

Project technical documentation

Source directory: /home/kacper/A-my projekts/TrangorgeOS

Public API/ABI: 4381 | Internal: 1377 | .md files: 20

1. Public API / ABI

Ada (spec)

```
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 4)
[package] package DS_Buddy_Math with
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 17)
[function] function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 23)
[function] function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return
Boolean with
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 32)
[function] function Get_Buddy_Address (
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 47)
[function] function Get_Left_Child_Address (
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 55)
[function] function Get_Right_Child_Address (
driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 69)
[function] function Safe_Calculate_DMA_End (
kernel_Workspace/kernel/src/policy/ada/policy.ads (line 3)
[package] package Policy
kernel_Workspace/kernel/src/policy/ada/policy.ads (line 21)
[function] function Evaluate (Ring : Ring_Id;
kernel_Workspace/kernel/src/policy/ada/policy_c.ads (line 4)
[package] package Policy_C is
kernel_Workspace/kernel/src/policy/ada/policy_c.ads (line 6)
[function] function policy_evaluate (ring : Unsigned_8;
```

Assembly

```
comgrub/src/boot.asm (line 30)
[label] global _start
kernel_Workspace/kernel-bin/src/boot.asm (line 2)
[label] global kernel_bin_entry
kernel_Workspace/kernel/src/cpu/trampoline.s (line 5)
[label] .global trampoline_start
kernel_Workspace/kernel/src/cpu/trampoline.s (line 6)
[label] .global trampoline_end
kernel_Workspace/kernel/src/cpu/trampoline.s (line 7)
[label] .global trampoline_cr3
kernel_Workspace/kernel/src/cpu/trampoline.s (line 8)
[label] .global trampoline_entry
kernel_Workspace/kernel/src/cpu/trampoline.s (line 9)
[label] .global trampoline_stack
kernel_Workspace/kernel/src/cpu/trampoline.s (line 10)
[label] .global trampoline_arg
kernel_Workspace/kernel/src/cpu/trampoline.s (line 100)
[label] .global trampoline_end
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 3)
[label] .global tlb_asm_invlpg
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 9)
[label] .global tlb_asm_read_cr3
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 15)
[label] .global tlb_asm_write_cr3
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 21)
[label] .global tlb_asm_read_cr4
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 27)
[label] .global tlb_asm_write_cr4
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 34)
[label] .global tlb_asm_invcid
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 40)
[label] .global tlb_asm_wbinvd
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 46)
[label] .global tlb_asm_clflush
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 102)
[label] .global tr_init
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 111)
[label] .global tr_restore_ctx
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 9)
[label] .global trap_entry
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 98)
[label] .global tr_init
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 114)
[label] .global tr_restore_ctx
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 3)
[label] .global tr_gdt
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 12)
[label] .global tr_tss_desc
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 21)
[label] .global tr_tss
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 27)
[label] .global tr_init
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 54)
[label] .global isr_hypercall
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 94)
[label] .global isr_default
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 99)
[label] .global tr_restore_ctx
triang-lang/out/x86_64/prog.asm (line 7)
[label] global main

```

C

```

comlimine/src/main.c (line 6)
[declaration] extern void kernel_main(uint64_t magic, void *info);
comlimine/src/main.c (line 28)
[function] void _start(void) {
comlimine/src/main.c (line 35)
[declaration] kernel_main(0, (void *)framebuffer.response);

```

```

drivers/vgpu/vgpu_driver.c (line 15)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 17)
[declaration] ds_take(id, &r);
drivers/vgpu/vgpu_driver.c (line 23)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 26)
[function] if (ds_take(id, &r) && r.status == 0) {
drivers/vgpu/vgpu_driver.c (line 35)
[declaration] pci_void(PORT_WRITE, VBE_INDEX, idx, 16);
drivers/vgpu/vgpu_driver.c (line 36)
[declaration] pci_void(PORT_WRITE, VBE_DATA, val, 16);
drivers/vgpu/vgpu_driver.c (line 42)
[function] if (g_ready) {
drivers/vgpu/vgpu_driver.c (line 47)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 51)
[function] if (!ds_take(id, &r) || r.status != 0) {
drivers/vgpu/vgpu_driver.c (line 57)
[function] if (r.arg1 != (0x1234 | (0x1111 << 16))) {
drivers/vgpu/vgpu_driver.c (line 63)
[function] if (fb_phys == 0) {
drivers/vgpu/vgpu_driver.c (line 67)
[declaration] pci_void(PCI_ENABLE, bdf, 0, 0);
drivers/vgpu/vgpu_driver.c (line 69)
[declaration] vbe_write(4, 0);
drivers/vgpu/vgpu_driver.c (line 70)
[declaration] vbe_write(0, 0xB0C4);
drivers/vgpu/vgpu_driver.c (line 71)
[declaration] vbe_write(1, w);
drivers/vgpu/vgpu_driver.c (line 72)
[declaration] vbe_write(2, h);
drivers/vgpu/vgpu_driver.c (line 73)
[declaration] vbe_write(3, 32);
drivers/vgpu/vgpu_driver.c (line 74)
[declaration] vbe_write(4, 0x01 | 0x40 | 0x80);
drivers/vgpu/vgpu_driver.c (line 78)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 82)
[function] if (!ds_take(mid, &mr) || mr.status != 0) {
drivers/vgpu/vgpu_driver.c (line 106)
[function] if (!g_ready || x >= g_info.width || y >= g_info.height) {
drivers/vgpu/vgpu_driver.c (line 115)
[function] if (!g_ready) {
drivers/vgpu/vgpu_driver.c (line 133)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 137)
[function] if (!ds_take(aid, &ar) || ar.status == 0 ? ar.arg0 == 0 : 1) {

```

```

drivers/vgpu/vgpu_driver.c (line 144)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 148)
[function] if (!ds_take(pid, &pr) || pr.status != 0) {
drivers/vgpu/vgpu_driver.c (line 153)
[function] if (!g_surf[i].used) {
drivers/vgpu/vgpu_driver.c (line 170)
[function] if (id < 0 || id >= VGPU_MAX_SURFACES || !g_surf[id].used) {
drivers/vgpu/vgpu_driver.c (line 181)
[function] if ((c & 0xFF000000) != 0) {
drivers/vgpu/vgpu_driver.c (line 192)
[function] if (ring == (void *)0) {
drivers/vgpu/vgpu_driver.c (line 199)
[function] if (s->id == 0 || s->done != 0) {
drivers/vgpu/vgpu_driver.c (line 203)
[function] switch (s->op) {
drivers/vgpu/vgpu_driver.c (line 215)
[function] if (sid < 0) {
drivers/vgpu/vgpu_driver.c (line 248)
[declaration] ds_init(params_va);
drivers/vgpu/vgpu_driver.c (line 250)
[function] if (!initd) {
drivers/vgpu/vgpu_driver.c (line 251)
[function] if (vgpu_init(1024, 768)) {
drivers/vgpu/vgpu_driver.c (line 253)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 257)
[function] if (ds_take(rid, &rr) && rr.status == 0) {
drivers/vgpu/vgpu_driver.c (line 261)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 265)
[function] if (ds_take(pid, &pr) && pr.status == 0) {
drivers/vgpu/vgpu_driver.c (line 266)
[declaration] ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
drivers/vgpu/vgpu_driver.c (line 267)
[declaration] ds_poll();
drivers/vgpu/vgpu_driver.c (line 270)
[declaration] ds_take(0, &zr);
drivers/vgpu/vgpu_driver.c (line 274)
[declaration] vgpu_clear(0xFF101020);
drivers/vgpu/vgpu_driver.c (line 280)
[declaration] vgpu_process_ring(g_ring);
drivers/vgpu/vgpu_driver.c (line 282)
[declaration] ds_yield();
kernel_Workspace/kernel/src/battery/init.c (line 8)
[function] if (is_ready) {
kernel_Workspace/kernel/src/battery/operation.c (line 23)
[function] if (!model.present) {

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 40)
[function] if (b != (void *)0 && b->status != (void *)0) {
kernel_Workspace/kernel/src/battery/operation.c (line 52)
[function] if (out == (void *)0) {
kernel_Workspace/kernel/src/battery/operation.c (line 63)
[function] if (!battery_status(&s)) {
kernel_Workspace/kernel/src/battery/operation.c (line 79)
[function] if (low_pct > 50) {
kernel_Workspace/kernel/src/battery/operation.c (line 103)
[function] if (!model.present || model.state == BATT_STATE_CHARGING) {
kernel_Workspace/kernel/src/battery/operation.c (line 109)
[function] if (tick_counter % 2000 == 0 && model.percent > 0) {
kernel_Workspace/kernel/src/battery/operation.c (line 112)
[function] if (model.percent <= low_threshold && !low_pending) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 14)
[function] if (ret_len != 0 && ret == NULL) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 19)
[function] if (!bt_event_poll(buf, EVT_MAX, &len)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 23)
[function] if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 27)
[function] if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >>
8)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 31)
[function] if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 47)
[function] if (bt_is_ready) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 51)
[declaration] bt_op_model_init();
kernel_Workspace/kernel/src/bluetooth/init.c (line 53)
[function] if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 58)
[function] if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 62)
[function] if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 66)
[function] if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 72)
[function] if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 76)
[function] if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 91)
[function] if (hci_ver != NULL) {
kernel_Workspace/kernel/src/bluetooth/init.c (line 95)
[function] if (bdaddr != NULL) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 18)
[function] if (data == NULL || len == 0 || len > EVT_MAX) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 23)
[function] if (next == evt_tail) {

```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 49)
[function] if (len != 0 && params == NULL) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 53)
[function] switch (opcode) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 62)
[declaration] return evt_push(ev, 7);
kernel_Workspace/kernel/src/bluetooth/operation.c (line 80)
[declaration] return evt_push(ev, 15);
kernel_Workspace/kernel/src/bluetooth/operation.c (line 93)
[declaration] return evt_push(ev, 13);
kernel_Workspace/kernel/src/bluetooth/operation.c (line 103)
[declaration] return evt_push(ev, 7);
kernel_Workspace/kernel/src/bluetooth/operation.c (line 109)
[function] if (!op_ready) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 113)
[declaration] return bt_op_model_run_cmd(opcode, params, len);
kernel_Workspace/kernel/src/bluetooth/operation.c (line 118)
[function] if (buf == NULL || len == NULL || evt_head == evt_tail) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 123)
[function] if (n > cap) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 138)
[function] if (!op_ready || (len != 0 && data == NULL) || len > ACL_MAX) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 143)
[function] if (next == acl_tail) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 158)
[function] if (data == NULL || len == NULL || acl_head == acl_tail) {
kernel_Workspace/kernel/src/bluetooth/operation.c (line 163)
[function] if (n > cap) {
kernel_Workspace/kernel/src/camera/init.c (line 7)
[function] if (is_ready) {
kernel_Workspace/kernel/src/camera/operation.c (line 43)
[function] if (((x + y + f * 8) & 0xFF) < 16) {
kernel_Workspace/kernel/src/camera/operation.c (line 58)
[function] if (!streaming) {
kernel_Workspace/kernel/src/camera/operation.c (line 62)
[declaration] gen_frame();
kernel_Workspace/kernel/src/camera/operation.c (line 66)
[function] if (cap < n) {
kernel_Workspace/kernel/src/camera/operation.c (line 78)
[function] if (fid) {
kernel_Workspace/kernel/src/camera/operation.c (line 96)
[function] if (b != (void *)0 && b->frame != (void *)0) {
kernel_Workspace/kernel/src/camera/operation.c (line 108)
[function] if (out == (void *)0 || !camera_present()) {
kernel_Workspace/kernel/src/camera/operation.c (line 123)
[function] if (!camera_present()) {
kernel_Workspace/kernel/src/camera/operation.c (line 137)
[function] if (buf == (void *)0) {

```

```

kernel_Workspace/kernel/src/core-lang/ast.c (line 14)
[function] if (a->pos + aligned > a->cap) {
kernel_Workspace/kernel/src/core-lang/ast.c (line 29)
[function] if (n == (void *)0) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 3)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 4)
[declaration] extern int32_t k_input_key(void);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 5)
[declaration] extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 6)
[declaration] extern uint32_t k_getpid(void);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 7)
[declaration] extern uint64_t k_world_cr3(void);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 8)
[declaration] extern uint64_t k_kernel_cr3(void);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 9)
[declaration] extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t
cr3);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 10)
[declaration] extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 11)
[declaration] extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 12)
[declaration] extern uint64_t k_tick(void);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 16)
[declaration] extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t
*a2);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 26)
[declaration] kprintf("%llu\n", (unsigned long long)v);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 34)
[declaration] kprintf("%c", (char)v);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 45)
[function] if (g_ring == 0) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 49)
[function] while (s[i] && i < 127) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 56)
[declaration] extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 58)
[function] if (!k_user_cstr(ptr, buf, sizeof(buf))) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 63)
[declaration] kprintf("%s\n", buf);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 81)
[declaration] return k_getpid();
kernel_Workspace/kernel/src/core-lang/bridge.c (line 88)
[declaration] return k_tick();
kernel_Workspace/kernel/src/core-lang/bridge.c (line 96)
[function] if (g_vm != (void *)0) {

```



```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 108)
[function] if (g_ring >= 3) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 114)
[declaration] return (uint64_t)k_spawn((const char *)ptr, k_getpid(), cr3);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 121)
[declaration] return (uint64_t)(int64_t)k_ipc_send((uint32_t)dst, a0, a1);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 140)
[function] if (g_ring >= 3) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 153)
[function] if (g_ring >= 3) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 171)
[function] if (!battery_status_packed(&a0, &a1, &a2)) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 181)
[declaration] return cl_vm_register_extern(vm, name, fn);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 189)
[declaration] cl_bridge_add(vm, "put", br_put);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 190)
[declaration] cl_bridge_add(vm, "putc", br_putc);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 191)
[declaration] cl_bridge_add(vm, "log", br_log);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 192)
[declaration] cl_bridge_add(vm, "key", br_key);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 193)
[declaration] cl_bridge_add(vm, "tick", br_tick);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 194)
[declaration] cl_bridge_add(vm, "bat", br_bat);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 195)
[declaration] cl_bridge_add(vm, "exit", br_exit);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 197)
[function] if (ring <= 1) {
kernel_Workspace/kernel/src/core-lang/bridge.c (line 198)
[declaration] cl_bridge_add(vm, "getpid", br_getpid);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 199)
[declaration] cl_bridge_add(vm, "spawn", br_spawn);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 200)
[declaration] cl_bridge_add(vm, "ipc_send", br_ipc_send);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 201)
[declaration] cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 202)
[declaration] cl_bridge_add(vm, "fsread", br_fsread);
kernel_Workspace/kernel/src/core-lang/bridge.c (line 203)
[declaration] cl_bridge_add(vm, "vfill", br_vfill);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 22)
[function] if (P->code_len >= P->code_cap) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 45)
[function] if (P->gnames[i][k] != name[k]) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 63)
[function] if (g->locals[i].name[k] != name[k]) {

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 79)
[function] if (i >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 83)
[function] if (g->nlocals >= 32) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 112)
[function] if (neg) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 115)
[declaration] cl_sub(sub, z, P->consts[P->nconsts].b, w);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 119)
[declaration] cl_mask(P->consts[P->nconsts].b, w, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 126)
[declaration] static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 134)
[function] switch (n->kind) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 136)
[declaration] emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false),
0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 142)
[function] if (li >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 143)
[declaration] emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 147)
[function] if (gi >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 148)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 150)
[declaration] emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 157)
[function] if (n->pool_kind == 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 158)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 161)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 162)
[declaration] emit(g, OP_NEG, bits, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 163)
[declaration] emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 164)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 167)
[declaration] emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 168)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 175)
[function] if (li >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 176)
[declaration] emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 178)
[declaration] emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 184)
[declaration] gen_expr(g, n->a, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 186)
[function] if (n->op == '-') {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 187)
[declaration] emit(g, OP_NEG, bits, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 189)
[declaration] emit(g, OP_NOT, bits, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 196)
[function] switch (n->op) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 215)
[declaration] gen_expr(g, n->a, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 216)
[declaration] gen_expr(g, n->b, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 217)
[declaration] emit(g, op, bits, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 222)
[declaration] gen_expr(g, n->b, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 224)
[function] if (n->a->kind == ND_VAR) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 227)
[function] if (li >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 228)
[declaration] emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 230)
[declaration] emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 237)
[declaration] gen_expr(g, n->a, bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 241)
[function] if (n->a != (void *)0 && n->a->kind == ND_VAR) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 248)
[function] if (P->ext_names[i][k] != n->a->name[k]) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 254)
[function] if (same) { ei = (int)i; break; }
kernel_Workspace/kernel/src/core-lang/codegen.c (line 258)
[declaration] gen_expr(g, n->list[i], 64);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 261)
[function] if (ei >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 262)
[declaration] emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 270)
[function] if (P->fn_names[i][k] != n->a->name[k]) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 276)
[function] if (same) { fi = (int)i; break; }
kernel_Workspace/kernel/src/core-lang/codegen.c (line 279)
[function] if (fi >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 280)
[declaration] emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 296)
[function] switch (n->kind) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 300)
[declaration] gen_expr(g, n->a, n->type.bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 301)
[declaration] emit(g, OP_STORE, n->type.bits, slot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 308)
[function] if (li >= 0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 309)
[declaration] emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 311)
[declaration] emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 317)
[function] if (n->a != (void *)0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 318)
[declaration] gen_expr(g, n->a, 1);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 322)
[declaration] gen_stmt(g, n->b);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 324)
[function] if (n->c != (void *)0) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 328)
[declaration] gen_stmt(g, n->c);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 334)
[declaration] gen_stmt(g, n->b);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 344)
[function] if (li2 >= 0 && g->locals[li2].slot != tslot) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 345)
[declaration] emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 346)
[declaration] emit(g, OP_STORE, 64, tslot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 348)
[declaration] emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 349)
[declaration] emit(g, OP_STORE, 64, tslot, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 359)
[declaration] gen_stmt(g, n->list[i]);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 364)
[declaration] gen_expr(g, n, 64);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 365)
[declaration] emit(g, OP_POP, 0, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 387)
[function] if (n->kind == ND_LET && P->nglobals < 64) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 403)
[declaration] cl_wbytes(n->type.bits), n->type.bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 436)
[function] if (P->fn_names[k][x] != n->name[x]) {
kernel_Workspace/kernel/src/core-lang/codegen.c (line 442)
[function] if (same) { fi = (int)k; break; }

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 450)
[declaration] add_local(&g, n->list[k]->name, n->list[k]->type.bits);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 455)
[declaration] gen_stmt(&g, n->a);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 457)
[declaration] emit(&g, OP_RET, 0, 0, 0, 0);
kernel_Workspace/kernel/src/core-lang/codegen.c (line 465)
[function] if (m[k] != n->name[k]) { is_main = false; break; }
kernel_Workspace/kernel/src/core-lang/codegen.c (line 468)
[function] if (is_main) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 49)
[function] if (c == '\n') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 77)
[function] if (c == ' ' || c == '\t' || c == '\r') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 78)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 80)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 82)
[function] while (l->pos < l->len && peek(l) != '\n') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 83)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 86)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 87)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 91)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 94)
[function] if (l->pos < l->len) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 95)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 96)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 111)
[declaration] return is_id_start(c) || (c >= '0' && c <= '9');
kernel_Workspace/kernel/src/core-lang/lexer.c (line 133)
[function] if (peek(l) == '0' && (peek2(l) == 'x' || peek2(l) == 'X')) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 134)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 135)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 137)
[function] while (hex_val(peek(l)) >= 0) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 141)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 142)
[declaration] take(l);

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 144)
[function] while (peek(l) == '0' || peek(l) == '1') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 148)
[function] while (is_digit(peek(l))) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 163)
[function] while (is_id_char(peek(l))) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 164)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 175)
[function] if (kwlen == len) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 179)
[function] if (start[j] != k->kw[j]) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 185)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 186)
[declaration] emit(l, t, k->kind, start, len, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 193)
[function] if (len == 1 && start[0] == '_') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 194)
[declaration] emit(l, t, TK_UNDERSCORE, start, len, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 198)
[declaration] emit(l, t, TK_IDENT, start, len, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 204)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 208)
[function] while (l->pos < l->len && peek(l) != '"') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 209)
[function] if (peek(l) == '\\') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 210)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 213)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 218)
[function] if (peek(l) == '"') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 219)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 221)
[declaration] emit(l, t, TK_ERR, start, len, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 225)
[declaration] emit(l, t, TK_STR, start, len, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 231)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 235)
[function] if (peek(l) == '\\') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 236)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 240)
[function] switch (e) {

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 252)
[function] if (peek(l) == '\') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 253)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 255)
[declaration] emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 259)
[declaration] emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 265)
[declaration] skip_ws(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 270)
[function] if (l->pos >= l->len) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 271)
[declaration] emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 277)
[function] if (is_digit(c)) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 278)
[declaration] lex_number(l, t, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 282)
[function] if (is_id_start(c)) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 283)
[declaration] lex_ident(l, t, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 287)
[function] if (c == '"') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 288)
[declaration] lex_string(l, t, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 292)
[function] if (c == '\\') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 293)
[declaration] lex_char(l, t, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 299)
[function] if (c == '$') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 300)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 302)
[function] if (peek(l) == '!') {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 303)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 304)
[declaration] emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 306)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 307)
[declaration] emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 309)
[declaration] emit(l, t, TK_DOLLAR, start, 1, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 317)
[function] if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2,
line, col); return true; }

```



```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 318)
[function] if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start,
2, line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 319)
[function] if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON,
start, 2, line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 320)
[function] if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2,
line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 321)
[function] if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2,
line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 322)
[function] if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2,
line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 323)
[function] if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2,
line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 324)
[function] if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2,
line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 325)
[function] if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start,
2, line, col); return true; }
kernel_Workspace/kernel/src/core-lang/lexer.c (line 328)
[declaration] take(l);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 330)
[function] switch (c) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 355)
[declaration] emit(l, t, TK_ERR, start, 1, line, col);
kernel_Workspace/kernel/src/core-lang/lexer.c (line 374)
[function] if (n >= cap) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 380)
[function] if (!ok) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 388)
[function] if (k == TK_EOF) {
kernel_Workspace/kernel/src/core-lang/lexer.c (line 399)
[function] switch (kind) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 19)
[function] if (toks == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 25)
[declaration] cl_lexer_init(&l, src, len);
kernel_Workspace/kernel/src/core-lang/loader.c (line 27)
[function] if (cl_lex_all(&l, toks, tcap, &n) != 0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 36)
[function] if (prog == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 44)
[function] if (s == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 50)
[declaration] cl_sema_run(s, prog);

```



```

kernel_Workspace/kernel/src/core-lang/loader.c (line 52)
[function] if (cl_sema_diag_count(s) > 0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 62)
[function] if (P == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 88)
[function] if (!w->ok || w->o >= w->cap) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 99)
[declaration] wr_u8(w, (uint8_t)(v >> (i * 8)));
kernel_Workspace/kernel/src/core-lang/loader.c (line 106)
[declaration] wr_u8(w, ((const uint8_t *)src)[i]);
kernel_Workspace/kernel/src/core-lang/loader.c (line 112)
[function] if (!r->ok || r->o >= r->len) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 142)
[declaration] wr_bytes(&w, "CLBC1", 5);
kernel_Workspace/kernel/src/core-lang/loader.c (line 144)
[declaration] wr_u32(&w, (uint32_t)P->code_len);
kernel_Workspace/kernel/src/core-lang/loader.c (line 149)
[declaration] wr_u8(&w, in->op);
kernel_Workspace/kernel/src/core-lang/loader.c (line 150)
[declaration] wr_u8(&w, in->bits);
kernel_Workspace/kernel/src/core-lang/loader.c (line 151)
[declaration] wr_u8(&w, (uint8_t)(in->a & 0xFF));
kernel_Workspace/kernel/src/core-lang/loader.c (line 152)
[declaration] wr_u8(&w, (uint8_t)(in->a >> 8));
kernel_Workspace/kernel/src/core-lang/loader.c (line 153)
[declaration] wr_u8(&w, (uint8_t)(in->b & 0xFF));
kernel_Workspace/kernel/src/core-lang/loader.c (line 154)
[declaration] wr_u8(&w, (uint8_t)(in->b >> 8));
kernel_Workspace/kernel/src/core-lang/loader.c (line 155)
[declaration] wr_u32(&w, in->c);
kernel_Workspace/kernel/src/core-lang/loader.c (line 158)
[declaration] wr_u32(&w, (uint32_t)P->nconsts);
kernel_Workspace/kernel/src/core-lang/loader.c (line 161)
[declaration] wr_bytes(&w, P->consts[i].b, 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 162)
[declaration] wr_u8(&w, P->const_bits[i]);
kernel_Workspace/kernel/src/core-lang/loader.c (line 165)
[declaration] wr_u32(&w, (uint32_t)P->nfns);
kernel_Workspace/kernel/src/core-lang/loader.c (line 168)
[declaration] wr_u32(&w, P->fns[i].start);
kernel_Workspace/kernel/src/core-lang/loader.c (line 169)
[declaration] wr_u8(&w, (uint8_t)P->fns[i].nslots);
kernel_Workspace/kernel/src/core-lang/loader.c (line 170)
[declaration] wr_u8(&w, (uint8_t)P->fns[i].nargs);
kernel_Workspace/kernel/src/core-lang/loader.c (line 171)
[declaration] wr_bytes(&w, P->fn_names[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 174)
[declaration] wr_u32(&w, (uint32_t)P->nglobals);

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 177)
[declaration] wr_bytes(&w, P->gnames[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 178)
[declaration] wr_u8(&w, P->gbits[i]);
kernel_Workspace/kernel/src/core-lang/loader.c (line 179)
[declaration] wr_u8(&w, P->gneg[i]);
kernel_Workspace/kernel/src/core-lang/loader.c (line 180)
[declaration] wr_bytes(&w, P->ginit[i].b, 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 183)
[declaration] wr_u32(&w, (uint32_t)P->nexts);
kernel_Workspace/kernel/src/core-lang/loader.c (line 186)
[declaration] wr_bytes(&w, P->ext_names[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 189)
[declaration] wr_u32(&w, (uint32_t)P->main_fn);
kernel_Workspace/kernel/src/core-lang/loader.c (line 200)
[declaration] rd_bytes(&r, magic, 5);
kernel_Workspace/kernel/src/core-lang/loader.c (line 209)
[function] if (P == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 218)
[function] if (P->code == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 224)
[function] if (cl > P->code_cap) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 242)
[function] if (P->nconsts > 64) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 247)
[declaration] rd_bytes(&r, P->consts[i].b, 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 253)
[function] if (P->nfns > 16) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 261)
[declaration] rd_bytes(&r, P->fn_names[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 266)
[function] if (P->nglobals > 64) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 271)
[declaration] rd_bytes(&r, P->gnames[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 274)
[declaration] rd_bytes(&r, P->ginit[i].b, 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 279)
[function] if (P->nexts > 16) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 284)
[declaration] rd_bytes(&r, P->ext_names[i], 64);
kernel_Workspace/kernel/src/core-lang/loader.c (line 289)
[function] if (!r.ok) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 305)
[function] if (f == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 311)
[declaration] fclose(f);
kernel_Workspace/kernel/src/core-lang/loader.c (line 316)
[declaration] extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 320)
[declaration] return (long)k_fs_read(path, buf, (uint32_t)cap);
kernel_Workspace/kernel/src/core-lang/loader.c (line 331)
[function] if (n <= 0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 337)
[function] if (m == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 343)
[function] if (n > 5 && io[0] == 'C' && io[1] == 'L' && io[2] == 'B') {
kernel_Workspace/kernel/src/core-lang/loader.c (line 353)
[function] if (m->bc == (void *)0) {
kernel_Workspace/kernel/src/core-lang/loader.c (line 357)
[function] if (try_native) {
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 27)
[declaration] nb_u8(d, 0x48); nb_u8(d, 0xB8); nb_u64(d, v);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 37)
[declaration] nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a
* 8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 38)
[declaration] push_rax(d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 43)
[declaration] pop_rax(d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 44)
[declaration] nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a
* 8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 49)
[declaration] nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a
* 8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 50)
[declaration] push_rax(d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 55)
[declaration] pop_rax(d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 56)
[declaration] nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a
* 8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 76)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xFB);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 77)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xF5);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 78)
[declaration] nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 89)
[function] switch (in.op) {
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 97)
[declaration] mov_rax_u64(&d, v);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 98)
[declaration] push_rax(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 102)
[declaration] case OP_LOAD: load_slot(&d, in.a); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 103)
[declaration] case OP_STORE: store_slot(&d, in.a); break;

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 104)
[declaration] case OP_GLOAD: load_glob(&d, in.a); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 105)
[declaration] case OP_GSTORE: store_glob(&d, in.a); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 108)
[declaration] pop_rax(&d); pop_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 109)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 110)
[declaration] push_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 114)
[declaration] pop_rax(&d); pop_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 115)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 116)
[declaration] push_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 120)
[declaration] pop_rax(&d); pop_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 121)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 122)
[declaration] push_rcx(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 129)
[declaration] pop_rax(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 131)
[function] switch (i) {
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 132)
[declaration] case 0: nb_u8(&d,0x48); nb_u8(&d,0x89); nb_u8(&d,0xC7); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 133)
[declaration] case 1: nb_u8(&d,0x48); nb_u8(&d,0x89); nb_u8(&d,0xC6); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 134)
[declaration] case 2: nb_u8(&d,0x48); nb_u8(&d,0x89); nb_u8(&d,0xC2); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 135)
[declaration] case 3: nb_u8(&d,0x48); nb_u8(&d,0x89); nb_u8(&d,0xC1); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 136)
[declaration] case 4: nb_u8(&d,0x49); nb_u8(&d,0x89); nb_u8(&d,0xC0); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 137)
[declaration] case 5: nb_u8(&d,0x49); nb_u8(&d,0x89); nb_u8(&d,0xC1); break;
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 142)
[declaration] nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 143)
[declaration] nb_u32(&d, (uint32_t)in.a * 8);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 144)
[declaration] nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 145)
[declaration] push_rax(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 149)
[declaration] case OP_POP: pop_rax(&d); break;

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 152)
[declaration] nb_u8(&d, 0xE9);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 156)
[declaration] nb_u32(&d, 0);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 160)
[declaration] pop_rax(&d);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 161)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 162)
[declaration] nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 166)
[declaration] nb_u32(&d, 0);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 171)
[declaration] nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 172)
[declaration] nb_u8(&d, 0xC3);
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 197)
[declaration] return sizeof(g_nb_mem);
kernel_Workspace/kernel/src/core-lang/parser.c (line 7)
[function] if (n >= cap) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 27)
[function] if (i >= p->n) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 38)
[function] if (p->pos + 1 < p->n) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 52)
[function] if (check(p, k)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 53)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 62)
[function] if (!p->failed) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 71)
[function] if (!check(p, k)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 72)
[declaration] perr(p, m);
kernel_Workspace/kernel/src/core-lang/parser.c (line 73)
[declaration] return peek(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 76)
[declaration] return advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 81)
[declaration] return cl_node(p->ar, kind, peek(p)->line);
kernel_Workspace/kernel/src/core-lang/parser.c (line 99)
[function] if (l->n < LIST_MAX) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 102)
[declaration] perr(p, "list too long");
kernel_Workspace/kernel/src/core-lang/parser.c (line 111)
[function] if (arr == (void *)0) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 112)
[declaration] perr(p, "arena full");

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 128)
[function] if (check(p, TK_TYPE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 131)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 135)
[declaration] perr(p, "expected type (u4..u512/str)");
kernel_Workspace/kernel/src/core-lang/parser.c (line 145)
[declaration] expect(p, TK_LET, "let");
kernel_Workspace/kernel/src/core-lang/parser.c (line 147)
[declaration] parse_type(p, &n->type);
kernel_Workspace/kernel/src/core-lang/parser.c (line 149)
[declaration] copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 151)
[declaration] expect(p, TK_ASSIGN, "let =");
kernel_Workspace/kernel/src/core-lang/parser.c (line 158)
[function] if (match(p, TK_UNDERSCORE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 159)
[declaration] expect(p, TK_ARROW, "_=>");
kernel_Workspace/kernel/src/core-lang/parser.c (line 174)
[declaration] expect(p, TK_STATIC, "static");
kernel_Workspace/kernel/src/core-lang/parser.c (line 175)
[declaration] expect(p, TK_DATA, "static data");
kernel_Workspace/kernel/src/core-lang/parser.c (line 176)
[declaration] expect(p, TK_LBRACK, "[");
kernel_Workspace/kernel/src/core-lang/parser.c (line 178)
[declaration] parse_type(p, &n->type);
kernel_Workspace/kernel/src/core-lang/parser.c (line 180)
[declaration] expect(p, TK_COLON, ":");
kernel_Workspace/kernel/src/core-lang/parser.c (line 188)
[declaration] expect(p, TK_RBRACK, "]");
kernel_Workspace/kernel/src/core-lang/parser.c (line 200)
[declaration] expect(p, TK_FN, "fn");
kernel_Workspace/kernel/src/core-lang/parser.c (line 202)
[declaration] copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "fn name"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 204)
[declaration] expect(p, TK_LPAREN, "fn (");
kernel_Workspace/kernel/src/core-lang/parser.c (line 208)
[function] while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 211)
[declaration] parse_type(p, &arg->type);
kernel_Workspace/kernel/src/core-lang/parser.c (line 213)
[declaration] expect(p, TK_IDENT, "param name"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 215)
[declaration] nl_push(p, &params, arg);
kernel_Workspace/kernel/src/core-lang/parser.c (line 217)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 222)
[declaration] expect(p, TK_RPAREN, "fn )");

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 224)
[declaration] nl_commit(p, &params, n);
kernel_Workspace/kernel/src/core-lang/parser.c (line 235)
[declaration] expect(p, TK_ENUM, "enum");
kernel_Workspace/kernel/src/core-lang/parser.c (line 237)
[declaration] copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "enum name"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 244)
[function] while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 248)
[declaration] expect(p, TK_IDENT, "enum item"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 252)
[declaration] nl_push(p, &items, it);
kernel_Workspace/kernel/src/core-lang/parser.c (line 254)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 259)
[declaration] expect(p, TK_RBRACE, "enum }");
kernel_Workspace/kernel/src/core-lang/parser.c (line 261)
[declaration] nl_commit(p, &items, n);
kernel_Workspace/kernel/src/core-lang/parser.c (line 270)
[declaration] expect(p, TK_EXTERN, "extern");
kernel_Workspace/kernel/src/core-lang/parser.c (line 281)
[function] while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 285)
[declaration] expect(p, TK_IDENT, "extern fn"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 287)
[declaration] nl_push(p, &fns, f);
kernel_Workspace/kernel/src/core-lang/parser.c (line 289)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 294)
[declaration] expect(p, TK_RBRACE, "extern }");
kernel_Workspace/kernel/src/core-lang/parser.c (line 296)
[declaration] nl_commit(p, &fns, n);
kernel_Workspace/kernel/src/core-lang/parser.c (line 305)
[declaration] expect(p, TK_DATAIMPL, "dataimpl");
kernel_Workspace/kernel/src/core-lang/parser.c (line 312)
[function] while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 313)
[function] if (match(p, TK_DATA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 314)
[declaration] expect(p, TK_COLON, "data:");
kernel_Workspace/kernel/src/core-lang/parser.c (line 318)
[declaration] nl_push(p, &base, parse_expr(p));
kernel_Workspace/kernel/src/core-lang/parser.c (line 320)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 325)
[declaration] expect(p, TK_RBRACE, "dataimpl }");
kernel_Workspace/kernel/src/core-lang/parser.c (line 327)
[declaration] expect(p, TK_ARROW, "=>");

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 330)
[declaration] nl_commit(p, &base, baseblk);
kernel_Workspace/kernel/src/core-lang/parser.c (line 348)
[function] while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 351)
[function] switch (peek(p)->kind) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 358)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 361)
[declaration] expect(p, TK_IDENT, "set_free var");
kernel_Workspace/kernel/src/core-lang/parser.c (line 362)
[declaration] expect(p, TK_RBACE, "set_free }");
kernel_Workspace/kernel/src/core-lang/parser.c (line 371)
[function] if (s == (void *)1) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 372)
[function] if (check(p, TK_IF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 384)
[declaration] nl_push(p, &stmts, s);
kernel_Workspace/kernel/src/core-lang/parser.c (line 387)
[declaration] expect(p, TK_RBACE, "}");
kernel_Workspace/kernel/src/core-lang/parser.c (line 389)
[declaration] nl_commit(p, &stmts, n);
kernel_Workspace/kernel/src/core-lang/parser.c (line 398)
[declaration] expect(p, TK_IF, "if");
kernel_Workspace/kernel/src/core-lang/parser.c (line 400)
[function] if (!check(p, TK_LBRACE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 407)
[function] if (match(p, TK_ELSE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 408)
[function] if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 409)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 413)
[function] if (!check(p, TK_LBRACE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 423)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 424)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 435)
[declaration] expect(p, TK_WHEN, "when");
kernel_Workspace/kernel/src/core-lang/parser.c (line 437)
[declaration] copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "when
target"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 439)
[declaration] expect(p, TK_EQCOLON, "=:");
kernel_Workspace/kernel/src/core-lang/parser.c (line 444)
[function] while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 450)
[declaration] nl_push(p, &cases, c);

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 453)
[declaration] expect(p, TK_RBRACE, "when ");
kernel_Workspace/kernel/src/core-lang/parser.c (line 455)
[declaration] nl_commit(p, &cases, n);
kernel_Workspace/kernel/src/core-lang/parser.c (line 466)
[function] if (check(p, TK_NUM)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 472)
[function] if (check(p, TK_STR)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 475)
[declaration] copy_name(n->name, sizeof(n->name), t);
kernel_Workspace/kernel/src/core-lang/parser.c (line 479)
[function] if (check(p, TK_CHAR)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 486)
[function] check(p, TK_DOLLAR_AT)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 494)
[declaration] expect(p, TK_IDENT, "$var");
kernel_Workspace/kernel/src/core-lang/parser.c (line 498)
[function] if (check(p, TK_CALL) && peek_at(p, 1)->kind == TK_LBRACK) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 500)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 501)
[declaration] expect(p, TK_LBRACK, "call[");
kernel_Workspace/kernel/src/core-lang/parser.c (line 503)
[declaration] expect(p, TK_IDENT, "call[x]");
kernel_Workspace/kernel/src/core-lang/parser.c (line 504)
[declaration] expect(p, TK_RBRACK, "]);
kernel_Workspace/kernel/src/core-lang/parser.c (line 508)
[function] if (match(p, TK_LPAREN)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 510)
[declaration] expect(p, TK_RPAREN, "));
kernel_Workspace/kernel/src/core-lang/parser.c (line 514)
[function] if (check(p, TK_IDENT)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 516)
[declaration] copy_name(n->name, sizeof(n->name), advance(p));
kernel_Workspace/kernel/src/core-lang/parser.c (line 520)
[declaration] perr(p, "unexpected token in expr");
kernel_Workspace/kernel/src/core-lang/parser.c (line 521)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 523)
[declaration] return new_node(p, ND_NUM);
kernel_Workspace/kernel/src/core-lang/parser.c (line 531)
[function] if (match(p, TK_DOT)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 536)
[declaration] expect(p, TK_IDENT, "member");
kernel_Workspace/kernel/src/core-lang/parser.c (line 539)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 547)
[function] if (n->kind == ND_VAR) {

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 554)
[declaration] expect(p, TK_IDENT, "scope fn"));
kernel_Workspace/kernel/src/core-lang/parser.c (line 556)
[declaration] expect(p, TK_LPAREN, "scope (");
kernel_Workspace/kernel/src/core-lang/parser.c (line 560)
[function] while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 561)
[declaration] nl_push(p, &args, parse_expr(p));
kernel_Workspace/kernel/src/core-lang/parser.c (line 563)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 568)
[declaration] expect(p, TK_RPAREN, "::");
kernel_Workspace/kernel/src/core-lang/parser.c (line 570)
[declaration] nl_commit(p, &args, m);
kernel_Workspace/kernel/src/core-lang/parser.c (line 574)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 582)
[function] while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 583)
[declaration] nl_push(p, &args, parse_expr(p));
kernel_Workspace/kernel/src/core-lang/parser.c (line 585)
[function] if (!match(p, TK_COMMA)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 590)
[declaration] expect(p, TK_RPAREN, "call ");
kernel_Workspace/kernel/src/core-lang/parser.c (line 592)
[declaration] nl_commit(p, &args, m);
kernel_Workspace/kernel/src/core-lang/parser.c (line 606)
[function] check(p, TK_TILDE)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 615)
[declaration] return parse_postfix(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 626)
[function] switch (k) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 638)
[function] if (lv > level) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 642)
[declaration] advance(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 662)
[function] if (match(p, TK_ASSIGN)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 676)
[declaration] return parse_assign(p);
kernel_Workspace/kernel/src/core-lang/parser.c (line 694)
[function] while (!check(p, TK_EOF)) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 697)
[function] switch (peek(p)->kind) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 705)
[declaration] perr(p, "expected top-level decl");
kernel_Workspace/kernel/src/core-lang/parser.c (line 706)
[declaration] advance(p);

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 710)
[declaration] nl_push(p, &tops, d);
kernel_Workspace/kernel/src/core-lang/parser.c (line 712)
[function] if (p->failed) {
kernel_Workspace/kernel/src/core-lang/parser.c (line 717)
[declaration] nl_commit(p, &tops, prog);
kernel_Workspace/kernel/src/core-lang/parser.c (line 719)
[function] if (p->failed) {
kernel_Workspace/kernel/src/core-lang/run.c (line 4)
[declaration] extern long cl_kernel_read(const char *path, uint8_t *buf, size_t
cap);
kernel_Workspace/kernel/src/core-lang/run.c (line 13)
[function] if (n <= 0) {
kernel_Workspace/kernel/src/core-lang/run.c (line 22)
[function] if (P == (void *)0) {
kernel_Workspace/kernel/src/core-lang/run.c (line 23)
[declaration] extern void kprintf(const char *, ...);
kernel_Workspace/kernel/src/core-lang/run.c (line 24)
[declaration] kprintf("core-lang: %s line %u: %s\n", path, el, em);
kernel_Workspace/kernel/src/core-lang/run.c (line 30)
[declaration] cl_vm_init(&vm, P);
kernel_Workspace/kernel/src/core-lang/run.c (line 31)
[declaration] cl_bridge_init(&vm, ring);
kernel_Workspace/kernel/src/core-lang/run.c (line 35)
[function] if (code != (void *)0) {
kernel_Workspace/kernel/src/core-lang/run.c (line 36)
[declaration] extern void cl_make_exec(void *p, size_t len);
kernel_Workspace/kernel/src/core-lang/run.c (line 37)
[declaration] cl_make_exec(code, 16384);
kernel_Workspace/kernel/src/core-lang/run.c (line 57)
[declaration] cl_vm_run(&vm);
kernel_Workspace/kernel/src/core-lang/sema.c (line 45)
[function] if (s == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 58)
[function] if (s->ndiags < 64) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 66)
[function] if (name != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 77)
[function] if (s->top < 15) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 86)
[function] if (sym->kind == SYM_VAR && sym->live) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 92)
[function] if (s->top > 0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 104)
[function] if (name[i] != sym->name[i]) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 110)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 111)
[declaration] diag(s, line, "redeclaration in same scope", name, false);

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 118)
[function] if (sym == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 149)
[function] if (name[k] != sym->name[k]) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 155)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 167)
[function] if (t.bits == 0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 171)
[function] if (t.bits >= 64) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 175)
[function] if (negative) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 176)
[function] if (!t.neg) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 183)
[function] if (t.neg) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 197)
[declaration] static etype_t check_expr(sema_ctx_t *s, ast_node_t *n);
kernel_Workspace/kernel/src/core-lang/sema.c (line 202)
[function] if (s->sh_count < 2) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 216)
[function] if (s->sh_count == 1) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 220)
[function] if (s->sh_type[0].bits != s->sh_type[1].bits) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 233)
[function] if (n == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 237)
[function] switch (n->kind) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 255)
[function] if (n->op == 45 ) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 256)
[function] if (r.is_const) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 259)
[declaration] diag(s, n->line, "negation needs _=> range", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 268)
[function] if (sym == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 269)
[declaration] diag(s, n->line, "undeclared identifier", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 273)
[function] if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 274)
[function] if (!sym->live) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 275)
[declaration] diag(s, n->line, "use after set_free", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 282)
[declaration] diag(s, n->line, "not a value", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 291)
[function] if (sym == (void *)0) {

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 292)
[declaration] diag(s, n->line, "undeclared identifier", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 296)
[function] if (sym->depth != 0 || sym->kind != SYM_VAR) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 301)
[function] if (!sym->live) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 302)
[declaration] diag(s, n->line, "use after set_free", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 307)
[function] if (n->pool_kind == 1 && !sym->type.neg) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 308)
[declaration] diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 311)
[function] if (n->pool_kind == 2) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 313)
[declaration] shared_note(s, n->name, sym->type, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 323)
[function] if (sym == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 324)
[declaration] diag(s, n->line, "call[] on undeclared", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 328)
[function] if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 329)
[declaration] diag(s, n->line, "use after set_free", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 340)
[function] if (sc == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 341)
[declaration] diag(s, n->line, "unknown scope", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 345)
[function] if (sc->kind == SYM_ENUM) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 348)
[function] if (sc->node != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 355)
[function] if (it->name[k] != n->name2[k]) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 361)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 370)
[function] if (!found) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 371)
[declaration] diag(s, n->line, "no such enum item", n->name2, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 377)
[function] if (sc->kind == SYM_DATA) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 380)
[function] if (sc->node != (void *)0 && sc->node->a != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 386)
[function] if (e->kind == ND_VAR) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 390)
[function] if (e->name[k] != n->name2[k]) {

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 396)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 404)
[function] if (!found) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 405)
[declaration] diag(s, n->line, "no such data entry", n->name2, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 411)
[declaration] diag(s, n->line, "scope is not enum/data", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 416)
[function] if (n->a == (void *)0 || n->a->kind != ND_VAR) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 417)
[declaration] diag(s, n->line, "bad call target", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 423)
[function] if (fn == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 424)
[declaration] diag(s, n->line, "call of undeclared", n->a->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 428)
[function] if (fn->kind == SYM_FN) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 429)
[function] if (fn->nparams != n->nlist) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 430)
[declaration] diag(s, n->line, "arity mismatch", fn->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 438)
[declaration] diag(s, n->line, "arg out of range", fn->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 446)
[function] if (fn->kind == SYM_EXTERN) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 448)
[declaration] check_expr(s, n->list[i]);
kernel_Workspace/kernel/src/core-lang/sema.c (line 455)
[declaration] diag(s, n->line, "not callable", n->a->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 467)
[function] if (l.t.bits != 0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 468)
[function] if (rhs.is_const) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 469)
[function] if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 470)
[declaration] diag(s, n->line, "value out of range", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 473)
[function] if (rhs.t.bits > l.t.bits) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 474)
[declaration] diag(s, n->line, "narrowing assignment", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 493)
[function] if (cmp) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 498)
[function] if (a.t.bits == 0 && b.t.bits != 0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 499)
[function] if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 500)
[declaration] diag(s, n->line, "value out of range", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 505)
[function] if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 506)
[declaration] diag(s, n->line, "value out of range", (void *)0, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 524)
[declaration] static void check_block(sema_ctx_t *s, ast_node_t *n);
kernel_Workspace/kernel/src/core-lang/sema.c (line 530)
[function] switch (n->kind) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 536)
[declaration] diag(s, n->line, "literal out of type range", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 539)
[function] if (init.t.bits != 0 && init.t.bits > n->type.bits) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 540)
[declaration] diag(s, n->line, "narrowing assignment", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 545)
[function] if (sym != (void *)0 && n->type.neg) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 553)
[declaration] declare(s, "data", SYM_STATIC, n->type, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 559)
[function] if (sym == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 560)
[declaration] diag(s, n->line, "set_free on undeclared", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 562)
[declaration] diag(s, n->line, "double set_free", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 572)
[function] if (n->a != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 573)
[declaration] check_expr(s, n->a);
kernel_Workspace/kernel/src/core-lang/sema.c (line 576)
[declaration] check_block(s, n->b);
kernel_Workspace/kernel/src/core-lang/sema.c (line 578)
[function] if (n->c != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 579)
[function] if (n->c->kind == ND_BLOCK) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 580)
[declaration] check_block(s, n->c);
kernel_Workspace/kernel/src/core-lang/sema.c (line 582)
[declaration] check_stmt(s, n->c);
kernel_Workspace/kernel/src/core-lang/sema.c (line 591)
[function] if (tgt == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 592)
[declaration] diag(s, n->line, "when on undeclared", n->name, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 606)
[function] if (cn[k] != n->name[k]) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 612)
[function] if (!same) {

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 613)
[declaration] diag(s, c->line, "case target mismatch", cn, false);
kernel_Workspace/kernel/src/core-lang/sema.c (line 624)
[declaration] check_block(s, c->b);
kernel_Workspace/kernel/src/core-lang/sema.c (line 631)
[declaration] check_block(s, n);
kernel_Workspace/kernel/src/core-lang/sema.c (line 635)
[declaration] check_expr(s, n);
kernel_Workspace/kernel/src/core-lang/sema.c (line 636)
[declaration] shared_validate(s);
kernel_Workspace/kernel/src/core-lang/sema.c (line 640)
[function] if (n->kind != ND_BLOCK) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 641)
[declaration] shared_validate(s);
kernel_Workspace/kernel/src/core-lang/sema.c (line 647)
[declaration] push_scope(s);
kernel_Workspace/kernel/src/core-lang/sema.c (line 650)
[declaration] check_stmt(s, n->list[i]);
kernel_Workspace/kernel/src/core-lang/sema.c (line 653)
[declaration] pop_scope(s);
kernel_Workspace/kernel/src/core-lang/sema.c (line 659)
[function] switch (n->kind) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 664)
[function] if (sym == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 677)
[declaration] declare(s, n->name, SYM_VAR, n->type, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 681)
[declaration] declare(s, "data", SYM_STATIC, n->type, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 688)
[function] if (sym != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 698)
[declaration] declare(s, n->list[i]->name, SYM_EXTERN, t, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 707)
[function] if (sym != (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 721)
[function] if (prog == (void *)0) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 726)
[declaration] declare_top(s, prog->list[i]);
kernel_Workspace/kernel/src/core-lang/sema.c (line 732)
[function] if (n->kind != ND_FN) {
kernel_Workspace/kernel/src/core-lang/sema.c (line 736)
[declaration] push_scope(s);
kernel_Workspace/kernel/src/core-lang/sema.c (line 739)
[declaration] declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
kernel_Workspace/kernel/src/core-lang/sema.c (line 742)
[declaration] check_block(s, n->a);
kernel_Workspace/kernel/src/core-lang/sema.c (line 744)
[declaration] pop_scope(s);

```



```

kernel_Workspace/kernel/src/core-lang/vm.c (line 19)
[declaration] for (int i = full + (rem ? 1 : 0); i < w; i++) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 23)
[function] if (rem) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 85)
[function] if (a[i] != b[i]) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 95)
[function] if (!sign) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 96)
[declaration] return cmp_u(a, b, w);
kernel_Workspace/kernel/src/core-lang/vm.c (line 113)
[declaration] return cmp_u(x, y, w);
kernel_Workspace/kernel/src/core-lang/vm.c (line 121)
[function] if (bit_get(a, i) && (int)(i + n) < w * 8) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 122)
[declaration] bit_set(t, i + (int)n);
kernel_Workspace/kernel/src/core-lang/vm.c (line 136)
[function] if (bit_get(a, i) && i >= (int)n) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 137)
[declaration] bit_set(t, i - (int)n);
kernel_Workspace/kernel/src/core-lang/vm.c (line 154)
[declaration] cl_shl(shl, rem, 1, w);
kernel_Workspace/kernel/src/core-lang/vm.c (line 160)
[function] if (bit_get(n, i)) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 164)
[function] if (cmp_u(rem, d, w) >= 0) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 166)
[declaration] cl_sub(sub, rem, d, w);
kernel_Workspace/kernel/src/core-lang/vm.c (line 172)
[declaration] bit_set(quo, i);
kernel_Workspace/kernel/src/core-lang/vm.c (line 207)
[function] if (vm->prog->ext_names[i][k] != name[k]) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 213)
[function] if (same) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 224)
[function] if (vm->sp < 256) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 233)
[function] if (vm->sp > 0) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 260)
[function] if (P->main_fn < 0) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 273)
[function] while (vm->depth > 0 && vm->err == CL_OK && !vm->halt_req) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 276)
[function] if (f->pc >= P->code_len) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 287)
[function] switch (in.op) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 292)
[declaration] cl_mask(r.b, w, in.bits);

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 293)
[declaration] push(vm, &r);
kernel_Workspace/kernel/src/core-lang/vm.c (line 297)
[function] if (!f->slive[in.a]) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 301)
[declaration] push(vm, &f->slots[in.a]);
kernel_Workspace/kernel/src/core-lang/vm.c (line 306)
[declaration] cl_mask(a.b, w, in.bits);
kernel_Workspace/kernel/src/core-lang/vm.c (line 312)
[function] if (!vm->glive[in.a]) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 316)
[declaration] push(vm, &vm->globals[in.a]);
kernel_Workspace/kernel/src/core-lang/vm.c (line 321)
[declaration] cl_mask(a.b, w, in.bits);
kernel_Workspace/kernel/src/core-lang/vm.c (line 335)
[function] switch (in.op) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 336)
[declaration] case OP_ADD: cl_add(r.b, a.b, b.b, w); break;
kernel_Workspace/kernel/src/core-lang/vm.c (line 337)
[declaration] case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
kernel_Workspace/kernel/src/core-lang/vm.c (line 338)
[declaration] case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
kernel_Workspace/kernel/src/core-lang/vm.c (line 342)
[declaration] case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
kernel_Workspace/kernel/src/core-lang/vm.c (line 343)
[declaration] case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
kernel_Workspace/kernel/src/core-lang/vm.c (line 352)
[function] if (zero) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 357)
[declaration] cl_divmod(a.b, b.b, w, q, m);
kernel_Workspace/kernel/src/core-lang/vm.c (line 366)
[declaration] cl_mask(r.b, w, in.bits);
kernel_Workspace/kernel/src/core-lang/vm.c (line 367)
[declaration] push(vm, &r);
kernel_Workspace/kernel/src/core-lang/vm.c (line 374)
[declaration] cl_sub(r.b, z, a.b, w);
kernel_Workspace/kernel/src/core-lang/vm.c (line 375)
[declaration] cl_mask(r.b, w, in.bits);
kernel_Workspace/kernel/src/core-lang/vm.c (line 377)
[declaration] push(vm, &r);
kernel_Workspace/kernel/src/core-lang/vm.c (line 383)
[declaration] cl_mask(r.b, w, in.bits);
kernel_Workspace/kernel/src/core-lang/vm.c (line 384)
[declaration] push(vm, &r);
kernel_Workspace/kernel/src/core-lang/vm.c (line 395)
[function] switch (in.op) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 407)
[declaration] push(vm, &r);

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 425)
[declaration] pop(vm);
kernel_Workspace/kernel/src/core-lang/vm.c (line 429)
[function] if (in.a == 1) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 439)
[function] if (!vm->glive[in.a]) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 445)
[function] if (in.a >= 16 || vm->externs[in.a] == (cl_ext_fn)0) {
kernel_Workspace/kernel/src/core-lang/vm.c (line 464)
[declaration] push(vm, &r);
kernel_Workspace/kernel/src/core-lang/vm.c (line 469)
[function] if (vm->depth >= 8) {
kernel_Workspace/kernel/src/displayport/aux.c (line 50)
[function] if (addr + len > 0x400) {
kernel_Workspace/kernel/src/displayport/aux.c (line 65)
[function] if (addr + len > 0x400) {
kernel_Workspace/kernel/src/displayport/aux.c (line 79)
[function] if (pat == 1) {
kernel_Workspace/kernel/src/displayport/aux.c (line 85)
[function] if (train_step >= 2) {
kernel_Workspace/kernel/src/displayport/aux.c (line 100)
[function] if (len > 128) {
kernel_Workspace/kernel/src/displayport/edid.c (line 8)
[function] if (b[0] != 0x00 || b[1] != 0xFF || b[7] != 0x00) {
kernel_Workspace/kernel/src/displayport/edid.c (line 17)
[function] if (clock == 0) {
kernel_Workspace/kernel/src/displayport/edid.c (line 32)
[function] if (hact == 0 || vact == 0) {
kernel_Workspace/kernel/src/displayport/edid.c (line 36)
[function] if (*count >= max) {
kernel_Workspace/kernel/src/displayport/init.c (line 13)
[function] if (dp_is_ready) {
kernel_Workspace/kernel/src/displayport/init.c (line 17)
[declaration] dp_aux_sink_init();
kernel_Workspace/kernel/src/displayport/init.c (line 19)
[function] if (!dp_link_train()) {
kernel_Workspace/kernel/src/displayport/init.c (line 25)
[function] if (!dp_aux_read_edid(raw, 128)) {
kernel_Workspace/kernel/src/displayport/init.c (line 29)
[function] if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
kernel_Workspace/kernel/src/displayport/init.c (line 51)
[function] if (i >= MODE_COUNT) {
kernel_Workspace/kernel/src/displayport/init.c (line 66)
[function] if (MODES[i].id == id) {
kernel_Workspace/kernel/src/displayport/init.c (line 67)
[declaration] dp_op_set_fb(0, MODES[i].w, MODES[i].h, MODES[i].w);
kernel_Workspace/kernel/src/displayport/init.c (line 77)
[declaration] dp_op_state(fb_phys, w, h, stride);

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 82)
[declaration] dp_op_set_fb(phys, w, h, stride);
kernel_Workspace/kernel/src/displayport/init.c (line 91)
[declaration] dp_op_fill(color, x, y, w, h);
kernel_Workspace/kernel/src/displayport/link.c (line 12)
[declaration] dp_aux_read(DPCD_MAX_LINK_RATE, &max_rate, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 13)
[declaration] dp_aux_read(DPCD_MAX_LANE_COUNT, &max_lanes, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 18)
[function] if (lanes > 4) {
kernel_Workspace/kernel/src/displayport/link.c (line 22)
[declaration] dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 23)
[declaration] dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 26)
[declaration] dp_aux_write(DPCD_TRAINING_SET, &tp, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 31)
[declaration] dp_aux_read(DPCD_LANE_STATUS, st, 2);
kernel_Workspace/kernel/src/displayport/link.c (line 33)
[function] if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
kernel_Workspace/kernel/src/displayport/link.c (line 39)
[declaration] dp_aux_write(DPCD_TRAINING_SET, &tp, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 42)
[declaration] dp_aux_read(DPCD_LANE_STATUS, st, 2);
kernel_Workspace/kernel/src/displayport/link.c (line 44)
[function] if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
kernel_Workspace/kernel/src/displayport/link.c (line 50)
[declaration] dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 52)
[function] if (lock != 1) {
kernel_Workspace/kernel/src/displayport/link.c (line 57)
[declaration] dp_aux_write(DPCD_TRAINING_SET, &tp, 1);
kernel_Workspace/kernel/src/displayport/link.c (line 59)
[function] switch (bw) {
kernel_Workspace/kernel/src/displayport/operation.c (line 27)
[function] if (fb_phys == 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 11)
[declaration] extern void console_set_enabled(int on);
kernel_Workspace/kernel/src/editor/editor.c (line 12)
[declaration] extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
kernel_Workspace/kernel/src/editor/editor.c (line 13)
[declaration] extern uint32_t k_input_keycode(void);
kernel_Workspace/kernel/src/editor/editor.c (line 88)
[function] if (bits & (0x80 >> rx)) {
kernel_Workspace/kernel/src/editor/editor.c (line 92)
[declaration] ed_fill(px, py, SCALE, SCALE, color);
kernel_Workspace/kernel/src/editor/editor.c (line 100)
[function] while (*s) {

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 101)
[declaration] ed_char(x, y, *s, color);
kernel_Workspace/kernel/src/editor/editor.c (line 111)
[function] if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }
kernel_Workspace/kernel/src/editor/editor.c (line 116)
[function] while (v > 0 && n < 11) { tmp[n++] = (char)('0' + v % 10); v /= 10; }
kernel_Workspace/kernel/src/editor/editor.c (line 147)
[function] if (v == 0) { tmp[n++] = '0'; }
kernel_Workspace/kernel/src/editor/editor.c (line 149)
[function] while (v > 0 && n < 20) { tmp[n++] = (char)('0' + v % 10); v /= 10; }
kernel_Workspace/kernel/src/editor/editor.c (line 154)
[declaration] log_append(buf);
kernel_Workspace/kernel/src/editor/editor.c (line 165)
[declaration] log_append(buf);
kernel_Workspace/kernel/src/editor/editor.c (line 172)
[function] switch (k) {
kernel_Workspace/kernel/src/editor/editor.c (line 201)
[function] if (tmp[k] == '/' && tmp[k + 1] == '/') { com_at = k; break; }
kernel_Workspace/kernel/src/editor/editor.c (line 208)
[declaration] cl_lexer_init(&l, tmp, ed_strlen(tmp));
kernel_Workspace/kernel/src/editor/editor.c (line 210)
[function] if (com_at >= 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 216)
[declaration] cl_lexer_init(&l, cut, (size_t)com_at);
kernel_Workspace/kernel/src/editor/editor.c (line 221)
[function] if (cl_lex_all(&l, toks, 64, &n) == 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 225)
[function] while (pos < ts) {
kernel_Workspace/kernel/src/editor/editor.c (line 226)
[declaration] ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
kernel_Workspace/kernel/src/editor/editor.c (line 240)
[function] while (tmp[pos] && (com_at < 0 || pos < com_at)) {
kernel_Workspace/kernel/src/editor/editor.c (line 241)
[declaration] ed_char(x + (uint32_t)pos * CHAR_W, y, tmp[pos], COL_FG);
kernel_Workspace/kernel/src/editor/editor.c (line 245)
[function] if (com_at >= 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 247)
[declaration] ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
kernel_Workspace/kernel/src/editor/editor.c (line 285)
[function] if (sx >= 0 && sx < (int)fw && sy >= 0 && sy < (int)fh) {
kernel_Workspace/kernel/src/editor/editor.c (line 296)
[declaration] ed_fill(0, 0, fw, fh, COL_BG);
kernel_Workspace/kernel/src/editor/editor.c (line 298)
[declaration] ed_fill(0, 0, fw, TOP_BAR, COL_BAR);
kernel_Workspace/kernel/src/editor/editor.c (line 299)
[declaration] ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
kernel_Workspace/kernel/src/editor/editor.c (line 300)
[declaration] ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 312)
[function] if (row == caret_l) {
kernel_Workspace/kernel/src/editor/editor.c (line 313)
[declaration] ed_fill(GUTTER, y, (uint32_t)cols * CHAR_W, CHAR_H, 0xFF2D2D2D);
kernel_Workspace/kernel/src/editor/editor.c (line 317)
[declaration] ed_num(nb, row + 1);
kernel_Workspace/kernel/src/editor/editor.c (line 318)
[declaration] ed_text(8, y, nb, COL_GUTTER);
kernel_Workspace/kernel/src/editor/editor.c (line 320)
[declaration] draw_hl(row, GUTTER + 4, y);
kernel_Workspace/kernel/src/editor/editor.c (line 326)
[function] if (r >= 0 && r < rows) {
kernel_Workspace/kernel/src/editor/editor.c (line 330)
[declaration] ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
kernel_Workspace/kernel/src/editor/editor.c (line 334)
[function] if (show_log) {
kernel_Workspace/kernel/src/editor/editor.c (line 337)
[declaration] ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);
kernel_Workspace/kernel/src/editor/editor.c (line 340)
[declaration] ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
kernel_Workspace/kernel/src/editor/editor.c (line 344)
[declaration] ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);
kernel_Workspace/kernel/src/editor/editor.c (line 351)
[declaration] ed_num(tmp, caret_l + 1);
kernel_Workspace/kernel/src/editor/editor.c (line 355)
[declaration] ed_num(tmp, caret_c + 1);
kernel_Workspace/kernel/src/editor/editor.c (line 359)
[declaration] ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
kernel_Workspace/kernel/src/editor/editor.c (line 360)
[declaration] ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);
kernel_Workspace/kernel/src/editor/editor.c (line 362)
[declaration] draw_mouse();
kernel_Workspace/kernel/src/editor/editor.c (line 413)
[function] if (caret_c > 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 425)
[function] if (plen + slen < ED_LINE_LEN - 1) {
kernel_Workspace/kernel/src/editor/editor.c (line 448)
[function] if (caret_c < len) {
kernel_Workspace/kernel/src/editor/editor.c (line 485)
[declaration] render();
kernel_Workspace/kernel/src/editor/editor.c (line 496)
[function] if (P == (void *)0) {
kernel_Workspace/kernel/src/editor/editor.c (line 499)
[function] while (pre[i]) { msg[i] = pre[i]; i++; }
kernel_Workspace/kernel/src/editor/editor.c (line 501)
[declaration] ed_num(tmp, (int)el);
kernel_Workspace/kernel/src/editor/editor.c (line 507)
[declaration] if (em) log_append(em);

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 511)
[declaration] cl_vm_init(&ed_vm, P);
kernel_Workspace/kernel/src/editor/editor.c (line 512)
[declaration] cl_bridge_init(&ed_vm, 0);
kernel_Workspace/kernel/src/editor/editor.c (line 514)
[declaration] cl_vm_register_extern(&ed_vm, "put", ed_put);
kernel_Workspace/kernel/src/editor/editor.c (line 515)
[declaration] cl_vm_register_extern(&ed_vm, "putc", ed_putc);
kernel_Workspace/kernel/src/editor/editor.c (line 517)
[declaration] log_append("== run ==");
kernel_Workspace/kernel/src/editor/editor.c (line 523)
[function] while (ok[i]) { msg[i] = ok[i]; i++; }
kernel_Workspace/kernel/src/editor/editor.c (line 536)
[function] if (gfx_fb_info_raw(&w, &h, &s, &base, &fliprows) != 0 || base == 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 549)
[function] if (path != (void *)0) {
kernel_Workspace/kernel/src/editor/editor.c (line 554)
[function] if (n > 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 563)
[function] if (c == '\n') {
kernel_Workspace/kernel/src/editor/editor.c (line 572)
[function] if (col > 0 || nlines == 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 579)
[declaration] console_set_enabled(0);
kernel_Workspace/kernel/src/editor/editor.c (line 580)
[declaration] mouse_init();
kernel_Workspace/kernel/src/editor/editor.c (line 584)
[function] while (running) {
kernel_Workspace/kernel/src/editor/editor.c (line 590)
[function] if (mouse_poll(&dx, &dy, &dz, &btn)) {
kernel_Workspace/kernel/src/editor/editor.c (line 599)
[function] if (dz != 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 601)
[declaration] clamp_scroll(rows);
kernel_Workspace/kernel/src/editor/editor.c (line 605)
[function] if (btn & 1 && !(mbuttons & 1)) {
kernel_Workspace/kernel/src/editor/editor.c (line 606)
[function] if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
kernel_Workspace/kernel/src/editor/editor.c (line 618)
[declaration] clamp_scroll(rows);
kernel_Workspace/kernel/src/editor/editor.c (line 629)
[function] if (k != 0) {
kernel_Workspace/kernel/src/editor/editor.c (line 630)
[function] switch (k) {
kernel_Workspace/kernel/src/editor/editor.c (line 632)
[declaration] case EDK_ENTER: do_enter(); dirty = 1; break;
kernel_Workspace/kernel/src/editor/editor.c (line 633)
[declaration] case EDK_BACKSPACE: do_backspace(); dirty = 1; break;

```



```

kernel_Workspace/kernel/src/editor/editor.c (line 634)
[declaration] case EDK_DELETE: do_delete(); dirty = 1; break;
kernel_Workspace/kernel/src/editor/editor.c (line 635)
[declaration] case EDK_TAB: ins_char(' '); ins_char(' ');
kernel_Workspace/kernel/src/editor/editor.c (line 636)
[declaration] ins_char(' '); ins_char(' '); dirty = 1; break;
kernel_Workspace/kernel/src/editor/editor.c (line 643)
[declaration] case EDK_F5: ed_compile_run(); dirty = 1; break;
kernel_Workspace/kernel/src/editor/editor.c (line 646)
[function] if (k >= 32 && k < 0x100) {
kernel_Workspace/kernel/src/editor/editor.c (line 647)
[declaration] ins_char((char)k);
kernel_Workspace/kernel/src/editor/editor.c (line 653)
[declaration] clamp_scroll(rows);
kernel_Workspace/kernel/src/editor/editor.c (line 656)
[function] if (dirty) {
kernel_Workspace/kernel/src/editor/editor.c (line 657)
[declaration] render();
kernel_Workspace/kernel/src/editor/editor.c (line 660)
[declaration] __asm__ volatile("pause");
kernel_Workspace/kernel/src/editor/editor.c (line 663)
[declaration] console_set_enabled(1);
kernel_Workspace/kernel/src/editor/mouse.c (line 5)
[declaration] __asm__ volatile("outb %0, %1" :: "a"(val), "Nd"(port));
kernel_Workspace/kernel/src/editor/mouse.c (line 11)
[declaration] __asm__ volatile("inb %1, %0" : "=a"(v) : "Nd"(port));
kernel_Workspace/kernel/src/editor/mouse.c (line 17)
[function] while (inb(0x64) & 0x02) { }
kernel_Workspace/kernel/src/editor/mouse.c (line 22)
[function] while (!(inb(0x64) & 0x01)) { }
kernel_Workspace/kernel/src/editor/mouse.c (line 27)
[declaration] wait_write(); outb(0x64, 0xD4);
kernel_Workspace/kernel/src/editor/mouse.c (line 28)
[declaration] wait_write(); outb(0x60, cmd);
kernel_Workspace/kernel/src/editor/mouse.c (line 29)
[declaration] wait_read(); inb(0x60);
kernel_Workspace/kernel/src/editor/mouse.c (line 34)
[declaration] wait_write(); outb(0x64, 0xD4);
kernel_Workspace/kernel/src/editor/mouse.c (line 35)
[declaration] wait_write(); outb(0x60, 0xF3);
kernel_Workspace/kernel/src/editor/mouse.c (line 36)
[declaration] wait_read(); inb(0x60);
kernel_Workspace/kernel/src/editor/mouse.c (line 38)
[declaration] wait_write(); outb(0x64, 0xD4);
kernel_Workspace/kernel/src/editor/mouse.c (line 39)
[declaration] wait_write(); outb(0x60, r);
kernel_Workspace/kernel/src/editor/mouse.c (line 40)
[declaration] wait_read(); inb(0x60);

```



```

kernel_Workspace/kernel/src/editor/mouse.c (line 45)
[declaration] wait_write(); outb(0x64, 0xA8);
kernel_Workspace/kernel/src/editor/mouse.c (line 47)
[declaration] mouse_cmd(0xF6);
kernel_Workspace/kernel/src/editor/mouse.c (line 49)
[declaration] mouse_rate(200);
kernel_Workspace/kernel/src/editor/mouse.c (line 50)
[declaration] mouse_rate(100);
kernel_Workspace/kernel/src/editor/mouse.c (line 51)
[declaration] mouse_rate(80);
kernel_Workspace/kernel/src/editor/mouse.c (line 53)
[declaration] mouse_cmd(0xF4);
kernel_Workspace/kernel/src/editor/mouse.c (line 62)
[function] while (inb(0x64) & 0x01) {
kernel_Workspace/kernel/src/editor/mouse.c (line 66)
[function] if (!(st & 0x20)) {
kernel_Workspace/kernel/src/editor/mouse.c (line 70)
[function] if (idx == 0 && !(d & 0x08)) {
kernel_Workspace/kernel/src/editor/mouse.c (line 76)
[function] if (idx == 4) {
kernel_Workspace/kernel/src/hdmi/init.c (line 8)
[function] if (hdmi_is_ready) {
kernel_Workspace/kernel/src/hdmi/init.c (line 12)
[function] if (fb_phys == 0 || w == 0 || h == 0 || stride < w) {
kernel_Workspace/kernel/src/hdmi/init.c (line 16)
[declaration] hdmi_op_set_fb(fb_phys, w, h, stride);
kernel_Workspace/kernel/src/hdmi/init.c (line 17)
[function] if (!hdmi_op_ready()) {
kernel_Workspace/kernel/src/hdmi/init.c (line 22)
[function] if (mode != NULL && !hdmi_mode_apply(mode)) {
kernel_Workspace/kernel/src/hdmi/init.c (line 32)
[declaration] return hdmi_is_ready && hdmi_op_ready();
kernel_Workspace/kernel/src/hdmi/init.c (line 37)
[declaration] hdmi_op_state(out);
kernel_Workspace/kernel/src/hdmi/init.c (line 42)
[function] if (phys == 0 || w == 0 || h == 0 || stride < w) {
kernel_Workspace/kernel/src/hdmi/init.c (line 46)
[declaration] hdmi_op_set_fb(phys, w, h, stride);
kernel_Workspace/kernel/src/hdmi/init.c (line 53)
[function] if (!hdmi_ready()) {
kernel_Workspace/kernel/src/hdmi/init.c (line 59)
[declaration] hdmi_op_state(&caps);
kernel_Workspace/kernel/src/hdmi/init.c (line 60)
[function] if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
kernel_Workspace/kernel/src/hdmi/init.c (line 64)
[declaration] return hdmi_mode_apply(mode);
kernel_Workspace/kernel/src/hdmi/init.c (line 70)
[function] if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {

```

```

kernel_Workspace/kernel/src/hdmi/interface.c (line 9)
[function] if (owner == 0) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 14)
[function] if (atomic_compare_exchange_strong(&port_owner, &expected, owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 29)
[declaration] return atomic_load(&port_owner);
kernel_Workspace/kernel/src/hdmi/interface.c (line 39)
[function] if (owner != 0 && !hdmi_iface_check(owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 43)
[declaration] return hdmi_init_with(fb_phys, w, h, stride);
kernel_Workspace/kernel/src/hdmi/interface.c (line 48)
[declaration] return hdmi_ready();
kernel_Workspace/kernel/src/hdmi/interface.c (line 53)
[function] if (!hdmi_iface_check(owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 57)
[declaration] return hdmi_mode_set_by_id(id);
kernel_Workspace/kernel/src/hdmi/interface.c (line 62)
[declaration] return hdmi_mode_current_raw(id, w, h, r);
kernel_Workspace/kernel/src/hdmi/interface.c (line 67)
[declaration] return hdmi_mode_at_raw(i, id, w, h, r);
kernel_Workspace/kernel/src/hdmi/interface.c (line 72)
[function] if (!hdmi_iface_check(owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 76)
[declaration] return hdmi_submit_fill(color, x, y, w, h);
kernel_Workspace/kernel/src/hdmi/interface.c (line 81)
[function] if (!hdmi_iface_check(owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 85)
[declaration] return hdmi_poll(out_seq);
kernel_Workspace/kernel/src/hdmi/interface.c (line 90)
[declaration] hdmi_caps_raw(w, h, s, phys);
kernel_Workspace/kernel/src/hdmi/interface.c (line 95)
[function] if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL ||
s == NULL) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 101)
[declaration] hdmi_caps_raw(&ww, &hh, &ss, &p);
kernel_Workspace/kernel/src/hdmi/interface.c (line 103)
[function] if (p == 0 || !hdmi_fb_grant()) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 116)
[function] if (!hdmi_iface_check(owner)) {
kernel_Workspace/kernel/src/hdmi/interface.c (line 120)
[declaration] hdmi_fb_revoke();
kernel_Workspace/kernel/src/hdmi/mode.c (line 17)
[declaration] return (uint32_t)(sizeof(MODES) / sizeof(MODES[0]));
kernel_Workspace/kernel/src/hdmi/mode.c (line 22)
[function] if (i >= hdmi_mode_count()) {
kernel_Workspace/kernel/src/hdmi/mode.c (line 32)
[function] if (MODES[i].w == w && MODES[i].h == h && MODES[i].refresh == refresh) {
kernel_Workspace/kernel/src/hdmi/mode.c (line 43)
[function] if (MODES[i].id == id) {

```

```

kernel_Workspace/kernel/src/hdmi/mode.c (line 53)
[function] if (m == NULL || m->w == 0 || m->h == 0 || m->refresh == 0 || m->clock_khz
== 0) {
kernel_Workspace/kernel/src/hdmi/mode.c (line 58)
[declaration] hdmi_op_state(&caps);
kernel_Workspace/kernel/src/hdmi/mode.c (line 64)
[function] if (!hdmi_mode_valid(m)) {
kernel_Workspace/kernel/src/hdmi/mode.c (line 68)
[declaration] hdmi_op_set_mode(m);
kernel_Workspace/kernel/src/hdmi/mode.c (line 81)
[function] if (m == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 7)
[declaration] return (void *) (DIRECT_BASE + phys);
kernel_Workspace/kernel/src/hdmi/operation.c (line 22)
[function] if (out != NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 44)
[function] if (t == NULL || !op_ready) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 50)
[function] if (stride < cur.w || dst_phys == 0) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 55)
[function] if (dst == NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 59)
[function] if (t->kind == HDMI_TR_FILL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 60)
[function] if (t->x >= cur.w || t->y >= cur.h) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 73)
[function] if (t->kind == HDMI_TR_BLIT) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 74)
[function] if (t->src_phys == 0 || t->stride == 0 || t->x >= cur.w || t->y >= cur.h)
{
kernel_Workspace/kernel/src/hdmi/operation.c (line 78)
[function] if (src == NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 91)
[function] if (t->kind == HDMI_TR_FLIP && t->dst_phys != 0) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 98)
[function] if (t == NULL || !op_ready || pending >= HDMI_MAX_TRANSFERS) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 102)
[function] if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind !=
HDMI_TR_FLIP) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 107)
[function] if (!slot_used[i]) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 110)
[function] if (next_seq == 0) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 116)
[declaration] hdmi_op_exec(&slots[i]);
kernel_Workspace/kernel/src/hdmi/operation.c (line 127)
[function] if (out_seq == NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 132)
[function] if (slot_used[i] && slot_done[i]) {

```

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 159)
[declaration] return hdmi_submit(&t);
kernel_Workspace/kernel/src/hdmi/operation.c (line 164)
[function] if (m == NULL) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 178)
[function] if (cur.fb_phys == 0) {
kernel_Workspace/kernel/src/hdmi/operation.c (line 198)
[function] if (w == NULL || h == NULL || s == NULL || phys == NULL) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 8)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 32)
[declaration] return heap_usable_size(ptr);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 38)
[declaration] return dbg_alloc(size);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 40)
[declaration] return heap_alloc(size);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 44)
[function] void kfree(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 63)
[declaration] return kzalloc(count * size);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 70)
[declaration] return heap_alloc_aligned(size, align);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 74)
[function] size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 79)
[declaration] kfree(ptr);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 95)
[declaration] return heap_alloc(pages * ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 98)
[function] void kfree_pages(void *ptr, const size_t pages) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 103)
[declaration] kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc
bloku=%zu B) dla ptr=%p – odmowa zwolnienia\n", pages, usable, ptr);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 118)
[function] uint64_t kvirt_to_phys(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 123)
[declaration] return arch_virt_to_phys(ptr);
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 126)
[function] void kalloc_dump(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 11)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 56)
[function] void dbg_free(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 61)
[declaration] kprintf("dbg_free: bad magic at %p\n", ptr);
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 73)
[function] if (!leak_untrack(ptr, &leaked_size, &alloc_caller)) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 74)
[declaration] kprintf("dbg_free: double free or unknown ptr %p\n", ptr);

```

```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 90)
[function] size_t dbg_usable_size(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 97)
[function] bool dbg_verify(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 104)
[function] void mm_debug_dump(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 4)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 22)
[function] bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t
*out_id) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 25)
[function] if (!leak_table[i].used) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 33)
[declaration] smp_lock_release(&leak_smp_lock);
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 41)
[function] bool leak_track(void *ptr, size_t size, uint64_t caller) { return
leak_track_id(ptr, size, caller, NULL); }
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 42)
[function] bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 45)
[function] if (leak_table[i].used && leak_table[i].ptr == ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 49)
[declaration] smp_lock_release(&leak_smp_lock);
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 56)
[function] bool leak_contains(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 59)
[function] if (leak_table[i].used && leak_table[i].ptr == ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 60)
[declaration] smp_lock_release(&leak_smp_lock);
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 67)
[function] size_t leak_count(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 74)
[function] void leak_dump(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 4)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 15)
[function] void alloc_stats_note_alloc(size_t size) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 23)
[function] void alloc_stats_note_free(size_t size) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 31)
[function] size_t alloc_stats_live_count(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 37)
[function] size_t alloc_stats_live_bytes(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 43)
[function] size_t alloc_stats_peak_bytes(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 49)
[function] size_t alloc_stats_total_allocs(void) {

```

```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 55)
[function] size_t alloc_stats_total_frees(void) {
kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 61)
[function] void alloc_stats_dump(void) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 6)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 49)
[function] if ((size_t)*p == idx) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 56)
[function] bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb,
buddy_unmap_cb unmap_cb) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 95)
[function] bool buddy_ready(void) { return buddy_initialized; }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 109)
[declaration] fl_push(cur, buddy_idx);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 116)
[declaration] fl_push(o, idx);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 133)
[declaration] return buddy_alloc(p);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 136)
[function] void buddy_free(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 153)
[declaration] fl_remove(cur, buddy_idx);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 163)
[function] size_t buddy_block_size(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 171)
[function] size_t buddy_stat_used_bytes(void) { return buddy_used_bytes; }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 172)
[function] size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 173)
[function] void buddy_dump(void) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 175)
[declaration] kprintf("BUDDY: not initialized\n");
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 32)
[function] if (!pmm_alloc_frame(&phys)) {
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 37)
[declaration] mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 39)
[function] if (p != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 40)
[declaration] pmm_free_frame(p);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 52)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 58)
[declaration] mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 60)
[function] if (p != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 61)
[declaration] pmm_free_frame(p);

```

```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 80)
[function] if (p != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 81)
[declaration] mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 82)
[declaration] pmm_free_frame(p);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 147)
[declaration] return slab_alloc(size);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 151)
[declaration] return buddy_alloc(size);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 167)
[declaration] return slab_alloc(size);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 171)
[declaration] return buddy_alloc_aligned(size, align);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 197)
[declaration] slab_free(ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 203)
[declaration] buddy_free(ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 217)
[declaration] return slab_usable_size(ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 222)
[declaration] return buddy_block_size(ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 238)
[declaration] slab_dump();
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 9)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 92)
[function] if ((uint32_t)*p == page) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 116)
[declaration] bitmap_free(&slab_page_bitmap, page);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 125)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 126)
[declaration] bitmap_free(&slab_page_bitmap, page);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 207)
[declaration] slab_bitmap_clear_all(&slab_descs[i]);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 218)
[function] if (slab_caches[i].objects_per_slab > SLAB_MAX_OBJECTS_PER_SLAB) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 255)
[function] if (!slab_grow(cache_id)) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 256)
[declaration] slab_unlock();
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 328)
[declaration] slab_unlock();
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 346)
[declaration] partial_remove(c, page);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 350)
[declaration] mapping_unmap_range(MAPPING_KERNEL, slab_va, ARCH_PAGE_SIZE);

```



```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 352)
[function] if (phys != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 353)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 356)
[declaration] bitmap_free(&slab_page_bitmap, page);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 400)
[function] if (d->state != 1) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 424)
[function] if (d->state != 1) {
kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 439)
[declaration] kprintf("SLAB: not initialized\n");
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 65)
[declaration] return bitmap_words_for_bits(bit_count) * sizeof(uint64_t);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 132)
[declaration] bitmap_mark_tail_used(bm);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 298)
[function] if (bm->bits[i] != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 329)
[function] if (idx < bm->bit_count) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 339)
[function] if (~w != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 343)
[function] if (idx < bm->bit_count) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 377)
[function] if (i == start_word && start_bit != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 381)
[function] if (i == end_word) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 385)
[function] if (w != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 389)
[function] if (idx >= start && idx < limit) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 474)
[declaration] return bitmap_alloc(bm);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 489)
[function] if (free_bit == BITMAP_INVALID) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 500)
[function] if (bitmap_test_range_free(bm, candidate, count)) {
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 501)
[declaration] bitmap_set_range(bm, candidate, count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 509)
[function] if (used == BITMAP_INVALID) {
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 64)
[declaration] return arch_phys_to_virt(frame);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 159)
[function] if (r->type != ARCH_MEM_TYPE_USABLE) {
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 171)
[function] if (frame_count == 0) {

```



```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 175)
[declaration] bitmap_clear_range(&frame_bitmap, pfn_start, frame_count);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 177)
[function] if (allocated_frames >= frame_count) {
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 183)
[declaration] dma32_account_free(pfn_start, frame_count);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 230)
[declaration] frame_free(frame);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 345)
[declaration] __asm__ volatile("" ::: "memory");
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 377)
[function] if (aligned > limit - count) {
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 381)
[function] if (bitmap_test_range_free(&frame_bitmap, aligned, count)) {
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 382)
[declaration] bitmap_set_range(&frame_bitmap, aligned, count);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 385)
[declaration] dma32_account_alloc(aligned, count);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 6)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 8)
[declaration] static void pmm_panic(const char *msg) __attribute__((noreturn));
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 31)
[declaration] pmm_panic("pmm_unlock without lock");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 80)
[function] if (!frame_zero(phys)) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 93)
[declaration] pmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 98)
[declaration] pmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 99)
[declaration] kprintf("pmm_init: arch memory not initialized\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 104)
[declaration] pmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 105)
[declaration] kprintf("pmm_init: frame allocator initialization failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 220)
[declaration] return pmm_alloc_frames(frames, out_phys);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 236)
[declaration] return pmm_alloc_zero_frame(out_phys);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 251)
[declaration] pmm_free_frames(frame, frames);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 275)
[declaration] return pmm_alloc_frames_aligned(frames, align_frames, out_phys);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 328)
[declaration] return pmm_free_frames(phys, frames);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 337)
[declaration] return frame_total();

```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 346)
[declaration] return frame_free_count();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 355)
[declaration] return frame_allocated();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 394)
[declaration] return frame_zone_dma32_total();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 403)
[declaration] return frame_zone_dma32_free();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 412)
[declaration] return frame_zone_normal_total();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 421)
[declaration] return frame_zone_normal_free();
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 452)
[declaration] kprintf("PMM: not initialized\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 490)
[function] if (!pmm_alloc_zero_frame(&frames[i])) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 491)
[declaration] kprintf("pmm_self_test: single frame alloc failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 495)
[function] if (!arch_is_page_aligned(frames[i])) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 496)
[declaration] kprintf("pmm_self_test: frame not aligned\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 504)
[function] if (p[j] != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 505)
[declaration] kprintf("pmm_self_test: zero frame not zero\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 512)
[function] if (!pmm_free_frame(frames[i])) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 513)
[declaration] kprintf("pmm_self_test: single frame free failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 522)
[declaration] kprintf("pmm_self_test: contiguous alloc failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 527)
[declaration] kprintf("pmm_self_test: contiguous not aligned\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 532)
[declaration] kprintf("pmm_self_test: contiguous free failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 540)
[declaration] kprintf("pmm_self_test: 2M aligned alloc failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 547)
[declaration] kprintf("pmm_self_test: 2M alignment failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 552)
[declaration] kprintf("pmm_self_test: 2M aligned free failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 560)
[declaration] kprintf("pmm_self_test: bytes alloc failed\n");
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 572)
[function] if (p[i] != 0) {
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 573)
[declaration] kprintf("pmm_self_test: bytes alloc not zero\n");

```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 579)
[declaration] kprintf("pmm_self_test: bytes free failed\n");
kernel_Workspace/kernel/src/mm/alloc/special/contiguous.c (line 9)
[declaration] return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
kernel_Workspace/kernel/src/mm/alloc/special/dma.c (line 10)
[declaration] return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
kernel_Workspace/kernel/src/mm/alloc/special/dma.c (line 47)
[declaration] pmm_free_frames(phys, frames);
kernel_Workspace/kernel/src/mm/alloc/special/dma.c (line 76)
[declaration] vmm_unmap_device((uint64_t)(uintptr_t)virt, len);
kernel_Workspace/kernel/src/mm/alloc/special/dma.c (line 88)
[declaration] __asm__ volatile("clflush (%0)" :: "r"(a) : "memory");
kernel_Workspace/kernel/src/mm/alloc/special/dma.c (line 91)
[declaration] __asm__ volatile("" ::: "memory");
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 27)
[declaration] mapping_init();
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 70)
[function] if (!paging_map_page_in(pml4, v, p, pte_flags)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 102)
[declaration] paging_unmap_page_in(pml4, virt + (uint64_t)i * ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 128)
[function] if (!paging_is_mapped_in(pml4, v)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 136)
[function] if (!paging_set_flags_in(pml4, v, pte_flags)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 147)
[declaration] return paging_translate_in(resolve_space(space), virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 152)
[declaration] return paging_is_mapped_in(resolve_space(space), virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 164)
[declaration] __asm__ volatile("" ::: "memory");
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.c (line 199)
[declaration] copy_page_by_phys(dp, sp);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 5)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 78)
[declaration] kprintf("page_init: pmm not ready\n");
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 85)
[declaration] kprintf("page_init: no arch mem info\n");
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 141)
[declaration] page_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 168)
[declaration] page_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 205)
[declaration] page_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 226)
[declaration] page_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 246)
[declaration] page_unlock();

```

```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 253)
[declaration] page_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 289)
[declaration] return arch_phys_to_virt(phys);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 331)
[declaration] return page_from_phys(arch_virt_to_phys(va));
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 379)
[function] if (page_descs[i].refcount != 0) {
kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 390)
[declaration] kprintf("PAGE: not initialized\n");
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 7)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 201)
[declaration] vmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 213)
[declaration] : pmm_alloc_frame(&phys);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 215)
[function] if (!got) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 222)
[function] if (!paging_map_page(virt, phys, pte)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 223)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 235)
[function] if (paging_is_mapped(virt)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 237)
[declaration] paging_unmap_page(virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 238)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 242)
[declaration] bitmap_free_range(&vmm_bitmap, start, pages);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 243)
[declaration] vmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 258)
[declaration] return vmm_alloc_aligned(bytes, ARCH_PAGE_SIZE, flags, out_virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 284)
[declaration] vmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 295)
[function] if (!paging_map_page(virt, frame_phys, pte)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 305)
[declaration] paging_unmap_page(vmm_page_virt(start + i));
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 308)
[declaration] bitmap_free_range(&vmm_bitmap, start, pages);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 309)
[declaration] vmm_unlock();
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 349)
[function] if (paging_is_mapped(v)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 351)
[declaration] paging_unmap_page(v);

```

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 352)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 395)
[function] if (paging_is_mapped(v)) {
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 396)
[declaration] paging_unmap_page(v);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 419)
[declaration] return paging_translate(virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 466)
[declaration] kprintf("VMM: not initialized\n");
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 115)
[declaration] arch_memory_boot_alloc(4096, 4096, &phys);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 139)
[declaration] tlb_flush_page_addr(virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 172)
[declaration] tlb_flush_page_addr(virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 184)
[declaration] tlb_flush_page_addr(virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 216)
[declaration] return (e1 & 0x0000FFFFFFFFE00000ULL) | (virt & 0x1FFFFFFULL);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 227)
[declaration] return (e2 & 0x0000FFFFFFFFF00000ULL) | (virt & 0xFFFFFULL);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 273)
[declaration] __asm__ volatile("mrs %0, ttbr1_el1" : "=r"(cur));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 297)
[function] if (idx == 0 && pa >= 0x200000 * 512) {
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 306)
[declaration] __asm__ volatile("msr mair_el1, %0" :: "r"(mair));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 320)
[declaration] __asm__ volatile("msr tcr_el1, %0" :: "r"(tcr));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 322)
[declaration] __asm__ volatile("msr ttbr0_el1, %0" :: "r"(kernel_l0));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 323)
[declaration] __asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 325)
[declaration] __asm__ volatile("isb");
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 326)
[declaration] __asm__ volatile("tlbi vmalle1");
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 327)
[declaration] __asm__ volatile("dsb sy");
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 328)
[declaration] __asm__ volatile("isb");
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 340)
[declaration] __asm__ volatile("mrs %0, ttbr1_el1" : "=r"(v));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 347)
[declaration] __asm__ volatile("msr ttbr0_el1, %0" :: "r"(pml4_phys));
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 348)
[declaration] __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));

```

```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 349)
[declaration] __asm__ volatile("isb");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 18)
[declaration] __asm__ volatile("dsb sy");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 19)
[declaration] __asm__ volatile("tlbi vmalle1");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 20)
[declaration] __asm__ volatile("dsb sy");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 21)
[declaration] __asm__ volatile("isb");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 31)
[declaration] __asm__ volatile("dsb ishst");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 32)
[declaration] __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 33)
[declaration] __asm__ volatile("dsb ish");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 34)
[declaration] __asm__ volatile("isb");
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 45)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 50)
[declaration] tlb_flush_page_addr(addr + i * 4096);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 37)
[declaration] arch_mem_panic("bad multiboot2 magic");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 41)
[declaration] arch_mem_panic("bad multiboot2 info address");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 50)
[declaration] arch_mem_panic("multiboot2 info too small");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 60)
[function] if (tag->size < 8) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 64)
[function] if (tag->type == MULTIBOOT_TAG_TYPE_END) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 68)
[function] if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 75)
[function] if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 79)
[function] while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 83)
[function] if (n < ARCH_MAX_MEM_REGIONS) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 106)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 108)
[declaration] static void arch_mem_panic(const char *msg) __attribute__((noreturn));
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 221)
[function] while (j > 0 && entries[j - 1].base > key.base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 244)
[function] if (r.base < last_end) {

```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 247)
[function] if (r.len <= delta) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 264)
[declaration] arch_mem_panic("too many memory regions");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 293)
[function] if (r_end <= base || r.base >= end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 294)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 299)
[function] if (r.base < base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 305)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 312)
[function] if (ov_end > ov_base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 318)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, reserved);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 322)
[function] if (r_end > end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 328)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, after);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 333)
[declaration] arch_mem_panic("memory reserve produced too many regions");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 351)
[function] if (r.type != ARCH_MEM_TYPE_USABLE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 352)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 359)
[function] if (aligned_end <= aligned_base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 365)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, tiny);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 370)
[function] if (r.base < aligned_base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 376)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 386)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, usable);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 390)
[function] if (aligned_end < r_end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 396)
[declaration] map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, after);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 401)
[declaration] arch_mem_panic("memory alignment produced too many regions");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 421)
[function] if (end > max_address) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 425)
[function] if (r->type == ARCH_MEM_TYPE_USABLE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 426)
[function] if (total_usable > ARCH_U64_MAX - r->len) {

```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 432)
[function] if (end > max_usable_address) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 452)
[declaration] arch_mem_panic("arch_memory_init() called twice");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 456)
[declaration] arch_mem_panic("empty memory map");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 460)
[declaration] arch_mem_panic("memory map too large");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 475)
[function] if (base >= end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 479)
[function] if (mem.count > 0) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 483)
[function] if (base < last_end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 487)
[function] if (base >= end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 497)
[declaration] map_add(mem.regions, &mem.count, ARCH_MAX_MEM_REGIONS, r);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 501)
[declaration] arch_mem_panic("no valid memory regions after parsing");
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 571)
[function] if (r->type != ARCH_MEM_TYPE_USABLE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 577)
[function] if (base >= r->base && end <= r_end) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 619)
[function] if (r->type != ARCH_MEM_TYPE_USABLE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 626)
[function] if (candidate < r->base) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 630)
[function] if (candidate <= r_end && (r_end - candidate) >= len) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 6)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 8)
[declaration] static void paging_panic(const char *msg) __attribute__((noreturn));
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 11)
[declaration] kprintf("arch/x86_64/paging.c panic: %s\n", msg);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 25)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 26)
[declaration] paging_panic("boot physical offset not set");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 29)
[declaration] return (void *) (uintptr_t) (phys + boot_phys_offset);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 40)
[declaration] __asm__ volatile("" ::: "memory");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 45)
[declaration] return (size_t) ((virt >> 39) & 0x1FF);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 50)
[declaration] return (size_t) ((virt >> 30) & 0x1FF);

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 55)
[declaration] return (size_t)((virt >> 21) & 0x1FF);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 60)
[declaration] return (size_t)((virt >> 12) & 0x1FF);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 67)
[declaration] __asm__ volatile("mov %%cr3, %0" : "=r"(cr3));
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 74)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 79)
[declaration] tlb_flush_page_addr(addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 93)
[function] if (pmm_ready()) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 94)
[function] if (!pmm_alloc_zero_frame(&phys)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 95)
[declaration] paging_panic("cannot allocate page table page (pmm)");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 103)
[declaration] paging_panic("cannot allocate page table page");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 106)
[declaration] zero_page_phys(phys);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 117)
[function] if (entry & PTE_PRESENT) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 118)
[function] if (entry & PTE_PAGE_SIZE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 138)
[function] if (i < pages) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 152)
[function] if (large_flags & PTE_USER) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 171)
[function] if (flags & PTE_USER) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 182)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 214)
[declaration] paging_flush_page(virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 222)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 250)
[function] if ((old & PTE_PRESENT) && !(old & PTE_PAGE_SIZE)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 251)
[declaration] paging_panic("cannot map 2M page over existing 4K table");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 265)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 269)
[function] if (len == 0) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 279)
[function] if (len > UINT64_MAX - virt || len > UINT64_MAX - phys) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 286)
[function] while (virt < virt_end) {

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 294)
[function] if (can_use_2m) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 295)
[function] if (!map_page_2m(virt, phys, flags)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 302)
[function] if (!paging_map_page(virt, phys, flags)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 311)
[declaration] paging_flush_tlb_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 318)
[function] if ((phys_offset & PAGING_PAGE_MASK) != 0) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 319)
[declaration] paging_panic("boot physical offset must be page aligned");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 333)
[function] if (!arch_memory_ready()) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 334)
[declaration] paging_panic("arch memory not initialized");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 337)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 338)
[declaration] paging_panic("boot physical offset not set");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 345)
[function] if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 351)
[function] if (info == NULL) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 352)
[declaration] paging_panic("no arch memory info");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 357)
[function] if (phys_limit == 0 || phys_limit == UINT64_MAX) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 358)
[declaration] paging_panic("invalid physical memory limit");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 365)
[declaration] paging_panic("failed to create direct map");
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 368)
[declaration] paging_flush_tlb_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 373)
[declaration] paging_set_boot_phys_offset(phys_offset);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 374)
[declaration] paging_init_direct_map();
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 384)
[declaration] * tables are allocated (with kernel RW flags); otherwise the walk stops
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 399)
[function] if (!(entry & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 400)
[function] if (!create) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 403)
[declaration] ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 410)
[function] if (!(entry & PTE_PRESENT)) {

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 411)
[function] if (!create) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 414)
[declaration] ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 417)
[function] if (entry & PTE_PAGE_SIZE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 424)
[function] if (!(entry & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 425)
[function] if (!create) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 428)
[declaration] ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 431)
[function] if (entry & PTE_PAGE_SIZE) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 447)
[function] if (!boot_phys_offset_valid) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 450)
[function] if ((virt & PAGING_PAGE_MASK) != 0 || (phys & PAGING_PAGE_MASK) != 0) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 456)
[function] if (pte == NULL) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 461)
[declaration] paging_flush_page(virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 469)
[function] if (pte == NULL || !(*pte & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 473)
[declaration] paging_flush_page(virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 481)
[function] if (pte == NULL || !(*pte & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 484)
[declaration] return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 498)
[function] if (pte == NULL || !(*pte & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 503)
[declaration] paging_flush_page(virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 511)
[function] if (pte == NULL || !(*pte & PTE_PRESENT)) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 514)
[function] if (out_flags != NULL) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 522)
[declaration] return paging_is_mapped_in(paging_read_cr3(), virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 527)
[declaration] return paging_translate_in(paging_read_cr3(), virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 532)
[declaration] return paging_unmap_page_in(paging_read_cr3(), virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 537)
[declaration] return paging_set_flags_in(paging_read_cr3(), virt, flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 542)
[declaration] return paging_get_flags_in(paging_read_cr3(), virt, out_flags);

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 552)
[function] if (nx_enabled) {
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 555)
[declaration] __asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 557)
[declaration] __asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 568)
[declaration] __asm__ volatile("mov %0, %%cr3" : : "r"(pml4_phys) : "memory");
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 77)
[declaration] tlb_asm_invpcid(INVPCID_ALL, &desc);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 87)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 94)
[declaration] tlb_asm_write_cr4(cr4 & ~CR4_PGE);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 95)
[declaration] tlb_asm_write_cr4(cr4);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 97)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 109)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 123)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 130)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 135)
[declaration] tlb_asm_invlpg(a);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 143)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 154)
[declaration] tlb_asm_invpcid(INVPCID_SINGLE_PCID, &desc);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 159)
[declaration] tlb_asm_write_cr3(tlb_asm_read_cr3());
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 167)
[declaration] tlb_asm_invpcid(INVPCID_ADDR_IN_PCID, &desc);
kernel_Workspace/kernel/src/mm/cache/cache.c (line 3)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/cache/cache.c (line 10)
[function] if (cache_initialized) {
kernel_Workspace/kernel/src/mm/cache/cache.c (line 14)
[declaration] per_cpu_init();
kernel_Workspace/kernel/src/mm/cache/cache.c (line 29)
[function] if (c == NULL) {
kernel_Workspace/kernel/src/mm/cache/cache.c (line 41)
[function] while (c != NULL) {
kernel_Workspace/kernel/src/mm/cache/cache.c (line 45)
[function] while (*a != '\0' && *a == *b) {
kernel_Workspace/kernel/src/mm/cache/cache.c (line 50)
[function] if (*a == '\0' && *b == '\0') {

```

```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 62)
[function] if (!cache_initialized) {
kernel_Workspace/kernel/src/mm/cache/cache.c (line 63)
[declaration] kprintf("CACHE: not initialized\n");
kernel_Workspace/kernel/src/mm/cache/cache.c (line 67)
[declaration] kprintf("CACHE:\n");
kernel_Workspace/kernel/src/mm/cache/cache.c (line 71)
[function] while (c != NULL) {
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 82)
[declaration] buddy_free(va);
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 96)
[function] if (c->ctor != NULL) {
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 180)
[declaration] buddy_free((void *)(uintptr_t)ch->va);
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 181)
[declaration] heap_free(ch);
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 208)
[declaration] cache_unlock();
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 213)
[function] if (!kcache_grow(c)) {
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 214)
[declaration] cache_unlock();
kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 253)
[declaration] cache_unlock();
kernel_Workspace/kernel/src/mm/core/mm.c (line 15)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/core/mm.c (line 29)
[declaration] kprintf("mm_init: bad params\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 46)
[declaration] kprintf("mm_init: pmm_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 52)
[declaration] kprintf("mm_init: vmm_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 58)
[declaration] kprintf("mm_init: page_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 66)
[declaration] kprintf("mm_init: heap_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 72)
[declaration] kprintf("mm_init: cache_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 78)
[declaration] kprintf("mm_init: paging_subsystem_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 86)
[declaration] kprintf("mm_init: aspace_subsystem_init FAILED\n");
kernel_Workspace/kernel/src/mm/core/mm.c (line 104)
[declaration] return pmm_stat_total_bytes();
kernel_Workspace/kernel/src/mm/core/mm.c (line 109)
[declaration] return pmm_stat_free_bytes();
kernel_Workspace/kernel/src/mm/core/range.c (line 53)
[function] if (raw_end > UINT64_MAX - pad) {

```

```

kernel_Workspace/kernel/src/mm/core/range.c (line 65)
[function] if (!range_is_canonical(start) || !range_is_canonical(end - 1)) {
kernel_Workspace/kernel/src/mm/core/region.c (line 64)
[function] if (region_valid(before)) {
kernel_Workspace/kernel/src/mm/core/region.c (line 75)
[function] if (region_valid(after)) {
kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 5)
[declaration] __asm__ volatile("pause" ::: "memory");
kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 64)
[function] __atomic_load_n(&lock->owner_cpu, __ATOMIC_ACQUIRE) == me) {
kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 73)
[declaration] cpu_pause();
kernel_Workspace/kernel/src/mm/kprintf.c (line 18)
[declaration] __asm__ volatile("inb %%dx, %0" : "=a"(v) : "d"(port));
kernel_Workspace/kernel/src/mm/kprintf.c (line 25)
[declaration] __asm__ volatile("outb %0, %%dx" : : "a"(val), "d"(port));
kernel_Workspace/kernel/src/mm/kprintf.c (line 58)
[declaration] serial_putc(*s++);
kernel_Workspace/kernel/src/mm/kprintf.c (line 68)
[declaration] serial_putc('0');
kernel_Workspace/kernel/src/mm/kprintf.c (line 80)
[declaration] serial_putc(buf[i++]);
kernel_Workspace/kernel/src/mm/kprintf.c (line 93)
[function] if (*p != '%') {
kernel_Workspace/kernel/src/mm/kprintf.c (line 94)
[declaration] serial_putc(*p);
kernel_Workspace/kernel/src/mm/kprintf.c (line 99)
[function] if (*p == '\\0') {
kernel_Workspace/kernel/src/mm/kprintf.c (line 107)
[function] while (*p == 'l' || *p == 'z' || *p == 'h') {
kernel_Workspace/kernel/src/mm/kprintf.c (line 108)
[function] if (*p == 'l') {
kernel_Workspace/kernel/src/mm/kprintf.c (line 109)
[function] if (p[1] == 'l') {
kernel_Workspace/kernel/src/mm/kprintf.c (line 121)
[function] switch (*p) {
kernel_Workspace/kernel/src/mm/kprintf.c (line 125)
[declaration] serial_puts(s ? s : "(null)");
kernel_Workspace/kernel/src/mm/kprintf.c (line 129)
[declaration] serial_putc((char)va_arg(ap, int));
kernel_Workspace/kernel/src/mm/kprintf.c (line 137)
[function] if (v < 0) {
kernel_Workspace/kernel/src/mm/kprintf.c (line 138)
[declaration] serial_putc('-');
kernel_Workspace/kernel/src/mm/kprintf.c (line 139)
[declaration] serial_put_uint((uint64_t)(-v), 10, false);
kernel_Workspace/kernel/src/mm/kprintf.c (line 141)
[declaration] serial_put_uint((uint64_t)v, 10, false);

```



```

kernel_Workspace/kernel/src/mm/kprintf.c (line 151)
[declaration] serial_put_uint(v, 10, false);
kernel_Workspace/kernel/src/mm/kprintf.c (line 161)
[declaration] serial_put_uint(v, 16, *p == 'X');
kernel_Workspace/kernel/src/mm/kprintf.c (line 167)
[declaration] serial_puts("0x");
kernel_Workspace/kernel/src/mm/kprintf.c (line 168)
[declaration] serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
kernel_Workspace/kernel/src/mm/kprintf.c (line 172)
[declaration] serial_putc('%');
kernel_Workspace/kernel/src/mm/kprintf.c (line 175)
[declaration] serial_putc('%');
kernel_Workspace/kernel/src/mm/kprintf.c (line 176)
[declaration] serial_putc(*p);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 68)
[function] if (pml_entry_present(e) && !pml_entry_large(e)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 69)
[declaration] free_tables(pml_entry_addr(e), level - 1);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 99)
[declaration] pmm_free_frame(pml4_phys);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 120)
[function] if (pml_entry_present(e) && !pml_entry_large(e)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 121)
[declaration] free_tables(pml_entry_addr(e), PML_LEVEL_PDPT);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 173)
[function] if (!paging_map_page_in(pml4, v, p, pte)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 175)
[declaration] paging_unmap_page_in(pml4, virt + (uint64_t)j * ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 207)
[function] if (paging_unmap_page_in(pml4, v)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 208)
[declaration] tlb_batch_add(&batch, v);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 238)
[function] if (!paging_is_mapped_in(pml4, v)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 249)
[function] if (!paging_set_flags_in(pml4, v, pte)) {
kernel_Workspace/kernel/src/mm/paging/paging.c (line 250)
[declaration] tlb_batch_commit(&batch);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 254)
[declaration] tlb_batch_add(&batch, v);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 265)
[declaration] return paging_translate_in(paging_kernel_pml4, virt);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 268)
[declaration] return paging_translate_in(as->pml4_phys, virt);
kernel_Workspace/kernel/src/mm/paging/paging.c (line 277)
[function] return paging_aspace_map((address_space_t *)&(address_space_t){
kernel_Workspace/kernel/src/mm/paging/paging.c (line 290)
[function] return paging_aspace_unmap((address_space_t *)&(address_space_t){

```

```

kernel_Workspace/kernel/src/mm/paging/tlb.c (line 33)
[declaration] tlb_flush_all();
kernel_Workspace/kernel/src/mm/paging/tlb.c (line 40)
[declaration] tlb_flush_page_addr(batch->pages[i]);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 63)
[function] if (!pmm_alloc_zero_frame(&phys)) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 69)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 82)
[declaration] paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 84)
[function] if (phys != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 85)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 97)
[declaration] paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 99)
[function] if (phys != UINT64_MAX) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 100)
[declaration] pmm_free_frame(phys);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 122)
[function] if (*p == v) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 136)
[function] if (addr >= v->start && addr < v->end) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 163)
[function] if (v->end <= cand) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 168)
[function] if (cand <= USER_MMAP_MAX - len && cand + len <= v->start) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 174)
[function] if (cand > USER_MMAP_MAX - len) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 199)
[declaration] heap_free(pa);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 212)
[declaration] paging_aspace_destroy(pa->as);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 213)
[declaration] heap_free(pa);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 226)
[declaration] heap_free(sv);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 227)
[declaration] paging_aspace_destroy(pa->as);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 228)
[declaration] heap_free(pa);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 246)
[declaration] unmap_pages_free(pa, v->start, v->end);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 249)
[declaration] heap_free(v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 281)
[declaration] as_unlock();

```

```
kernel_Workspace/kernel/src/mm/process/address_space.c (line 288)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 300)
[declaration] vma_remove(pa, v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 301)
[declaration] heap_free(v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 302)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 330)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 342)
[declaration] vma_remove(pa, v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 343)
[declaration] heap_free(v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 344)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 371)
[function] while (v != NULL) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 372)
[function] if (v->start < b && v->end > a) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 380)
[function] if (hit == NULL) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 390)
[function] if (needs_split) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 393)
[function] if (tail == NULL) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 394)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 399)
[declaration] unmap_pages_free(pa, s, e);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 401)
[function] if (needs_split) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 415)
[declaration] vma_remove(pa, hit);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 416)
[declaration] heap_free(hit);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 445)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 450)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 455)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 468)
[declaration] return aspace_protect_checked(pa, addr, len, prot, prot, NULL);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 496)
[function] if (v->start < b && v->end > a) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 497)
[declaration] as_unlock();
```

```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 507)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 533)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 538)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 545)
[function] if (!map_anon_pages(pa, pa->brk, nb, PROT_READ | PROT_WRITE)) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 546)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 552)
[function] if (v == NULL) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 555)
[function] if (v == NULL) {
kernel_Workspace/kernel/src/mm/process/address_space.c (line 556)
[declaration] unmap_pages_free(pa, pa->brk, nb);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 557)
[declaration] as_unlock();
kernel_Workspace/kernel/src/mm/process/address_space.c (line 566)
[declaration] vma_insert(pa, v);
kernel_Workspace/kernel/src/mm/process/address_space.c (line 573)
[declaration] aspace_unmap(pa, nb, pa->brk - nb);
kernel_Workspace/kernel/src/mm/process/mmap.c (line 17)
[declaration] return aspace_map_at(pa, addr, len, prot);
kernel_Workspace/kernel/src/mm/process/mmap.c (line 20)
[declaration] return aspace_map_anon(pa, addr, len, prot);
kernel_Workspace/kernel/src/mm/process/mmap.c (line 25)
[declaration] return aspace_unmap(pa, addr, len);
kernel_Workspace/kernel/src/mm/protection/guard.c (line 6)
[declaration] return aspace_reserve_at(pa, addr, len, VMA_FLAG_GUARD);
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 5)
[declaration] extern void kprintf(const char *fmt, ...);
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 24)
[declaration] __asm__ volatile("mov %%cr4, %0" : "=r"(v));
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 31)
[declaration] __asm__ volatile("mov %0, %%cr4" :: "r"(v) : "memory");
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 81)
[function] if (!pml_entry_present(e)) {
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 85)
[function] if (e & PTE_USER) {
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 89)
[function] if (!pml_entry_large(e) && level > PML_LEVEL_PT) {
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 91)
[declaration] audit_tables(pml_entry_addr(e), level - 1, violations);
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 108)
[function] if (!pml_entry_present(e)) {
kernel_Workspace/kernel/src/mm/protection/isolation.c (line 112)
[function] if (e & PTE_USER) {

```

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 116)

[function] if (!pml_entry_large(e)) {

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 118)

[declaration] audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);

kstd/src/kernel/audio.c (line 3)

[declaration] extern int32_t k_audio_play(uint64_t phys, uint32_t len);

kstd/src/kernel/audio.c (line 4)

[declaration] extern int32_t k_audio_stop(void);

kstd/src/kernel/audio.c (line 5)

[declaration] extern int32_t k_audio_jack(void);

kstd/src/kernel/audio.c (line 6)

[declaration] extern int32_t k_audio_amp(int32_t on);

kstd/src/kernel/audio.c (line 7)

[declaration] extern uint64_t kvirt_to_phys(void *ptr);

kstd/src/kernel/audio.c (line 13)

[function] if (phys == (uint64_t)-1) {

kstd/src/kernel/audio.c (line 22)

[declaration] k_audio_stop();

kstd/src/kernel/audio.c (line 28)

[function] if (!present) {

kstd/src/kernel/audio.c (line 38)

[declaration] k_audio_amp(on ? 1 : 0);

kstd/src/kernel/bt.c (line 3)

[declaration] extern bool bt_ready(void);

kstd/src/kernel/bt.c (line 4)

[declaration] extern void bt_info(uint8_t *ver, uint8_t *addr);

kstd/src/kernel/bt.c (line 5)

[declaration] extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);

kstd/src/kernel/bt.c (line 6)

[declaration] extern bool bt_event_poll(uint8_t *buf, uint8_t *len);

kstd/src/kernel/bt.c (line 7)

[declaration] extern bool bt_acl_send(const uint8_t *d, uint16_t len);

kstd/src/kernel/bt.c (line 8)

[declaration] extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

kstd/src/kernel/bt.c (line 12)

[function] if (!bt_ready()) {

kstd/src/kernel/bt.c (line 16)

[declaration] bt_info(hci_ver, addr6);

kstd/src/kernel/fs.c (line 3)

[declaration] extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

kstd/src/kernel/fs.c (line 4)

[declaration] extern int32_t k_fs_exists(const char *path);

kstd/src/kernel/fs.c (line 10)

[function] if (n < 0) {

kstd/src/kernel/fs.c (line 14)

[function] if (out) {

kstd/src/kernel/input.c (line 3)

[declaration] extern int32_t k_input_key(void);

```

kstd/src/kernel/input.c (line 7)
[declaration] return k_input_key();
kstd/src/kernel/log.c (line 3)
[declaration] extern void kprintf(const char *fmt, ...);
kstd/src/kernel/log.c (line 7)
[declaration] kprintf("%s\n", s);
kstd/src/kernel/mem.c (line 4)
[declaration] extern void kfree(void *ptr);
kstd/src/kernel/mem.c (line 5)
[declaration] extern bool vmm_map_device(uint64_t phys, unsigned long len, uint64_t
*virt);
kstd/src/kernel/mem.c (line 6)
[declaration] extern uint64_t kvirt_to_phys(void *ptr);
kstd/src/kernel/mem.c (line 10)
[declaration] return kmalloc(bytes);
kstd/src/kernel/mem.c (line 16)
[declaration] kfree(ptr);
kstd/src/kernel/mem.c (line 23)
[function] if (vmm_map_device(phys, len, &v)) {
kstd/src/kernel/video.c (line 8)
[function] tr_status_t tr_video_init(void) {
kstd/src/kernel/video.c (line 9)
[function] if (dp_ready()) {
kstd/src/kernel/video.c (line 13)
[function] if (hdmi_ready()) {
kstd/src/kernel/video.c (line 20)
[function] bool tr_video_is_ready(void) {
kstd/src/kernel/video.c (line 24)
[function] tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
kstd/src/kernel/video.c (line 27)
[function] if (use_dp) {
kstd/src/kernel/video.c (line 28)
[declaration] dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
kstd/src/kernel/video.c (line 31)
[declaration] hdmi_caps_raw(&caps->width, &caps->height, &caps->stride,
&caps->fb_phys);
kstd/src/kernel/video.c (line 37)
[function] tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y,
uint32_t w, uint32_t h) {
kstd/src/kernel/video.c (line 41)
[function] if (use_dp) {
kstd/src/user/audio.c (line 7)
[declaration] extern void ds_poll(void);
kstd/src/user/audio.c (line 8)
[declaration] extern int ds_take(uint64_t id, ds_msg_t *out);
kstd/src/user/audio.c (line 15)
[function] if (bounce == TR_NULL) {
kstd/src/user/audio.c (line 18)
[function] if (bounce == TR_NULL) {

```

```

kstd/src/user/audio.c (line 23)
[declaration] ds_poll();
kstd/src/user/audio.c (line 27)
[function] if (!ds_take(id, &r) || r.status != 0) {
kstd/src/user/audio.c (line 34)
[function] if (len > 4096 * 4) {
kstd/src/user/audio.c (line 46)
[declaration] ds_poll();
kstd/src/user/audio.c (line 50)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/audio.c (line 59)
[declaration] ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
kstd/src/user/audio.c (line 65)
[function] if (!present) {
kstd/src/user/audio.c (line 70)
[declaration] ds_poll();
kstd/src/user/audio.c (line 74)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/audio.c (line 84)
[declaration] ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
kstd/src/user/bt.c (line 6)
[declaration] extern void ds_poll(void);
kstd/src/user/bt.c (line 7)
[declaration] extern int ds_take(uint64_t id, ds_msg_t *out);
kstd/src/user/bt.c (line 14)
[declaration] ds_poll();
kstd/src/user/bt.c (line 18)
[function] if (!ds_take(id, &r) || r.status != 0) {
kstd/src/user/bt.c (line 40)
[declaration] ds_poll();
kstd/src/user/bt.c (line 44)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/bt.c (line 54)
[declaration] ds_poll();
kstd/src/user/bt.c (line 58)
[function] if (!ds_take(id, &r) || r.status != 0) {
kstd/src/user/bt.c (line 83)
[declaration] ds_poll();
kstd/src/user/bt.c (line 87)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/bt.c (line 97)
[declaration] ds_poll();
kstd/src/user/bt.c (line 101)
[function] if (!ds_take(id, &r) || r.status != 0) {
kstd/src/user/fs.c (line 6)
[declaration] extern void ds_poll(void);
kstd/src/user/fs.c (line 7)
[declaration] extern int ds_take(uint64_t id, ds_msg_t *out);

```



```

kstd/src/user/fs.c (line 15)
[function] while (path[n] && n < 255) {
kstd/src/user/fs.c (line 23)
[declaration] ds_poll();
kstd/src/user/fs.c (line 27)
[function] if (!ds_take(id, &r) || r.status != 0) {
kstd/src/user/fs.c (line 33)
[function] if (got > cap) {
kstd/src/user/fs.c (line 43)
[function] if (out) {
kstd/src/user/fs.c (line 54)
[function] while (path[n] && n < 255) {
kstd/src/user/fs.c (line 62)
[declaration] ds_poll();
kstd/src/user/fs.c (line 66)
[function] if (ds_take(id, &r) && r.status == 0 && r.arg0 == 1) {
kstd/src/user/input.c (line 6)
[declaration] extern void ds_poll(void);
kstd/src/user/input.c (line 7)
[declaration] extern int ds_take(uint64_t id, ds_msg_t *out);
kstd/src/user/input.c (line 12)
[declaration] ds_poll();
kstd/src/user/input.c (line 16)
[function] if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
kstd/src/user/log.c (line 10)
[function] while (s[n] && n < DS_SCRATCH_SIZE - 1) {
kstd/src/user/log.c (line 15)
[declaration] ds_call(SVC_SYS, OP_LOG, n, 0, 0);
kstd/src/user/mem.c (line 6)
[declaration] extern void ds_poll(void);
kstd/src/user/mem.c (line 7)
[declaration] extern int ds_take(uint64_t id, ds_msg_t *out);
kstd/src/user/mem.c (line 16)
[declaration] ds_poll();
kstd/src/user/mem.c (line 20)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/mem.c (line 30)
[declaration] ds_call(SVC_SYS, OP_FREE, (uint64_t)ptr, 0, 0);
kstd/src/user/mem.c (line 39)
[declaration] ds_poll();
kstd/src/user/mem.c (line 43)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/video.c (line 6)
[function] tr_status_t tr_video_init(void) {
kstd/src/user/video.c (line 11)
[function] bool tr_video_is_ready(void) {
kstd/src/user/video.c (line 13)
[declaration] ds_poll();

```

```

kstd/src/user/video.c (line 15)
[function] if (ds_take(id, &r) && r.status == 0) {
kstd/src/user/video.c (line 21)
[function] tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y,
uint32_t w, uint32_t h) {
kstd/src/user/video.c (line 26)
[declaration] ds_poll();
kstd/src/user/video.c (line 29)
[function] if (ds_take(id, &r)) {
libs/dsclient.c (line 30)
[function] if (next == d2k->tail) {
libs/dsclient.c (line 51)
[function] while (k2d->tail != k2d->head) {
libs/dsclient.c (line 58)
[function] if (!cache_used[i]) {
libs/dsclient.c (line 70)
[function] if (cache_used[i] && cache[i].id == id) {
triang-lang/out/x86_64/prog.c (line 15)
[declaration] memset(buffer, (int)0ull, 16);

```

C (header)

```

drivers/vgpu/vgpu.h (line 22)
[declaration] bool vgpu_init(uint32_t w, uint32_t h);
drivers/vgpu/vgpu.h (line 23)
[declaration] vgpu_info_t vgpu_get_info(void);
drivers/vgpu/vgpu.h (line 25)
[declaration] void vgpu_clear(uint32_t color);
drivers/vgpu/vgpu.h (line 26)
[declaration] void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);
drivers/vgpu/vgpu.h (line 28)
[declaration] int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t
*phys_out);
drivers/vgpu/vgpu.h (line 29)
[declaration] bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);
drivers/vgpu/vgpu.h (line 31)
[declaration] void vgpu_process_ring(volatile void *ring);
kernel_Workspace/kernel/src/battery/battery.h (line 21)
[declaration] bool battery_init(void);
kernel_Workspace/kernel/src/battery/battery.h (line 22)
[declaration] bool battery_present(void);
kernel_Workspace/kernel/src/battery/battery.h (line 23)
[declaration] bool battery_status(battery_status_t *out);
kernel_Workspace/kernel/src/battery/battery.h (line 24)
[declaration] bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
kernel_Workspace/kernel/src/battery/battery.h (line 25)
[declaration] bool battery_set_threshold(uint32_t low_pct);
kernel_Workspace/kernel/src/battery/battery.h (line 26)
[declaration] uint32_t battery_threshold(void);

```

kernel_Workspace/kernel/src/battery/battery.h (line 27)
[declaration] bool battery_low_pending(void);

kernel_Workspace/kernel/src/battery/battery.h (line 28)
[declaration] void battery_sim_tick(void);

kernel_Workspace/kernel/src/battery/operation.h (line 7)
[declaration] bool (*present)(void);

kernel_Workspace/kernel/src/battery/operation.h (line 8)
[declaration] bool (*status)(battery_status_t *out);

kernel_Workspace/kernel/src/battery/operation.h (line 11)
[declaration] void battery_register_backend(const battery_backend_t *b);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 8)
[declaration] bool bt_init(void);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 9)
[declaration] bool bt_ready(void);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 10)
[declaration] void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 11)
[declaration] bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 12)
[declaration] bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 13)
[declaration] bool bt_acl_send(const uint8_t *data, uint16_t len);

kernel_Workspace/kernel/src/bluetooth/bt.h (line 14)
[declaration] bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

kernel_Workspace/kernel/src/bluetooth/operation.h (line 11)
[declaration] void bt_op_model_init(void);

kernel_Workspace/kernel/src/bluetooth/operation.h (line 12)
[declaration] bool bt_op_model_run_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

kernel_Workspace/kernel/src/camera/camera.h (line 19)
[declaration] bool camera_init(void);

kernel_Workspace/kernel/src/camera/camera.h (line 20)
[declaration] bool camera_present(void);

kernel_Workspace/kernel/src/camera/camera.h (line 21)
[declaration] bool camera_caps_get(camera_caps_t *out);

kernel_Workspace/kernel/src/camera/camera.h (line 22)
[declaration] bool camera_start(void);

kernel_Workspace/kernel/src/camera/camera.h (line 23)
[declaration] bool camera_stop(void);

kernel_Workspace/kernel/src/camera/camera.h (line 24)
[declaration] bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);

kernel_Workspace/kernel/src/camera/camera.h (line 25)
[declaration] bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

kernel_Workspace/kernel/src/camera/operation.h (line 7)
[declaration] bool (*present)(void);

kernel_Workspace/kernel/src/camera/operation.h (line 8)
[declaration] bool (*start)(void);

kernel_Workspace/kernel/src/camera/operation.h (line 9)
[declaration] bool (*stop)(void);

kernel_Workspace/kernel/src/camera/operation.h (line 10)
[declaration] bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);

kernel_Workspace/kernel/src/camera/operation.h (line 13)
[declaration] void camera_register_backend(const camera_backend_t *b);

kernel_Workspace/kernel/src/caps/caps.h (line 10)
[declaration] unsigned int caps_self_bits(void);

kernel_Workspace/kernel/src/caps/caps.h (line 12)
[declaration] unsigned int caps_world_bits(unsigned int world_id);

kernel_Workspace/kernel/src/caps/caps.h (line 14)
[declaration] int caps_has(unsigned int world_id, unsigned char cap_id);

kernel_Workspace/kernel/src/caps/caps.h (line 16)
[declaration] int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

kernel_Workspace/kernel/src/caps/caps.h (line 18)
[declaration] int caps_request(unsigned int target, unsigned char cap_id);

kernel_Workspace/kernel/src/caps/caps.h (line 20)
[declaration] int caps_release(unsigned int world_id, unsigned char cap_id);

kernel_Workspace/kernel/src/caps/caps.h (line 22)
[declaration] unsigned int caps_world_count(void);

kernel_Workspace/kernel/src/caps/caps.h (line 24)
[declaration] unsigned long long caps_audit_count(void);

kernel_Workspace/kernel/src/core-lang/ast.h (line 50)
[declaration] void cl_arena_init(arena_t *a, uint8_t *buf, size_t cap);

kernel_Workspace/kernel/src/core-lang/bridge.h (line 6)
[declaration] int cl_bridge_init(cl_vm_t *vm, uint8_t ring);

kernel_Workspace/kernel/src/core-lang/bridge.h (line 8)
[declaration] int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn);

kernel_Workspace/kernel/src/core-lang/lexer.h (line 16)
[declaration] void cl_lexer_init(lexer_t *l, const char *src, size_t len);

kernel_Workspace/kernel/src/core-lang/lexer.h (line 18)
[declaration] bool cl_lex_next(lexer_t *l, token_t *out);

kernel_Workspace/kernel/src/core-lang/lexer.h (line 20)
[declaration] int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

kernel_Workspace/kernel/src/core-lang/loader.h (line 15)
[declaration] size_t cl_save_bc(cl_prog_t *P, uint8_t *buf, size_t cap);

kernel_Workspace/kernel/src/core-lang/native.h (line 9)
[declaration] typedef uint64_t (*cl_native_fn)(uint64_t *, uint64_t *, cl_ext_fn *);

kernel_Workspace/kernel/src/core-lang/native.h (line 14)
[declaration] size_t cl_native_size(void);

kernel_Workspace/kernel/src/core-lang/sema.h (line 16)
[declaration] int cl_sema_run(sema_ctx_t *s, ast_node_t *prog);

kernel_Workspace/kernel/src/core-lang/sema.h (line 18)
[declaration] size_t cl_sema_diag_count(const sema_ctx_t *s);

kernel_Workspace/kernel/src/core-lang/vm.h (line 44)
[declaration] void cl_vm_init(cl_vm_t *vm, cl_prog_t *prog);

kernel_Workspace/kernel/src/core-lang/vm.h (line 45)
[declaration] int cl_vm_register_extern(cl_vm_t *vm, const char *name, cl_ext_fn fn);

kernel_Workspace/kernel/src/core-lang/vm.h (line 46)
[declaration] cl_vm_err_t cl_vm_run(cl_vm_t *vm);

kernel_Workspace/kernel/src/core-lang/vm.h (line 48)
[declaration] int cl_wbytes(int bits);

kernel_Workspace/kernel/src/core-lang/vm.h (line 49)
[declaration] void cl_mask(uint8_t *r, int w, int bits);

kernel_Workspace/kernel/src/core-lang/vm.h (line 50)
[declaration] void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);

kernel_Workspace/kernel/src/core-lang/vm.h (line 51)
[declaration] void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);

kernel_Workspace/kernel/src/core-lang/vm.h (line 52)
[declaration] void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);

kernel_Workspace/kernel/src/core-lang/vm.h (line 53)
[declaration] int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);

kernel_Workspace/kernel/src/core-lang/vm.h (line 54)
[declaration] void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);

kernel_Workspace/kernel/src/core-lang/vm.h (line 55)
[declaration] void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);

kernel_Workspace/kernel/src/displayport/aux.h (line 17)
[declaration] void dp_aux_sink_init(void);

kernel_Workspace/kernel/src/displayport/aux.h (line 18)
[declaration] bool dp_aux_read(uint32_t addr, void *buf, uint32_t len);

kernel_Workspace/kernel/src/displayport/aux.h (line 19)
[declaration] bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);

kernel_Workspace/kernel/src/displayport/aux.h (line 20)
[declaration] bool dp_aux_read_edid(void *buf, uint32_t len);

kernel_Workspace/kernel/src/displayport/dp.h (line 10)
[declaration] bool dp_init(void);

kernel_Workspace/kernel/src/displayport/dp.h (line 11)
[declaration] bool dp_ready(void);

kernel_Workspace/kernel/src/displayport/dp.h (line 13)
[declaration] void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

kernel_Workspace/kernel/src/displayport/dp.h (line 15)
[declaration] uint32_t dp_mode_count(void);

kernel_Workspace/kernel/src/displayport/dp.h (line 18)
[declaration] bool dp_mode_set_by_id(uint32_t id);

kernel_Workspace/kernel/src/displayport/dp.h (line 20)
[declaration] void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);

kernel_Workspace/kernel/src/displayport/dp.h (line 21)
[declaration] bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/displayport/link.h (line 7)
[declaration] bool dp_link_train(void);

kernel_Workspace/kernel/src/displayport/link.h (line 8)
[declaration] void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

kernel_Workspace/kernel/src/displayport/operation.h (line 6)

[declaration] void dp_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/displayport/operation.h (line 7)

[declaration] void dp_op_state(uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *stride);

kernel_Workspace/kernel/src/displayport/operation.h (line 8)

[declaration] void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

kernel_Workspace/kernel/src/editor/editor.h (line 21)

[declaration] int editor_run(const char *path);

kernel_Workspace/kernel/src/editor/mouse.h (line 6)

[declaration] void mouse_init(void);

kernel_Workspace/kernel/src/editor/mouse.h (line 8)

[declaration] int mouse_poll(int *dx, int *dy, int *dz, int *buttons);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 41)

[declaration] bool hdmi_init_with(uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 42)

[declaration] bool hdmi_ready(void);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 43)

[declaration] void hdmi_caps(hdmi_caps_t *out);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 44)

[declaration] bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 45)

[declaration] bool hdmi_mode_set_by_id(uint32_t id);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 46)

[declaration] bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 47)

[declaration] uint64_t hdmi_submit(const hdmi_transfer_t *t);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 48)

[declaration] bool hdmi_poll(uint64_t *out_seq);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 49)

[declaration] uint32_t hdmi_pending(void);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 50)

[declaration] uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 51)

[declaration] void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 53)

[declaration] bool hdmi_fb_grant(void);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 54)

[declaration] void hdmi_fb_revoke(void);

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 55)

[declaration] bool hdmi_fb_granted(void);

kernel_Workspace/kernel/src/hdmi/interface.h (line 9)

[declaration] bool hdmi_iface_acquire(uint32_t owner);

kernel_Workspace/kernel/src/hdmi/interface.h (line 10)
[declaration] bool hdmi_iface_release(uint32_t owner);

kernel_Workspace/kernel/src/hdmi/interface.h (line 11)
[declaration] uint32_t hdmi_iface_owner(void);

kernel_Workspace/kernel/src/hdmi/interface.h (line 12)
[declaration] bool hdmi_iface_check(uint32_t owner);

kernel_Workspace/kernel/src/hdmi/interface.h (line 14)
[declaration] bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/hdmi/interface.h (line 15)
[declaration] bool hdmi_iface_ready(void);

kernel_Workspace/kernel/src/hdmi/interface.h (line 17)
[declaration] bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);

kernel_Workspace/kernel/src/hdmi/interface.h (line 18)
[declaration] bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

kernel_Workspace/kernel/src/hdmi/interface.h (line 19)
[declaration] bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

kernel_Workspace/kernel/src/hdmi/interface.h (line 21)
[declaration] uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

kernel_Workspace/kernel/src/hdmi/interface.h (line 22)
[declaration] bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

kernel_Workspace/kernel/src/hdmi/interface.h (line 24)
[declaration] void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

kernel_Workspace/kernel/src/hdmi/interface.h (line 26)
[declaration] bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);

kernel_Workspace/kernel/src/hdmi/interface.h (line 27)
[declaration] bool hdmi_iface_fb_revoke(uint32_t owner);

kernel_Workspace/kernel/src/hdmi/mode.h (line 23)
[declaration] uint32_t hdmi_mode_count(void);

kernel_Workspace/kernel/src/hdmi/mode.h (line 27)
[declaration] bool hdmi_mode_valid(const hdmi_mode_t *m);

kernel_Workspace/kernel/src/hdmi/mode.h (line 28)
[declaration] bool hdmi_mode_apply(const hdmi_mode_t *m);

kernel_Workspace/kernel/src/hdmi/operation.h (line 7)
[declaration] void hdmi_op_set_mode(const hdmi_mode_t *m);

kernel_Workspace/kernel/src/hdmi/operation.h (line 8)
[declaration] void hdmi_op_exec(hdmi_transfer_t *t);

kernel_Workspace/kernel/src/hdmi/operation.h (line 9)
[declaration] void hdmi_op_state(hdmi_caps_t *out);

kernel_Workspace/kernel/src/hdmi/operation.h (line 10)
[declaration] void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

kernel_Workspace/kernel/src/hdmi/operation.h (line 11)
[declaration] bool hdmi_op_ready(void);

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 13)
[declaration] void kfree(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 15)
[declaration] size_t kmalloc_usable_size(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 18)
[declaration] void kfree_pages(void *ptr, size_t pages);

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 22)
[declaration] uint64_t kvirt_to_phys(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 24)
[declaration] void kalloc_dump(void);

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 9)
[declaration] void dbg_free(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 11)
[declaration] bool dbg_verify(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 12)
[declaration] size_t dbg_usable_size(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 13)
[declaration] void mm_debug_dump(void);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 8)
[declaration] bool leak_track(void *ptr, size_t size, uint64_t caller);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 10)
[declaration] bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 12)
[declaration] bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 13)
[declaration] bool leak_contains(void *ptr);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 14)
[declaration] size_t leak_count(void);

kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 16)
[declaration] void leak_dump(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 8)
[declaration] void alloc_stats_note_alloc(size_t size);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 9)
[declaration] void alloc_stats_note_free(size_t size);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 11)
[declaration] size_t alloc_stats_live_count(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 12)
[declaration] size_t alloc_stats_live_bytes(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 13)
[declaration] size_t alloc_stats_peak_bytes(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 14)
[declaration] size_t alloc_stats_total_allocs(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 15)
[declaration] size_t alloc_stats_total_frees(void);

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 17)
[declaration] void alloc_stats_dump(void);

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 8)
[declaration] typedef bool (*buddy_map_cb)(uint64_t virt, size_t size);

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 9)
[declaration] typedef void (*buddy_unmap_cb)(uint64_t virt, size_t size);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 16)
[declaration] bool buddy_ready(void);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 20)
[declaration] void buddy_free(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 22)
[declaration] size_t buddy_block_size(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 24)
[declaration] size_t buddy_stat_used_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 25)
[declaration] size_t buddy_stat_free_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 27)
[declaration] void buddy_dump(void);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 8)
[declaration] bool heap_init(void);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 9)
[declaration] bool heap_ready(void);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 15)
[declaration] void heap_free(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 17)
[declaration] size_t heap_usable_size(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 19)
[declaration] void heap_dump(void);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 10)
[declaration] bool slab_init(uint64_t base, size_t size);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 11)
[declaration] bool slab_ready(void);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 14)
[declaration] void slab_free(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 16)
[declaration] size_t slab_usable_size(void *ptr);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 18)
[declaration] size_t slab_stat_used_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 19)
[declaration] size_t slab_stat_free_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 21)
[declaration] void slab_dump(void);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 30)
[declaration] size_t bitmap_words_for_bits(size_t bit_count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 31)
[declaration] size_t bitmap_bytes_for_bits(size_t bit_count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 33)
[declaration] void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 35)
[declaration] bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 37)
[declaration] void bitmap_fill(bitmap_t *bm, bool value);

```
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 39)
[declaration] void bitmap_set(bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 40)
[declaration] void bitmap_clear(bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 41)
[declaration] bool bitmap_test(const bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 43)
[declaration] void bitmap_fill(bitmap_t *bm, bool value);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 45)
[declaration] void bitmap_set(bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 46)
[declaration] void bitmap_clear(bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 47)
[declaration] bool bitmap_test(const bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 49)
[declaration] void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 50)
[declaration] void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 52)
[declaration] bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t
count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 54)
[declaration] size_t bitmap_alloc(bitmap_t *bm);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 57)
[declaration] size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 59)
[declaration] size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 61)
[declaration] void bitmap_free(bitmap_t *bm, size_t bit);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 62)
[declaration] void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);
kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 64)
[declaration] size_t bitmap_count_free(const bitmap_t *bm);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 12)
[declaration] bool frame_init(uint64_t bitmap_phys, size_t bit_count);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 14)
[declaration] bool frame_init_from_memory(void);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 16)
[declaration] bool frame_ready(void);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 18)
[declaration] bool frame_alloc(frame_t *out);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 19)
[declaration] bool frame_alloc_zero(frame_t *out);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 29)
[declaration] bool frame_free(frame_t frame);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 30)
[declaration] bool frame_free_contiguous(frame_t start, size_t count);
kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 32)
[declaration] bool frame_is_valid(frame_t frame);
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 33)
[declaration] bool frame_zero(frame_t frame);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 36)
[declaration] uint64_t frame_phys(frame_t frame);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 38)
[declaration] size_t frame_to_pfn(frame_t frame);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 39)
[declaration] frame_t frame_from_pfn(size_t pfn);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 41)
[declaration] size_t frame_total(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 42)
[declaration] size_t frame_allocated(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 43)
[declaration] size_t frame_free_count(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 46)
[declaration] size_t frame_zone_dma32_total(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 47)
[declaration] size_t frame_zone_dma32_free(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 48)
[declaration] size_t frame_zone_normal_total(void);

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 49)
[declaration] size_t frame_zone_normal_free(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 10)
[declaration] bool pmm_init(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 12)
[declaration] bool pmm_ready(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 14)
[declaration] bool pmm_alloc_frame(uint64_t *out_phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 16)
[declaration] bool pmm_alloc_zero_frame(uint64_t *out_phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 18)
[declaration] bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 23)
[declaration] bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 24)
[declaration] bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 28)
[declaration] bool pmm_free_frame(uint64_t phys);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 33)
[declaration] bool pmm_free_frames(uint64_t phys, size_t count);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 34)
[declaration] bool pmm_free_bytes(uint64_t phys, size_t bytes);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 35)
[declaration] size_t pmm_stat_total_frames(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 36)
[declaration] size_t pmm_stat_free_frames(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 37)
[declaration] size_t pmm_stat_allocated_frames(void);

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 39)
[declaration] uint64_t pmm_stat_total_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 40)
[declaration] uint64_t pmm_stat_free_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 43)
[declaration] size_t pmm_stat_dma32_total_frames(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 44)
[declaration] size_t pmm_stat_dma32_free_frames(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 45)
[declaration] size_t pmm_stat_normal_total_frames(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 46)
[declaration] size_t pmm_stat_normal_free_frames(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 48)
[declaration] void pmm_dump(void);
kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 51)
[declaration] bool pmm_self_test(void);
kernel_Workspace/kernel/src/mm/alloc/special/contiguous.h (line 13)
[declaration] void contig_free(uint64_t phys, size_t bytes);
kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 16)
[declaration] void dma_free_coherent(uint64_t phys, void *virt, size_t bytes);
kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 18)
[declaration] void dma_sync_for_device(void *virt, size_t len);
kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 19)
[declaration] void dma_sync_for_cpu(void *virt, size_t len);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 12)
[declaration] void mapping_init(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 13)
[declaration] mapping_space_t mapping_kernel_space(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 30)
[declaration] uint64_t mapping_translate(mapping_space_t space, uint64_t virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 31)
[declaration] bool mapping_is_mapped(mapping_space_t space, uint64_t virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 22)
[declaration] bool page_init(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 23)
[declaration] bool page_ready(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 28)
[declaration] bool page_attach(uint64_t phys, uint32_t type);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 30)
[declaration] bool page_get(page_t *page);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 31)
[declaration] bool page_put(page_t *page);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 33)
[declaration] uint64_t page_phys(const page_t *page);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 35)
[declaration] size_t page_pfn(const page_t *page);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 41)
[declaration] uint32_t page_refcount(const page_t *page);

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 42)
[declaration] uint32_t page_type(const page_t *page);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 43)
[declaration] void page_set_type(page_t *page, uint32_t type);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 45)
[declaration] size_t page_stat_total(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 46)
[declaration] size_t page_stat_referenced(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 48)
[declaration] void page_dump(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 18)
[declaration] bool vmm_init(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 19)
[declaration] bool vmm_ready(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 21)
[declaration] bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 22)
[declaration] bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 23)
[declaration] bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 25)
[declaration] bool vmm_free(uint64_t virt, size_t bytes);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 26)
[declaration] bool vmm_unmap_device(uint64_t virt, size_t len);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 28)
[declaration] uint64_t vmm_translate(uint64_t virt);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 30)
[declaration] size_t vmm_stat_total_pages(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 31)
[declaration] size_t vmm_stat_free_pages(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 32)
[declaration] size_t vmm_stat_allocated_pages(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 33)
[declaration] uint64_t vmm_stat_total_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 34)
[declaration] uint64_t vmm_stat_free_bytes(void);
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 36)
[declaration] void vmm_dump(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 25)
[declaration] bool arch_memory_ready(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 27)
[declaration] bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 28)
[declaration] void arch_memory_reserve_range(uint64_t base, uint64_t len);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 30)
[declaration] bool arch_is_page_aligned(uint64_t a);

kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 31)
[declaration] uint64_t arch_page_align_up(uint64_t a);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 32)
[declaration] uint64_t arch_page_align_down(uint64_t a);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 35)
[declaration] uint64_t arch_virt_to_phys(void *virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 37)
[declaration] uint64_t arch_memory_max_phys(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 38)
[declaration] void arch_memory_dump(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 17)
[declaration] void paging_init(uint64_t boot_phys_offset);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 19)
[declaration] bool paging_enable_nx(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 21)
[declaration] uint64_t paging_read_cr3(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 22)
[declaration] void paging_write_cr3(uint64_t pml4_phys);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 24)
[declaration] bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 25)
[declaration] bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 26)
[declaration] bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 27)
[declaration] bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 28)
[declaration] uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 8)
[declaration] bool tlb_init(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 9)
[declaration] bool tlb_ready(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 11)
[declaration] void tlb_flush_all(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 12)
[declaration] void tlb_flush_all_including_global(void);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 14)
[declaration] void tlb_flush_page(const void *addr);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 15)
[declaration] void tlb_flush_page_addr(uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 17)
[declaration] void tlb_flush_range(const void *addr, size_t pages);
kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 18)
[declaration] void tlb_flush_range_addr(uint64_t addr, size_t pages);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 71)
[declaration] bool arch_memory_ready(void);


```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 73)
[declaration] bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t
*out_base);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 76)
[declaration] size_t arch_memory_regions(const arch_mem_region_t **out);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 77)
[declaration] uint64_t arch_memory_total_usable(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 78)
[declaration] bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 79)
[declaration] void arch_memory_reserve_range(uint64_t base, uint64_t len);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 83)
[declaration] void arch_memory_dump(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 92)
[function] if (v > UINT64_MAX - ARCH_PAGE_MASK) {
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 107)
[declaration] return (void *) (uintptr_t) (phys + ARCH_DIRECT_MAP_BASE);
kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 112)
[declaration] return (uint64_t) ((uintptr_t) virt - ARCH_DIRECT_MAP_BASE);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 35)
[declaration] void paging_set_boot_phys_offset(uint64_t phys_offset);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 37)
[declaration] bool paging_boot_phys_offset_valid(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 39)
[declaration] void paging_init_direct_map(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 41)
[declaration] void paging_init(uint64_t boot_phys_offset);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 43)
[declaration] uint64_t paging_read_cr3(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 44)
[declaration] void paging_flush_tlb_all(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 45)
[declaration] void paging_flush_page(uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 47)
[declaration] bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 49)
[declaration] bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len,
uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 51)
[declaration] uint64_t paging_translate(uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 52)
[declaration] bool paging_is_mapped(uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 53)
[declaration] bool paging_unmap_page(uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 54)
[declaration] bool paging_set_flags(uint64_t virt, uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 55)
[declaration] bool paging_get_flags(uint64_t virt, uint64_t *out_flags);

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 56)
[declaration] void paging_enable_nx(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 57)
[declaration] bool paging_nx_enabled(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 63)
[declaration] bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 64)
[declaration] uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 65)
[declaration] bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 66)
[declaration] bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 67)
[declaration] bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 69)
[declaration] uint64_t paging_create_pml4(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 70)
[declaration] void paging_destroy_pml4(uint64_t pml4_phys);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 71)
[declaration] void paging_switch_pml4(uint64_t pml4_phys);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 73)
[declaration] bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 75)
[declaration] void paging_enable_write_protect(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 76)
[declaration] void paging_disable_write_protect(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 77)
[declaration] bool paging_write_protect_enabled(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 79)
[declaration] void paging_enable_smep(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 80)
[declaration] void paging_enable_smap(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 81)
[declaration] bool paging_smep_enabled(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 82)
[declaration] bool paging_smap_enabled(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 84)
[declaration] bool paging_pcid_supported(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 85)
[declaration] void paging_enable_pcid(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 86)
[declaration] bool paging_pcid_enabled(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 87)
[declaration] void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 90)
[declaration] bool paging_la57_supported(void);

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 91)
[declaration] void paging_assert_4level_paging(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 92)
[declaration] void paging_write_cr3(uint64_t pml4_phys);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 14)
[declaration] bool tlb_init(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 15)
[declaration] bool tlb_ready(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 16)
[declaration] bool tlb_has_pcid(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 17)
[declaration] bool tlb_has_invpcid(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 19)
[declaration] void tlb_flush_all(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 20)
[declaration] void tlb_flush_all_including_global(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 22)
[declaration] void tlb_flush_page(const void *addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 23)
[declaration] void tlb_flush_page_addr(uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 25)
[declaration] void tlb_flush_range(const void *addr, size_t pages);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 26)
[declaration] void tlb_flush_range_addr(uint64_t addr, size_t pages);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 28)
[declaration] void tlb_flush_pcid(uint16_t pcid);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 29)
[declaration] void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 31)
[declaration] void tlb_wbinvd(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 32)
[declaration] void tlb_clflush(uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 6)
[declaration] void tlb_asm_invlpg(uint64_t addr);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 8)
[declaration] uint64_t tlb_asm_read_cr3(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 9)
[declaration] void tlb_asm_write_cr3(uint64_t value);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 11)
[declaration] uint64_t tlb_asm_read_cr4(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 12)
[declaration] void tlb_asm_write_cr4(uint64_t value);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 14)
[declaration] void tlb_asm_invpcid(uint64_t type, const void *desc);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 16)
[declaration] void tlb_asm_wbinvd(void);
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 17)
[declaration] void tlb_asm_clflush(uint64_t addr);

kernel_Workspace/kernel/src/mm/cache/cache.h (line 7)
[declaration] bool cache_init(void);
kernel_Workspace/kernel/src/mm/cache/cache.h (line 8)
[declaration] bool cache_ready(void);
kernel_Workspace/kernel/src/mm/cache/cache.h (line 10)
[declaration] void cache_register(kcache_t *c);
kernel_Workspace/kernel/src/mm/cache/cache.h (line 13)
[declaration] void cache_dump(void);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 12)
[declaration] typedef void (*kcache_ctor_t)(void *obj);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 47)
[declaration] void kcache_destroy(kcache_t *c);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 51)
[declaration] void kcache_free(kcache_t *c, void *ptr);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 53)
[declaration] size_t kcache_object_size(kcache_t *c);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 54)
[declaration] size_t kcache_free_count(kcache_t *c);
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 55)
[declaration] size_t kcache_total_count(kcache_t *c);
kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 12)
[declaration] bool per_cpu_init(void);
kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 13)
[declaration] size_t per_cpu_count(void);
kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 14)
[declaration] size_t per_cpu_id(void);
kernel_Workspace/kernel/src/mm/core/address.h (line 10)
[declaration] bool addr_is_canonical(uint64_t a);
kernel_Workspace/kernel/src/mm/core/address.h (line 11)
[declaration] bool addr_is_user(uint64_t a);
kernel_Workspace/kernel/src/mm/core/address.h (line 12)
[declaration] bool addr_is_kernel(uint64_t a);
kernel_Workspace/kernel/src/mm/core/address.h (line 13)
[declaration] bool addr_is_direct_map(uint64_t a);
kernel_Workspace/kernel/src/mm/core/address.h (line 15)
[declaration] uint64_t addr_align_up(uint64_t a, uint64_t align);
kernel_Workspace/kernel/src/mm/core/address.h (line 16)
[declaration] uint64_t addr_align_down(uint64_t a, uint64_t align);
kernel_Workspace/kernel/src/mm/core/address.h (line 18)
[declaration] uint64_t addr_phys_to_direct(uint64_t phys);
kernel_Workspace/kernel/src/mm/core/address.h (line 19)
[declaration] uint64_t addr_direct_to_phys(uint64_t va);
kernel_Workspace/kernel/src/mm/core/mm.h (line 26)
[declaration] bool mm_init(const mm_boot_params_t *params);
kernel_Workspace/kernel/src/mm/core/mm.h (line 27)
[declaration] bool mm_ready(void);
kernel_Workspace/kernel/src/mm/core/mm.h (line 29)
[declaration] uint64_t mm_total_ram(void);

kernel_Workspace/kernel/src/mm/core/mm.h (line 30)
[declaration] uint64_t mm_free_ram(void);

kernel_Workspace/kernel/src/mm/core/mm.h (line 32)
[declaration] void mm_dump(void);

kernel_Workspace/kernel/src/mm/core/range.h (line 18)
[declaration] bool range_is_canonical(uint64_t addr);

kernel_Workspace/kernel/src/mm/core/region.h (line 13)
[declaration] region_t region_make(uint64_t start, uint64_t len);

kernel_Workspace/kernel/src/mm/core/region.h (line 14)
[declaration] bool region_valid(region_t r);

kernel_Workspace/kernel/src/mm/core/region.h (line 15)
[declaration] uint64_t region_len(region_t r);

kernel_Workspace/kernel/src/mm/core/region.h (line 17)
[declaration] bool region_contains(region_t r, uint64_t addr);

kernel_Workspace/kernel/src/mm/core/region.h (line 18)
[declaration] bool region_overlaps(region_t a, region_t b);

kernel_Workspace/kernel/src/mm/core/region.h (line 19)
[declaration] region_t region_intersect(region_t a, region_t b);

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 8)
[declaration] bool size_is_pow2(size_t v);

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 11)
[declaration] bool size_round_up_pow2(size_t v, size_t *out);

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 21)
[declaration] void smp_lock_init(smp_ticket_lock_t *lock);

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 24)
[declaration] void smp_lock_acquire(smp_ticket_lock_t *lock);

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 27)
[declaration] bool smp_lock_release(smp_ticket_lock_t *lock);

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 30)
[declaration] int64_t smp_current_cpu_id(void);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 19)
[declaration] bool paging_subsystem_init(void);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 22)
[declaration] void paging_aspace_destroy(address_space_t *as);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 23)
[declaration] void paging_aspace_switch(address_space_t *as);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 24)
[declaration] uint64_t paging_aspace_cr3(const address_space_t *as);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 32)
[declaration] bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 39)
[declaration] uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 41)
[declaration] bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);

kernel_Workspace/kernel/src/mm/paging/paging.h (line 42)
[declaration] bool paging_kernel_unmap(uint64_t virt, size_t len);

kernel_Workspace/kernel/src/mm/paging/pml.h (line 14)
[declaration] uint64_t pml_level_shift(int level);

kernel_Workspace/kernel/src/mm/paging/pml.h (line 15)
[declaration] uint64_t pml_index(uint64_t virt, int level);

kernel_Workspace/kernel/src/mm/paging/pml.h (line 17)
[declaration] bool pml_entry_present(uint64_t entry);

kernel_Workspace/kernel/src/mm/paging/pml.h (line 18)
[declaration] bool pml_entry_large(uint64_t entry);

kernel_Workspace/kernel/src/mm/paging/pml.h (line 19)
[declaration] uint64_t pml_entry_addr(uint64_t entry);

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 16)
[declaration] void tlb_batch_begin(tlb_batch_t *batch);

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 17)
[declaration] void tlb_batch_add(tlb_batch_t *batch, uint64_t virt);

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 18)
[declaration] void tlb_batch_full(tlb_batch_t *batch);

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 19)
[declaration] void tlb_batch_commit(tlb_batch_t *batch);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 30)
[declaration] bool aspace_subsystem_init(void);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 33)
[declaration] void aspace_destroy(proc_aspace_t *pa);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 37)
[declaration] uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 38)
[declaration] uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 39)
[declaration] uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 40)
[declaration] bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 41)
[declaration] bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 51)
[declaration] uint64_t aspace_stack_base(void);

kernel_Workspace/kernel/src/mm/process/address_space.h (line 53)
[declaration] uint64_t aspace_brk(proc_aspace_t *pa, uint64_t new_brk);

kernel_Workspace/kernel/src/mm/process/mmap.h (line 20)
[declaration] bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len);

kernel_Workspace/kernel/src/mm/process/mmap.h (line 21)
[declaration] bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

kernel_Workspace/kernel/src/mm/protection/guard.h (line 9)
[declaration] uint64_t guard_install(proc_aspace_t *pa, uint64_t addr, size_t len);

kernel_Workspace/kernel/src/mm/protection/guard.h (line 10)
[declaration] bool guard_user_stack(proc_aspace_t *pa);

```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 8)
[declaration] bool isolation_init(void);
kernel_Workspace/kernel/src/mm/protection/isolation.h (line 10)
[declaration] bool isolation_has_smep(void);
kernel_Workspace/kernel/src/mm/protection/isolation.h (line 11)
[declaration] bool isolation_has_smmap(void);
kernel_Workspace/kernel/src/mm/protection/isolation.h (line 12)
[declaration] bool isolation_enable_smep(void);
kernel_Workspace/kernel/src/mm/protection/isolation.h (line 13)
[declaration] bool isolation_enable_smmap(void);
kernel_Workspace/kernel/src/mm/protection/isolation.h (line 15)
[declaration] size_t isolation_audit_kernel(void);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 8)
[declaration] uint32_t perm_sanitizize(uint32_t prot);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 9)
[declaration] bool perm_is_wx(uint32_t prot);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 10)
[declaration] bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 11)
[declaration] void perm_set_strict_wx(bool on);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 13)
[declaration] uint32_t perm_kernel_default(void);
kernel_Workspace/kernel/src/mm/protection/permissions.h (line 14)
[declaration] uint32_t perm_user_default(void);
kstd/include/kstd.h (line 12)
[declaration] void tr_log(const char *s);
kstd/include/kstd_audio.h (line 7)
[declaration] void tr_free(void *ptr, uint32_t bytes);
kstd/include/kstd_audio.h (line 8)
[declaration] tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);
kstd/include/kstd_bt.h (line 6)
[declaration] tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6);
kstd/include/kstd_bt.h (line 7)
[declaration] tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len);
kstd/include/kstd_bt.h (line 8)
[declaration] tr_status_t tr_bt_evt(void *buf, uint8_t *len);
kstd/include/kstd_bt.h (line 9)
[declaration] tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
kstd/include/kstd_bt.h (line 10)
[declaration] tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);
kstd/include/kstd_fs.h (line 6)
[declaration] tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out);
kstd/include/kstd_fs.h (line 7)
[declaration] tr_status_t tr_fs_exists(const char *path);
kstd/include/kstd_input.h (line 6)
[declaration] int32_t tr_input_key(void);

```



```

kstd/include/kstd_mem.h (line 7)
[declaration] void tr_free(void *ptr, uint32_t bytes);
kstd/include/kstd_mem.h (line 8)
[declaration] tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);
kstd/include/kstd_video.h (line 22)
[declaration] tr_status_t tr_video_init(void);
kstd/include/kstd_video.h (line 23)
[declaration] bool tr_video_is_ready(void);
kstd/include/kstd_video.h (line 24)
[declaration] tr_status_t tr_video_get_caps(tr_video_caps_t *caps);
kstd/include/kstd_video.h (line 27)
[declaration] uint32_t tr_video_mode_count(void);
kstd/include/kstd_video.h (line 28)
[declaration] tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
kstd/include/kstd_video.h (line 29)
[declaration] tr_status_t tr_video_mode_set(uint32_t mode_id);
kstd/include/kstd_video.h (line 32)
[declaration] tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y,
uint32_t w, uint32_t h);
libs/dsabi.h (line 93)
[declaration] __asm__ volatile("int $0x80" :: "a"(1));
libs/dsabi.h (line 124)
[declaration] void ds_init(uint64_t params_va);
libs/dsabi.h (line 127)
[declaration] int ds_take(uint64_t id, ds_msg_t *out);
libs/dsabi.h (line 128)
[declaration] void ds_poll(void);
libs/dsasync.h (line 21)
[function] if (r->done) {
libs/dsasync.h (line 25)
[declaration] ds_poll();
libs/dsasync.h (line 27)
[function] if (r->id != 0 && ds_take(r->id, &r->msg)) {
libs/dsaudio.h (line 7)
[declaration] return ds_call(SVC_AUDIO, AUD_PLAY, phys, len, 0);
libs/dsaudio.h (line 12)
[declaration] ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
libs/dsaudio.h (line 17)
[declaration] return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
libs/dsaudio.h (line 22)
[declaration] ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
libs/dsinput.h (line 7)
[declaration] return ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
libs/dsinput.h (line 14)
[function] if (!ds_take(id, &m) || m.arg0 == 0) {
libs/dslock.h (line 7)
[declaration] return ds_call(SVC_BLOCK, BLK_COUNT, disk, 0, 0);
libs/dslock.h (line 12)
[declaration] return ds_call(SVC_BLOCK, BLK_READ, disk, block, 0);

```

```

libs/dslock.h (line 17)
[declaration] return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
libs/dslog.h (line 10)
[function] while (s[n] != '\0' && n < DS_SCRATCH_SIZE - 1) {
libs/dslog.h (line 15)
[declaration] ds_call(SVC_SYS, OP_LOG, n, 0, 0);
libs/dsmem.h (line 7)
[declaration] return ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
libs/dsmem.h (line 12)
[declaration] return ds_call(SVC_SYS, OP_FREE, va, 0, 0);
libs/dsmem.h (line 17)
[declaration] return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
libs/dsmem.h (line 22)
[declaration] return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
libs/dsvgpu.h (line 6)
[declaration] extern uint64_t ds_call(uint32_t, uint32_t, uint64_t, uint64_t,
uint64_t);
libs/dsvgpu.h (line 7)
[declaration] extern void ds_poll(void);
libs/dsvgpu.h (line 8)
[declaration] extern int ds_take(uint64_t, ds_msg_t *);
libs/dsvgpu.h (line 14)
[declaration] ds_poll();
libs/dsvgpu.h (line 18)
[function] if (!ds_take(id, &r) || r.status != 0) {
libs/dsvgpu.h (line 31)
[declaration] ds_poll();
libs/dsvideo.h (line 14)
[declaration] return ds_call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0);
libs/dsvideo.h (line 21)
[function] if (!ds_take(id, &m) || m.status != 0) {
libs/dsvideo.h (line 35)
[declaration] ds_call(SVC_VIDEO, VID_FB_TAKEOVER, 0, 0, 0);
libs/dsvideo.h (line 40)
[declaration] ds_call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0);

```

Odin

```

drivers/audiodriver/src/odin/driver.odin (line 8)
[struct] Bdl_Entry :: struct {
drivers/audiodriver/src/odin/driver.odin (line 16)
[proc] nam16 :: proc(off: u32) -> ^u16 {
drivers/audiodriver/src/odin/driver.odin (line 21)
[proc] bm8 :: proc(off: u32) -> ^u8 {
drivers/audiodriver/src/odin/driver.odin (line 25)
[proc] bm16 :: proc(off: u32) -> ^u16 {
drivers/audiodriver/src/odin/driver.odin (line 29)
[proc] bm32 :: proc(off: u32) -> ^u32 {
drivers/audiodriver/src/odin/driver.odin (line 50)
[proc] ad_init :: proc "C" (nam_va: u64, bm_va: u64) -> i32 {

```

```

drivers/audiodriver/src/odin/driver.odin (line 73)
[proc] ad_stop :: proc "C" () -> i32 {
drivers/audiodriver/src/odin/driver.odin (line 80)
[proc] ad_position :: proc "C" () -> u32 {
drivers/audiodriver/src/odin/jack.odin (line 8)
[proc] ad_jack_present :: proc "C" () -> i32 {
drivers/audiodriver/src/odin/jack.odin (line 21)
[proc] ad_set_amp :: proc "C" (on: i32) -> i32 {
drivers/audiodriver/src/odin/microphone.odin (line 6)
[proc] ad_capture :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64)
-> i32 {
drivers/audiodriver/src/odin/speaker.odin (line 6)
[proc] ad_play :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) ->
i32 {

```

Python

```

doc.py (line 131)
[class] class Definition:
doc.py (line 144)
[class] class MarkdownFile:
doc.py (line 153)
[function] def iter_source_files(root: str, exts: set[str] | None, exclude_dirs:
set[str]):
doc.py (line 170)
[function] def read_text(path: str) -> str:
doc.py (line 185)
[function] def extract_preceding_comment(lines: list[str], lineno_0based: int, ext:
str) -> str:
doc.py (line 222)
[function] def parse_python(rel_path: str, full_path: str, source: str) ->
list[Definition]:
doc.py (line 457)
[function] def parse_generic(rel_path: str, ext: str, source: str, max_body_lines:
int) -> list[Definition]:
doc.py (line 526)
[function] def collect(root: str, exts: set[str] | None, exclude_dirs: set[str],
max_body_lines: int):
doc.py (line 550)
[function] def build_styles():
doc.py (line 587)
[function] def group_by_language(defs: list[Definition]):
doc.py (line 594)
[function] def path_line_label(d: Definition) -> str:
doc.py (line 598)
[function] def render_signature_section(story, styles, title: str, defs:
list[Definition], max_items: int):
doc.py (line 622)
[function] def render_fragments_section(story, styles, defs: list[Definition],
max_items: int):
doc.py (line 647)
[function] def render_markdown_section(story, styles, md_files: list[MarkdownFile]):

```

doc.py (line 680)

```
[function] def build_pdf(output_path: str, root: str, public_defs, internal_defs,
all_defs_for_fragments, ...
```

doc.py (line 734)

```
[function] def main():
```

Rust

drivers/audiodriver/src/jacklib.rs (line 4)

```
[struct] pub struct JackMgr {
```

drivers/audiodriver/src/jacklib.rs (line 14)

```
[function] pub fn tick(&mut self) {
```

drivers/audiodriver/src/jacklib.rs (line 30)

```
[function] pub fn set_amp(&mut self, on: bool) {
```

drivers/audiodriver/src/jacklib.rs (line 35)

```
[function] pub fn present(&self) -> bool {
```

drivers/audiodriver/src/jacklib.rs (line 40)

```
[function] pub fn amp_enabled(&self) -> bool {
```

drivers/audiodriver/src/lib.rs (line 16)

```
[function] pub fn init(nam_va: u64, bm_va: u64) -> bool {
```

drivers/audiodriver/src/lib.rs (line 20)

```
[function] pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool
{
```

drivers/audiodriver/src/lib.rs (line 24)

```
[function] pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) ->
bool {
```

drivers/audiodriver/src/lib.rs (line 28)

```
[function] pub fn stop() {
```

drivers/audiodriver/src/lib.rs (line 32)

```
[function] pub fn jack_present() -> bool {
```

drivers/audiodriver/src/lib.rs (line 36)

```
[function] pub fn set_amp(on: bool) {
```

drivers/audiodriver/src/lib.rs (line 40)

```
[function] pub fn position() -> u32 {
```

drivers/audiodriver/src/tone.rs (line 1)

```
[function] pub fn fill_square(buf: &mut [u8], periods: u32) {
```

driverspace/src/drivers/storage.rs (line 9)

```
[struct] pub struct StorageDrv {
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 4)

```
[struct] pub struct RegistryEntry {
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 11)

```
[struct] pub struct Registry {
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 20)

```
[function] pub fn entries(&self) -> &[RegistryEntry] {
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 24)

```
[function] pub fn find(&self, vendor_id: u16, device_id: u16) ->
Option<&RegistryEntry> {
```

driverspace_workspace/crates/ds-registry/src/matching.rs (line 5)

```
[function] pub fn find_driver_for_device(vendor_id: u16, device_id: u16) ->
Option<DriverId> {
```

```

driverspace_workspace/crates/ds-registry/src/matching.rs (line 10)
[function] pub fn get_driver_name(driver_id: DriverId) -> Option<&'static str> {
driverspace_workspace/crates/ds-registry/src/state.rs (line 5)
[enum] pub enum DeviceStatus {
driverspace_workspace/crates/ds-registry/src/state.rs (line 17)
[struct] pub struct DeviceState {
driverspace_workspace/crates/ds-registry/src/state.rs (line 34)
[function] pub fn set_attached(&mut self, driver_id: u32) {
driverspace_workspace/crates/ds-registry/src/state.rs (line 39)
[function] pub fn set_detached(&mut self) {
driverspace_workspace/crates/ds-registry/src/state.rs (line 44)
[function] pub fn set_error(&mut self) {
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 7)
[function] pub fn ds_spark_safe_add_32(a: u32, b: u32) -> u32;
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 10)
[function] pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 21)
[function] pub fn safe_add(a: u32, b: u32) -> u32 {
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 28)
[function] pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 29)
[function] pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 35)
[function] pub fn get_buddy(addr: u64, exp: u64) -> u64 {
driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 1)
[function] pub fn add(left: u64, right: u64) -> u64 {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 7)
[enum] pub enum RequestState {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 15)
[struct] pub struct IoRequest {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 28)
[function] pub fn new(id: u64, lba: u64, block_count: u32, buffer: *mut u8, len:
usize, is_write: bool) -> Self {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 42)
[function] pub fn state(&self) -> RequestState {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 52)
[function] pub fn mark_in_progress(&self) {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 56)
[function] pub fn mark_completed(&self) {
driverspace_workspace/lib/ds-fw-block/src/request.rs (line 63)
[function] pub fn mark_failed(&self) {
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 6)
[struct] pub struct BlockRequest {
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 16)
[trait] pub trait BlockDevice {
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 40)
[struct] pub struct BlockGeometry {
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 47)
[function] pub fn total_bytes(&self) -> u64 {

```

```

driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 1)
[function] pub fn add(left: u64, right: u64) -> u64 {
driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 1)
[function] pub fn add(left: u64, right: u64) -> u64 {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 6)
[struct] pub struct ManagerClient {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 15)
[function] pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64,
DsError> {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 20)
[function] pub fn bind_irq(&self, irq_num: u32) -> Result<(), DsError> {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 25)
[function] pub fn alloc_dma(&self, size: u64, flags: u32) -> Result<(u64, u64),
DsError> {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 30)
[function] pub fn pci_find(&self, class_code: u32) -> Result<u64, DsError> {
driverspace_workspace/lib/ds-ipc/src/client.rs (line 35)
[function] pub fn get_page_phys(&self, virt_addr: u64) -> Result<u64, DsError> {
driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 10)
[struct] pub struct MessageDispatcher {
driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 21)
[function] pub fn register(&mut self, opcode: Opcode, handler: HandlerFn) ->
Result<(), DsError> {
driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 30)
[function] pub fn dispatch(&self, sender: Handle, opcode_raw: u32, payload_ptr:
*const u8, payload_len: u32) -> Result<(), DsError> {
driverspace_workspace/lib/ds-log/src/formatter.rs (line 7)
[struct] pub struct LogBuffer {
driverspace_workspace/lib/ds-log/src/formatter.rs (line 20)
[function] pub fn as_bytes(&self) -> &[u8] {
driverspace_workspace/lib/ds-log/src/formatter.rs (line 24)
[function] pub fn reset(&mut self) {
driverspace_workspace/lib/ds-log/src/levels.rs (line 6)
[enum] pub enum LogLevel {
driverspace_workspace/lib/ds-log/src/lib.rs (line 15)
[struct] pub struct EndpointRef;
driverspace_workspace/lib/ds-log/src/lib.rs (line 20)
[function] pub fn is_valid(&self) -> bool {
driverspace_workspace/lib/ds-log/src/lib.rs (line 25)
[function] pub fn handle(&self) -> u32 {
driverspace_workspace/lib/ds-log/src/lib.rs (line 29)
[function] pub fn set(&self, handle: Handle) {
driverspace_workspace/lib/ds-log/src/transport.rs (line 15)
[function] pub fn emit_log(level: LogLevel, module: &str, buf: &LogBuffer) {
driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 5)
[struct] pub struct BuddyAllocator {
driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 20)
[function] pub fn alloc_pages(&mut self, order: usize) -> Option<usize> {
driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 28)
[function] pub fn free_pages(&mut self, addr: usize, order: usize) {

```

```

driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 6)
[struct] pub struct DriverHeap {
driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 18)
[function] pub fn init(&mut self, base_addr: usize, total_pages: usize) {
driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 1)
[struct] pub struct SlabAllocator {
driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 14)
[function] pub fn alloc(&mut self) -> Option<usize> {
driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 19)
[function] pub fn free(&mut self, ptr: usize) {
driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 7)
[struct] pub struct DmaFlags(pub u32);
driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 15)
[struct] pub struct DmaBuffer {
driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 23)
[function] pub fn allocate(manager_ep: Handle, size: u64, flags: DmaFlags) ->
Result<Self, DsError> {
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 6)
[struct] pub struct MmioRegion {
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 14)
[function] pub fn map(manager_ep: Handle, phys_base: u64, size: u64) -> Result<Self,
DsError> {
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 30)
[struct] pub struct Volatile<T>(pub T);
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 34)
[function] pub fn read(&self) -> T {
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 39)
[function] pub fn write(&mut self, value: T) {
driverspace_workspace/lib/ds-mem/src/vmm.rs (line 6)
[function] pub fn map_memory(
driverspace_workspace/lib/ds-mem/src/vmm.rs (line 17)
[function] pub fn unmap_memory(_manager_ep: Handle, virt_addr: u64, size: u64) ->
Result<(), DsError> {
driverspace_workspace/lib/kapi-abi/src/errors.rs (line 7)
[enum] pub enum DsError {
driverspace_workspace/lib/kapi-abi/src/errors.rs (line 44)
[function] pub fn from_u32(val: u32) -> Self {
driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 5)
[enum] pub enum DsCmd {
driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 30)
[function] pub fn from_u32(val: u32) -> Option<Self> {
driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 10)
[enum] pub enum BusType {
driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 22)
[struct] pub struct DeviceFlags: u32 {
driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 38)
[struct] pub struct DevAttachPayload {
driverspace_workspace/lib/kapi-abi/src/payloads/irq.rs (line 6)
[struct] pub struct IrqBindPayload {

```



```

driverspace_workspace/lib/kapi-abi/src/payloads/mem.rs (line 5)
[struct] pub struct DmaAllocPayload {
driverspace_workspace/lib/kapi-abi/src/payloads/mem.rs (line 13)
[struct] pub struct MmioMapPayload {
driverspace_workspace/lib/kapi-abi/src/payloads/sys.rs (line 9)
[struct] pub struct LogPayload {
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 5)
[struct] pub struct Handle(pub u32);
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 9)
[struct] pub struct CapId(pub u64);
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 13)
[enum] pub enum Status {
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 41)
[struct] pub struct PhysAddr(pub u64);
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 45)
[struct] pub struct VirtAddr(pub u64);
driverspace_workspace/lib/kapi-abi/src/wire.rs (line 6)
[struct] pub struct DsMsg {
driverspace_workspace/lib/kapi-abi/src/wire.rs (line 31)
[function] pub fn is_ok(&self) -> bool {
driverspace_workspace/lib/kapi-abi/src/wire.rs (line 35)
[function] pub fn error_status(&self) -> crate::DsError {
driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 5)
[struct] pub struct RingBuffer {
driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 24)
[function] pub fn push(&self, msg: &DsMsg) -> bool {
driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 40)
[function] pub fn pop(&self) -> Option<DsMsg> {
driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 21)
[function] pub fn sys_ipc_call(cmd: DsCmd, arg0: u64, arg1: u64, arg2: u64) -> DsMsg
{
driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 40)
[function] pub fn sys_poll() {
driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 48)
[function] pub fn sys_yield() {
driverspace_workspace/src/drivers/vgpu.rs (line 8)
[function] pub fn init() -> Result<(), Status> {
driverspace_workspace/src/ipc.rs (line 7)
[struct] pub struct Message {
driverspace_workspace/src/ipc.rs (line 21)
[function] pub fn send(msg: Message) -> Result<(), Status> {
driverspace_workspace/src/ipc.rs (line 38)
[function] pub fn receive() -> Option<Message> {
driverspace_workspace/src/ipc.rs (line 55)
[function] pub fn reply(status: Status, data: u64) {
driverspace_workspace/src/memory.rs (line 5)
[function] pub fn alloc(size: usize) -> Result<*mut u8, Status> {
driverspace_workspace/src/memory.rs (line 23)
[function] pub fn free(ptr: *mut u8, size: usize) {

```

```

driverspace_workspace/src/self_test.rs (line 6)
[function] pub fn run_all() -> Result<usize, Status> {
driverspacelib/src/abi.rs (line 53)
[enum] pub enum DsCmd {
driverspacelib/src/abi.rs (line 83)
[struct] pub struct DsRing {
driverspacelib/src/abi.rs (line 94)
[struct] pub struct DsMsg {
driverspacelib/src/abi.rs (line 110)
[struct] pub struct DsInitParams {
driverspacelib/src/abi.rs (line 123)
[enum] pub enum DsError {
driverspacelib/src/audio.rs (line 4)
[struct] pub struct AudioBars {
driverspacelib/src/audio.rs (line 9)
[function] pub fn info_req() -> u64 {
driverspacelib/src/audio.rs (line 13)
[function] pub fn info_take(id: u64) -> Option<AudioBars> {
driverspacelib/src/audio.rs (line 23)
[function] pub fn page_phys_req(va: u64) -> u64 {
driverspacelib/src/audio.rs (line 27)
[function] pub fn page_phys_take(id: u64) -> Option<u64> {
driverspacelib/src/driver.rs (line 6)
[struct] pub struct DeviceInfo {
driverspacelib/src/driver.rs (line 14)
[trait] pub trait Driver {
driverspacelib/src/input.rs (line 4)
[function] pub fn key_req() -> u64 {
driverspacelib/src/input.rs (line 8)
[function] pub fn key_take(id: u64) -> Option<u8> {
driverspacelib/src/jack.rs (line 4)
[struct] pub struct JackInfo {
driverspacelib/src/jack.rs (line 9)
[function] pub fn query_req() -> u64 {
driverspacelib/src/jack.rs (line 13)
[function] pub fn query_take(id: u64) -> Option<JackInfo> {
driverspacelib/src/jack.rs (line 22)
[function] pub fn set_amp(on: bool) {
driverspacelib/src/jack.rs (line 26)
[function] pub fn play(va: u64, len: u32) -> u64 {
driverspacelib/src/jack.rs (line 30)
[function] pub fn stop() {
driverspacelib/src/log.rs (line 10)
[function] pub fn ds_log(msg: &str) {
driverspacelib/src/log.rs (line 15)
[function] pub fn ds_log_raw(ptr: *const u8, len: usize) {
driverspacelib/src/runtime.rs (line 38)
[function] pub fn init_once(params_va: u64) {

```

```

driverspacelib/src/runtime.rs (line 51)
[function] pub fn register<D: Driver>(drv: &mut D) {
driverspacelib/src/runtime.rs (line 94)
[function] pub fn request(cmd: DsCmd, a0: u64, a1: u64, a2: u64) -> u64 {
driverspacelib/src/runtime.rs (line 99)
[function] pub fn request_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
driverspacelib/src/runtime.rs (line 104)
[function] pub fn tick() {
driverspacelib/src/runtime.rs (line 128)
[function] pub fn take_resp(id: u64) -> Option<DsMsg> {
driverspacelib/src/svc.rs (line 4)
[function] pub fn call(class: u32, op: u32, a0: u64, a1: u64, a2: u64) -> u64 {
driverspacelib/src/svc.rs (line 8)
[function] pub fn take(id: u64) -> Option<DsMsg> {
driverspacelib/src/video.rs (line 4)
[struct] pub struct FbInfo {
driverspacelib/src/video.rs (line 11)
[function] pub fn fb_info_req() -> u64 {
driverspacelib/src/video.rs (line 15)
[function] pub fn fb_info_take(id: u64) -> Option<FbInfo> {
driverspacelib/src/video.rs (line 26)
[function] pub fn takeover() -> u64 {
driverspacelib/src/video.rs (line 30)
[function] pub fn release() -> u64 {
gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 6)
[struct] pub struct GfxContext {
gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 11)
[function] pub fn init() -> Self {
gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 15)
[function] pub fn create_surface(&self, width: u32, height: u32) ->
Option<SurfaceId> {
gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 35)
[function] pub fn commit(&self, surface: SurfaceId) {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 14)
[struct] pub struct DisplayServer {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 23)
[function] pub fn new(width: u32, height: u32) -> Self {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 31)
[function] pub fn create_surface(&mut self, width: u32, height: u32) -> u32 {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 48)
[function] pub fn set_position(&mut self, id: u32, x: i32, y: i32) {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 56)
[function] pub fn commit(&mut self, _id: u32) {}
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 58)
[function] pub fn composite(&mut self) {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 84)
[function] pub fn framebuffer_ptr(&self) -> *const u32 { self.fb }
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 85)
[function] pub fn fb_size(&self) -> usize { (self.width * self.height * 4) as usize }

```

```

gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 5)
[struct] pub struct GpuBridge {
gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 14)
[function] pub fn new() -> Self {
gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 18)
[function] pub fn init(&mut self) -> bool {
gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 47)
[function] pub fn present(&self, src_buffer: *const u32, size: usize) {
gfx_protocol_workspace/lib/gfx-protocol/src/commands.rs (line 7)
[enum] pub enum GfxCmd {
gfx_protocol_workspace/lib/gfx-protocol/src/commands.rs (line 24)
[struct] pub struct GfxIpcMsg {
gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 5)
[struct] pub struct SurfaceId(pub u32);
gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 9)
[struct] pub struct BufferId(pub u32);
gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 13)
[enum] pub enum PixelFormat {
gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 20)
[struct] pub struct SurfaceConfig {
kernel-drivers/audio/src/lib.rs (line 19)
[function] pub fn init(base: u32) -> bool {
kernel-drivers/audio/src/lib.rs (line 23)
[function] pub fn poll_jack() -> bool {
kernel-drivers/audio/src/lib.rs (line 36)
[function] pub fn query() -> u32 {
kernel-drivers/audio/src/lib.rs (line 40)
[function] pub fn set_amp(on: bool) {
kernel-drivers/audio/src/lib.rs (line 45)
[function] pub fn play_phys(phys: u64, len: u32) -> bool {
kernel-drivers/audio/src/lib.rs (line 49)
[function] pub fn stop() {
kernel-drivers/battery/src/lib.rs (line 9)
[function] pub fn init() -> bool {
kernel-drivers/bluetooth/src/lib.rs (line 11)
[function] pub fn init() -> bool {
kernel-drivers/bluetooth/src/lib.rs (line 15)
[function] pub fn ready() -> bool {
kernel-drivers/camera/src/lib.rs (line 9)
[function] pub fn init() -> bool {
kernel-drivers/displayport/src/lib.rs (line 10)
[function] pub fn init() -> bool {
kernel-drivers/displayport/src/lib.rs (line 14)
[function] pub fn ready() -> bool {
kernel-drivers/gfx/src/lib.rs (line 9)
[struct] pub struct FramebufferInfo {
kernel-drivers/gfx/src/lib.rs (line 16)
[function] pub fn fb_info() -> Option<FramebufferInfo> {

```

```

kernel-drivers/hdmi/src/lib.rs (line 10)
[function] pub fn init() -> bool {
kernel-drivers/hdmi/src/lib.rs (line 22)
[function] pub fn ready() -> bool {
kernel-drivers/hw-sys/src/lib.rs (line 13)
[function] pub fn kprintf(fmt: *const u8, ...);
kernel-drivers/hw-sys/src/lib.rs (line 16)
[function] pub fn dma_alloc_coherent(
kernel-drivers/hw-sys/src/lib.rs (line 23)
[function] pub fn dma_free_coherent(phys: u64, virt: *mut u8, bytes: usize);
kernel-drivers/hw-sys/src/lib.rs (line 26)
[function] pub fn mmio_map(phys: u64, len: usize, out_virt: *mut u64) -> bool;
kernel-drivers/hw-sys/src/lib.rs (line 28)
[function] pub fn mmio_unmap(virt: u64, len: usize);
kernel-drivers/hw-sys/src/lib.rs (line 31)
[function] pub fn paging_is_mapped(virt: u64) -> bool;
kernel-drivers/hw-sys/src/lib.rs (line 34)
[function] pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32,
out_phys: *mut u64) -> bool;
kernel-drivers/hw-sys/src/lib.rs (line 39)
[function] pub fn info(msg: &str) {
kernel-drivers/pcie/src/lib.rs (line 20)
[struct] pub struct PciDev {
kernel-drivers/pcie/src/lib.rs (line 28)
[struct] pub struct PciDevice {
kernel-drivers/pcie/src/lib.rs (line 62)
[function] pub fn read32(d: PciDev, off: u32) -> u32 {
kernel-drivers/pcie/src/lib.rs (line 69)
[function] pub fn write32(d: PciDev, off: u32, v: u32) {
kernel-drivers/pcie/src/lib.rs (line 76)
[function] pub fn read16(d: PciDev, off: u32) -> u16 {
kernel-drivers/pcie/src/lib.rs (line 80)
[function] pub fn read8(d: PciDev, off: u32) -> u8 {
kernel-drivers/pcie/src/lib.rs (line 85)
[function] pub fn vendor(self) -> u16 {
kernel-drivers/pcie/src/lib.rs (line 89)
[function] pub fn device_id(self) -> u16 {
kernel-drivers/pcie/src/lib.rs (line 93)
[function] pub fn class(self) -> u8 {
kernel-drivers/pcie/src/lib.rs (line 97)
[function] pub fn subclass(self) -> u8 {
kernel-drivers/pcie/src/lib.rs (line 101)
[function] pub fn prog_if(self) -> u8 {
kernel-drivers/pcie/src/lib.rs (line 106)
[function] pub fn bar(self, idx: u32) -> u64 {
kernel-drivers/pcie/src/lib.rs (line 117)
[function] pub fn enable_mmio(self) {

```

```

kernel-drivers/pcie/src/lib.rs (line 124)
[function] pub fn find_class(class: u8, subclass: u8, prog_if: u8) -> Option<PciDev>
{
kernel-drivers/pcie/src/lib.rs (line 170)
[function] pub fn enumerate() -> Vec<PciDevice> {
kernel-drivers/usb/src/class/hid/keyboard.rs (line 18)
[function] pub fn key_to_ascii(key: u8, shift: bool) -> Option<u8> {
kernel-drivers/usb/src/class/hid/keyboard.rs (line 44)
[function] pub fn push_char(c: u8) {
kernel-drivers/usb/src/class/hid/keyboard.rs (line 55)
[function] pub fn take_char() -> Option<u8> {
kernel-drivers/usb/src/class/hid/mod.rs (line 16)
[struct] pub struct HidKeyboard {
kernel-drivers/usb/src/class/hid/mod.rs (line 31)
[function] pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool,
UsbError> {
kernel-drivers/usb/src/class/hid/mod.rs (line 105)
[function] pub fn poll(x: &mut Xhci) {
kernel-drivers/usb/src/class/hid/report.rs (line 1)
[struct] pub struct BootReport {
kernel-drivers/usb/src/class/hid/report.rs (line 6)
[function] pub fn parse_boot_keyboard(report: &[u8]) -> Option<BootReport> {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 6)
[enum] pub enum DriverError {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 10)
[trait] pub trait BlockDevice {
kernel-drivers/usb/src/class/mass/mod.rs (line 18)
[struct] pub struct UsbMass {
kernel-drivers/usb/src/class/mass/mod.rs (line 32)
[function] pub fn bulk_out(&mut self, x: &mut Xhci, phys: u64, len: u32) ->
Result<(), UsbError> {
kernel-drivers/usb/src/class/mass/mod.rs (line 45)
[function] pub fn bulk_in(&mut self, x: &mut Xhci, phys: u64, len: u32) -> Result<(),
UsbError> {
kernel-drivers/usb/src/class/mass/mod.rs (line 58)
[function] pub fn scsi_cmd(&mut self, x: &mut Xhci, cdb: &[u8],
kernel-drivers/usb/src/class/mass/mod.rs (line 102)
[function] pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool,
UsbError> {
kernel-drivers/usb/src/class/mass/mod.rs (line 182)
[function] pub fn mass0() -> Option<&'static mut UsbMass> {
kernel-drivers/usb/src/class/mass/mod.rs (line 186)
[function] pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> R {
kernel-drivers/usb/src/class/mass/scsi.rs (line 5)
[function] pub fn read_capacity(x: &mut Xhci, m: &mut UsbMass) -> Result<(u32, u32),
UsbError> {
kernel-drivers/usb/src/class/mass/scsi.rs (line 20)
[function] pub fn read10(x: &mut Xhci, m: &mut UsbMass,
kernel-drivers/usb/src/class/mass/scsi.rs (line 33)
[function] pub fn write10(x: &mut Xhci, m: &mut UsbMass,

```

```

kernel-drivers/usb/src/class/mod.rs (line 6)
[trait] pub trait ClassDriver {
kernel-drivers/usb/src/core/descriptor.rs (line 2)
[struct] pub struct DeviceDesc {
kernel-drivers/usb/src/core/descriptor.rs (line 13)
[struct] pub struct InterfaceDesc {
kernel-drivers/usb/src/core/descriptor.rs (line 22)
[struct] pub struct EndpointDesc {
kernel-drivers/usb/src/core/descriptor.rs (line 29)
[function] pub fn parse_device(buf: &[u8]) -> Option<DeviceDesc> {
kernel-drivers/usb/src/core/descriptor.rs (line 45)
[function] pub fn parse_config(buf: &[u8],
kernel-drivers/usb/src/core/device.rs (line 7)
[struct] pub struct UsbDevice {
kernel-drivers/usb/src/core/device.rs (line 24)
[function] pub fn new(slot: u8, speed: u32, ctx_size: usize) -> Result<Self,
UsbError> {
kernel-drivers/usb/src/core/enumerate.rs (line 11)
[function] pub fn enumerate(x: &mut Xhci, port: u32) -> Result<UsbDevice, UsbError>
{
kernel-drivers/usb/src/core/speed.rs (line 6)
[function] pub fn default_ep0_mps(speed: u32) -> u16 {
kernel-drivers/usb/src/dma.rs (line 4)
[struct] pub struct DmaBuf {
kernel-drivers/usb/src/dma.rs (line 11)
[function] pub fn new(len: usize) -> Result<Self, UsbError> {
kernel-drivers/usb/src/dma.rs (line 24)
[function] pub fn zero(&mut self) {
kernel-drivers/usb/src/host/xhci/context.rs (line 4)
[struct] pub struct Contexts {
kernel-drivers/usb/src/host/xhci/context.rs (line 11)
[function] pub fn setup_configure_ep(&mut self, speed: u32, port: u32,
kernel-drivers/usb/src/host/xhci/context.rs (line 37)
[function] pub fn setup_configure_bulk_pair(&mut self, speed: u32, port: u32,
kernel-drivers/usb/src/host/xhci/context.rs (line 71)
[function] pub fn new(ctx_size: usize) -> Result<Self, UsbError> {
kernel-drivers/usb/src/host/xhci/context.rs (line 97)
[function] pub fn setup_address_device(&mut self, speed: u32, port: u32,
kernel-drivers/usb/src/host/xhci/context.rs (line 120)
[function] pub fn setup_evaluate_mps(&mut self, mps: u16) {
kernel-drivers/usb/src/host/xhci/control.rs (line 6)
[function] pub fn wait_transfer(x: &mut Xhci, slot: u8) -> Result<u8, UsbError> {
kernel-drivers/usb/src/host/xhci/control.rs (line 26)
[function] pub fn wait_transfer_ep(x: &mut Xhci, slot: u8, ep: u8) -> Result<u8,
UsbError> {
kernel-drivers/usb/src/host/xhci/control.rs (line 46)
[function] pub fn control(x: &mut Xhci,
kernel-drivers/usb/src/host/xhci/control.rs (line 94)
[function] pub fn control_in(x: &mut Xhci, dev: &mut UsbDevice,

```



```

kernel-drivers/usb/src/host/xhci/control.rs (line 102)
[function] pub fn control_out(x: &mut Xhci, dev: &mut UsbDevice,
kernel-drivers/usb/src/host/xhci/event.rs (line 6)
[function] pub fn drain_events(&mut self) {
kernel-drivers/usb/src/host/xhci/init.rs (line 14)
[struct] pub struct Xhci {
kernel-drivers/usb/src/host/xhci/init.rs (line 41)
[function] pub(super) fn rt_write64(regs: &XhciRegs, off: usize, v: u64) {
kernel-drivers/usb/src/host/xhci/init.rs (line 46)
[function] pub fn init(regs: XhciRegs) -> Result<Xhci, UsbError> {
kernel-drivers/usb/src/host/xhci/init.rs (line 100)
[function] pub fn command(&mut self, trb: Trb) -> Result<Trb, UsbError> {
kernel-drivers/usb/src/host/xhci/init.rs (line 127)
[function] pub fn scan_ports(&mut self) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 35)
[struct] pub struct XhciRegs {
kernel-drivers/usb/src/host/xhci/regs.rs (line 48)
[function] pub fn new(phys: u64) -> Result<Self, UsbError> {
kernel-drivers/usb/src/host/xhci/regs.rs (line 102)
[function] pub fn op_read(&self, off: usize) -> u32 {
kernel-drivers/usb/src/host/xhci/regs.rs (line 106)
[function] pub fn op_write(&self, off: usize, v: u32) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 110)
[function] pub fn rt_read(&self, off: usize) -> u32 {
kernel-drivers/usb/src/host/xhci/regs.rs (line 114)
[function] pub fn rt_write(&self, off: usize, v: u32) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 118)
[function] pub fn doorbell(&self, slot: u32, target: u32, task: u32) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 125)
[function] pub fn port_sc(&self, port: u32) -> u32 {
kernel-drivers/usb/src/host/xhci/regs.rs (line 129)
[function] pub fn port_sc_write(&self, port: u32, v: u32) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 133)
[function] pub fn port_speed(&self, port: u32) -> u32 {
kernel-drivers/usb/src/host/xhci/ring.rs (line 5)
[struct] pub struct CmdRing {
kernel-drivers/usb/src/host/xhci/ring.rs (line 13)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {
kernel-drivers/usb/src/host/xhci/ring.rs (line 24)
[function] pub fn phys(&self) -> u64 {
kernel-drivers/usb/src/host/xhci/ring.rs (line 28)
[function] pub fn enqueue(&mut self, mut trb: Trb) {
kernel-drivers/usb/src/host/xhci/ring.rs (line 48)
[struct] pub struct EventRing {
kernel-drivers/usb/src/host/xhci/ring.rs (line 56)
[struct] pub struct TransferRing {
kernel-drivers/usb/src/host/xhci/ring.rs (line 64)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {

```

```

kernel-drivers/usb/src/host/xhci/ring.rs (line 75)
[function] pub fn phys(&self) -> u64 {
kernel-drivers/usb/src/host/xhci/ring.rs (line 79)
[function] pub fn enqueue(&mut self, mut trb: Trb) {
kernel-drivers/usb/src/host/xhci/ring.rs (line 100)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {
kernel-drivers/usb/src/host/xhci/ring.rs (line 113)
[function] pub fn erst_phys(&self) -> u64 {
kernel-drivers/usb/src/host/xhci/ring.rs (line 117)
[function] pub fn pending(&self) -> Option<Trb> {
kernel-drivers/usb/src/host/xhci/ring.rs (line 129)
[function] pub fn pop(&mut self) {
kernel-drivers/usb/src/host/xhci/ring.rs (line 138)
[function] pub fn erdp(&self) -> u64 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 29)
[struct] pub struct Trb {
kernel-drivers/usb/src/host/xhci/trb.rs (line 36)
[function] pub fn typ(&self) -> u32 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 40)
[function] pub fn cycle(&self) -> bool {
kernel-drivers/usb/src/host/xhci/trb.rs (line 44)
[function] pub fn completion_code(&self) -> u8 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 48)
[function] pub fn slot_id(&self) -> u8 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 52)
[function] pub fn ep_id(&self) -> u8 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 56)
[function] pub fn transfer_len(&self) -> u32 {
kernel-drivers/usb/src/host/xhci/trb.rs (line 60)
[function] pub fn link(addr: u64) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 68)
[function] pub fn enable_slot() -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 72)
[function] pub fn address_device(slot: u8, ctx_phys: u64) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 80)
[function] pub fn configure_ep(slot: u8, ctx_phys: u64) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 88)
[function] pub fn noop_cmd() -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 92)
[function] pub fn setup_stage(raw_setup: u64, trt: u32) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 100)
[function] pub fn data_stage(addr: u64, len: u32, dir_in: bool) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 108)
[function] pub fn status_stage(dir_in: bool) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 116)
[function] pub fn normal(addr: u64, len: u32) -> Self {
kernel-drivers/usb/src/host/xhci/trb.rs (line 124)
[function] pub fn evaluate_ctx(slot: u8, ctx_phys: u64) -> Self {

```

```

kernel-drivers/usb/src/host/xhci/trb.rs (line 133)
[function] pub fn pack_setup(bm_request: u8, b_request: u8, value: u16,
kernel-drivers/usb/src/lib.rs (line 23)
[enum] pub enum UsbError {
kernel-drivers/usb/src/lib.rs (line 36)
[function] pub fn init() -> Result<(), UsbError> {
kernel-drivers/usb/src/lib.rs (line 57)
[function] pub fn poll() {
kernel-drivers/usb/src/lib.rs (line 64)
[function] pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> Option<R> {
kernel-drivers/usb/src/pci_glue.rs (line 8)
[struct] pub struct XhciPci {
kernel-drivers/usb/src/pci_glue.rs (line 13)
[function] pub fn find_xhci() -> Result<XhciPci, UsbError> {
kernel_Workspace/C_base/kc-abi/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {
kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {
kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {
kernel_Workspace/base/kw-C-abi/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {
kernel_Workspace/base/kw-base/src/api/dispatch.rs (line 21)
[function] pub fn dispatch(num: u64, a0: u64, a1: u64, a2: u64, a3: u64) -> i64 {
kernel_Workspace/base/kw-base/src/api/fs_ops.rs (line 4)
[function] pub fn open(path_ptr: *const u8, flags: u32) -> KResult<u32> {
kernel_Workspace/base/kw-base/src/api/fs_ops.rs (line 12)
[function] pub fn read(fd: u32, buf: *mut u8, len: usize) -> KResult<usize> {
kernel_Workspace/base/kw-base/src/api/fs_ops.rs (line 18)
[function] pub fn write(fd: u32, buf: *const u8, len: usize) -> KResult<usize> {
kernel_Workspace/base/kw-base/src/api/fs_ops.rs (line 24)
[function] pub fn close(fd: u32) -> KResult<()> {
kernel_Workspace/base/kw-base/src/api/memory.rs (line 8)
[function] pub fn mmap(addr: u64, len: usize, prot: u32, _flags: u32) -> KResult<u64>
{
kernel_Workspace/base/kw-base/src/api/memory.rs (line 20)
[function] pub fn munmap(addr: u64, len: usize) -> KResult<()> {
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 10)
[function] pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 15)
[function] pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) ->
KResult<usize> {
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 21)
[function] pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize>
{
kernel_Workspace/base/kw-base/src/api/process.rs (line 10)
[function] pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
kernel_Workspace/base/kw-base/src/api/process.rs (line 15)
[function] pub fn exit(code: i32) -> ! {

```

```

kernel_Workspace/base/kw-base/src/api/process.rs (line 20)
[function] pub fn yield_now() {
kernel_Workspace/base/kw-base/src/api/process.rs (line 24)
[function] pub fn get_pid() -> u32 {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 4)
[function] pub fn disable() {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 10)
[function] pub fn enable() {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 16)
[function] pub fn halt() {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 21)
[function] pub fn are_enabled() -> bool {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 32)
[struct] pub struct InterruptGuard {
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 37)
[function] pub fn disable() -> Self {
kernel_Workspace/base/kw-base/src/arch/registers.rs (line 3)
[struct] pub struct CpuContext {
kernel_Workspace/base/kw-base/src/arch/x86_64.rs (line 30)
[function] pub fn read_cr3() -> u64 {
kernel_Workspace/base/kw-base/src/arch/x86_64.rs (line 37)
[function] pub fn write_cr3(val: u64) {
kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 5)
[function] pub fn init_serial() {
kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 21)
[function] pub fn print_char(c: u8) {
kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 26)
[function] pub fn print_str(s: &str) {
kernel_Workspace/base/kw-base/src/boot/memory_detect.rs (line 5)
[struct] pub struct MemoryRegion {
kernel_Workspace/base/kw-base/src/boot/memory_detect.rs (line 11)
[function] pub fn extract_memory_map(info: &BootInfo) -> (usize, [MemoryRegion; 32])
{
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 6)
[struct] pub struct Multiboot2Info {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 12)
[struct] pub struct Multiboot2Tag {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 27)
[struct] pub struct MmapTag {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 34)
[struct] pub struct MmapEntry {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 41)
[struct] pub struct BootInfo {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 53)
[function] pub fn first_tag(&self) -> *const Multiboot2Tag {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 57)
[function] pub fn next_tag(tag: *const Multiboot2Tag) -> *const Multiboot2Tag {
kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 65)
[function] pub fn find_tag(&self, tag_type: u32) -> *const Multiboot2Tag {

```

```

kernel_Workspace/base/kw-base/src/boot/stack.rs (line 2)
[struct] pub struct BootStack {
kernel_Workspace/base/kw-base/src/boot/stack.rs (line 11)
[function] pub fn top(&self) -> u64 {
kernel_Workspace/base/kw-base/src/core/error.rs (line 5)
[enum] pub enum KernelError {
kernel_Workspace/base/kw-base/src/core/result.rs (line 5)
[trait] pub trait IntoKResult<T> {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 4)
[trait] pub trait BlockDevice {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 11)
[trait] pub trait CharDevice {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 16)
[trait] pub trait NetworkDevice {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 22)
[trait] pub trait FileSystem {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 31)
[struct] pub struct FileStat {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 42)
[trait] pub trait Scheduler {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 49)
[trait] pub trait InterruptController {
kernel_Workspace/base/kw-base/src/core/types.rs (line 5)
[struct] pub struct Pid(pub u32);
kernel_Workspace/base/kw-base/src/core/types.rs (line 9)
[struct] pub struct Tid(pub u32);
kernel_Workspace/base/kw-base/src/core/types.rs (line 13)
[struct] pub struct Fd(pub i32);
kernel_Workspace/base/kw-base/src/core/types.rs (line 29)
[enum] pub enum PrivilegeLevel {
kernel_Workspace/base/kw-base/src/core/types.rs (line 38)
[enum] pub enum Arch {
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 8)
[trait] pub trait BlockCipher {
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 17)
[struct] pub struct Aes256Cbc;
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 8)
[trait] pub trait Hasher {
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 13)
[struct] pub struct Sha256;
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 26)
[struct] pub struct Sha512;
kernel_Workspace/base/kw-base/src/crypto/keys.rs (line 4)
[struct] pub struct SymmetricKey {
kernel_Workspace/base/kw-base/src/crypto/keys.rs (line 9)
[function] pub fn generate(len: usize) -> KResult<Self> {
kernel_Workspace/base/kw-base/src/crypto/keys.rs (line 15)
[function] pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {

```

```

kernel_Workspace/base/kw-base/src/crypto/keys.rs (line 21)
[function] pub fn as_slice(&self) -> &[u8] {
kernel_Workspace/base/kw-base/src/crypto/random.rs (line 8)
[function] pub fn fill_random(buf: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/random.rs (line 16)
[function] pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/random.rs (line 24)
[function] pub fn next_u32() -> KResult<u32> {
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 5)
[function] pub fn secure_zero(ptr: *mut u8, len: usize) {
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 14)
[struct] pub struct SecureBuffer {
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 20)
[function] pub fn new(len: usize) -> Option<Self> {
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 27)
[function] pub fn as_slice(&self) -> &[u8] {
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 31)
[function] pub fn as_mut_slice(&mut self) -> &mut [u8] {
kernel_Workspace/base/kw-base/src/drivers/ps2.rs (line 7)
[struct] pub struct Ps2Controller;
kernel_Workspace/base/kw-base/src/drivers/ps2.rs (line 10)
[function] pub fn init() {
kernel_Workspace/base/kw-base/src/drivers/ps2.rs (line 25)
[function] pub fn read_scancode() -> Option<u8> {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 4)
[struct] pub struct Uart16550 {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 11)
[function] pub fn new(port: u16) -> Self {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 33)
[function] pub fn write_byte(&mut self, b: u8) {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 5)
[enum] pub enum FileType { Regular, Directory, CharDevice, BlockDevice, Pipe, Socket
}
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 7)
[trait] pub trait Inode {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 14)
[trait] pub trait File {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 19)
[trait] pub trait FileSystem {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 25)
[struct] pub struct Vfs;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 28)
[function] pub fn open(path: &str, flags: u32) -> KResult<Handle> {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 35)
[function] pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 41)
[function] pub fn write(fd: Handle, buf: &[u8]) -> KResult<usize> {
kernel_Workspace/base/kw-base/src/hal/acpi.rs (line 8)
[function] pub fn get_rsdp() -> Option<*const u8> {

```

```

kernel_Workspace/base/kw-base/src/hal/acpi.rs (line 13)
[function] pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 5)
[struct] pub struct DmaBuffer {
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 11)
[function] pub fn allocate(pages: usize) -> KResult<Self> {
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 21)
[function] pub fn phys_addr(&self) -> PhysAddr { self.frames.phys_addr() }
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 22)
[function] pub fn virt_addr(&self) -> VirtAddr { self.virt }
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 23)
[function] pub fn size(&self) -> usize { self.frames.byte_size() }
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 25)
[function] pub fn as_slice(&self) -> &[u8] {
kernel_Workspace/base/kw-base/src/hal/dma.rs (line 29)
[function] pub fn as_mut_slice(&mut self) -> &mut [u8] {
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 7)
[struct] pub struct PciAddress {
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 28)
[struct] pub struct PciDevice {
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 36)
[function] pub fn read_config32(addr: PciAddress, offset: u8) -> u32 {
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 43)
[function] pub fn write_config32(addr: PciAddress, offset: u8, value: u32) {
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 50)
[function] pub fn scan_bus() -> kstd_data::Vec<PciDevice> {
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 10)
[function] pub fn get_ticks() -> u64 {
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 14)
[function] pub fn get_frequency() -> u64 {
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 18)
[function] pub fn sleep_ms(ms: u32) {
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 22)
[function] pub fn set_one_shot(ns: u64) -> KResult<()> {
kernel_Workspace/base/kw-base/src/hal/usb.rs (line 9)
[function] pub fn init() -> KResult<()> {
kernel_Workspace/base/kw-base/src/hal/usb.rs (line 17)
[function] pub fn enumerate_devices() -> KResult<u32> {
kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 9)
[struct] pub struct IpcMessage {
kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 15)
[function] pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 20)
[function] pub fn recv() -> KResult<IpcMessage> {
kernel_Workspace/base/kw-base/src/ipc/shared_mem.rs (line 5)
[struct] pub struct SharedMemory {
kernel_Workspace/base/kw-base/src/ipc/shared_mem.rs (line 11)
[function] pub fn allocate(pages: usize) -> KResult<Self> {

```



```

kernel_Workspace/base/kw-base/src/ipc/shared_mem.rs (line 27)
[function] pub fn virt_addr(&self) -> VirtAddr { self.virt }
kernel_Workspace/base/kw-base/src/ipc/shared_mem.rs (line 28)
[function] pub fn size(&self) -> usize { self.frames.byte_size() }
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 4)
[struct] pub struct ContiguousFrames {
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 10)
[function] pub fn alloc(count: usize) -> KResult<Self> {
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 15)
[function] pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self>
{
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 20)
[function] pub fn phys_addr(&self) -> PhysAddr { self.addr }
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 21)
[function] pub fn frame_count(&self) -> usize { self.count }
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 22)
[function] pub fn byte_size(&self) -> usize { self.count * 4096 }
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 24)
[function] pub fn into_raw(self) -> (PhysAddr, usize) {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 4)
[struct] pub struct Frame {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 9)
[function] pub fn alloc() -> KResult<Self> {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 14)
[function] pub fn alloc_zeroed() -> KResult<Self> {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 19)
[function] pub fn phys_addr(&self) -> PhysAddr {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 23)
[function] pub fn into_raw(self) -> PhysAddr {
kernel_Workspace/base/kw-base/src/mem/heap.rs (line 4)
[function] pub fn alloc<T>() -> KResult<*mut T> {
kernel_Workspace/base/kw-base/src/mem/heap.rs (line 10)
[function] pub fn alloc_zeroed<T>() -> KResult<*mut T> {
kernel_Workspace/base/kw-base/src/mem/heap.rs (line 21)
[function] pub fn alloc_slice<T>(len: usize) -> KResult<*mut T> {
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 18)
[function] pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) ->
KResult<()> {
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 26)
[function] pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 34)
[function] pub fn flush_tlb_page(virt: VirtAddr) {
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 38)
[function] pub fn flush_tlb_all() {
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 10)
[struct] pub struct SlabCache {
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 18)
[function] pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {

```

```

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 28)
[function] pub fn alloc<T>(&self) -> KResult<*mut T> {
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 37)
[function] pub fn free<T>(&self, ptr: *mut T) {
kernel_Workspace/base/kw-base/src/sync/atomic.rs (line 5)
[function] pub fn compiler_fence() {
kernel_Workspace/base/kw-base/src/sync/atomic.rs (line 10)
[function] pub fn memory_barrier() {
kernel_Workspace/base/kw-base/src/sync/futex.rs (line 11)
[function] pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) ->
KResult<()> {
kernel_Workspace/base/kw-base/src/sync/futex.rs (line 16)
[function] pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 11)
[struct] pub struct Mutex<T> {
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 27)
[function] pub fn lock(&self) -> MutexGuard<'_, T> {
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 39)
[function] pub fn try_lock(&self) -> Option<MutexGuard<'_, T>> {
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 55)
[struct] pub struct MutexGuard<'a, T> {
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 5)
[struct] pub struct RwLock<T> {
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 19)
[function] pub fn read(&self) -> RwLockReadGuard<'_, T> {
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 33)
[function] pub fn write(&self) -> RwLockWriteGuard<'_, T> {
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 45)
[struct] pub struct RwLockReadGuard<'a, T> { lock: &'a RwLock<T> }
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 54)
[struct] pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
kernel_Workspace/base/kw-base/src/sync/semaphore.rs (line 3)
[struct] pub struct Semaphore {
kernel_Workspace/base/kw-base/src/sync/semaphore.rs (line 12)
[function] pub fn acquire(&self) {
kernel_Workspace/base/kw-base/src/sync/semaphore.rs (line 26)
[function] pub fn try_acquire(&self) -> bool {
kernel_Workspace/base/kw-base/src/sync/semaphore.rs (line 37)
[function] pub fn release(&self) {
kernel_Workspace/base/kw-base/src/task/context.rs (line 6)
[function] pub fn context_switch(old_ctx: *mut TaskContext, new_ctx: *mut
TaskContext);
kernel_Workspace/base/kw-base/src/task/process.rs (line 6)
[enum] pub enum ProcessState {
kernel_Workspace/base/kw-base/src/task/process.rs (line 14)
[struct] pub struct Process {
kernel_Workspace/base/kw-base/src/task/process.rs (line 22)
[function] pub fn new(pid: Pid, world_id: WorldId) -> KResult<Self> {

```

```

kernel_Workspace/base/kw-base/src/task/process.rs (line 32)
[function] pub fn transition(&mut self, new_state: ProcessState) -> KResult<()> {
kernel_Workspace/base/kw-base/src/task/queue.rs (line 5)
[struct] pub struct WaitQueue {
kernel_Workspace/base/kw-base/src/task/queue.rs (line 18)
[function] pub fn init(&mut self) {
kernel_Workspace/base/kw-base/src/task/queue.rs (line 22)
[function] pub fn sleep(&self, thread: &mut Thread) {
kernel_Workspace/base/kw-base/src/task/queue.rs (line 29)
[function] pub fn wake_one(&self) {
kernel_Workspace/base/kw-base/src/task/queue.rs (line 36)
[function] pub fn wake_all(&self) {
kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 4)
[function] pub fn yield_now() {
kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 10)
[function] pub fn halt_cpu() {
kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 17)
[function] pub fn set_current_priority(prio: i32) -> KResult<()> {
kernel_Workspace/base/kw-base/src/task/thread.rs (line 5)
[enum] pub enum ThreadState {
kernel_Workspace/base/kw-base/src/task/thread.rs (line 12)
[struct] pub struct Thread {
kernel_Workspace/base/kw-base/src/task/thread.rs (line 20)
[function] pub fn new(tid: Tid, entry_point: u64, stack_top: u64) -> Self {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 3)
[struct] pub struct Bitmap {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 10)
[function] pub fn new_external(ptr: *mut u64, bits: usize) -> Self {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 14)
[function] pub fn words_needed(bits: usize) -> usize {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 28)
[function] pub fn set(&mut self, bit: usize) {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 34)
[function] pub fn clear(&mut self, bit: usize) {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 40)
[function] pub fn test(&self, bit: usize) -> bool {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 45)
[function] pub fn find_first_zero(&self) -> Option<usize> {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 57)
[function] pub fn find_first_set(&self) -> Option<usize> {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 69)
[function] pub fn alloc_first_zero(&mut self) -> Option<usize> {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 75)
[function] pub fn count_set(&self) -> usize {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 83)
[function] pub fn count_free(&self) -> usize {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 36)
[struct] pub struct BTreeMap<K, V> {

```

```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 45)
[function] pub fn new() -> Self {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 49)
[function] pub fn insert(&mut self, key: K, val: V) {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 129)
[function] pub fn get(&self, key: &K) -> Option<&V> {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 146)
[function] pub fn len(&self) -> usize { self.len }
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 147)
[function] pub fn is_empty(&self) -> bool { self.len == 0 }
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 25)
[struct] pub struct HashMap<K, V> {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 35)
[function] pub fn new() -> Self {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 64)
[function] pub fn insert(&mut self, key: K, val: V) {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 87)
[function] pub fn get(&self, key: &K) -> Option<&V> {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 101)
[function] pub fn get_mut(&mut self, key: &K) -> Option<&mut V> {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 115)
[function] pub fn remove(&mut self, key: &K) -> Option<V> {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 133)
[function] pub fn len(&self) -> usize { self.len }
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 134)
[function] pub fn is_empty(&self) -> bool { self.len == 0 }
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 1)
[struct] pub struct RingBuffer<T, const N: usize> {
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 18)
[function] pub fn push(&mut self, val: T) -> bool {
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 26)
[function] pub fn pop(&mut self) -> Option<T> {
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 34)
[function] pub fn peek(&self) -> Option<&T> {
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 39)
[function] pub fn len(&self) -> usize { self.count }
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 40)
[function] pub fn is_empty(&self) -> bool { self.count == 0 }
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 41)
[function] pub fn is_full(&self) -> bool { self.count == N }
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 42)
[function] pub fn capacity(&self) -> usize { N }
kernel_Workspace/base/kw-lib/src/kernel/syscall.rs (line 7)
[function] pub fn syscall0(num: u64) -> i64 {
kernel_Workspace/base/kw-lib/src/kernel/syscall.rs (line 22)
[function] pub fn syscall3(num: u64, a1: u64, a2: u64, a3: u64) -> i64 {
kernel_Workspace/base/kw-lib/src/kernel/time.rs (line 3)
[function] pub fn ticks() -> u64 {

```

```

kernel_Workspace/base/kw-lib/src/kernel/time.rs (line 7)
[function] pub fn frequency() -> u64 {
kernel_Workspace/base/kw-lib/src/kernel/time.rs (line 11)
[function] pub fn sleep_ms(ms: u32) {
kernel_Workspace/base/kw-lib/src/kernel/time.rs (line 15)
[function] pub fn sleep_ticks(ticks: u64) {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 14)
[struct] pub struct SlabCache {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 26)
[function] pub fn new(obj_size: usize, align: usize) -> Self {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 69)
[function] pub fn alloc(&mut self) -> KResult<*mut u8> {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 99)
[function] pub fn free(&mut self, obj: *mut u8) {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 17)
[struct] pub struct BuddyAllocator {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 40)
[function] pub fn init(&mut self, base: PhysAddr, frames: usize, bitmap_ptr: *mut
u64) {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 100)
[function] pub fn alloc(&mut self, order: usize) -> KResult<PhysAddr> {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 133)
[function] pub fn free(&mut self, phys: PhysAddr, order: usize) {
kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 7)
[struct] pub struct AddressSpace {
kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 17)
[function] pub fn new(cr3: u64) -> Self {
kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 25)
[function] pub fn map_region(&mut self, start: VirtAddr, size: usize, flags: u32,
vma_type: VmaType) -> KResult<()> {
kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 53)
[function] pub fn find_vma(&self, addr: VirtAddr) -> Option<&Vma> {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 6)
[enum] pub enum VmaType {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 14)
[struct] pub struct Vma {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 24)
[function] pub fn new(start: VirtAddr, end: VirtAddr, flags: u32, vma_type: VmaType)
-> Self {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 35)
[function] pub fn size(&self) -> usize {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 39)
[function] pub fn contains(&self, addr: VirtAddr) -> bool {
kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 43)
[function] pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
kernel_Workspace/base/kw-lib/src/utils/string.rs (line 20)
[struct] pub struct StrBuffer<const N: usize> {
kernel_Workspace/base/kw-lib/src/utils/string.rs (line 30)
[function] pub fn push_str(&mut self, s: &str) -> bool {

```

```

kernel_Workspace/base/kw-lib/src/Utils/string.rs (line 38)
[function] pub fn as_str(&self) -> &str {
kernel_Workspace/base/kw-lib/src/Utils/string.rs (line 42)
[function] pub fn as_cstr_ptr(&mut self) -> *const u8 {
kernel_Workspace/base/kw-odin-abi/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {
kernel_Workspace/core/gluecore/src/bindings.rs (line 6)
[struct] pub struct CPhysRange {
kernel_Workspace/core/gluecore/src/boot.rs (line 9)
[struct] pub struct BootMemoryMap {
kernel_Workspace/core/gluecore/src/boot.rs (line 14)
[function] pub fn get_memory_map() -> Option<BootMemoryMap> {
kernel_Workspace/core/gluecore/src/boot.rs (line 24)
[struct] pub struct BootFramebuffer {
kernel_Workspace/core/gluecore/src/boot.rs (line 31)
[function] pub fn get_framebuffer() -> Option<BootFramebuffer> {
kernel_Workspace/core/gluecore/src/bridge/caps/check.rs (line 6)
[function] pub fn has_cap(world_id: u32, cap: CapId) -> bool {
kernel_Workspace/core/gluecore/src/bridge/caps/check.rs (line 10)
[function] pub fn request_cap(target_world: u32, cap: CapId) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/caps/check.rs (line 19)
[function] pub fn release_cap(world_id: u32, cap: CapId) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 7)
[function] pub fn alloc_frame() -> Result<PhysAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 17)
[function] pub fn alloc_zero_frame() -> Result<PhysAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 27)
[function] pub fn alloc_frames(count: usize) -> Result<PhysAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 37)
[function] pub fn alloc_frames_aligned(count: usize, align_frames: usize) ->
Result<PhysAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 47)
[function] pub fn free_frame(addr: PhysAddr) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/pmm.rs (line 56)
[function] pub fn free_frames(addr: PhysAddr, count: usize) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 14)
[function] pub fn alloc(bytes: usize, flags: u32) -> Result<VirtAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 24)
[function] pub fn alloc_aligned(bytes: usize, align: usize, flags: u32) ->
Result<VirtAddr, Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 34)
[function] pub fn free(addr: VirtAddr, bytes: usize) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 43)
[function] pub fn map_device(phys: PhysAddr, len: usize) -> Result<VirtAddr, Status>
{
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 53)
[function] pub fn unmap_device(addr: VirtAddr, len: usize) -> Result<(), Status> {
kernel_Workspace/core/gluecore/src/bridge/mm/vmm.rs (line 62)
[function] pub fn virt_to_phys(vaddr: VirtAddr) -> PhysAddr {

```

```

kernel_Workspace/core/gluecore/src/context.rs (line 11)
[function] pub fn read_cr3() -> PhysAddr {
kernel_Workspace/core/gluecore/src/context.rs (line 15)
[function] pub fn write_cr3(pml4: PhysAddr) {
kernel_Workspace/core/gluecore/src/context.rs (line 19)
[function] pub fn flush_tlb_all() {
kernel_Workspace/core/gluecore/src/context.rs (line 23)
[function] pub fn flush_page(addr: VirtAddr) {
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 8)
[function] pub fn pmm_alloc_frame(out_phys: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 9)
[function] pub fn pmm_alloc_zero_frame(out_phys: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 10)
[function] pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 11)
[function] pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize,
out_phys: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 12)
[function] pub fn pmm_free_frame(phys: u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 13)
[function] pub fn pmm_free_frames(phys: u64, count: usize) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 16)
[function] pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 17)
[function] pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32,
out_virt: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 18)
[function] pub fn vmm_free(virt: u64, bytes: usize) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 19)
[function] pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 20)
[function] pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 23)
[function] pub fn kmalloc(size: usize) -> *mut u8;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 24)
[function] pub fn kcalloc(size: usize) -> *mut u8;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 25)
[function] pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 26)
[function] pub fn kfree(ptr: *mut u8);
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 27)
[function] pub fn kalloc_pages(pages: usize) -> *mut u8;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 28)
[function] pub fn kfree_pages(ptr: *mut u8, pages: usize);
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 29)
[function] pub fn kvirt_to_phys(ptr: *const u8) -> u64;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 32)
[function] pub fn paging_read_cr3() -> u64;

```



```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 33)
[function] pub fn paging_write_cr3(pml4_phys: u64);
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 34)
[function] pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 35)
[function] pub fn paging_unmap_page(virt: u64) -> bool;
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 36)
[function] pub fn paging_flush_tlb_all();
kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 37)
[function] pub fn paging_flush_page(addr: u64);
kernel_Workspace/core/gluecore/src/interrupts.rs (line 4)
[function] pub fn disable() {
kernel_Workspace/core/gluecore/src/interrupts.rs (line 9)
[function] pub fn enable() {
kernel_Workspace/core/gluecore/src/interrupts.rs (line 14)
[function] pub fn halt() {
kernel_Workspace/core/gluecore/src/interrupts.rs (line 19)
[function] pub fn are_enabled() -> bool {
kernel_Workspace/core/gluecore/src/interrupts.rs (line 25)
[struct] pub struct InterruptGuard {
kernel_Workspace/core/gluecore/src/interrupts.rs (line 30)
[function] pub fn disable() -> Self {
kernel_Workspace/core/gluecore/src/syscalls.rs (line 7)
[function] pub fn invoke(num: u64, args: [u64; 4]) -> Result<u64, Status> {
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 13)
[struct] pub struct KernelHeap;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 45)
[function] pub fn alloc_zeroed(size: usize) -> Result<*mut u8, Status> {
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 50)
[function] pub fn alloc_array(n: usize, elem_size: usize) -> Result<*mut u8, Status>
{
kernel_Workspace/core/kstd_alloc/src/lib.rs (line 21)
[struct] pub struct KernelAllocator;
kernel_Workspace/core/kstd_alloc/src/omm.rs (line 5)
[trait] pub trait OomHandler: Fn() -> ! + Send + Sync {}
kernel_Workspace/core/kstd_alloc/src/omm.rs (line 10)
[function] pub fn set_handler(handler: &'static dyn OomHandler) {
kernel_Workspace/core/kstd_alloc/src/omm.rs (line 14)
[function] pub fn invoke() -> ! {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 9)
[function] pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 18)
[function] pub fn free_pages(addr: VirtAddr, order: usize) {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 23)
[function] pub fn alloc_single_page() -> Result<VirtAddr, Status> {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 27)
[function] pub fn free_single_page(addr: VirtAddr) {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 31)
[function] pub fn pages_for_size(size: usize) -> usize {

```

```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 11)
[struct] pub struct SlabCache {
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 19)
[function] pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self,
Status> {
kernel_Workspace/core/kstd_alloc/src/traits.rs (line 3)
[trait] pub trait Allocator {
kernel_Workspace/core/kstd_alloc/src/traits.rs (line 19)
[struct] pub struct Layout {
kernel_Workspace/core/kstd_alloc/src/traits.rs (line 41)
[function] pub fn padded(&self) -> usize {
kernel_Workspace/core/kstd_base/src/errors.rs (line 3)
[enum] pub enum Status {
kernel_Workspace/core/kstd_base/src/intrinsics.rs (line 12)
[function] pub fn memory_barrier() {
kernel_Workspace/core/kstd_base/src/markers.rs (line 3)
[struct] pub struct SendSync<T>(PhantomData<T>);
kernel_Workspace/core/kstd_base/src/markers.rs (line 7)
[struct] pub struct Immutable<T>(PhantomData<T>);
kernel_Workspace/core/kstd_base/src/markers.rs (line 14)
[struct] pub struct Unpin<T>(PhantomData<T>);
kernel_Workspace/core/kstd_base/src/primitives.rs (line 7)
[enum] pub enum Ring {
kernel_Workspace/core/kstd_base/src/primitives.rs (line 15)
[struct] pub struct ProtFlags(pub u32);
kernel_Workspace/core/kstd_base/src/primitives.rs (line 46)
[struct] pub struct MapFlags(pub u32);
kernel_Workspace/core/kstd_base/src/primitives.rs (line 58)
[struct] pub struct IrqFlags(pub u32);
kernel_Workspace/core/kstd_base/src/trails.rs (line 3)
[trait] pub trait PageAlign {
kernel_Workspace/core/kstd_base/src/trails.rs (line 43)
[trait] pub trait IoPort {
kernel_Workspace/core/kstd_base/src/trails.rs (line 52)
[trait] pub trait MmioRegion {
kernel_Workspace/core/kstd_base/src/types.rs (line 3)
[struct] pub struct PhysAddr(pub u64);
kernel_Workspace/core/kstd_base/src/types.rs (line 7)
[struct] pub struct VirtAddr(pub u64);
kernel_Workspace/core/kstd_base/src/types.rs (line 11)
[struct] pub struct Handle(pub u32);
kernel_Workspace/core/kstd_base/src/types.rs (line 30)
[struct] pub struct CapId(pub u64);
kernel_Workspace/core/kstd_base/src/types.rs (line 34)
[struct] pub struct WorldId(pub u32);
kernel_Workspace/core/kstd_base/src/types.rs (line 38)
[struct] pub struct TaskId(pub u64);
kernel_Workspace/core/kstd_base/src/types.rs (line 42)
[struct] pub struct Size(pub usize);

```

```

kernel_Workspace/core/kstd_core/src/panic.rs (line 11)
[function] pub fn panic(info: &PanicInfo) -> ! {
kernel_Workspace/core/kstd_core/src/sync.rs (line 5)
[struct] pub struct SpinLock<T> {
kernel_Workspace/core/kstd_core/src/sync.rs (line 21)
[function] pub fn lock(&self) -> SpinLockGuard<'_, T> {
kernel_Workspace/core/kstd_core/src/sync.rs (line 30)
[function] pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
kernel_Workspace/core/kstd_core/src/sync.rs (line 39)
[struct] pub struct SpinLockGuard<'a, T> {
kernel_Workspace/core/kstd_core/src/task.rs (line 10)
[struct] pub struct Task {
kernel_Workspace/core/kstd_core/src/task.rs (line 16)
[function] pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
kernel_Workspace/core/kstd_core/src/task.rs (line 25)
[function] pub fn destroy(&self) -> Result<(), Status> {
kernel_Workspace/core/kstd_core/src/task.rs (line 33)
[function] pub fn current() -> TaskId {
kernel_Workspace/core/kstd_core/src/task.rs (line 37)
[function] pub fn yield_now() {
kernel_Workspace/core/kstd_core/src/thread.rs (line 9)
[struct] pub struct Thread {
kernel_Workspace/core/kstd_core/src/thread.rs (line 15)
[function] pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) ->
Result<Self, Status> {
kernel_Workspace/core/kstd_core/src/thread.rs (line 24)
[function] pub fn exit() -> ! {
kernel_Workspace/core/kstd_core/src/thread.rs (line 28)
[function] pub fn current_id() -> u64 {
kernel_Workspace/core/kstd_core/src/time.rs (line 6)
[function] pub fn kernel_tick() -> u64 {
kernel_Workspace/core/kstd_core/src/time.rs (line 11)
[function] pub fn rdtsc() -> u64 {
kernel_Workspace/core/kstd_data/src/list.rs (line 4)
[struct] pub struct ListHead {
kernel_Workspace/core/kstd_data/src/list.rs (line 14)
[function] pub fn init(&mut self) {
kernel_Workspace/core/kstd_data/src/list.rs (line 19)
[function] pub fn add(&mut self, new_node: *mut ListHead) {
kernel_Workspace/core/kstd_data/src/list.rs (line 29)
[function] pub fn add_tail(&mut self, new_node: *mut ListHead) {
kernel_Workspace/core/kstd_data/src/list.rs (line 39)
[function] pub fn del(&mut self) {
kernel_Workspace/core/kstd_data/src/list.rs (line 49)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/core/kstd_data/src/map.rs (line 17)
[struct] pub struct HashMap<K, V> {
kernel_Workspace/core/kstd_data/src/map.rs (line 23)
[function] pub fn new() -> Self {

```

```

kernel_Workspace/core/kstd_data/src/map.rs (line 35)
[function] pub fn insert(&mut self, key: K, val: V) {
kernel_Workspace/core/kstd_data/src/map.rs (line 59)
[function] pub fn get(&self, key: &K) -> Option<&V> {
kernel_Workspace/core/kstd_data/src/map.rs (line 73)
[function] pub fn remove(&mut self, key: &K) -> Option<V> {
kernel_Workspace/core/kstd_data/src/map.rs (line 107)
[function] pub fn len(&self) -> usize { self.len }
kernel_Workspace/core/kstd_data/src/map.rs (line 108)
[function] pub fn is_empty(&self) -> bool { self.len == 0 }
kernel_Workspace/core/kstd_data/src/string.rs (line 4)
[struct] pub struct String { vec: Vec<u8> }
kernel_Workspace/core/kstd_data/src/string.rs (line 9)
[function] pub fn from_str(s: &str) -> Self {
kernel_Workspace/core/kstd_data/src/string.rs (line 15)
[function] pub fn push_str(&mut self, s: &str) {
kernel_Workspace/core/kstd_data/src/string.rs (line 19)
[function] pub fn as_str(&self) -> &str {
kernel_Workspace/core/kstd_data/src/string.rs (line 23)
[function] pub fn len(&self) -> usize { self.vec.len() }
kernel_Workspace/core/kstd_data/src/string.rs (line 24)
[function] pub fn is_empty(&self) -> bool { self.vec.is_empty() }
kernel_Workspace/core/kstd_data/src/tree.rs (line 9)
[struct] pub struct TreeMap<K, V> {
kernel_Workspace/core/kstd_data/src/tree.rs (line 16)
[function] pub fn new() -> Self {
kernel_Workspace/core/kstd_data/src/tree.rs (line 104)
[function] pub fn insert(&mut self, key: K, val: V) {
kernel_Workspace/core/kstd_data/src/tree.rs (line 108)
[function] pub fn get(&self, key: &K) -> Option<&V> {
kernel_Workspace/core/kstd_data/src/tree.rs (line 119)
[function] pub fn len(&self) -> usize { self.nodes.len() - self.free_list.len() }
kernel_Workspace/core/kstd_data/src/tree.rs (line 120)
[function] pub fn is_empty(&self) -> bool { self.root == 0 }
kernel_Workspace/core/kstd_data/src/vec.rs (line 4)
[struct] pub struct Vec<T> {
kernel_Workspace/core/kstd_data/src/vec.rs (line 18)
[function] pub fn with_capacity(cap: usize) -> Self {
kernel_Workspace/core/kstd_data/src/vec.rs (line 40)
[function] pub fn push(&mut self, val: T) {
kernel_Workspace/core/kstd_data/src/vec.rs (line 49)
[function] pub fn pop(&mut self) -> Option<T> {
kernel_Workspace/core/kstd_data/src/vec.rs (line 57)
[function] pub fn len(&self) -> usize { self.len }
kernel_Workspace/core/kstd_data/src/vec.rs (line 58)
[function] pub fn is_empty(&self) -> bool { self.len == 0 }
kernel_Workspace/core/kstd_data/src/vec.rs (line 59)
[function] pub fn capacity(&self) -> usize { self.cap }

```

```

kernel_Workspace/core/kstd_data/src/vec.rs (line 61)
[function] pub fn as_slice(&self) -> &[T] {
kernel_Workspace/core/kstd_data/src/vec.rs (line 65)
[function] pub fn as_mut_slice(&mut self) -> &mut [T] {
kernel_Workspace/core/kstd_data/src/vec.rs (line 71)
[function] pub fn extend_from_slice(&mut self, slice: &[T]) {
kernel_Workspace/core/kstd_io/src/buffer.rs (line 1)
[struct] pub struct RingBuffer<const N: usize> {
kernel_Workspace/core/kstd_io/src/buffer.rs (line 12)
[function] pub fn push(&mut self, b: u8) -> bool {
kernel_Workspace/core/kstd_io/src/buffer.rs (line 20)
[function] pub fn pop(&mut self) -> Option<u8> {
kernel_Workspace/core/kstd_io/src/buffer.rs (line 27)
[function] pub fn is_empty(&self) -> bool { self.head == self.tail }
kernel_Workspace/core/kstd_io/src/buffer.rs (line 28)
[function] pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
kernel_Workspace/core/kstd_io/src/buffer.rs (line 29)
[function] pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
kernel_Workspace/core/kstd_io/src/console.rs (line 8)
[struct] pub struct Serial;
kernel_Workspace/core/kstd_io/src/console.rs (line 24)
[struct] pub struct VgaConsole;
kernel_Workspace/core/kstd_io/src/input.rs (line 7)
[function] pub fn raw_scancode() -> Option<u32> {
kernel_Workspace/core/kstd_io/src/input.rs (line 12)
[function] pub fn cooked_char() -> Option<u8> {
kernel_Workspace/core/kstd_io/src/input.rs (line 17)
[function] pub fn buffered_byte() -> Option<u8> {
kernel_Workspace/core/kstd_io/src/input.rs (line 22)
[struct] pub struct RawInput;
kernel_Workspace/core/kstd_io/src/input.rs (line 29)
[struct] pub struct CookedInput;
kernel_Workspace/core/kstd_io/src/traits.rs (line 1)
[trait] pub trait Write {
kernel_Workspace/core/kstd_io/src/traits.rs (line 22)
[trait] pub trait Read {
kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 14)
[enum] pub enum PartitionKind {
kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 24)
[enum] pub enum AbiFlavor {
kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 32)
[struct] pub struct SandboxRules {
kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 53)
[struct] pub struct OsWorkspace {
kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 81)
[function] pub fn check_memory_access(&self, addr: VirtAddr, size: usize) -> bool {
kernel_Workspace/core/linux_abi/src/caps.rs (line 15)
[function] pub fn requires_native_cap(linux_cap: u32) -> Option<kstd_base::CapId> {

```

```

kernel_Workspace/core/linux_abi/src/errcodes.rs (line 25)
[function] pub fn status_to_errno(s: Status) -> u32 {
kernel_Workspace/core/linux_abi/src/errcodes.rs (line 44)
[function] pub fn errno_to_status(e: u32) -> Status {
kernel_Workspace/core/linux_abi/src/ipc.rs (line 4)
[struct] pub struct IpcPerm {
kernel_Workspace/core/linux_abi/src/ipc.rs (line 16)
[struct] pub struct ShmidDs {
kernel_Workspace/core/linux_abi/src/sysnums.rs (line 25)
[function] pub fn map_syscall_to_native_op(sys_num: u64) -> Option<u32> {
kernel_Workspace/core/linux_abi/src/uctx.rs (line 4)
[struct] pub struct UserContext {
kernel_Workspace/core/linux_abi/src/uctx.rs (line 21)
[function] pub fn syscall_args(&self) -> (u64, u64, u64, u64, u64, u64) {
kernel_Workspace/core/linux_abi/src/uctx.rs (line 27)
[function] pub fn set_return(&mut self, val: u64) {
kernel_Workspace/core/linux_abi_com/src/msgpass.rs (line 5)
[struct] pub struct LinuxMsgBuf {
kernel_Workspace/core/linux_abi_com/src/msgpass.rs (line 11)
[struct] pub struct PosixMqShim {
kernel_Workspace/core/linux_abi_com/src/msgpass.rs (line 17)
[function] pub fn send_msg(&self, _mtype: u64, _payload: &[u8]) -> Result<(),
Status> {
kernel_Workspace/core/linux_abi_com/src/msgpass.rs (line 23)
[function] pub fn recv_msg(&self, _out: &mut [u8]) -> Result<u64, Status> {
kernel_Workspace/core/linux_abi_com/src/sockets.rs (line 15)
[struct] pub struct SockAddrIn {
kernel_Workspace/core/linux_abi_com/src/sockets.rs (line 24)
[struct] pub struct SockAddrIn6 {
kernel_Workspace/core/linux_abi_com/src/sockets.rs (line 34)
[function] pub fn map_socket_type(family: u16, sock_type: u16) -> Option<u8> {
kernel_Workspace/core/linux_abi_driverspace/src/devmodel.rs (line 4)
[struct] pub struct LinuxDevice {
kernel_Workspace/core/linux_abi_driverspace/src/devmodel.rs (line 10)
[trait] pub trait LinuxDriver {
kernel_Workspace/core/linux_abi_driverspace/src/devmodel.rs (line 17)
[function] pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(),
Status> {
kernel_Workspace/core/linux_abi_driverspace/src/dma.rs (line 4)
[enum] pub enum DmaDirection {
kernel_Workspace/core/linux_abi_driverspace/src/dma.rs (line 12)
[struct] pub struct DmaMapping {
kernel_Workspace/core/linux_abi_driverspace/src/dma.rs (line 20)
[function] pub fn map_single(_cpu_addr: VirtAddr, _size: Size, _dir: DmaDirection)
-> Result<PhysAddr, Status> {
kernel_Workspace/core/linux_abi_driverspace/src/dma.rs (line 26)
[function] pub fn unmap_single(_phys: PhysAddr, _size: Size, _dir: DmaDirection) {
kernel_Workspace/core/windows-com/src/lib.rs (line 2)
[function] pub fn add(left: u64, right: u64) -> u64 {

```

```

kernel_Workspace/kernel-bin/src/lib.rs (line 13)
[function] pub fn announce_boot() {
kernel_Workspace/kernel-bin/src/lib.rs (line 36)
[function] pub fn kernel_ipc_loop() -> ! {
kernel_Workspace/kernel/src/arch/mod.rs (line 13)
[function] pub fn init() {
kernel_Workspace/kernel/src/arch/mod.rs (line 18)
[function] pub fn hlt_loop() -> ! {
kernel_Workspace/kernel/src/arch/mod.rs (line 23)
[function] pub fn now() -> u64 {
kernel_Workspace/kernel/src/arch/mod.rs (line 28)
[function] pub fn current_cpu() -> usize {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 20)
[function] pub fn heap_init() {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 27)
[struct] pub struct KernelHeap;
kernel_Workspace/kernel/src/arch/riscv64.rs (line 51)
[function] pub fn now() -> u64 {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 66)
[function] pub fn init() {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 74)
[function] pub fn current_cpu() -> usize {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 78)
[function] pub fn hlt_loop() -> ! {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 86)
[function] pub fn early_uart(s: &str) {
kernel_Workspace/kernel/src/arch/x86_64.rs (line 9)
[function] pub fn init() {
kernel_Workspace/kernel/src/arch/x86_64.rs (line 17)
[function] pub fn hlt_loop() -> ! {
kernel_Workspace/kernel/src/arch/x86_64.rs (line 23)
[function] pub fn current_cpu() -> usize {
kernel_Workspace/kernel/src/arch/x86_64.rs (line 35)
[function] pub fn now() -> u64 {
kernel_Workspace/kernel/src/audio/jack.rs (line 6)
[function] pub fn init(base: u32) -> bool {
kernel_Workspace/kernel/src/audio/jack.rs (line 10)
[function] pub fn poll_jack() -> bool {
kernel_Workspace/kernel/src/audio/jack.rs (line 23)
[function] pub fn query() -> u32 {
kernel_Workspace/kernel/src/audio/jack.rs (line 29)
[function] pub fn set_amp(on: bool) {
kernel_Workspace/kernel/src/audio/jack.rs (line 34)
[function] pub fn play_phys(phys: u64, len: u32) -> bool {
kernel_Workspace/kernel/src/audio/jack.rs (line 38)
[function] pub fn stop() {
kernel_Workspace/kernel/src/battery/aut.rs (line 4)
[function] pub fn authorize(ring: u8, op: u8) -> bool {

```



```

kernel_Workspace/kernel/src/battery/bridge.rs (line 9)
[function] pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
kernel_Workspace/kernel/src/battery/init.rs (line 5)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/bluetooth/aut.rs (line 8)
[function] pub fn authorize(ring: u8, op: u32) -> bool {
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 20)
[function] pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
kernel_Workspace/kernel/src/bluetooth/init.rs (line 6)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/bluetooth/init.rs (line 10)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/camera/aut.rs (line 6)
[function] pub fn authorize(ring: u8, op: u8) -> bool {
kernel_Workspace/kernel/src/camera/bridge.rs (line 15)
[function] pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
kernel_Workspace/kernel/src/camera/init.rs (line 5)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/caps/audit.rs (line 6)
[enum] pub enum EventKind {
kernel_Workspace/kernel/src/caps/audit.rs (line 16)
[struct] pub struct AuditEvent {
kernel_Workspace/kernel/src/caps/audit.rs (line 41)
[function] pub fn init_audit_log() -> Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/audit.rs (line 73)
[function] pub fn log_check(world: u32, cap: Capability, ok: bool) {
kernel_Workspace/kernel/src/caps/audit.rs (line 78)
[function] pub fn log_grant(granter: u32, target: u32, cap: Capability, ok: bool) {
kernel_Workspace/kernel/src/caps/audit.rs (line 84)
[function] pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
kernel_Workspace/kernel/src/caps/audit.rs (line 90)
[function] pub fn log_register(world: u32) {
kernel_Workspace/kernel/src/caps/audit.rs (line 94)
[function] pub fn log_unregister(world: u32) {
kernel_Workspace/kernel/src/caps/audit.rs (line 98)
[function] pub fn count() -> usize {
kernel_Workspace/kernel/src/caps/audit.rs (line 102)
[function] pub fn recent(n: usize) -> Vec<AuditEvent> {
kernel_Workspace/kernel/src/caps/audit.rs (line 119)
[function] pub fn by_world(world: u32, limit: usize) -> Vec<AuditEvent> {
kernel_Workspace/kernel/src/caps/audit.rs (line 126)
[function] pub fn by_kind(kind: EventKind, limit: usize) -> Vec<AuditEvent> {
kernel_Workspace/kernel/src/caps/audit.rs (line 133)
[function] pub fn deny_count() -> usize {
kernel_Workspace/kernel/src/caps/check.rs (line 10)
[function] pub fn kernel_world_id() -> u32 {
kernel_Workspace/kernel/src/caps/check.rs (line 14)
[function] pub fn set_kernel_world_id(id: u32) {

```

```

kernel_Workspace/kernel/src/caps/check.rs (line 18)
[function] pub fn current_world_id_pub() -> u32 {
kernel_Workspace/kernel/src/caps/check.rs (line 22)
[function] pub fn set_current_world(world_id: u32) {
kernel_Workspace/kernel/src/caps/check.rs (line 30)
[function] pub fn has_cap(cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/check.rs (line 35)
[function] pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/check.rs (line 39)
[function] pub fn require_cap(cap: Capability) -> CapResult<()> {
kernel_Workspace/kernel/src/caps/check.rs (line 54)
[function] pub fn require_world_cap(world_id: u32, cap: Capability) -> CapResult<()>
{
kernel_Workspace/kernel/src/caps/check.rs (line 67)
[function] pub fn require_caps(caps: &[Capability]) -> CapResult<()> {
kernel_Workspace/kernel/src/caps/check.rs (line 74)
[function] pub fn with_cap<T, F>(cap: Capability, f: F) -> CapResult<T>
kernel_Workspace/kernel/src/caps/check.rs (line 82)
[function] pub fn with_caps<T, F>(caps: &[Capability], f: F) -> CapResult<T>
kernel_Workspace/kernel/src/caps/check.rs (line 104)
[function] pub fn assert_cap(cap: Capability) {
kernel_Workspace/kernel/src/caps/check.rs (line 111)
[function] pub fn fast_check(cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/check.rs (line 115)
[function] pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
kernel_Workspace/kernel/src/caps/check.rs (line 126)
[struct] pub struct TemporaryCaps {
kernel_Workspace/kernel/src/caps/check.rs (line 132)
[function] pub fn enter(extra: CapabilitySet) -> CapResult<Self> {
kernel_Workspace/kernel/src/caps/defaults.rs (line 6)
[function] pub fn default_for_ring(ring: u8) -> CapabilitySet {
kernel_Workspace/kernel/src/caps/defaults.rs (line 15)
[function] pub fn install_defaults() -> Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/defaults.rs (line 22)
[function] pub fn register_for_ring(ring: u8, parent: Option<u32>) -> Result<u32,
&'static str> {
kernel_Workspace/kernel/src/caps/grant.rs (line 18)
[function] pub fn grant_cap(granter: u32, target: u32, cap: Capability) ->
CapResult<()> {
kernel_Workspace/kernel/src/caps/grant.rs (line 31)
[function] pub fn delegate_caps(granter: u32, target: u32, caps: CapabilitySet) ->
CapResult<()> {
kernel_Workspace/kernel/src/caps/grant.rs (line 38)
[function] pub fn inherit_set(parent_world: u32) -> CapabilitySet {
kernel_Workspace/kernel/src/caps/grant.rs (line 51)
[function] pub fn spawn_child(parent_world: u32) -> Result<u32, &'static str> {
kernel_Workspace/kernel/src/caps/grant.rs (line 78)
[function] pub fn grant_temporary(granter: u32, target: u32, cap: Capability,
kernel_Workspace/kernel/src/caps/grant.rs (line 97)
[function] pub fn prune_expired() {

```

```

kernel_Workspace/kernel/src/caps/grant.rs (line 110)
[function] pub fn is_temporary(world_id: u32, cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 5)
[function] pub fn parent(cap: Capability) -> Option<Capability> {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 47)
[function] pub fn implies(held: Capability, required: Capability) -> bool {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 62)
[function] pub fn set_implies(held: CapabilitySet, required: Capability) -> bool {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 71)
[function] pub fn expand_hierarchy(set: CapabilitySet) -> CapabilitySet {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 85)
[function] pub fn path_to_root(cap: Capability) -> Vec<Capability> {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 96)
[function] pub fn depth(cap: Capability) -> usize {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 106)
[function] pub fn subtree(root: Capability) -> Vec<Capability> {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 123)
[function] pub fn can_delegate(holder: CapabilitySet, candidate: Capability) -> bool
{
kernel_Workspace/kernel/src/caps/mod.rs (line 31)
[function] pub fn init() -> Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/mod.rs (line 38)
[function] pub fn snapshot() -> SystemSnapshot {
kernel_Workspace/kernel/src/caps/mod.rs (line 48)
[struct] pub struct SystemSnapshot {
kernel_Workspace/kernel/src/caps/mod.rs (line 55)
[function] pub fn self_test() -> crate::testing::TestResult {
kernel_Workspace/kernel/src/caps/policy.rs (line 9)
[function] pub fn set_hook(h: PolicyHook) {
kernel_Workspace/kernel/src/caps/policy.rs (line 23)
[function] pub fn enforce(world: u32, cap: Capability) -> CapResult<()> {
kernel_Workspace/kernel/src/caps/policy.rs (line 35)
[function] pub fn enforce_self(cap: Capability) -> CapResult<()> {
kernel_Workspace/kernel/src/caps/policy.rs (line 40)
[function] pub fn allowed(world: u32, cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/revoke.rs (line 7)
[function] pub fn revoke_from_world(world_id: u32, cap: Capability) -> CapResult<()>
{
kernel_Workspace/kernel/src/caps/revoke.rs (line 15)
[function] pub fn revoke_subtree(world_id: u32, cap: Capability) -> CapResult<()> {
kernel_Workspace/kernel/src/caps/revoke.rs (line 23)
[function] pub fn revoke_global(cap: Capability) {
kernel_Workspace/kernel/src/caps/revoke.rs (line 28)
[function] pub fn restore_global(cap: Capability) {
kernel_Workspace/kernel/src/caps/revoke.rs (line 32)
[function] pub fn revoked_list() -> Vec<Capability> {
kernel_Workspace/kernel/src/caps/revoke.rs (line 43)
[function] pub fn is_globally_revoked(cap: Capability) -> bool {

```

```

kernel_Workspace/kernel/src/caps/revoke.rs (line 47)
[function] pub fn lockdown(world_id: u32) {
kernel_Workspace/kernel/src/caps/sets.rs (line 4)
[struct] pub struct CapSetBuilder {
kernel_Workspace/kernel/src/caps/sets.rs (line 9)
[function] pub fn new() -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 13)
[function] pub fn empty() -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 17)
[function] pub fn add(mut self, cap: Capability) -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 22)
[function] pub fn add_many(mut self, caps: &[Capability]) -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 29)
[function] pub fn add_category(mut self, cat: super::types::CapCategory) -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 38)
[function] pub fn remove(mut self, cap: Capability) -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 43)
[function] pub fn with_hierarchy(self) -> Self {
kernel_Workspace/kernel/src/caps/sets.rs (line 47)
[function] pub fn build(self) -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 55)
[function] pub fn minimal_user() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 67)
[function] pub fn standard_user() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 84)
[function] pub fn privileged_user() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 98)
[function] pub fn driver() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 111)
[function] pub fn kernel() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 115)
[function] pub fn sandbox() -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 125)
[function] pub fn validate_hierarchy(set: CapabilitySet) -> Result<(), &'static str>
{
kernel_Workspace/kernel/src/caps/sets.rs (line 141)
[function] pub fn effective(set: CapabilitySet) -> CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 145)
[function] pub fn effective_diff(a: CapabilitySet, b: CapabilitySet) ->
CapabilitySet {
kernel_Workspace/kernel/src/caps/sets.rs (line 151)
[function] pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset:
CapabilitySet) -> bool {
kernel_Workspace/kernel/src/caps/store.rs (line 9)
[struct] pub struct WorldCaps {
kernel_Workspace/kernel/src/caps/store.rs (line 28)
[function] pub fn init_store() -> Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 36)
[function] pub fn global_caps() -> CapabilitySet {

```

```

kernel_Workspace/kernel/src/caps/store.rs (line 41)
[function] pub fn register_world(parent_world: Option<u32>, initial: CapabilitySet)
-> Result<u32, &'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 82)
[function] pub fn get_world_caps(world_id: u32) -> Result<CapabilitySet, &'static
str> {
kernel_Workspace/kernel/src/caps/store.rs (line 88)
[function] pub fn set_world_caps(world_id: u32, new_caps: CapabilitySet) ->
Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 115)
[function] pub fn add_world_cap(world_id: u32, cap: Capability) -> Result<(),
&'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 120)
[function] pub fn remove_world_cap(world_id: u32, cap: Capability) -> Result<(),
&'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 138)
[function] pub fn unregister_world(world_id: u32) -> Result<(), &'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 153)
[function] pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/store.rs (line 163)
[function] pub fn world_count() -> usize {
kernel_Workspace/kernel/src/caps/store.rs (line 168)
[function] pub fn iter_worlds<F: FnMut(u32, CapabilitySet)>(mut f: F) {
kernel_Workspace/kernel/src/caps/store.rs (line 177)
[function] pub fn add_global_revoked(cap: Capability) {
kernel_Workspace/kernel/src/caps/store.rs (line 182)
[function] pub fn remove_global_revoked(cap: Capability) {
kernel_Workspace/kernel/src/caps/syscalls.rs (line 17)
[function] pub fn cap_syscall(num: u64, a0: u64, a1: u64, a2: u64) -> u64 {
kernel_Workspace/kernel/src/caps/types.rs (line 10)
[enum] pub enum Capability {
kernel_Workspace/kernel/src/caps/types.rs (line 99)
[function] pub fn iter_all() -> impl Iterator<Item = Capability> {
kernel_Workspace/kernel/src/caps/types.rs (line 112)
[function] pub fn category(self) -> CapCategory {
kernel_Workspace/kernel/src/caps/types.rs (line 127)
[enum] pub enum CapCategory {
kernel_Workspace/kernel/src/caps/types.rs (line 154)
[struct] pub struct CapabilitySet {
kernel_Workspace/kernel/src/caps/types.rs (line 215)
[function] pub fn iter(self) -> impl Iterator<Item = Capability> {
kernel_Workspace/kernel/src/caps/types.rs (line 234)
[struct] pub struct CapabilityError {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 4)
[struct] pub struct CpuEntry {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 9)
[struct] pub struct IoApic {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 15)
[struct] pub struct MadtInfo {

```

```

kernel_Workspace/kernel/src/cpu/acpi.rs (line 21)
[struct] pub struct FadtInfo {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 25)
[struct] pub struct Rsdp {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 57)
[function] pub fn find_rsdp(phys_offset: u64) -> Option<u64> {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 24)
[function] pub fn init(base_phys: u64) -> bool {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 46)
[function] pub fn is_x2apic() -> bool {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 50)
[function] pub fn read(reg: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 60)
[function] pub fn write(reg: u32, val: u32) {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 70)
[function] pub fn id() -> u32 {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 78)
[function] pub fn version() -> u32 {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 82)
[function] pub fn enable_bsp() {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 87)
[function] pub fn enable_ap() {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 93)
[function] pub fn eoi() {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 97)
[function] pub fn send_ipi(icr: u32, dest_apic_id: u32) {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 109)
[function] pub fn send_init_ipi() {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 116)
[function] pub fn send_startup_ipi(apic_id: u32, vector: u8) {
kernel_Workspace/kernel/src/cpu/mod.rs (line 18)
[function] pub fn init_riscv() {
kernel_Workspace/kernel/src/cpu/mod.rs (line 24)
[function] pub fn total_cpus() -> u32 {
kernel_Workspace/kernel/src/cpu/mod.rs (line 29)
[function] pub fn total_cpus() -> u32 {
kernel_Workspace/kernel/src/cpu/riscv.rs (line 7)
[function] pub fn init() {
kernel_Workspace/kernel/src/cpu/riscv.rs (line 12)
[function] pub fn current_hart() -> u64 {
kernel_Workspace/kernel/src/cpu/riscv.rs (line 17)
[function] pub fn poweroff() -> ! {
kernel_Workspace/kernel/src/cpu/riscv.rs (line 22)
[function] pub fn reboot() -> ! {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 639)
[function] pub fn get_switch_count() -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 643)
[function] pub fn get_fpu_switch_count() -> u64 {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 647)
[function] pub fn enable_lazy_fpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 651)
[function] pub fn disable_lazy_fpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 655)
[function] pub fn is_lazy_fpu_enabled() -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 692)
[function] pub fn satp_mode_name(satp: u64) -> &'static str {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 703)
[function] pub fn satp_asid(satp: u64) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 707)
[function] pub fn satp_ppn(satp: u64) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 731)
[function] pub fn get_switch_count() -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 735)
[function] pub fn get_fpu_switch_count() -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 739)
[function] pub fn enable_lazy_fpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 743)
[function] pub fn disable_lazy_fpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 747)
[function] pub fn is_lazy_fpu_enabled() -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 49)
[function] pub fn register_runqueue(cpu: u32, rq: *mut RunQueue) {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 55)
[function] pub fn register_apic_id(cpu: u32, apic_id: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 61)
[function] pub fn snapshot_registry() -> [*mut RunQueue; MAX_CPUS] {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 81)
[function] pub fn set_tsc_frequency(hz: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 85)
[function] pub fn tsc_frequency() -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 89)
[function] pub fn booted_cpu_count() -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 183)
[function] pub fn now_ns() -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 270)
[function] pub fn current_cpu_id() -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 338)
[struct] pub struct IdtEntry {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 353)
[function] pub fn set(&mut self, handler: usize, selector: u16, ist: u8, type_attr:
u8) {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 363)
[function] pub fn is_present(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 12)
[struct] pub struct SchedClassOps {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 336)
[function] pub fn class_of(class: SchedClass) -> &'static SchedClassOps {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 27)
[struct] pub struct Bitmap<const WORDS: usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 43)
[function] pub fn set(&mut self, bit: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 50)
[function] pub fn clear(&mut self, bit: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 57)
[function] pub fn toggle(&mut self, bit: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 64)
[function] pub fn test(&self, bit: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 68)
[function] pub fn set_range(&mut self, start: usize, end: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 77)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 81)
[function] pub fn is_full(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 85)
[function] pub fn weight(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 89)
[function] pub fn find_first_set(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 98)
[function] pub fn find_first_zero(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 111)
[function] pub fn find_next_set(&self, after: usize) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 126)
[function] pub fn and(&self, other: &Self) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 134)
[function] pub fn or(&self, other: &Self) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 142)
[function] pub fn xor(&self, other: &Self) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 150)
[function] pub fn andnot(&self, other: &Self) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 158)
[function] pub fn intersects(&self, other: &Self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 162)
[function] pub fn iter(&self) -> BitmapIter<'_> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 173)
[struct] pub struct BitmapIter<'a> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 200)
[struct] pub struct BitmapSlice<'a> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 206)
[function] pub fn new(words: &'a mut [u64], nbits: usize) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 211)
[function] pub fn capacity(&self) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 215)
[function] pub fn set(&mut self, bit: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 221)
[function] pub fn clear(&mut self, bit: usize) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 227)
[function] pub fn test(&self, bit: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 231)
[function] pub fn weight(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 235)
[function] pub fn find_first_zero(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 248)
[function] pub fn clear_all(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 263)
[struct] pub struct AtomicBitmap<const WORDS: usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 275)
[function] pub fn test(&self, bit: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 280)
[function] pub fn test_and_set(&self, bit: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 289)
[function] pub fn test_and_clear(&self, bit: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 297)
[function] pub fn weight(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 301)
[function] pub fn is_full(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 311)
[function] pub fn find_first_zero_and_set(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 17)
[struct] pub struct AtomicCpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 29)
[function] pub fn set(&self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 37)
[function] pub fn clear(&self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 45)
[function] pub fn test(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 51)
[function] pub fn snapshot(&self) -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 74)
[function] pub fn mark_possible(cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 79)
[function] pub fn mark_present(cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 84)
[function] pub fn mark_online(cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 90)
[function] pub fn mark_offline(cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 96)
[function] pub fn is_online(cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 101)
[function] pub fn online_mask() -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 106)
[function] pub fn active_mask() -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 111)
[function] pub fn set_topology(cpu: u32, pkg: u32, core: u32) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 119)
[function] pub fn sibling_mask(cpu: u32) -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 137)
[function] pub fn package_mask(cpu: u32) -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 44)
[struct] pub struct PList {
kernel_Workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 53)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 152)
[function] pub fn first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 167)
[function] pub fn last(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 62)
[struct] pub struct RbTree {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 4)
[struct] pub struct RtArray {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 20)
[function] pub fn set_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 27)
[function] pub fn clear_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 41)
[function] pub fn highest_prio(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 149)
[function] pub fn active_levels(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 15)
[struct] pub struct PidAllocator {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 31)
[function] pub fn alloc(&self) -> Option<TaskId> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 54)
[function] pub fn free(&self, pid: TaskId) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 64)
[function] pub fn reserve(&self, pid: TaskId) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 77)
[function] pub fn is_used(&self, pid: TaskId) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 85)
[function] pub fn count_allocated(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 94)
[struct] pub struct TaskTable {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 100)
[struct] pub struct RtFields {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 119)
[function] pub fn insert(&self, task: *mut TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 154)
[function] pub fn remove(&self, pid: TaskId) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 185)
[function] pub fn lookup_pid(&self, pid: TaskId) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 196)
[function] pub fn lookup_tgid(&self, tgid: TaskId) -> *mut TaskStruct {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 207)
[function] pub fn count(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 221)
[struct] pub struct TaskIterator<'a> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 279)
[function] pub fn new(table: &'a TaskTable) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 299)
[struct] pub struct ThreadGroupIterator {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 334)
[struct] pub struct ChildIterator {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 358)
[struct] pub struct DescendantIterator {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 386)
[struct] pub struct ProcessGroup {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 397)
[function] pub fn new(pgid: TaskId, leader: *mut TaskStruct) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 427)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 443)
[struct] pub struct Session {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 453)
[function] pub fn new(sid: TaskId, leader: *mut TaskStruct) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 473)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 477)
[function] pub fn set_foreground(&self, pgid: TaskId) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 481)
[function] pub fn get_foreground(&self) -> TaskId {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 488)
[struct] pub struct UidTaskCount {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 498)
[function] pub fn inc(&self, uid: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 503)
[function] pub fn dec(&self, uid: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 510)
[function] pub fn get(&self, uid: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 516)
[enum] pub enum TaskRegistryError {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 524)
[struct] pub struct TaskRegistry {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 585)
[function] pub fn find_by_pid(&self, pid: TaskId) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 589)
[function] pub fn find_by_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 685)
[function] pub fn iter(&self) -> TaskIterator {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 700)
[function] pub fn set_init_task(&self, task: *mut TaskStruct) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 704)
[function] pub fn stats(&self) -> (u32, u64, u64) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 713)
[struct] pub struct PidNamespace {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 22)
[function] pub fn alloc_pid() -> Option<TaskId> {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 26)
[function] pub fn free_pid(pid: TaskId) {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 34)
[function] pub fn reserve_pid(pid: TaskId) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 41)
[function] pub fn pid_in_use(pid: TaskId) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 219)
[function] pub fn sys_sched_get_priority_max(policy: SchedPolicy) -> i32 {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 227)
[function] pub fn sys_sched_get_priority_min(policy: SchedPolicy) -> i32 {
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 1)
[struct] pub struct Tracepoint {}
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 3)
[struct] pub struct TracePointId {}
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 5)
[struct] pub struct Tracepointkind{}
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 7)
[struct] pub struct tracepoints {
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 11)
[struct] pub struct TraceEvent{
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 15)
[enum] pub enum TraceData{
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 19)
[struct] pub struct ContextSwichTraceData{
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 23)
[struct] pub struct TraceContext{
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 27)
[struct] pub struct TracepointRegistry{
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 31)
[struct] pub struct RegisteredTracepoint{}
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 33)
[struct] pub struct TraceBuffer{}
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 35)
[struct] pub struct PerCpuTracebuffer{}
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 19)
[struct] pub struct EnqueueFlags: u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 30)
[struct] pub struct DequeueFlags: u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 240)
[function] pub fn insert(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct,
node: *mut TaskStruct, key_of: KeyOf) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 357)
[function] pub fn delete(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct,
z: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 457)
[struct] pub struct RqFair {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 478)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 482)
[function] pub fn leftmost(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 499)
[function] pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 510)
[function] pub fn dequeue(&mut self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 520)
[function] pub fn pick_first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 524)
[function] pub fn charge_exec(&mut self, task: *mut TaskStruct, delta_exec: u64,
now: u64) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 540)
[function] pub fn should_preempt(&self, current: *const TaskStruct, candidate:
*const TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 552)
[function] pub fn rb_invariants_ok(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 557)
[function] pub fn rb_count(&self) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 568)
[struct] pub struct RqDl {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 591)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 595)
[function] pub fn leftmost(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 599)
[function] pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool
{
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 618)
[function] pub fn enqueue(&mut self, task: *mut TaskStruct, now: u64, flags:
EnqueueFlags) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 639)
[function] pub fn dequeue(&mut self, task: *mut TaskStruct, removing_permanently:
bool) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 649)
[function] pub fn pick_first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 653)
[function] pub fn update_curr(&mut self, task: *mut TaskStruct, delta_exec: u64,
now: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 670)
[function] pub fn should_preempt(&self, current: *const TaskStruct, candidate:
*const TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 675)
[function] pub fn rb_invariants_ok(&self) -> bool {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 684)
[struct] pub struct RqRt {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 694)
[function] pub fn new() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 710)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 735)
[function] pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 769)
[function] pub fn dequeue(&mut self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 793)
[function] pub fn pick_first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 809)
[function] pub fn requeue(&mut self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 820)
[function] pub fn highest_priority(&self) -> i32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 828)
[struct] pub struct RqStop {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 833)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 837)
[function] pub fn enqueue(&mut self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 843)
[function] pub fn dequeue(&mut self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 850)
[function] pub fn pick_first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 857)
[struct] pub struct RunQueue {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 882)
[function] pub fn new(cpu: u32, idle_task: *mut TaskStruct) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 914)
[function] pub fn bind_idle_task(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 930)
[function] pub fn current(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 938)
[function] pub fn nr_running(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 942)
[function] pub fn is_idle(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1214)
[function] pub fn fair_root_for_test(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 21)
[function] pub fn account_switch() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 25)
[function] pub fn account_voluntary_switch(task: &mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 30)
[function] pub fn account_involuntary_switch(task: &mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 35)
[function] pub fn account_migration(task: &mut TaskStruct) {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 41)
[function] pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 48)
[function] pub fn account_system_time(task: &mut TaskStruct, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 55)
[function] pub fn account_idle_time(delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 59)
[function] pub fn account_iowait_time(delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 64)
[function] pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 65)
[function] pub fn total_voluntary_switches() -> u64 {
TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 66)
[function] pub fn total_involuntary_switches() -> u64 {
TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 67)
[function] pub fn total_migrations() -> u64 {
TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 68)
[function] pub fn total_user_time_ns() -> u64 {
TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 69)
[function] pub fn total_system_time_ns() -> u64 {
TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 70)
[function] pub fn total_idle_time_ns() -> u64 {
TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 71)
[function] pub fn total_iowait_time_ns() -> u64 {
TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 72)
[function] pub fn total_running_time_ns() -> u64 {
TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 77)
[struct] pub struct RqStats {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 126)
[function] pub fn record_switch(&mut self, voluntary: bool) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 143)
[function] pub fn record_migration(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 149)
[function] pub fn add_user_time(&mut self, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 157)
[function] pub fn add_system_time(&mut self, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 165)
[function] pub fn add_idle_time(&mut self, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 171)
[function] pub fn add_iowait_time(&mut self, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 177)
[function] pub fn set_nr_running(&mut self, nr: u32) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 182)
[function] pub fn set_nr_uninterruptible(&mut self, nr: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 187)
[function] pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 193)
[function] pub fn load_percent(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 204)
[function] pub fn load_avg_1min(&self) -> f32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 213)
[struct] pub struct ExecTimeHistogram {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 229)
[function] pub fn add_sample(&mut self, exec_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 251)
[function] pub fn average_ns(&self) -> f32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 259)
[function] pub fn total_samples(&self) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 273)
[struct] pub struct SystemStats {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 287)
[struct] pub struct CpuStatsSnapshot {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 298)
[function] pub fn collect_system_stats() -> SystemStats {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 44)
[struct] pub struct TaskFlags: u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 73)
[struct] pub struct AtomicTaskFlags(AtomicU32);
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 85)
[function] pub fn load(&self, order: Ordering) -> TaskFlags {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 90)
[function] pub fn store(&self, flags: TaskFlags, order: Ordering) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 95)
[function] pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 102)
[function] pub fn fetch_remove(&self, flags: TaskFlags) -> TaskFlags {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 108)
[function] pub fn contains(&self, flags: TaskFlags) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 113)
[function] pub fn test_and_set(&self, flags: TaskFlags) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 127)
[enum] pub enum TaskState {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 208)
[enum] pub enum SchedPolicy {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 230)
[enum] pub enum SchedClass {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 253)
[struct] pub struct CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 266)
[function] pub fn single(cpu: u32) -> Self {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 272)
[function] pub fn first_n(n: u32) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 281)
[function] pub fn set(&mut self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 288)
[function] pub fn clear(&mut self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 295)
[function] pub fn is_set(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 300)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 304)
[function] pub fn count(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 308)
[function] pub fn intersects(&self, other: &CpuMask) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 312)
[function] pub fn and(&self, other: &CpuMask) -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 320)
[function] pub fn or(&self, other: &CpuMask) -> CpuMask {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 328)
[function] pub fn first(&self) -> Option<u32> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 337)
[function] pub fn next_after(&self, after: u32) -> Option<u32> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 352)
[function] pub fn iter(&self) -> CpuMaskIter {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 363)
[struct] pub struct CpuMaskIter {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 406)
[function] pub fn nice_to_weight(nice: i8) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 411)
[function] pub fn nice_to_wmult(nice: i8) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 416)
[function] pub fn weight_to_nice(weight: u64) -> i8 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 431)
[function] pub fn calc_delta_fair(delta_exec: u64, weight: u64, inv_weight: u32) ->
u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 442)
[struct] pub struct InterruptFrame {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 452)
[struct] pub struct FxSaveArea {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 476)
[struct] pub struct CpuContext {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 548)
[function] pub fn xsave_area_size() -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 557)
[function] pub fn rb_parent(pc: usize) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 562)
[function] pub fn rb_color(pc: usize) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 567)
[function] pub fn rb_is_red(pc: usize) -> bool {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 572)
[function] pub fn rb_is_black(pc: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 577)
[function] pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) ->
usize {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 583)
[struct] pub struct LoadAvg {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 595)
[function] pub fn accumulate(&mut self, now: u64, delta_ns: u64, weight: u64,
running: bool) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 624)
[struct] pub struct SchedEntity {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 671)
[struct] pub struct RtSchedEntity {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 703)
[struct] pub struct PlistNode {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 711)
[function] pub fn init(&mut self, owner: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 731)
[struct] pub struct DlSchedEntity {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 762)
[struct] pub struct TaskStats {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 781)
[struct] pub struct Credentials {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 831)
[struct] pub struct RLimit {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 848)
[enum] pub enum RlimitResource {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 872)
[struct] pub struct SignalState {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 878)
[function] pub fn has_pending(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 882)
[function] pub fn raise(&mut self, signum: u8) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 888)
[function] pub fn clear(&mut self, signum: u8) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 898)
[struct] pub struct ListHead {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 908)
[function] pub fn init(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 914)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 949)
[function] pub fn is_linked(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 997)
[struct] pub struct SpinLock {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1012)
[function] pub fn lock(&self) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1024)
[function] pub fn unlock(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1028)
[function] pub fn try_lock(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1034)
[function] pub fn is_locked(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1038)
[function] pub fn lock_irqsave(&self) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1043)
[function] pub fn unlock_irqrestore(&self, flags: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1048)
[function] pub fn lock_guard(&self) -> SpinLockGuard<'_> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1053)
[function] pub fn lock_irqsave_guard(&self) -> SpinLockGuard<'_> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1060)
[struct] pub struct SpinLockGuard<'a> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1076)
[enum] pub enum TaskError {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1112)
[struct] pub struct TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1403)
[function] pub fn state(&self) -> TaskState {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1410)
[function] pub fn set_state(&self, new_state: TaskState) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1433)
[function] pub fn wake_up(&self) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1441)
[function] pub fn sleep(&self, interruptible: bool) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1446)
[function] pub fn is_kernel_thread(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1450)
[function] pub fn is_idle_task(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1454)
[function] pub fn is_zombie(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1458)
[function] pub fn is_runnable(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1462)
[function] pub fn needs_resched(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1466)
[function] pub fn set_need_resched(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1470)
[function] pub fn clear_need_resched(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1474)
[function] pub fn set_nice(&mut self, nice: i8) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1487)
[function] pub fn set_rt_priority(&mut self, rt_priority: u8) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1501)
[function] pub fn effective_prio(&self) -> i32 {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1505)
[function] pub fn set_affinity(&mut self, mask: CpuMask) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1517)
[function] pub fn can_run_on(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1522)
[function] pub fn rq_ptr(&self) -> *mut core::ffi::c_void {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1526)
[function] pub fn set_rq_ptr(&self, rq: *mut core::ffi::c_void) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1530)
[function] pub fn charge_cputime(&mut self, delta_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1549)
[function] pub fn record_voluntary_switch(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1553)
[function] pub fn record_involuntary_switch(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1557)
[function] pub fn record_migration(&mut self, new_cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1564)
[function] pub fn set_comm(&mut self, name: &str) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1571)
[function] pub fn comm_str(&self) -> &str {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1608)
[struct] pub struct ThreadInfoView<'a> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1618)
[function] pub fn thread_info(&mut self) -> ThreadInfoView<'_> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1641)
[function] pub fn fair_has_priority(a: &TaskStruct, b: &TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1645)
[function] pub fn deadline_has_priority(a: &TaskStruct, b: &TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1653)
[function] pub fn blank() -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1697)
[function] pub fn init_test_stub(&mut self, pid: TaskId, policy: SchedPolicy, nice:
i8) {
kernel_Workspace/kernel/src/cpu/scheduler/mod.rs (line 69)
[function] pub fn self_test() -> Result<&'static str, &'static str> {
kernel_Workspace/kernel/src/cpu/scheduler/mod.rs (line 111)
[function] pub fn current_cpu_id() -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 16)
[struct] pub struct CapacityState {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 54)
[function] pub fn compute_cost(&mut self, max_cap: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 64)
[function] pub fn is_valid(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 70)
[struct] pub struct PerformanceDomain {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 120)
[function] pub fn init(&mut self, id: u32, cpus: CpuMask) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 132)
[function] pub fn add_state(&mut self, state: CapacityState) -> bool {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 151)
[function] pub fn finalize(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 173)
[function] pub fn find_state_for_capacity(&self, target_cap: u32) ->
Option<&CapacityState> {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 186)
[function] pub fn find_state_for_frequency(&self, target_freq: u32) ->
Option<&CapacityState> {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 203)
[function] pub fn get_effective_capacity(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 207)
[function] pub fn update_thermal_pressure(&self, pressure: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 218)
[function] pub fn compute_power_at_state(&self, state_idx: usize, temp_millideg:
u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 237)
[function] pub fn update_temperature(&self, power_mw: u32, delta_time_ms: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 252)
[function] pub fn contains_cpu(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 256)
[function] pub fn get_active_state_idx(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 260)
[function] pub fn set_active_state_idx(&self, idx: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 268)
[struct] pub struct EnergyModel {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 298)
[function] pub fn init(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 307)
[function] pub fn register_domain(&mut self, pd: PerformanceDomain) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 329)
[function] pub fn get_pd_for_cpu(&self, cpu: u32) -> Option<&PerformanceDomain> {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 340)
[function] pub fn get_pd_for_cpu_mut(&mut self, cpu: u32) -> Option<&mut
PerformanceDomain> {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 351)
[function] pub fn compute_system_energy(&self, delta_time_ms: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 365)
[function] pub fn compute_task_energy_on_cpu(&self, task_util: u32, cpu: u32) -> u32
{
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 381)
[function] pub fn find_best_state_for_domain(&self, pd_idx: u32, target_util: u32)
-> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 404)
[function] pub fn invert_capacity_for_thermal(&mut self, cpu: u32, thermal_pressure:
u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 410)
[function] pub fn clear_thermal_pressure(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 416)
[function] pub fn get_max_capacity(&self) -> u32 {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 420)
[function] pub fn get_min_capacity(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 424)
[function] pub fn is_cpu_big(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 432)
[function] pub fn is_cpu_little(&self, cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 440)
[function] pub fn dump_state(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 484)
[function] pub fn em_build_synthetic_little_pd(cpus: CpuMask) -> PerformanceDomain {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 502)
[function] pub fn em_build_synthetic_big_pd(cpus: CpuMask) -> PerformanceDomain {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 521)
[function] pub fn em_build_synthetic_huge_pd(cpus: CpuMask) -> PerformanceDomain {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 28)
[struct] pub struct PidState {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 45)
[function] pub fn update(&mut self, measurement: i64) -> i64 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 54)
[function] pub fn reset(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 62)
[struct] pub struct ThermalZone {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 89)
[function] pub fn init(&mut self, pd_idx: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 99)
[function] pub fn update_temp(&self, temp_millideg: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 103)
[function] pub fn get_temp(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 107)
[function] pub fn get_level(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 111)
[function] pub fn evaluate_level(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 148)
[function] pub fn compute_pid_output(&mut self) -> i64 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 155)
[struct] pub struct PowerScheduler {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 186)
[function] pub fn init(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 199)
[function] pub fn register_zone(&mut self, pd_idx: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 209)
[function] pub fn get_zone_for_pd(&self, pd_idx: u32) -> Option<&ThermalZone> {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 218)
[function] pub fn get_zone_for_pd_mut(&mut self, pd_idx: u32) -> Option<&mut
ThermalZone> {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 227)
[function] pub fn update_tick(&mut self, current_time_ns: u64) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 258)
[function] pub fn apply_mitigation(&mut self) {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 308)
[function] pub fn get_max_allowed_capacity(&self, cpu: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 315)
[function] pub fn is_eas_enabled(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 319)
[function] pub fn enable_eas(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 323)
[function] pub fn disable_eas(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 327)
[function] pub fn is_emergency_shutdown(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 331)
[function] pub fn dump_thermal_state(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/power/placement.rs (line 14)
[struct] pub struct CpuSnapshot {
kernel_Workspace/kernel/src/cpu/scheduler/power/placement.rs (line 35)
[struct] pub struct DomainSnapshot {
kernel_Workspace/kernel/src/cpu/scheduler/power/placement.rs (line 54)
[struct] pub struct EnergyCtx {
kernel_Workspace/kernel/src/cpu/scheduler/power/placement.rs (line 77)
[struct] pub struct PlacementResult {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 17)
[enum] pub enum GroupType {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 28)
[struct] pub struct LoadCalculation {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 18)
[struct] pub struct LoadBalanceEnv {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 41)
[function] pub fn set_flag(&mut self, flag: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 45)
[function] pub fn clear_flag(&mut self, flag: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 49)
[function] pub fn has_flag(&self, flag: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 55)
[function] pub fn should_stop(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 59)
[function] pub fn record_attempt(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 63)
[function] pub fn record_failure(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 68)
[function] pub fn reset_for_next_iteration(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 8)
[enum] pub enum IpiType {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 10)
[enum] pub enum StopperState {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 19)
[struct] pub struct StopperWork {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 41)
[function] pub fn get_state(&self) -> StopperState {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 52)
[function] pub fn set_state(&self, new_state: StopperState) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 60)
[struct] pub struct CpuStopper {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 83)
[function] pub fn init(&mut self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 88)
[function] pub fn queue_work(&self, func: StopperFn, arg: *mut core::ffi::c_void) ->
bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 120)
[function] pub fn execute_work(&self) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 146)
[function] pub fn wait_for_completion(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 166)
[struct] pub struct StopperRegistry {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 178)
[function] pub fn init_cpu(&mut self, cpu: u32) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 184)
[function] pub fn get(&self, cpu: u32) -> Option<&CpuStopper> {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 24)
[enum] pub enum CacheLevel {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 33)
[struct] pub struct CacheTopology {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 41)
[struct] pub struct NumaNode {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 60)
[function] pub fn distance_to(&self, other_node: u32) -> u8 {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 67)
[struct] pub struct SchedGroup {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 96)
[struct] pub struct SchedDomain {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 137)
[function] pub fn has_flag(&self, flag: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 141)
[function] pub fn is_numa(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 145)
[function] pub fn shares_cache(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 149)
[function] pub fn is_smt(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 155)
[struct] pub struct CpuTopology {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 192)
[struct] pub struct TopologyState {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 226)
[function] pub fn alloc_domain(&mut self) -> Option<&mut SchedDomain> {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 234)
[function] pub fn alloc_group(&mut self) -> Option<&mut SchedGroup> {
kernel_Workspace/kernel/src/cpu/scheduler/time/plist.rs (line 42)
[struct] pub struct PList {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/time/plist.rs (line 51)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/time/plist.rs (line 150)
[function] pub fn first(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/time/plist.rs (line 165)
[function] pub fn last(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtree.rs (line 62)
[struct] pub struct RbTree {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 4)
[struct] pub struct RtArray {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 20)
[function] pub fn set_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 27)
[function] pub fn clear_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 41)
[function] pub fn highest_prio(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 149)
[function] pub fn active_levels(&self) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/time/task.rs (line 7)
[struct] pub struct ListHead {
kernel_Workspace/kernel/src/cpu/scheduler/time/task.rs (line 18)
[function] pub fn is_empty(&self) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/time/task.rs (line 52)
[struct] pub struct RtFields {
kernel_Workspace/kernel/src/cpu/scheduler/time/task.rs (line 58)
[struct] pub struct FairFields {
kernel_Workspace/kernel/src/cpu/scheduler/time/task.rs (line 63)
[struct] pub struct TaskStruct {
kernel_Workspace/kernel/src/cpu/smp.rs (line 91)
[function] pub fn flush_serial_no_panic() {
kernel_Workspace/kernel/src/cpu/smp.rs (line 141)
[function] pub fn init(boot_info: &'static bootloader::BootInfo) {
kernel_Workspace/kernel/src/cpu/smp.rs (line 248)
[function] pub fn total_cpus() -> u32 {
kernel_Workspace/kernel/src/cpu/smp.rs (line 252)
[function] pub fn poweroff() -> bool {
kernel_Workspace/kernel/src/cpu/smp.rs (line 276)
[function] pub fn reboot() -> ! {
kernel_Workspace/kernel/src/cpu/smp.rs (line 293)
[function] pub fn self_test() -> TestResult {
kernel_Workspace/kernel/src/cpu/trampoline.rs (line 23)
[function] pub fn install(cr3: u64, entry: u64) {
kernel_Workspace/kernel/src/cpu/trampoline.rs (line 48)
[function] pub fn set_stack_and_arg(stack_top: u64, arg: u64) {
kernel_Workspace/kernel/src/ctrlinstall/common.rs (line 2)
[struct] pub struct PackageId {
kernel_Workspace/kernel/src/ctrlinstall/common.rs (line 8)
[struct] pub struct Version {

```

```

kernel_Workspace/kernel/src/ctrlinstall/common.rs (line 15)
[enum] pub enum ComponentKind {
kernel_Workspace/kernel/src/ctrlinstall/common.rs (line 23)
[enum] pub enum PackageStatus {
kernel_Workspace/kernel/src/ctrlinstall/common.rs (line 31)
[enum] pub enum CtrlInstallError {
kernel_Workspace/kernel/src/ctrlinstall/init/bootstrap.rs (line 1)
[struct] pub struct BootstrapResult {
kernel_Workspace/kernel/src/ctrlinstall/init/bootstrap.rs (line 6)
[function] pub fn bootstrap() -> Result<BootstrapResult, CtrlInstallError> {
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 1)
[struct] pub struct SystemState {
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 6)
[struct] pub struct InstalledPackage {
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 16)
[function] pub fn is_installed(&self, name: &str) -> bool { todo!() }
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 17)
[function] pub fn get_package(&self, name: &str) -> Option<&InstalledPackage> {
todo!() }
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 18)
[function] pub fn register(&mut self, pkg: InstalledPackage) { todo!() }
kernel_Workspace/kernel/src/ctrlinstall/init/state.rs (line 19)
[function] pub fn unregister(&mut self, name: &str) -> Result<(), CtrlInstallError>
{ todo!() }
kernel_Workspace/kernel/src/ctrlinstall/install/executor.rs (line 1)
[struct] pub struct InstallExecutor<'a> {
kernel_Workspace/kernel/src/ctrlinstall/install/executor.rs (line 6)
[function] pub fn execute(&mut self, tx: &Transaction) -> Result<(),
CtrlInstallError> {
kernel_Workspace/kernel/src/ctrlinstall/install/resolver.rs (line 1)
[struct] pub struct ResolvedPlan {
kernel_Workspace/kernel/src/ctrlinstall/install/resolver.rs (line 5)
[function] pub fn resolve(
kernel_Workspace/kernel/src/ctrlinstall/install/transaction.rs (line 1)
[enum] pub enum TransactionStep {
kernel_Workspace/kernel/src/ctrlinstall/install/transaction.rs (line 8)
[struct] pub struct Transaction {
kernel_Workspace/kernel/src/ctrlinstall/mod.rs (line 9)
[struct] pub struct CtrlInstall {
kernel_Workspace/kernel/src/ctrlinstall/mod.rs (line 15)
[function] pub fn new() -> Result<Self, CtrlInstallError> {
kernel_Workspace/kernel/src/ctrlinstall/mod.rs (line 21)
[function] pub fn install(&mut self, name: &str) -> Result<(), CtrlInstallError> {
kernel_Workspace/kernel/src/ctrlinstall/mod.rs (line 33)
[function] pub fn update(&mut self) -> Result<(), CtrlInstallError> {
kernel_Workspace/kernel/src/ctrlinstall/mod.rs (line 39)
[function] pub fn list_installed(&self) -> &[init::state::InstalledPackage] {
kernel_Workspace/kernel/src/ctrlinstall/repo/fetch.rs (line 1)
[trait] pub trait PackageFetcher {

```

```

kernel_Workspace/kernel/src/ctrlinstall/repo/fetch.rs (line 5)
[struct] pub struct LocalFetcher {
kernel_Workspace/kernel/src/ctrlinstall/repo/fetch.rs (line 9)
[struct] pub struct RemoteFetcher {
kernel_Workspace/kernel/src/ctrlinstall/repo/index.rs (line 1)
[struct] pub struct RepositoryIndex {
kernel_Workspace/kernel/src/ctrlinstall/repo/index.rs (line 6)
[function] pub fn find(&self, name: &str) -> Option<&PackageManifest> { todo!() }
kernel_Workspace/kernel/src/ctrlinstall/repo/index.rs (line 7)
[function] pub fn find_by_kind(&self, kind: ComponentKind) ->
alloc::vec::Vec<&PackageManifest> { todo!() }
kernel_Workspace/kernel/src/ctrlinstall/repo/index.rs (line 8)
[function] pub fn search(&self, query: &str) -> alloc::vec::Vec<&PackageManifest> {
todo!() }
kernel_Workspace/kernel/src/ctrlinstall/repo/manifest.rs (line 1)
[struct] pub struct PackageManifest {
kernel_Workspace/kernel/src/ctrlinstall/repo/manifest.rs (line 10)
[struct] pub struct Dependency {
kernel_Workspace/kernel/src/ctrlinstall/repo/manifest.rs (line 15)
[struct] pub struct FileEntry {
kernel_Workspace/kernel/src/ctrlinstall/update/diff.rs (line 1)
[enum] pub enum UpdateAction {
kernel_Workspace/kernel/src/ctrlinstall/update/diff.rs (line 7)
[function] pub fn diff(
kernel_Workspace/kernel/src/ctrlinstall/update/upgrade.rs (line 1)
[struct] pub struct UpgradePlan {
kernel_Workspace/kernel/src/ctrlinstall/update/upgrade.rs (line 6)
[function] pub fn plan_upgrade(
kernel_Workspace/kernel/src/ctrlinstall/update/upgrade.rs (line 13)
[function] pub fn execute_upgrade(plan: &UpgradePlan, executor: &mut
InstallExecutor) -> Result<(), CtrlInstallError> {
kernel_Workspace/kernel/src/displayport/aut.rs (line 7)
[function] pub fn authorize(ring: u8, op: u8) -> bool {
kernel_Workspace/kernel/src/displayport/bridge.rs (line 13)
[function] pub fn dp_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
kernel_Workspace/kernel/src/displayport/init.rs (line 6)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/displayport/init.rs (line 10)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 2)
[struct] pub struct PciDev {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 22)
[function] pub fn read32(d: PciDev, off: u32) -> u32 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 29)
[function] pub fn write32(d: PciDev, off: u32, v: u32) {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 36)
[function] pub fn port_out(port: u16, val: u32, size: u32) {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 46)
[function] pub fn port_in(port: u16, size: u32) -> u32 {

```

```

kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 68)
[function] pub fn read16(d: PciDev, off: u32) -> u16 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 72)
[function] pub fn read8(d: PciDev, off: u32) -> u8 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 77)
[function] pub fn vendor(self) -> u16 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 81)
[function] pub fn device_id(self) -> u16 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 85)
[function] pub fn class(self) -> u8 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 89)
[function] pub fn subclass(self) -> u8 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 93)
[function] pub fn prog_if(self) -> u8 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 97)
[function] pub fn bar(self, idx: u32) -> u64 {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 108)
[function] pub fn enable_mmio(self) {
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 114)
[function] pub fn find_class(class: u8, subclass: u8, prog_if: u8) -> Option<PciDev>
{
kernel_Workspace/kernel/src/drivers/usb/class/hid/keyboard.rs (line 18)
[function] pub fn key_to_ascii(key: u8, shift: bool) -> Option<u8> {
kernel_Workspace/kernel/src/drivers/usb/class/hid/keyboard.rs (line 44)
[function] pub fn push_char(c: u8) {
kernel_Workspace/kernel/src/drivers/usb/class/hid/keyboard.rs (line 55)
[function] pub fn take_char() -> Option<u8> {
kernel_Workspace/kernel/src/drivers/usb/class/hid/mod.rs (line 16)
[struct] pub struct HidKeyboard {
kernel_Workspace/kernel/src/drivers/usb/class/hid/mod.rs (line 31)
[function] pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool,
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/class/hid/mod.rs (line 105)
[function] pub fn poll(x: &mut Xhci) {
kernel_Workspace/kernel/src/drivers/usb/class/hid/report.rs (line 1)
[struct] pub struct BootReport {
kernel_Workspace/kernel/src/drivers/usb/class/hid/report.rs (line 6)
[function] pub fn parse_boot_keyboard(report: &[u8]) -> Option<BootReport> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 17)
[struct] pub struct UsbMass {
kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 31)
[function] pub fn bulk_out(&mut self, x: &mut Xhci, phys: u64, len: u32) ->
Result<(), UsbError> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 44)
[function] pub fn bulk_in(&mut self, x: &mut Xhci, phys: u64, len: u32) -> Result<(),
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 57)
[function] pub fn scsi_cmd(&mut self, x: &mut Xhci, cdb: &[u8],

```



```

kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 101)
[function] pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool,
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/mod.rs (line 186)
[function] pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> R {
kernel_Workspace/kernel/src/drivers/usb/class/mass/scsi.rs (line 5)
[function] pub fn read_capacity(x: &mut Xhci, m: &mut UsbMass) -> Result<(u32, u32),
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/scsi.rs (line 20)
[function] pub fn read10(x: &mut Xhci, m: &mut UsbMass,
kernel_Workspace/kernel/src/drivers/usb/class/mass/scsi.rs (line 33)
[function] pub fn write10(x: &mut Xhci, m: &mut UsbMass,
kernel_Workspace/kernel/src/drivers/usb/class/mod.rs (line 5)
[trait] pub trait ClassDriver {
kernel_Workspace/kernel/src/drivers/usb/core/descriptor.rs (line 2)
[struct] pub struct DeviceDesc {
kernel_Workspace/kernel/src/drivers/usb/core/descriptor.rs (line 13)
[struct] pub struct InterfaceDesc {
kernel_Workspace/kernel/src/drivers/usb/core/descriptor.rs (line 22)
[struct] pub struct EndpointDesc {
kernel_Workspace/kernel/src/drivers/usb/core/descriptor.rs (line 29)
[function] pub fn parse_device(buf: &[u8]) -> Option<DeviceDesc> {
kernel_Workspace/kernel/src/drivers/usb/core/descriptor.rs (line 45)
[function] pub fn parse_config(buf: &[u8],
kernel_Workspace/kernel/src/drivers/usb/core/device.rs (line 7)
[struct] pub struct UsbDevice {
kernel_Workspace/kernel/src/drivers/usb/core/device.rs (line 24)
[function] pub fn new(slot: u8, speed: u32, ctx_size: usize) -> Result<Self,
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/core/enumerate.rs (line 14)
[function] pub fn enumerate(x: &mut Xhci, port: u32) -> Result<UsbDevice, UsbError>
{
kernel_Workspace/kernel/src/drivers/usb/core/speed.rs (line 6)
[function] pub fn default_ep0_mps(speed: u32) -> u16 {
kernel_Workspace/kernel/src/drivers/usb/dma.rs (line 5)
[struct] pub struct DmaBuf {
kernel_Workspace/kernel/src/drivers/usb/dma.rs (line 12)
[function] pub fn new(len: usize) -> Result<Self, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/dma.rs (line 31)
[function] pub fn zero(&mut self) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 4)
[struct] pub struct Contexts {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 11)
[function] pub fn setup_configure_ep(&mut self, speed: u32, port: u32,
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 37)
[function] pub fn setup_configure_bulk_pair(&mut self, speed: u32, port: u32,
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 71)
[function] pub fn new(ctx_size: usize) -> Result<Self, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 97)
[function] pub fn setup_address_device(&mut self, speed: u32, port: u32,

```

```

kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 120)
[function] pub fn setup_evaluate_mps(&mut self, mps: u16) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/control.rs (line 6)
[function] pub fn wait_transfer(x: &mut Xhci, slot: u8) -> Result<u8, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/control.rs (line 26)
[function] pub fn wait_transfer_ep(x: &mut Xhci, slot: u8, ep: u8) -> Result<u8,
UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/control.rs (line 46)
[function] pub fn control(x: &mut Xhci,
kernel_Workspace/kernel/src/drivers/usb/host/xhci/control.rs (line 94)
[function] pub fn control_in(x: &mut Xhci, dev: &mut UsbDevice,
kernel_Workspace/kernel/src/drivers/usb/host/xhci/control.rs (line 102)
[function] pub fn control_out(x: &mut Xhci, dev: &mut UsbDevice,
kernel_Workspace/kernel/src/drivers/usb/host/xhci/event.rs (line 9)
[function] pub fn drain_events(&mut self) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 17)
[struct] pub struct Xhci {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 44)
[function] pub(super) fn rt_write64(regs: &XhciRegs, off: usize, v: u64) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 49)
[function] pub fn init(regs: XhciRegs) -> Result<Xhci, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 103)
[function] pub fn command(&mut self, trb: Trb) -> Result<Trb, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 130)
[function] pub fn scan_ports(&mut self) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 36)
[struct] pub struct XhciRegs {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 49)
[function] pub fn new(phys: u64) -> Result<Self, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 103)
[function] pub fn op_read(&self, off: usize) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 107)
[function] pub fn op_write(&self, off: usize, v: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 111)
[function] pub fn rt_read(&self, off: usize) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 115)
[function] pub fn rt_write(&self, off: usize, v: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 119)
[function] pub fn doorbell(&self, slot: u32, target: u32, task: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 126)
[function] pub fn port_sc(&self, port: u32) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 130)
[function] pub fn port_sc_write(&self, port: u32, v: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 134)
[function] pub fn port_speed(&self, port: u32) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 5)
[struct] pub struct CmdRing {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 13)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {

```

```

kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 24)
[function] pub fn phys(&self) -> u64 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 28)
[function] pub fn enqueue(&mut self, mut trb: Trb) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 50)
[struct] pub struct EventRing {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 58)
[struct] pub struct TransferRing {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 66)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 77)
[function] pub fn phys(&self) -> u64 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 81)
[function] pub fn enqueue(&mut self, mut trb: Trb) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 102)
[function] pub fn new(count: usize) -> Result<Self, UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 115)
[function] pub fn erst_phys(&self) -> u64 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 119)
[function] pub fn pending(&self) -> Option<Trb> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 131)
[function] pub fn pop(&mut self) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/ring.rs (line 140)
[function] pub fn erdp(&self) -> u64 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 29)
[struct] pub struct Trb {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 36)
[function] pub fn typ(&self) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 40)
[function] pub fn cycle(&self) -> bool {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 44)
[function] pub fn completion_code(&self) -> u8 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 48)
[function] pub fn slot_id(&self) -> u8 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 52)
[function] pub fn ep_id(&self) -> u8 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 56)
[function] pub fn transfer_len(&self) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 60)
[function] pub fn link(addr: u64) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 68)
[function] pub fn enable_slot() -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 72)
[function] pub fn address_device(slot: u8, ctx_phys: u64) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 80)
[function] pub fn configure_ep(slot: u8, ctx_phys: u64) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 88)
[function] pub fn noop_cmd() -> Self {

```

```

kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 92)
[function] pub fn setup_stage(raw_setup: u64, trt: u32) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 100)
[function] pub fn data_stage(addr: u64, len: u32, dir_in: bool) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 108)
[function] pub fn status_stage(dir_in: bool) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 116)
[function] pub fn normal(addr: u64, len: u32) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 124)
[function] pub fn evaluate_ctx(slot: u8, ctx_phys: u64) -> Self {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/trb.rs (line 133)
[function] pub fn pack_setup(bm_request: u8, b_request: u8, value: u16,
kernel_Workspace/kernel/src/drivers/usb/mod.rs (line 10)
[enum] pub enum UsbError {
kernel_Workspace/kernel/src/drivers/usb/mod.rs (line 24)
[function] pub fn init() -> Result<(), UsbError> {
kernel_Workspace/kernel/src/drivers/usb/mod.rs (line 61)
[function] pub fn poll() {
kernel_Workspace/kernel/src/drivers/usb/mod.rs (line 67)
[function] pub fn self_test() -> TestResult {
kernel_Workspace/kernel/src/drivers/usb/pci_glue.rs (line 8)
[struct] pub struct XhciPci {
kernel_Workspace/kernel/src/drivers/usb/pci_glue.rs (line 13)
[function] pub fn find_xhci() -> Result<XhciPci, UsbError> {
kernel_Workspace/kernel/src/driverspaceinit/abi/abi.rs (line 9)
[enum] pub enum DsCmd {
kernel_Workspace/kernel/src/driverspaceinit/abi/abi.rs (line 38)
[struct] pub struct DsMsg {
kernel_Workspace/kernel/src/driverspaceinit/abi/abi.rs (line 53)
[struct] pub struct DsRing {
kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 3)
[function] pub fn ring_bytes(cap: u64) -> usize {
kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 7)
[struct] pub struct RingView {
kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 20)
[function] pub fn init(&self, cap: u64) {
kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 28)
[function] pub fn push(&self, msg: &DsMsg) -> bool {
kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 49)
[function] pub fn pop(&self) -> Option<DsMsg> {
kernel_Workspace/kernel/src/driverspaceinit/init/enter.rs (line 61)
[function] pub fn enter() -> Result<(), DsError> {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 9)
[struct] pub struct Driverspace {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 24)
[function] pub fn prepare() -> Result<(), DsError> {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 91)
[function] pub fn self_test() -> Result<(), DsError> {

```

```

kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 134)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 138)
[function] pub fn k2d_view() -> Option<RingView> {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 142)
[function] pub fn d2k_view() -> Option<RingView> {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 146)
[function] pub fn scratch_view() -> Option<*mut u8> {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 150)
[function] pub fn map_into_ds(va: u64, phys: u64, len: usize, prot:
space::ProtFlags) -> bool {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 159)
[function] pub fn unmap_from_ds(va: u64, len: usize) -> bool {
kernel_Workspace/kernel/src/driverspaceinit/init/initabi.rs (line 7)
[struct] pub struct DsSwitch {
kernel_Workspace/kernel/src/driverspaceinit/init/initabi.rs (line 47)
[struct] pub struct DsInitParams {
kernel_Workspace/kernel/src/driverspaceinit/init/initcommand.rs (line 5)
[enum] pub enum DsError {
kernel_Workspace/kernel/src/driverspaceinit/init/initcommand.rs (line 14)
[struct] pub struct InitHandshake {
kernel_Workspace/kernel/src/driverspaceinit/init/initcommand.rs (line 21)
[function] pub fn send(&mut self, cmd: DsCmd,
kernel_Workspace/kernel/src/driverspaceinit/init/initcommand.rs (line 60)
[function] pub fn run(&mut self, params_va: u64) -> Result<(), DsError> {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 86)
[function] pub fn vgpu_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring_lvl: u8) -> i32 {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 155)
[function] pub fn pci_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 247)
[function] pub fn video_call(op: u32, m: &DsMsg, r: &mut DsMsg) -> i32 {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 430)
[function] pub fn poll() {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 459)
[function] pub fn post_event(cmd: DsCmd, a0: u64, a1: u64) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 24)
[enum] pub enum BufferTransferMode {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 32)
[enum] pub enum BufferError {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 56)
[struct] pub struct BufferFlags: u32 {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 66)
[struct] pub struct IpcBufferDescriptor {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 74)
[struct] pub struct IpcBufferSlot {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 89)
[function] pub fn new(id: u64, owner: u64) -> Self {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 106)
[function] pub fn is_free(&self) -> bool {

```

```

kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 110)
[function] pub fn mark_in_transfer(&self) -> bool {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 119)
[function] pub fn release(&self) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 124)
[struct] pub struct IpcBufferPool {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 137)
[function] pub fn allocate_slot(&self, owner_task_id: u64) -> Result<u64,
BufferError> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 150)
[function] pub fn register_buffer(
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 192)
[function] pub fn get_slot(&self, slot_id: u64) ->
Option<core::sync::atomic::AtomicPtr<()>> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 196)
[function] pub fn with_slot<F, R>(&self, slot_id: u64, f: F) -> Option<R>
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 206)
[struct] pub struct TransferContext<'a> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 216)
[function] pub fn determine_mode(&self) -> BufferTransferMode {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 235)
[function] pub fn execute_transfer(&mut self) -> Result<(), BufferError> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 498)
[function] pub fn revoke_ipc_buffer_capabilities(cnode: &mut CNode, buffer_slot_id:
u64) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 508)
[function] pub fn cleanup_task_ipc_buffers(pool: &IpcBufferPool, task_id: u64) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 551)
[function] pub fn sys_ipc_buffer_register(
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 582)
[function] pub fn sys_ipc_buffer_unregister(
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 8)
[struct] pub struct Endpoint {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 23)
[function] pub fn new(badge: u64) -> Self {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 34)
[function] pub fn enqueue_sender(&self, task: *mut Task) {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 39)
[function] pub fn enqueue_receiver(&self, task: *mut Task) {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 44)
[function] pub fn dequeue_sender(&self) -> Option<*mut Task> {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 49)
[function] pub fn dequeue_receiver(&self) -> Option<*mut Task> {
kernel_Workspace/kernel/src/ds_ipc/msg.rs (line 5)
[struct] pub struct MessageInfo {
kernel_Workspace/kernel/src/ds_ipc/msg.rs (line 27)
[struct] pub struct MessageRegisters {
kernel_Workspace/kernel/src/ds_ipc/msg.rs (line 39)
[struct] pub struct IpcMessage {

```

```

kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 8)
[enum] pub enum IpcSyscall {
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 15)
[function] pub fn sys_ipc_call(
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 43)
[function] pub fn sys_ipc_recv(ep_cap_idx: u64) -> Result<IpMessage, DsError> {
kernel_Workspace/kernel/src/fs/disc.rs (line 4)
[enum] pub enum DiscError {
kernel_Workspace/kernel/src/fs/disc.rs (line 17)
[trait] pub trait BlockDevice {
kernel_Workspace/kernel/src/fs/disc.rs (line 33)
[enum] pub enum PartitionTableKind {
kernel_Workspace/kernel/src/fs/disc.rs (line 40)
[struct] pub struct Partition {
kernel_Workspace/kernel/src/fs/disc.rs (line 49)
[function] pub fn end_lba(&self) -> Option<u64> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 26)
[struct] pub struct AtaPio {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 66)
[function] pub fn is_present(&self) -> bool {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 129)
[function] pub fn identify(&self) -> Result<[u16; 256], DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 253)
[function] pub fn probe() -> usize {
kernel_Workspace/kernel/src/fs/driver/block.rs (line 2)
[enum] pub enum DriverError {
kernel_Workspace/kernel/src/fs/driver/block.rs (line 11)
[trait] pub trait BlockDevice {
kernel_Workspace/kernel/src/fs/driver/lock.rs (line 3)
[struct] pub struct IrqGuard {
kernel_Workspace/kernel/src/fs/driver/lock.rs (line 8)
[function] pub fn lock() -> Self {
kernel_Workspace/kernel/src/fs/driver/mod.rs (line 7)
[function] pub fn init() {
kernel_Workspace/kernel/src/fs/driver/mod.rs (line 17)
[function] pub fn self_test() -> bool {
kernel_Workspace/kernel/src/fs/driver/registry.rs (line 7)
[function] pub fn register(dev: &'static dyn BlockDevice) -> bool {
kernel_Workspace/kernel/src/fs/driver/registry.rs (line 22)
[function] pub fn get(index: usize) -> Option<&'static dyn BlockDevice> {
kernel_Workspace/kernel/src/fs/driver/registry.rs (line 28)
[function] pub fn first() -> Option<&'static dyn BlockDevice> {
kernel_Workspace/kernel/src/fs/driver/registry.rs (line 32)
[function] pub fn count() -> usize {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 6)
[enum] pub enum ExtError {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 15)
[struct] pub struct Superblock {

```



```

kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 27)
[struct] pub struct RawInode {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 33)
[struct] pub struct DirEntry {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 39)
[struct] pub struct Ext4 {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 65)
[function] pub fn mount(disk: &'static dyn BlockDevice) -> Result<Self, ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 129)
[function] pub fn read_blk(&self, blk: u64) -> Result<Vec<u8>, ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 156)
[function] pub fn read_inode(&self, ino: u32) -> Result<RawInode, ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 227)
[function] pub fn read_file(&self, inode: &RawInode,
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 262)
[function] pub fn read_dir(&self, inode: &RawInode) -> Result<Vec<DirEntry>,
ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 297)
[function] pub fn resolve(&self, path: &str) -> Result<RawInode, ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 332)
[function] pub fn read_path(&self, path: &str, buf: &mut [u8]) -> Result<usize,
ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 337)
[function] pub fn list_path(&self, path: &str) -> Result<Vec<DirEntry>, ExtError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 6)
[enum] pub enum FatError {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 14)
[struct] pub struct Fat32 {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 23)
[struct] pub struct FatEntry {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 39)
[function] pub fn mount(disk: &'static dyn BlockDevice) -> Result<Self, FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 142)
[function] pub fn read_dir_cluster(&self, start: u32) -> Result<Vec<FatEntry>,
FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 226)
[function] pub fn resolve(&self, path: &str) -> Result<FatEntry, FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 264)
[function] pub fn read_file(&self, entry: &FatEntry,
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 290)
[function] pub fn read_path(&self, path: &str, buf: &mut [u8]) -> Result<usize,
FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 295)
[function] pub fn list_path(&self, path: &str) -> Result<Vec<FatEntry>, FatError> {
kernel_Workspace/kernel/src/fs/mbr.rs (line 9)
[struct] pub struct MbrEntry {
kernel_Workspace/kernel/src/fs/mbr.rs (line 16)
[struct] pub struct Partition {
kernel_Workspace/kernel/src/fs/mbr.rs (line 57)
[function] pub fn parse_mbr(buf: &[u8]) -> Option<[MbrEntry; 4]> {

```

```

kernel_Workspace/kernel/src/fs/mbr.rs (line 80)
[function] pub fn probe_disk(d: &'static dyn BlockDevice) -> usize {
kernel_Workspace/kernel/src/fs/mbr.rs (line 138)
[function] pub fn init() {
kernel_Workspace/kernel/src/fs/mod.rs (line 9)
[function] pub fn init() {
kernel_Workspace/kernel/src/fs/mod.rs (line 14)
[function] pub fn self_test() -> TestResult {
kernel_Workspace/kernel/src/fs/mod.rs (line 109)
[function] pub fn root_device() -> Option<&'static dyn driver::block::BlockDevice> {
kernel_Workspace/kernel/src/fs/mod.rs (line 119)
[function] pub fn ensure_formatted(dev: &dyn BlockDevice) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 9)
[struct] pub struct BTreeNode {
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 21)
[function] pub fn lookup_dir_entry(fs: &TangFs, dir_ino: u64, name: &str) ->
Result<u64> {
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 91)
[function] pub fn read_dir_entries(fs: &TangFs, dir_ino: u64) ->
Result<Vec<DirEntry>> {
kernel_Workspace/kernel/src/fs/tangfs/dir.rs (line 2)
[struct] pub struct DirEntry {
kernel_Workspace/kernel/src/fs/tangfs/extern.rs (line 4)
[function] pub fn allocate_block(fs: &TangFs) -> Result<u64> {
kernel_Workspace/kernel/src/fs/tangfs/extern.rs (line 35)
[function] pub fn free_block(fs: &TangFs, block: u64) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/file.rs (line 4)
[function] pub fn read_file(fs: &TangFs, inode: &Inode, mut offset: u64, buf: &mut
[u8]) -> Result<usize> {
kernel_Workspace/kernel/src/fs/tangfs/file.rs (line 50)
[function] pub fn write_file(fs: &TangFs, inode: &mut Inode, mut offset: u64, data:
&[u8]) -> Result<usize> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 7)
[struct] pub struct Inode {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 28)
[struct] pub struct Extent {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 35)
[struct] pub struct InodeHandle {
kernel_Workspace/kernel/src/fs/tangfs/journal.rs (line 5)
[struct] pub struct Journal {
kernel_Workspace/kernel/src/fs/tangfs/journal.rs (line 13)
[function] pub fn open(device: &'static dyn crate::fs::driver::BlockDevice, sb:
&super::Superblock) -> Result<Self> {
kernel_Workspace/kernel/src/fs/tangfs/journal.rs (line 22)
[function] pub fn write_block(&mut self, block: u64, data: &[u8]) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/journal.rs (line 39)
[function] pub fn replay(&self) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 18)
[struct] pub struct TangFs {

```

```

kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 26)
[function] pub fn mount(device: &'static dyn crate::fs::driver::BlockDevice) ->
Result<Self> {
kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 49)
[function] pub fn read_block(&self, block: u64) -> Result<Vec<u8>> {
kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 63)
[function] pub fn write_block(&self, block: u64, data: &[u8]) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 5)
[struct] pub struct Superblock {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 26)
[function] pub fn read(device: &dyn BlockDevice) -> Result<Self> {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 45)
[function] pub fn write(&self, device: &dyn BlockDevice) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 77)
[function] pub fn generate_uuid() -> [u8; 16] {
kernel_Workspace/kernel/src/fs/tfs.rs (line 19)
[enum] pub enum FsError {
kernel_Workspace/kernel/src/fs/tfs.rs (line 33)
[struct] pub struct Superblock {
kernel_Workspace/kernel/src/fs/tfs.rs (line 48)
[function] pub fn format(dev: &dyn BlockDevice) -> Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 70)
[function] pub fn read_superblock(dev: &dyn BlockDevice) -> Result<Superblock> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 85)
[function] pub fn write_superblock(dev: &dyn BlockDevice, sb: &Superblock) ->
Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 156)
[function] pub fn list_dir(dev: &dyn BlockDevice, dir: u32, out: &mut impl Write) ->
Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 186)
[function] pub fn write_file(dev: &dyn BlockDevice, dir: u32, name: &str, data:
&[u8]) -> Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 246)
[function] pub fn remove(dev: &dyn BlockDevice, dir: u32, name: &str) -> Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 274)
[function] pub fn mkdir(dev: &dyn BlockDevice, dir: u32, name: &str) -> Result<()> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 325)
[function] pub fn find_dir(dev: &dyn BlockDevice, dir: u32, name: &str) ->
Result<u32> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 333)
[function] pub fn entries(dev: &dyn BlockDevice, dir: u32) ->
Result<alloc::vec::Vec<alloc::string::String, u32, u8>> {
kernel_Workspace/kernel/src/fs/tfs.rs (line 352)
[function] pub fn read_file(dev: &dyn BlockDevice, dir: u32, name: &str) ->
Result<alloc::vec::Vec<u8>> {
kernel_Workspace/kernel/src/fs/vfs.rs (line 6)
[struct] pub struct FsEntry {
kernel_Workspace/kernel/src/fs/vfs.rs (line 12)
[enum] pub enum Mounted {

```

```

kernel_Workspace/kernel/src/fs/vfs.rs (line 19)
[function] pub fn read_path(&self, path: &str, buf: &mut [u8]) -> Option<usize> {
kernel_Workspace/kernel/src/fs/vfs.rs (line 27)
[function] pub fn list_path(&self, path: &str) -> Option<Vec<FsEntry>> {
kernel_Workspace/kernel/src/fs/vfs.rs (line 56)
[function] pub fn mount_all() {
kernel_Workspace/kernel/src/fs/vfs.rs (line 74)
[function] pub fn root() -> Option<&'static Mounted> {
kernel_Workspace/kernel/src/gdt.rs (line 52)
[function] pub fn init() {
kernel_Workspace/kernel/src/gfx/console.rs (line 32)
[function] pub fn cols() -> usize {
kernel_Workspace/kernel/src/gfx/console.rs (line 36)
[function] pub fn rows() -> usize {
kernel_Workspace/kernel/src/gfx/console.rs (line 40)
[function] pub fn fb_info() -> (u32, u32, u32, u64) {
kernel_Workspace/kernel/src/gfx/console.rs (line 75)
[function] pub(crate) fn framebuffer_info_pub() -> Option<(u32, u32, u32, u64,
bool)> {
kernel_Workspace/kernel/src/gfx/console.rs (line 79)
[function] pub fn set_enabled(enabled: bool) {
kernel_Workspace/kernel/src/gfx/console.rs (line 145)
[function] pub fn test_fill(r: u32, g: u32, b: u32) -> bool {
kernel_Workspace/kernel/src/gfx/console.rs (line 164)
[function] pub fn resync_background() -> bool {
kernel_Workspace/kernel/src/gfx/console.rs (line 188)
[function] pub fn init(fb_addr: u64, width: u32, height: u32, stride: u32, format:
PixelFormat) -> bool {
kernel_Workspace/kernel/src/gfx/console.rs (line 313)
[function] pub fn refresh() {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 8)
[enum] pub enum PixelFormat {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 21)
[struct] pub struct Framebuffer {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 69)
[function] pub fn get(&self, x: usize, y: usize) -> u32 {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 85)
[function] pub fn set(&mut self, x: usize, y: usize, c: u32) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 107)
[function] pub fn offset(&self, x: usize, y: usize) -> usize {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 111)
[function] pub fn add(&mut self, x: usize, y: usize, r: u32, g: u32, b: u32) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 153)
[function] pub fn rgb(r: u32, g: u32, b: u32) -> u32 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 96)
[function] pub fn render(fb: &mut Framebuffer, t: u32) {
kernel_Workspace/kernel/src/gfx/mod.rs (line 16)
[function] pub fn init() -> bool {

```

```

kernel_Workspace/kernel/src/gfx/mod.rs (line 25)
[function] pub fn init_mode(fb_phys: u64, width: u32, height: u32, stride: u32) ->
bool {
kernel_Workspace/kernel/src/gfx/mod.rs (line 30)
[function] pub fn set_resolution(mode: vga::VideoMode) -> bool {
kernel_Workspace/kernel/src/gfx/mod.rs (line 47)
[function] pub fn set_resolution_w_h(width: u32, height: u32) -> bool {
kernel_Workspace/kernel/src/gfx/mod.rs (line 73)
[function] pub fn current_resolution() -> (u32, u32) {
kernel_Workspace/kernel/src/gfx/mod.rs (line 77)
[function] pub fn refresh() {
kernel_Workspace/kernel/src/gfx/mod.rs (line 81)
[function] pub fn self_test() -> Result<&'static str, &'static str> {
kernel_Workspace/kernel/src/gfx/panic_screen.rs (line 11)
[function] pub fn show(info: &PanicInfo) -> ! {
kernel_Workspace/kernel/src/gfx/vga.rs (line 4)
[enum] pub enum VideoMode {
kernel_Workspace/kernel/src/gfx/vga.rs (line 47)
[function] pub fn set_mode(mode: VideoMode) {
kernel_Workspace/kernel/src/gfx/vga.rs (line 105)
[function] pub fn bochs_version() -> Option<u16> {
kernel_Workspace/kernel/src/gfx/vga.rs (line 114)
[function] pub fn bochs_disable() {
kernel_Workspace/kernel/src/gfx/vga.rs (line 120)
[function] pub fn bochs_set_mode(width: u32, height: u32, bpp: u32) -> bool {
kernel_Workspace/kernel/src/gfx/vga.rs (line 139)
[function] pub fn bochs_lfb_base() -> Option<u64> {
kernel_Workspace/kernel/src/hdmi/aut.rs (line 18)
[function] pub fn authorize(ring: u8, op: u32) -> bool {
kernel_Workspace/kernel/src/hdmi/aut.rs (line 46)
[function] pub fn tick_reset() {
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 24)
[function] pub fn hdmi_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8, owner: u32)
-> i32 {
kernel_Workspace/kernel/src/hdmi/init.rs (line 6)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/hdmi/init.rs (line 11)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/interrupts.rs (line 50)
[enum] pub enum InterruptIndex {
kernel_Workspace/kernel/src/interrupts.rs (line 83)
[function] pub fn init_idt() {
kernel_Workspace/kernel/src/main.rs (line 146)
[function] pub fn init() {
kernel_Workspace/kernel/src/main.rs (line 150)
[function] pub fn hlt_loop() -> ! {
kernel_Workspace/kernel/src/main.rs (line 154)
[function] pub fn init_permissions() {

```

```

kernel_Workspace/kernel/src/main.rs (line 167)
[function] pub fn kernel_main(boot_info: &'static bootloader::BootInfo) -> ! {
kernel_Workspace/kernel/src/main.rs (line 239)
[function] pub fn kernel_main_riscv() -> ! {
kernel_Workspace/kernel/src/mm/api.rs (line 5)
[function] pub fn kmalloc(size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/api.rs (line 11)
[function] pub fn kzalloc(size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/api.rs (line 17)
[function] pub fn krealloc(ptr: *mut u8, size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/api.rs (line 23)
[function] pub fn kfree(ptr: *mut u8) {
kernel_Workspace/kernel/src/mm/api.rs (line 27)
[function] pub fn kalloc_pages(pages: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/api.rs (line 33)
[function] pub fn kfree_pages(ptr: *mut u8, _pages: usize) {
kernel_Workspace/kernel/src/mm/api.rs (line 37)
[function] pub fn virt_to_phys(ptr: *mut u8) -> u64 {
kernel_Workspace/kernel/src/mm/api.rs (line 41)
[struct] pub struct KernelAlloc;
kernel_Workspace/kernel/src/mm/api.rs (line 72)
[function] pub fn self_test() -> Result<&'static str, &'static str> {
kernel_Workspace/kernel/src/mm/ffi.rs (line 5)
[struct] pub struct RawMemEntry {
kernel_Workspace/kernel/src/mm/ffi.rs (line 14)
[struct] pub struct MmBootParams {
kernel_Workspace/kernel/src/mm/ffi.rs (line 25)
[function] pub fn mm_init(params: *const MmBootParams) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 26)
[function] pub fn mm_ready() -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 27)
[function] pub fn mm_total_ram() -> u64;
kernel_Workspace/kernel/src/mm/ffi.rs (line 28)
[function] pub fn mm_free_ram() -> u64;
kernel_Workspace/kernel/src/mm/ffi.rs (line 29)
[function] pub fn mm_dump();
kernel_Workspace/kernel/src/mm/ffi.rs (line 31)
[function] pub fn arch_memory_ready() -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 32)
[function] pub fn arch_memory_reserve_range(base: u64, len: u64);
kernel_Workspace/kernel/src/mm/ffi.rs (line 33)
[function] pub fn arch_memory_boot_alloc(len: u64, align: u64, out: *mut u64) ->
bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 35)
[function] pub fn paging_init(boot_phys_offset: u64);
kernel_Workspace/kernel/src/mm/ffi.rs (line 36)
[function] pub fn paging_enable_nx();
kernel_Workspace/kernel/src/mm/ffi.rs (line 37)
[function] pub fn paging_read_cr3() -> u64;

```

```
kernel_Workspace/kernel/src/mm/ffi.rs (line 38)
[function] pub fn paging_is_mapped(virt: u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 39)
[function] pub fn paging_aspace_switch(aspaces: *mut c_void);
kernel_Workspace/kernel/src/mm/ffi.rs (line 40)
[function] pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 42)
[function] pub fn pmm_init() -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 43)
[function] pub fn pmm_alloc_frame(out: *mut u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 44)
[function] pub fn pmm_alloc_zero_frame(out: *mut u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 45)
[function] pub fn pmm_alloc_frames(count: usize, out: *mut u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 46)
[function] pub fn pmm_alloc_frames_aligned(count: usize,
kernel_Workspace/kernel/src/mm/ffi.rs (line 49)
[function] pub fn pmm_free_frame(phys: u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 50)
[function] pub fn pmm_free_frames(phys: u64, count: usize) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 53)
[function] pub fn vmm_init() -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 54)
[function] pub fn vmm_alloc(bytes: usize, flags: u32, out: *mut u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 55)
[function] pub fn vmm_free(virt: u64, bytes: usize) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 56)
[function] pub fn vmm_map_device(phys: u64, len: usize, out: *mut u64) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 57)
[function] pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 59)
[function] pub fn kmalloc(size: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 60)
[function] pub fn kcalloc(size: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 61)
[function] pub fn kcalloc(count: usize, size: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 62)
[function] pub fn krealloc(ptr: *mut c_void, size: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 63)
[function] pub fn kmalloc_aligned(size: usize, align: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 64)
[function] pub fn kfree(ptr: *mut c_void);
kernel_Workspace/kernel/src/mm/ffi.rs (line 65)
[function] pub fn kalloc_pages(pages: usize) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 66)
[function] pub fn kfree_pages(ptr: *mut c_void, pages: usize);
kernel_Workspace/kernel/src/mm/ffi.rs (line 67)
[function] pub fn kvirt_to_phys(ptr: *mut c_void) -> u64;
```



```

kernel_Workspace/kernel/src/mm/ffi.rs (line 69)
[function] pub fn contig_alloc(bytes: usize,
kernel_Workspace/kernel/src/mm/ffi.rs (line 73)
[function] pub fn contig_free(phys: u64, bytes: usize);
kernel_Workspace/kernel/src/mm/ffi.rs (line 74)
[function] pub fn dma_alloc_coherent(bytes: usize,
kernel_Workspace/kernel/src/mm/ffi.rs (line 78)
[function] pub fn dma_free_coherent(phys: u64, virt: *mut c_void, bytes: usize);
kernel_Workspace/kernel/src/mm/ffi.rs (line 81)
[function] pub fn aspace_subsystem_init() -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 82)
[function] pub fn aspace_create() -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 83)
[function] pub fn aspace_destroy(pa: *mut c_void);
kernel_Workspace/kernel/src/mm/ffi.rs (line 84)
[function] pub fn aspace_paging_handle(pa: *mut c_void) -> *mut c_void;
kernel_Workspace/kernel/src/mm/ffi.rs (line 85)
[function] pub fn aspace_map_anon(pa: *mut c_void,
kernel_Workspace/kernel/src/mm/ffi.rs (line 89)
[function] pub fn aspace_unmap(pa: *mut c_void, addr: u64, len: usize) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 90)
[function] pub fn aspace_protect(pa: *mut c_void,
kernel_Workspace/kernel/src/mm/ffi.rs (line 94)
[function] pub fn aspace_brk(pa: *mut c_void, new_brk: u64) -> u64;
kernel_Workspace/kernel/src/mm/ffi.rs (line 95)
[function] pub fn mmap(pa: *mut c_void,
kernel_Workspace/kernel/src/mm/ffi.rs (line 100)
[function] pub fn paging_aspace_map(aspace: *mut c_void,
kernel_Workspace/kernel/src/mm/ffi.rs (line 105)
[function] pub fn munmap(pa: *mut c_void, addr: u64, len: usize) -> bool;
kernel_Workspace/kernel/src/mm/ffi.rs (line 106)
[function] pub fn paging_aspace_cr3(aspace: *mut c_void) -> u64;
kernel_Workspace/kernel/src/mm/init.rs (line 9)
[struct] pub struct Driverspace {
kernel_Workspace/kernel/src/mm/init.rs (line 24)
[function] pub fn prepare() -> Result<(), DsError> {
kernel_Workspace/kernel/src/mm/init.rs (line 91)
[function] pub fn self_test() -> Result<(), DsError> {
kernel_Workspace/kernel/src/mm/init.rs (line 134)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/mm/init.rs (line 138)
[function] pub fn k2d_view() -> Option<RingView> {
kernel_Workspace/kernel/src/mm/init.rs (line 142)
[function] pub fn d2k_view() -> Option<RingView> {
kernel_Workspace/kernel/src/mm/init.rs (line 146)
[function] pub fn scratch_view() -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/init.rs (line 150)
[function] pub fn map_into_ds(va: u64, phys: u64, len: usize, prot:
space::ProtFlags) -> bool {

```

```

kernel_Workspace/kernel/src/mm/init.rs (line 159)
[function] pub fn unmap_from_ds(va: u64, len: usize) -> bool {
kernel_Workspace/kernel/src/mm/mm_bridge.rs (line 14)
[struct] pub struct RawMemEntry {
kernel_Workspace/kernel/src/mm/mod.rs (line 20)
[function] pub fn init(params: &ffi::MmBootParams) -> bool {
kernel_Workspace/kernel/src/mm/mod.rs (line 25)
[function] pub fn ready() -> bool {
kernel_Workspace/kernel/src/mm/mod.rs (line 30)
[function] pub fn dump() {
kernel_Workspace/kernel/src/mm/mod.rs (line 35)
[function] pub fn init_riscv() -> bool {
kernel_Workspace/kernel/src/mm/mod.rs (line 57)
[function] pub fn init_from_boot_info(boot_info: &'static bootloader::BootInfo) ->
bool {
kernel_Workspace/kernel/src/mm/mod.rs (line 110)
[function] pub fn self_test() -> Result<&'static str, &'static str> {
kernel_Workspace/kernel/src/mm/phys.rs (line 3)
[function] pub fn alloc_frame() -> Option<u64> {
kernel_Workspace/kernel/src/mm/phys.rs (line 13)
[function] pub fn alloc_zero_frame() -> Option<u64> {
kernel_Workspace/kernel/src/mm/phys.rs (line 23)
[function] pub fn alloc_frames(count: usize) -> Option<u64> {
kernel_Workspace/kernel/src/mm/phys.rs (line 33)
[function] pub fn alloc_frames_aligned(count: usize, align: usize) -> Option<u64> {
kernel_Workspace/kernel/src/mm/phys.rs (line 43)
[function] pub fn free_frame(phys: u64) -> bool {
kernel_Workspace/kernel/src/mm/phys.rs (line 47)
[function] pub fn free_frames(phys: u64, count: usize) -> bool {
kernel_Workspace/kernel/src/mm/phys.rs (line 51)
[function] pub fn reserve(base: u64, len: u64) {
kernel_Workspace/kernel/src/mm/phys.rs (line 55)
[function] pub fn total_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/phys.rs (line 59)
[function] pub fn free_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/phys.rs (line 65)
[function] pub fn phys_to_virt(phys: u64) -> *mut u8 {
kernel_Workspace/kernel/src/mm/phys.rs (line 69)
[function] pub fn self_test() -> Result<&'static str, &'static str> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 29)
[function] pub fn init() {
kernel_Workspace/kernel/src/mm/riscv.rs (line 47)
[function] pub fn reserve(base: u64, len: u64) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 91)
[function] pub fn alloc_frame() -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 95)
[function] pub fn alloc_zero_frame() -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 103)
[function] pub fn alloc_frames(count: usize) -> Option<u64> {

```

```

kernel_Workspace/kernel/src/mm/riscv.rs (line 108)
[function] pub fn alloc_frames_aligned(count: usize, align_frames: usize) ->
Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 121)
[function] pub fn free_frame(pa: u64) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 125)
[function] pub fn free_frames(pa: u64, count: usize) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 152)
[function] pub fn total_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 156)
[function] pub fn free_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 165)
[function] pub fn phys_to_virt(phys: u64) -> *mut u8 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 195)
[function] pub fn init() {
kernel_Workspace/kernel/src/mm/riscv.rs (line 206)
[function] pub fn pool_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 210)
[function] pub fn heap_free_bytes() -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 334)
[function] pub fn kmalloc(size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 342)
[function] pub fn kmalloc_aligned(size: usize, align: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 350)
[function] pub fn kzalloc(size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 356)
[function] pub fn kcalloc(count: usize, size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 360)
[function] pub fn kfree(ptr: *mut u8) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 378)
[function] pub fn krealloc(ptr: *mut u8, size: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 395)
[function] pub fn kalloc_pages(pages: usize) -> Option<*mut u8> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 399)
[function] pub fn kfree_pages(ptr: *mut u8, _pages: usize) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 403)
[function] pub fn kvirt_to_phys(ptr: *mut u8) -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 420)
[struct] pub struct VmmFlags(u32);
kernel_Workspace/kernel/src/mm/riscv.rs (line 470)
[function] pub fn init() {
kernel_Workspace/kernel/src/mm/riscv.rs (line 504)
[function] pub fn alloc(bytes: usize, _flags: VmmFlags) -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 513)
[function] pub fn free(va: u64, bytes: usize) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 525)
[function] pub fn map_device(phys: u64, len: usize) -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 536)
[function] pub fn unmap_device(va: u64, len: usize) -> bool {

```

```

kernel_Workspace/kernel/src/mm/riscv.rs (line 570)
[struct] pub struct ProtFlags(u32);
kernel_Workspace/kernel/src/mm/riscv.rs (line 601)
[struct] pub struct MapFlags(u32);
kernel_Workspace/kernel/src/mm/riscv.rs (line 663)
[struct] pub struct AddressSpace {
kernel_Workspace/kernel/src/mm/riscv.rs (line 674)
[function] pub fn new() -> Option<Self> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 687)
[function] pub fn handle(&self) -> *mut c_void {
kernel_Workspace/kernel/src/mm/riscv.rs (line 691)
[function] pub fn cr3(&self) -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 732)
[function] pub fn translate(&self, va: u64) -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 758)
[function] pub fn map_phys(&self, virt: u64, phys: u64, len: usize, prot: ProtFlags)
-> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 773)
[function] pub fn map_anon(&self, hint: u64, len: usize, prot: ProtFlags) ->
Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 807)
[function] pub fn mmap(
kernel_Workspace/kernel/src/mm/riscv.rs (line 821)
[function] pub fn munmap(&self, addr: u64, len: usize) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 840)
[function] pub fn protect(&self, addr: u64, len: usize, prot: ProtFlags) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 878)
[function] pub fn brk(&self, new_brk: u64) -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 886)
[function] pub fn switch(&self) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 931)
[function] pub fn init() -> bool {
kernel_Workspace/kernel/src/mm/space.rs (line 7)
[struct] pub struct ProtFlags(u32);
kernel_Workspace/kernel/src/mm/space.rs (line 39)
[struct] pub struct MapFlags(u32);
kernel_Workspace/kernel/src/mm/space.rs (line 64)
[struct] pub struct AddressSpace {
kernel_Workspace/kernel/src/mm/space.rs (line 69)
[function] pub fn new() -> Option<Self> {
kernel_Workspace/kernel/src/mm/space.rs (line 79)
[function] pub fn handle(&self) -> *mut c_void {
kernel_Workspace/kernel/src/mm/space.rs (line 83)
[function] pub fn cr3(&self) -> u64 {
kernel_Workspace/kernel/src/mm/space.rs (line 86)
[function] pub fn map_phys(&self, virt: u64, phys: u64, len: usize, prot: ProtFlags)
-> bool {
kernel_Workspace/kernel/src/mm/space.rs (line 90)
[function] pub fn map_anon(&self, hint: u64, len: usize, prot: ProtFlags) ->
Option<u64> {

```

kernel_workspace/kernel/src/mm/space.rs (line 96)

```
[function] pub fn mmap(&self, addr: u64, len: usize, prot: ProtFlags, flags: MapFlags) -> Option<u64> {
```

kernel_workspace/kernel/src/mm/space.rs (line 102)

```
[function] pub fn munmap(&self, addr: u64, len: usize) -> bool {
```

kernel_workspace/kernel/src/mm/space.rs (line 106)

```
[function] pub fn protect(&self, addr: u64, len: usize, prot: ProtFlags) -> bool {
```

(List truncated to 4000 entries to keep the document size manageable. Use --max-items-per-section to change this.)

2. Internal functions

Ada (body)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 6)

[function] function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 11)

[function] function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 18)

[function] function Get_Buddy_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 31)

[function] function Get_Left_Child_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 40)

[function] function Get_Right_Child_Address (

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 50)

[function] function Safe_Calculate_DMA_End (

kernel_Workspace/kernel/src/policy/ada/policy.adb (line 5)

[function] function Evaluate (Ring : Ring_Id;

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 5)

[function] function To_Ring (V : Unsigned_8) return Ring_Id is

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 11)

[function] function To_Class (V : Unsigned_8) return Call_Class is

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 20)

[function] function policy_evaluate (ring : Unsigned_8;

Assembly

comgrub/src/boot.asm (line 10)

[label] mboot_header_start:

comgrub/src/boot.asm (line 19)

[label] mboot_header_end:

comgrub/src/boot.asm (line 23)

[label] stack_bottom:

comgrub/src/boot.asm (line 25)

[label] stack_top:

comgrub/src/boot.asm (line 33)

[label] _start:

kernel_Workspace/kernel-bin/src/boot.asm (line 8)

[label] kernel_bin_entry:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 13)

[label] trampoline_start:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 31)

[label] trampoline_32:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 59)

[label] trampoline_64:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 81)

[label] trampoline_gdt:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 87)

[label] trampoline_gdt_end:

kernel_Workspace/kernel/src/cpu/trampoline.s (line 90)
[label] trampoline_gdtr:
kernel_Workspace/kernel/src/cpu/trampoline.s (line 95)
[label] trampoline_cr3: .quad 0
kernel_Workspace/kernel/src/cpu/trampoline.s (line 96)
[label] trampoline_entry: .quad 0
kernel_Workspace/kernel/src/cpu/trampoline.s (line 97)
[label] trampoline_stack: .quad 0
kernel_Workspace/kernel/src/cpu/trampoline.s (line 98)
[label] trampoline_arg: .quad 0
kernel_Workspace/kernel/src/cpu/trampoline.s (line 101)
[label] trampoline_end:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 5)
[label] tlb_asm_invlpg:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 11)
[label] tlb_asm_read_cr3:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 17)
[label] tlb_asm_write_cr3:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 23)
[label] tlb_asm_read_cr4:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 29)
[label] tlb_asm_write_cr4:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 36)
[label] tlb_asm_invpclid:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 42)
[label] tlb_asm_wbinvd:
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 48)
[label] tlb_asm_clflush:
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 4)
[label] vector_table:
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 41)
[label] vector_default:
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 44)
[label] vector_hyper:
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 104)
[label] tr_init:
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 113)
[label] tr_restore_ctx:
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 3)
[label] kstack_top:
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 10)
[label] trap_entry:
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 100)
[label] tr_init:
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 116)
[label] tr_restore_ctx:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 4)
[label] tr_gdt:


```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 13)
[label] tr_tss_desc:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 16)
[label] gdt_ptr:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 22)
[label] tr_tss:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 29)
[label] tr_init:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 56)
[label] isr_hypercall:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 96)
[label] isr_default:
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 101)
[label] tr_restore_ctx:
triang-lang/out/x86_64/prog.asm (line 4)
[label] buffer: resb 16
triang-lang/out/x86_64/prog.asm (line 8)
[label] main:
triang-lang/out/x86_64/prog.asm (line 9)
[label] fn_main:
triang-lang/out/x86_64/prog.asm (line 17)
[label] fill_1:
triang-lang/out/x86_64/prog.asm (line 24)
[label] fill_end_2:
triang-lang/out/x86_64/prog.asm (line 41)
[label] else_1:
triang-lang/out/x86_64/prog.asm (line 43)
[label] end_2:
triang-lang/out/x86_64/prog.asm (line 44)
[label] loop_3:
triang-lang/out/x86_64/prog.asm (line 51)
[label] loop_end_4:

```

C

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 30)
[function] static void push_rax(nb_t *d) { nb_u8(d, 0x50); }
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 31)
[function] static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 32)
[function] static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 33)
[function] static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 10)
[function] static void k_memset(const void *dst, const uint8 value, const size_t n) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 15)
[function] static void k_memcpy(const void *dst, const void *src, const size_t n) {
kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 21)
[function] static size_t k_strlen(const char *s) {

```

```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 27)
[function] static size_t k_usable_size(void *ptr) {
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 25)
[function] static uint64_t dbg_tail_canary(size_t size) { return ~((uint64_t)size) ^
DBG_MAGIC; }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 25)
[function] static inline uint64_t order_size(size_t o) { return ARCH_PAGE_SIZE << o;
}
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 26)
[function] static size_t page_of(uint64_t va) { return (size_t)((va - buddy_base) /
ARCH_PAGE_SIZE); }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 27)
[function] static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page *
ARCH_PAGE_SIZE; }
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 28)
[function] static size_t size_to_order(size_t size) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 36)
[function] static void fl_push(size_t o, size_t idx) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 40)
[function] static size_t fl_pop(size_t o) {
kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 46)
[function] static void fl_remove(size_t o, size_t idx) {

```

Nim

```

kernel_Workspace/kernel/src/nimcore/banner.nim (line 13)
[proc] proc nim_banner(buf: ptr uint8, cap: uint32): uint32 {.exportc, cdecl.} =
kernel_Workspace/kernel/src/nimcore/format.nim (line 1)
[proc] proc nim_u64_to_str(v: uint64, base: uint8, buf: ptr uint8, cap: uint32):
uint32
kernel_Workspace/kernel/src/nimcore/format.nim (line 29)
[proc] proc nim_hex_dump(src: ptr uint8, len: uint32, buf: ptr uint8, cap: uint32):
uint32
kernel_Workspace/kernel/src/nimcore/parse.nim (line 1)
[proc] proc nim_parse_u64(s: ptr uint8, len: uint32, base: uint8, out_v: ptr
uint64): uint8
kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 1)
[type] type Rb = object
kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 8)
[proc] proc nim_rb_push(b: uint8): uint8 {.exportc, cdecl.} =
kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 15)
[proc] proc nim_rb_pop(): int32 {.exportc, cdecl.} =
kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 21)
[proc] proc nim_rb_len(): uint32 {.exportc, cdecl.} =
kernel_Workspace/kernel/src/nimcore/shell.nim (line 1)
[type] type Handler = proc(arg: ptr uint8, len: uint32): int32 {.cdecl.}
kernel_Workspace/kernel/src/nimcore/shell.nim (line 3)
[type] type Cmd = object
kernel_Workspace/kernel/src/nimcore/shell.nim (line 11)
[proc] proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8

```

```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 28)
[proc] proc eq_name(c: var Cmd, line: ptr uint8, nlen: uint32): bool =
kernel_Workspace/kernel/src/nimcore/shell.nim (line 36)
[proc] proc nim_shell_run(line: ptr uint8, len: uint32): int32
kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 10)
[proc] proc nim_policy_log(ring, cls, op, dec: uint8) {.exportc, cdecl.} =
kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 17)
[proc] proc nim_policy_denies(): uint64 {.exportc, cdecl.} =
kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 20)
[proc] proc nim_policy_get(idx: uint32,

```

Python

```

doc.py (line 298)
[function] def _rules_js_ts():
doc.py (line 313)
[function] def _rules_java_cs():
doc.py (line 322)
[function] def _rules_go():
doc.py (line 331)
[function] def _rules_rust():
doc.py (line 342)
[function] def _rules_c_cpp():
doc.py (line 350)
[function] def _rules_php():
doc.py (line 359)
[function] def _rules_ruby():
doc.py (line 367)
[function] def _rules_kotlin_swift():
doc.py (line 378)
[function] def _rules_d():
doc.py (line 387)
[function] def _rules_nim():
doc.py (line 399)
[function] def _rules_odin():
doc.py (line 410)
[function] def _rules_ada_spec():
doc.py (line 420)
[function] def _rules_ada_body():
doc.py (line 428)
[function] def _rules_asm():
doc.py (line 660)
[function] def _new_doc(path: str):
doc.py (line 667)
[function] def _summary_paragraph(styles, root, public_defs, internal_defs,
md_files):

```

Rust

```

c.rs (line 57)
[struct] struct BlockHeader* next; /* Wskaźnik do następnego bloku */

```

```

c.rs (line 58)
[struct] struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
comgrub/build.rs (line 1)
[function] fn main() {
comgrub/src/main.rs (line 11)
[function] fn kernel_main(magic: u32, info: *const u8) -> !;
comgrub/src/main.rs (line 15)
[function] fn panic(_info: &PanicInfo) -> ! {
comlimine/build.rs (line 1)
[function] fn main() {
comlimine/src/main.rs (line 1)
[function] fn main() {}
drivers/amdgpu_driver/src/main.rs (line 1)
[function] fn main() {
drivers/audiodriver/build.rs (line 12)
[function] fn main() {
drivers/audiodriver/src/lib.rs (line 7)
[function] fn ad_init(nam_va: u64, bm_va: u64) -> i32;
drivers/audiodriver/src/lib.rs (line 8)
[function] fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
drivers/audiodriver/src/lib.rs (line 9)
[function] fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) ->
i32;
drivers/audiodriver/src/lib.rs (line 10)
[function] fn ad_stop() -> i32;
drivers/audiodriver/src/lib.rs (line 11)
[function] fn ad_jack_present() -> i32;
drivers/audiodriver/src/lib.rs (line 12)
[function] fn ad_set_amp(on: i32) -> i32;
drivers/audiodriver/src/lib.rs (line 13)
[function] fn ad_position() -> u32;
drivers/audiodriver/src/main.rs (line 8)
[function] fn main() {
drivers/intelgpu_driver/src/main.rs (line 1)
[function] fn main() {
drivers/netcam_driver/src/main.rs (line 1)
[function] fn main() {
drivers/wacomgraphic_driver/src/main.rs (line 1)
[function] fn main() {
driverspace/src/drivers/storage.rs (line 20)
[function] fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
driverspace/src/main.rs (line 32)
[function] fn panic(_info: &core::panic::PanicInfo) -> ! {
driverspace_workspace/crates/ds-manager/src/main.rs (line 1)
[function] fn main() {
driverspace_workspace/drivers/amdgpu-driver/src/main.rs (line 1)
[function] fn main() {
driverspace_workspace/drivers/audiodriver/src/main.rs (line 1)
[function] fn main() {

```

```

driverspace_workspace/drivers/intelgpu-driver/src/main.rs (line 1)
[function] fn main() {
driverspace_workspace/drivers/netcam_driver/src/main.rs (line 1)
[function] fn main() {
driverspace_workspace/drivers/vgpu/src/main.rs (line 42)
[struct] struct VgpuInfo {
driverspace_workspace/drivers/vgpu/src/main.rs (line 67)
[function] fn vgpu_init(w: u32, h: u32) -> bool {
driverspace_workspace/drivers/vgpu/src/main.rs (line 138)
[function] fn find_vga_device() -> Option<u64> {
driverspace_workspace/drivers/vgpu/src/main.rs (line 151)
[function] fn read_pci_config(bdf: u64, offset: u32) -> u32 {
driverspace_workspace/drivers/vgpu/src/main.rs (line 161)
[function] fn read_pci_bar(bdf: u64, bar: u32) -> u64 {
driverspace_workspace/drivers/vgpu/src/main.rs (line 172)
[function] fn enable_pci_device(bdf: u64) {
driverspace_workspace/drivers/vgpu/src/main.rs (line 185)
[function] fn vbe_write(idx: u16, val: u16) {
driverspace_workspace/drivers/vgpu/src/main.rs (line 192)
[function] fn vgpu_clear(color: u32) {
driverspace_workspace/drivers/vgpu/src/main.rs (line 207)
[function] fn vgpu_pixel(x: u32, y: u32, color: u32) {
driverspace_workspace/drivers/vgpu/src/main.rs (line 219)
[function] fn panic(_info: &core::panic::PanicInfo) -> ! {
driverspace_workspace/drivers/wacomgraphic_driver/src/main.rs (line 1)
[function] fn main() {
driverspace_workspace/lib/ds-formal-ffi/build.rs (line 10)
[function] fn main() {
driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 10)
[function] fn it_works() {
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 18)
[function] fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8])
-> Result<(), DsError>;
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 21)
[function] fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) ->
Result<(), DsError>;
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 24)
[function] fn flush(&mut self) -> Result<(), DsError>;
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 27)
[function] fn geometry(&self) -> BlockGeometry;
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 30)
[function] fn supports_trim(&self) -> bool { false }
driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 33)
[function] fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 10)
[function] fn it_works() {
driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 10)
[function] fn it_works() {

```

```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 30)
[function] fn write_str(&mut self, s: &str) -> fmt::Result {
driverspace_workspace/lib/ds-log/src/transport.rs (line 8)
[function] fn uart_putchar(c: u8) {
driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 34)
[function] fn drop(&mut self) {
driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 25)
[function] fn drop(&mut self) {
driverspace_workspace/lib/kapi-abi/build.rs (line 9)
[function] fn main() {
driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 24)
[function] fn from(val: i32) -> Self {
driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 17)
[function] fn slot_ptr(&self, index: u64) -> *mut DsMsg {
driverspace_workspace/src/self_test.rs (line 21)
[function] fn test_ipc_ring() -> Result<(), Status> {
driverspace_workspace/src/self_test.rs (line 39)
[function] fn test_memory_alloc() -> Result<(), Status> {
driverspace_workspace/src/self_test.rs (line 62)
[function] fn test_capability_check() -> Result<(), Status> {
driverspacelib/src/abi.rs (line 133)
[function] fn from(v: i32) -> Self {
driverspacelib/src/driver.rs (line 15)
[function] fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 42)
[function] fn send_ipc(&self, msg: GfxIpcMsg) -> GfxIpcMsg {
gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 4)
[struct] struct Surface {
gfx_protocol_workspace/crates/gfx-server/src/main.rs (line 49)
[function] fn wait_for_ipc() -> GfxIpcMsg {
gfx_protocol_workspace/crates/gfx-server/src/main.rs (line 54)
[function] fn reply(cmd: u32, status: i32, data: u64) {
kernel-drivers/audio/src/lib.rs (line 9)
[function] fn ad_init(base: u32) -> bool;
kernel-drivers/audio/src/lib.rs (line 10)
[function] fn ad_jack_present() -> bool;
kernel-drivers/audio/src/lib.rs (line 11)
[function] fn ad_set_amp(on: bool);
kernel-drivers/audio/src/lib.rs (line 12)
[function] fn ad_play_phys(phys: u64, len: u32) -> bool;
kernel-drivers/audio/src/lib.rs (line 13)
[function] fn ad_stop();
kernel-drivers/battery/src/lib.rs (line 6)
[function] fn battery_init() -> bool;
kernel-drivers/bluetooth/src/lib.rs (line 7)
[function] fn bt_init() -> bool;
kernel-drivers/bluetooth/src/lib.rs (line 8)
[function] fn bt_ready() -> bool;

```

```

kernel-drivers/camera/src/lib.rs (line 6)
[function] fn camera_init() -> bool;
kernel-drivers/displayport/src/lib.rs (line 6)
[function] fn dp_init() -> bool;
kernel-drivers/displayport/src/lib.rs (line 7)
[function] fn dp_ready() -> bool;
kernel-drivers/hdmi/src/lib.rs (line 6)
[function] fn hdmi_init_with(fb_phys: u64, w: u32, h: u32, stride: u32) -> bool;
kernel-drivers/hdmi/src/lib.rs (line 7)
[function] fn hdmi_ready() -> bool;
kernel-drivers/pcie/src/lib.rs (line 54)
[function] fn cfg_addr(bus: u32, dev: u32, func: u32, off: u32) -> u32 {
kernel-drivers/pcie/src/lib.rs (line 152)
[function] fn probe(bus: u8, dev: u8, func: u8) -> Option<PciDevice> {
kernel-drivers/usb/src/class/hid/mod.rs (line 26)
[function] fn submit(x: &mut Xhci, kb: &mut HidKeyboard) {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 11)
[function] fn name(&self) -> &'static str;
kernel-drivers/usb/src/class/mass/blockdev.rs (line 12)
[function] fn block_size(&self) -> usize;
kernel-drivers/usb/src/class/mass/blockdev.rs (line 13)
[function] fn block_count(&self) -> u64;
kernel-drivers/usb/src/class/mass/blockdev.rs (line 14)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError>;
kernel-drivers/usb/src/class/mass/blockdev.rs (line 15)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
kernel-drivers/usb/src/class/mass/blockdev.rs (line 19)
[function] fn name(&self) -> &'static str {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 23)
[function] fn block_size(&self) -> usize {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 27)
[function] fn block_count(&self) -> u64 {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 31)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel-drivers/usb/src/class/mass/blockdev.rs (line 46)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>
{
kernel-drivers/usb/src/class/mod.rs (line 7)
[function] fn probe(&self, dev: &UsbDevice) -> bool;
kernel-drivers/usb/src/dma.rs (line 30)
[function] fn drop(&mut self) {
kernel-drivers/usb/src/host/xhci/context.rs (line 80)
[function] fn in_add_drop(&mut self, add: u32, drop: u32) {
kernel-drivers/usb/src/host/xhci/context.rs (line 89)
[function] fn slot_ptr(&mut self) -> *mut u32 {
kernel-drivers/usb/src/host/xhci/context.rs (line 93)
[function] fn ep0_ptr(&mut self) -> *mut u32 {

```



```

kernel-drivers/usb/src/host/xhci/init.rs (line 24)
[function] fn spin_wait<F: Fn() -> bool>(f: F) -> Result<(), UsbError> {
kernel-drivers/usb/src/host/xhci/init.rs (line 36)
[function] fn op_write64(regs: &XhciRegs, off: usize, v: u64) {
kernel-drivers/usb/src/host/xhci/init.rs (line 170)
[function] fn attach_port(&mut self, p: u32) {
kernel-drivers/usb/src/host/xhci/regs.rs (line 90)
[function] fn op(&self) -> *mut u8 {
kernel-drivers/usb/src/host/xhci/regs.rs (line 94)
[function] fn rt(&self) -> *mut u8 {
kernel-drivers/usb/src/host/xhci/regs.rs (line 98)
[function] fn db(&self) -> *mut u8 {
kernel_Workspace/C_base/kc-abi-menager/src/main.rs (line 2)
[function] fn main() {
kernel_Workspace/C_base/kc-abi/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/Odin_base/odin-bin-loader/src/main.rs (line 2)
[function] fn main() {
kernel_Workspace/base/kw-C-abi/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 4)
[function] fn k_net_socket(domain: u32, sock_type: u32, proto: u32) -> i32;
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 5)
[function] fn k_net_bind(fd: i32, addr: *const u8, len: u32) -> i32;
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 6)
[function] fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 7)
[function] fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;
kernel_Workspace/base/kw-base/src/api/process.rs (line 4)
[function] fn k_proc_spawn(world_id: u32, entry: u64, stack: u64) -> i64;
kernel_Workspace/base/kw-base/src/api/process.rs (line 5)
[function] fn k_proc_exit(code: i32);
kernel_Workspace/base/kw-base/src/api/process.rs (line 6)
[function] fn k_proc_yield();
kernel_Workspace/base/kw-base/src/api/process.rs (line 7)
[function] fn k_proc_get_pid() -> u32;
kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 45)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 17)
[function] fn is_transmitter_empty() -> bool {
kernel_Workspace/base/kw-base/src/core/error.rs (line 65)
[function] fn from(s: kstd_base::Status) -> Self {
kernel_Workspace/base/kw-base/src/core/error.rs (line 84)
[function] fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {

```

```

kernel_Workspace/base/kw-base/src/core/result.rs (line 6)
[function] fn into_kresult(self) -> KResult<T>;
kernel_Workspace/base/kw-base/src/core/result.rs (line 10)
[function] fn into_kresult(self) -> KResult<T> {
kernel_Workspace/base/kw-base/src/core/result.rs (line 16)
[function] fn into_kresult(self) -> KResult<T> {
kernel_Workspace/base/kw-base/src/core/traits.rs (line 5)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 6)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 7)
[function] fn block_size(&self) -> usize;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 8)
[function] fn block_count(&self) -> u64;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 12)
[function] fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 13)
[function] fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 17)
[function] fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 18)
[function] fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 19)
[function] fn mac_address(&self) -> [u8; 6];
kernel_Workspace/base/kw-base/src/core/traits.rs (line 23)
[function] fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 24)
[function] fn close(&self, handle: Handle) -> Result<(), KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 25)
[function] fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) ->
Result<usize, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 26)
[function] fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize,
KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 27)
[function] fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 43)
[function] fn schedule(&mut self) -> Option<TaskId>;
kernel_Workspace/base/kw-base/src/core/traits.rs (line 44)
[function] fn enqueue(&mut self, task: TaskId);
kernel_Workspace/base/kw-base/src/core/traits.rs (line 45)
[function] fn dequeue(&mut self, task: TaskId);
kernel_Workspace/base/kw-base/src/core/traits.rs (line 46)
[function] fn yield_current(&mut self);
kernel_Workspace/base/kw-base/src/core/traits.rs (line 50)
[function] fn enable_irq(&self, irq: u32);
kernel_Workspace/base/kw-base/src/core/traits.rs (line 51)
[function] fn disable_irq(&self, irq: u32);

```

```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 52)
[function] fn ack_irq(&self, irq: u32);
kernel_Workspace/base/kw-base/src/core/traits.rs (line 53)
[function] fn end_of_interrupt(&self);
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 4)
[function] fn k_aes256_enc(key: *const u8, iv: *const u8, data: *mut u8, len: usize)
-> bool;
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 5)
[function] fn k_aes256_dec(key: *const u8, iv: *const u8, data: *mut u8, len: usize)
-> bool;
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 13)
[function] fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()>;
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 14)
[function] fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()>;
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 23)
[function] fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 37)
[function] fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 4)
[function] fn k_sha256(data: *const u8, len: usize, out: *mut u8) -> bool;
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 5)
[function] fn k_sha512(data: *const u8, len: usize, out: *mut u8) -> bool;
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 10)
[function] fn hash(data: &[u8], out: &mut [u8]) -> KResult<()>;
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 16)
[function] fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 29)
[function] fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
kernel_Workspace/base/kw-base/src/crypto/random.rs (line 4)
[function] fn k_get_random(buf: *mut u8, len: usize) -> bool;
kernel_Workspace/base/kw-base/src/crypto/random.rs (line 5)
[function] fn k_get_hw_random(buf: *mut u8, len: usize) -> bool;
kernel_Workspace/base/kw-base/src/crypto/secure_mem.rs (line 37)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 17)
[function] fn init(&mut self) {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 29)
[function] fn is_transmitter_empty(&self) -> bool {
kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 40)
[function] fn write_str(&mut self, s: &str) -> Result<(), ()> {
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 8)
[function] fn stat(&self) -> KResult<FileStat>;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 9)
[function] fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 10)
[function] fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 11)
[function] fn lookup(&self, name: &str) -> KResult<Handle>;

```

```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 15)
[function] fn inode(&self) -> &dyn Inode;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 16)
[function] fn flags(&self) -> u32;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 20)
[function] fn name(&self) -> &str;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 21)
[function] fn root_inode(&self) -> Handle;
kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 22)
[function] fn sync(&self) -> KResult<>;
kernel_Workspace/base/kw-base/src/hal/acpi.rs (line 4)
[function] fn k_acpi_get_rsd() -> *const u8;
kernel_Workspace/base/kw-base/src/hal/acpi.rs (line 5)
[function] fn k_acpi_find_table(sig: *const u8, idx: u32) -> *const u8;
kernel_Workspace/base/kw-base/src/hal/pci.rs (line 18)
[function] fn config_addr(self, offset: u8) -> u32 {
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 4)
[function] fn k_timer_get_ticks() -> u64;
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 5)
[function] fn k_timer_get_frequency() -> u64;
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 6)
[function] fn k_timer_sleep_ms(ms: u32);
kernel_Workspace/base/kw-base/src/hal/timers.rs (line 7)
[function] fn k_timer_set_oneshot(ns: u64) -> bool;
kernel_Workspace/base/kw-base/src/hal/usb.rs (line 4)
[function] fn k_usb_init() -> bool;
kernel_Workspace/base/kw-base/src/hal/usb.rs (line 5)
[function] fn k_usb_enum_devices() -> u32;
kernel_Workspace/base/kw-base/src/hal/usb.rs (line 6)
[function] fn k_usb_get_device_count() -> u32;
kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 4)
[function] fn k_ipc_send(dst: u32, a0: u64, a1: u64) -> i32;
kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 5)
[function] fn k_ipc_recv(a0: *mut u64, a1: *mut u64) -> i32;
kernel_Workspace/base/kw-base/src/ipc/shared_mem.rs (line 32)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 37)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/mem/frame.rs (line 35)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 14)
[function] fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
kernel_Workspace/base/kw-base/src/mem/paging.rs (line 15)
[function] fn paging_unmap_page(virt: u64) -> bool;
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 4)
[function] fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) ->
*mut u8;
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 5)
[function] fn slab_cache_destroy(cache: *mut u8);

```

```

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 6)
[function] fn slab_alloc(cache: *mut u8) -> *mut u8;
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 7)
[function] fn slab_free(cache: *mut u8, ptr: *mut u8);
kernel_Workspace/base/kw-base/src/mem/slab.rs (line 45)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/sync/futex.rs (line 4)
[function] fn k_futex_wait(uaddr: *const u32, expected_val: u32, timeout_ns: u64) ->
i32;
kernel_Workspace/base/kw-base/src/sync/futex.rs (line 5)
[function] fn k_futex_wake(uaddr: *const u32, wake_count: u32) -> i32;
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 49)
[function] fn unlock(&self) {
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 61)
[function] fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 65)
[function] fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() }
}
kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 69)
[function] fn drop(&mut self) { self.lock.unlock(); }
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 48)
[function] fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 51)
[function] fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 57)
[function] fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 60)
[function] fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() }
}
kernel_Workspace/base/kw-base/src/sync/rwlock.rs (line 63)
[function] fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
kernel_Workspace/base/kw-base/src/task/process.rs (line 48)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 19)
[function] fn word_idx(&self, bit: usize) -> usize {
kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 24)
[function] fn bit_mask(&self, bit: usize) -> u64 {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 9)
[struct] struct Node<K, V> {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 18)
[function] fn new_leaf() -> *mut Self {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 29)
[function] fn new_internal() -> *mut Self {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 69)
[function] fn split_child(&mut self, parent: *mut Node<K, V>, i: usize) {
kernel_Workspace/base/kw-base/src/utils/btree.rs (line 102)
[function] fn insert_non_full(&mut self, node: *mut Node<K, V>, key: K, val: V) {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 6)
[struct] struct FnvHasher(u64);

```

```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 8)
[function] fn finish(&self) -> u64 { self.0 }
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 9)
[function] fn write(&mut self, bytes: &[u8]) {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 17)
[function] fn hash_key<K: Hash>(key: &K) -> usize {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 39)
[function] fn grow(&mut self, new_cap: usize) {
kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 138)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-base/src/utils/ring_buffer.rs (line 46)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 8)
[struct] struct SlabPage {
kernel_Workspace/base/kw-lib/src/mem/allocators/slab.rs (line 45)
[function] fn alloc_page(&self) -> KResult<*mut SlabPage> {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 9)
[struct] struct FreeList {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 78)
[function] fn frame_to_phys(&self, frame: usize) -> PhysAddr {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 82)
[function] fn phys_to_frame(&self, phys: PhysAddr) -> usize {
kernel_Workspace/base/kw-lib/src/mem/physical/buddy.rs (line 86)
[function] fn free_block(&mut self, frame: usize, order: usize) {
kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 67)
[function] fn drop(&mut self) {
kernel_Workspace/base/kw-odin-abi/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/core/gluecore/build.rs (line 1)
[function] fn main() {}
kernel_Workspace/core/gluecore/src/bindings.rs (line 12)
[function] fn from(val: CPhysRange) -> Self {
kernel_Workspace/core/gluecore/src/boot.rs (line 4)
[function] fn boot_get_memory_map(out_ptr: *mut u64, out_size: *mut usize) -> bool;
kernel_Workspace/core/gluecore/src/boot.rs (line 5)
[function] fn boot_get_cmdline(out_ptr: *mut u64, out_len: *mut usize) -> bool;
kernel_Workspace/core/gluecore/src/boot.rs (line 6)
[function] fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut
u32, out_pitch: *mut u32) -> bool;
kernel_Workspace/core/gluecore/src/context.rs (line 4)
[function] fn paging_read_cr3() -> u64;
kernel_Workspace/core/gluecore/src/context.rs (line 5)
[function] fn paging_write_cr3(pml4_phys: u64);
kernel_Workspace/core/gluecore/src/context.rs (line 6)
[function] fn paging_flush_tlb_all();
kernel_Workspace/core/gluecore/src/context.rs (line 7)
[function] fn paging_flush_page(addr: u64);
kernel_Workspace/core/gluecore/src/context.rs (line 8)
[function] fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);

```

```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 38)
[function] fn drop(&mut self) {
kernel_Workspace/core/gluecore/src/syscalls.rs (line 4)
[function] fn syscall_dispatch(num: u64, arg1: u64, arg2: u64, arg3: u64, arg4: u64)
-> i64;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 5)
[function] fn kmalloc(size: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 6)
[function] fn kcalloc(size: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 7)
[function] fn kcalloc(n: usize, size: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 8)
[function] fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 9)
[function] fn kfree(ptr: *mut u8);
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 10)
[function] fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 16)
[function] fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 29)
[function] fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 35)
[function] fn realloc(&self, ptr: *mut u8, _old: Layout, new_size: usize) ->
Result<*mut u8, Status> {
kernel_Workspace/core/kstd_alloc/src/lib.rs (line 15)
[function] fn oom() -> ! {
kernel_Workspace/core/kstd_alloc/src/page.rs (line 5)
[function] fn kalloc_pages(order: usize) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/page.rs (line 6)
[function] fn kfree_pages(ptr: *mut u8, pages: usize);
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 5)
[function] fn slab_alloc(cache: *mut u8) -> *mut u8;
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 6)
[function] fn slab_free(cache: *mut u8, ptr: *mut u8);
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 7)
[function] fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) ->
*mut u8;
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 8)
[function] fn slab_cache_destroy(cache: *mut u8);
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 31)
[function] fn alloc(&self, _layout: Layout) -> Result<*mut u8, Status> {
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 36)
[function] fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
kernel_Workspace/core/kstd_alloc/src/slab.rs (line 44)
[function] fn drop(&mut self) {
kernel_Workspace/core/kstd_alloc/src/traits.rs (line 4)
[function] fn alloc(&self, layout: Layout) -> Result<*mut u8, Status>;
kernel_Workspace/core/kstd_alloc/src/traits.rs (line 5)
[function] fn dealloc(&self, ptr: *mut u8, layout: Layout);

```



```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 6)
[function] fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) ->
Result<*mut u8, Status> {
kernel_Workspace/core/kstd_base/src/errors.rs (line 20)
[function] fn from(val: u32) -> Self {
kernel_Workspace/core/kstd_base/src/errors.rs (line 40)
[function] fn from(val: Status) -> Self {
kernel_Workspace/core/kstd_base/src/primitives.rs (line 39)
[function] fn bitor(self, rhs: Self) -> Self {
kernel_Workspace/core/kstd_base/src/trails.rs (line 4)
[function] fn align_up(self) -> Self;
kernel_Workspace/core/kstd_base/src/trails.rs (line 5)
[function] fn align_down(self) -> Self;
kernel_Workspace/core/kstd_base/src/trails.rs (line 6)
[function] fn is_aligned(self) -> bool;
kernel_Workspace/core/kstd_base/src/trails.rs (line 11)
[function] fn align_up(self) -> Self {
kernel_Workspace/core/kstd_base/src/trails.rs (line 16)
[function] fn align_down(self) -> Self {
kernel_Workspace/core/kstd_base/src/trails.rs (line 21)
[function] fn is_aligned(self) -> bool {
kernel_Workspace/core/kstd_base/src/trails.rs (line 28)
[function] fn align_up(self) -> Self {
kernel_Workspace/core/kstd_base/src/trails.rs (line 33)
[function] fn align_down(self) -> Self {
kernel_Workspace/core/kstd_base/src/trails.rs (line 38)
[function] fn is_aligned(self) -> bool {
kernel_Workspace/core/kstd_base/src/trails.rs (line 44)
[function] fn read_u8(&self, port: u16) -> u8;
kernel_Workspace/core/kstd_base/src/trails.rs (line 45)
[function] fn write_u8(&self, port: u16, val: u8);
kernel_Workspace/core/kstd_base/src/trails.rs (line 46)
[function] fn read_u16(&self, port: u16) -> u16;
kernel_Workspace/core/kstd_base/src/trails.rs (line 47)
[function] fn write_u16(&self, port: u16, val: u16);
kernel_Workspace/core/kstd_base/src/trails.rs (line 48)
[function] fn read_u32(&self, port: u16) -> u32;
kernel_Workspace/core/kstd_base/src/trails.rs (line 49)
[function] fn write_u32(&self, port: u16, val: u32);
kernel_Workspace/core/kstd_base/src/trails.rs (line 53)
[function] fn read_volatile_u32(&self, offset: usize) -> u32;
kernel_Workspace/core/kstd_base/src/trails.rs (line 54)
[function] fn write_volatile_u32(&self, offset: usize, val: u32);
kernel_Workspace/core/kstd_base/src/trails.rs (line 55)
[function] fn read_volatile_u64(&self, offset: usize) -> u64;
kernel_Workspace/core/kstd_base/src/trails.rs (line 56)
[function] fn write_volatile_u64(&self, offset: usize, val: u64);
kernel_Workspace/core/kstd_base/src/types.rs (line 23)
[function] fn default() -> Self {

```

```

kernel_workspace/core/kstd_core/src/panic.rs (line 5)
[function] fn write_all(serial: &mut kstd_io::Serial, vga: &mut kstd_io::VgaConsole,
s: &str) {
kernel_workspace/core/kstd_core/src/panic.rs (line 34)
[struct] struct FmtWriter<'a, W: Write>(&'a mut W);
kernel_workspace/core/kstd_core/src/panic.rs (line 37)
[function] fn write_str(&mut self, s: &str) -> core::fmt::Result {
kernel_workspace/core/kstd_core/src/panic.rs (line 45)
[function] fn heapless_msg(message: core::panic::PanicMessage) ->
kstd_data::string::String {
kernel_workspace/core/kstd_core/src/panic.rs (line 52)
[struct] struct FmtMsg<'a>(&'a mut kstd_data::string::String);
kernel_workspace/core/kstd_core/src/panic.rs (line 55)
[function] fn write_str(&mut self, s: &str) -> core::fmt::Result {
kernel_workspace/core/kstd_core/src/sync.rs (line 45)
[function] fn deref(&self) -> &T {
kernel_workspace/core/kstd_core/src/sync.rs (line 51)
[function] fn deref_mut(&mut self) -> &mut T {
kernel_workspace/core/kstd_core/src/sync.rs (line 57)
[function] fn drop(&mut self) {
kernel_workspace/core/kstd_core/src/task.rs (line 4)
[function] fn sched_create_task(world_id: u32, flags: u32, out_task_id: *mut u64) ->
bool;
kernel_workspace/core/kstd_core/src/task.rs (line 5)
[function] fn sched_destroy_task(task_id: u64) -> bool;
kernel_workspace/core/kstd_core/src/task.rs (line 6)
[function] fn sched_yield() -> bool;
kernel_workspace/core/kstd_core/src/task.rs (line 7)
[function] fn sched_get_current_task() -> u64;
kernel_workspace/core/kstd_core/src/thread.rs (line 4)
[function] fn thread_create(task_id: u64, entry: u64, stack_ptr: u64, arg: u64,
out_tid: *mut u64) -> bool;
kernel_workspace/core/kstd_core/src/thread.rs (line 5)
[function] fn thread_exit() -> !;
kernel_workspace/core/kstd_core/src/thread.rs (line 6)
[function] fn thread_get_current_id() -> u64;
kernel_workspace/core/kstd_core/src/time.rs (line 2)
[function] fn k_tick() -> u64;
kernel_workspace/core/kstd_data/src/map.rs (line 4)
[struct] struct FnvHasher(u64);
kernel_workspace/core/kstd_data/src/map.rs (line 6)
[function] fn finish(&self) -> u64 { self.0 }
kernel_workspace/core/kstd_data/src/map.rs (line 7)
[function] fn write(&mut self, bytes: &[u8]) {
kernel_workspace/core/kstd_data/src/map.rs (line 27)
[function] fn hash_key(key: &K) -> usize {
kernel_workspace/core/kstd_data/src/map.rs (line 33)
[function] fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }
kernel_workspace/core/kstd_data/src/map.rs (line 91)
[function] fn resize(&mut self, new_cap: usize) {

```

```

kernel_Workspace/core/kstd_data/src/string.rs (line 29)
[function] fn deref(&self) -> &str { self.as_str() }
kernel_Workspace/core/kstd_data/src/string.rs (line 33)
[function] fn write_str(&mut self, s: &str) -> fmt::Result {
kernel_Workspace/core/kstd_data/src/tree.rs (line 3)
[struct] struct Node<K, V> {
kernel_Workspace/core/kstd_data/src/tree.rs (line 20)
[function] fn alloc_node(&mut self, key: K, val: V) -> usize {
kernel_Workspace/core/kstd_data/src/tree.rs (line 30)
[function] fn free_node(&mut self, idx: usize) {
kernel_Workspace/core/kstd_data/src/tree.rs (line 34)
[function] fn height(&self, idx: usize) -> i32 {
kernel_Workspace/core/kstd_data/src/tree.rs (line 38)
[function] fn update_height(&mut self, idx: usize) {
kernel_Workspace/core/kstd_data/src/tree.rs (line 44)
[function] fn balance_factor(&self, idx: usize) -> i32 {
kernel_Workspace/core/kstd_data/src/tree.rs (line 49)
[function] fn rotate_right(&mut self, y: usize) -> usize {
kernel_Workspace/core/kstd_data/src/tree.rs (line 59)
[function] fn rotate_left(&mut self, x: usize) -> usize {
kernel_Workspace/core/kstd_data/src/tree.rs (line 69)
[function] fn balance(&mut self, idx: usize) -> usize {
kernel_Workspace/core/kstd_data/src/tree.rs (line 89)
[function] fn insert_rec(&mut self, idx: usize, key: K, val: V) -> usize {
kernel_Workspace/core/kstd_data/src/vec.rs (line 24)
[function] fn grow_to(&mut self, new_cap: usize) {
kernel_Workspace/core/kstd_data/src/vec.rs (line 87)
[function] fn deref(&self) -> &[T] { self.as_slice() }
kernel_Workspace/core/kstd_data/src/vec.rs (line 91)
[function] fn deref_mut(&mut self) -> &mut [T] { self.as_mut_slice() }
kernel_Workspace/core/kstd_data/src/vec.rs (line 95)
[function] fn drop(&mut self) {
kernel_Workspace/core/kstd_io/src/buffer.rs (line 33)
[function] fn write_str(&mut self, s: &str) -> Result<(), ()> {
kernel_Workspace/core/kstd_io/src/console.rs (line 4)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/core/kstd_io/src/console.rs (line 5)
[function] fn serial_write_str(s: *const u8);
kernel_Workspace/core/kstd_io/src/console.rs (line 10)
[function] fn write_str(&mut self, s: &str) -> Result<(), ()> {
kernel_Workspace/core/kstd_io/src/console.rs (line 26)
[function] fn write_str(&mut self, s: &str) -> Result<(), ()> {
kernel_Workspace/core/kstd_io/src/input.rs (line 2)
[function] fn k_input_key() -> i32;
kernel_Workspace/core/kstd_io/src/input.rs (line 3)
[function] fn k_input_keycode() -> u32;
kernel_Workspace/core/kstd_io/src/input.rs (line 4)
[function] fn kbuf_pop() -> i32;

```

```

kernel_Workspace/core/kstd_io/src/input.rs (line 24)
[function] fn read_byte(&mut self) -> Option<u8> {
kernel_Workspace/core/kstd_io/src/input.rs (line 31)
[function] fn read_byte(&mut self) -> Option<u8> {
kernel_Workspace/core/kstd_io/src/traits.rs (line 2)
[function] fn write_str(&mut self, s: &str) -> Result<(), ()>;
kernel_Workspace/core/kstd_io/src/traits.rs (line 4)
[function] fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
kernel_Workspace/core/kstd_io/src/traits.rs (line 23)
[function] fn read_byte(&mut self) -> Option<u8>;
kernel_Workspace/core/linux_abi_driverspace/src/devmodel.rs (line 11)
[function] fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
kernel_Workspace/core/linux_abi_driverspace/src/devmodel.rs (line 12)
[function] fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
kernel_Workspace/core/windows-com/src/lib.rs (line 11)
[function] fn it_works() {
kernel_Workspace/kernel-bin/build.rs (line 13)
[function] fn main() {
kernel_Workspace/kernel-bin/src/main.rs (line 17)
[function] fn kernel_main(boot_info: &'static BootInfo) -> ! {
kernel_Workspace/kernel/build.rs (line 6)
[function] fn collect_c_files(dir: &Path, out: &mut Vec<PathBuf>) {
kernel_Workspace/kernel/build.rs (line 23)
[function] fn clang_target(arch: &str) -> &'static str {
kernel_Workspace/kernel/build.rs (line 33)
[function] fn main() {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 9)
[struct] struct HeapSpace([u8; HEAP_SIZE]);
kernel_Workspace/kernel/src/arch/riscv64.rs (line 13)
[struct] struct Bump {
kernel_Workspace/kernel/src/arch/riscv64.rs (line 60)
[function] fn riscv_panic(info: &PanicInfo) -> ! {
kernel_Workspace/kernel/src/arch/x86_64.rs (line 5)
[function] fn x86_panic(info: &PanicInfo) -> ! {
kernel_Workspace/kernel/src/battery/bridge.rs (line 5)
[function] fn battery_status_packed(a0: *mut u64, a1: *mut u64, a2: *mut u64) ->
bool;
kernel_Workspace/kernel/src/battery/bridge.rs (line 6)
[function] fn battery_set_threshold(pct: u32) -> bool;
kernel_Workspace/kernel/src/battery/init.rs (line 2)
[function] fn battery_init() -> bool;
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 7)
[function] fn bt_init() -> bool;
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 8)
[function] fn bt_ready() -> bool;
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 9)
[function] fn bt_info(ver: *mut u8, bdaddr: *mut u8);
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 10)
[function] fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;

```

```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 11)
[function] fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 12)
[function] fn bt_acl_send(data: *const u8, len: u16) -> bool;
kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 13)
[function] fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
kernel_Workspace/kernel/src/bluetooth/init.rs (line 2)
[function] fn bt_init() -> bool;
kernel_Workspace/kernel/src/bluetooth/init.rs (line 3)
[function] fn bt_ready() -> bool;
kernel_Workspace/kernel/src/camera/bridge.rs (line 5)
[function] fn camera_caps_get(w: *mut u32, h: *mut u32, fmt: *mut u32, fps: *mut
u32) -> bool;
kernel_Workspace/kernel/src/camera/bridge.rs (line 6)
[function] fn camera_start() -> bool;
kernel_Workspace/kernel/src/camera/bridge.rs (line 7)
[function] fn camera_stop() -> bool;
kernel_Workspace/kernel/src/camera/bridge.rs (line 8)
[function] fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
kernel_Workspace/kernel/src/camera/bridge.rs (line 11)
[function] fn grant_phys(va: u64) -> Option<u64> {
kernel_Workspace/kernel/src/camera/init.rs (line 2)
[function] fn camera_init() -> bool;
kernel_Workspace/kernel/src/caps/audit.rs (line 27)
[struct] struct AuditInner {
kernel_Workspace/kernel/src/caps/audit.rs (line 52)
[function] fn now_tick() -> u64 {
kernel_Workspace/kernel/src/caps/audit.rs (line 56)
[function] fn push(kind: EventKind, world: u32, target: u32, cap: Capability) {
kernel_Workspace/kernel/src/caps/audit.rs (line 142)
[function] fn test_audit() {
kernel_Workspace/kernel/src/caps/check.rs (line 26)
[function] fn current_world_id() -> u32 {
kernel_Workspace/kernel/src/caps/check.rs (line 147)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/caps/check.rs (line 158)
[function] fn test_require() {
kernel_Workspace/kernel/src/caps/export.rs (line 7)
[function] fn cap_from_id(id: u8) -> Option<Capability> {
kernel_Workspace/kernel/src/caps/grant.rs (line 58)
[struct] struct TempGrant {
kernel_Workspace/kernel/src/caps/grant.rs (line 74)
[function] fn now_tick() -> u64 {
kernel_Workspace/kernel/src/caps/grant.rs (line 121)
[function] fn test_inherit() {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 132)
[function] fn test_hierarchy() {
kernel_Workspace/kernel/src/caps/hierarchy.rs (line 149)
[function] fn test_depth() {

```

```

kernel_Workspace/kernel/src/caps/policy.rs (line 14)
[function] fn policy_allows(world: u32, cap: Capability) -> bool {
kernel_Workspace/kernel/src/caps/revoke.rs (line 68)
[function] fn test_revoke() {
kernel_Workspace/kernel/src/caps/sets.rs (line 162)
[function] fn test_presets() {
kernel_Workspace/kernel/src/caps/sets.rs (line 174)
[function] fn test_effective() {
kernel_Workspace/kernel/src/caps/store.rs (line 22)
[struct] struct StoreInner {
kernel_Workspace/kernel/src/caps/store.rs (line 71)
[function] fn find_world_inner(s: &StoreInner, world_id: u32) -> Result<WorldCaps,
&'static str> {
kernel_Workspace/kernel/src/caps/store.rs (line 192)
[function] fn test_world_registration() {
kernel_Workspace/kernel/src/caps/syscalls.rs (line 13)
[function] fn cap_from_id(id: u8) -> Option<Capability> {
kernel_Workspace/kernel/src/caps/tests.rs (line 8)
[function] fn test_full_flow() {
kernel_Workspace/kernel/src/caps/tests.rs (line 29)
[function] fn test_hierarchy_enforcement() {
kernel_Workspace/kernel/src/caps/tests.rs (line 40)
[function] fn test_global_revocation() {
kernel_Workspace/kernel/src/caps/tests.rs (line 54)
[function] fn test_audit_trail() {
kernel_Workspace/kernel/src/caps/types.rs (line 221)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/caps/types.rs (line 240)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/caps/types.rs (line 256)
[function] fn test_set_operations() {
kernel_Workspace/kernel/src/cpu/acpi.rs (line 43)
[function] fn checksum(bytes: &[u8]) -> bool {
kernel_Workspace/kernel/src/cpu/lapic.rs (line 20)
[function] fn x2apic_msr(reg: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/riscv.rs (line 26)
[function] fn system_reset(reset_type: u32) -> ! {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_switch.rs (line 716)
[function] fn test_fpu_save_area_size() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_switch.rs (line 722)
[function] fn test_satp_constants() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_switch.rs (line 730)
[function] fn test_sstatus_fs_mask() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/risc_v/context_switch.rs (line 739)
[function] fn test_satp_mode_name() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_switch.rs (line 45)
[struct] struct XSaveArea {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_switch.rs (line 56)
[struct] struct ExtendedContext {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 458)
[struct] struct TssStruct {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 769)
[function] fn test_xsave_area_alignment() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 774)
[function] fn test_msr_constants() {
kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 781)
[function] fn test_cr3_masks() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 69)
[function] fn cpu_to_apic_id(cpu: u32) -> Option<u32> {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 175)
[function] fn ns_from_tsc(tsc_delta: u64, hz: u64) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 217)
[function] fn timer_count_for_hz(hz: u64) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 225)
[function] fn cmp_max_1(v: u32) -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 233)
[function] fn default_timer_count() -> u32 {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 369)
[struct] struct IdtDescriptor {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 437)
[function] fn arch_context_switch(prev_ctx: *mut CpuContext, next_ctx: *const CpuContext);
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 462)
[function] fn maybe_balance(cpu: u32, registry: &[*mut RunQueue; MAX_CPUS]) {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 587)
[function] fn registry_defaults_to_null_for_untouched_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 593)
[function] fn apic_id_table_defaults_to_none_then_roundtrips() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 600)
[function] fn register_runqueue_ignores_out_of_range_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 605)
[function] fn ns_from_tsc_is_zero_without_calibration() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 610)
[function] fn ns_from_tsc_scales_correctly() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 618)
[function] fn timer_count_for_hz_has_sane_fallback() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 624)
[function] fn timer_count_for_hz_never_returns_zero() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 629)
[function] fn tlb_pending_flag_sets_and_clears() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 637)
[function] fn idt_entry_encodes_and_reports_present() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 645)
[function] fn idt_entry_missing_is_not_present() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 651)
[function] fn cpuid_leaf_zero_reports_a_nonzero_max_leaf() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 660)
[function] fn rdtsc_does_not_go_backwards_across_two_reads() {

```



```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 670)
[function] fn cpuid_and_rdtsc_are_zero_off_x86_64() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 678)
[function] fn set_and_read_tsc_frequency_is_stable_within_this_test() {
kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 684)
[function] fn booted_cpu_count_starts_at_or_above_zero() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 514)
[function] fn make_idle(pid: TaskId) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 520)
[function] fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 526)
[function] fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 531)
[function] fn chain_is_ordered_stop_dl_rt_fair_idle() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 545)
[function] fn class_of_maps_every_variant_correctly() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 554)
[function] fn pick_next_walks_the_chain_in_priority_order() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 596)
[function] fn rt_get_rr_interval_is_nonzero_only_for_round_robin() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 606)
[function] fn fair_and_deadline_and_idle_have_zero_rr_interval() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 618)
[function] fn check_preempt_cross_class_ignores_intra_class_ops() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 639)
[function] fn switched_to_reschedules_when_new_class_outranks_current() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 664)
[function] fn change_task_class_moves_task_between_underlying_queues() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 688)
[function] fn change_task_class_on_non_queued_task_does_not_touch_queues() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 706)
[function] fn rt_task_tick_only_requeues_round_robin_on_slice_expiry() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 727)
[function] fn rt_task_tick_marks_resched_when_round_robin_slice_expires() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 750)
[function] fn dl_task_tick_dequeues_when_throttled() {
kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 776)
[function] fn fair_charge_dispatches_to_underlying_rqfair() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 168)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 182)
[function] fn next(&mut self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 340)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 350)
[function] fn words_for_bits_rounds_up() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 358)
[function] fn set_clear_test_roundtrip() {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 369)
[function] fn out_of_range_access_is_a_safe_noop() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 377)
[function] fn weight_counts_set_bits_across_words() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 387)
[function] fn find_first_set_crosses_word_boundary() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 394)
[function] fn find_first_zero_skips_full_words() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 401)
[function] fn find_first_zero_none_when_full() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 408)
[function] fn find_next_set_after_given_bit() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 417)
[function] fn set_range_sets_contiguous_span() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 429)
[function] fn boolean_ops_behave_as_expected() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 446)
[function] fn iter_yields_set_bits_in_order() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 460)
[function] fn slice_bitmap_respects_logical_bit_count_not_word_count() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 472)
[function] fn slice_bitmap_clear_all_resets_backing_storage() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 481)
[function] fn atomic_bitmap_test_and_set_reports_previous_state() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 489)
[function] fn atomic_bitmap_test_and_clear_reports_previous_state() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 498)
[function] fn atomic_bitmap_find_first_zero_and_set_claims_sequentially() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 507)
[function] fn atomic_bitmap_find_first_zero_and_set_returns_none_when_full() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 517)
[function] fn atomic_bitmap_weight_matches_manual_count() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 526)
[function] fn atomic_bitmap_out_of_range_set_is_harmless() {
kernel_Workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 20)
[function] fn plist_prio(node: *mut TaskStruct) -> i32 {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 23)
[function] fn rb_parent(node: *mut TaskStruct) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 29)
[function] fn rb_color(node: *mut TaskStruct) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 35)
[function] fn rb_is_red(node: *mut TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 40)
[function] fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct,
color: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 48)
[function] fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 54)
[function] fn rb_set_color(node: *mut TaskStruct, color: usize) {

```

```
kernel_Workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 116)
[function] fn rb_insert_fixup(&mut self, mut node: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 34)
[function] fn bit_is_set(&self, prio: usize) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 116)
[function] fn hash_pid(pid: TaskId) -> usize { (pid as usize) % MAX_TASKS }
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 117)
[function] fn hash_tgid(tgid: TaskId) -> usize { (tgid as usize) % TGID_SLOTS }
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 287)
[function] fn next(&mut self) -> Option<Self::Item> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 319)
[function] fn next(&mut self) -> Option<Self::Item> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 350)
[function] fn next(&mut self) -> Option<Self::Item> {
kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 375)
[function] fn next(&mut self) -> Option<Self::Item> {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 240)
[function] fn make_idle(pid: TaskId) -> TaskStructT {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 246)
[function] fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStructT {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 252)
[function] fn ptr_of(t: &mut TaskStructT) -> *mut TaskStructT {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 257)
[function] fn two_consecutive_allocations_are_distinct() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 266)
[function] fn reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 278)
[function] fn reserve_pid_rejects_out_of_range() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 283)
[function] fn boot_bsp_registers_apic_and_runqueue() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 294)
[function] fn wake_up_new_task_enqueues_and_marks_runnable() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 312)
[function] fn exit_bookkeeping_dequeues_and_marks_zombie() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 332)
[function] fn exit_bookkeeping_on_unqueued_task_returns_null_rq() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 342)
[function] fn reap_zombie_destroys_and_frees_pid() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 358)
[function] fn maybe_reschedule_is_noop_when_not_needed() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 369)
[function] fn sys_nice_clamps_to_valid_range() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 380)
[function] fn sys_nice_rejects_non_fair_policy() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 388)
[function] fn sys_sched_setscheduler_moves_task_to_new_class() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 407)
[function] fn sys_sched_rr_get_interval_matches_class_dispatch() {
```

```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 417)
[function] fn sys_sched_affinity_roundtrips() {
kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 427)
[function] fn priority_range_helpers_match_posix_expectations() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 189)
[function] fn insert_fixup(root: &mut *mut TaskStruct, mut z: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 293)
[function] fn delete_fixup(root: &mut *mut TaskStruct, mut x: *mut TaskStruct, mut
x_parent: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 423)
[function] fn check(node: *mut TaskStruct, key_of: KeyOf) -> Result<usize, &'static
str> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 452)
[function] fn fair_key_of(node: *const TaskStruct) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 466)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 486)
[function] fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 563)
[function] fn dl_key_of(node: *const TaskStruct) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 579)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 607)
[function] fn task_bw(task: *const TaskStruct) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 714)
[function] fn set_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 718)
[function] fn clear_bit(&mut self, prio: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 722)
[function] fn sched_find_first_bit(&self) -> Option<usize> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 920)
[function] fn now(&self) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 934)
[function] fn set_current(&self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1001)
[function] fn nr_uninterruptible_dec_if(&self, task: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1202)
[function] fn fair_max_vruntime(&self) -> Option<u64> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1219)
[function] fn fair_root_for_test(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1223)
[function] fn fair_root_internal(&self) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1246)
[function] fn fmt(&self, f: &mut core::fmt::Formatter<'>) -> core::fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1294)
[function] fn can_migrate_task(task: *const TaskStruct, dst_cpu: u32) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1489)
[function] fn make_idle(pid: TaskId, cpu: u32) -> TaskStruct {

```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1496)
[function] fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1502)
[function] fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1507)
[function] fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1537)
[function] fn fair_tree_inorder_traversal_is_sorted_by_vruntime() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1566)
[function] fn fair_tree_survives_random_removals_keeping_invariants() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1588)
[function] fn deadline_tree_orders_by_earliest_deadline() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1609)
[function] fn rt_queue_fifo_order_at_same_priority_without_cycles() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1648)
[function] fn rt_queue_picks_highest_priority_across_levels() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1667)
[function] fn rt_queue_requeue_moves_task_to_back_of_its_level() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1689)
[function] fn
pick_next_task_respects_stop_over_deadline_over_rt_over_fair_over_idle() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1739)
[function] fn pick_next_task_never_returns_null_even_when_fully_empty() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1754)
[function] fn deadline_task_is_picked_even_before_its_deadline_has_passed() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1783)
[function] fn min_vruntime_never_decreases_across_updates() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1797)
[function] fn newly_woken_task_does_not_start_before_min_vruntime() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1817)
[function] fn higher_class_always_preempts_lower_class() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1842)
[function] fn fair_preemption_requires_minimum_granularity() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1860)
[function] fn deadline_preempts_via_earliest_deadline_first() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1874)
[function] fn deadline_task_gets_throttled_when_budget_is_exhausted() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1892)
[function] fn deadline_task_is_replenished_after_period_rollover() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1909)
[function] fn deadline_admission_control_rejects_overcommitted_bandwidth() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1924)
[function] fn activate_and_deactivate_round_trip_updates_bookkeeping() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1951)
[function] fn double_enqueue_is_rejected_in_debug_builds() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1966)
[function] fn select_task_rq_prefers_idle_cpu_with_cache_affinity() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1985)
[function] fn select_task_rq_picks_least_loaded_when_no_affinity_hit() {
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2011)
[function] fn wake_up_process_transitions_state_and_enqueues_remotely() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2030)
[function] fn wake_up_process_is_idempotent_when_already_queued() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2049)
[function] fn idle_balance_pulls_one_task_from_busy_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2081)
[function] fn idle_balance_does_nothing_when_balanced() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2105)
[function] fn migrated_task_gets_updated_cpu_and_rq_pointer() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2140)
[function] fn no_migration_respects_cpus_allowed_mask() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2169)
[function] fn yield_task_pushes_vruntime_to_back_of_fair_queue() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2192)
[function] fn schedule_tail_records_voluntary_vs_involuntary_switch() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2218)
[function] fn clock_advances_and_update_curr_charges_real_delta() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2243)
[function] fn update_curr_is_a_noop_on_idle_task() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2257)
[function] fn lower_nice_task_accumulates_vruntime_slower() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2281)
[function] fn load_balance_moves_task_between_two_uneven_queues() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2310)
[function] fn load_balance_is_noop_below_imbalance_threshold() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 265)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 315)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 332)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 120)
[function] fn clone(&self) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 200)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 239)
[function] fn from(policy: SchedPolicy) -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 358)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 371)
[function] fn next(&mut self) -> Option<u32> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 463)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 469)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 493)
[function] fn default() -> Self {

```



```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 647)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 682)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 720)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 744)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 824)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 955)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1066)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1089)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1407)
[function] fn set_state_unchecked(&self, new_state: TaskState) {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1429)
[function] fn set_state_owned(&mut self, new_state: TaskState) -> Result<(),
TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1578)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1592)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1761)
[function] fn nice_to_weight_is_monotonically_decreasing() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1771)
[function] fn nice_zero_has_default_weight() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1776)
[function] fn nice_to_weight_clamps_out_of_range() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1782)
[function] fn weight_to_nice_round_trip_is_close() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1790)
[function] fn calc_delta_fair_is_identity_at_nice_zero() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1796)
[function] fn calc_delta_fair_grows_for_lower_weight() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1803)
[function] fn cpumask_basic_set_clear() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1818)
[function] fn cpumask_all_contains_every_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1826)
[function] fn cpumask_intersects() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1837)
[function] fn cpumask_and_or() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1850)
[function] fn cpumask_next_after_wraps() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1859)
[function] fn cpumask_first_n() {
```



```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1867)
[function] fn task_state_valid_transitions() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1875)
[function] fn task_state_invalid_transitions_are_rejected() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1882)
[function] fn default_rlimits_have_sane_stack_and_nofile() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1890)
[function] fn credentials_kernel_has_full_capabilities() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1897)
[function] fn credentials_user_has_no_extra_capabilities() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1904)
[function] fn signal_state_pending_respects_blocked_mask() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1915)
[function] fn list_head_insert_and_remove_roundtrip() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1943)
[function] fn list_head_remove_never_creates_a_cycle_with_three_nodes() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1967)
[function] fn spinlock_lock_unlock_cycle() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1977)
[function] fn spinlock_try_lock_fails_when_held() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1987)
[function] fn spinlock_irqsave_guard_unlocks_on_drop() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1997)
[function] fn spinlock_plain_guard_unlocks_on_drop() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2007)
[function] fn atomic_task_flags_insert_remove_are_bitwise() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2020)
[function] fn atomic_task_flags_test_and_set_is_edge_triggered() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2027)
[function] fn rb_parent_color_encoding_roundtrips() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2038)
[function] fn fx_save_area_default_is_zeroed() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2046)
[function] fn cpu_context_default_has_no_valid_fpu_state() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2052)
[function] fn sched_policy_maps_to_expected_class() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2063)
[function] fn sched_class_ordering_matches_pick_next_priority() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2071)
[function] fn default_time_slice_matches_policy_expectations() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2078)
[function] fn load_avg_accumulate_grows_then_decays() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2087)
[function] fn bare_task_for_state_tests() -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2132)
[function] fn set_state_via_shared_reference_does_not_need_mut() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2142)
[function] fn set_state_rejects_illegal_transition_via_shared_reference() {
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2150)
[function] fn wake_up_process_pattern_from_remote_cpu_context() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2154)
[function] fn remote_wake(t: &TaskStruct) -> Result<(), TaskError> {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2163)
[function] fn rq_backpointer_roundtrips() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2172)
[function] fn need_resched_flag_is_settable_from_shared_reference() {
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2182)
[function] fn rt_sched_entity_starts_with_initialized_list_and_no_queued_prio() {
kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 160)
[function] fn sort_states(&mut self) {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 13)
[struct] struct ActiveBalanceArg {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 84)
[function] fn active_balance_is_noop_with_zero_imbalance() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 95)
[function] fn active_balance_is_noop_when_target_equals_busiest() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 244)
[function] fn load_calculation_defaults() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 251)
[function] fn group_classification_idle() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 260)
[function] fn group_classification_has_idle_beats_overloaded() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 271)
[function] fn group_classification_overloaded_when_busy_and_full() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 281)
[function] fn calculate_imbalance_moves_load_toward_equilibrium() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 304)
[function] fn calculate_imbalance_is_zero_for_balanced_groups() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 79)
[function] fn env() -> LoadBalanceEnv {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 84)
[function] fn flags_set_clear_roundtrip() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 94)
[function] fn should_stop_once_loop_max_reached() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 106)
[function] fn record_failure_sets_some_pinned() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 115)
[function] fn reset_clears_loop_counter_and_need_break() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 75)
[function] fn migration_stop_ipi_increments_dedicated_counter() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 85)
[function] fn out_of_range_cpu_is_a_safe_noop() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 92)
[function] fn handle_ipi_entry_ignores_unimplemented_types_without_panic() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 108)
[function] fn wake_stopper_task(&self) {
```

```
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 280)
[function] fn stopper_state_transitions() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 295)
[function] fn stopper_registry_bounds_check() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 302)
[function] fn queue_execute_and_wait_roundtrip_returns_value_from_function() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 317)
[function] fn queue_work_rejects_a_second_job_while_busy() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 325)
[function] fn execute_work_without_queued_job_is_a_noop() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 393)
[function] fn topology_alloc_domains_respects_limits() {
kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 406)
[function] fn sched_domain_flags_logic() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 5)
[function] fn make_idle(pid: u64, cpu: u32) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 12)
[function] fn make_task(pid: u64, policy: SchedPolicy, nice: i8) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 18)
[function] fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 23)
[function] fn select_task_rq_returns_preferred_cpu_when_idle() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 42)
[function] fn select_task_rq_falls_back_to_least_loaded() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 68)
[function] fn select_task_rq_respects_cpus_allowed_mask() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 88)
[function] fn wake_up_process_enqueues_to_target_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 107)
[function] fn idle_balance_pulls_task_from_overloaded_cpu() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 135)
[function] fn load_balance_moves_task_across_uneven_queues() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 161)
[function] fn no_migration_when_cpus_allowed_forbids_it() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 189)
[function] fn migrated_task_updates_cpu_and_rq_pointer() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 224)
[function] fn pf_no_migrate_blocks_migration() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 250)
[function] fn load_balance_is_noop_below_threshold() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 4)
[function] fn default_load_avg_is_zero() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 14)
[function] fn accumulate_zero_delta_does_not_change_state_except_time() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 23)
[function] fn accumulate_running_task_increases_util_and_load() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 34)
[function] fn accumulate_idle_task_increases_load_but_not_util() {
```

```

kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 44)
[function] fn decay_reduces_load_avg_over_time() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 56)
[function] fn load_avg_never_exceeds_theoretical_maximum() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 67)
[function] fn higher_weight_produces_higher_load_avg() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 78)
[function] fn saturating_add_prevents_overflow_on_huge_delta() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 86)
[function] fn alternating_running_and_idle_smooths_util_avg() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 99)
[function] fn period_contrib_increments_on_every_accumulate() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 109)
[function] fn util_sum_accumulates_only_when_running() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 118)
[function] fn load_sum_accumulates_regardless_of_running_state() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 127)
[function] fn pelt_precision_under_one_millisecond() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 134)
[function] fn pelt_handles_max_weight_without_panic() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_pelt.rs (line 141)
[function] fn consecutive_accumulates_update_last_time_correctly() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 5)
[function] fn fair_key(node: *const TaskStruct) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 9)
[function] fn dl_key(node: *const TaskStruct) -> u64 {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 13)
[function] fn make_task(pid: u64, vruntime: u64, deadline: u64) -> TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 22)
[function] fn empty_tree_has_zero_count_and_ok_invariants() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 33)
[function] fn single_node_tree_is_black_and_valid() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 46)
[function] fn sequential_inserts_maintain_sorted_order_and_invariants() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 77)
[function] fn reverse_inserts_maintain_invariants() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 97)
[function] fn random_inserts_and_deletes_keep_tree_valid() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 136)
[function] fn delete_root_repeatedly_destroys_tree_cleanly() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 160)
[function] fn delete_successor_maintains_inorder_traversal() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 206)
[function] fn duplicate_vruntime_inserts_are_placed_to_the_right() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 234)
[function] fn deadline_key_orders_by_earliest_deadline() {
kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 254)
[function] fn subtree_min_and_max_return_extremes() {

```

```

kernel_Workspace/kernel/src/cpu/scheduler/tests/test_rbtrees.rs (line 276)
[function] fn stress_test_insert_delete_cycles() {
kernel_Workspace/kernel/src/cpu/scheduler/time/plist.rs (line 20)
[function] fn plist_prio(node: *mut TaskStruct) -> i32 {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 23)
[function] fn rb_parent(node: *mut TaskStruct) -> *mut TaskStruct {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 29)
[function] fn rb_color(node: *mut TaskStruct) -> usize {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 35)
[function] fn rb_is_red(node: *mut TaskStruct) -> bool {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 40)
[function] fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct,
color: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 48)
[function] fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 54)
[function] fn rb_set_color(node: *mut TaskStruct, color: usize) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rbtrees.rs (line 116)
[function] fn rb_insert_fixup(&mut self, mut node: *mut TaskStruct) {
kernel_Workspace/kernel/src/cpu/scheduler/time/rt_array.rs (line 34)
[function] fn bit_is_set(&self, prio: usize) -> bool {
kernel_Workspace/kernel/src/cpu/smp.rs (line 32)
[struct] struct ApStack([u8; AP_STACK_SIZE]);
kernel_Workspace/kernel/src/cpu/smp.rs (line 36)
[function] fn stack_top_of(i: usize) -> u64 {
kernel_Workspace/kernel/src/cpu/smp.rs (line 42)
[function] fn delay(iterations: u64) {
kernel_Workspace/kernel/src/cpu/smp.rs (line 48)
[function] fn delay_ms(ms: u64) {
kernel_Workspace/kernel/src/cpu/smp.rs (line 52)
[function] fn load_cpu_gdt(ist_stack_top: VirtAddr) {
kernel_Workspace/kernel/src/cpu/smp.rs (line 76)
[function] fn debug_halt(msg: &str) -> ! {
kernel_Workspace/kernel/src/cpu/smp.rs (line 131)
[function] fn wait_for(flag: &AtomicBool, max_ms: u64) -> bool {
kernel_Workspace/kernel/src/ctrlinstall/repo/fetch.rs (line 2)
[function] fn fetch(&mut self, manifest: &PackageManifest) ->
Result<alloc::vec::Vec<u8>, CtrlInstallError>;
kernel_Workspace/kernel/src/displayport/bridge.rs (line 5)
[function] fn dp_ready() -> bool;
kernel_Workspace/kernel/src/displayport/bridge.rs (line 6)
[function] fn dp_link_info(rate: *mut u32, lanes: *mut u32);
kernel_Workspace/kernel/src/displayport/bridge.rs (line 7)
[function] fn dp_mode_at(i: u32, id: *mut u32, w: *mut u32,
kernel_Workspace/kernel/src/displayport/bridge.rs (line 9)
[function] fn dp_mode_set_by_id(id: u32) -> bool;
kernel_Workspace/kernel/src/displayport/bridge.rs (line 10)
[function] fn dp_submit_fill(color: u32, x: u32, y: u32, w: u32, h: u32) -> u64;

```

```

kernel_Workspace/kernel/src/displayport/init.rs (line 2)
[function] fn dp_init() -> bool;
kernel_Workspace/kernel/src/displayport/init.rs (line 3)
[function] fn dp_ready() -> bool;
kernel_Workspace/kernel/src/drivers/pci/mod.rs (line 18)
[function] fn cfg_addr(bus: u32, dev: u32, func: u32, off: u32) -> u32 {
kernel_Workspace/kernel/src/drivers/usb/class/hid/mod.rs (line 26)
[function] fn submit(x: &mut Xhci, kb: &mut HidKeyboard) {
kernel_Workspace/kernel/src/drivers/usb/class/mass/blockdev.rs (line 5)
[function] fn name(&self) -> &'static str {
kernel_Workspace/kernel/src/drivers/usb/class/mass/blockdev.rs (line 9)
[function] fn block_size(&self) -> usize {
kernel_Workspace/kernel/src/drivers/usb/class/mass/blockdev.rs (line 13)
[function] fn block_count(&self) -> u64 {
kernel_Workspace/kernel/src/drivers/usb/class/mass/blockdev.rs (line 17)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/drivers/usb/class/mass/blockdev.rs (line 32)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>
{
kernel_Workspace/kernel/src/drivers/usb/class/mod.rs (line 6)
[function] fn probe(&self, dev: &UsbDevice) -> bool;
kernel_Workspace/kernel/src/drivers/usb/core/enumerate.rs (line 11)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/drivers/usb/dma.rs (line 37)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 80)
[function] fn in_add_drop(&mut self, add: u32, drop: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 89)
[function] fn slot_ptr(&mut self) -> *mut u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/context.rs (line 93)
[function] fn ep0_ptr(&mut self) -> *mut u32 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/event.rs (line 5)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 8)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 27)
[function] fn spin_wait<F: Fn() -> bool>(f: F) -> Result<(), UsbError> {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 39)
[function] fn op_write64(regs: &XhciRegs, off: usize, v: u64) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/init.rs (line 173)
[function] fn attach_port(&mut self, p: u32) {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 91)
[function] fn op(&self) -> *mut u8 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 95)
[function] fn rt(&self) -> *mut u8 {
kernel_Workspace/kernel/src/drivers/usb/host/xhci/regs.rs (line 99)
[function] fn db(&self) -> *mut u8 {

```



```

kernel_Workspace/kernel/src/driverspaceinit/abi/src.rs (line 16)
[function] fn hdr(&self) -> *mut DsRing {
kernel_Workspace/kernel/src/driverspaceinit/init/init.rs (line 20)
[function] fn kv(phys: u64) -> *mut u8 {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 10)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 17)
[struct] struct Grant {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 33)
[function] fn ac97_bars() -> (u64, u64) {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 48)
[function] fn grant_add(va: u64, phys: u64, pages: u64, kind: u8) -> bool {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 65)
[struct] struct Slot {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 73)
[function] fn arch_phys_to_virt(phys: u64) -> *mut u8;
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 76)
[function] fn ring() -> Option<*mut Slot> {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 222)
[function] fn grant_take(va: u64) -> Option<Grant> {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 235)
[function] fn grant_phys(va: u64) -> Option<(u64, u64)> {
kernel_Workspace/kernel/src/driverspaceinit/init/service.rs (line 271)
[function] fn handle(m: &DsMsg, r: &mut DsMsg) -> i32 {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 45)
[function] fn from(e: BufferError) -> Self {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 231)
[function] fn can_do_zero_copy(&self) -> bool {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 243)
[function] fn transfer_via_registers(&mut self) -> Result<(), BufferError> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 256)
[function] fn transfer_zero_copy(&mut self) -> Result<(), BufferError> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 344)
[function] fn transfer_via_bounce(&mut self) -> Result<(), BufferError> {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 522)
[function] fn arch_flush_tlb_single(task_id: u64, vaddr: VirtAddr) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 534)
[function] fn arch_clean_invalidate_dcache_region(vaddr: VirtAddr, size: u64) {
kernel_Workspace/kernel/src/ds_ipc/buffer.rs (line 609)
[function] fn test_buffer_alignment_check() {
kernel_Workspace/kernel/src/ds_ipc/endpoint.rs (line 14)
[struct] struct EndpointState {
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 60)
[function] fn transfer_message(_target: *mut Task, _info: MessageInfo, _regs:
&MessageRegisters) {
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 64)
[function] fn extract_message(_sender: *mut Task) -> IpcMessage {
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 72)
[function] fn block_current_task() {}

```



```

kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 73)
[function] fn wake_task(_task: *mut Task) {}
kernel_Workspace/kernel/src/ds_ipc/syscall.rs (line 74)
[function] fn switch_context() {}
kernel_Workspace/kernel/src/fs/disc.rs (line 18)
[function] fn block_size(&self) -> u64;
kernel_Workspace/kernel/src/fs/disc.rs (line 20)
[function] fn read_block(&mut self, lba: u64, buf: &[u8]) -> Result<()>;
kernel_Workspace/kernel/src/fs/disc.rs (line 22)
[function] fn write_block(&mut self, lba: u64, buf: &mut [u8]) -> Result<()>{
kernel_Workspace/kernel/src/fs/disc.rs (line 27)
[function] fn flush(&mut self) -> Result<()> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 58)
[function] fn drive_lba(&self) -> u8 {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 62)
[function] fn drive_identify(&self) -> u8 {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 70)
[function] fn disable_irq(&self) {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 74)
[function] fn status(&self) -> u8 {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 78)
[function] fn wait_not_bsy(&self) -> Result<(), DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 88)
[function] fn wait_drq(&self) -> Result<(), DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 104)
[function] fn wait_ready(&self) -> Result<(), DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 116)
[function] fn setup_lba28(&self, lba: u32, count: u8, cmd: u8) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 162)
[function] fn read_sector(&self, lba: u32, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 177)
[function] fn write_sector(&self, lba: u32, buf: &[u8]) -> Result<(), DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 196)
[function] fn name(&self) -> &'static str {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 200)
[function] fn block_size(&self) -> usize {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 204)
[function] fn block_count(&self) -> u64 {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 208)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 224)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>
{
kernel_Workspace/kernel/src/fs/driver/ata_pio.rs (line 241)
[function] fn probe_dev(dev: &AtaPio) -> bool {
kernel_Workspace/kernel/src/fs/driver/block.rs (line 12)
[function] fn name(&self) -> &'static str;

```

```

kernel_Workspace/kernel/src/fs/driver/block.rs (line 13)
[function] fn block_size(&self) -> usize;
kernel_Workspace/kernel/src/fs/driver/block.rs (line 14)
[function] fn block_count(&self) -> u64;
kernel_Workspace/kernel/src/fs/driver/block.rs (line 16)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError>;
kernel_Workspace/kernel/src/fs/driver/block.rs (line 17)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
kernel_Workspace/kernel/src/fs/driver/block.rs (line 19)
[function] fn read_blocks(&self, start: u64, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/fs/driver/block.rs (line 36)
[function] fn write_blocks(&self, start: u64, buf: &[u8]) -> Result<(), DriverError>
{
kernel_Workspace/kernel/src/fs/driver/lock.rs (line 20)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 49)
[function] fn u16le(b: &[u8], o: usize) -> u16 {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 53)
[function] fn u32le(b: &[u8], o: usize) -> u32 {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 57)
[function] fn u64le(b: &[u8], o: usize) -> u64 {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 107)
[function] fn read_at(disk: &'static dyn BlockDevice,
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 138)
[function] fn inode_table(&self, ino: u32) -> Result<u64, ExtError> {
kernel_Workspace/kernel/src/fs/ext4/mod.rs (line 173)
[function] fn block_map(&self, inode: &RawInode, file_blk: u32) -> Option<u64> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 30)
[function] fn u16le(b: &[u8], o: usize) -> u16 {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 34)
[function] fn u32le(b: &[u8], o: usize) -> u32 {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 65)
[function] fn read_at(&self, off: u64, buf: &mut [u8]) -> Result<(), FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 85)
[function] fn cluster_off(&self, clus: u32) -> u64 {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 89)
[function] fn read_cluster(&self, clus: u32) -> Result<Vec<u8>, FatError> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 95)
[function] fn next_cluster(&self, clus: u32) -> Option<u32> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 113)
[function] fn short_name(raw: &[u8]) -> Vec<u8> {
kernel_Workspace/kernel/src/fs/fat32/mod.rs (line 138)
[function] fn lfn_char(c: u16) -> u8 {
kernel_Workspace/kernel/src/fs/mbr.rs (line 5)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/fs/mbr.rs (line 28)
[function] fn name(&self) -> &'static str {

```

```

kernel_Workspace/kernel/src/fs/mbr.rs (line 32)
[function] fn block_size(&self) -> usize {
kernel_Workspace/kernel/src/fs/mbr.rs (line 36)
[function] fn block_count(&self) -> u64 {
kernel_Workspace/kernel/src/fs/mbr.rs (line 40)
[function] fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(),
DriverError> {
kernel_Workspace/kernel/src/fs/mbr.rs (line 48)
[function] fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>
{
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 46)
[function] fn search_leaf_node(node: &BTreeNode, name: &str) -> Result<u64> {
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 67)
[function] fn find_child_in_internal(node: &BTreeNode, name: &str) -> Result<u64> {
kernel_Workspace/kernel/src/fs/tangfs/btree.rs (line 134)
[function] fn collect_leaf_entries(node: &BTreeNode, entries: &mut Vec<DirEntry>) ->
Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/file.rs (line 85)
[function] fn allocate_block_for_inode(fs: &TangFs, inode: &mut Inode,
logical_block: u64) -> Result<u64> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 41)
[function] fn load_inode(&self) -> Result<Inode> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 55)
[function] fn save_inode(&self, inode: &Inode) -> Result<()> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 76)
[function] fn lookup(&self, name: &str) -> Result<Box<dyn VfsInode>> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 95)
[function] fn readdir(&self) -> Result<Vec<DirEntry>> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 105)
[function] fn read(&self, offset: u64, buf: &mut [u8]) -> Result<usize> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 109)
[function] fn write(&self, offset: u64, data: &[u8]) -> Result<usize> {
kernel_Workspace/kernel/src/fs/tangfs/inode.rs (line 116)
[function] fn stat(&self) -> Result<crate::fs::vfs::Stat> {
kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 81)
[function] fn root_inode(&self) -> Box<dyn VfsInode> {
kernel_Workspace/kernel/src/fs/tangfs/mod.rs (line 89)
[function] fn statfs(&self) -> Result<crate::fs::vfs::StatFs> {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 55)
[function] fn calculate_checksum(sb: &Superblock) -> u32 {
kernel_Workspace/kernel/src/fs/tangfs/superblock.rs (line 89)
[function] fn clone(&self) -> Self {
kernel_Workspace/kernel/src/fs/tfs.rs (line 39)
[function] fn rd32(b: &[u8], off: usize) -> u32 {
kernel_Workspace/kernel/src/fs/tfs.rs (line 43)
[function] fn wr32(b: &mut [u8], off: usize, v: u32) {
kernel_Workspace/kernel/src/fs/tfs.rs (line 98)
[struct] struct DirEntry {
kernel_Workspace/kernel/src/fs/tfs.rs (line 112)
[function] fn read_entry(block: &[u8], idx: usize) -> DirEntry {

```

```

kernel_Workspace/kernel/src/fs/tfs.rs (line 124)
[function] fn write_entry(block: &mut [u8], idx: usize, e: &DirEntry) {
kernel_Workspace/kernel/src/fs/tfs.rs (line 132)
[function] fn entry_name(e: &DirEntry) -> &str {
kernel_Workspace/kernel/src/fs/tfs.rs (line 137)
[function] fn find_entry(dev: &dyn BlockDevice, dir: u32, name: &str) ->
Result<(usize, DirEntry)> {
kernel_Workspace/kernel/src/gdt.rs (line 18)
[struct] struct Selectors {
kernel_Workspace/kernel/src/gfx/console.rs (line 28)
[function] fn fb() -> &'static mut Framebuffer {
kernel_Workspace/kernel/src/gfx/console.rs (line 54)
[function] fn framebuffer_info() -> Option<(u32, u32, u32, u64, bool)> {
kernel_Workspace/kernel/src/gfx/console.rs (line 90)
[function] fn set_palette_rgb332() {
kernel_Workspace/kernel/src/gfx/console.rs (line 111)
[function] fn set_palette16() {
kernel_Workspace/kernel/src/gfx/console.rs (line 127)
[function] fn disable_text_cursor() {
kernel_Workspace/kernel/src/gfx/console.rs (line 139)
[function] fn delay() {
kernel_Workspace/kernel/src/gfx/console.rs (line 363)
[function] fn draw_cell(row: usize, col: usize, ch: u8, attr: u8) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 29)
[function] fn rgb332_index(r: u32, g: u32, b: u32) -> u8 {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 33)
[function] fn rgb332_from_index(idx: u8) -> (u32, u32, u32) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 40)
[function] fn rgb_to_index4(r: u32, g: u32, b: u32) -> u8 {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 56)
[function] fn index4_to_rgb(idx: u8) -> (u32, u32, u32) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 61)
[function] fn ry(&self, y: usize) -> usize {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 65)
[function] fn rx(&self, x: usize) -> usize {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 119)
[function] fn planar_set(&self, x: usize, y: usize, color: u8) {
kernel_Workspace/kernel/src/gfx/framebuffer.rs (line 134)
[function] fn planar_get(&self, x: usize, y: usize) -> u8 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 6)
[function] fn hash2(x: i64, y: i64) -> u32 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 14)
[function] fn vnoise(xq: i64, yq: i64) -> i64 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 34)
[function] fn fbm(xq: i64, yq: i64) -> i64 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 42)
[function] fn band_fall(x: i64, y: i64, w: i64, h: i64) -> i64 {
kernel_Workspace/kernel/src/gfx/galaxy.rs (line 60)
[function] fn green_fall(x: i64, y: i64, w: i64, _h: i64) -> i64 {

```

```

kernel_Workspace/kernel/src/gfx/galaxy.rs (line 83)
[function] fn px_add(fb: &mut Framebuffer, x: i64, y: i64,
kernel_Workspace/kernel/src/gfx/vga.rs (line 9)
[function] fn write_regs(misc: u8, seq: &[u8], crtc: &[u8], gfx: &[u8], attr: &[u8])
{
kernel_Workspace/kernel/src/gfx/vga.rs (line 91)
[function] fn dispi_write(index: u16, value: u16) {
kernel_Workspace/kernel/src/gfx/vga.rs (line 98)
[function] fn dispi_read(index: u16) -> u16 {
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 5)
[function] fn hdmi_iface_acquire(owner: u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 6)
[function] fn hdmi_iface_release(owner: u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 8)
[function] fn hdmi_iface_init(owner: u32, fb_phys: u64, w: u32, h: u32, stride: u32)
-> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 9)
[function] fn hdmi_iface_ready() -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 11)
[function] fn hdmi_iface_mode_set(owner: u32, id: u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 12)
[function] fn hdmi_iface_mode_current(id: *mut u32, w: *mut u32, h: *mut u32, r: *mut
u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 13)
[function] fn hdmi_iface_mode_at(i: u32, id: *mut u32, w: *mut u32, h: *mut u32, r:
*mut u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 15)
[function] fn hdmi_iface_submit_fill(owner: u32, color: u32, x: u32, y: u32, w: u32,
h: u32) -> u64;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 16)
[function] fn hdmi_iface_poll(owner: u32, out: *mut u64) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 18)
[function] fn hdmi_iface_caps(w: *mut u32, h: *mut u32, s: *mut u32, phys: *mut u64);
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 20)
[function] fn hdmi_iface_fb_grant(owner: u32, phys: *mut u64, w: *mut u32, h: *mut
u32, s: *mut u32) -> bool;
kernel_Workspace/kernel/src/hdmi/bridge.rs (line 21)
[function] fn hdmi_iface_fb_revoke(owner: u32) -> bool;
kernel_Workspace/kernel/src/hdmi/init.rs (line 2)
[function] fn hdmi_init_with(fb_phys: u64, w: u32, h: u32, stride: u32) -> bool;
kernel_Workspace/kernel/src/hdmi/init.rs (line 3)
[function] fn hdmi_ready() -> bool;
kernel_Workspace/kernel/src/interrupts.rs (line 56)
[function] fn as_u8(self) -> u8 {
kernel_Workspace/kernel/src/interrupts.rs (line 61)
[function] fn breakpoint_handler(stack_frame: InterruptStackFrame) {
kernel_Workspace/kernel/src/kernel_glue.rs (line 3)
[function] fn cstr_to_str<'a>(p: *const u8) -> Option<&'a str> {

```

```

kernel_Workspace/kernel/src/kernel_glue.rs (line 96)
[function] fn tfs_read_path(dev: &dyn crate::fs::driver::block::BlockDevice, path:
&str) -> Option<alloc::vec::Vec<u8>> {
kernel_Workspace/kernel/src/kstd_glue.rs (line 3)
[function] fn cstr_to_str<'a>(p: *const u8) -> Option<&'a str> {
kernel_Workspace/kernel/src/main.rs (line 229)
[function] fn kernel_bin_entry(boot_info_ptr: *const u8) -> !;
kernel_Workspace/kernel/src/mm/init.rs (line 20)
[function] fn kv(phys: u64) -> *mut u8 {
kernel_Workspace/kernel/src/mm/mm_bridge.rs (line 33)
[function] fn arch_memory_init(
kernel_Workspace/kernel/src/mm/mm_bridge.rs (line 42)
[function] fn arch_memory_dump();
kernel_Workspace/kernel/src/mm/mod.rs (line 51)
[struct] struct EntryStorage(UnsafeCell<[ffi::RawMemEntry; MAX_RAW_ENTRIES]>);
kernel_Workspace/kernel/src/mm/riscv.rs (line 10)
[struct] struct FramePool([u8; FRAME_POOL_BYTES]);
kernel_Workspace/kernel/src/mm/riscv.rs (line 14)
[struct] struct State {
kernel_Workspace/kernel/src/mm/riscv.rs (line 24)
[function] fn pool_base() -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 35)
[function] fn bit_set(b: &mut [u64; BITMAP_WORDS], idx: usize) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 39)
[function] fn bit_clear(b: &mut [u64; BITMAP_WORDS], idx: usize) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 43)
[function] fn bit_get(b: &[u64; BITMAP_WORDS], idx: usize) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 66)
[function] fn scan_run(s: &mut State, count: usize, align_frames: usize) ->
Option<usize> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 180)
[struct] struct HeapPool([u8; HEAP_BYTES]);
kernel_Workspace/kernel/src/mm/riscv.rs (line 184)
[struct] struct HeapState {
kernel_Workspace/kernel/src/mm/riscv.rs (line 190)
[function] fn pool_base() -> usize {
kernel_Workspace/kernel/src/mm/riscv.rs (line 224)
[function] fn push_free(s: &mut HeapState, block: usize) {
kernel_Workspace/kernel/src/mm/riscv.rs (line 231)
[function] fn raw_alloc(size: usize, align: usize) -> *mut u8 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 438)
[function] fn bitor(self, rhs: VmmFlags) -> VmmFlags {
kernel_Workspace/kernel/src/mm/riscv.rs (line 452)
[struct] struct DeviceMap {
kernel_Workspace/kernel/src/mm/riscv.rs (line 458)
[struct] struct State {
kernel_Workspace/kernel/src/mm/riscv.rs (line 477)
[function] fn overlaps(s: &State, va: u64, len: u64) -> bool {

```

```

kernel_Workspace/kernel/src/mm/riscv.rs (line 483)
[function] fn take_range(s: &mut State, len: u64) -> Option<u64> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 588)
[function] fn bitor(self, rhs: ProtFlags) -> ProtFlags {
kernel_Workspace/kernel/src/mm/riscv.rs (line 617)
[function] fn bitor(self, rhs: MapFlags) -> MapFlags {
kernel_Workspace/kernel/src/mm/riscv.rs (line 626)
[function] fn sfence_all() {
kernel_Workspace/kernel/src/mm/riscv.rs (line 632)
[function] fn pte_for_prot(prot: ProtFlags) -> u64 {
kernel_Workspace/kernel/src/mm/riscv.rs (line 650)
[struct] struct Range {
kernel_Workspace/kernel/src/mm/riscv.rs (line 656)
[struct] struct Inner {
kernel_Workspace/kernel/src/mm/riscv.rs (line 668)
[function] fn alloc_table() -> Option<usize> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 695)
[function] fn ensure_table(entry: u64, tables: &mut Vec<u64>) -> Option<usize> {
kernel_Workspace/kernel/src/mm/riscv.rs (line 704)
[function] fn map_page(
kernel_Workspace/kernel/src/mm/riscv.rs (line 852)
[function] fn set_pte_flags(&self, va: u64, flags: u64) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 891)
[function] fn unmap_page_root(root: usize, va: u64) -> bool {
kernel_Workspace/kernel/src/mm/riscv.rs (line 916)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/mm/space.rs (line 26)
[function] fn bitor(self, rhs: ProtFlags) -> ProtFlags {
kernel_Workspace/kernel/src/mm/space.rs (line 55)
[function] fn bitor(self, rhs: MapFlags) -> MapFlags {
kernel_Workspace/kernel/src/mm/space.rs (line 121)
[function] fn drop(&mut self) {
kernel_Workspace/kernel/src/mm/virt.rs (line 24)
[function] fn bitor(self, rhs: VmmFlags) -> VmmFlags {
kernel_Workspace/kernel/src/nic/arp.rs (line 106)
[struct] struct CacheEntry {
kernel_Workspace/kernel/src/nic/arp.rs (line 173)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/nic/arp.rs (line 183)
[function] fn arp_round_trip_and_cache_expiry() {
kernel_Workspace/kernel/src/nic/checksum.rs (line 47)
[function] fn checksum_round_trip() {
kernel_Workspace/kernel/src/nic/command.rs (line 85)
[function] fn parse_octet(input: &str) -> Option<u8> {
kernel_Workspace/kernel/src/nic/command.rs (line 107)
[function] fn parses_ipv4() {
kernel_Workspace/kernel/src/nic/command.rs (line 112)
[function] fn rejects_invalid_ipv4() {

```



```

kernel_Workspace/kernel/src/nic/device.rs (line 29)
[function] fn init(&mut self) -> Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/device.rs (line 30)
[function] fn mac_address(&self) -> MacAddress;
kernel_Workspace/kernel/src/nic/device.rs (line 31)
[function] fn mtu(&self) -> usize;
kernel_Workspace/kernel/src/nic/device.rs (line 32)
[function] fn submit_tx(&mut self, frame: TxFrame<'>) -> Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/device.rs (line 33)
[function] fn poll(&mut self) -> Result<PollResult, NetworkError>;
kernel_Workspace/kernel/src/nic/device.rs (line 34)
[function] fn take_rx(&mut self) -> Option<RxFrame<'>>;
kernel_Workspace/kernel/src/nic/device.rs (line 35)
[function] fn recycle_rx(&mut self, buffer_id: u16) -> Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/driver.rs (line 41)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/nic/ethernet.rs (line 97)
[function] fn ethernet_round_trip_and_padding() {
kernel_Workspace/kernel/src/nic/icmp.rs (line 83)
[function] fn echo_request_and_reply_keep_identity_and_payload() {
kernel_Workspace/kernel/src/nic/ipv4.rs (line 105)
[function] fn ipv4_build_then_parse() {
kernel_Workspace/kernel/src/nic/packet.rs (line 69)
[function] fn self_test_passes() {
kernel_Workspace/kernel/src/nic/ping.rs (line 82)
[function] fn send_pending(
kernel_Workspace/kernel/src/nic/ping.rs (line 115)
[function] fn map_packet_error(error: PacketError) -> NetworkError {
kernel_Workspace/kernel/src/nic/runtime.rs (line 23)
[struct] struct NetworkRuntime {
kernel_Workspace/kernel/src/nic/runtime.rs (line 67)
[function] fn legacy_virtio_device() -> Option<PciDevice> {
kernel_Workspace/kernel/src/nic/stack.rs (line 149)
[function] fn process_arp(
kernel_Workspace/kernel/src/nic/stack.rs (line 182)
[function] fn process_ipv4(&mut self, frame: EthernetFrame<'>) ->
Result<StackEvent, PacketError> {
kernel_Workspace/kernel/src/nic/stack.rs (line 215)
[function] fn stack() -> NetworkStack<4> {
kernel_Workspace/kernel/src/nic/stack.rs (line 226)
[function] fn outside_subnet_uses_gateway() {
kernel_Workspace/kernel/src/nic/stack.rs (line 232)
[function] fn arp_request_and_ping_are_ethernet_padded() {
kernel_Workspace/kernel/src/nic/types.rs (line 38)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/nic/types.rs (line 82)
[function] fn fmt(&self, f: &mut fmt::Formatter<'>) -> fmt::Result {
kernel_Workspace/kernel/src/nic/types.rs (line 92)
[function] fn subnet_check_uses_mask() {

```

```

kernel_Workspace/kernel/src/nic/virtio/device.rs (line 12)
[function] fn mac_address(&self) -> MacAddress;
kernel_Workspace/kernel/src/nic/virtio/device.rs (line 13)
[function] fn mtu(&self) -> usize;
kernel_Workspace/kernel/src/nic/virtio/device.rs (line 14)
[function] fn capabilities(&self) -> Capabilities;
kernel_Workspace/kernel/src/nic/virtio/device.rs (line 15)
[function] fn transmit(&mut self, packet: &[u8]) -> Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/virtio/device.rs (line 16)
[function] fn receive(&mut self) -> Option<&[u8]>;
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 58)
[struct] struct LegacyQueue {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 75)
[function] fn allocate(size: u16) -> Result<Self, NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 101)
[function] fn descriptor_offset(&self, index: u16) -> usize {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 105)
[function] fn avail_offset(&self) -> usize {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 109)
[function] fn used_offset(&self) -> usize {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 164)
[struct] struct UsedElement {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 200)
[function] fn initialize(&mut self) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 246)
[function] fn configure_queue(&mut self, index: u16, queue: LegacyQueue) ->
Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 260)
[function] fn post_rx(&mut self, index: u16) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 282)
[function] fn reclaim_tx(&mut self) -> u16 {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 291)
[function] fn notify(&self, queue: u16) {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 295)
[function] fn fail(&mut self) {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 300)
[function] fn mmio_read(&self, offset: usize) -> u32 {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 304)
[function] fn mmio_write(&self, offset: usize, value: u32) {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 312)
[function] fn init(&mut self) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 316)
[function] fn mac_address(&self) -> MacAddress {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 320)
[function] fn mtu(&self) -> usize {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 324)
[function] fn submit_tx(&mut self, frame: TxFrame<'_>) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 358)
[function] fn poll(&mut self) -> Result<PollResult, NetworkError> {

```

```

kernel_Workspace/kernel/src/nic/virtio/net.rs (line 375)
[function] fn take_rx(&mut self) -> Option<RxFram<'_>> {
kernel_Workspace/kernel/src/nic/virtio/net.rs (line 393)
[function] fn recycle_rx(&mut self, buffer_id: u16) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 38)
[struct] struct VirtioNetHeader {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 48)
[struct] struct LegacyQueue {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 65)
[function] fn allocate(size: u16) -> Result<Self, NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 91)
[function] fn descriptor_offset(&self, index: u16) -> usize {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 95)
[function] fn avail_offset(&self) -> usize {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 99)
[function] fn used_offset(&self) -> usize {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 154)
[struct] struct UsedElement {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 192)
[function] fn initialize(&mut self) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 233)
[function] fn configure_queue(&mut self, index: u16, queue: LegacyQueue) ->
Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 245)
[function] fn post_rx(&mut self, index: u16) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 267)
[function] fn reclaim_tx(&mut self) -> u16 {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 276)
[function] fn notify(&self, queue: u16) {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 280)
[function] fn fail(&mut self) {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 285)
[function] fn read_u8(&self, offset: u16) -> u8 {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 290)
[function] fn read_u16(&self, offset: u16) -> u16 {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 295)
[function] fn read_u32(&self, offset: u16) -> u32 {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 300)
[function] fn write_u8(&self, offset: u16, value: u8) {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 307)
[function] fn write_u16(&self, offset: u16, value: u16) {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 314)
[function] fn write_u32(&self, offset: u16, value: u32) {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 323)
[function] fn init(&mut self) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 327)
[function] fn mac_address(&self) -> MacAddress {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 331)
[function] fn mtu(&self) -> usize {

```

```

kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 335)
[function] fn submit_tx(&mut self, frame: TxFrame<'_>) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 369)
[function] fn poll(&mut self) -> Result<PollResult, NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 383)
[function] fn take_rx(&mut self) -> Option<RxFrame<'_>> {
kernel_Workspace/kernel/src/nic/virtio/pci_legacy.rs (line 401)
[function] fn recycle_rx(&mut self, buffer_id: u16) -> Result<(), NetworkError> {
kernel_Workspace/kernel/src/nic/virtio/queue.rs (line 135)
[function] fn default() -> Self {
kernel_Workspace/kernel/src/nic/virtio/queue.rs (line 173)
[function] fn pool_allocates_and_releases_chain_once() {
kernel_Workspace/kernel/src/nic/virtio/queue.rs (line 186)
[function] fn zero_sized_pool_fails_cleanly() {
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 12)
[function] fn reset(&mut self) -> Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 13)
[function] fn status(&self) -> u8;
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 14)
[function] fn set_status(&mut self, status: u8);
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 16)
[function] fn device_features(&self) -> u64;
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 17)
[function] fn set_driver_features(&mut self, features: u64);
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 19)
[function] fn queue_max_size(&self, queue_index: u16) -> u16;
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 20)
[function] fn configure_queue(&mut self, queue_index: u16, setup: QueueSetup) ->
Result<(), NetworkError>;
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 22)
[function] fn notify_queue(&mut self, queue_index: u16);
kernel_Workspace/kernel/src/nic/virtio/transport.rs (line 24)
[function] fn read_config(&self, offset: u16, out: &mut [u8]) -> Result<(),
NetworkError>;
kernel_Workspace/kernel/src/nimcore.rs (line 2)
[function] fn nim_u64_to_str(v: u64, base: u8, buf: *mut u8, cap: u32) -> u32;
kernel_Workspace/kernel/src/nimcore.rs (line 3)
[function] fn nim_parse_u64(s: *const u8, len: u32, base: u8, out: *mut u64) -> u8;
kernel_Workspace/kernel/src/nimcore.rs (line 4)
[function] fn nim_rb_push(b: u8) -> u8;
kernel_Workspace/kernel/src/nimcore.rs (line 5)
[function] fn nim_rb_pop() -> i32;
kernel_Workspace/kernel/src/nimcore.rs (line 6)
[function] fn nim_shell_register(name: *const u8, nlen: u32,
kernel_Workspace/kernel/src/nimcore.rs (line 8)
[function] fn nim_shell_run(line: *const u8, len: u32) -> i32;
kernel_Workspace/kernel/src/nimcore.rs (line 9)
[function] fn nim_banner(buf: *mut u8, cap: u32) -> u32;

```

kernel_Workspace/kernel/src/pci.rs (line 28)

```
[function] fn config_address(addr: PciAddress, offset: u8) -> u32 {
```

kernel_Workspace/kernel/src/pci.rs (line 66)

```
[function] fn probe_function(bus: u8, device: u8, function: u8) -> Option<PciDevice>  
{
```

kernel_Workspace/kernel/src/policy/bridge.rs (line 2)

```
[function] fn policy_evaluate(ring: u8, cls: u8, op: u8, arg: u64) -> u8;
```

kernel_Workspace/kernel/src/policy/bridge.rs (line 3)

```
[function] fn nim_policy_log(ring: u8, cls: u8, op: u8, dec: u8);
```

kernel_Workspace/kernel/src/policy/mod.rs (line 68)

```
[struct] struct LogInner {
```

kernel_Workspace/kernel/src/process/fd.rs (line 32)

```
[function] fn kprintf(fmt: *const u8, ...);
```

kernel_Workspace/kernel/src/process/fd.rs (line 35)

```
[function] fn pool_take() -> Option<i16> {
```

kernel_Workspace/kernel/src/process/fd.rs (line 48)

```
[function] fn pool_drop(slot: i16) {
```

kernel_Workspace/kernel/src/serial.rs (line 27)

```
[function] fn tx_empty() -> bool {
```

kernel_Workspace/kernel/src/serial.rs (line 50)

```
[function] fn tx_empty() -> bool {
```

kernel_Workspace/kernel/src/serial.rs (line 83)

```
[function] fn write_str(&mut self, s: &str) -> fmt::Result {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 7)

```
[function] fn editor_run(path: *const u8) -> i32;
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 24)

```
[function] fn kbuf_push(c: u8) {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 34)

```
[function] fn kbuf_pop() -> Option<u8> {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 45)

```
[function] fn scancode_to_char(code: u8) -> Option<char> {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 133)

```
[function] fn scancode_to_keycode(code: u8) -> Option<u32> {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 152)

```
[function] fn keycode_push(k: u32) {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 195)

```
[function] fn line_as_str(buf: &mut [u8]) -> &str {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 203)

```
[function] fn line_clear() {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 207)

```
[function] fn draw_prompt() {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 226)

```
[function] fn redraw_line() {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 247)

```
[function] fn handle_char(c: char) {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 296)

```
[function] fn split_once_space(s: &str) -> (&str, &str) {
```

kernel_Workspace/kernel/src/terminal/mod.rs (line 304)

```
[function] fn parse_resolution(s: &str) -> Option<(u32, u32)> {
```

```

kernel_Workspace/kernel/src/terminal/mod.rs (line 315)
[function] fn dev() -> Option<&'static dyn BlockDevice> {
kernel_Workspace/kernel/src/terminal/mod.rs (line 319)
[function] fn execute(line: &str) {
kernel_Workspace/kernel/src/terminal/mod.rs (line 528)
[function] fn print_ping_result(result: crate::nic::PingResult) {
kernel_Workspace/kernel/src/terminal/mod.rs (line 544)
[function] fn poll_network() {
kernel_Workspace/kernel/src/terminal/mod.rs (line 553)
[struct] struct Sink;
kernel_Workspace/kernel/src/terminal/mod.rs (line 556)
[function] fn write_str(&mut self, s: &str) -> core::fmt::Result {
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.rs (line 23)
[function] fn tr_init();
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.rs (line 68)
[function] fn write_ttbr0(v: u64) {
kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.rs (line 74)
[function] fn pick_next(from: Option<usize>) -> usize {
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.rs (line 25)
[function] fn tr_init(kernel_stack_top: u64);
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.rs (line 73)
[function] fn write_satp(v: u64) {
kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.rs (line 79)
[function] fn pick_next(from: Option<usize>) -> usize {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 29)
[function] fn tr_init(rsp0: u64);
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 37)
[struct] struct IdtGate {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 51)
[function] fn set_gate(i: usize, handler: u64, dpl: u8) {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 80)
[function] fn sel_for(ring: u8) -> (u64, u64) {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 122)
[struct] struct IdtPtr {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 154)
[function] fn write_cr3(v: u64) {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 158)
[function] fn pick_next(from: Option<usize>) -> usize {
kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.rs (line 227)
[function] fn ctx_mut(&mut self) -> &mut CpuCtx {
kernel_Workspace/kernel/src/userspace/process/runc1.rs (line 2)
[function] fn k_fs_read(path: *const u8, buf: *mut u8, cap: u32) -> i32;
kernel_Workspace/kernel/src/userspace/process/runc1.rs (line 4)
[function] fn cl_compile_source(src: *const u8, len: usize, ar: *mut u8,
kernel_Workspace/kernel/src/userspace/process/runc1.rs (line 8)
[function] fn cl_vm_init(vm: *mut u8, prog: *mut u8);
kernel_Workspace/kernel/src/userspace/process/runc1.rs (line 9)
[function] fn cl_bridge_init(vm: *mut u8, ring: u8) -> i32;

```

```

kernel_Workspace/kernel/src/userspace/process/runcl.rs (line 10)
[function] fn cl_vm_run(vm: *mut u8) -> i32;
kernel_Workspace/kernel/src/userspace/process/runcl.rs (line 16)
[struct] struct ArenaBuf([u8; 196608]);
kernel_Workspace/kernel/src/userspace/process/runcl.rs (line 20)
[struct] struct ProgBuf([u8; 65536]);
kernel_Workspace/kernel/src/userspace/process/runcl.rs (line 24)
[struct] struct VmBuf([u8; 98304]);
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 17)
[function] fn paging_translate_in(pml4: u64, virt: u64) -> u64;
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 18)
[function] fn kprintf(fmt: *const u8, ...);
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 23)
[function] fn user_copy_in(cr3: u64, src: u64, dst: &mut [u8]) -> bool {
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 50)
[function] fn user_cstr(cr3: u64, ptr: u64, buf: &mut [u8]) -> Option<&str> {
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 70)
[function] fn do_spawn(cr3: u64, path_ptr: u64, parent_pid: u32) -> i64 {
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 207)
[function] fn hdmi_caps_raw(w: *mut u32, h: *mut u32,
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 209)
[function] fn kvirt_to_phys(p: *const u8) -> u64;
kernel_Workspace/kernel/src/userspace/process/syscall.rs (line 210)
[function] fn paging_map_page_in(pml4: u64, virt: u64,
kernel_Workspace/kernel/src/vga_buffer.rs (line 34)
[struct] struct ColorCode(u8);
kernel_Workspace/kernel/src/vga_buffer.rs (line 37)
[function] fn new(foreground: Color, background: Color) -> ColorCode {
kernel_Workspace/kernel/src/vga_buffer.rs (line 44)
[struct] struct ScreenChar {
kernel_Workspace/kernel/src/vga_buffer.rs (line 53)
[struct] struct Buffer {
kernel_Workspace/kernel/src/vga_buffer.rs (line 83)
[function] fn new_line(&mut self) {
kernel_Workspace/kernel/src/vga_buffer.rs (line 94)
[function] fn clear_row(&mut self, row: usize) {
kernel_Workspace/kernel/src/vga_buffer.rs (line 153)
[function] fn write_str(&mut self, s: &str) -> fmt::Result {
mp4_to_bmp/src/main.rs (line 23)
[function] fn main() {
mp4_to_bmp/src/main.rs (line 30)
[function] fn run() -> Result<(), String> {
mp4_to_bmp/src/main.rs (line 103)
[function] fn check_tool(name: &str) -> Result<(), String> {
mp4_to_bmp/src/main.rs (line 114)
[function] fn probe_resolution(input: &str) -> Result<(usize, usize), String> {
mp4_to_bmp/src/main.rs (line 152)
[function] fn read_exact_or_eof<R: Read>(reader: &mut R, buf: &mut [u8]) ->
Result<bool, String> {

```


mp4_to_bmp/src/main.rs (line 173)

```
[function] fn write_bmp(path: &str, pixels: &[u8], width: usize, height: usize) ->  
std::io::Result<()> {
```

net/lib/net-buffer/src/lib.rs (line 10)

```
[function] fn it_works() {
```

net/lib/net-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

net/lib/net-types/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/c/c-bridge/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/compute/dag-exec/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/compute/jit-compiler/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/compute/wasm-runtime/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-alloc/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-crypto/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-ffi/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-io/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-math/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-sync/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/core/core-telemetry/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/auth-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/catalog-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/graph-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/identity-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/ledger-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/orders-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/domain/ts-core/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/net/dns-resolver/src/lib.rs (line 10)

```
[function] fn it_works() {
```

server_workspace/net/proto-grpc/src/lib.rs (line 10)

```
[function] fn it_works() {
```

```

server_workspace/net/proto-rpc/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/net/stack-http3/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/net/stack-quick/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/net/stack-tcp/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/net/stack-udp/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/storage/store-btree/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/storage/store-disk/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/storage/store-lsm/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/storage/store-page/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/storage/store-wal/src/lib.rs (line 10)
[function] fn it_works() {
server_workspace/svc/svc-api/src/main.rs (line 1)
[function] fn main() {
server_workspace/svc/svc-gateway/src/main.rs (line 1)
[function] fn main() {
server_workspace/svc/svc-ingest/src/main.rs (line 1)
[function] fn main() {
server_workspace/svc/svc-query/src/main.rs (line 1)
[function] fn main() {
server_workspace/svc/svc-sync/src/main.rs (line 1)
[function] fn main() {
server_workspace/tools/tool-bench/src/main.rs (line 1)
[function] fn main() {
server_workspace/tools/tool-codegen/src/main.rs (line 1)
[function] fn main() {
server_workspace/tools/tool-migrator/src/main.rs (line 1)
[function] fn main() {
trangorgellibc/src/lib.rs (line 31)
[function] fn sc0(n: u64) -> u64 {
trangorgellibc/src/lib.rs (line 38)
[function] fn sc1(n: u64, a0: u64) -> u64 {
trangorgellibc/src/lib.rs (line 45)
[function] fn sc3(n: u64, a0: u64, a1: u64, a2: u64) -> u64 {
trangorgellibc/src/lib.rs (line 58)
[function] fn cstr(s: &str) -> ([u8; 256], usize) {
triang-lang/src/codegen/asm.rs (line 8)
[struct] struct Emitter {
triang-lang/src/codegen/asm.rs (line 14)
[function] fn label(&mut self, tag: &str) -> String {

```

```

triang-lang/src/codegen/asm.rs (line 19)
[function] fn line(&mut self, s: &str) {
triang-lang/src/codegen/asm.rs (line 26)
[function] fn op(t: &Target, v: &Val) -> String {
triang-lang/src/codegen/asm.rs (line 33)
[function] fn mem_len(ir: &[Ir], name: &str) -> u64 {
triang-lang/src/codegen/c.rs (line 4)
[struct] struct FnChunk {
triang-lang/src/codegen/c.rs (line 11)
[function] fn val(v: &Val) -> String {
triang-lang/src/codegen/c.rs (line 18)
[function] fn binop(op: BinOp) -> &'static str {
triang-lang/src/codegen/c.rs (line 30)
[function] fn cmpop(op: CmpOp) -> &'static str {
triang-lang/src/codegen/c.rs (line 37)
[function] fn is_param(name: &str, args: usize) -> bool {
triang-lang/src/codegen/c.rs (line 47)
[function] fn split(ir: &[Ir]) -> Vec<FnChunk> {
triang-lang/src/ir.rs (line 58)
[function] fn label(&mut self, tag: &str) -> String {
triang-lang/src/ir.rs (line 70)
[function] fn lower_function(&mut self, f: &Function) {
triang-lang/src/ir.rs (line 83)
[function] fn lower_stmt(&mut self, s: &Stmt) {
triang-lang/src/ir.rs (line 143)
[function] fn lower_op(&mut self, call: &OpCall) {
triang-lang/src/ir.rs (line 263)
[function] fn val(e: &Expr) -> Val {
triang-lang/src/ir.rs (line 271)
[function] fn invert(op: CmpOp) -> CmpOp {
triang-lang/src/ir.rs (line 278)
[function] fn binop(s: &str) -> BinOp {
triang-lang/src/iso.rs (line 8)
[function] fn dstring(s: &str, len: usize) -> Vec<u8> {
triang-lang/src/iso.rs (line 18)
[function] fn push_both_u16(out: &mut Vec<u8>, v: u16) {
triang-lang/src/iso.rs (line 23)
[function] fn push_both_u32(out: &mut Vec<u8>, v: u32) {
triang-lang/src/iso.rs (line 28)
[function] fn put_both_u16(buf: &mut [u8], off: usize, v: u16) {
triang-lang/src/iso.rs (line 33)
[function] fn put_both_u32(buf: &mut [u8], off: usize, v: u32) {
triang-lang/src/iso.rs (line 38)
[function] fn dir_record(name: &[u8], extent: u32, size: u32, flags: u8) -> Vec<u8> {
triang-lang/src/lexer.rs (line 28)
[function] fn peek(&self) -> Option<char> {
triang-lang/src/lexer.rs (line 32)
[function] fn peek2(&self) -> Option<char> {

```

```

triang-lang/src/lexer.rs (line 36)
[function] fn bump(&mut self) -> Option<char> {
triang-lang/src/lexer.rs (line 98)
[function] fn skip_ws_and_comments(&mut self) {
triang-lang/src/lexer.rs (line 114)
[function] fn read_number(&mut self) -> Result<TokenKind, LexError> {
triang-lang/src/lexer.rs (line 161)
[function] fn read_ident(&mut self) -> TokenKind {
triang-lang/src/lexer.rs (line 192)
[function] fn is_ident_start(c: char) -> bool {
triang-lang/src/lexer.rs (line 196)
[function] fn is_ident_cont(c: char) -> bool {
triang-lang/src/main.rs (line 18)
[function] fn main() {
triang-lang/src/main.rs (line 29)
[function] fn read_libs(dir: &str) -> Vec<String> {
triang-lang/src/main.rs (line 50)
[function] fn pipeline(srcs: &[String]) -> Vec<ir::Ir> {
triang-lang/src/main.rs (line 83)
[function] fn cmd_new(args: &[String]) {
triang-lang/src/main.rs (line 114)
[function] fn cmd_build(args: &[String]) {
triang-lang/src/main.rs (line 178)
[function] fn cmd_file(args: &[String]) {
triang-lang/src/main.rs (line 256)
[function] fn cmd_iso(_args: &[String]) {
triang-lang/src/main.rs (line 288)
[function] fn route_c(ir: &[ir::Ir], text_path: &str, elf_path: &str, bin_path:
&str, want_elf: bool, want_bin: bool) {
triang-lang/src/main.rs (line 309)
[function] fn route_asm(
triang-lang/src/main.rs (line 357)
[function] fn run_objcopy(elf: &str, bin: &str) -> bool {
triang-lang/src/parser.rs (line 21)
[function] fn cur(&self) -> &Token {
triang-lang/src/parser.rs (line 25)
[function] fn kind(&self) -> TokenKind {
triang-lang/src/parser.rs (line 29)
[function] fn at(&self, kind: &TokenKind) -> bool {
triang-lang/src/parser.rs (line 33)
[function] fn bump(&mut self) -> TokenKind {
triang-lang/src/parser.rs (line 41)
[function] fn err(&self, msg: String) -> ParseError {
triang-lang/src/parser.rs (line 49)
[function] fn expect(&mut self, want: TokenKind) -> Result<TokenKind, ParseError> {
triang-lang/src/parser.rs (line 57)
[function] fn expect_ident(&mut self) -> Result<String, ParseError> {
triang-lang/src/parser.rs (line 64)
[function] fn expect_int(&mut self) -> Result<u64, ParseError> {

```

```

triang-lang/src/parser.rs (line 79)
[function] fn parse_function(&mut self) -> Result<Function, ParseError> {
triang-lang/src/parser.rs (line 111)
[function] fn parse_block(&mut self) -> Result<Vec<Stmt>, ParseError> {
triang-lang/src/parser.rs (line 120)
[function] fn parse_stmt(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 134)
[function] fn parse_type(&mut self) -> Result<Type, ParseError> {
triang-lang/src/parser.rs (line 148)
[function] fn parse_layout(&mut self) -> Result<Layout, ParseError> {
triang-lang/src/parser.rs (line 160)
[function] fn parse_const(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 169)
[function] fn parse_static(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 187)
[function] fn parse_reg(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 195)
[function] fn parse_mem(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 207)
[function] fn parse_target(&mut self) -> Result<Target, ParseError> {
triang-lang/src/parser.rs (line 219)
[function] fn parse_op_call(&mut self) -> Result<OpCall, ParseError> {
triang-lang/src/parser.rs (line 224)
[function] fn parse_op_suffix(&mut self, target: Target) -> Result<OpCall,
ParseError> {
triang-lang/src/parser.rs (line 239)
[function] fn parse_op_stmt(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 245)
[function] fn parse_expr(&mut self) -> Result<Expr, ParseError> {
triang-lang/src/parser.rs (line 266)
[function] fn parse_cond(&mut self) -> Result<Cond, ParseError> {
triang-lang/src/parser.rs (line 277)
[function] fn parse_if(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 294)
[function] fn parse_while(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/parser.rs (line 305)
[function] fn parse_return(&mut self) -> Result<Stmt, ParseError> {
triang-lang/src/project.rs (line 19)
[function] fn parse(text: &str) -> Project {
triang-lang/src/sema.rs (line 28)
[function] fn err(&self, msg: String) -> SemaError {
triang-lang/src/sema.rs (line 43)
[function] fn check_function(&mut self, f: &Function) -> Result<(), SemaError> {
triang-lang/src/sema.rs (line 62)
[function] fn declare(&mut self, name: &str, sym: Symbol) -> Result<(), SemaError> {
triang-lang/src/sema.rs (line 70)
[function] fn check_stmt(&mut self, s: &Stmt) -> Result<(), SemaError> {
triang-lang/src/sema.rs (line 104)
[function] fn check_cond(&self, cond: &Cond) -> Result<(), SemaError> {

```

```

triang-lang/src/sema.rs (line 109)
[function] fn check_operand(&self, e: &Expr) -> Result<(), SemaError> {
triang-lang/src/sema.rs (line 125)
[function] fn check_mem_access(&self, name: &str, idx: u64) -> Result<(), SemaError>
{
triang-lang/src/sema.rs (line 141)
[function] fn expect_arity(&self, call: &OpCall, n: usize) -> Result<(), SemaError>
{
triang-lang/src/sema.rs (line 149)
[function] fn check_op(&self, call: &OpCall) -> Result<(), SemaError> {
userspace-legacy/demo/src/main.rs (line 24)
[function] fn panic(_i: &core::panic::PanicInfo) -> ! { loop {} }
userspace-legacy/init/src/main.rs (line 6)
[function] fn load_autostart() -> bool {
userspace-legacy/init/src/main.rs (line 73)
[function] fn panic(_i: &core::panic::PanicInfo) -> ! {
userspace-legacy/shell/src/main.rs (line 13)
[function] fn cstr(buf: &[u8]) -> &str {
userspace-legacy/shell/src/main.rs (line 48)
[function] fn run(cmd: &str) {
userspace-legacy/shell/src/main.rs (line 141)
[function] fn panic(_i: &core::panic::PanicInfo) -> ! { loop {} }
userspace-legacy/terminal/src/main.rs (line 16)
[function] fn push_hist(line: &[u8]) {
userspace-legacy/terminal/src/main.rs (line 107)
[function] fn panic(_i: &core::panic::PanicInfo) -> ! {
vise_lg_workspace/lang/vise-codegen/src/main.rs (line 1)
[function] fn main() {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 165)
[function] fn advance(&mut self) {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 178)
[function] fn eat(&mut self, expected: u8) -> bool {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 188)
[function] fn skip_whitespace(&mut self) {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 215)
[function] fn lex_string(&mut self) -> TokenKind {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 229)
[function] fn lex_number(&mut self) -> TokenKind {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 255)
[function] fn lex_identifier(&mut self) -> TokenKind {
vise_lg_workspace/lang/vise-lexer/src/lib.rs (line 317)
[function] fn current_span(&self) -> Span {
vise_lg_workspace/lang/vise-optimizer/src/main.rs (line 1)
[function] fn main() {
vise_lg_workspace/lang/vise-parser/src/parser.rs (line 36)
[function] fn parse_top_level(&mut self) -> Option<TopLevel> {
vise_lg_workspace/lang/vise-parser/src/parser.rs (line 46)
[function] fn parse_function(&mut self) -> Option<Function> {

```

```

viselg_workspace/lang/viselg-parser/src/parser.rs (line 93)
[function] fn parse_struct(&mut self) -> Option<StructDef> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 113)
[function] fn parse_enum(&mut self) -> Option<EnumDef> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 134)
[function] fn parse_static(&mut self) -> Option<StaticData> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 149)
[function] fn parse_block(&mut self) -> Option<Vec<Stmt>> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 165)
[function] fn parse_stmt(&mut self) -> Option<Stmt> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 188)
[function] fn parse_let(&mut self) -> Option<Stmt> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 208)
[function] fn parse_if(&mut self) -> Option<Stmt> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 232)
[function] fn parse_when(&mut self) -> Option<Stmt> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 252)
[function] fn parse_expr(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 256)
[function] fn parse_assignment(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 271)
[function] fn parse_or(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 286)
[function] fn parse_and(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 301)
[function] fn parse_equality(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 322)
[function] fn parse_comparison(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 345)
[function] fn parse_addition(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 366)
[function] fn parse_multiplication(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 388)
[function] fn parse_unary(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 405)
[function] fn parse_postfix(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 446)
[function] fn parse_primary(&mut self) -> Option<Expr> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 476)
[function] fn parse_type(&mut self) -> Option<Type> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 508)
[function] fn peek(&self) -> Option<&Token> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 512)
[function] fn advance(&mut self) -> Option<&Token> {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 519)
[function] fn check(&self, kind: TokenKind) -> bool {
viselg_workspace/lang/viselg-parser/src/parser.rs (line 523)
[function] fn match_token(&mut self, kind: TokenKind) -> bool {

```



```
viselg_workspace/lang/vise-parser/src/parser.rs (line 532)
[function] fn expect(&mut self, kind: TokenKind) -> Option<&Token> {
viselg_workspace/lang/vise-parser/src/parser.rs (line 540)
[function] fn expect_ident(&mut self) -> Option<String> {
viselg_workspace/lang/vise-parser/src/parser.rs (line 551)
[function] fn expect_number(&mut self) -> Option<i64> {
viselg_workspace/lang/vise-parser/src/parser.rs (line 562)
[function] fn is_at_end(&self) -> bool {
viselg_workspace/lang/vise-sema/src/sema.rs (line 39)
[function] fn declare_top(&mut self, item: &TopLevel) {
viselg_workspace/lang/vise-sema/src/sema.rs (line 68)
[function] fn check_top(&mut self, item: &TopLevel) {
viselg_workspace/lang/vise-sema/src/sema.rs (line 101)
[function] fn check_stmt(&mut self, stmt: &Stmt) {
viselg_workspace/lang/vise-sema/src/sema.rs (line 155)
[function] fn check_expr(&mut self, expr: &Expr) -> ClType {
viselg_workspace/runtime/vise-api/src/lib.rs (line 76)
[function] fn compile_shader(stage: ShaderStage, source: &[u8]) -> Option<Vec<u8>> {
viselg_workspace/runtime/vise-device/src/lib.rs (line 10)
[function] fn it_works() {
viselg_workspace/runtime/vise-pipeline/src/lib.rs (line 10)
[function] fn it_works() {
viselg_workspace/tools/vise-compiler/src/main.rs (line 5)
[function] fn main() {
viselg_workspace/tools/vise-compiler/src/main.rs (line 37)
[function] fn compile(source: &str) -> Option<Vec<u8>> {
viselg_workspace/tools/vise-inspector/src/main.rs (line 1)
[function] fn main() {
```

3. Code fragments with comments

Ada (body)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 6) - [internal]

```
Power_Of_Two

function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is
begin
    return Phys_Addr (Shift_Left (1, Integer (Exponent)));
end Power_Of_Two;

function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 11) - [internal]

```
Is_Aligned

function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 18) - [internal]

```
Get_Buddy_Address

function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    Bit_Mask : constant Phys_Addr := Power_Of_Two (Size_Exp);
begin
    -- XOR z maską odwraca dokładnie ten jeden bit, który definiuje pozycję buddy.
    -- SPARK udowodni, że ponieważ Addr jest wyrównane (Pre condition z .ads),
    -- to wynik również będzie wyrównany, a XOR nie spowoduje przepełnienia.
    return Addr xor Bit_Mask;
end Get_Buddy_Address;
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 31) - [internal]

```
Get_Left_Child_Address

function Get_Left_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
begin
    -- Lewe dziecko zawsze zaczyna się tam, gdzie rodzic
    return Parent_Addr;
end Get_Left_Child_Address;

function Get_Right_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 40) - [internal]

Get_Right_Child_Address

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr is
  -- Prawe dziecko to rodzic + połowa rozmiaru rodzica
  Half_Size : constant Phys_Addr := Power_Of_Two (Parent_Exp - 1);
begin
  return Parent_Addr + Half_Size;
end Get_Right_Child_Address;

function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 50) - [internal]

Safe_Calculate_DMA_End

```
function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  Buffer_Len : Phys_Addr
) return Phys_Addr is
begin
  -- Ponieważ Pre condition w .ads gwarantuje, że Base + Len <= Last,
  -- kompilator SPARK wie, że to dodawanie nigdy nie "owinie" (wrap-around) pamięci.
  return Base_Addr + Buffer_Len;
end Safe_Calculate_DMA_End;

end DS_Buddy_Math;
```

kernel_Workspace/kernel/src/policy/ada/policy.adb (line 5) - [internal] Evaluate

```
function Evaluate (Ring : Ring_Id;
  Class : Call_Class;
  Op    : Unsigned_8;
  Arg   : U64)
return Decision
is
  pragma Unreferenced (Arg);
begin
  if Ring = Ring_Kernel then
    return Allow;
  end if;

  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 5) - [internal] To_Ring

```
function To_Ring (V : Unsigned_8) return Ring_Id is
  (case V is
    when 0 => Ring_Kernel,
    when 1 => Ring_Driver,
    when others => Ring_User);

function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    // ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 11) - [internal] To_Class
function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    when 4 => Cls_Block,
    when others => Cls_Net);

function policy_evaluate (ring : Unsigned_8;
                          cls : Unsigned_8;
                          op : Unsigned_8;
                          // ... (truncated, see source file for more)

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 20) - [internal] policy_evaluate
function policy_evaluate (ring : Unsigned_8;
                          cls : Unsigned_8;
                          op : Unsigned_8;
                          arg : Unsigned_64)
  return Unsigned_8
is
  D : constant Decision :=
    Evaluate (To_Ring (ring), To_Class (cls), op, U64 (arg));
begin
  return (case D is
    when Allow => 0,
    when Deny => 1);
  // ... (truncated, see source file for more)

```

Ada (spec)

```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 4) - [public]
DS_Buddy_Math
package DS_Buddy_Math with
  SPARK_Mode,
  Pure
is
  -- Typy bazowe dla fizycznej pamieci (bare-metal)
  type Phys_Addr is new Unsigned_64;
  type Block_Size is new Unsigned_64;

  -- Minimalny i maksymalny rozmiar bloku (np. 4KB do 1GB)
  MIN_BLOCK_SIZE : constant Block_Size := 12; -- 2^12 = 4096 bytes (4KB)
  MAX_BLOCK_SIZE : constant Block_Size := 30; -- 2^30 = 1GB

  // ... (truncated, see source file for more)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 17) - [public]
Power_Of_Two
  -- Funkcja pomocnicza: Obliczanie 2^N (Shift left)

  function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
    Pre => Exponent <= 63,
    -- Use a bit-shift to produce a value of type Unsigned_64 / Phys_Addr
    Post => Power_Of_Two'Result = Shift_Left (Phys_Addr (1), Integer (Exponent));

  -- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku
  function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
    Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);

  -- =====
  -- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
  -- =====
  // ... (truncated, see source file for more)

```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 23) - [public]

Is_Aligned

-- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku

```
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
  Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);
```

```
-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).
```

```
function Get_Buddy_Address (
  Addr      : Phys_Addr;
  Size_Exp  : Block_Size
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 32) - [public]

Get_Buddy_Address

```
-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
```

```
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).
```

```
function Get_Buddy_Address (
  Addr      : Phys_Addr;
  Size_Exp  : Block_Size
) return Phys_Addr with
  Pre => Size_Exp >= MIN_BLOCK_SIZE and then Size_Exp <= MAX_BLOCK_SIZE,
  Post => Is_Aligned (Get_Buddy_Address'Result, Size_Exp),
  Post => Get_Buddy_Address'Result /= Addr, -- Buddy NIGDY nie jest tym samym adresem
  Post => Shift_Right (Get_Buddy_Address'Result, Integer (Size_Exp)) =
    (Shift_Right (Addr, Integer (Size_Exp)) xor Phys_Addr (1)); -- Udowodniona różnica dok
```

```
-- =====
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 47) - [public]

Get_Left_Child_Address

```
-- =====
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)
-- =====
```

-- Dzielenie bloku o rozmiarze 2^N na dwa bloki o rozmiarze $2^{(N-1)}$.

```
function Get_Left_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  Pre => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),
  Post => Get_Left_Child_Address'Result = Parent_Addr,
  Post => Is_Aligned (Get_Left_Child_Address'Result, Parent_Exp - 1);
```

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 55) - [public]

Get_Right_Child_Address

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  Pre  => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),
  Post => Get_Right_Child_Address'Result = Parent_Addr + Power_Of_Two (Parent_Exp - 1),
  Post => Is_Aligned (Get_Right_Child_Address'Result, Parent_Exp - 1),
  Post => Get_Right_Child_Address'Result > Parent_Addr; -- Prawe dziecko jest zawsze wyżej w pamięci

-- =====
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA
-- =====
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 69) - [public]

Safe_Calculate_DMA_End

```
-- =====
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA
-- =====
-- Krytyczne dla sterowników: obliczanie adresu końcowego bufora bez przepełnienia (Overflow).

function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  Buffer_Len : Phys_Addr
) return Phys_Addr with
  Pre  => Base_Addr <= Phys_Addr'Last - Buffer_Len, -- Gwarancja braku przepełnienia UInt64
  Post => Safe_Calculate_DMA_End'Result = Base_Addr + Buffer_Len;

-- Eksport do C/Rust (FFI)
pragma Export (C, Get_Buddy_Address, "ds_spark_buddy_get_buddy");
pragma Export (C, Get_Left_Child_Address, "ds_spark_buddy_left_child");
pragma Export (C, Get_Right_Child_Address, "ds_spark_buddy_right_child");
pragma Export (C, Safe_Calculate_DMA_End, "ds_spark_safe_dma_end");
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 3) - [public] Policy

```
package Policy
with SPARK_Mode
is

  type Ring_Id is (Ring_Kernel, Ring_Driver, Ring_User)
    with Size => 8;

  type Call_Class is (Cls_Sys, Cls_Video, Cls_Audio,
    Cls_Input, Cls_Block, Cls_Net)
    with Size => 8;

  type Decision is (Allow, Deny)
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 21) - [public] Evaluate

```
function Evaluate (Ring  : Ring_Id;
  Class : Call_Class;
  Op    : Unsigned_8;
  Arg   : U64)
  return Decision

with
  Global => null,
  Post  =>
    (if Ring = Ring_Kernel then Evaluate'Result = Allow)
    and then
    (if Ring = Ring_User
      and then Class = Cls_Block
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/policy/ada/policy_c.ads (line 4) - [public] Policy_C

```
package Policy_C is

    function policy_evaluate (ring : Unsigned_8;
                              cls  : Unsigned_8;
                              op   : Unsigned_8;
                              arg  : Unsigned_64)
        return Unsigned_8

    with Convention => C,
         Export,
         External_Name => "policy_evaluate";

end Policy_C;
```

kernel_workspace/kernel/src/policy/ada/policy_c.ads (line 6) - [public] policy_evaluate

```
function policy_evaluate (ring : Unsigned_8;
                          cls  : Unsigned_8;
                          op   : Unsigned_8;
                          arg  : Unsigned_64)
    return Unsigned_8

    with Convention => C,
         Export,
         External_Name => "policy_evaluate";

end Policy_C;
```

Assembly

comgrub/src/boot.asm (line 10) - [internal] mboot_header_start

```
mboot_header_start:
    dd MBOOT2_MAGIC
    dd MBOOT2_ARCH
    dd MBOOT2_LENGTH
    dd MBOOT2_CHECKSUM

    dw 0
    dw 0
    dd 8
mboot_header_end:

section .bss
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 19) - [internal] mboot_header_end

```
mboot_header_end:

section .bss
    align 16
stack_bottom:
    resb 65536
stack_top:

section .text
    bits 32

global _start
    // ... (truncated, see source file for more)
```


comgrub/src/boot.asm (line 23) - [internal] stack_bottom

```
stack_bottom:
    resb 65536
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 25) - [internal] stack_top

```
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 30) - [public] _start

```
global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 33) - [internal] _start

```
_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    cli
    hlt
    jmp .hang
```

kernel_Workspace/kernel-bin/src/boot.asm (line 2) - [public] kernel_bin_entry

```
; kernel_Workspace/kernel-bin/src/boot.asm
```

```
global kernel_bin_entry
extern kernel_bin_main

section .text
bits 64

kernel_bin_entry:
    ; Wywołaj główną funkcję Rust w kernel-bin
    call kernel_bin_main

    ; Jeśli kernel_bin_main somehow wróci
.hang:
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/boot.asm (line 8) - [internal] kernel_bin_entry

```
kernel_bin_entry:
    ; Wywołaj główną funkcję Rust w kernel-bin
    call kernel_bin_main

    ; Jeśli kernel_bin_main somehow wróci
.hang:
    cli
    hlt
    jmp .hang
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 5) - [public] trampoline_start

```
.global trampoline_start
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 6) - [public] trampoline_end

```
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 7) - [public] trampoline_cr3

```
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 8) - [public] trampoline_entry

```
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 9) - [public] trampoline_stack

```
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 10) - [public] trampoline_arg

```
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 13) - [internal] trampoline_start

```
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    lgdt %cs:(trampoline_gdtr - trampoline_start)

    mov %cr0, %eax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 31) - [internal] trampoline_32

```
trampoline_32:

    mov $0x10, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    mov %cr4, %eax
    or $((1 << 5) | (1 << 9) | (1 << 10)), %eax
    mov %eax, %cr4

    movl $(TRAMPOLINE_BASE + (trampoline_cr3 - trampoline_start)), %eax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 59) - [internal] trampoline_64

```
trampoline_64:

    mov $0x20, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    xor %eax, %eax
    mov %ax, %fs
    mov %ax, %gs

    movabs $(TRAMPOLINE_BASE + (trampoline_arg - trampoline_start)), %rcx
    mov (%rcx), %rdi
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 81) - [internal] trampoline_gdt

```
trampoline_gdt:
    .quad 0x0000000000000000
    .quad 0x00CF9A000000FFFF
    .quad 0x00CF92000000FFFF
    .quad 0x00AF9A000000FFFF
    .quad 0x00AF92000000FFFF
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 87) - [internal] trampoline_gdt_end

```
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

.align 8
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 90) - [internal] trampoline_gdtr
    trampoline_gdtr:
        .word (trampoline_gdt_end - trampoline_gdt - 1)
        .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

        .align 8
    trampoline_cr3:    .quad 0
    trampoline_entry:  .quad 0
    trampoline_stack:  .quad 0
    trampoline_arg:    .quad 0

    .global trampoline_end
    trampoline_end:
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/trampoline.s (line 95) - [internal] trampoline_cr3
    trampoline_cr3:    .quad 0
    trampoline_entry:  .quad 0
    trampoline_stack:  .quad 0
    trampoline_arg:    .quad 0

    .global trampoline_end
    trampoline_end:
        .quad 0

kernel_Workspace/kernel/src/cpu/trampoline.s (line 96) - [internal] trampoline_entry
    trampoline_entry:  .quad 0
    trampoline_stack:  .quad 0
    trampoline_arg:    .quad 0

    .global trampoline_end
    trampoline_end:
        .quad 0

kernel_Workspace/kernel/src/cpu/trampoline.s (line 97) - [internal] trampoline_stack
    trampoline_stack:  .quad 0
    trampoline_arg:    .quad 0

    .global trampoline_end
    trampoline_end:
        .quad 0

kernel_Workspace/kernel/src/cpu/trampoline.s (line 98) - [internal] trampoline_arg
    trampoline_arg:    .quad 0

    .global trampoline_end
    trampoline_end:
        .quad 0

kernel_Workspace/kernel/src/cpu/trampoline.s (line 100) - [public] trampoline_end
    .global trampoline_end
    trampoline_end:
        .quad 0

kernel_Workspace/kernel/src/cpu/trampoline.s (line 101) - [internal] trampoline_end
    trampoline_end:
        .quad 0

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 3) - [public] tlb_asm_invlpg

```
.global tlb_asm_invlpg
.type tlb_asm_invlpg, @function
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 5) - [internal] tlb_asm_invlpg

```
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 9) - [public] tlb_asm_read_cr3

```
.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 11) - [internal] tlb_asm_read_cr3

```
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 15) - [public] tlb_asm_write_cr3
.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 17) - [internal] tlb_asm_write_cr3
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 21) - [public] tlb_asm_read_cr4
.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 23) - [internal] tlb_asm_read_cr4
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

.global tlb_asm_invpclid
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 27) - [public] tlb_asm_write_cr4

```
.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret
```

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 29) - [internal] tlb_asm_write_cr4

```
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret
```

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 34) - [public] tlb_asm_invpcid

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 36) - [internal] tlb_asm_invpcid

```
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 40) - [public] tlb_asm_wbinvd

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret

.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 42) - [internal] tlb_asm_wbinvd

```
tlb_asm_wbinvd:
    wbinvd
    ret

.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 46) - [public] tlb_asm_clflush

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 48) - [internal] tlb_asm_clflush

```
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 4) -
[internal] vector_table

```
vector_table:
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default

    .align 7
    b vector_default
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 41) -
[internal] vector_default

```
vector_default:
    eret

vector_hyper:
    sub sp, sp, #272

    stp x0, x1, [sp, #0]
    stp x2, x3, [sp, #16]
    stp x4, x5, [sp, #32]
    stp x6, x7, [sp, #48]
    stp x8, x9, [sp, #64]
    stp x10, x11, [sp, #80]
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 44) -
[internal] vector_hyper
    vector_hyper:
        sub sp, sp, #272

        stp x0, x1, [sp, #0]
        stp x2, x3, [sp, #16]
        stp x4, x5, [sp, #32]
        stp x6, x7, [sp, #48]
        stp x8, x9, [sp, #64]
        stp x10, x11, [sp, #80]
        stp x12, x13, [sp, #96]
        stp x14, x15, [sp, #112]
        stp x16, x17, [sp, #128]
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 102) -
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    adr x1, vector_table
    msr vbar_el1, x1
    isb
    ret

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 104) -
[internal] tr_init
    tr_init:
        adr x1, vector_table
        msr vbar_el1, x1
        isb
        ret

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 111) -
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 113) -
[internal] tr_restore_ctx

```
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    ldp x26, x27, [x0, #208]
    ldp x24, x25, [x0, #192]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 3) -
[internal] kstack_top

```
kstack_top:
    .dword 0

.section .text
.align 2

.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 9) -
[public] trap_entry

```
.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 10) -
[internal] trap_entry

```
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    sd x9, 72(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 98) -

```
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    la t0, kstack_top
    sd a0, 0(t0)

    csrw sscratch, a0

    la t0, trap_entry
    csrw stvec, t0

    li t0, (1 << 18)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 100) -

```
[internal] tr_init
    tr_init:
        la t0, kstack_top
        sd a0, 0(t0)

        csrw sscratch, a0

        la t0, trap_entry
        csrw stvec, t0

        li t0, (1 << 18)
        csrs sstatus, t0

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 114) -

```
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    ld t0, 264(a0)
    csrw sepc, t0
    ld t0, 272(a0)
    csrw sstatus, t0

    la t0, kstack_top
    ld t0, 0(t0)
    csrw sscratch, t0

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 116) -

```
[internal] tr_restore_ctx
    tr_restore_ctx:
        ld t0, 264(a0)
        csrw sepc, t0
        ld t0, 272(a0)
        csrw sstatus, t0

        la t0, kstack_top
        ld t0, 0(t0)
        csrw sscratch, t0

        ld sp, 256(a0)

        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 3) -
[public] tr_gdt
    .global tr_gdt
    tr_gdt:
        .quad 0x0000000000000000
        .quad 0x00209A0000000000
        .quad 0x0000920000000000
        .quad 0x0020BA0000000000
        .quad 0x0000B20000000000
        .quad 0x0020FA0000000000
        .quad 0x0000F20000000000
    .global tr_tss_desc
    tr_tss_desc:
        .space 16
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 4) -
[internal] tr_gdt
    tr_gdt:
        .quad 0x0000000000000000
        .quad 0x00209A0000000000
        .quad 0x0000920000000000
        .quad 0x0020BA0000000000
        .quad 0x0000B20000000000
        .quad 0x0020FA0000000000
        .quad 0x0000F20000000000
    .global tr_tss_desc
    tr_tss_desc:
        .space 16

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 12) -
[public] tr_tss_desc
    .global tr_tss_desc
    tr_tss_desc:
        .space 16

    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
    tr_tss:
        .space 104
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 13) -
[internal] tr_tss_desc
    tr_tss_desc:
        .space 16

    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
    tr_tss:
        .space 104

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 16) -
[internal] gdt_ptr
    gdt_ptr:
        .short (gdt_ptr - tr_gdt) - 1
        .quad tr_gdt

    .align 16
    .global tr_tss
tr_tss:
    .space 104

    .section .text

    .global tr_init
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 21) -
[public] tr_tss
    .global tr_tss
tr_tss:
    .space 104

    .section .text

    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 22) -
[internal] tr_tss
    tr_tss:
        .space 104

    .section .text

    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 27) -
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)

    lgdt gdt_ptr

    mov $0x10, %ax
    mov %ax, %ss
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 29) -
[internal] tr_init
    tr_init:
        mov %rdi, %rax
        mov %eax, (tr_tss + 4)
        shr $32, %rax
        mov %eax, (tr_tss + 8)

        lgdt gdt_ptr

        mov $0x10, %ax
        mov %ax, %ss
        mov %ax, %ds
        mov %ax, %es
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 54) -
[public] isr_hypercall
    .global isr_hypercall
    .type isr_hypercall, @function
isr_hypercall:
    push %rax
    push %rbx
    push %rcx
    push %rdx
    push %rsi
    push %rdi
    push %rbp
    push %r8
    push %r9
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 56) -
[internal] isr_hypercall
    isr_hypercall:
        push %rax
        push %rbx
        push %rcx
        push %rdx
        push %rsi
        push %rdi
        push %rbp
        push %r8
        push %r9
        push %r10
        push %r11
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 94) -
[public] isr_default
    .global isr_default
    .type isr_default, @function
isr_default:
    iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 96) -
[internal] isr_default
    isr_default:
        iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 99) -
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    pop %r10
    pop %r9
    pop %r8
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 101) -
[internal] tr_restore_ctx
    tr_restore_ctx:
        mov %rdi, %rsp
        pop %r15
        pop %r14
        pop %r13
        pop %r12
        pop %r11
        pop %r10
        pop %r9
        pop %r8
        pop %rbp
        pop %rdi
        // ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.asm (line 4) - [internal] buffer
    buffer: resb 16

    section .text
    global main
    main:
    fn_main:
        mov rax, 16
        mov rbx, 32
        mov rcx, rax
        add rcx, rbx
        lea r14, [rel buffer]
        mov r15, 16
        // ... (truncated, see source file for more)

```

triang-lang/out/x86_64/prog.asm (line 7) - [public] main

```
global main
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 8) - [internal] main

```
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 9) - [internal] fn_main

```
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 17) - [internal] fill_1

```
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    inc r14
    dec r15
    jmp fill_1
fill_end_2:
    lea r14, [rel buffer]
    mov byte [r14 + 0], 65
    lea r14, [rel buffer]
    mov byte [r14 + 1], 66
    // ... (truncated, see source file for more)
```

```

triang-lang/out/x86_64/prog.asm (line 24) - [internal] fill_end_2
    fill_end_2:
        lea r14, [rel buffer]
        mov byte [r14 + 0], 65
        lea r14, [rel buffer]
        mov byte [r14 + 1], 66
        lea r14, [rel buffer]
        mov byte [r14 + 2], 67
        lea r14, [rel buffer]
        movzx rax, byte [r14 + 0]
        lea r14, [rel buffer]
        movzx rbx, byte [r14 + 1]
        mov rcx, rax
        // ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.asm (line 41) - [internal] else_1
    else_1:
        mov rax, 0
    end_2:
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 43) - [internal] end_2
    end_2:
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 44) - [internal] loop_3
    loop_3:
        cmp rax, 0
        je loop_end_4
        mov rbx, rax
        sub rbx, 1
        mov rax, rbx
        jmp loop_3
    loop_end_4:
        ret

triang-lang/out/x86_64/prog.asm (line 51) - [internal] loop_end_4
    loop_end_4:
        ret

```

C

```

comlimine/src/main.c (line 6) - [public] kernel_main
extern void kernel_main(uint64_t magic, void *info);

LIMINE_BASE_REVISION(1);

static volatile struct limine_framebuffer_request framebuffer = {
    .id = LIMINE_FRAMEBUFFER_REQUEST,
    .revision = 0,
    .response = NULL,
};

static volatile struct limine_memmap_request memmap = {
    .id = LIMINE_MEMMAP_REQUEST,
    // ... (truncated, see source file for more)

comlimine/src/main.c (line 28) - [public] _start
void _start(void) {
    if (framebuffer.response == NULL ||
        memmap.response == NULL ||
        hhdm.response == NULL) {
        for (;;) { asm volatile ("hlt"); }
    }

    kernel_main(0, (void *)framebuffer.response);

    for (;;) { asm volatile ("hlt"); }
}

comlimine/src/main.c (line 35) - [public] kernel_main
kernel_main(0, (void *)framebuffer.response);

    for (;;) { asm volatile ("hlt"); }
}

drivers/vgpu/vgpu_driver.c (line 15) - [public] ds_poll
ds_poll();
ds_msg_t r;
ds_take(id, &r);
}

static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
{
    uint64_t id = ds_call(SVC_PCI, op, a0, a1, a2);
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 17) - [public] ds_take
ds_take(id, &r);
}

static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
{
    uint64_t id = ds_call(SVC_PCI, op, a0, a1, a2);
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }
    // ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 23) - [public] ds_poll
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }

    return 0;
}

static void vbe_write(uint16_t idx, uint16_t val)
{
    // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 26) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }

    return 0;
}

static void vbe_write(uint16_t idx, uint16_t val)
{
    pci_void(PORT_WRITE, VBE_INDEX, idx, 16);
    pci_void(PORT_WRITE, VBE_DATA, val, 16);
}

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 35) - [public] pci_void
    pci_void(PORT_WRITE, VBE_INDEX, idx, 16);
    pci_void(PORT_WRITE, VBE_DATA, val, 16);
}

bool vgpu_init(uint32_t w, uint32_t h)
{
    if (g_ready) {
        return true;
    }

    uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
    // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 36) - [public] pci_void
    pci_void(PORT_WRITE, VBE_DATA, val, 16);
}

bool vgpu_init(uint32_t w, uint32_t h)
{
    if (g_ready) {
        return true;
    }

    uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)
}

```

```
drivers/vgpu/vgpu_driver.c (line 42) - [public] if
    if (g_ready) {
        return true;
    }

    uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 47) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return false;
    }

    uint64_t bdf = r.arg0;

    if (r.arg1 != (0x1234 | (0x1111 << 16))) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 51) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return false;
    }

    uint64_t bdf = r.arg0;

    if (r.arg1 != (0x1234 | (0x1111 << 16))) {
        return false;
    }

    uint64_t fb_phys = pci_val(PCI_BAR, bdf, 0, 0);

    // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 57) - [public] if
    if (r.arg1 != (0x1234 | (0x1111 << 16))) {
        return false;
    }

    uint64_t fb_phys = pci_val(PCI_BAR, bdf, 0, 0);

    if (fb_phys == 0) {
        return false;
    }

    pci_void(PCI_ENABLE, bdf, 0, 0);

    // ... (truncated, see source file for more)
```


drivers/vgpu/vgpu_driver.c (line 63) - [public] if

```
    if (fb_phys == 0) {
        return false;
    }

    pci_void(PCI_ENABLE, bdf, 0, 0);

    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 67) - [public] pci_void

```
    pci_void(PCI_ENABLE, bdf, 0, 0);

    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 69) - [public] vbe_write

```
    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 70) - [public] vbe_write

```
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 71) - [public] vbe_write

```
vbe_write(1, w);
vbe_write(2, h);
vbe_write(3, 32);
vbe_write(4, 0x01 | 0x40 | 0x80);

uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                      (uint64_t)w * h * 4, 0x45000000ULL);
ds_poll();

ds_msg_t mr;

if (!ds_take(mid, &mr) || mr.status != 0) {
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 72) - [public] vbe_write

```
vbe_write(2, h);
vbe_write(3, 32);
vbe_write(4, 0x01 | 0x40 | 0x80);

uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                      (uint64_t)w * h * 4, 0x45000000ULL);
ds_poll();

ds_msg_t mr;

if (!ds_take(mid, &mr) || mr.status != 0) {
    return false;
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 73) - [public] vbe_write

```
vbe_write(3, 32);
vbe_write(4, 0x01 | 0x40 | 0x80);

uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                      (uint64_t)w * h * 4, 0x45000000ULL);
ds_poll();

ds_msg_t mr;

if (!ds_take(mid, &mr) || mr.status != 0) {
    return false;
}
// ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 74) - [public] vbe_write

```
vbe_write(4, 0x01 | 0x40 | 0x80);

uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                      (uint64_t)w * h * 4, 0x45000000ULL);
ds_poll();

ds_msg_t mr;

if (!ds_take(mid, &mr) || mr.status != 0) {
    return false;
}

// ... (truncated, see source file for more)
```

```

drivers/vgpu/vgpu_driver.c (line 78) - [public] ds_poll
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }

    g_info.width = w;
    g_info.height = h;
    g_info.bpp = 32;
    g_info.stride = w;
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 82) - [public] if
    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }

    g_info.width = w;
    g_info.height = h;
    g_info.bpp = 32;
    g_info.stride = w;
    g_info.fb_phys = fb_phys;
    g_info.fb = (void *)0x45000000ULL;

    g_ready = true;
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 106) - [public] if
    if (!g_ready || x >= g_info.width || y >= g_info.height) {
        return;
    }

    ((uint32_t *)g_info.fb)[y * g_info.stride + x] = color;
}

void vgpu_clear(uint32_t color)
{
    if (!g_ready) {
        return;
    }
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 115) - [public] if
    if (!g_ready) {
        return;
    }

    uint32_t *fb = (uint32_t *)g_info.fb;
    uint32_t n = g_info.width * g_info.height;

    for (uint32_t i = 0; i < n; i++) {
        fb[i] = color;
    }
}

// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 133) - [public] ds_poll
    ds_poll();

    ds_msg_t ar;

    if (!ds_take(aid, &ar) || ar.status == 0 ? ar.arg0 == 0 : 1) {
        return -1;
    }

    uint64_t va = ar.arg0;

    uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 137) - [public] if
    if (!ds_take(aid, &ar) || ar.status == 0 ? ar.arg0 == 0 : 1) {
        return -1;
    }

    uint64_t va = ar.arg0;

    uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
    ds_poll();

    ds_msg_t pr;

    if (!ds_take(pid, &pr) || pr.status != 0) {
        // ... (truncated, see source file for more)
    }

```

```

drivers/vgpu/vgpu_driver.c (line 144) - [public] ds_poll
    ds_poll();

    ds_msg_t pr;

    if (!ds_take(pid, &pr) || pr.status != 0) {
        return -1;
    }

    for (int32_t i = 0; i < VGPU_MAX_SURFACES; i++) {
        if (!g_surf[i].used) {
            g_surf[i].used = true;
            g_surf[i].phys = pr.arg0;
        }
        // ... (truncated, see source file for more)
    }

```

```

drivers/vgpu/vgpu_driver.c (line 148) - [public] if
    if (!ds_take(pid, &pr) || pr.status != 0) {
        return -1;
    }

    for (int32_t i = 0; i < VGPU_MAX_SURFACES; i++) {
        if (!g_surf[i].used) {
            g_surf[i].used = true;
            g_surf[i].phys = pr.arg0;
            g_surf[i].va = (void *)va;
            g_surf[i].w = w;
            g_surf[i].h = h;
        }
        // ... (truncated, see source file for more)
    }

```

```

drivers/vgpu/vgpu_driver.c (line 153) - [public] if
    if (!g_surf[i].used) {
        g_surf[i].used = true;
        g_surf[i].phys = pr.arg0;
        g_surf[i].va = (void *)va;
        g_surf[i].w = w;
        g_surf[i].h = h;

        *phys_out = pr.arg0;
        return i;
    }
}

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 170) - [public] if
if (id < 0 || id >= VGPU_MAX_SURFACES || !g_surf[id].used) {
    return false;
}

vgpu_surface_t *s = &g_surf[id];
uint32_t *src = (uint32_t *)s->va;

for (uint32_t yy = 0; yy < s->h && y + yy < g_info.height; yy++) {
    for (uint32_t xx = 0; xx < s->w && x + xx < g_info.width; xx++) {
        uint32_t c = src[yy * s->w + xx];

        if ((c & 0xFF000000) != 0) {
            // ... (truncated, see source file for more)
        }
    }
}

drivers/vgpu/vgpu_driver.c (line 181) - [public] if
    if ((c & 0xFF000000) != 0) {
        ((uint32_t *)g_info.fb)[(y + yy) * g_info.stride + (x + xx)] = c;
    }
}

return true;
}

void vgpu_process_ring(volatile vgpu_slot_t *ring)
{
    if (ring == (void *)0) {
        // ... (truncated, see source file for more)
    }
}

drivers/vgpu/vgpu_driver.c (line 192) - [public] if
if (ring == (void *)0) {
    return;
}

for (uint32_t i = 0; i < VGPU_RING_SLOTS; i++) {
    volatile vgpu_slot_t *s = &ring[i];

    if (s->id == 0 || s->done != 0) {
        continue;
    }

    switch (s->op) {
        // ... (truncated, see source file for more)
    }
}

```

```

drivers/vgpu/vgpu_driver.c (line 199) - [public] if
    if (s->id == 0 || s->done != 0) {
        continue;
    }

    switch (s->op) {
    case VGPU_INFO:
        s->r0 = ((uint64_t)g_info.width << 32) | g_info.height;
        s->r1 = g_info.fb_phys;
        s->status = 0;
        break;

        case VGPU_SURF_CREATE: {
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 203) - [public] switch
    switch (s->op) {
    case VGPU_INFO:
        s->r0 = ((uint64_t)g_info.width << 32) | g_info.height;
        s->r1 = g_info.fb_phys;
        s->status = 0;
        break;

        case VGPU_SURF_CREATE: {
            uint64_t phys = 0;
            int32_t sid = vgpu_surface_create((uint32_t)s->a0,
                                              (uint32_t)s->a1, &phys);

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 215) - [public] if
    if (sid < 0) {
        s->status = -1;
    } else {
        s->r0 = (uint64_t)(uint32_t)sid;
        s->r1 = phys;
        s->status = 0;
    }
    break;
}

    case VGPU_PRESENT:
        s->status = vgpu_surface_present((int32_t)s->a0,
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 248) - [public] ds_init
    ds_init(params_va);

    if (!initd) {
        if (vgpu_init(1024, 768)) {
            uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
            ds_poll();

            ds_msg_t rr;

            if (ds_take(rid, &rr) && rr.status == 0) {
                g_ring = (volatile vgpu_slot_t *)rr.arg0;
            }

// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 250) - [public] if
    if (!inited) {
        if (vgpu_init(1024, 768)) {
            uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
            ds_poll();

            ds_msg_t rr;

            if (ds_take(rid, &rr) && rr.status == 0) {
                g_ring = (volatile vgpu_slot_t *)rr.arg0;

                uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
                ds_poll();
            }
            // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 251) - [public] if
    if (vgpu_init(1024, 768)) {
        uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
        ds_poll();

        ds_msg_t rr;

        if (ds_take(rid, &rr) && rr.status == 0) {
            g_ring = (volatile vgpu_slot_t *)rr.arg0;

            uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
            ds_poll();

            // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 253) - [public] ds_poll
    ds_poll();

    ds_msg_t rr;

    if (ds_take(rid, &rr) && rr.status == 0) {
        g_ring = (volatile vgpu_slot_t *)rr.arg0;

        uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
        ds_poll();

        ds_msg_t pr;

        // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 257) - [public] if
    if (ds_take(rid, &rr) && rr.status == 0) {
        g_ring = (volatile vgpu_slot_t *)rr.arg0;

        uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
        ds_poll();

        ds_msg_t pr;

        if (ds_take(pid, &pr) && pr.status == 0) {
            ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
            ds_poll();

            // ... (truncated, see source file for more)
}

```

```

drivers/vgpu/vgpu_driver.c (line 261) - [public] ds_poll
    ds_poll();

    ds_msg_t pr;

    if (ds_take(pid, &pr) && pr.status == 0) {
        ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
        ds_poll();

        ds_msg_t zr;
        ds_take(0, &zr);
    }
}
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 265) - [public] if
    if (ds_take(pid, &pr) && pr.status == 0) {
        ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
        ds_poll();

        ds_msg_t zr;
        ds_take(0, &zr);
    }
}

    vgpu_clear(0xFF101020);
}

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 266) - [public] ds_call
    ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
    ds_poll();

    ds_msg_t zr;
    ds_take(0, &zr);
}

    vgpu_clear(0xFF101020);
}

    initied = true;
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 267) - [public] ds_poll
    ds_poll();

    ds_msg_t zr;
    ds_take(0, &zr);
}

    vgpu_clear(0xFF101020);
}

    initied = true;
}
// ... (truncated, see source file for more)

```



```
drivers/vgpu/vgpu_driver.c (line 270) - [public] ds_take
        ds_take(0, &zr);
    }
}

    vgpu_clear(0xFF101020);
}
```

```
    initied = true;
}
```

```
vgpu_process_ring(g_ring);
```

```
// ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 274) - [public] vgpu_clear
    vgpu_clear(0xFF101020);
}
```

```
    initied = true;
}
```

```
vgpu_process_ring(g_ring);
```

```
ds_yield();
```

```
}
```

```
drivers/vgpu/vgpu_driver.c (line 280) - [public] vgpu_process_ring
    vgpu_process_ring(g_ring);
```

```
    ds_yield();
```

```
}
```

```
drivers/vgpu/vgpu_driver.c (line 282) - [public] ds_yield
    ds_yield();
}
```

```
kernel_Workspace/kernel/src/battery/init.c (line 8) - [public] if
```

```
    if (is_ready) {
        return true;
    }
```

```
    is_ready = true;
```

```
    return true;
```

```
}
```

```
kernel_Workspace/kernel/src/battery/operation.c (line 23) - [public] if
```

```
    if (!model.present) {
        return false;
    }
```

```
    *out = model;
```

```
    return true;
```

```
}
```

```
static const battery_backend_t model_backend = {
    .present = model_present,
    .status = model_status,
};

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/battery/operation.c (line 40) - [public] if
    if (b != (void *)0 && b->status != (void *)0) {
        backend = b;
    }
}

bool battery_present(void)
{
    return backend->present();
}

bool battery_status(battery_status_t *out)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/battery/operation.c (line 52) - [public] if
    if (out == (void *)0) {
        return false;
    }

    return backend->status(out);
}

bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2)
{
    battery_status_t s;

    if (!battery_status(&s)) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/battery/operation.c (line 63) - [public] if
    if (!battery_status(&s)) {
        return false;
    }

    *a0 = ((uint64_t)s.present << 32)
        | ((uint64_t)s.state << 24)
        | (s.percent & 0xFF);

    *a1 = ((uint64_t)s.voltage_mv << 16) | (s.temp_c & 0xFFFF);
    *a2 = s.health;

    return true;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/battery/operation.c (line 79) - [public] if
    if (low_pct > 50) {
        return false;
    }

    low_threshold = low_pct;
    low_pending = false;

    return true;
}

uint32_t battery_threshold(void)
{
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/kernel/src/battery/operation.c (line 103) - [public] if

```
    if (!model.present || model.state == BATT_STATE_CHARGING) {
        return;
    }

    tick_counter++;

    if (tick_counter % 2000 == 0 && model.percent > 0) {
        model.percent--;

        if (model.percent <= low_threshold && !low_pending) {
            low_pending = true;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/battery/operation.c (line 109) - [public] if

```
    if (tick_counter % 2000 == 0 && model.percent > 0) {
        model.percent--;

        if (model.percent <= low_threshold && !low_pending) {
            low_pending = true;
        }
    }
}
```

kernel_Workspace/kernel/src/battery/operation.c (line 112) - [public] if

```
    if (model.percent <= low_threshold && !low_pending) {
        low_pending = true;
    }
}
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 14) - [public] if

```
    if (ret_len != 0 && ret == NULL) {
        return false;
    }

    for (uint8_t i = 0; i < 32; i++) {
        if (!bt_event_poll(buf, EVT_MAX, &len)) {
            continue;
        }

        if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
            continue;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 19) - [public] if

```
    if (!bt_event_poll(buf, EVT_MAX, &len)) {
        continue;
    }

    if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
        continue;
    }

    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 23) - [public] if
    if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
        continue;
    }

    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 27) - [public] if
    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    for (uint8_t k = 0; k < ret_len; k++) {
        ret[k] = buf[7 + k];
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 31) - [public] if
    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    for (uint8_t k = 0; k < ret_len; k++) {
        ret[k] = buf[7 + k];
    }

    return true;
}

return false;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 47) - [public] if
    if (bt_is_ready) {
        return true;
    }

    bt_op_model_init();

    if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
        return false;
    }

    uint8_t tmp[8];
    if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 51) - [public] bt_op_model_init
    bt_op_model_init();

    if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
        return false;
    }

    uint8_t tmp[8];
    if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
        return false;
    }

    if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/bluetooth/init.c (line 53) - [public] if
    if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
        return false;
    }

    uint8_t tmp[8];
    if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
        return false;
    }

    if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 58) - [public] if
    if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
        return false;
    }

    if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 62) - [public] if
    if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
        return false;
    }

    saved_ver = tmp[0];

    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 66) - [public] if
    if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
        return false;
    }

    saved_ver = tmp[0];

    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 72) - [public] if
    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }

    bt_is_ready = true;
    return true;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 76) - [public] if
    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }

    bt_is_ready = true;
    return true;
}

bool bt_ready(void)
{
    return bt_is_ready;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 91) - [public] if
    if (hci_ver != NULL) {
        *hci_ver = saved_ver;
    }

    if (bdaddr != NULL) {
        for (uint8_t i = 0; i < 6; i++) {
            bdaddr[i] = saved_addr[i];
        }
    }
}

kernel_Workspace/kernel/src/bluetooth/init.c (line 95) - [public] if
    if (bdaddr != NULL) {
        for (uint8_t i = 0; i < 6; i++) {
            bdaddr[i] = saved_addr[i];
        }
    }
}

```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 18) - [public] if
    if (data == NULL || len == 0 || len > EVT_MAX) {
        return false;
    }

    uint32_t next = (evt_head + 1) % EVT_Q;
    if (next == evt_tail) {
        return false;
    }

    for (uint8_t i = 0; i < len; i++) {
        evt_q[evt_head][i] = data[i];
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 23) - [public] if
    if (next == evt_tail) {
        return false;
    }

    for (uint8_t i = 0; i < len; i++) {
        evt_q[evt_head][i] = data[i];
    }

    evt_len[evt_head] = len;
    evt_head = next;
    return true;
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 49) - [public] if
    if (len != 0 && params == NULL) {
        return false;
    }

    switch (opcode) {
    case HCI_RESET:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 4;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 53) - [public] switch
    switch (opcode) {
    case HCI_RESET:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 4;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        return evt_push(ev, 7);

    case HCI_READ_LOCAL_VERSION:
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 62) - [public] evt_push
    return evt_push(ev, 7);

    case HCI_READ_LOCAL_VERSION:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 12;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        ev[7] = hci_version;
        ev[8] = 0;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 80) - [public] evt_push
    return evt_push(ev, 15);

    case HCI_READ_BD_ADDR:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 10;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        for (uint8_t i = 0; i < 6; i++) {
            ev[7 + i] = bdaddr[i];
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 93) - [public] evt_push
    return evt_push(ev, 13);

    default:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 4;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        return evt_push(ev, 7);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 103) - [public] evt_push
    return evt_push(ev, 7);
}

bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len)
{
    if (!op_ready) {
        return false;
    }

    return bt_op_model_run_cmd(opcode, params, len);
}
// ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/bluetooth/operation.c (line 109) - [public] if

```
    if (!op_ready) {
        return false;
    }

    return bt_op_model_run_cmd(opcode, params, len);
}

bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len)
{
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 113) - [public] bt_op_model_run_cmd

```
    return bt_op_model_run_cmd(opcode, params, len);
}

bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len)
{
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }

    uint8_t n = evt_len[evt_tail];
    if (n > cap) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 118) - [public] if

```
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }

    uint8_t n = evt_len[evt_tail];
    if (n > cap) {
        return false;
    }

    for (uint8_t i = 0; i < n; i++) {
        buf[i] = evt_q[evt_tail][i];
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 123) - [public] if

```
    if (n > cap) {
        return false;
    }

    for (uint8_t i = 0; i < n; i++) {
        buf[i] = evt_q[evt_tail][i];
    }

    *len = n;
    evt_tail = (evt_tail + 1) % EVT_Q;
    return true;
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 138) - [public] if
    if (!op_ready || (len != 0 && data == NULL) || len > ACL_MAX) {
        return false;
    }

    uint32_t next = (acl_head + 1) % ACL_Q;
    if (next == acl_tail) {
        return false;
    }

    for (uint16_t i = 0; i < len; i++) {
        acl_q[acl_head][i] = data[i];
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 143) - [public] if
    if (next == acl_tail) {
        return false;
    }

    for (uint16_t i = 0; i < len; i++) {
        acl_q[acl_head][i] = data[i];
    }

    acl_len[acl_head] = len;
    acl_head = next;
    return true;
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 158) - [public] if
    if (data == NULL || len == NULL || acl_head == acl_tail) {
        return false;
    }

    uint16_t n = acl_len[acl_tail];
    if (n > cap) {
        return false;
    }

    for (uint16_t i = 0; i < n; i++) {
        data[i] = acl_q[acl_tail][i];
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/operation.c (line 163) - [public] if
    if (n > cap) {
        return false;
    }

    for (uint16_t i = 0; i < n; i++) {
        data[i] = acl_q[acl_tail][i];
    }

    *len = n;
    acl_tail = (acl_tail + 1) % ACL_Q;
    return true;
}
```

```
kernel_Workspace/kernel/src/camera/init.c (line 7) - [public] if
    if (is_ready) {
        return true;
    }

    is_ready = true;

    return true;
}
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 43) - [public] if
    if (((x + y + f * 8) & 0xFF) < 16) {
        r = g = b = 255;
    }

    uint32_t i = (y * CAM_W + x) * CAM_BPP;

    framebuf[i + 0] = r;
    framebuf[i + 1] = g;
    framebuf[i + 2] = b;
}
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 58) - [public] if
    if (!streaming) {
        return false;
    }

    gen_frame();

    uint32_t n = sizeof(framebuf);

    if (cap < n) {
        n = cap;
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 62) - [public] gen_frame
    gen_frame();

    uint32_t n = sizeof(framebuf);

    if (cap < n) {
        n = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < n; i++) {
        dst[i] = framebuf[i];
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 66) - [public] if
    if (cap < n) {
        n = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < n; i++) {
        dst[i] = framebuf[i];
    }

    frame_id++;

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/camera/operation.c (line 78) - [public] if
    if (fid) {
        *fid = frame_id;
    }

    return true;
}

static const camera_backend_t model_backend = {
    .present = model_present,
    .start = model_start,
    .stop = model_stop,
    .frame = model_frame,
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/camera/operation.c (line 96) - [public] if
    if (b != (void *)0 && b->frame != (void *)0) {
        backend = b;
    }
}

bool camera_present(void)
{
    return backend->present();
}

bool camera_caps_get(camera_caps_t *out)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/camera/operation.c (line 108) - [public] if
    if (out == (void *)0 || !camera_present()) {
        return false;
    }

    out->present = 1;
    out->width = CAM_W;
    out->height = CAM_H;
    out->format = CAM_FMT_RGB888;
    out->fps = 30;

    return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/camera/operation.c (line 123) - [public] if
    if (!camera_present()) {
        return false;
    }

    return backend->start();
}

bool camera_stop(void)
{
    return backend->stop();
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/camera/operation.c (line 137) - [public] if
    if (buf == (void *)0) {
        return false;
    }

    return backend->frame(buf, cap, fid);
}

bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *fid)
{
    return backend->frame(arch_phys_to_virt(phys), cap, fid);
}

kernel_Workspace/kernel/src/core-lang/ast.c (line 14) - [public] if
    if (a->pos + aligned > a->cap) {
        return (void *)0;
    }

    void *p = a->buf + a->pos;

    a->pos += aligned;

    return p;
}

ast_node_t *cl_node(arena_t *ar, uint8_t kind, uint32_t line)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/ast.c (line 29) - [public] if
    if (n == (void *)0) {
        return (void *)0;
    }

    n->kind = kind;
    n->type.bits = 0;
    n->type.neg = false;
    n->line = line;
    n->op = 0;
    n->pool_kind = 0;
    n->is_array = false;
    n->value = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 3) - [public] kprintf
extern void kprintf(const char *fmt, ...);
extern int32_t k_input_key(void);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 4) - [public] k_input_key

```
extern int32_t k_input_key(void);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 5) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 6) - [public] k_getpid

```
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 7) - [public] k_world_cr3

```
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 8) - [public] k_kernel_cr3

```
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 9) - [public] k_spawn

```
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 10) - [public] k_ipc_send

```
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 11) - [public] k_ipc_recv

```
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 12) - [public] k_tick

```
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 16) - [public] battery_status_packed

```
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;
    kprintf("%llu\n", (unsigned long long)v);
    return 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 26) - [public] kprintf

```
kprintf("%llu\n", (unsigned long long)v);
return 0;
}

static uint64_t br_putc(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;
    kprintf("%c", (char)v);
    return 0;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 34) - [public] kprintf

```
kprintf("%c", (char)v);
return 0;
}

static uint64_t br_log(uint64_t ptr, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    char buf[128];

    if (g_ring == 0) {
        // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 45) - [public] if
    if (g_ring == 0) {
        const char *s = (const char *)ptr;
        int i = 0;

        while (s[i] && i < 127) {
            buf[i] = s[i];
            i++;
        }

        buf[i] = '\\0';
    } else {
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 49) - [public] while
        while (s[i] && i < 127) {
            buf[i] = s[i];
            i++;
        }

        buf[i] = '\\0';
    } else {
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);

        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 56) - [public] k_user_cstr
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);

        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
    }

    kprintf("%s\\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 58) - [public] if
        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
    }

    kprintf("%s\\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
    uint64_t d, uint64_t e, uint64_t f)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 63) - [public] kprintf
    kprintf("%s\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;

    int32_t k = k_input_key();

    return k < 0 ? 0 : (uint64_t)k;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 81) - [public] k_getpid
    return k_getpid();
}

static uint64_t br_tick(uint64_t a, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;
    return k_tick();
}

static uint64_t br_exit(uint64_t code, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 88) - [public] k_tick
    return k_tick();
}

static uint64_t br_exit(uint64_t code, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    if (g_vm != (void *)0) {
        g_vm->halt_req = true;
    }

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 96) - [public] if
    if (g_vm != (void *)0) {
        g_vm->halt_req = true;
    }

    return code;
}

static uint64_t br_spawn(uint64_t ptr, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 108) - [public] if
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    uint64_t cr3 = (g_ring == 0) ? k_kernel_cr3() : k_world_cr3();

    return (uint64_t)k_spawn((const char *)ptr, k_getpid(), cr3);
}

static uint64_t br_ipc_send(uint64_t dst, uint64_t a0, uint64_t a1,
                           uint64_t d, uint64_t e, uint64_t f)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 114) - [public] return
    return (uint64_t)k_spawn((const char *)ptr, k_getpid(), cr3);
}

static uint64_t br_ipc_send(uint64_t dst, uint64_t a0, uint64_t a1,
                           uint64_t d, uint64_t e, uint64_t f)
{
    (void)d; (void)e; (void)f;
    return (uint64_t)(int64_t)k_ipc_send((uint32_t)dst, a0, a1);
}

static uint64_t br_ipc_recv(uint64_t a, uint64_t b, uint64_t c,
                           uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 121) - [public] return
    return (uint64_t)(int64_t)k_ipc_send((uint32_t)dst, a0, a1);
}

static uint64_t br_ipc_recv(uint64_t a, uint64_t b, uint64_t c,
                           uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;

    uint64_t a0 = 0, a1 = 0;
    int32_t from = k_ipc_recv(&a0, &a1);

    return (uint64_t)(int64_t)from;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 140) - [public] if
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    return (uint64_t)(int64_t)k_fs_read((const char *)path,
                                       (void *)buf, (uint32_t)cap);
}

static uint64_t br_vfill(uint64_t color, uint64_t xy, uint64_t wh,
                        uint64_t d, uint64_t e, uint64_t f)
{
    (void)d; (void)e; (void)f;
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 153) - [public] if

```
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    return hdmi_submit_fill((uint32_t)color,
                           (uint32_t)(xy & 0xFFFF),
                           (uint32_t)(xy >> 16),
                           (uint32_t)(wh & 0xFFFF),
                           (uint32_t)(wh >> 16));
}

static uint64_t br_bat(uint64_t a, uint64_t b, uint64_t c,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 171) - [public] if

```
    if (!battery_status_packed(&a0, &a1, &a2)) {
        return (uint64_t)-1;
    }

    return a0;
}
```

```
int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn)
{
    return cl_vm_register_extern(vm, name, fn);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 181) - [public] cl_vm_register_extern

```
    return cl_vm_register_extern(vm, name, fn);
}
```

```
int cl_bridge_init(cl_vm_t *vm, uint8_t ring)
{
    g_vm = vm;
    g_ring = ring;

    cl_bridge_add(vm, "put", br_put);
    cl_bridge_add(vm, "putc", br_putc);
    cl_bridge_add(vm, "log", br_log);
    cl_bridge_add(vm, "key", br_key);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 189) - [public] cl_bridge_add

```
    cl_bridge_add(vm, "put", br_put);
    cl_bridge_add(vm, "putc", br_putc);
    cl_bridge_add(vm, "log", br_log);
    cl_bridge_add(vm, "key", br_key);
    cl_bridge_add(vm, "tick", br_tick);
    cl_bridge_add(vm, "bat", br_bat);
    cl_bridge_add(vm, "exit", br_exit);

    if (ring <= 1) {
        cl_bridge_add(vm, "getpid", br_getpid);
        cl_bridge_add(vm, "spawn", br_spawn);
        cl_bridge_add(vm, "ipc_send", br_ipc_send);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 190) - [public] cl_bridge_add

```
cl_bridge_add(vm, "putc", br_putc);
cl_bridge_add(vm, "log", br_log);
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 191) - [public] cl_bridge_add

```
cl_bridge_add(vm, "log", br_log);
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 192) - [public] cl_bridge_add

```
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 193) - [public] cl_bridge_add

```
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 194) - [public] cl_bridge_add

```
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 195) - [public] cl_bridge_add

```
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 197) - [public] if

```
if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 198) - [public] cl_bridge_add

```
cl_bridge_add(vm, "getpid", br_getpid);
cl_bridge_add(vm, "spawn", br_spawn);
cl_bridge_add(vm, "ipc_send", br_ipc_send);
cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
cl_bridge_add(vm, "fsread", br_fsread);
cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 199) - [public] cl_bridge_add

```
cl_bridge_add(vm, "spawn", br_spawn);
cl_bridge_add(vm, "ipc_send", br_ipc_send);
cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
cl_bridge_add(vm, "fsread", br_fsread);
cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 200) - [public] cl_bridge_add
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 201) - [public] cl_bridge_add
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 202) - [public] cl_bridge_add
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 203) - [public] cl_bridge_add
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 22) - [public] if
    if (P->code_len >= P->code_cap) {
        return 0;
    }

    cl_insn_t in;

    in.op = op;
    in.bits = bits;
    in.a = a;
    in.b = b;
    in.c = c;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 45) - [public] if
    if (P->gnames[i][k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) return (int)i;
}

    return -1;
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 63) - [public] if
    if (g->locals[i].name[k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) return i;
}

return -1;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 79) - [public] if
    if (i >= 0) {
        return g->locals[i].slot;
    }

    if (g->nlocals >= 32) {
        return 0;
    }

    for (int k = 0; name[k] && k < 63; k++) {
        g->locals[g->nlocals].name[k] = name[k];
        g->locals[g->nlocals].name[k + 1] = '\0';
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 83) - [public] if
    if (g->nlocals >= 32) {
        return 0;
    }

    for (int k = 0; name[k] && k < 63; k++) {
        g->locals[g->nlocals].name[k] = name[k];
        g->locals[g->nlocals].name[k + 1] = '\0';
    }

    g->locals[g->nlocals].bits = bits;
    g->locals[g->nlocals].slot = g->maxslots;

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 112) - [public] if
    if (neg) {
        uint8_t z[64] = { 0 };
        uint8_t sub[64];
        cl_sub(sub, z, P->consts[P->nconsts].b, w);
        for (int i = 0; i < w; i++) P->consts[P->nconsts].b[i] = sub[i];
    }

    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 115) - [public] cl_sub
    cl_sub(sub, z, P->consts[P->nconsts].b, w);
    for (int i = 0; i < w; i++) P->consts[P->nconsts].b[i] = sub[i];
}

    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
}

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 119) - [public] cl_mask
    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
}

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits)
{
    cl_prog_t *P = g->P;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 126) - [public] gen_expr
static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits)
{
    cl_prog_t *P = g->P;

    if (n == (void *)0) return;

    switch (n->kind) {
    case ND_NUM:
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
        break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 134) - [public] switch
    switch (n->kind) {
    case ND_NUM:
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
        break;

    case ND_VAR: {
        int li = find_local(g, n->name);

        if (li >= 0) {
            emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
        } else {
            int gi = find_global(P, n->name);
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 136) - [public] emit
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
    break;

case ND_VAR: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        int gi = find_global(P, n->name);

        if (gi >= 0) {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 142) - [public] if
    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        int gi = find_global(P, n->name);

        if (gi >= 0) {
            emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        } else {
            emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
        }
    }
    break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 143) - [public] emit
    emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
} else {
    int gi = find_global(P, n->name);

    if (gi >= 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
    }
}
break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 147) - [public] if
    if (gi >= 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
    }
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 148) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
}
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 150) - [public] emit
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
}
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 157) - [public] if
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_NEG, bits, 0, 0, 0);
        emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 158) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
} else if (n->pool_kind == 1) {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 161) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 162) - [public] emit
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 163) - [public] emit
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 164) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 167) - [public] emit
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 168) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 175) - [public] if
    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 176) - [public] emit
    emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
} else {
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
}
break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    }
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 178) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 184) - [public] gen_expr
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 186) - [public] if
    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
        case '+': op = OP_ADD; break;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 187) - [public] emit
    emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
        case '+': op = OP_ADD; break;
        case '-': op = OP_SUB; break;
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 189) - [public] emit

```
    emit(g, OP_NOT, bits, 0, 0, 0);
}
break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
    case '+': op = OP_ADD; break;
    case '-': op = OP_SUB; break;
    case '*': op = OP_MUL; break;
    case '/': op = OP_DIV; break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 196) - [public] switch

```
    switch (n->op) {
    case '+': op = OP_ADD; break;
    case '-': op = OP_SUB; break;
    case '*': op = OP_MUL; break;
    case '/': op = OP_DIV; break;
    case '%': op = OP_MOD; break;
    case '&': op = OP_AND; break;
    case '|': op = OP_OR; break;
    case '^': op = OP_XOR; break;
    case TK_SHL: op = OP_SHL; break;
    case TK_EQ: op = OP_EQ; break;
    case TK_NEQ: op = OP_NE; break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 215) - [public] gen_expr

```
    gen_expr(g, n->a, bits);
    gen_expr(g, n->b, bits);
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 216) - [public] gen_expr

```
    gen_expr(g, n->b, bits);
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

        if (li >= 0) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 217) - [public] emit

```
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

        if (li >= 0) {
            emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 222) - [public] gen_expr

```
gen_expr(g, n->b, bits);

if (n->a->kind == ND_VAR) {
    int li = find_local(g, n->a->name);

    if (li >= 0) {
        emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
}
break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 224) - [public] if

```
if (n->a->kind == ND_VAR) {
    int li = find_local(g, n->a->name);

    if (li >= 0) {
        emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
}
break;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 227) - [public] if

```
if (li >= 0) {
    emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
} else {
    emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
}
}
break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;
// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 228) - [public] emit
    emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
    }
    break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 230) - [public] emit
    emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
    }
    break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;

case ND_CALL: {
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 237) - [public] gen_expr
    gen_expr(g, n->a, bits);
    break;

case ND_CALL: {
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        int ei = -1;

        for (size_t i = 0; i < P->nexts; i++) {
            bool same = true;

            for (int k = 0; P->ext_names[i][k] || n->a->name[k]; k++) {
                if (P->ext_names[i][k] != n->a->name[k]) {
                    // ... (truncated, see source file for more)
                }
            }
        }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 241) - [public] if
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        int ei = -1;

        for (size_t i = 0; i < P->nexts; i++) {
            bool same = true;

            for (int k = 0; P->ext_names[i][k] || n->a->name[k]; k++) {
                if (P->ext_names[i][k] != n->a->name[k]) {
                    same = false;
                    break;
                }
            }
        }

        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 248) - [public] if

```
        if (P->ext_names[i][k] != n->a->name[k]) {
            same = false;
            break;
        }
    }

    if (same) { ei = (int)i; break; }
}

for (size_t i = 0; i < n->nlist; i++) {
    gen_expr(g, n->list[i], 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 254) - [public] if

```
    if (same) { ei = (int)i; break; }
}

for (size_t i = 0; i < n->nlist; i++) {
    gen_expr(g, n->list[i], 64);
}

if (ei >= 0) {
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 258) - [public] gen_expr

```
    gen_expr(g, n->list[i], 64);
}

if (ei >= 0) {
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    for (size_t i = 0; i < P->nfns; i++) {
        bool same = true;

        for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 261) - [public] if

```
    if (ei >= 0) {
        emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
    } else {
        int fi = -1;

        for (size_t i = 0; i < P->nfns; i++) {
            bool same = true;

            for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
                if (P->fn_names[i][k] != n->a->name[k]) {
                    same = false;
                    break;
                }
            }

            // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 262) - [public] emit
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    for (size_t i = 0; i < P->nfns; i++) {
        bool same = true;

        for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
            if (P->fn_names[i][k] != n->a->name[k]) {
                same = false;
                break;
            }
        }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 270) - [public] if
        if (P->fn_names[i][k] != n->a->name[k]) {
            same = false;
            break;
        }
    }

    if (same) { fi = (int)i; break; }
}

    if (fi >= 0) {
        emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 276) - [public] if
        if (same) { fi = (int)i; break; }
    }

    if (fi >= 0) {
        emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
    }
}
}
break;
}

default:
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 279) - [public] if
        if (fi >= 0) {
            emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
        }
    }
    break;
}

default:
    break;
}
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 280) - [public] emit
        emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
    }
}
break;
}

default:
    break;
}
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 296) - [public] switch
switch (n->kind) {
case ND_LET: {
    uint16_t slot = add_local(g, n->name, n->type.bits);

    gen_expr(g, n->a, n->type.bits);
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 300) - [public] gen_expr
    gen_expr(g, n->a, n->type.bits);
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 301) - [public] emit
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 308) - [public] if
    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 309) - [public] emit
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 311) - [public] emit
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

        gen_stmt(g, n->b);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 317) - [public] if
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

        gen_stmt(g, n->b);

        if (n->c != (void *)0) {
            uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

            P->code[jz].c = (uint32_t)P->code_len;
            gen_stmt(g, n->c);
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 318) - [public] gen_expr
    gen_expr(g, n->a, 1);

    uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

    gen_stmt(g, n->b);

    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 322) - [public] gen_stmt
    gen_stmt(g, n->b);

    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    } else {
        P->code[jz].c = (uint32_t)P->code_len;
    }
    } else {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 324) - [public] if
    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    } else {
        P->code[jz].c = (uint32_t)P->code_len;
    }
    } else {
        gen_stmt(g, n->b);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 328) - [public] gen_stmt
    gen_stmt(g, n->c);
    P->code[jmp].c = (uint32_t)P->code_len;
    } else {
        P->code[jz].c = (uint32_t)P->code_len;
    }
    } else {
        gen_stmt(g, n->b);
    }
    break;
}

case ND_WHEN: {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 334) - [public] gen_stmt
    gen_stmt(g, n->b);
}
break;
}

case ND_WHEN: {
    uint16_t tslot = add_local(g, n->name, 64);

    int li2 = find_local(g, n->name);

    if (li2 >= 0 && g->locals[li2].slot != tslot) {
        emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 344) - [public] if
    if (li2 >= 0 && g->locals[li2].slot != tslot) {
        emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
        emit(g, OP_STORE, 64, tslot, 0, 0);
    } else if (li2 < 0) {
        emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
        emit(g, OP_STORE, 64, tslot, 0, 0);
    }

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 345) - [public] emit
    emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
} else if (li2 < 0) {
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 346) - [public] emit
    emit(g, OP_STORE, 64, tslot, 0, 0);
} else if (li2 < 0) {
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 348) - [public] emit
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
    for (size_t i = 0; i < n->nlist; i++) {
        gen_stmt(g, n->list[i]);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 349) - [public] emit
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
    for (size_t i = 0; i < n->nlist; i++) {
        gen_stmt(g, n->list[i]);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 359) - [public] gen_stmt
    gen_stmt(g, n->list[i]);
}
break;

default:
    gen_expr(g, n, 64);
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}

cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 364) - [public] gen_expr
    gen_expr(g, n, 64);
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}

cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
{
    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) return (void *)0;

    // ... (truncated, see source file for more)
}

```



```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 365) - [public] emit
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}
```

```
cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
{
    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) return (void *)0;

    P->ar = ar;
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 387) - [public] if
    if (n->kind == ND_LET && P->nglobals < 64) {
        for (int k = 0; n->name[k] && k < 63; k++) {
            P->gnames[P->nglobals][k] = n->name[k];
            P->gnames[P->nglobals][k + 1] = '\0';
        }

        P->gbits[P->nglobals] = n->type.bits;
        P->gneg[P->nglobals] = n->type.neg;

        for (int k = 0; k < 8; k++) {
            P->ginit[P->nglobals].b[k] =
                (n->a != (void *)0 && n->a->kind == ND_NUM)
            // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 403) - [public] cl_wbytes
    cl_wbytes(n->type.bits, n->type.bits);

    P->nglobals++;
} else if (n->kind == ND_EXTERN) {
    for (size_t k = 0; k < n->nlist && P->nexts < 16; k++) {
        for (int x = 0; n->list[k]->name[x] && x < 63; x++) {
            P->ext_names[P->nexts][x] = n->list[k]->name[x];
            P->ext_names[P->nexts][x + 1] = '\0';
        }
        P->nexts++;
    }
} else if (n->kind == ND_FN && P->nfns < 16) {
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 436) - [public] if
    if (P->fn_names[k][x] != n->name[x]) {
        same = false;
        break;
    }
}

if (same) { fi = (int)k; break; }
}

cg_t g = { 0 };

g.P = P;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 442) - [public] if
    if (same) { fi = (int)k; break; }
}

cg_t g = { 0 };

g.P = P;

for (size_t k = 0; k < n->nlist; k++) {
    add_local(&g, n->list[k]->name, n->list[k]->type.bits);
}

P->fns[fi].start = (uint32_t)P->code_len;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 450) - [public] add_local
    add_local(&g, n->list[k]->name, n->list[k]->type.bits);
}

P->fns[fi].start = (uint32_t)P->code_len;

gen_stmt(&g, n->a);

emit(&g, OP_RET, 0, 0, 0, 0);

P->fns[fi].nslots = g.maxslots;

bool is_main = true;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 455) - [public] gen_stmt
    gen_stmt(&g, n->a);

    emit(&g, OP_RET, 0, 0, 0, 0);

    P->fns[fi].nslots = g.maxslots;

    bool is_main = true;
    const char *m = "main";

    for (int k = 0; m[k] || n->name[k]; k++) {
        if (m[k] != n->name[k]) { is_main = false; break; }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 457) - [public] emit
    emit(&g, OP_RET, 0, 0, 0, 0);

    P->fns[fi].nslots = g.maxslots;

    bool is_main = true;
    const char *m = "main";

    for (int k = 0; m[k] || n->name[k]; k++) {
        if (m[k] != n->name[k]) { is_main = false; break; }
    }

    if (is_main) {
// ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 465) - [public] if
    if (m[k] != n->name[k]) { is_main = false; break; }
    }

    if (is_main) {
        P->main_fn = fi;
        P->code[P->code_len - 1].op = OP_HALT;
    }
}

return P;
}
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 468) - [public] if
    if (is_main) {
        P->main_fn = fi;
        P->code[P->code_len - 1].op = OP_HALT;
    }
}

return P;
}
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 49) - [public] if
    if (c == '\n') {
        l->line++;
        l->col = 1;
    } else {
        l->col++;
    }
}

return c;
}
```

```
static void emit(lexer_t *l, token_t *t, tok_kind_t kind,
    const char *start, uint32_t len,
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 77) - [public] if
    if (c == ' ' || c == '\t' || c == '\r') {
        take(l);
    } else if (c == '\n') {
        take(l);
    } else if (c == '/' && peek2(l) == '/') {
        while (l->pos < l->len && peek(l) != '\n') {
            take(l);
        }
    } else if (c == '/' && peek2(l) == '*') {
        take(l);
        take(l);
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 78) - [public] take
    take(l);
} else if (c == '\n') {
    take(l);
} else if (c == '/' && peek2(l) == '/') {
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);
}

while (l->pos < l->len &&
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 80) - [public] take

```
    take(l);
} else if (c == '/' && peek2(l) == '/') {
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 82) - [public] while

```
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 83) - [public] take

```
    take(l);
}
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 86) - [public] take

```
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 87) - [public] take

```
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 91) - [public] take

```
    take(l);
}

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    return;
}
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 94) - [public] if

```
    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    return;
}
}
```

```
static bool is_id_start(char c)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 95) - [public] take

```
    take(l);
    take(l);
}
} else {
    return;
}
}

static bool is_id_start(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 96) - [public] take

```
        take(l);
    }
} else {
    return;
}
}

static bool is_id_start(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 111) - [public] is_id_start

```
return is_id_start(c) || (c >= '0' && c <= '9');
}
```

```
static bool is_digit(char c)
{
    return c >= '0' && c <= '9';
}
```

```
static int hex_val(char c)
{
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 133) - [public] if

```
if (peek(l) == '0' && (peek2(l) == 'x' || peek2(l) == 'X')) {
    take(l);
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 134) - [public] take

```
    take(l);
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 135) - [public] take
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 137) - [public] while
    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 141) - [public] take
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 142) - [public] take
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 144) - [public] while

```
    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
      (uint32_t)(l->src + l->pos - start), line, col);
t->num = val;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 148) - [public] while

```
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
      (uint32_t)(l->src + l->pos - start), line, col);
t->num = val;
}
```

```
static void lex_ident(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 163) - [public] while

```
while (is_id_char(peek(l))) {
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

for (size_t i = 0; i < sizeof(KEYWORDS) / sizeof(KEYWORDS[0]); i++) {
    const kw_t *k = &KEYWORDS[i];
    size_t kwlen = 0;

    while (k->kw[kwlen]) kwlen++;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 164) - [public] take

```
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

for (size_t i = 0; i < sizeof(KEYWORDS) / sizeof(KEYWORDS[0]); i++) {
    const kw_t *k = &KEYWORDS[i];
    size_t kwlen = 0;

    while (k->kw[kwlen]) kwlen++;

    if (kwlen == len) {
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/lexer.c (line 175) - [public] if

```
    if (kwlen == len) {
        bool same = true;

        for (size_t j = 0; j < len; j++) {
            if (start[j] != k->kw[j]) {
                same = false;
                break;
            }
        }

        if (same) {
            emit(l, t, k->kind, start, len, line, col);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 179) - [public] if

```
        if (start[j] != k->kw[j]) {
            same = false;
            break;
        }
    }

    if (same) {
        emit(l, t, k->kind, start, len, line, col);
        t->type_bits = k->bits;
        return;
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 185) - [public] if

```
        if (same) {
            emit(l, t, k->kind, start, len, line, col);
            t->type_bits = k->bits;
            return;
        }
    }

    if (len == 1 && start[0] == '_') {
        emit(l, t, TK_UNDERSCORE, start, len, line, col);
        return;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 186) - [public] emit

```
        emit(l, t, k->kind, start, len, line, col);
        t->type_bits = k->bits;
        return;
    }
}

    if (len == 1 && start[0] == '_') {
        emit(l, t, TK_UNDERSCORE, start, len, line, col);
        return;
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 193) - [public] if
    if (len == 1 && start[0] == '_') {
        emit(l, t, TK_UNDERSCORE, start, len, line, col);
        return;
    }

    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 194) - [public] emit
    emit(l, t, TK_UNDERSCORE, start, len, line, col);
    return;
}

    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 198) - [public] emit
    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);

    const char *start = l->src + l->pos;

    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 204) - [public] take
    take(l);

    const char *start = l->src + l->pos;

    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            take(l);
        }

        take(l);
    }

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 208) - [public] while
    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            take(l);
        }

        take(l);
    }

    uint32_t len = (uint32_t)(l->src + l->pos - start);

    if (peek(l) == '') {
        take(l);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 209) - [public] if
    if (peek(l) == '\\') {
        take(l);
    }

    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 210) - [public] take
    take(l);
}

    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 213) - [public] take
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 218) - [public] if
    if (peek(l) == '') {
        take(l);
    } else {
        emit(l, t, TK_ERR, start, len, line, col);
        return;
    }

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 219) - [public] take
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 221) - [public] emit
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    take(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 225) - [public] emit
    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    take(l);

    uint64_t v = 0;

    if (peek(l) == '\\') {
        take(l);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 231) - [public] take

```
take(l);

uint64_t v = 0;

if (peek(l) == '\\') {
    take(l);

    char e = take(l);

    switch (e) {
        case 'n': v = '\\n'; break;
        case 't': v = '\\t'; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 235) - [public] if

```
if (peek(l) == '\\') {
    take(l);

    char e = take(l);

    switch (e) {
        case 'n': v = '\\n'; break;
        case 't': v = '\\t'; break;
        case '0': v = 0; break;
        case '\\': v = '\\'; break;
        case '\\': v = '\\'; break;
        default: v = (uint64_t)e; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 236) - [public] take

```
take(l);

char e = take(l);

switch (e) {
    case 'n': v = '\\n'; break;
    case 't': v = '\\t'; break;
    case '0': v = 0; break;
    case '\\': v = '\\'; break;
    case '\\': v = '\\'; break;
    default: v = (uint64_t)e; break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 240) - [public] switch

```
switch (e) {
    case 'n': v = '\\n'; break;
    case 't': v = '\\t'; break;
    case '0': v = 0; break;
    case '\\': v = '\\'; break;
    case '\\': v = '\\'; break;
    default: v = (uint64_t)e; break;
}
} else {
    v = (uint64_t)(unsigned char)take(l);
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 252) - [public] if
    if (peek(l) == '\\') {
        take(l);
    } else {
        emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
        return;
    }

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 253) - [public] take
    take(l);
} else {
    emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
    return;
}

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 255) - [public] emit
    emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
    return;
}

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    skip_ws(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 259) - [public] emit
    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    skip_ws(l);

    uint32_t line = l->line;
    uint32_t col = l->col;

    if (l->pos >= l->len) {
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 265) - [public] skip_ws

```
    skip_ws(l);

    uint32_t line = l->line;
    uint32_t col = l->col;

    if (l->pos >= l->len) {
        emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
        return true;
    }

    char c = peek(l);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 270) - [public] if

```
    if (l->pos >= l->len) {
        emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
        return true;
    }

    char c = peek(l);

    if (is_digit(c)) {
        lex_number(l, t, line, col);
        return true;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 271) - [public] emit

```
        emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
        return true;
    }

    char c = peek(l);

    if (is_digit(c)) {
        lex_number(l, t, line, col);
        return true;
    }

    if (is_id_start(c)) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 277) - [public] if

```
    if (is_digit(c)) {
        lex_number(l, t, line, col);
        return true;
    }

    if (is_id_start(c)) {
        lex_ident(l, t, line, col);
        return true;
    }

    if (c == '"') {
        lex_string(l, t, line, col);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 278) - [public] lex_number

```
    lex_number(l, t, line, col);
    return true;
}

if (is_id_start(c)) {
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 282) - [public] if

```
if (is_id_start(c)) {
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 283) - [public] lex_ident

```
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 287) - [public] if

```
if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 288) - [public] lex_string
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

if (c == '$') {
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/lexer.c (line 292) - [public] if
    if (c == '\\') {
        lex_char(l, t, line, col);
        return t->kind != TK_ERR;
    }

    const char *start = l->src + l->pos;

    if (c == '$') {
        take(l);

        if (peek(l) == '!') {
            take(l);
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/lexer.c (line 293) - [public] lex_char
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

if (c == '$') {
    take(l);

    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/lexer.c (line 299) - [public] if
    if (c == '$') {
        take(l);

        if (peek(l) == '!') {
            take(l);
            emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
        } else if (peek(l) == '@') {
            take(l);
            emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
        } else {
            emit(l, t, TK_DOLLAR, start, 1, line, col);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 300) - [public] take
    take(l);

    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
    } else if (peek(l) == '@') {
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 302) - [public] if
    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
    } else if (peek(l) == '@') {
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 303) - [public] take
    take(l);
    emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
} else if (peek(l) == '@') {
    take(l);
    emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
} else {
    emit(l, t, TK_DOLLAR, start, 1, line, col);
}

    return true;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 304) - [public] emit
    emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
} else if (peek(l) == '@') {
    take(l);
    emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
} else {
    emit(l, t, TK_DOLLAR, start, 1, line, col);
}

    return true;
}

char d = peek2(l);
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 306) - [public] take

```
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 307) - [public] emit

```
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 309) - [public] emit

```
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 317) - [public] if

```
if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 318) - [public] if

```
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 319) - [public] if

```
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 320) - [public] if

```
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 321) - [public] if

```
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 322) - [public] if

```
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 323) - [public] if

```
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 324) - [public] if

```
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 325) - [public] if

```
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 328) - [public] take

```
/* jednoznaki */
```

```
take(l);
```

```
switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
case '>': emit(l, t, TK_GT, start, 1, line, col); return true;
case '+': emit(l, t, TK_PLUS, start, 1, line, col); return true;
case '-': emit(l, t, TK_MINUS, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 330) - [public] switch

```
switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
case '>': emit(l, t, TK_GT, start, 1, line, col); return true;
case '+': emit(l, t, TK_PLUS, start, 1, line, col); return true;
case '-': emit(l, t, TK_MINUS, start, 1, line, col); return true;
case '*': emit(l, t, TK_STAR, start, 1, line, col); return true;
case '/': emit(l, t, TK_SLASH, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 355) - [public] emit

```
    emit(l, t, TK_ERR, start, 1, line, col);
    return false;
}
}
```

```
void cl_lexer_init(lexer_t *l, const char *src, size_t len)
```

```
{
    l->src = src;
    l->len = len;
    l->pos = 0;
    l->line = 1;
    l->col = 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 374) - [public] if

```
    if (n >= cap) {
        return -1;
    }

    bool ok = cl_lex_next(l, &buf[n]);

    if (!ok) {
        return -2;
    }

    tok_kind_t k = buf[n].kind;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 380) - [public] if

```
    if (!ok) {
        return -2;
    }

    tok_kind_t k = buf[n].kind;

    n++;

    if (k == TK_EOF) {
        break;
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 388) - [public] if

```
    if (k == TK_EOF) {
        break;
    }
}

*count = n;
return 0;
}
```

```
const char *cl_tok_name(tok_kind_t kind)
{
    switch (kind) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 399) - [public] switch

```
switch (kind) {
case TK_EOF: return "EOF";
case TK_ERR: return "ERR";
case TK_IDENT: return "IDENT";
case TK_TYPE: return "TYPE";
case TK_NUM: return "NUM";
case TK_STR: return "STR";
case TK_CHAR: return "CHAR";
case TK_IF: return "if";
case TK_ELSE: return "else";
case TK_WHEN: return "when";
case TK_CASE: return "case";
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 19) - [public] if

```
if (toks == (void *)0) {
    *err_line = 0;
    *err_msg = "arena full";
    return (void *)0;
}

cl_lexer_init(&l, src, len);

if (cl_lex_all(&l, toks, tcap, &n) != 0) {
    *err_line = l.line;
    *err_msg = "lex error";
    return (void *)0;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 25) - [public] cl_lexer_init
    cl_lexer_init(&l, src, len);

    if (cl_lex_all(&l, toks, tcap, &n) != 0) {
        *err_line = l.line;
        *err_msg = "lex error";
        return (void *)0;
    }

    parser_t p;
    ast_node_t *prog = cl_parse(&p, toks, n, ar);

    if (prog == (void *)0) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 27) - [public] if
    if (cl_lex_all(&l, toks, tcap, &n) != 0) {
        *err_line = l.line;
        *err_msg = "lex error";
        return (void *)0;
    }

    parser_t p;
    ast_node_t *prog = cl_parse(&p, toks, n, ar);

    if (prog == (void *)0) {
        *err_line = p.err_line;
        *err_msg = p.err;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 36) - [public] if
    if (prog == (void *)0) {
        *err_line = p.err_line;
        *err_msg = p.err;
        return (void *)0;
    }

    sema_ctx_t *s = cl_sema_new(ar);

    if (s == (void *)0) {
        *err_line = 0;
        *err_msg = "arena full";
        return (void *)0;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 44) - [public] if
    if (s == (void *)0) {
        *err_line = 0;
        *err_msg = "arena full";
        return (void *)0;
    }

    cl_sema_run(s, prog);

    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        // ... (truncated, see source file for more)
    }

```



```

kernel_Workspace/kernel/src/core-lang/loader.c (line 50) - [public] cl_sema_run
    cl_sema_run(s, prog);

    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        *err_msg = d->msg;
        return (void *)0;
    }

    cl_prog_t *P = cl_codegen(prog, ar);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 52) - [public] if
    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        *err_msg = d->msg;
        return (void *)0;
    }

    cl_prog_t *P = cl_codegen(prog, ar);

    if (P == (void *)0) {
        *err_line = 0;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 62) - [public] if
    if (P == (void *)0) {
        *err_line = 0;
        *err_msg = "codegen failed";
        return (void *)0;
    }

    return P;
}

typedef struct {
    uint8_t *b;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 88) - [public] if
    if (!w->ok || w->o >= w->cap) {
        w->ok = false;
        return;
    }

    w->b[w->o++] = v;
}

static void wr_u32(wr_t *w, uint32_t v)
{
    for (int i = 0; i < 4; i++) {
        wr_u8(w, (uint8_t)(v >> (i * 8)));
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 99) - [public] wr_u8

```
    wr_u8(w, (uint8_t)(v >> (i * 8)));
}

static void wr_bytes(wr_t *w, const void *src, size_t n)
{
    for (size_t i = 0; i < n; i++) {
        wr_u8(w, ((const uint8_t *)src)[i]);
    }
}

static uint8_t rd_u8(rd_t *r)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 106) - [public] wr_u8

```
    wr_u8(w, ((const uint8_t *)src)[i]);
}

static uint8_t rd_u8(rd_t *r)
{
    if (!r->ok || r->o >= r->len) {
        r->ok = false;
        return 0;
    }

    return r->b[r->o++];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 112) - [public] if

```
    if (!r->ok || r->o >= r->len) {
        r->ok = false;
        return 0;
    }

    return r->b[r->o++];
}

static uint32_t rd_u32(rd_t *r)
{
    uint32_t v = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 142) - [public] wr_bytes

```
    wr_bytes(&w, "CLBC1", 5);

    wr_u32(&w, (uint32_t)P->code_len);

    for (size_t i = 0; i < P->code_len; i++) {
        const cl_insn_t *in = &P->code[i];

        wr_u8(&w, in->op);
        wr_u8(&w, in->bits);
        wr_u8(&w, (uint8_t)(in->a & 0xFF));
        wr_u8(&w, (uint8_t)(in->a >> 8));
        wr_u8(&w, (uint8_t)(in->b & 0xFF));
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 144) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->code_len);

for (size_t i = 0; i < P->code_len; i++) {
    const cl_insn_t *in = &P->code[i];

    wr_u8(&w, in->op);
    wr_u8(&w, in->bits);
    wr_u8(&w, (uint8_t)(in->a & 0xFF));
    wr_u8(&w, (uint8_t)(in->a >> 8));
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 149) - [public] wr_u8

```
wr_u8(&w, in->op);
wr_u8(&w, in->bits);
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 150) - [public] wr_u8

```
wr_u8(&w, in->bits);
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 151) - [public] wr_u8

```
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 152) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)(in->a >> 8));
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 153) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 154) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 155) - [public] wr_u32

```
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 158) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 161) - [public] wr_bytes

```
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 162) - [public] wr_u8

```
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 165) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 168) - [public] wr_u32

```
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 169) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 170) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 171) - [public] wr_bytes

```
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 174) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 177) - [public] wr_bytes

```
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 178) - [public] wr_u8

```
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 179) - [public] wr_u8

```
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 180) - [public] wr_bytes
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

return w.ok ? w.o : 0;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 183) - [public] wr_u32
    wr_u32(&w, (uint32_t)P->nexts);

    for (size_t i = 0; i < P->nexts; i++) {
        wr_bytes(&w, P->ext_names[i], 64);
    }

    wr_u32(&w, (uint32_t)P->main_fn);

    return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 186) - [public] wr_bytes
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
{
    rd_t r = { buf, len, 0, true };

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 189) - [public] wr_u32
    wr_u32(&w, (uint32_t)P->main_fn);

    return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
{
    rd_t r = { buf, len, 0, true };

    uint8_t magic[5];

    rd_bytes(&r, magic, 5);
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/loader.c (line 200) - [public] rd_bytes
    rd_bytes(&r, magic, 5);

    if (!r.ok || magic[0] != 'C' || magic[1] != 'L' ||
        magic[2] != 'B' || magic[3] != 'C' || magic[4] != '1') {
        return (void *)0;
    }

    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 209) - [public] if
    if (P == (void *)0) {
        return (void *)0;
    }

    P->ar = ar;
    P->code_cap = 4096;
    P->code = (cl_insn_t *)cl_arena_alloc(ar,
        P->code_cap * sizeof(cl_insn_t));

    if (P->code == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 218) - [public] if
    if (P->code == (void *)0) {
        return (void *)0;
    }

    uint32_t cl = rd_u32(&r);

    if (cl > P->code_cap) {
        return (void *)0;
    }

    P->code_len = cl;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 224) - [public] if
    if (cl > P->code_cap) {
        return (void *)0;
    }

    P->code_len = cl;

    for (size_t i = 0; i < P->code_len; i++) {
        cl_insn_t *in = &P->code[i];

        in->op = rd_u8(&r);
        in->bits = rd_u8(&r);
        in->a = (uint16_t)(rd_u8(&r) | ((uint16_t)rd_u8(&r) << 8));
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 242) - [public] if

```
    if (P->nconsts > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nconsts; i++) {
        rd_bytes(&r, P->consts[i].b, 64);
        P->const_bits[i] = rd_u8(&r);
    }

    P->nfns = rd_u32(&r);

    if (P->nfns > 16) {
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 247) - [public] rd_bytes

```
        rd_bytes(&r, P->consts[i].b, 64);
        P->const_bits[i] = rd_u8(&r);
    }

    P->nfns = rd_u32(&r);

    if (P->nfns > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nfns; i++) {
        P->fns[i].start = rd_u32(&r);
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 253) - [public] if

```
    if (P->nfns > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nfns; i++) {
        P->fns[i].start = rd_u32(&r);
        P->fns[i].nslots = rd_u8(&r);
        P->fns[i].nargs = rd_u8(&r);
        rd_bytes(&r, P->fn_names[i], 64);
    }

    P->nglobals = rd_u32(&r);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 261) - [public] rd_bytes

```
        rd_bytes(&r, P->fn_names[i], 64);
    }

    P->nglobals = rd_u32(&r);

    if (P->nglobals > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nglobals; i++) {
        rd_bytes(&r, P->gnames[i], 64);
        P->gbits[i] = rd_u8(&r);
        // ... (truncated, see source file for more)
    }
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 266) - [public] if
    if (P->nglobals > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nglobals; i++) {
        rd_bytes(&r, P->gnames[i], 64);
        P->gbits[i] = rd_u8(&r);
        P->gneg[i] = rd_u8(&r);
        rd_bytes(&r, P->ginit[i].b, 64);
    }

    P->nexts = rd_u32(&r);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 271) - [public] rd_bytes
    rd_bytes(&r, P->gnames[i], 64);
    P->gbits[i] = rd_u8(&r);
    P->gneg[i] = rd_u8(&r);
    rd_bytes(&r, P->ginit[i].b, 64);
}

    P->nexts = rd_u32(&r);

    if (P->nexts > 16) {
        return (void *)0;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 274) - [public] rd_bytes
    rd_bytes(&r, P->ginit[i].b, 64);
}

    P->nexts = rd_u32(&r);

    if (P->nexts > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nexts; i++) {
        rd_bytes(&r, P->ext_names[i], 64);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 279) - [public] if
    if (P->nexts > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nexts; i++) {
        rd_bytes(&r, P->ext_names[i], 64);
    }

    P->main_fn = (int)rd_u32(&r);

    if (!r.ok) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 284) - [public] rd_bytes
    rd_bytes(&r, P->ext_names[i], 64);
}

    P->main_fn = (int)rd_u32(&r);

    if (!r.ok) {
        return (void *)0;
    }

    return P;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 289) - [public] if
    if (!r.ok) {
        return (void *)0;
    }

    return P;
}

#ifdef CL_HOST
#include <stdio.h>

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 305) - [public] if
    if (f == (void *)0) {
        return -1;
    }

    size_t n = fread(buf, 1, cap, f);

    fclose(f);

    return (long)n;
}
#else
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 311) - [public] fclose
    fclose(f);

    return (long)n;
}
#else
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

static long read_all(const char *path, uint8_t *buf, size_t cap)
{
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 316) - [public] k_fs_read
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

static long read_all(const char *path, uint8_t *buf, size_t cap)
{
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif

cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native)
{
    static uint8_t io[64 * 1024];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 320) - [public] return
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif

cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native)
{
    static uint8_t io[64 * 1024];

    long n = read_all(path, io, sizeof(io));

    if (n <= 0) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 331) - [public] if
        if (n <= 0) {
            return (void *)0;
        }

        cl_module_t *m = (cl_module_t *)cl_arena_alloc(ar, sizeof(cl_module_t));

        if (m == (void *)0) {
            return (void *)0;
        }

        m->native = (void *)0;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 337) - [public] if
        if (m == (void *)0) {
            return (void *)0;
        }

        m->native = (void *)0;

        if (n > 5 && io[0] == 'C' && io[1] == 'L' && io[2] == 'B') {
            m->bc = cl_load_bc(io, (size_t)n, ar);
        } else {
            uint32_t el = 0;
            const char *em = (void *)0;

            // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/loader.c (line 343) - [public] if
    if (n > 5 && io[0] == 'C' && io[1] == 'L' && io[2] == 'B') {
        m->bc = cl_load_bc(io, (size_t)n, ar);
    } else {
        uint32_t el = 0;
        const char *em = (void *)0;

        m->bc = cl_compile_source((const char *)io, (size_t)n,
                                ar, &el, &em);
    }

    if (m->bc == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/loader.c (line 353) - [public] if
    if (m->bc == (void *)0) {
        return (void *)0;
    }

    if (try_native) {
        m->native = cl_native_compile(m->bc);
    }

    return m;
}
```

```
kernel_Workspace/kernel/src/core-lang/loader.c (line 357) - [public] if
    if (try_native) {
        m->native = cl_native_compile(m->bc);
    }

    return m;
}
```

```
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 27) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0xB8); nb_u64(d, v);
}

static void push_rax(nb_t *d) { nb_u8(d, 0x50); }
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 30) - [internal] push_rax
static void push_rax(nb_t *d) { nb_u8(d, 0x50); }
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 31) - [internal] pop_rax
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 32) - [internal] pop_rcx
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 33) - [internal] push_rcx
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 37) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 38) - [public] push_rax
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 43) - [public] pop_rax
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 44) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 49) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    // ... (truncated, see source file for more)
}

```



```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 50) - [public] push_rax
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 55) - [public] pop_rax
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
} nb_patch_t;

static uint8_t g_nb_mem[16384];

void *cl_native_compile(cl_prog_t *P)
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 56) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
} nb_patch_t;

static uint8_t g_nb_mem[16384];

void *cl_native_compile(cl_prog_t *P)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 76) - [public] nb_u8
/* prolog: rbx=slots, rbp=globals, r10=externs */

    nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xFB);
    nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xF5);
    nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);

    uint32_t pc = P->fns[P->main_fn].start;

    for (;;) {
        if (pc >= P->code_len) return (void *)0;

        off_by_insn[pc] = d.len;

        cl_insn_t in = P->code[pc++];
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 77) - [public] nb_u8

```
nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xF5);
nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);
```

```
uint32_t pc = P->fns[P->main_fn].start;
```

```
for (;;) {
    if (pc >= P->code_len) return (void *)0;
```

```
    off_by_insn[pc] = d.len;
```

```
    cl_insn_t in = P->code[pc++];
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 78) - [public] nb_u8

```
nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);
```

```
uint32_t pc = P->fns[P->main_fn].start;
```

```
for (;;) {
    if (pc >= P->code_len) return (void *)0;
```

```
    off_by_insn[pc] = d.len;
```

```
    cl_insn_t in = P->code[pc++];
```

```
    switch (in.op) {
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 89) - [public] switch

```
switch (in.op) {
```

```
case OP_CONST: {
```

```
    uint64_t v = 0;
```

```
    for (int i = 7; i >= 0; i--) {
        v = (v << 8) | P->consts[in.a].b[i];
    }
```

```
    mov_rax_u64(&d, v);
```

```
    push_rax(&d);
```

```
    break;
```

```
}
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 97) - [public] mov_rax_u64

```
mov_rax_u64(&d, v);
```

```
push_rax(&d);
```

```
break;
```

```
}
```

```
case OP_LOAD: load_slot(&d, in.a); break;
```

```
case OP_STORE: store_slot(&d, in.a); break;
```

```
case OP_GLOAD: load_glob(&d, in.a); break;
```

```
case OP_GSTORE: store_glob(&d, in.a); break;
```

```
case OP_ADD:
```

```
    pop_rax(&d); pop_rcx(&d);
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 98) - [public] push_rax

```
    push_rax(&d);
    break;
}

case OP_LOAD: load_slot(&d, in.a); break;
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 102) - [public] load_slot

```
case OP_LOAD: load_slot(&d, in.a); break;
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 103) - [public] store_slot

```
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
    pop_rax(&d); pop_rcx(&d);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 104) - [public] load_glob

```
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 105) - [public] store_glob
    case OP_GSTORE: store_glob(&d, in.a); break;

    case OP_ADD:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 108) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 109) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 110) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 114) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 115) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 116) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 120) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 121) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
                case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 122) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
                case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
                case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 129) - [public] pop_rax
    pop_rax(&d);

    switch (i) {
        case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
        case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
        case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
        case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 131) - [public] switch
    switch (i) {
        case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
        case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
        case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
        case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

    /* rax = externs[a]; call rax; push rax */
    nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 132) - [public] nb_u8
    case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
    case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 133) - [public] nb_u8
    case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 134) - [public] nb_u8
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 135) - [public] nb_u8
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 136) - [public] nb_u8
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 137) - [public] nb_u8
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 142) - [public] nb_u8
/* rax = externs[a]; call rax; push rax */
    nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
    nb_u32(&d, (uint32_t)in.a * 8);
    nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
    push_rax(&d);
    break;
}

case OP_POP: pop_rax(&d); break;

case OP_JMP:
    nb_u8(&d, 0xE9);
    patches[npatches].insn_idx = in.c;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 143) - [public] nb_u32
    nb_u32(&d, (uint32_t)in.a * 8);
    nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
    push_rax(&d);
    break;
}

case OP_POP: pop_rax(&d); break;

case OP_JMP:
    nb_u8(&d, 0xE9);
    patches[npatches].insn_idx = in.c;
    patches[npatches].patch_off = d.len;
// ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/core-lang/native_x86.c (line 144) - [public] nb_u8

```
        nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
        push_rax(&d);
        break;
    }

    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 145) - [public] push_rax

```
        push_rax(&d);
        break;
    }

    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 149) - [public] pop_rax

```
    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_JZ:
        pop_rax(&d);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 152) - [public] nb_u8

```
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_JZ:
        pop_rax(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 156) - [public] nb_u32

```
    nb_u32(&d, 0);
    break;

    case OP_JZ:
        pop_rax(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 160) - [public] pop_rax

```
    pop_rax(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
    nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
    patches[npatches].insn_idx = in.c;
    patches[npatches].patch_off = d.len;
    npatches++;
    nb_u32(&d, 0);
    break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 161) - [public] nb_u8

```
    nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
    nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
    patches[npatches].insn_idx = in.c;
    patches[npatches].patch_off = d.len;
    npatches++;
    nb_u32(&d, 0);
    break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 162) - [public] nb_u8

```
    nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
    patches[npatches].insn_idx = in.c;
    patches[npatches].patch_off = d.len;
    npatches++;
    nb_u32(&d, 0);
    break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
        goto done;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 166) - [public] nb_u32
    nb_u32(&d, 0);
    break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
        goto done;

    default:
        return (void *)0; /* subset v1 */
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 171) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
    nb_u8(&d, 0xC3);
    goto done;

    default:
        return (void *)0; /* subset v1 */
    }
}

done:
for (size_t i = 0; i < npatches; i++) {
    uint32_t target = (uint32_t)off_by_insn[patches[i].insn_idx];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 172) - [public] nb_u8
    nb_u8(&d, 0xC3);
    goto done;

    default:
        return (void *)0; /* subset v1 */
    }
}

done:
for (size_t i = 0; i < npatches; i++) {
    uint32_t target = (uint32_t)off_by_insn[patches[i].insn_idx];
    uint32_t rel = (uint32_t)(target - (patches[i].patch_off + 4));
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 197) - [public] sizeof
    return sizeof(g_nb_mem);
}

#else /* aarch64 / riscv64 – na razie NULL, VM przejmuje */

void *cl_native_compile(cl_prog_t *P)
{
    (void)P;
    return (void *)0;
}

size_t cl_native_size(void)
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 7) - [public] if
    if (n >= cap) {
        n = cap - 1;
    }

    for (size_t i = 0; i < n; i++) {
        dst[i] = t->start[i];
    }

    dst[n] = '\\0';
}

static token_t *peek(parser_t *p)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 27) - [public] if
    if (i >= p->n) {
        i = p->n - 1;
    }

    return &p->toks[i];
}

static token_t *advance(parser_t *p)
{
    token_t *t = &p->toks[p->pos];

    if (p->pos + 1 < p->n) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 38) - [public] if
    if (p->pos + 1 < p->n) {
        p->pos++;
    }

    return t;
}

static bool check(parser_t *p, tok_kind_t k)
{
    return peek(p)->kind == k;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 52) - [public] if
    if (check(p, k)) {
        advance(p);
        return true;
    }

    return false;
}

static void perr(parser_t *p, const char *m)
{
    if (!p->failed) {
        p->failed = true;
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 53) - [public] advance

```
    advance(p);
    return true;
}

return false;
}

static void perr(parser_t *p, const char *m)
{
    if (!p->failed) {
        p->failed = true;
        p->err = m;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 62) - [public] if

```
    if (!p->failed) {
        p->failed = true;
        p->err = m;
        p->err_line = peek(p)->line;
    }
}

static token_t *expect(parser_t *p, tok_kind_t k, const char *m)
{
    if (!check(p, k)) {
        perr(p, m);
        return peek(p);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 71) - [public] if

```
    if (!check(p, k)) {
        perr(p, m);
        return peek(p);
    }

    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 72) - [public] perr

```
    perr(p, m);
    return peek(p);
}

return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 73) - [public] peek

```
    return peek(p);
}

    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 76) - [public] advance

```
    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
static ast_node_t *parse_block(parser_t *p);

/* ---- lista node'ów (rozmie przez arena-realloc nie jest możliwy,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 81) - [public] cl_node

```
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
static ast_node_t *parse_block(parser_t *p);

/* ---- lista node'ów (rozmie przez arena-realloc nie jest możliwy,
    więc zbieramy do tymczasowej tablicy na stacku i kopiujemy) ---- */

#define LIST_MAX 128

typedef struct {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 99) - [public] if

```
    if (l->n < LIST_MAX) {
        l->v[l->n++] = x;
    } else {
        perr(p, "list too long");
    }
}

static void nl_commit(parser_t *p, nlist_t *l, ast_node_t *parent)
{
    ast_node_t **arr = (ast_node_t **)cl_arena_alloc(
        p->ar, l->n * sizeof(ast_node_t *));

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 102) - [public] perr
    perr(p, "list too long");
}
}

static void nl_commit(parser_t *p, nlist_t *l, ast_node_t *parent)
{
    ast_node_t **arr = (ast_node_t **)cl_arena_alloc(
        p->ar, l->n * sizeof(ast_node_t *));

    if (arr == (void *)0) {
        perr(p, "arena full");
        return;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 111) - [public] if
    if (arr == (void *)0) {
        perr(p, "arena full");
        return;
    }

    for (size_t i = 0; i < l->n; i++) {
        arr[i] = l->v[i];
    }

    parent->list = arr;
    parent->nlist = l->n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 112) - [public] perr
    perr(p, "arena full");
    return;
}

    for (size_t i = 0; i < l->n; i++) {
        arr[i] = l->v[i];
    }

    parent->list = arr;
    parent->nlist = l->n;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 128) - [public] if
    if (check(p, TK_TYPE)) {
        out->bits = peek(p)->type_bits;
        out->neg = false;
        advance(p);
        return true;
    }

    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 131) - [public] advance
    advance(p);
    return true;
}

    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */

static ast_node_t *parse_let(parser_t *p)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 135) - [public] perr
    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */

static ast_node_t *parse_let(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_LET);

    expect(p, TK_LET, "let");

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 145) - [public] expect
    expect(p, TK_LET, "let");

    parse_type(p, &n->type);

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 147) - [public] parse_type
    parse_type(p, &n->type);

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        // ... (truncated, see source file for more)
    }
}

```



```
kernel_Workspace/kernel/src/core-lang/parser.c (line 149) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 151) - [public] expect
    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        expect(p, TK_SEMI, "range ;");

        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 158) - [public] if
    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        expect(p, TK_SEMI, "range ;");

        n->type.neg = true;
        n->b = r;
    }

    return n;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 159) - [public] expect
    expect(p, TK_ARROW, "_=>");
    ast_node_t *r = parse_expr(p);
    expect(p, TK_SEMI, "range ;");

    n->type.neg = true;
    n->b = r;
}

return n;
}

static ast_node_t *parse_static(parser_t *p)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 174) - [public] expect

```
expect(p, TK_STATIC, "static");
expect(p, TK_DATA, "static data");
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 175) - [public] expect

```
expect(p, TK_DATA, "static data");
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 176) - [public] expect

```
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 178) - [public] parse_type

```
parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;

expect(p, TK_RBRACK, "]");
expect(p, TK_SEMI, "static ;");
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 180) - [public] expect
    expect(p, TK_COLON, ":");

    n->arr_count = expect(p, TK_NUM, "count")->num;

    expect(p, TK_SEMI, ";");

    n->arr_init = expect(p, TK_NUM, "init")->num;

    expect(p, TK_RBRACK, "];");
    expect(p, TK_SEMI, "static ;");

    n->is_array = true;
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 188) - [public] expect
    expect(p, TK_RBRACK, "];");
    expect(p, TK_SEMI, "static ;");

    n->is_array = true;

    return n;
}

static ast_node_t *parse_fn(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_FN);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 200) - [public] expect
    expect(p, TK_FN, "fn");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "fn name"));

    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 202) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "fn name"));

    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 204) - [public] expect
    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));

        nl_push(p, &params, arg);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 208) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));

        nl_push(p, &params, arg);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 211) - [public] parse_type
    parse_type(p, &arg->type);
    copy_name(arg->name, sizeof(arg->name),
        expect(p, TK_IDENT, "param name"));

    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 213) - [public] expect
    expect(p, TK_IDENT, "param name"));

    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 215) - [public] nl_push
    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);

n->a = parse_block(p);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 217) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);

n->a = parse_block(p);

return n;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 222) - [public] expect
    expect(p, TK_RPAREN, "fn ");

    nl_commit(p, &params, n);

    n->a = parse_block(p);

    return n;
}

static ast_node_t *parse_enum(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_ENUM);
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 224) - [public] nl_commit
    nl_commit(p, &params, n);

    n->a = parse_block(p);

    return n;
}

static ast_node_t *parse_enum(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_ENUM);

    expect(p, TK_ENUM, "enum");
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 235) - [public] expect
    expect(p, TK_ENUM, "enum");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "enum name"));

    expect(p, TK_LBRACE, "enum {");

    nlist_t items = { { 0 }, 0 };
    uint64_t val = 0;

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 237) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "enum name"));

    expect(p, TK_LBRACE, "enum {");

    nlist_t items = { { 0 }, 0 };
    uint64_t val = 0;

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        copy_name(it->name, sizeof(it->name),
            expect(p, TK_IDENT, "enum item"));
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 244) - [public] while
    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        copy_name(it->name, sizeof(it->name),
            expect(p, TK_IDENT, "enum item"));

        it->value = val++;

        nl_push(p, &items, it);

        if (!match(p, TK_COMMA)) {
            break;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 248) - [public] expect
        expect(p, TK_IDENT, "enum item"));

        it->value = val++;

        nl_push(p, &items, it);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBRACE, "enum }");
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 252) - [public] nl_push
    nl_push(p, &items, it);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "enum ");

nl_commit(p, &items, n);

return n;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 254) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "enum ");

nl_commit(p, &items, n);

return n;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 259) - [public] expect
    expect(p, TK_RBRACE, "enum ");

    nl_commit(p, &items, n);

    return n;
}

static ast_node_t *parse_extern(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_EXTERN);

    expect(p, TK_EXTERN, "extern");
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 261) - [public] nl_commit
    nl_commit(p, &items, n);

    return n;
}

static ast_node_t *parse_extern(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_EXTERN);

    expect(p, TK_EXTERN, "extern");

    token_t *abi = expect(p, TK_CHAR, "extern 'C'");
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 270) - [public] expect
    expect(p, TK_EXTERN, "extern");

    token_t *abi = expect(p, TK_CHAR, "extern 'C'");

    n->name[0] = (char)abi->num;
    n->name[1] = '\0';

    expect(p, TK_LBRACE, "extern {");

    nlist_t fns = { { 0 }, 0 };

    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 281) - [public] while
    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *f = new_node(p, ND_VAR);

        copy_name(f->name, sizeof(f->name),
            expect(p, TK_IDENT, "extern fn"));

        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 285) - [public] expect
        expect(p, TK_IDENT, "extern fn"));

        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBACE, "extern }");

    nl_commit(p, &fns, n);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 287) - [public] nl_push
        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBACE, "extern }");

    nl_commit(p, &fns, n);

    return n;
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 289) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "extern ");

nl_commit(p, &fns, n);

return n;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 294) - [public] expect
expect(p, TK_RBRACE, "extern ");

nl_commit(p, &fns, n);

return n;
}

static ast_node_t *parse_dataimpl(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_DATAIMPL);

    expect(p, TK_DATAIMPL, "dataimpl");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 296) - [public] nl_commit
    nl_commit(p, &fns, n);

    return n;
}

static ast_node_t *parse_dataimpl(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_DATAIMPL);

    expect(p, TK_DATAIMPL, "dataimpl");

    expect(p, TK_LBRACE, "dataimpl {");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 305) - [public] expect
    expect(p, TK_DATAIMPL, "dataimpl");

    expect(p, TK_LBRACE, "dataimpl {");

    /* baza: data: ... a■ do } */
    nlist_t base = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        if (match(p, TK_DATA)) {
            expect(p, TK_COLON, "data:");
            continue;
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 312) - [public] while
    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        if (match(p, TK_DATA)) {
            expect(p, TK_COLON, "data:");
            continue;
        }

        nl_push(p, &base, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 313) - [public] if
    if (match(p, TK_DATA)) {
        expect(p, TK_COLON, "data:");
        continue;
    }

    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 314) - [public] expect
    expect(p, TK_COLON, "data:");
    continue;
}

    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RBACE, "dataimpl }");
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 318) - [public] nl_push
    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RBACE, "dataimpl }");

    expect(p, TK_ARROW, "=>");

    ast_node_t *baseblk = new_node(p, ND_BLOCK);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 320) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "dataimpl }");

expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 325) - [public] expect
expect(p, TK_RBRACE, "dataimpl }");

expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

n->a = baseblk;
n->b = parse_block(p);

return n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 327) - [public] expect
expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

n->a = baseblk;
n->b = parse_block(p);

return n;
}

/* ---- statementy ---- */
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 330) - [public] nl_commit
    nl_commit(p, &base, baseblk);

    n->a = baseblk;
    n->b = parse_block(p);

    return n;
}

/* ---- statementy ---- */

static ast_node_t *parse_block(parser_t *p)
{
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 348) - [public] while

```
while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
    ast_node_t *s = (void *)0;

    switch (peek(p)->kind) {
    case TK_LET:      s = parse_let(p); break;
    case TK_STATIC:   s = parse_static(p); break;
    case TK_IF:       s = (void *)1; break; /* ni■ej */
    case TK_WHEN:     s = (void *)1; break;
    case TK_SET_FREE: {
        s = new_node(p, ND_SET_FREE);
        advance(p);
        expect(p, TK_LBRACE, "set_free {");
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 351) - [public] switch

```
switch (peek(p)->kind) {
case TK_LET:      s = parse_let(p); break;
case TK_STATIC:   s = parse_static(p); break;
case TK_IF:       s = (void *)1; break; /* ni■ej */
case TK_WHEN:     s = (void *)1; break;
case TK_SET_FREE: {
    s = new_node(p, ND_SET_FREE);
    advance(p);
    expect(p, TK_LBRACE, "set_free {");
    copy_name(s->name, sizeof(s->name),
               expect(p, TK_IDENT, "set_free var"));
    expect(p, TK_RBACE, "set_free }");
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 358) - [public] advance

```
advance(p);
expect(p, TK_LBRACE, "set_free {");
copy_name(s->name, sizeof(s->name),
           expect(p, TK_IDENT, "set_free var"));
expect(p, TK_RBACE, "set_free }");
expect(p, TK_SEMI, "set_free ;");
break;
}
default:
    s = (void *)1;
    break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 361) - [public] expect

```
expect(p, TK_IDENT, "set_free var"));
expect(p, TK_RBACE, "set_free }");
expect(p, TK_SEMI, "set_free ;");
break;
}
default:
    s = (void *)1;
    break;
}

if (s == (void *)1) {
    if (check(p, TK_IF)) {
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 362) - [public] expect
    expect(p, TK_RBRACE, "set_free }");
    expect(p, TK_SEMI, "set_free ;");
    break;
}
default:
    s = (void *)1;
    break;
}

    if (s == (void *)1) {
        if (check(p, TK_IF)) {
            extern ast_node_t *cl_parse_if(parser_t *p);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 371) - [public] if
    if (s == (void *)1) {
        if (check(p, TK_IF)) {
            extern ast_node_t *cl_parse_if(parser_t *p);
            s = cl_parse_if(p);
        } else if (check(p, TK_WHEN)) {
            extern ast_node_t *cl_parse_when(parser_t *p);
            s = cl_parse_when(p);
        } else {
            s = parse_expr(p);
            expect(p, TK_SEMI, "stmt ;");
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 372) - [public] if
    if (check(p, TK_IF)) {
        extern ast_node_t *cl_parse_if(parser_t *p);
        s = cl_parse_if(p);
    } else if (check(p, TK_WHEN)) {
        extern ast_node_t *cl_parse_when(parser_t *p);
        s = cl_parse_when(p);
    } else {
        s = parse_expr(p);
        expect(p, TK_SEMI, "stmt ;");
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 384) - [public] nl_push
    nl_push(p, &stmts, s);
}

    expect(p, TK_RBRACE, "}");

    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 387) - [public] expect
    expect(p, TK_RBRACE, "}");

    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_IF);

    expect(p, TK_IF, "if");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 389) - [public] nl_commit
    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_IF);

    expect(p, TK_IF, "if");

    if (!check(p, TK_LBRACE)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 398) - [public] expect
    expect(p, TK_IF, "if");

    if (!check(p, TK_LBRACE)) {
        n->a = parse_expr(p);
    }

    n->b = parse_block(p);

    /* else / else if / if else */
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 400) - [public] if
    if (!check(p, TK_LBRACE)) {
        n->a = parse_expr(p);
    }

    n->b = parse_block(p);

    /* else / else if / if else */
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);
        }

        ast_node_t *inner = new_node(p, ND_IF);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 407) - [public] if

/ else / else if / if else */*

```
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);

            ast_node_t *inner = new_node(p, ND_IF);

            if (!check(p, TK_LBRACE)) {
                inner->a = parse_expr(p);
            }

            inner->b = parse_block(p);
            n->c = inner;
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 408) - [public] if

```
    if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
        advance(p);

        ast_node_t *inner = new_node(p, ND_IF);

        if (!check(p, TK_LBRACE)) {
            inner->a = parse_expr(p);
        }

        inner->b = parse_block(p);
        n->c = inner;
    } else {
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 409) - [public] advance

```
    advance(p);

    ast_node_t *inner = new_node(p, ND_IF);

    if (!check(p, TK_LBRACE)) {
        inner->a = parse_expr(p);
    }

    inner->b = parse_block(p);
    n->c = inner;
} else {
    n->c = parse_block(p);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 413) - [public] if

```
    if (!check(p, TK_LBRACE)) {
        inner->a = parse_expr(p);
    }

    inner->b = parse_block(p);
    n->c = inner;
} else {
    n->c = parse_block(p);
}
} else if (check(p, TK_IF) && peek_at(p, 1)->kind == TK_ELSE) {
    advance(p);
    advance(p);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 423) - [public] advance

```
    advance(p);
    advance(p);
    n->c = parse_block(p);
}

return n;
}

ast_node_t *cl_parse_when(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_WHEN);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 424) - [public] advance

```
    advance(p);
    n->c = parse_block(p);
}

return n;
}

ast_node_t *cl_parse_when(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_WHEN);

    expect(p, TK_WHEN, "when");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 435) - [public] expect

```
    expect(p, TK_WHEN, "when");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "when target"));

    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 437) - [public] copy_name

```
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "when target"));

    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);
        // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 439) - [public] expect
    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);

        nl_push(p, &cases, c);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 444) - [public] while
    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);

        nl_push(p, &cases, c);
    }

    expect(p, TK_RBRACE, "when }");

    nl_commit(p, &cases, n);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 450) - [public] nl_push
    nl_push(p, &cases, c);
}

    expect(p, TK_RBRACE, "when }");

    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 453) - [public] expect
    expect(p, TK_RBRACE, "when }");

    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

static ast_node_t *parse_primary(parser_t *p)
{
    ast_node_t *n;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 455) - [public] nl_commit
    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

static ast_node_t *parse_primary(parser_t *p)
{
    ast_node_t *n;

    if (check(p, TK_NUM)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 466) - [public] if
        if (check(p, TK_NUM)) {
            n = new_node(p, ND_NUM);
            n->value = advance(p)->num;
            return n;
        }

        if (check(p, TK_STR)) {
            n = new_node(p, ND_STR);
            token_t *t = advance(p);
            copy_name(n->name, sizeof(n->name), t);
            return n;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 472) - [public] if
        if (check(p, TK_STR)) {
            n = new_node(p, ND_STR);
            token_t *t = advance(p);
            copy_name(n->name, sizeof(n->name), t);
            return n;
        }

        if (check(p, TK_CHAR)) {
            n = new_node(p, ND_CHAR);
            n->value = advance(p)->num;
            return n;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 475) - [public] copy_name
        copy_name(n->name, sizeof(n->name), t);
        return n;
    }

    if (check(p, TK_CHAR)) {
        n = new_node(p, ND_CHAR);
        n->value = advance(p)->num;
        return n;
    }

    if (check(p, TK_DOLLAR) || check(p, TK_DOLLAR_BANG) ||
        check(p, TK_DOLLAR_AT)) {
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 479) - [public] if

```
if (check(p, TK_CHAR)) {
    n = new_node(p, ND_CHAR);
    n->value = advance(p)->num;
    return n;
}

if (check(p, TK_DOLLAR) || check(p, TK_DOLLAR_BANG) ||
    check(p, TK_DOLLAR_AT)) {
    n = new_node(p, ND_POOL);
    token_t *t = advance(p);

    n->pool_kind = (t->kind == TK_DOLLAR) ? 0
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 486) - [public] check

```
check(p, TK_DOLLAR_AT)) {
    n = new_node(p, ND_POOL);
    token_t *t = advance(p);

    n->pool_kind = (t->kind == TK_DOLLAR) ? 0
                  : (t->kind == TK_DOLLAR_BANG) ? 1 : 2;

    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "$var"));
    return n;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 494) - [public] expect

```
expect(p, TK_IDENT, "$var"));
return n;
}

if (check(p, TK_CALL) && peek_at(p, 1)->kind == TK_LBRACK) {
    n = new_node(p, ND_CALLIDX);
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);
    return n;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 498) - [public] if

```
if (check(p, TK_CALL) && peek_at(p, 1)->kind == TK_LBRACK) {
    n = new_node(p, ND_CALLIDX);
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 500) - [public] advance

```
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 501) - [public] expect

```
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 503) - [public] expect

```
    expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}

if (check(p, TK_IDENT)) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 504) - [public] expect

```
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}

if (check(p, TK_IDENT)) {
    n = new_node(p, ND_VAR);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 508) - [public] if

```
    if (match(p, TK_LPAREN)) {
        n = parse_expr(p);
        expect(p, TK_RPAREN, "");
        return n;
    }

    if (check(p, TK_IDENT)) {
        n = new_node(p, ND_VAR);
        copy_name(n->name, sizeof(n->name), advance(p));
        return n;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 510) - [public] expect

```
    expect(p, TK_RPAREN, "");
    return n;
}

if (check(p, TK_IDENT)) {
    n = new_node(p, ND_VAR);
    copy_name(n->name, sizeof(n->name), advance(p));
    return n;
}

perr(p, "unexpected token in expr");
advance(p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 514) - [public] if

```
    if (check(p, TK_IDENT)) {
        n = new_node(p, ND_VAR);
        copy_name(n->name, sizeof(n->name), advance(p));
        return n;
    }

    perr(p, "unexpected token in expr");
    advance(p);

    return new_node(p, ND_NUM);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 516) - [public] copy_name

```
    copy_name(n->name, sizeof(n->name), advance(p));
    return n;
}

perr(p, "unexpected token in expr");
advance(p);

return new_node(p, ND_NUM);
}

static ast_node_t *parse_postfix(parser_t *p)
{
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 520) - [public] perr
    perr(p, "unexpected token in expr");
    advance(p);

    return new_node(p, ND_NUM);
}
```

```
static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 521) - [public] advance
    advance(p);

    return new_node(p, ND_NUM);
}
```

```
static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            ast_node_t *m = new_node(p, ND_MEMBER);
            // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 523) - [public] new_node
    return new_node(p, ND_NUM);
}
```

```
static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            ast_node_t *m = new_node(p, ND_MEMBER);

            m->a = n;
            // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 531) - [public] if
    if (match(p, TK_DOT)) {
        ast_node_t *m = new_node(p, ND_MEMBER);

        m->a = n;
        copy_name(m->name, sizeof(m->name),
            expect(p, TK_IDENT, "member"));
        n = m;
    } else if (check(p, TK_DCOLON)) {
        advance(p);

        ast_node_t *m = new_node(p, ND_SCOPECALL);

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 536) - [public] expect
        expect(p, TK_IDENT, "member"));
        n = m;
    } else if (check(p, TK_DCOLON)) {
        advance(p);

        ast_node_t *m = new_node(p, ND_SCOPECALL);

        for (size_t i = 0; i < sizeof(m->name); i++) {
            m->name[i] = 0;
        }

        if (n->kind == ND_VAR) {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 539) - [public] advance
        advance(p);

        ast_node_t *m = new_node(p, ND_SCOPECALL);

        for (size_t i = 0; i < sizeof(m->name); i++) {
            m->name[i] = 0;
        }

        if (n->kind == ND_VAR) {
            for (size_t i = 0; n->name[i] && i < 63; i++) {
                m->name[i] = n->name[i];
            }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 547) - [public] if
        if (n->kind == ND_VAR) {
            for (size_t i = 0; n->name[i] && i < 63; i++) {
                m->name[i] = n->name[i];
            }
        }

        copy_name(m->name2, sizeof(m->name2),
            expect(p, TK_IDENT, "scope fn"));

        expect(p, TK_LPAREN, "scope (");

        nlist_t args = { { 0 }, 0 };
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 554) - [public] expect
        expect(p, TK_IDENT, "scope fn"));

        expect(p, TK_LPAREN, "scope (");

        nlist_t args = { { 0 }, 0 };

        while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
            nl_push(p, &args, parse_expr(p));

            if (!match(p, TK_COMMA)) {
                break;
            }
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 556) - [public] expect
    expect(p, TK_LPAREN, "scope (");

    nlist_t args = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 560) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 561) - [public] nl_push
    nl_push(p, &args, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    n = m;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 563) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 568) - [public] expect
    expect(p, TK_RPAREN, "::-");

    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 570) - [public] nl_commit
    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    nlist_t args = { { 0 }, 0 };

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 574) - [public] advance
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    nlist_t args = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 582) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RPAREN, "call ");

    nl_commit(p, &args, m);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 583) - [public] nl_push
    nl_push(p, &args, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "call ");

nl_commit(p, &args, m);

n = m;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 585) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "call ");

nl_commit(p, &args, m);

n = m;
} else {
    break;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 590) - [public] expect
    expect(p, TK_RPAREN, "call ");

    nl_commit(p, &args, m);

    n = m;
} else {
    break;
}

return n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 592) - [public] nl_commit
    nl_commit(p, &args, m);

    n = m;
} else {
    break;
}

return n;
}

static ast_node_t *parse_unary(parser_t *p)
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 606) - [public] check

```
    check(p, TK_TILDE)) {
        ast_node_t *n = new_node(p, ND_UN);

        n->op = advance(p)->kind;
        n->a = parse_unary(p);

        return n;
    }

    return parse_postfix(p);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 615) - [public] parse_postfix

```
    return parse_postfix(p);
}

static ast_node_t *parse_bin(parser_t *p, int level)
{
    ast_node_t *lhs = parse_unary(p);

    for (;;) {
        tok_kind_t k = peek(p)->kind;
        int lv = 0;

        switch (k) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 626) - [public] switch

```
        switch (k) {
            case TK_STAR: case TK_SLASH: case TK_PERCENT: lv = 1; break;
            case TK_PLUS: case TK_MINUS: lv = 2; break;
            case TK_SHL: case TK_SHR: lv = 3; break;
            case TK_LT: case TK_GT: case TK_LEQ: case TK_GEQ: lv = 4; break;
            case TK_EQ: case TK_NEQ: lv = 5; break;
            case TK_AMP: lv = 6; break;
            case TK_CARET: lv = 7; break;
            case TK_PIPE: lv = 8; break;
            default: return lhs;
        }

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 638) - [public] if

```
        if (lv > level) {
            return lhs;
        }

        advance(p);

        ast_node_t *rhs = parse_bin(p, lv + 1 > 8 ? 8 : lv + 1);

        ast_node_t *b = new_node(p, ND_BIN);

        b->op = k;
        b->a = lhs;
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 642) - [public] advance
    advance(p);

    ast_node_t *rhs = parse_bin(p, lv + 1 > 8 ? 8 : lv + 1);

    ast_node_t *b = new_node(p, ND_BIN);

    b->op = k;
    b->a = lhs;
    b->b = rhs;

    lhs = b;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 662) - [public] if
    if (match(p, TK_ASSIGN)) {
        ast_node_t *n = new_node(p, ND_ASSIGN);

        n->a = lhs;
        n->b = parse_assign(p);

        return n;
    }

    return lhs;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 676) - [public] parse_assign
    return parse_assign(p);
}

/* ---- top ---- */

ast_node_t *cl_parse(parser_t *p, token_t *toks, size_t n, arena_t *ar)
{
    p->toks = toks;
    p->n = n;
    p->pos = 0;
    p->ar = ar;
    p->failed = false;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 694) - [public] while
    while (!check(p, TK_EOF)) {
        ast_node_t *d;

        switch (peek(p)->kind) {
            case TK_FN:      d = parse_fn(p); break;
            case TK_LET:     d = parse_let(p); break;
            case TK_STATIC:  d = parse_static(p); break;
            case TK_ENUM:    d = parse_enum(p); break;
            case TK_EXTERN:  d = parse_extern(p); break;
            case TK_DATAIMPL: d = parse_dataimpl(p); break;
            default:
                perr(p, "expected top-level decl");
        }
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 697) - [public] switch

```
    switch (peek(p)->kind) {
    case TK_FN:      d = parse_fn(p); break;
    case TK_LET:     d = parse_let(p); break;
    case TK_STATIC:  d = parse_static(p); break;
    case TK_ENUM:    d = parse_enum(p); break;
    case TK_EXTERN:  d = parse_extern(p); break;
    case TK_DATAIMPL: d = parse_dataimpl(p); break;
    default:
        perr(p, "expected top-level decl");
        advance(p);
        continue;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 705) - [public] perr

```
    perr(p, "expected top-level decl");
    advance(p);
    continue;
}

nl_push(p, &tops, d);

if (p->failed) {
    break;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 706) - [public] advance

```
    advance(p);
    continue;
}

nl_push(p, &tops, d);

if (p->failed) {
    break;
}

nl_commit(p, &tops, prog);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 710) - [public] nl_push

```
    nl_push(p, &tops, d);

    if (p->failed) {
        break;
    }

    nl_commit(p, &tops, prog);

    if (p->failed) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 712) - [public] if
    if (p->failed) {
        break;
    }
}

nl_commit(p, &tops, prog);

if (p->failed) {
    return (void *)0;
}

return prog;
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 717) - [public] nl_commit
    nl_commit(p, &tops, prog);

    if (p->failed) {
        return (void *)0;
    }

    return prog;
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 719) - [public] if
    if (p->failed) {
        return (void *)0;
    }

    return prog;
}
```

```
kernel_Workspace/kernel/src/core-lang/run.c (line 4) - [public] cl_kernel_read
extern long cl_kernel_read(const char *path, uint8_t *buf, size_t cap);
extern void *cl_native_compile(cl_prog_t *P);

int cl_run_script(const char *path, uint8_t ring, arena_t *ar)
{
    static uint8_t io[64 * 1024];

    long n = cl_kernel_read(path, io, sizeof(io));

    if (n <= 0) {
        return -1;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/run.c (line 13) - [public] if
    if (n <= 0) {
        return -1;
    }

    uint32_t el = 0;
    const char *em = (void *)0;

    cl_prog_t *P = cl_compile_source((const char *)io, (size_t)n, ar, &el, &em);

    if (P == (void *)0) {
        extern void kprintf(const char *, ...);
        kprintf("core-lang: %s line %u: %s\n", path, el, em);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 22) - [public] if

```
    if (P == (void *)0) {
        extern void kprintf(const char *, ...);
        kprintf("core-lang: %s line %u: %s\n", path, el, em);
        return -1;
    }

    cl_vm_t vm;

    cl_vm_init(&vm, P);
    cl_bridge_init(&vm, ring);

    void *code = cl_native_compile(P);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 23) - [public] kprintf

```
    extern void kprintf(const char *, ...);
    kprintf("core-lang: %s line %u: %s\n", path, el, em);
    return -1;
}

cl_vm_t vm;

cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 24) - [public] kprintf

```
    kprintf("core-lang: %s line %u: %s\n", path, el, em);
    return -1;
}

cl_vm_t vm;

cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

if (code != (void *)0) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 30) - [public] cl_vm_init

```
cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

if (code != (void *)0) {
    extern void cl_make_exec(void *p, size_t len);
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/run.c (line 31) - [public] cl_bridge_init
    cl_bridge_init(&vm, ring);

    void *code = cl_native_compile(P);

    if (code != (void *)0) {
        extern void cl_make_exec(void *p, size_t len);
        cl_make_exec(code, 16384);

        static uint64_t slots[64], globs[64];
        static cl_ext_fn exts[16];

        for (size_t i = 0; i < 64; i++) {
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/run.c (line 35) - [public] if
    if (code != (void *)0) {
        extern void cl_make_exec(void *p, size_t len);
        cl_make_exec(code, 16384);

        static uint64_t slots[64], globs[64];
        static cl_ext_fn exts[16];

        for (size_t i = 0; i < 64; i++) {
            globs[i] = 0;

            for (int k = 0; k < 8; k++) {
                globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/run.c (line 36) - [public] cl_make_exec
    extern void cl_make_exec(void *p, size_t len);
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    for (size_t i = 0; i < 64; i++) {
        globs[i] = 0;

        for (int k = 0; k < 8; k++) {
            globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/run.c (line 37) - [public] cl_make_exec
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    for (size_t i = 0; i < 64; i++) {
        globs[i] = 0;

        for (int k = 0; k < 8; k++) {
            globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/run.c (line 57) - [public] cl_vm_run
    cl_vm_run(&vm);
}

return 0;
}

```


kernel_Workspace/kernel/src/core-lang/sema.c (line 45) - [public] if

```
    if (s == (void *)0) {
        return (void *)0;
    }

    s->ar = ar;
    s->top = 0;

    return s;
}

static void diag(sema_ctx_t *s, uint32_t line, const char *msg,
                const char *name, bool warn)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 58) - [public] if

```
    if (s->ndiags < 64) {
        cl_diag_t *d = &s->diags[s->ndiags++];

        d->line = line;
        d->msg = msg;
        d->is_warn = warn;
        d->name[0] = '\0';

        if (name != (void *)0) {
            for (int i = 0; name[i] && i < 63; i++) {
                d->name[i] = name[i];
                d->name[i + 1] = '\0';
            }
        }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 66) - [public] if

```
        if (name != (void *)0) {
            for (int i = 0; name[i] && i < 63; i++) {
                d->name[i] = name[i];
                d->name[i + 1] = '\0';
            }
        }
    }
}

static void push_scope(sema_ctx_t *s)
{
    if (s->top < 15) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 77) - [public] if

```
    if (s->top < 15) {
        s->top++;
        s->scopes[s->top].head = (void *)0;
    }
}

static void pop_scope(sema_ctx_t *s)
{
    for (symbol_t *sym = s->scopes[s->top].head; sym; sym = sym->next) {
        if (sym->kind == SYM_VAR && sym->live) {
            diag(s, 0, "warning: variable left alive without set_free",
                sym->name, true);
        }
// ... (truncated, see source file for more)
    }
}
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 86) - [public] if
    if (sym->kind == SYM_VAR && sym->live) {
        diag(s, 0, "warning: variable left alive without set_free",
            sym->name, true);
    }
}

if (s->top > 0) {
    s->top--;
}

static symbol_t *declare(sema_ctx_t *s, const char *name, uint8_t kind,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 92) - [public] if
    if (s->top > 0) {
        s->top--;
    }
}

static symbol_t *declare(sema_ctx_t *s, const char *name, uint8_t kind,
    cl_type_t type, uint32_t line)
{
    for (symbol_t *sym = s->scopes[s->top].head; sym; sym = sym->next) {
        bool same = true;

        for (int i = 0; name[i] || sym->name[i]; i++) {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 104) - [public] if
        if (name[i] != sym->name[i]) {
            same = false;
            break;
        }
    }

    if (same) {
        diag(s, line, "redeclaration in same scope", name, false);
        return sym;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 110) - [public] if
    if (same) {
        diag(s, line, "redeclaration in same scope", name, false);
        return sym;
    }
}

symbol_t *sym = (symbol_t *)cl_arena_alloc(s->ar, sizeof(symbol_t));

if (sym == (void *)0) {
    return (void *)0;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 111) - [public] diag
    diag(s, line, "redeclaration in same scope", name, false);
    return sym;
}
}

symbol_t *sym = (symbol_t *)cl_arena_alloc(s->ar, sizeof(symbol_t));

if (sym == (void *)0) {
    return (void *)0;
}

for (int i = 0; name[i] && i < 63; i++) {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 118) - [public] if
    if (sym == (void *)0) {
        return (void *)0;
    }

    for (int i = 0; name[i] && i < 63; i++) {
        sym->name[i] = name[i];
        sym->name[i + 1] = '\0';
    }

    sym->kind = kind;
    sym->type = type;
    sym->live = true;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 149) - [public] if
    if (name[k] != sym->name[k]) {
        same = false;
        break;
    }
}

if (same) {
    return sym;
}
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 155) - [public] if
    if (same) {
        return sym;
    }
}

return (void *)0;
}

```

```

static bool value_fits(uint64_t v, bool negative, cl_type_t t)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 167) - [public] if
    if (t.bits == 0) {
        return true;
    }

    if (t.bits >= 64) {
        return true;
    }

    if (negative) {
        if (!t.neg) {
            return false;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 171) - [public] if
    if (t.bits >= 64) {
        return true;
    }

    if (negative) {
        if (!t.neg) {
            return false;
        }
    }

    return v <= (1ULL << (t.bits - 1));
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 175) - [public] if
    if (negative) {
        if (!t.neg) {
            return false;
        }
    }

    return v <= (1ULL << (t.bits - 1));
}

    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 176) - [public] if
    if (!t.neg) {
        return false;
    }

    return v <= (1ULL << (t.bits - 1));
}

    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    return v < (1ULL << t.bits);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 183) - [public] if
    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    return v < (1ULL << t.bits);
}

typedef struct {
    cl_type_t t;
    bool is_const;
    uint64_t cval;
    bool cneg;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 197) - [public] check_expr
static etype_t check_expr(sema_ctx_t *s, ast_node_t *n);

static void shared_note(sema_ctx_t *s, const char *name, cl_type_t t,
                        uint32_t line)
{
    if (s->sh_count < 2) {
        for (int i = 0; name[i] && i < 63; i++) {
            s->sh_name[s->sh_count][i] = name[i];
            s->sh_name[s->sh_count][i + 1] = '\0';
        }

        s->sh_type[s->sh_count] = t;
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 202) - [public] if
    if (s->sh_count < 2) {
        for (int i = 0; name[i] && i < 63; i++) {
            s->sh_name[s->sh_count][i] = name[i];
            s->sh_name[s->sh_count][i + 1] = '\0';
        }

        s->sh_type[s->sh_count] = t;
        s->sh_line[s->sh_count] = line;
        s->sh_count++;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 216) - [public] if
    if (s->sh_count == 1) {
        diag(s, s->sh_line[0], "$@ requires two participating variables",
            s->sh_name[0], false);
    } else if (s->sh_count == 2) {
        if (s->sh_type[0].bits != s->sh_type[1].bits) {
            diag(s, s->sh_line[1], "$@ pair must share the same range",
                s->sh_name[1], false);
        }
    }

    s->sh_count = 0;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 220) - [public] if
    if (s->sh_type[0].bits != s->sh_type[1].bits) {
        diag(s, s->sh_line[1], "$@ pair must share the same range",
            s->sh_name[1], false);
    }
}

s->sh_count = 0;
}

static etype_t check_expr(sema_ctx_t *s, ast_node_t *n)
{
    etype_t r = { { 0, false }, false, 0, false };
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 233) - [public] if
    if (n == (void *)0) {
        return r;
    }

    switch (n->kind) {
    case ND_NUM:
        r.is_const = true;
        r.cval = n->value;
        return r;

    case ND_CHAR:
        r.is_const = true;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 237) - [public] switch
    switch (n->kind) {
    case ND_NUM:
        r.is_const = true;
        r.cval = n->value;
        return r;

    case ND_CHAR:
        r.is_const = true;
        r.cval = n->value;
        r.t.bits = 64;
        return r;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 255) - [public] if
    if (n->op == 45 ) {
        if (r.is_const) {
            r.cneg = true;
        } else if (r.t.bits != 0 && !r.t.neg) {
            diag(s, n->line, "negation needs _=> range", (void *)0, false);
        }
    }

    return r;

    case ND_VAR: {
        symbol_t *sym = lookup(s, n->name);
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 256) - [public] if
    if (r.is_const) {
        r.cneg = true;
    } else if (r.t.bits != 0 && !r.t.neg) {
        diag(s, n->line, "negation needs _=> range", (void *)0, false);
    }
}

return r;

case ND_VAR: {
    symbol_t *sym = lookup(s, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 259) - [public] diag
    diag(s, n->line, "negation needs _=> range", (void *)0, false);
}

return r;

case ND_VAR: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 268) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }

    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 269) - [public] diag
    diag(s, n->line, "undeclared identifier", n->name, false);
    return r;
}

    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        r.t.bits = 64;
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 273) - [public] if
    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        r.t.bits = 64;
    } else {
        diag(s, n->line, "not a value", n->name, false);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 274) - [public] if
    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = sym->type;
} else if (sym->kind == SYM_ENUM) {
    r.t.bits = 64;
} else {
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 275) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

    r.t = sym->type;
} else if (sym->kind == SYM_ENUM) {
    r.t.bits = 64;
} else {
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 282) - [public] diag
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
}

case ND_POOL: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 291) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }

    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 292) - [public] diag
    diag(s, n->line, "undeclared identifier", n->name, false);
    return r;
}

    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 296) - [public] if
    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 301) - [public] if
    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 302) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (_=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 307) - [public] if
    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (_=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 308) - [public] diag
    diag(s, n->line, "$! needs neg range (_=>-1)", n->name, false);
}

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 311) - [public] if
    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}

case ND_CALLIDX: {
    symbol_t *sym = lookup(s, n->name);

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 313) - [public] shared_note
    shared_note(s, n->name, sym->type, n->line);
}

    r.t = sym->type;
    return r;
}

case ND_CALLIDX: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "call[] on undeclared", n->name, false);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 323) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "call[] on undeclared", n->name, false);
        return r;
    }

    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 324) - [public] diag
    diag(s, n->line, "call[] on undeclared", n->name, false);
    return r;
}

    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
}
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 328) - [public] if
    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
}

case ND_SCOPECALL: {
    symbol_t *sc = lookup(s, n->name);

    // ... (truncated, see source file for more)
}

```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 329) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
    ? (cl_type_t){ 64, false } : sym->type;
return r;
}
```

```
case ND_SCOPECALL: {
    symbol_t *sc = lookup(s, n->name);

    if (sc == (void *)0) {
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 340) - [public] if
    if (sc == (void *)0) {
        diag(s, n->line, "unknown scope", n->name, false);
        return r;
    }
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];
```

```
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 341) - [public] diag
    diag(s, n->line, "unknown scope", n->name, false);
    return r;
}
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];
```

```
                bool same = true;
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 345) - [public] if
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];

                bool same = true;

                for (int k = 0; it->name[k] || n->name2[k]; k++) {
                    if (it->name[k] != n->name2[k]) {
                        same = false;
                    }
                }
            }
        }
// ... (truncated, see source file for more)
}
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 348) - [public] if
    if (sc->node != (void *)0) {
        for (size_t i = 0; i < sc->node->nlist; i++) {
            ast_node_t *it = sc->node->list[i];

            bool same = true;

            for (int k = 0; it->name[k] || n->name2[k]; k++) {
                if (it->name[k] != n->name2[k]) {
                    same = false;
                    break;
                }
            }
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 355) - [public] if
        if (it->name[k] != n->name2[k]) {
            same = false;
            break;
        }

        if (same) {
            found = true;
            r.is_const = true;
            r.cval = it->value;
            r.t.bits = 64;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 361) - [public] if
        if (same) {
            found = true;
            r.is_const = true;
            r.cval = it->value;
            r.t.bits = 64;
        }
    }

    if (!found) {
        diag(s, n->line, "no such enum item", n->name2, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 370) - [public] if
    if (!found) {
        diag(s, n->line, "no such enum item", n->name2, false);
    }

    return r;
}

if (sc->kind == SYM_DATA) {
    bool found = false;

    if (sc->node != (void *)0 && sc->node->a != (void *)0) {
        ast_node_t *base = sc->node->a;
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 371) - [public] diag
    diag(s, n->line, "no such enum item", n->name2, false);
}

return r;
}

if (sc->kind == SYM_DATA) {
    bool found = false;

    if (sc->node != (void *)0 && sc->node->a != (void *)0) {
        ast_node_t *base = sc->node->a;

        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 377) - [public] if
    if (sc->kind == SYM_DATA) {
        bool found = false;

        if (sc->node != (void *)0 && sc->node->a != (void *)0) {
            ast_node_t *base = sc->node->a;

            for (size_t i = 0; i < base->nlist; i++) {
                ast_node_t *e = base->list[i];

                if (e->kind == ND_VAR) {
                    bool same = true;

                    // ... (truncated, see source file for more)
                }

kernel_Workspace/kernel/src/core-lang/sema.c (line 380) - [public] if
    if (sc->node != (void *)0 && sc->node->a != (void *)0) {
        ast_node_t *base = sc->node->a;

        for (size_t i = 0; i < base->nlist; i++) {
            ast_node_t *e = base->list[i];

            if (e->kind == ND_VAR) {
                bool same = true;

                for (int k = 0; e->name[k] || n->name2[k]; k++) {
                    if (e->name[k] != n->name2[k]) {
                        same = false;
                    }
                }

                // ... (truncated, see source file for more)
            }

kernel_Workspace/kernel/src/core-lang/sema.c (line 386) - [public] if
    if (e->kind == ND_VAR) {
        bool same = true;

        for (int k = 0; e->name[k] || n->name2[k]; k++) {
            if (e->name[k] != n->name2[k]) {
                same = false;
                break;
            }
        }

        if (same) {
            found = true;
        }

        // ... (truncated, see source file for more)
    }
}

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 390) - [public] if
        if (e->name[k] != n->name2[k]) {
            same = false;
            break;
        }
    }

    if (same) {
        found = true;
        r.t.bits = 64;
    }
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 396) - [public] if
        if (same) {
            found = true;
            r.t.bits = 64;
        }
    }
}

    if (!found) {
        diag(s, n->line, "no such data entry", n->name2, false);
    }

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 404) - [public] if
    if (!found) {
        diag(s, n->line, "no such data entry", n->name2, false);
    }

    return r;
}

    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 405) - [public] diag
        diag(s, n->line, "no such data entry", n->name2, false);
    }

    return r;
}

    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 411) - [public] diag
    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
        diag(s, n->line, "bad call target", (void *)0, false);
        return r;
    }

    symbol_t *fn = lookup(s, n->a->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 416) - [public] if
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
        diag(s, n->line, "bad call target", (void *)0, false);
        return r;
    }

    symbol_t *fn = lookup(s, n->a->name);

    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 417) - [public] diag
    diag(s, n->line, "bad call target", (void *)0, false);
    return r;
}

    symbol_t *fn = lookup(s, n->a->name);

    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    if (fn->kind == SYM_FN) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 423) - [public] if
    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    if (fn->kind == SYM_FN) {
        if (fn->nparams != n->nlist) {
            diag(s, n->line, "arity mismatch", fn->name, false);
        }

        for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
            etype_t a = check_expr(s, n->list[i]);
            // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 424) - [public] diag
    diag(s, n->line, "call of undeclared", n->a->name, false);
    return r;
}

if (fn->kind == SYM_FN) {
    if (fn->nparams != n->nlist) {
        diag(s, n->line, "arity mismatch", fn->name, false);
    }

    for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
        etype_t a = check_expr(s, n->list[i]);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 428) - [public] if
    if (fn->kind == SYM_FN) {
        if (fn->nparams != n->nlist) {
            diag(s, n->line, "arity mismatch", fn->name, false);
        }

        for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
            etype_t a = check_expr(s, n->list[i]);

            if (a.is_const &&
                !value_fits(a.cval, a.cneg, fn->params[i])) {
                diag(s, n->line, "arg out of range", fn->name, false);
            }
        }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 429) - [public] if
    if (fn->nparams != n->nlist) {
        diag(s, n->line, "arity mismatch", fn->name, false);
    }

    for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
        etype_t a = check_expr(s, n->list[i]);

        if (a.is_const &&
            !value_fits(a.cval, a.cneg, fn->params[i])) {
            diag(s, n->line, "arg out of range", fn->name, false);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 430) - [public] diag
    diag(s, n->line, "arity mismatch", fn->name, false);
}

for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
    etype_t a = check_expr(s, n->list[i]);

    if (a.is_const &&
        !value_fits(a.cval, a.cneg, fn->params[i])) {
        diag(s, n->line, "arg out of range", fn->name, false);
    }
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 438) - [public] diag
        diag(s, n->line, "arg out of range", fn->name, false);
    }
}

    r.t.bits = 64;
    return r;
}

    if (fn->kind == SYM_EXTERN) {
        for (size_t i = 0; i < n->nlist; i++) {
            check_expr(s, n->list[i]);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 446) - [public] if
    if (fn->kind == SYM_EXTERN) {
        for (size_t i = 0; i < n->nlist; i++) {
            check_expr(s, n->list[i]);
        }

        r.t.bits = 64;
        return r;
    }

    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 448) - [public] check_expr
        check_expr(s, n->list[i]);
    }

    r.t.bits = 64;
    return r;
}

    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}

case ND_MEMBER:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 455) - [public] diag
    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}

case ND_MEMBER:
    r = check_expr(s, n->a);
    return r;

case ND_ASSIGN: {
    etype_t l = check_expr(s, n->a);
    etype_t rhs = check_expr(s, n->b);

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 467) - [public] if
    if (l.t.bits != 0) {
        if (rhs.is_const) {
            if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
                diag(s, n->line, "value out of range", (void *)0, false);
            }
        }
        } else if (rhs.t.bits != 0) {
            if (rhs.t.bits > l.t.bits) {
                diag(s, n->line, "narrowing assignment", (void *)0, false);
            } else if (rhs.t.bits == l.t.bits &&
                rhs.t.neg != l.t.neg) {
                diag(s, n->line, "signed/unsigned mismatch",
                    (void *)0, false);
            }
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 468) - [public] if
    if (rhs.is_const) {
        if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 469) - [public] if
    if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 470) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 473) - [public] if
    if (rhs.t.bits > l.t.bits) {
        diag(s, n->line, "narrowing assignment", (void *)0, false);
    } else if (rhs.t.bits == l.t.bits &&
        rhs.t.neg != l.t.neg) {
        diag(s, n->line, "signed/unsigned mismatch",
            (void *)0, false);
    }
}
}
return l;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 474) - [public] diag
    diag(s, n->line, "narrowing assignment", (void *)0, false);
} else if (rhs.t.bits == l.t.bits &&
    rhs.t.neg != l.t.neg) {
    diag(s, n->line, "signed/unsigned mismatch",
        (void *)0, false);
}
}
}
return l;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 493) - [public] if
    if (cmp) {
        r.t.bits = 1;
        return r;
    }

    if (a.t.bits == 0 && b.t.bits != 0) {
        if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = b.t;
    } else if (b.t.bits == 0 && a.t.bits != 0) {
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 498) - [public] if
    if (a.t.bits == 0 && b.t.bits != 0) {
        if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = b.t;
    } else if (b.t.bits == 0 && a.t.bits != 0) {
        if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = a.t;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 499) - [public] if
    if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = b.t;
} else if (b.t.bits == 0 && a.t.bits != 0) {
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 500) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
}

    r.t = b.t;
} else if (b.t.bits == 0 && a.t.bits != 0) {
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 505) - [public] if
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    r.t.neg = a.t.neg || b.t.neg;
}

    return r;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 506) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
}

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    r.t.neg = a.t.neg || b.t.neg;
}

    return r;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 524) - [public] check_block
static void check_block(sema_ctx_t *s, ast_node_t *n);

static void check_stmt(sema_ctx_t *s, ast_node_t *n)
{
    s->sh_count = 0;

    switch (n->kind) {
    case ND_LET: {
        etype_t init = check_expr(s, n->a);

        if (init.is_const &&
            !value_fits(init.cval, init.cneg, n->type)) {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 530) - [public] switch
switch (n->kind) {
case ND_LET: {
    etype_t init = check_expr(s, n->a);

    if (init.is_const &&
        !value_fits(init.cval, init.cneg, n->type)) {
        diag(s, n->line, "literal out of type range", n->name, false);
    }

    if (init.t.bits != 0 && init.t.bits > n->type.bits) {
        diag(s, n->line, "narrowing assignment", n->name, false);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 536) - [public] diag
diag(s, n->line, "literal out of type range", n->name, false);
}

if (init.t.bits != 0 && init.t.bits > n->type.bits) {
    diag(s, n->line, "narrowing assignment", n->name, false);
}

symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

if (sym != (void *)0 && n->type.neg) {
    sym->type.neg = true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 539) - [public] if
if (init.t.bits != 0 && init.t.bits > n->type.bits) {
    diag(s, n->line, "narrowing assignment", n->name, false);
}

symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

if (sym != (void *)0 && n->type.neg) {
    sym->type.neg = true;
}

break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 540) - [public] diag
    diag(s, n->line, "narrowing assignment", n->name, false);
}

symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

if (sym != (void *)0 && n->type.neg) {
    sym->type.neg = true;
}

break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 545) - [public] if
    if (sym != (void *)0 && n->type.neg) {
        sym->type.neg = true;
    }

    break;
}

case ND_STATIC:
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_SET_FREE: {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 553) - [public] declare
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_SET_FREE: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "set_free on undeclared", n->name, false);
    } else if (sym->freed) {
        diag(s, n->line, "double set_free", n->name, false);
    } else {
        sym->live = false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 559) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "set_free on undeclared", n->name, false);
    } else if (sym->freed) {
        diag(s, n->line, "double set_free", n->name, false);
    } else {
        sym->live = false;
        sym->freed = true;
    }

    break;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 560) - [public] diag
    diag(s, n->line, "set_free on undeclared", n->name, false);
} else if (sym->freed) {
    diag(s, n->line, "double set_free", n->name, false);
} else {
    sym->live = false;
    sym->freed = true;
}

break;
}

case ND_IF:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 562) - [public] diag
    diag(s, n->line, "double set_free", n->name, false);
} else {
    sym->live = false;
    sym->freed = true;
}

break;
}

case ND_IF:
    if (n->a != (void *)0) {
        check_expr(s, n->a);
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 572) - [public] if
    if (n->a != (void *)0) {
        check_expr(s, n->a);
    }

    check_block(s, n->b);

    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 573) - [public] check_expr
    check_expr(s, n->a);
}

check_block(s, n->b);

if (n->c != (void *)0) {
    if (n->c->kind == ND_BLOCK) {
        check_block(s, n->c);
    } else {
        check_stmt(s, n->c);
    }
}
// ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 576) - [public] check_block
    check_block(s, n->b);

    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }

    break;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 578) - [public] if
    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }

    break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 579) - [public] if
    if (n->c->kind == ND_BLOCK) {
        check_block(s, n->c);
    } else {
        check_stmt(s, n->c);
    }
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 580) - [public] check_block
    check_block(s, n->c);
} else {
    check_stmt(s, n->c);
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    if (tgt == (void *)0) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 582) - [public] check_stmt
    check_stmt(s, n->c);
    }
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    if (tgt == (void *)0) {
        diag(s, n->line, "when on undeclared", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 591) - [public] if
    if (tgt == (void *)0) {
        diag(s, n->line, "when on undeclared", n->name, false);
    }

    for (size_t i = 0; i < n->nlist; i++) {
        ast_node_t *c = n->list[i];

        if (c->a != (void *)0 && c->a->kind == ND_BIN &&
            c->a->a != (void *)0 && c->a->a->kind == ND_MEMBER &&
            c->a->a->a != (void *)0 &&
            c->a->a->a->kind == ND_VAR) {
            const char *cn = c->a->a->a->name;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 592) - [public] diag
        diag(s, n->line, "when on undeclared", n->name, false);
    }

    for (size_t i = 0; i < n->nlist; i++) {
        ast_node_t *c = n->list[i];

        if (c->a != (void *)0 && c->a->kind == ND_BIN &&
            c->a->a != (void *)0 && c->a->a->kind == ND_MEMBER &&
            c->a->a->a != (void *)0 &&
            c->a->a->a->kind == ND_VAR) {
            const char *cn = c->a->a->a->name;
            bool same = true;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 606) - [public] if
        if (cn[k] != n->name[k]) {
            same = false;
            break;
        }
    }

    if (!same) {
        diag(s, c->line, "case target mismatch", cn, false);
    }

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 612) - [public] if
    if (!same) {
        diag(s, c->line, "case target mismatch", cn, false);
    }

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
        !value_fits(c->a->b->value, false, tgt->type)) {
        diag(s, c->line, "case value out of range",
            n->name, false);
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 613) - [public] diag
    diag(s, c->line, "case target mismatch", cn, false);
}

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
        !value_fits(c->a->b->value, false, tgt->type)) {
        diag(s, c->line, "case value out of range",
            n->name, false);
    }
}

    check_block(s, c->b);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 624) - [public] check_block
    check_block(s, c->b);
}

    break;
}

case ND_BLOCK:
    check_block(s, n);
    break;

default:
    check_expr(s, n);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 631) - [public] check_block
    check_block(s, n);
    break;

default:
    check_expr(s, n);
    shared_validate(s);
    break;
}

if (n->kind != ND_BLOCK) {
    shared_validate(s);
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 635) - [public] check_expr

```
        check_expr(s, n);
        shared_validate(s);
        break;
    }

    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 636) - [public] shared_validate

```
        shared_validate(s);
        break;
    }

    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 640) - [public] if

```
    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 641) - [public] shared_validate

```
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 647) - [public] push_scope
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }

    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 650) - [public] check_stmt
    check_stmt(s, n->list[i]);
}

    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 653) - [public] pop_scope
    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        symbol_t *sym = declare(s, n->name, SYM_FN, t, n->line);

        if (sym == (void *)0) {
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/core-lang/sema.c (line 659) - [public] switch
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        symbol_t *sym = declare(s, n->name, SYM_FN, t, n->line);

        if (sym == (void *)0) {
            return;
        }

        for (size_t i = 0; i < n->nlist && i < 8; i++) {
            sym->params[sym->nparams++] = n->list[i]->type;
        }
        // ... (truncated, see source file for more)
    }
}

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 664) - [public] if
    if (sym == (void *)0) {
        return;
    }

    for (size_t i = 0; i < n->nlist && i < 8; i++) {
        sym->params[sym->nparams++] = n->list[i]->type;
    }

    sym->node = n;
    break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 677) - [public] declare
    declare(s, n->name, SYM_VAR, n->type, n->line);
    break;

case ND_STATIC:
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_ENUM: {
    cl_type_t t = { 64, false };
    symbol_t *sym = declare(s, n->name, SYM_ENUM, t, n->line);

    if (sym != (void *)0) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 681) - [public] declare
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_ENUM: {
    cl_type_t t = { 64, false };
    symbol_t *sym = declare(s, n->name, SYM_ENUM, t, n->line);

    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 688) - [public] if
    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
}

case ND_EXTERN:
    for (size_t i = 0; i < n->nlist; i++) {
        cl_type_t t = { 64, false };
        declare(s, n->list[i]->name, SYM_EXTERN, t, n->line);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 698) - [public] declare
    declare(s, n->list[i]->name, SYM_EXTERN, t, n->line);
}

    break;

case ND_DATAIMPL: {
    cl_type_t t = { 0, false };
    symbol_t *sym = declare(s, "data", SYM_DATA, t, n->line);

    if (sym != (void *)0) {
        sym->node = n;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 707) - [public] if
    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
}

default:
    break;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 721) - [public] if
    if (prog == (void *)0) {
        return -1;
    }

    for (size_t i = 0; i < prog->nlist; i++) {
        declare_top(s, prog->list[i]);
    }

    for (size_t i = 0; i < prog->nlist; i++) {
        ast_node_t *n = prog->list[i];

        if (n->kind != ND_FN) {
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 726) - [public] declare_top
    declare_top(s, prog->list[i]);
}

    for (size_t i = 0; i < prog->nlist; i++) {
        ast_node_t *n = prog->list[i];

        if (n->kind != ND_FN) {
            continue;
        }

        push_scope(s);

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 732) - [public] if
    if (n->kind != ND_FN) {
        continue;
    }

    push_scope(s);

    for (size_t k = 0; k < n->nlist; k++) {
        declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
    }

    check_block(s, n->a);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 736) - [public] push_scope
    push_scope(s);

    for (size_t k = 0; k < n->nlist; k++) {
        declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
    }

    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 739) - [public] declare
    declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
}

    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 742) - [public] check_block
    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
{
    return s->ndiags;
}

    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 744) - [public] pop_scope
    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
{
    return s->ndiags;
}

const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 19) - [public] for
    for (int i = full + (rem ? 1 : 0); i < w; i++) {
        r[i] = 0;
    }

    if (rem) {
        r[full] &= (uint8_t)((1 << rem) - 1);
    }
}

void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w)
{
    int c = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 23) - [public] if
    if (rem) {
        r[full] &= (uint8_t)((1 << rem) - 1);
    }
}

void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w)
{
    int c = 0;

    for (int i = 0; i < w; i++) {
        int s = a[i] + b[i] + c;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 85) - [public] if
    if (a[i] != b[i]) {
        return a[i] > b[i] ? 1 : -1;
    }
}

    return 0;
}

int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign)
{
    if (!sign) {
        return cmp_u(a, b, w);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 95) - [public] if

```
    if (!sign) {
        return cmp_u(a, b, w);
    }

    uint8_t x[64], y[64];

    for (int i = 0; i < w; i++) {
        x[i] = a[i];
        y[i] = b[i];
    }

    int sb = w * 8 - 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 96) - [public] cmp_u

```
    return cmp_u(a, b, w);
}

uint8_t x[64], y[64];

for (int i = 0; i < w; i++) {
    x[i] = a[i];
    y[i] = b[i];
}

int sb = w * 8 - 1;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 113) - [public] cmp_u

```
    return cmp_u(x, y, w);
}

void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    uint8_t t[64] = { 0 };

    for (int i = 0; i < w * 8; i++) {
        if (bit_get(a, i) && (int)(i + n) < w * 8) {
            bit_set(t, i + (int)n);
        }
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 121) - [public] if

```
    if (bit_get(a, i) && (int)(i + n) < w * 8) {
        bit_set(t, i + (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}

void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 122) - [public] bit_set
    bit_set(t, i + (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}
}

void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    uint8_t t[64] = { 0 };
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 136) - [public] if
    if (bit_get(a, i) && i >= (int)n) {
        bit_set(t, i - (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}
}

void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 137) - [public] bit_set
    bit_set(t, i - (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}
}

void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 154) - [public] cl_shl
    cl_shl(shl, rem, 1, w);

    for (int k = 0; k < w; k++) {
        rem[k] = shl[k];
    }

    if (bit_get(n, i)) {
        rem[0] |= 1;
    }

    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 160) - [public] if
    if (bit_get(n, i)) {
        rem[0] |= 1;
    }

    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        cl_sub(sub, rem, d, w);

        for (int k = 0; k < w; k++) {
            rem[k] = sub[k];
        }

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 164) - [public] if
    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        cl_sub(sub, rem, d, w);

        for (int k = 0; k < w; k++) {
            rem[k] = sub[k];
        }

        bit_set(quo, i);
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 166) - [public] cl_sub
    cl_sub(sub, rem, d, w);

    for (int k = 0; k < w; k++) {
        rem[k] = sub[k];
    }

    bit_set(quo, i);
}

for (int i = 0; i < w; i++) {
    q[i] = quo[i];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 172) - [public] bit_set
    bit_set(quo, i);
}

for (int i = 0; i < w; i++) {
    q[i] = quo[i];
    m[i] = rem[i];
}

/* ---- VM ---- */

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 207) - [public] if
    if (vm->prog->ext_names[i][k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) {
        vm->externs[i] = fn;
        return 0;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 213) - [public] if
    if (same) {
        vm->externs[i] = fn;
        return 0;
    }
}

    return -1;
}

static void push(cl_vm_t *vm, const cl_val_t *v)
{
    if (vm->sp < 256) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/vm.c (line 224) - [public] if
    if (vm->sp < 256) {
        vm->stack[vm->sp++] = *v;
    } else {
        vm->err = CL_ERR_STACK;
    }
}

static cl_val_t pop(cl_vm_t *vm)
{
    if (vm->sp > 0) {
        return vm->stack[--vm->sp];
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/vm.c (line 233) - [public] if
    if (vm->sp > 0) {
        return vm->stack[--vm->sp];
    }

    vm->err = CL_ERR_STACK;

    cl_val_t z = { { 0 } };
    return z;
}

static uint64_t val_u64(const cl_val_t *v)
{
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 260) - [public] if

```
    if (P->main_fn < 0) {
        return CL_ERR_BADOP;
    }

    vm->frames[0].pc = P->fns[P->main_fn].start;
    vm->frames[0].fn_idx = (uint16_t)P->main_fn;

    for (int i = 0; i < 64; i++) {
        vm->frames[0].slive[i] = 0;
    }

    vm->depth = 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 273) - [public] while

```
    while (vm->depth > 0 && vm->err == CL_OK && !vm->halt_req) {
        vm_frame_t *f = &vm->frames[vm->depth - 1];

        if (f->pc >= P->code_len) {
            break;
        }

        cl_insn_t in = P->code[f->pc++];

        int w = cl_wbytes(in.bits);
        bool sign = false;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 276) - [public] if

```
        if (f->pc >= P->code_len) {
            break;
        }

        cl_insn_t in = P->code[f->pc++];

        int w = cl_wbytes(in.bits);
        bool sign = false;

        cl_val_t a, b, r;

        switch (in.op) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 287) - [public] switch

```
        switch (in.op) {
            case OP_NOP: break;

            case OP_CONST:
                r = P->consts[in.a];
                cl_mask(r.b, w, in.bits);
                push(vm, &r);
                break;

            case OP_LOAD:
                if (!f->slive[in.a]) {
                    vm->err = CL_ERR_USE_AFTER_FREE;
                }
                // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 292) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_LOAD:
        if (!f->slive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
            break;
        }
        push(vm, &f->slots[in.a]);
        break;
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 293) - [public] push

```
    push(vm, &r);
    break;

    case OP_LOAD:
        if (!f->slive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
            break;
        }
        push(vm, &f->slots[in.a]);
        break;
```

case OP_STORE:

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 297) - [public] if

```
    if (!f->slive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &f->slots[in.a]);
    break;
```

case OP_STORE:

```
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 301) - [public] push

```
    push(vm, &f->slots[in.a]);
    break;
```

case OP_STORE:

```
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
    break;
```

case OP_GLOAD:

```
    if (!vm->glive[in.a]) {
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 306) - [public] cl_mask

```
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
    break;
```

```
case OP_GLOAD:
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &vm->globals[in.a]);
    break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 312) - [public] if

```
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &vm->globals[in.a]);
    break;
```

```
case OP_GSTORE:
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 316) - [public] push

```
    push(vm, &vm->globals[in.a]);
    break;
```

```
case OP_GSTORE:
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
    break;
```

```
case OP_ADD: case OP_SUB: case OP_MUL:
case OP_DIV: case OP_MOD:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 321) - [public] cl_mask

```
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
    break;
```

```
case OP_ADD: case OP_SUB: case OP_MUL:
case OP_DIV: case OP_MOD:
case OP_AND: case OP_OR: case OP_XOR:
case OP_SHL: case OP_SHR:
    b = pop(vm);
    a = pop(vm);
```

```
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/vm.c (line 335) - [public] switch

```
switch (in.op) {
case OP_ADD: cl_add(r.b, a.b, b.b, w); break;
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 336) - [public] cl_add

```
case OP_ADD: cl_add(r.b, a.b, b.b, w); break;
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 337) - [public] cl_sub

```
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

    for (int i = 0; i < w; i++) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 338) - [public] cl_mul

```
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

    for (int i = 0; i < w; i++) {
        if (b.b[i]) zero = false;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 342) - [public] cl_shl
    case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_DIV: case OP_MOD: {
        uint8_t q[64], m[64];
        bool zero = true;

        for (int i = 0; i < w; i++) {
            if (b.b[i]) zero = false;
        }

        if (zero) {
            vm->err = CL_ERR_DIV_ZERO;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 343) - [public] cl_shr
    case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_DIV: case OP_MOD: {
        uint8_t q[64], m[64];
        bool zero = true;

        for (int i = 0; i < w; i++) {
            if (b.b[i]) zero = false;
        }

        if (zero) {
            vm->err = CL_ERR_DIV_ZERO;
            break;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 352) - [public] if
    if (zero) {
        vm->err = CL_ERR_DIV_ZERO;
        break;
    }

    cl_divmod(a.b, b.b, w, q, m);

    for (int i = 0; i < w; i++) {
        r.b[i] = (in.op == OP_DIV) ? q[i] : m[i];
    }
    break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 357) - [public] cl_divmod
    cl_divmod(a.b, b.b, w, q, m);

    for (int i = 0; i < w; i++) {
        r.b[i] = (in.op == OP_DIV) ? q[i] : m[i];
    }
    break;
}
}

    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 366) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_NEG:
        a = pop(vm);
        {
            uint8_t z[64] = { 0 };
            cl_sub(r.b, z, a.b, w);
            cl_mask(r.b, w, in.bits);
        }
        push(vm, &r);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 367) - [public] push

```
    push(vm, &r);
    break;

    case OP_NEG:
        a = pop(vm);
        {
            uint8_t z[64] = { 0 };
            cl_sub(r.b, z, a.b, w);
            cl_mask(r.b, w, in.bits);
        }
        push(vm, &r);
        break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 374) - [public] cl_sub

```
    cl_sub(r.b, z, a.b, w);
    cl_mask(r.b, w, in.bits);
}
push(vm, &r);
break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 375) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
}
push(vm, &r);
break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 377) - [public] push

```
    push(vm, &r);
    break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 383) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
        b = pop(vm);
        a = pop(vm);
        {
            int c = cl_cmp(a.b, b.b, w, sign);
            bool t = false;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 384) - [public] push

```
    push(vm, &r);
    break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
        b = pop(vm);
        a = pop(vm);
        {
            int c = cl_cmp(a.b, b.b, w, sign);
            bool t = false;

            switch (in.op) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 395) - [public] switch

```
        switch (in.op) {
            case OP_EQ: t = (c == 0); break;
            case OP_NE: t = (c != 0); break;
            case OP_LT: t = (c < 0); break;
            case OP_GT: t = (c > 0); break;
            case OP_LE: t = (c <= 0); break;
            case OP_GE: t = (c >= 0); break;
        }

        for (int i = 0; i < 64; i++) r.b[i] = 0;
        r.b[0] = t ? 1 : 0;
    }
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 407) - [public] push
    push(vm, &r);
    break;

    case OP_JMP:
        f->pc = in.c;
        break;

    case OP_JZ:
        a = pop(vm);
        if (a.b[0] == 0) f->pc = in.c;
        break;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 425) - [public] pop
    pop(vm);
    break;

    case OP_FREE:
        if (in.a == 1) {
            vm->glive[in.b] = 0;
            for (int i = 0; i < 64; i++) vm->globals[in.b].b[i] = 0;
        } else {
            f->slive[in.b] = 0;
            for (int i = 0; i < 64; i++) f->slots[in.b].b[i] = 0;
        }
        break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 429) - [public] if
    if (in.a == 1) {
        vm->glive[in.b] = 0;
        for (int i = 0; i < 64; i++) vm->globals[in.b].b[i] = 0;
    } else {
        f->slive[in.b] = 0;
        for (int i = 0; i < 64; i++) f->slots[in.b].b[i] = 0;
    }
    break;

    case OP_SHARED:
        if (!vm->glive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
        }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 439) - [public] if
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
    }
    break;

    case OP_EXTERN: {
        if (in.a >= 16 || vm->externs[in.a] == (cl_ext_fn)0) {
            vm->err = CL_ERR_EXTERN;
            break;
        }

        uint64_t args[6] = { 0 };
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 445) - [public] if
    if (in.a >= 16 || vm->externs[in.a] == (cl_ext_fn)0) {
        vm->err = CL_ERR_EXTERN;
        break;
    }

    uint64_t args[6] = { 0 };
    int n = in.b > 6 ? 6 : in.b;

    for (int i = n - 1; i >= 0; i--) {
        cl_val_t v = pop(vm);
        args[i] = val_u64(&v);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 464) - [public] push
    push(vm, &r);
    break;
}

case OP_CALL: {
    if (vm->depth >= 8) {
        vm->err = CL_ERR_DEPTH;
        break;
    }

    cl_val_t argv[8];
    int n = in.b > 8 ? 8 : in.b;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 469) - [public] if
    if (vm->depth >= 8) {
        vm->err = CL_ERR_DEPTH;
        break;
    }

    cl_val_t argv[8];
    int n = in.b > 8 ? 8 : in.b;

    for (int i = n - 1; i >= 0; i--) {
        argv[i] = pop(vm);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 50) - [public] if
    if (addr + len > 0x400) {
        return false;
    }

    uint8_t *out = (uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        out[i] = dpd[addr + i];
    }

    return true;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/aux.c (line 65) - [public] if
    if (addr + len > 0x400) {
        return false;
    }

    const uint8_t *in = (const uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        dpcd[addr + i] = in[i];
    }

    if (addr <= DPCD_TRAINING_SET &&
        DPCD_TRAINING_SET < addr + len) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 79) - [public] if
    if (pat == 1) {
        dpcd[DPCD_LANE_STATUS] = 0x11;
        dpcd[DPCD_LANE_STATUS + 1] = 0x11;
    } else if (pat == 2) {
        train_step++;

        if (train_step >= 2) {
            dpcd[DPCD_LANE_STATUS] = 0x77;
            dpcd[DPCD_LANE_STATUS + 1] = 0x77;
            dpcd[DPCD_SYMBOL_LOCK] = 0x01;
        }
    } else {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 85) - [public] if
    if (train_step >= 2) {
        dpcd[DPCD_LANE_STATUS] = 0x77;
        dpcd[DPCD_LANE_STATUS + 1] = 0x77;
        dpcd[DPCD_SYMBOL_LOCK] = 0x01;
    }
    } else {
        train_step = 0;
    }
}

return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 100) - [public] if
    if (len > 128) {
        len = 128;
    }

    uint8_t *out = (uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        out[i] = EDID[i];
    }

    return true;
}

```

kernel_Workspace/kernel/src/displayport/edid.c (line 8) - [public] if

```
*count = 0;

if (b[0] != 0x00 || b[1] != 0xFF || b[7] != 0x00) {
    return false;
}

for (uint32_t d = 0; d < 4; d++) {
    const uint8_t *p = &b[54 + d * 18];

    uint32_t clock = (uint32_t)p[0] | ((uint32_t)p[1] << 8);

    if (clock == 0) {
        continue;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 17) - [public] if

```
if (clock == 0) {
    continue;
}

uint32_t hact = p[2] | (((uint32_t)p[4] >> 4) << 8);
uint32_t hblk = p[3] | (((uint32_t)p[4] & 0xF) << 8);
uint32_t vact = p[5] | (((uint32_t)p[7] >> 4) << 8);
uint32_t vblk = p[6] | (((uint32_t)p[7] & 0xF) << 8);

uint32_t hsync = p[9] | (((uint32_t)p[10] >> 4) & 0xF) << 8);
uint32_t vsync = (uint32_t)p[10] & 0xF;
uint32_t hoff = p[8] | (((uint32_t)p[11] >> 6) << 8);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 32) - [public] if

```
if (hact == 0 || vact == 0) {
    continue;
}

if (*count >= max) {
    break;
}

uint32_t htot = hact + hblk;
uint32_t vtot = vact + vblk;

uint32_t refresh = (clock * 1000u) / (htot * vtot);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 36) - [public] if

```
if (*count >= max) {
    break;
}

uint32_t htot = hact + hblk;
uint32_t vtot = vact + vblk;

uint32_t refresh = (clock * 1000u) / (htot * vtot);

hdmi_mode_t *m = &out[*count];

m->id = 100 + *count;
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/displayport/init.c (line 13) - [public] if

```
    if (dp_is_ready) {
        return true;
    }

    dp_aux_sink_init();

    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 17) - [public] dp_aux_sink_init

```
dp_aux_sink_init();

    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 19) - [public] if

```
    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 25) - [public] if

```
    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }

    dp_is_ready = true;

    return true;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/displayport/init.c (line 29) - [public] if
    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }

    dp_is_ready = true;

    return true;
}

bool dp_ready(void)
{
    return dp_is_ready;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 51) - [public] if
    if (i >= MODE_COUNT) {
        return false;
    }

    *id = MODES[i].id;
    *w = MODES[i].w;
    *h = MODES[i].h;
    *r = MODES[i].refresh;

    return true;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 66) - [public] if
    if (MODES[i].id == id) {
        dp_op_set_fb(0, MODES[i].w, MODES[i].h, MODES[i].w);
        return true;
    }

    return false;
}

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride)
{
    dp_op_state(fb_phys, w, h, stride);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 67) - [public] dp_op_set_fb
    dp_op_set_fb(0, MODES[i].w, MODES[i].h, MODES[i].w);
    return true;
}

    return false;
}

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride)
{
    dp_op_state(fb_phys, w, h, stride);
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 77) - [public] dp_op_state
    dp_op_state(fb_phys, w, h, stride);
}

bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    dp_op_set_fb(phys, w, h, stride);
    return true;
}

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 82) - [public] dp_op_set_fb
    dp_op_set_fb(phys, w, h, stride);
    return true;
}

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h)
{
    static uint64_t seq = 0;

    dp_op_fill(color, x, y, w, h);

    return ++seq;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 91) - [public] dp_op_fill
    dp_op_fill(color, x, y, w, h);

    return ++seq;
}

```

```

kernel_Workspace/kernel/src/displayport/link.c (line 12) - [public] dp_aux_read
    dp_aux_read(DPCD_MAX_LINK_RATE, &max_rate, 1);
    dp_aux_read(DPCD_MAX_LANE_COUNT, &max_lanes, 1);

    uint8_t bw = max_rate < 0x14 ? max_rate : 0x14;
    uint8_t lanes = max_lanes & 0x1F;

    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/link.c (line 13) - [public] dp_aux_read
    dp_aux_read(DPCD_MAX_LANE_COUNT, &max_lanes, 1);

    uint8_t bw = max_rate < 0x14 ? max_rate : 0x14;
    uint8_t lanes = max_lanes & 0x1F;

    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/displayport/link.c (line 18) - [public] if

```
    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/link.c (line 22) - [public] dp_aux_write

```
    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            // ... (truncated, see source file for more)
        }
    }
```

kernel_Workspace/kernel/src/displayport/link.c (line 23) - [public] dp_aux_write

```
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            break;
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/displayport/link.c (line 26) - [public] dp_aux_write

```
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            break;
        }
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/displayport/link.c (line 31) - [public] dp_aux_read
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
        break;
    }
}

tp = 2;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/displayport/link.c (line 33) - [public] if
    if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
        break;
    }
}

tp = 2;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/displayport/link.c (line 39) - [public] dp_aux_write
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 42) - [public] dp_aux_read
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/displayport/link.c (line 44) - [public] if
    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 50) - [public] dp_aux_read
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
}

tp = 0;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 52) - [public] if
if (lock != 1) {
    return false;
}

tp = 0;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
default:   cur_rate = 1620; break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 57) - [public] dp_aux_write
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
default:   cur_rate = 1620; break;
}

cur_lanes = lanes;

return true;
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/displayport/link.c (line 59) - [public] switch

```
    switch (bw) {
        case 0x0A: cur_rate = 2700; break;
        case 0x14: cur_rate = 5400; break;
        default:   cur_rate = 1620; break;
    }

    cur_lanes = lanes;

    return true;
}

void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/operation.c (line 27) - [public] if

```
    if (fb_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)arch_phys_to_virt(fb_phys);

    for (uint32_t yy = y; yy < y + fh && yy < h; yy++) {
        for (uint32_t xx = x; xx < x + fw && xx < w; xx++) {
            dst[yy * stride + xx] = color;
        }
    }
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 11) - [public] console_set_enabled

```
extern void console_set_enabled(int on);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_input_keycode(void);
```

```
#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 12) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_input_keycode(void);
```

```
#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2
```

```
#define COL_BG      0xFF1E1E1E
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 13) - [public] k_input_keycode
extern uint32_t k_input_keycode(void);

#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2

#define COL_BG 0xFF1E1E1E
#define COL_FG 0xFFD4D4D4
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 88) - [public] if
    if (bits & (0x80 >> rx)) {
        uint32_t px = x + (uint32_t)rx * SCALE;
        uint32_t py = y + (uint32_t)ry * SCALE;

        ed_fill(px, py, SCALE, SCALE, color);
    }
}

static void ed_text(uint32_t x, uint32_t y, const char *s, uint32_t color)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 92) - [public] ed_fill
    ed_fill(px, py, SCALE, SCALE, color);
}
}

static void ed_text(uint32_t x, uint32_t y, const char *s, uint32_t color)
{
    while (*s) {
        ed_char(x, y, *s, color);
        x += CHAR_W;
        s++;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 100) - [public] while
    while (*s) {
        ed_char(x, y, *s, color);
        x += CHAR_W;
        s++;
    }
}

static void ed_num(char *buf, int v)
{
    int i = 0;

    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }
    // ... (truncated, see source file for more)

```


kernel_workspace/kernel/src/editor/editor.c (line 101) - [public] ed_char

```
    ed_char(x, y, *s, color);
    x += CHAR_W;
    s++;
}
}
```

```
static void ed_num(char *buf, int v)
{
    int i = 0;

    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/editor/editor.c (line 111) - [public] if

```
    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }

    char tmp[12];
    int n = 0;

    while (v > 0 && n < 11) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;
}
```

```
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/editor/editor.c (line 116) - [public] while

```
    while (v > 0 && n < 11) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;
}
```

```
static void log_append(const char *s)
{
    for (int i = 0; i < 7; i++) {
        for (int k = 0; k < ED_LINE_LEN; k++) {
            log_lines[i][k] = log_lines[i + 1][k];
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/editor/editor.c (line 147) - [public] if

```
    if (v == 0) { tmp[n++] = '0'; }

    while (v > 0 && n < 20) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;

    log_append(buf);

    return 0;
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 149) - [public] while
    while (v > 0 && n < 20) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;

    log_append(buf);

    return 0;
}

static uint64_t ed_putc(uint64_t v, uint64_t b, uint64_t c,
                        uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 154) - [public] log_append
    log_append(buf);

    return 0;
}

static uint64_t ed_putc(uint64_t v, uint64_t b, uint64_t c,
                        uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    char buf[2] = { (char)v, 0 };
    log_append(buf);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 165) - [public] log_append
    log_append(buf);
    return 0;
}

static uint32_t tok_color(tok_kind_t k)
{
    switch (k) {
        case TK_IF: case TK_ELSE: case TK_WHEN: case TK_CASE:
        case TK_FN: case TK_LET: case TK_STATIC: case TK_DATA:
        case TK_DATAIMPL: case TK_ENUM: case TK_EXTERN:
        case TK_CALL: case TK_SET_FREE:
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 172) - [public] switch
    switch (k) {
        case TK_IF: case TK_ELSE: case TK_WHEN: case TK_CASE:
        case TK_FN: case TK_LET: case TK_STATIC: case TK_DATA:
        case TK_DATAIMPL: case TK_ENUM: case TK_EXTERN:
        case TK_CALL: case TK_SET_FREE:
            return COL_KEY;
        case TK_TYPE: return COL_TYPE;
        case TK_NUM: return COL_NUM;
        case TK_STR:
        case TK_CHAR: return COL_STR;
        case TK_IDENT: return COL_ID;
        default: return COL_OP;
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 201) - [public] if
    if (tmp[k] == '/' && tmp[k + 1] == '/') { com_at = k; break; }
}

lexer_t l;
token_t toks[64];
size_t n = 0;

cl_lexer_init(&l, tmp, ed_strlen(tmp));

if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 208) - [public] cl_lexer_init
cl_lexer_init(&l, tmp, ed_strlen(tmp));

if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    for (int k = 0; k < com_at; k++) cut[k] = tmp[k];
    cut[com_at] = 0;

    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 210) - [public] if
if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    for (int k = 0; k < com_at; k++) cut[k] = tmp[k];
    cut[com_at] = 0;

    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;

if (cl_lex_all(&l, toks, 64, &n) == 0) {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 216) - [public] cl_lexer_init
    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;

if (cl_lex_all(&l, toks, 64, &n) == 0) {
    for (size_t t = 0; t < n && toks[t].kind != TK_EOF; t++) {
        int ts = (int)(toks[t].start - l.src);

        while (pos < ts) {
            ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
            pos++;
        }
        // ... (truncated, see source file for more)
    }
}

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 221) - [public] if
    if (cl_lex_all(&l, toks, 64, &n) == 0) {
        for (size_t t = 0; t < n && toks[t].kind != TK_EOF; t++) {
            int ts = (int)(toks[t].start - l.src);

            while (pos < ts) {
                ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
                pos++;
            }

            uint32_t col = tok_color(toks[t].kind);

            for (uint32_t k = 0; k < toks[t].len; k++) {
                // ... (truncated, see source file for more)
            }
        }
    }

kernel_Workspace/kernel/src/editor/editor.c (line 225) - [public] while
    while (pos < ts) {
        ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
        pos++;
    }

    uint32_t col = tok_color(toks[t].kind);

    for (uint32_t k = 0; k < toks[t].len; k++) {
        ed_char(x + (uint32_t)pos * CHAR_W, y,
            toks[t].start[k], col);
        pos++;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 226) - [public] ed_char
    ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
    pos++;
}

    uint32_t col = tok_color(toks[t].kind);

    for (uint32_t k = 0; k < toks[t].len; k++) {
        ed_char(x + (uint32_t)pos * CHAR_W, y,
            toks[t].start[k], col);
        pos++;
    }
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 240) - [public] while
    while (tmp[pos] && (com_at < 0 || pos < com_at)) {
        ed_char(x + (uint32_t)pos * CHAR_W, y, tmp[pos], COL_FG);
        pos++;
    }

    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 241) - [public] ed_char
    ed_char(x + (uint32_t)pos * CHAR_W, y, tmp[pos], COL_FG);
    pos++;
}

    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 245) - [public] if
    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

static const char *CURSOR_SPRITE[] = {
    "X.....",
    "XX.....",
    "XoX.....",
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 247) - [public] ed_char
    ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
}
}

static const char *CURSOR_SPRITE[] = {
    "X.....",
    "XX.....",
    "XoX.....",
    "XooX.....",
    "XoooX.....",
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 285) - [public] if
    if (sx >= 0 && sx < (int)fw && sy >= 0 && sy < (int)fh) {
        int row = ed_flip_rows ? (int)fh - 1 - sy : sy;
        fb[row * fstride + sx] = col;
    }
}
}

static void render(void)
{
    ed_fill(0, 0, fw, fh, COL_BG);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 296) - [public] ed_fill
    ed_fill(0, 0, fw, fh, COL_BG);

    ed_fill(0, 0, fw, TOP_BAR, COL_BAR);
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 298) - [public] ed_fill
    ed_fill(0, 0, fw, TOP_BAR, COL_BAR);
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 299) - [public] ed_text
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        uint32_t y = TOP_BAR + (uint32_t)r * CHAR_H;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 300) - [public] ed_text
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        uint32_t y = TOP_BAR + (uint32_t)r * CHAR_H;

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 312) - [public] if
    if (row == caret_l) {
        ed_fill(GUTTER, y, (uint32_t)cols * CHAR_W, CHAR_H, 0xFF2D2D2D);
    }

    char nb[12];
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 313) - [public] ed_fill
    ed_fill(GUTTER, y, (uint32_t)cols * CHAR_W, CHAR_H, 0xFF2D2D2D);
}

    char nb[12];
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 317) - [public] ed_num
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 318) - [public] ed_text
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 320) - [public] draw_hl

```
    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

        ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 326) - [public] if

```
    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

        ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
    }
}

if (show_log) {
    uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 330) - [public] ed_fill

```
    ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
}

}

if (show_log) {
    uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

    for (int i = 0; i < 8; i++) {
        ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 334) - [public] if

```
    if (show_log) {
        uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

        ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

        for (int i = 0; i < 8; i++) {
            ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
        }
    }

    ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

    // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/editor/editor.c (line 337) - [public] ed_fill
    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

    for (int i = 0; i < 8; i++) {
        ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
    }
}

ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 340) - [public] ed_text
    ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
}
}

ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
while (*p1) st[i++] = *p1++;
char tmp[12];
ed_num(tmp, caret_l + 1);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 344) - [public] ed_fill
    ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
while (*p1) st[i++] = *p1++;
char tmp[12];
ed_num(tmp, caret_l + 1);
for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
const char *p2 = " col ";
while (*p2) st[i++] = *p2++;
ed_num(tmp, caret_c + 1);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 351) - [public] ed_num
    ed_num(tmp, caret_l + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    const char *p2 = " col ";
    while (*p2) st[i++] = *p2++;
    ed_num(tmp, caret_c + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    st[i] = 0;

    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 355) - [public] ed_num

```
    ed_num(tmp, caret_c + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    st[i] = 0;

    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 359) - [public] ed_text

```
    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 360) - [public] ed_text

```
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    if (len >= ED_LINE_LEN - 1 || caret_c > len) return;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 362) - [public] draw_mouse

```
    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    if (len >= ED_LINE_LEN - 1 || caret_c > len) return;

    for (int i = len; i >= caret_c; i--) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 413) - [public] if

```
    if (caret_c > 0) {
        int len = (int)ed_strlen(ln);

        for (int i = caret_c - 1; i < len; i++) {
            ln[i] = ln[i + 1];
        }

        caret_c--;
    } else if (caret_l > 0) {
        int plen = (int)ed_strlen(lines[caret_l - 1]);
        int slen = (int)ed_strlen(ln);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 425) - [public] if

```
    if (plen + slen < ED_LINE_LEN - 1) {
        for (int k = 0; k <= slen; k++) {
            lines[caret_l - 1][plen + k] = ln[k];
        }

        for (int i = caret_l; i < nlines - 1; i++) {
            for (int k = 0; k < ED_LINE_LEN; k++) {
                lines[i][k] = lines[i + 1][k];
            }
        }

        nlines--;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 448) - [public] if

```
    if (caret_c < len) {
        for (int i = caret_c; i < len; i++) {
            ln[i] = ln[i + 1];
        }
    }
}

static void clamp_scroll(int rows)
{
    if (caret_l < top_line) top_line = caret_l;
    if (caret_l >= top_line + rows) top_line = caret_l - rows + 1;
    if (top_line < 0) top_line = 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 485) - [public] render

```
    render();

    ed_arena.pos = 0;

    int n = flatten();

    uint32_t el = 0;
    const char *em = (void *)0;

    cl_prog_t *P = cl_compile_source(srcbuf, (size_t)n, &ed_arena, &el, &em);

    if (P == (void *)0) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 496) - [public] if

```
if (P == (void *)0) {
    const char *pre = "ERR ln ";
    int i = 0;
    while (pre[i]) { msg[i] = pre[i]; i++; }
    char tmp[12];
    ed_num(tmp, (int)el);
    for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
    msg[i++] = ' ';
    if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
    msg[i] = 0;

    if (em) log_append(em);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 499) - [public] while

```
while (pre[i]) { msg[i] = pre[i]; i++; }
char tmp[12];
ed_num(tmp, (int)el);
for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
msg[i++] = ' ';
if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
msg[i] = 0;

if (em) log_append(em);
return;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 501) - [public] ed_num

```
ed_num(tmp, (int)el);
for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
msg[i++] = ' ';
if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
msg[i] = 0;

if (em) log_append(em);
return;
}

cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 507) - [public] if

```
if (em) log_append(em);
return;
}

cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 511) - [public] cl_vm_init

```
cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 512) - [public] cl_bridge_init

```
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 514) - [public] cl_vm_register_extern

```
cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
msg[i] = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 515) - [public] cl_vm_register_extern

```
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
msg[i] = 0;

show_log = 1;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 517) - [public] log_append
    log_append("== run ==");

    cl_vm_err_t e = cl_vm_run(&ed_vm);

    const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
    int i = 0;
    while (ok[i]) { msg[i] = ok[i]; i++; }
    msg[i] = 0;

    show_log = 1;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 523) - [public] while
    while (ok[i]) { msg[i] = ok[i]; i++; }
    msg[i] = 0;

    show_log = 1;
}

int editor_run(const char *path)
{
    uint32_t w = 0, h = 0, s = 0;
    uint64_t base = 0;
    int32_t fliprows = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 536) - [public] if
    if (gfx_fb_info_raw(&w, &h, &s, &base, &fliprows) != 0 || base == 0) {
        return -1;
    }

    fb = (uint32_t *) (uintptr_t) base;
    fw = w; fh = h; fstride = s;
    ed_flip_rows = fliprows;

    nlines = 1;
    lines[0][0] = 0;
    caret_l = caret_c = 0;
    top_line = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 549) - [public] if
    if (path != (void *) 0) {
        static char rbuf[ED_MAX_LINES * ED_LINE_LEN];

        int32_t n = k_fs_read(path, rbuf, sizeof(rbuf) - 1);

        if (n > 0) {
            rbuf[n] = 0;

            nlines = 0;
            int col = 0;

            for (int i = 0; i < n && nlines < ED_MAX_LINES; i++) {
                // ... (truncated, see source file for more)
            }
        }
    }
}

```

kernel_Workspace/kernel/src/editor/editor.c (line 554) - [public] if

```
    if (n > 0) {
        rbuf[n] = 0;

        nlines = 0;
        int col = 0;

        for (int i = 0; i < n && nlines < ED_MAX_LINES; i++) {
            char c = rbuf[i];

            if (c == '\n') {
                lines[nlines][col] = 0;
                nlines++;
            }
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 563) - [public] if

```
        if (c == '\n') {
            lines[nlines][col] = 0;
            nlines++;
            col = 0;
        } else if (col < ED_LINE_LEN - 1) {
            lines[nlines][col++] = c;
        }
    }

    if (col > 0 || nlines == 0) {
        lines[nlines][col] = 0;
        nlines++;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 572) - [public] if

```
        if (col > 0 || nlines == 0) {
            lines[nlines][col] = 0;
            nlines++;
        }
    }
}
```

```
console_set_enabled(0);
mouse_init();
```

```
int running = 1;
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 579) - [public] console_set_enabled

```
console_set_enabled(0);
mouse_init();
```

```
int running = 1;
```

```
while (running) {
    int dirty = 0;
    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
```

```
    int dx = 0, dy = 0, dz = 0, btn = 0;
```

```
    if (mouse_poll(&dx, &dy, &dz, &btn)) {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 580) - [public] mouse_init
    mouse_init();

    int running = 1;

    while (running) {
        int dirty = 0;
        int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);

        int dx = 0, dy = 0, dz = 0, btn = 0;

        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/editor/editor.c (line 584) - [public] while
    while (running) {
        int dirty = 0;
        int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);

        int dx = 0, dy = 0, dz = 0, btn = 0;

        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            my += dy;

            if (mx < 0) mx = 0;
            if (my < 0) my = 0;
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/editor/editor.c (line 590) - [public] if
        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            my += dy;

            if (mx < 0) mx = 0;
            if (my < 0) my = 0;
            if (mx >= (int)fw) mx = (int)fw - 1;
            if (my >= (int)fh) my = (int)fh - 1;

            if (dz != 0) {
                top_line += dz;
                clamp_scroll(rows);
                // ... (truncated, see source file for more)
            }

kernel_Workspace/kernel/src/editor/editor.c (line 599) - [public] if
            if (dz != 0) {
                top_line += dz;
                clamp_scroll(rows);
                dirty = 1;
            }

            if (btn & 1 && !(mbuttons & 1)) {
                if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
                    int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
                    int col = (mx - (int)GUTTER - 4) / CHAR_W;

                    if (row >= 0 && row < nlines) caret_l = row;
                    // ... (truncated, see source file for more)
                }
            }
        }
    }
}

```



```

kernel_Workspace/kernel/src/editor/editor.c (line 601) - [public] clamp_scroll
    clamp_scroll(rows);
    dirty = 1;
}

    if (btn & 1 && !(mbuttons & 1)) {
        if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
            int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
            int col = (mx - (int)GUTTER - 4) / CHAR_W;

            if (row >= 0 && row < nlines) caret_l = row;

            int len = (int)ed_strlen(lines[caret_l]);
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 605) - [public] if
    if (btn & 1 && !(mbuttons & 1)) {
        if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
            int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
            int col = (mx - (int)GUTTER - 4) / CHAR_W;

            if (row >= 0 && row < nlines) caret_l = row;

            int len = (int)ed_strlen(lines[caret_l]);

            if (col < 0) col = 0;
            if (col > len) col = len;

            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 606) - [public] if
    if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
        int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
        int col = (mx - (int)GUTTER - 4) / CHAR_W;

        if (row >= 0 && row < nlines) caret_l = row;

        int len = (int)ed_strlen(lines[caret_l]);

        if (col < 0) col = 0;
        if (col > len) col = len;

        caret_c = col;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 618) - [public] clamp_scroll
    clamp_scroll(rows);
    dirty = 1;
    }
}

    mbuttons = btn;
    dirty = 1;
}

    uint32_t k = k_input_keycode();

    if (k != 0) {
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 629) - [public] if

```
if (k != 0) {
    switch (k) {
        case EDK_ESC:      running = 0; break;
        case EDK_ENTER:    do_enter(); dirty = 1; break;
        case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
        case EDK_DELETE:   do_delete(); dirty = 1; break;
        case EDK_TAB:      ins_char(' '); ins_char(' ');
                          ins_char(' '); ins_char(' '); dirty = 1; break;
        case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
        case EDK_RIGHT:    caret_c++; dirty = 1; break;
        case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
        case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 630) - [public] switch

```
switch (k) {
    case EDK_ESC:      running = 0; break;
    case EDK_ENTER:    do_enter(); dirty = 1; break;
    case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
    case EDK_DELETE:   do_delete(); dirty = 1; break;
    case EDK_TAB:      ins_char(' '); ins_char(' ');
                      ins_char(' '); ins_char(' '); dirty = 1; break;
    case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
    case EDK_RIGHT:    caret_c++; dirty = 1; break;
    case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
    case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
    case EDK_HOME:     caret_c = 0; dirty = 1; break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 632) - [public] do_enter

```
case EDK_ENTER:    do_enter(); dirty = 1; break;
case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
case EDK_DELETE:   do_delete(); dirty = 1; break;
case EDK_TAB:      ins_char(' '); ins_char(' ');
                  ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:    caret_c++; dirty = 1; break;
case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:     caret_c = 0; dirty = 1; break;
case EDK_END:      caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:       ed_compile_run(); dirty = 1; break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 633) - [public] do_backspace

```
case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
case EDK_DELETE:   do_delete(); dirty = 1; break;
case EDK_TAB:      ins_char(' '); ins_char(' ');
                  ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:    caret_c++; dirty = 1; break;
case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:     caret_c = 0; dirty = 1; break;
case EDK_END:      caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:       ed_compile_run(); dirty = 1; break;
case EDK_F8:       show_log = !show_log; dirty = 1; break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 634) - [public] do_delete

```
case EDK_DELETE:    do_delete(); dirty = 1; break;
case EDK_TAB:       ins_char(' '); ins_char(' ');
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 635) - [public] ins_char

```
case EDK_TAB:       ins_char(' '); ins_char(' ');
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 636) - [public] ins_char

```
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
        ins_char((char)k);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 643) - [public] ed_compile_run

```
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
        ins_char((char)k);
        dirty = 1;
    }
    break;
}

clamp_scroll(rows);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 646) - [public] if

```
        if (k >= 32 && k < 0x100) {
            ins_char((char)k);
            dirty = 1;
        }
        break;
    }

    clamp_scroll(rows);
}

if (dirty) {
    render();
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 647) - [public] ins_char

```
        ins_char((char)k);
        dirty = 1;
    }
    break;
}

    clamp_scroll(rows);
}

if (dirty) {
    render();
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 653) - [public] clamp_scroll

```
    clamp_scroll(rows);
}

if (dirty) {
    render();
}

    __asm__ volatile("pause");
}

console_set_enabled(1);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 656) - [public] if

```
    if (dirty) {
        render();
    }

    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 657) - [public] render

```
    render();
}

    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 660) - [public] volatile

```
    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 663) - [public] console_set_enabled

```
    console_set_enabled(1);

    return 0;
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 5) - [public] volatile

```
    __asm__ volatile("outb %0, %1" :: "a"(val), "Nd"(port));
}

static inline uint8_t inb(uint16_t port)
{
    uint8_t v;
    __asm__ volatile("inb %1, %0" : "=a"(v) : "Nd"(port));
    return v;
}
```

```
static void wait_write(void)
{
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 11) - [public] volatile

```
    __asm__ volatile("inb %1, %0" : "=a"(v) : "Nd"(port));
    return v;
}
```

```
static void wait_write(void)
{
    while (inb(0x64) & 0x02) { }
}
```

```
static void wait_read(void)
{
    while (!(inb(0x64) & 0x01)) { }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 17) - [public] while

```
    while (inb(0x64) & 0x02) { }
}
```

```
static void wait_read(void)
{
    while (!(inb(0x64) & 0x01)) { }
}
```

```
static void mouse_cmd(uint8_t cmd)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, cmd);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 22) - [public] while

```
    while (!(inb(0x64) & 0x01)) { }  
}  
  
static void mouse_cmd(uint8_t cmd)  
{  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, cmd);  
    wait_read();  inb(0x60);  
}  
  
static void mouse_rate(uint8_t r)  
{  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 27) - [public] wait_write

```
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, cmd);  
    wait_read();  inb(0x60);  
}  
  
static void mouse_rate(uint8_t r)  
{  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, 0xF3);  
    wait_read();  inb(0x60);  
  
    wait_write(); outb(0x64, 0xD4);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 28) - [public] wait_write

```
    wait_write(); outb(0x60, cmd);  
    wait_read();  inb(0x60);  
}  
  
static void mouse_rate(uint8_t r)  
{  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, 0xF3);  
    wait_read();  inb(0x60);  
  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, r);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 29) - [public] wait_read

```
    wait_read();  inb(0x60);  
}  
  
static void mouse_rate(uint8_t r)  
{  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, 0xF3);  
    wait_read();  inb(0x60);  
  
    wait_write(); outb(0x64, 0xD4);  
    wait_write(); outb(0x60, r);  
    wait_read();  inb(0x60);  
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 34) - [public] wait_write
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, 0xF3);
    wait_read(); inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 35) - [public] wait_write
    wait_write(); outb(0x60, 0xF3);
    wait_read(); inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 36) - [public] wait_read
    wait_read(); inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 38) - [public] wait_write
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 39) - [public] wait_write
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 40) - [public] wait_read
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 45) - [public] wait_write
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 47) - [public] mouse_cmd
    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/editor/mouse.c (line 49) - [public] mouse_rate

```
    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 50) - [public] mouse_rate

```
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 51) - [public] mouse_rate

```
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    while (inb(0x64) & 0x01) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 53) - [public] mouse_cmd

```
    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    while (inb(0x64) & 0x01) {
        uint8_t st = inb(0x64);
        uint8_t d = inb(0x60);
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 62) - [public] while
    while (inb(0x64) & 0x01) {
        uint8_t st = inb(0x64);
        uint8_t d = inb(0x60);

        if (!(st & 0x20)) {
            continue;
        }

        if (idx == 0 && !(d & 0x08)) {
            continue;
        }

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 66) - [public] if
    if (!(st & 0x20)) {
        continue;
    }

    if (idx == 0 && !(d & 0x08)) {
        continue;
    }

    pkt[idx++] = d;

    if (idx == 4) {
        idx = 0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 70) - [public] if
    if (idx == 0 && !(d & 0x08)) {
        continue;
    }

    pkt[idx++] = d;

    if (idx == 4) {
        idx = 0;

        *buttons = pkt[0] & 0x07;

        *dx = (int)pkt[1] - (int)((pkt[0] << 4) & 0x100);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 76) - [public] if
    if (idx == 4) {
        idx = 0;

        *buttons = pkt[0] & 0x07;

        *dx = (int)pkt[1] - (int)((pkt[0] << 4) & 0x100);
        *dy = (int)pkt[2] - (int)((pkt[0] << 3) & 0x100);
        *dz = (int)(int8_t)pkt[3];

        got = 1;
    }
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/hdmi/init.c (line 8) - [public] if

```
    if (hdmi_is_ready) {
        return true;
    }

    if (fb_phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 12) - [public] if

```
    if (fb_phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 16) - [public] hdmi_op_set_fb

```
    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 17) - [public] if

```
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/init.c (line 22) - [public] if
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
}

bool hdmi_ready(void)
{
    return hdmi_is_ready && hdmi_op_ready();
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 32) - [public] hdmi_op_ready
    return hdmi_is_ready && hdmi_op_ready();
}

void hdmi_caps(hdmi_caps_t *out)
{
    hdmi_op_state(out);
}

bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 37) - [public] hdmi_op_state
    hdmi_op_state(out);
}

bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 42) - [public] if
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
}

bool hdmi_mode_set_by_id(uint32_t id)
{
    if (!hdmi_ready()) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/hdmi/init.c (line 46) - [public] hdmi_op_set_fb
    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
}

bool hdmi_mode_set_by_id(uint32_t id)
{
    if (!hdmi_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_by_id(id);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 53) - [public] if
    if (!hdmi_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_by_id(id);
    hdmi_caps_t caps;
    hdmi_op_state(&caps);
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 59) - [public] hdmi_op_state
    hdmi_op_state(&caps);
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 60) - [public] if
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/hdmi/init.c (line 64) - [public] hdmi_mode_apply

```
    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = mode->id;
    *w = mode->w;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 70) - [public] if

```
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = mode->id;
    *w = mode->w;
    *h = mode->h;
    *r = mode->refresh;
    return true;
}
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 9) - [public] if

```
    if (owner == 0) {
        return false;
    }

    uint32_t expected = 0;
    if (atomic_compare_exchange_strong(&port_owner, &expected, owner)) {
        return true;
    }

    return expected == owner;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 14) - [public] if

```
    if (atomic_compare_exchange_strong(&port_owner, &expected, owner)) {
        return true;
    }

    return expected == owner;
}

bool hdmi_iface_release(uint32_t owner)
{
    uint32_t expected = owner;
    return owner != 0 && atomic_compare_exchange_strong(&port_owner, &expected, 0);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 29) - [public] atomic_load

```
    return atomic_load(&port_owner);
}

bool hdmi_iface_check(uint32_t owner)
{
    return owner != 0 && atomic_load(&port_owner) == owner;
}

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (owner != 0 && !hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 39) - [public] if

```
    if (owner != 0 && !hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_init_with(fb_phys, w, h, stride);
}

bool hdmi_iface_ready(void)
{
    return hdmi_ready();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 43) - [public] hdmi_init_with

```
    return hdmi_init_with(fb_phys, w, h, stride);
}

bool hdmi_iface_ready(void)
{
    return hdmi_ready();
}

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 48) - [public] hdmi_ready

```
    return hdmi_ready();
}

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_mode_set_by_id(id);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 53) - [public] if

```
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_mode_set_by_id(id);
}

bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_current_raw(id, w, h, r);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 57) - [public] hdmi_mode_set_by_id

```
    return hdmi_mode_set_by_id(id);
}

bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_current_raw(id, w, h, r);
}

bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_at_raw(i, id, w, h, r);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 62) - [public] hdmi_mode_current_raw

```
    return hdmi_mode_current_raw(id, w, h, r);
}

bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_at_raw(i, id, w, h, r);
}

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h)
{
    if (!hdmi_iface_check(owner)) {
        return 0;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 67) - [public] hdmi_mode_at_raw

```
    return hdmi_mode_at_raw(i, id, w, h, r);
}

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h)
{
    if (!hdmi_iface_check(owner)) {
        return 0;
    }

    return hdmi_submit_fill(color, x, y, w, h);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 72) - [public] if

```
    if (!hdmi_iface_check(owner)) {
        return 0;
    }

    return hdmi_submit_fill(color, x, y, w, h);
}

bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/hdmi/interface.c (line 76) - [public] hdmi_submit_fill
    return hdmi_submit_fill(color, x, y, w, h);
}

bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_poll(out_seq);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 81) - [public] if
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_poll(out_seq);
}

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys)
{
    hdmi_caps_raw(w, h, s, phys);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 85) - [public] hdmi_poll
    return hdmi_poll(out_seq);
}

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys)
{
    hdmi_caps_raw(w, h, s, phys);
}

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s)
{
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 90) - [public] hdmi_caps_raw
    hdmi_caps_raw(w, h, s, phys);
}

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s)
{
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }

    uint64_t p;
    uint32_t ww, hh, ss;
    hdmi_caps_raw(&ww, &hh, &ss, &p);
    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 95) - [public] if
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }

    uint64_t p;
    uint32_t ww, hh, ss;
    hdmi_caps_raw(&ww, &hh, &ss, &p);

    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 101) - [public] hdmi_caps_raw
    hdmi_caps_raw(&ww, &hh, &ss, &p);

    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    *phys = p;
    *w = ww;
    *h = hh;
    *s = ss;
    return true;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 103) - [public] if
    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    *phys = p;
    *w = ww;
    *h = hh;
    *s = ss;
    return true;
}

bool hdmi_iface_fb_revoke(uint32_t owner)
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 116) - [public] if
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    hdmi_fb_revoke();
    return true;
}
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 120) - [public] hdmi_fb_revoke
    hdmi_fb_revoke();
    return true;
}
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 17) - [public] return

```
    return (uint32_t)(sizeof(MODES) / sizeof(MODES[0]));
}

const hdmi_mode_t *hdmi_mode_at(uint32_t i)
{
    if (i >= hdmi_mode_count()) {
        return NULL;
    }

    return &MODES[i];
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 22) - [public] if

```
    if (i >= hdmi_mode_count()) {
        return NULL;
    }

    return &MODES[i];
}

const hdmi_mode_t *hdmi_mode_find(uint32_t w, uint32_t h, uint32_t refresh)
{
    for (uint32_t i = 0; i < hdmi_mode_count(); i++) {
        if (MODES[i].w == w && MODES[i].h == h && MODES[i].refresh == refresh) {
            return &MODES[i];
        }
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 32) - [public] if

```
    if (MODES[i].w == w && MODES[i].h == h && MODES[i].refresh == refresh) {
        return &MODES[i];
    }

    return NULL;
}

const hdmi_mode_t *hdmi_mode_by_id(uint32_t id)
{
    for (uint32_t i = 0; i < hdmi_mode_count(); i++) {
        if (MODES[i].id == id) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 43) - [public] if

```
    if (MODES[i].id == id) {
        return &MODES[i];
    }

    return NULL;
}

bool hdmi_mode_valid(const hdmi_mode_t *m)
{
    if (m == NULL || m->w == 0 || m->h == 0 || m->refresh == 0 || m->clock_khz == 0) {
        return false;
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/mode.c (line 53) - [public] if
    if (m == NULL || m->w == 0 || m->h == 0 || m->refresh == 0 || m->clock_khz == 0) {
        return false;
    }

    hdmi_caps_t caps;
    hdmi_op_state(&caps);
    return caps.ready != 0 && caps.fb_phys != 0;
}

bool hdmi_mode_apply(const hdmi_mode_t *m)
{
    if (!hdmi_mode_valid(m)) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 58) - [public] hdmi_op_state
    hdmi_op_state(&caps);
    return caps.ready != 0 && caps.fb_phys != 0;
}

bool hdmi_mode_apply(const hdmi_mode_t *m)
{
    if (!hdmi_mode_valid(m)) {
        return false;
    }

    hdmi_op_set_mode(m);
    current_mode = m;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 64) - [public] if
    if (!hdmi_mode_valid(m)) {
        return false;
    }

    hdmi_op_set_mode(m);
    current_mode = m;
    return true;
}

const hdmi_mode_t *hdmi_mode_current(void)
{
    return current_mode;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 68) - [public] hdmi_op_set_mode
    hdmi_op_set_mode(m);
    current_mode = m;
    return true;
}

const hdmi_mode_t *hdmi_mode_current(void)
{
    return current_mode;
}

bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 81) - [public] if
    if (m == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = m->id;
    *w = m->w;
    *h = m->h;
    *r = m->refresh;
    return true;
}

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 7) - [public] return

```
    return (void *)(DIRECT_BASE + phys);
}

static hdmi_caps_t cur;
static hdmi_mode_t cur_mode;
static bool op_ready;
static hdmi_transfer_t slots[HDMI_MAX_TRANSFERS];
static uint64_t slot_seq[HDMI_MAX_TRANSFERS];
static bool slot_used[HDMI_MAX_TRANSFERS];
static bool slot_done[HDMI_MAX_TRANSFERS];
static uint64_t next_seq = 1;
static uint32_t pending;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 22) - [public] if

```
    if (out != NULL) {
        *out = cur;
    }
}

bool hdmi_op_ready(void)
{
    return op_ready;
}

void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 44) - [public] if

```
    if (t == NULL || !op_ready) {
        return;
    }

    uint32_t stride = t->stride != 0 ? t->stride : cur.stride;
    uint64_t dst_phys = t->dst_phys != 0 ? t->dst_phys : cur.fb_phys;
    if (stride < cur.w || dst_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)phys_to_virt(dst_phys);
    if (dst == NULL) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 50) - [public] if

```
    if (stride < cur.w || dst_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)phys_to_virt(dst_phys);
    if (dst == NULL) {
        return;
    }

    if (t->kind == HDMI_TR_FILL) {
        if (t->x >= cur.w || t->y >= cur.h) {
            return;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 55) - [public] if

```
if (dst == NULL) {
    return;
}

if (t->kind == HDMI_TR_FILL) {
    if (t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 59) - [public] if

```
if (t->kind == HDMI_TR_FILL) {
    if (t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = t->color;
        }
    }
    return;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 60) - [public] if

```
if (t->x >= cur.w || t->y >= cur.h) {
    return;
}
uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
for (uint32_t y = 0; y < height; y++) {
    for (uint32_t x = 0; x < width; x++) {
        dst[(t->y + y) * stride + t->x + x] = t->color;
    }
}
return;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 73) - [public] if

```
if (t->kind == HDMI_TR_BLIT) {
    if (t->src_phys == 0 || t->stride == 0 || t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t *src = (uint32_t *)phys_to_virt(t->src_phys);
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            // ... (truncated, see source file for more)
        }
    }
}
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 74) - [public] if
    if (t->src_phys == 0 || t->stride == 0 || t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t *src = (uint32_t *)phys_to_virt(t->src_phys);
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = src[y * t->stride + x];
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 78) - [public] if
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = src[y * t->stride + x];
        }
    }
    return;
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 91) - [public] if
    if (t->kind == HDMI_TR_FLIP && t->dst_phys != 0) {
        cur.fb_phys = t->dst_phys;
    }
}

uint64_t hdmi_submit(const hdmi_transfer_t *t)
{
    if (t == NULL || !op_ready || pending >= HDMI_MAX_TRANSFERS) {
        return 0;
    }

    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 98) - [public] if
    if (t == NULL || !op_ready || pending >= HDMI_MAX_TRANSFERS) {
        return 0;
    }

    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        return 0;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (!slot_used[i]) {
            slots[i] = *t;
            slot_seq[i] = next_seq++;
        }
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 102) - [public] if
    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        return 0;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (!slot_used[i]) {
            slots[i] = *t;
            slot_seq[i] = next_seq++;
            if (next_seq == 0) {
                next_seq = 1;
            }
            slot_used[i] = true;
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/operation.c (line 107) - [public] if
        if (!slot_used[i]) {
            slots[i] = *t;
            slot_seq[i] = next_seq++;
            if (next_seq == 0) {
                next_seq = 1;
            }
            slot_used[i] = true;
            slot_done[i] = false;
            pending++;
            hdmi_op_exec(&slots[i]);
            slot_done[i] = true;
            return slot_seq[i];
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/operation.c (line 110) - [public] if
            if (next_seq == 0) {
                next_seq = 1;
            }
            slot_used[i] = true;
            slot_done[i] = false;
            pending++;
            hdmi_op_exec(&slots[i]);
            slot_done[i] = true;
            return slot_seq[i];
        }
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/operation.c (line 116) - [public] hdmi_op_exec
        hdmi_op_exec(&slots[i]);
        slot_done[i] = true;
        return slot_seq[i];
    }

    return 0;
}

bool hdmi_poll(uint64_t *out_seq)
{
    if (out_seq == NULL) {
        // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/hdmi/operation.c (line 127) - [public] if

```
    if (out_seq == NULL) {
        return false;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (slot_used[i] && slot_done[i]) {
            *out_seq = slot_seq[i];
            slot_used[i] = false;
            slot_done[i] = false;
            pending--;
            return true;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 132) - [public] if

```
        if (slot_used[i] && slot_done[i]) {
            *out_seq = slot_seq[i];
            slot_used[i] = false;
            slot_done[i] = false;
            pending--;
            return true;
        }
    }

    return false;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 159) - [public] hdmi_submit

```
    return hdmi_submit(&t);
}

void hdmi_op_set_mode(const hdmi_mode_t *m)
{
    if (m == NULL) {
        return;
    }
    cur_mode = *m;
    cur.w = m->w;
    cur.h = m->h;
    cur.ready = (cur.fb_phys != 0 && cur.w != 0 && cur.h != 0 && cur.stride >= cur.w) ? 1 : 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 164) - [public] if

```
    if (m == NULL) {
        return;
    }
    cur_mode = *m;
    cur.w = m->w;
    cur.h = m->h;
    cur.ready = (cur.fb_phys != 0 && cur.w != 0 && cur.h != 0 && cur.stride >= cur.w) ? 1 : 0;
    op_ready = cur.ready != 0;
}

static bool fb_granted = false;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 178) - [public] if

```
    if (cur.fb_phys == 0) {
        return false;
    }

    fb_granted = true;
    return true;
}
```

```
void hdmi_fb_revoke(void)
{
    fb_granted = false;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 198) - [public] if

```
    if (w == NULL || h == NULL || s == NULL || phys == NULL) {
        return;
    }
    *w = cur.w;
    *h = cur.h;
    *s = cur.stride;
    *phys = cur.fb_phys;
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 8) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void k_memset(const void *dst, const uint8 value, const size_t n) {
    ■uint8_t *p = (uint8_t *)dst;
    ■for (size_t i = 0; i < n; i++) p[i] = value;
}

static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 10) - [internal] k_memset

```
static void k_memset(const void *dst, const uint8 value, const size_t n) {
    ■uint8_t *p = (uint8_t *)dst;
    ■for (size_t i = 0; i < n; i++) p[i] = value;
}

static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

static size_t k_strlen(const char *s) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 15) - [internal] k_memcpy

```
static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

static size_t k_strlen(const char *s) {
    ■size_t n = 0;
    ■while (s[n] != '\0') n++;
    ■return n;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 21) - [internal] k_strlen

```
static size_t k_strlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0') n++;
    return n;
}

static size_t k_usable_size(void *ptr) {
#ifdef ALLOC_DEBUG
    size_t s = dbg_usable_size(ptr);
    if (s != 0) return s;
#endif
    return heap_usable_size(ptr);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 27) - [internal] k_usable_size

```
static size_t k_usable_size(void *ptr) {
#ifdef ALLOC_DEBUG
    size_t s = dbg_usable_size(ptr);
    if (s != 0) return s;
#endif
    return heap_usable_size(ptr);
}

void *kmalloc(const size_t size) {
    if (size == 0) return NULL;
#ifdef ALLOC_DEBUG
    return dbg_alloc(size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 32) - [public] heap_usable_size

```
return heap_usable_size(ptr);
}

void *kmalloc(const size_t size) {
    if (size == 0) return NULL;
#ifdef ALLOC_DEBUG
    return dbg_alloc(size);
#else
    return heap_alloc(size);
#endif
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 38) - [public] dbg_alloc

```
return dbg_alloc(size);
#else
return heap_alloc(size);
#endif
}

void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 40) - [public] heap_alloc

```
    return heap_alloc(size);
#endif
}

void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
#endif
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 44) - [public] kfree

```
void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
#endif
}

void *kzalloc(const size_t size) {
    void *ptr = kmalloc(size);
    if (ptr == NULL) return NULL;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 63) - [public] kzalloc

```
    return kzalloc(count * size);
}

void *kmalloc_aligned(const size_t size, const size_t align) {
    if (size == 0) return NULL;

    if (!size_is_pow2(align)) return NULL;
    return heap_alloc_aligned(size, align);
}

size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 70) - [public] heap_alloc_aligned

```
    return heap_alloc_aligned(size, align);
}

size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }

void *krealloc(void *ptr, const size_t new_size) {
    if (ptr == NULL) return kmalloc(new_size);
    if (new_size == 0) {
        kfree(ptr);
        return NULL;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 74) - [public] kmalloc_usable_size

```
size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }
```

```
void *krealloc(void *ptr, const size_t new_size) {  
    if (ptr == NULL) return kmalloc(new_size);  
    if (new_size == 0) {  
        kfree(ptr);  
        return NULL;  
    }  
    const size_t old_size = k_usable_size(ptr);  
    if (old_size == 0) return NULL;  
    if (old_size >= new_size) return ptr;  
    void *new_ptr = kmalloc(new_size);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 79) - [public] kfree

```
    kfree(ptr);  
    return NULL;  
}  
const size_t old_size = k_usable_size(ptr);  
if (old_size == 0) return NULL;  
if (old_size >= new_size) return ptr;  
void *new_ptr = kmalloc(new_size);  
if (new_ptr == NULL) return NULL;  
k_memcpy(new_ptr, ptr, old_size);  
kfree(ptr);  
return new_ptr;  
}  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 95) - [public] heap_alloc

```
return heap_alloc(pages * ARCH_PAGE_SIZE);  
}
```

```
void kfree_pages(void *ptr, const size_t pages) {  
    if (ptr == NULL) return;  
    size_t requested_bytes = 0;  
    const size_t usable = heap_usable_size(ptr);  
    if (!kfree_pages_validate(pages, ARCH_PAGE_SIZE, usable, &requested_bytes)) {  
        kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p – odmowa zwro  
        return;  
    }  
    heap_free(ptr);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 98) - [public] kfree_pages

```
void kfree_pages(void *ptr, const size_t pages) {  
    if (ptr == NULL) return;  
    size_t requested_bytes = 0;  
    const size_t usable = heap_usable_size(ptr);  
    if (!kfree_pages_validate(pages, ARCH_PAGE_SIZE, usable, &requested_bytes)) {  
        kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p – odmowa zwro  
        return;  
    }  
    heap_free(ptr);  
}
```

```
char *kstrdup(const char *s) {  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 103) - [public] kprintf

```
    kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p - odmowa zwolnienia\n", pages, block_size, ptr);
    return;
}

heap_free(ptr);
}

char *kstrdup(const char *s) {
    if (s == NULL) return NULL;
    const size_t n = k_strlen(s) + 1;
    char *ptr = (char *)kmalloc(n);
    if (ptr == NULL) return NULL;
    k_memcpy(ptr, s, n);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 118) - [public] kvirt_to_phys

```
uint64_t kvirt_to_phys(void *ptr) {
    if (ptr == NULL) return UINT64_MAX;
    const uint64_t va = (uint64_t)(uintptr_t)ptr;
    phys = mapping_translate(MAPPING_KERNEL, va);
    if (phys != UINT64_MAX) return phys;
    return arch_virt_to_phys(ptr);
}

void kalloc_dump(void) {
#ifdef ALLOC_DEBUG
    mm_debug_dump();
#else
    // ... (truncated, see source file for more)
}
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 123) - [public] arch_virt_to_phys

```
return arch_virt_to_phys(ptr);
}

void kalloc_dump(void) {
#ifdef ALLOC_DEBUG
    mm_debug_dump();
#else
    heap_dump();
#endif
}
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 126) - [public] kalloc_dump

```
void kalloc_dump(void) {
#ifdef ALLOC_DEBUG
    mm_debug_dump();
#else
    heap_dump();
#endif
}
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 11) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define DBG_MAGIC 0x444247414C4C4F43ULL
#define DBG_POISON_ALLOC 0xAA
#define DBG_POISON_FREE 0xFF

typedef struct dbg_header {
    uint64_t magic;
    uint64_t size;
    uint64_t caller;
    uint64_t tracked;
    uint64_t alloc_id;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 25) - [internal]

dbg_tail_canary

```
static uint64_t dbg_tail_canary(size_t size) { return ~((uint64_t)size) ^ DBG_MAGIC; }
static uint64_t *dbg_tail_ptr(void *user, size_t size) { return (uint64_t *)((uint8_t *)user + size); }

void *dbg_alloc(size_t size) {
    if (size == 0) return NULL;

    uint64_t caller = (uint64_t)(uintptr_t)__builtin_return_address(0);
    uint8_t *real = (uint8_t *)heap_alloc(sizeof(dbg_header_t) + size + sizeof(uint64_t));

    if (real == NULL) return NULL;

    dbg_header_t *h = (dbg_header_t *)real;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 56) - [public] dbg_free

```
void dbg_free(void *ptr) {
    if (ptr == NULL) return;

    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) {
        kprintf("dbg_free: bad magic at %p\n", ptr);
        return;
    }

    size_t size = h->size;
    if (*dbg_tail_ptr(ptr, size) != dbg_tail_canary(size)) kprintf("dbg_free: buffer overflow at %p caller 0\n", ptr);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 61) - [public] kprintf

```
kprintf("dbg_free: bad magic at %p\n", ptr);
return;
}

size_t size = h->size;
if (*dbg_tail_ptr(ptr, size) != dbg_tail_canary(size)) kprintf("dbg_free: buffer overflow at %p caller 0\n", ptr);

if (h->tracked) {
    size_t leaked_size = 0;
    uint64_t alloc_caller = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 73) - [public] if

```
if (!leak_untrack(ptr, &leaked_size, &alloc_caller)) {
    kprintf("dbg_free: double free or unknown ptr %p\n", ptr);
    return;
}

alloc_stats_note_free(size);

h->magic = 0;

uint8_t *real = (uint8_t *)h;
size_t total = sizeof(dbg_header_t) + size + sizeof(uint64_t);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 74) - [public] kprintf

```
    kprintf("dbg_free: double free or unknown ptr %p\n", ptr);
    return;
}
```

```
alloc_stats_note_free(size);
```

```
h->magic = 0;
```

```
uint8_t *real = (uint8_t *)h;
size_t total = sizeof(dbg_header_t) + size + sizeof(uint64_t);
for (size_t i = 0; i < total; i++) real[i] = DBG_POISON_FREE;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 90) - [public] dbg_usable_size

```
size_t dbg_usable_size(void *ptr) {
    if (ptr == NULL) return 0;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return 0;
    return h->size;
}
```

```
bool dbg_verify(void *ptr) {
    if (ptr == NULL) return false;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return false;
    return *dbg_tail_ptr(ptr, h->size) == dbg_tail_canary(h->size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 97) - [public] dbg_verify

```
bool dbg_verify(void *ptr) {
    if (ptr == NULL) return false;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return false;
    return *dbg_tail_ptr(ptr, h->size) == dbg_tail_canary(h->size);
}
```

```
void mm_debug_dump(void) {
    alloc_stats_dump();
    leak_dump();
    pmm_dump();
    vmm_dump();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 104) - [public] mm_debug_dump

```
void mm_debug_dump(void) {
    alloc_stats_dump();
    leak_dump();
    pmm_dump();
    vmm_dump();
    buddy_dump();
    slab_dump();
    cache_dump();
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 4) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);
```

```
#define LEAK_MAX 2048
```

```
typedef struct leak_entry {
    void *ptr;
    size_t size;
    uint64_t caller, alloc_id;
    int64_t cpu_id;
    bool used;
} leak_entry_t;
```

```
// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/debug/leak.c (line 22) - [public] leak_track_id

```
bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (!leak_table[i].used) {
            uint64_t id = next_alloc_id++;
            leak_table[i].ptr = ptr;
            leak_table[i].size = size;
            leak_table[i].caller = caller;
            leak_table[i].alloc_id = id;
            leak_table[i].cpu_id = smp_current_cpu_id();
            leak_table[i].used = true;
            smp_lock_release(&leak_smp_lock);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/mm/alloc/debug/leak.c (line 25) - [public] if

```
    if (!leak_table[i].used) {
        uint64_t id = next_alloc_id++;
        leak_table[i].ptr = ptr;
        leak_table[i].size = size;
        leak_table[i].caller = caller;
        leak_table[i].alloc_id = id;
        leak_table[i].cpu_id = smp_current_cpu_id();
        leak_table[i].used = true;
        smp_lock_release(&leak_smp_lock);
        if (out_id != NULL) *out_id = id;
        return true;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/alloc/debug/leak.c (line 33) - [public] smp_lock_release

```
    smp_lock_release(&leak_smp_lock);
    if (out_id != NULL) *out_id = id;
    return true;
}
}
smp_lock_release(&leak_smp_lock);
return false;
}
bool leak_track(void *ptr, size_t size, uint64_t caller) { return leak_track_id(ptr, size, caller, NULL); }
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/mm/alloc/debug/leak.c (line 41) - [public] leak_track

```
bool leak_track(void *ptr, size_t size, uint64_t caller) { return leak_track_id(ptr, size, caller, NULL); }
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (leak_table[i].used && leak_table[i].ptr == ptr) {
            if (out_size != NULL) *out_size = leak_table[i].size;
            if (out_caller != NULL) *out_caller = leak_table[i].caller;
            leak_table[i].used = false;
            smp_lock_release(&leak_smp_lock);
            return true;
        }
    }
}
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 42) - [public] leak_untrack

```
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (leak_table[i].used && leak_table[i].ptr == ptr) {
            if (out_size != NULL) *out_size = leak_table[i].size;
            if (out_caller != NULL) *out_caller = leak_table[i].caller;
            leak_table[i].used = false;
            smp_lock_release(&leak_smp_lock);
            return true;
        }
    }
    smp_lock_release(&leak_smp_lock);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 45) - [public] if

```
    if (leak_table[i].used && leak_table[i].ptr == ptr) {
        if (out_size != NULL) *out_size = leak_table[i].size;
        if (out_caller != NULL) *out_caller = leak_table[i].caller;
        leak_table[i].used = false;
        smp_lock_release(&leak_smp_lock);
        return true;
    }
}
smp_lock_release(&leak_smp_lock);
return false;
}
bool leak_contains(void *ptr) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 49) - [public] smp_lock_release

```
    smp_lock_release(&leak_smp_lock);
    return true;
}
}
smp_lock_release(&leak_smp_lock);
return false;
}
bool leak_contains(void *ptr) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (leak_table[i].used && leak_table[i].ptr == ptr) {
            smp_lock_release(&leak_smp_lock);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 56) - [public] leak_contains

```
bool leak_contains(void *ptr) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (leak_table[i].used && leak_table[i].ptr == ptr) {
            smp_lock_release(&leak_smp_lock);
            return true;
        }
    }
    smp_lock_release(&leak_smp_lock);
    return false;
}
size_t leak_count(void) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 59) - [public] if

```
■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
■        smp_lock_release(&leak_smp_lock);
■        return true;
■    }
■}
■smp_lock_release(&leak_smp_lock);
■return false;
}
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 60) - [public] smp_lock_release

```
■        smp_lock_release(&leak_smp_lock);
■        return true;
■    }
■}
■smp_lock_release(&leak_smp_lock);
■return false;
}
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■smp_lock_release(&leak_smp_lock);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 67) - [public] leak_count

```
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■smp_lock_release(&leak_smp_lock);
■return n;
}
void leak_dump(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■kprintf("LEAKS: %llu live\n", (unsigned long long)n);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 74) - [public] leak_dump

```
void leak_dump(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■kprintf("LEAKS: %llu live\n", (unsigned long long)n);
■for (size_t i = 0; i < LEAK_MAX; i++) {
■    if (!leak_table[i].used) continue;
■    kprintf("    leak: %p size %llu caller 0x%llx id %llu cpu %lld\n",
■        leak_table[i].ptr,
■        (unsigned long long)leak_table[i].size,
■        (unsigned long long)leak_table[i].caller,
■        (unsigned long long)leak_table[i].alloc_id,
■        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 4) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static size_t
■stat_live_count = 0,
■stat_live_bytes = 0,
■stat_peak_bytes = 0,
■stat_total_allocs = 0,
■stat_total_frees = 0;

static smp_ticket_lock_t stats_smp_lock = SMP_TICKET_LOCK_INIT;

void alloc_stats_note_alloc(size_t size) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 15) - [public] alloc_stats_note_alloc

```
void alloc_stats_note_alloc(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■stat_live_count++;
    ■stat_live_bytes += size;
    ■stat_total_allocs++;
    ■if (stat_live_bytes > stat_peak_bytes) stat_peak_bytes = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
}

void alloc_stats_note_free(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■if (stat_live_count > 0) stat_live_count--;
    ■if (stat_live_bytes >= size) stat_live_bytes -= size;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 23) - [public] alloc_stats_note_free

```
void alloc_stats_note_free(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■if (stat_live_count > 0) stat_live_count--;
    ■if (stat_live_bytes >= size) stat_live_bytes -= size;
    ■else stat_live_bytes = 0;
    ■stat_total_frees++;
    ■smp_lock_release(&stats_smp_lock);
}

size_t alloc_stats_live_count(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_count;
    ■smp_lock_release(&stats_smp_lock);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 31) - [public] alloc_stats_live_count

```
size_t alloc_stats_live_count(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_count;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}

size_t alloc_stats_live_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 37) - [public] alloc_stats_live_bytes

```
size_t alloc_stats_live_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_peak_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_peak_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 43) - [public] alloc_stats_peak_bytes

```
size_t alloc_stats_peak_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_peak_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_total_allocs(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_allocs;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 49) - [public]
alloc_stats_total_allocs

```
size_t alloc_stats_total_allocs(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_allocs;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_total_frees(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 55) - [public]
alloc_stats_total_frees

```
size_t alloc_stats_total_frees(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
void alloc_stats_dump(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t
    ■■live_count = stat_live_count,
    ■■live_bytes = stat_live_bytes,
    ■■peak_bytes = stat_peak_bytes,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 61) - [public] alloc_stats_dump

```
void alloc_stats_dump(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t
    ■■live_count = stat_live_count,
    ■■live_bytes = stat_live_bytes,
    ■■peak_bytes = stat_peak_bytes,
    ■■total_allocs = stat_total_allocs,
    ■■total_frees = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■kprintf("ALLOC STATS:\n");
    ■kprintf("  live: %llu objs, %llu B\n",
    ■■■(unsigned long long)live_count,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 6) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define BUDDY_STATE_FREE 0
#define BUDDY_STATE_USED 1
#define BUDDY_STATE_TAIL 2

static uint64_t buddy_base = 0;
static size_t buddy_pages = 0, buddy_max_order = 0;

static uint8_t *buddy_order = NULL, *buddy_state = NULL;
static int32_t *buddy_next = NULL, *buddy_heads = NULL;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 25) - [internal] order_size

```
static inline uint64_t order_size(size_t o) { return ARCH_PAGE_SIZE << o; }
static size_t page_of(uint64_t va) { return (size_t)((va - buddy_base) / ARCH_PAGE_SIZE); }
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    ■size_t o = 0, s = ARCH_PAGE_SIZE;
    ■while (s < size && o < buddy_max_order) {
    ■    s <= 1;
    ■    o++;
    ■}
    ■return o;
}
static void fl_push(size_t o, size_t idx) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 26) - [internal] page_of

```
static size_t page_of(uint64_t va) { return (size_t)((va - buddy_base) / ARCH_PAGE_SIZE); }
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    ■size_t o = 0, s = ARCH_PAGE_SIZE;
    ■while (s < size && o < buddy_max_order) {
    ■    s <= 1;
    ■    o++;
    ■}
    ■return o;
}
static void fl_push(size_t o, size_t idx) {
    ■buddy_next[idx] = buddy_heads[o];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 27) - [internal] va_of

```
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    size_t o = 0, s = ARCH_PAGE_SIZE;
    while (s < size && o < buddy_max_order) {
        s <= 1;
        o++;
    }
    return o;
}
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 28) - [internal] size_to_order

```
static size_t size_to_order(size_t size) {
    size_t o = 0, s = ARCH_PAGE_SIZE;
    while (s < size && o < buddy_max_order) {
        s <= 1;
        o++;
    }
    return o;
}
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 36) - [internal] fl_push

```
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
}
static size_t fl_pop(size_t o) {
    int32_t i = buddy_heads[o];
    if (i < 0) return (size_t)-1;
    buddy_heads[o] = buddy_next[i];
    return (size_t)i;
}
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 40) - [internal] fl_pop

```
static size_t fl_pop(size_t o) {
    int32_t i = buddy_heads[o];
    if (i < 0) return (size_t)-1;
    buddy_heads[o] = buddy_next[i];
    return (size_t)i;
}
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    while (*p >= 0) {
        if ((size_t)*p == idx) {
            *p = buddy_next[*p];
            return;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 46) - [internal] fl_remove

```
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    while (*p >= 0) {
        if ((size_t)*p == idx) {
            *p = buddy_next[*p];
            return;
        }
        p = &buddy_next[*p];
    }
}

bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 49) - [public] if

```
    if ((size_t)*p == idx) {
        *p = buddy_next[*p];
        return;
    }
    p = &buddy_next[*p];
}

bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    if (map_cb == NULL || unmap_cb == NULL) return false;
    if (size == 0 || (size & (size - 1)) != 0) return false;
    if (size < ARCH_PAGE_SIZE) return false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 56) - [public] buddy_init

```
bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    if (map_cb == NULL || unmap_cb == NULL) return false;
    if (size == 0 || (size & (size - 1)) != 0) return false;
    if (size < ARCH_PAGE_SIZE) return false;
    if (!arch_is_page_aligned(base)) return false;
    if ((base & (size - 1)) != 0) return false;
    buddy_base = base;
    buddy_pages = size / ARCH_PAGE_SIZE;
    buddy_max_order = 0;
    while (order_size(buddy_max_order) < size) buddy_max_order++;
    size_t
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 95) - [public] buddy_ready

```
bool buddy_ready(void) { return buddy_initialized; }
void *buddy_alloc(size_t size) {
    if (!buddy_initialized || size == 0) return NULL;
    size_t o = size_to_order(size);
    if (order_size(o) < size || o > buddy_max_order) return NULL;
    size_t cur = 0;
    while (buddy_heads[cur] < 0 && cur < buddy_max_order) cur++;
    if (buddy_heads[cur] < 0) return NULL;
    size_t idx = fl_pop(cur);
    while (cur > 0) {
        cur--;
        size_t half_pages = (size_t)1 << cur, buddy_idx = idx + half_pages;
        // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 109) - [public] fl_push

```
    fl_push(cur, buddy_idx);
}
buddy_order[idx] = (uint8_t)o;
buddy_state[idx] = BUDDY_STATE_USED;
size_t bytes = order_size(o);
if (!buddy_map(va_of(idx), bytes)) {
    buddy_state[idx] = BUDDY_STATE_FREE;
    fl_push(o, idx);
    return NULL;
}
buddy_used_bytes += bytes;
buddy_free_bytes -= bytes;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 116) - [public] fl_push

```
    fl_push(o, idx);
    return NULL;
}
buddy_used_bytes += bytes;
buddy_free_bytes -= bytes;
return (void *) (uintptr_t) va_of(idx);
}
```

```
void *buddy_alloc_aligned(size_t size, size_t align) {
    if (!buddy_initialized || size == 0) return NULL;
    if (!size_is_pow2(align)) return NULL;
    if (align <= ARCH_PAGE_SIZE) return buddy_alloc(size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 133) - [public] buddy_alloc

```
return buddy_alloc(p);
}
```

```
void buddy_free(void *ptr) {
    if (!buddy_initialized || ptr == NULL) return;
    uint64_t va = (uint64_t)(uintptr_t)ptr;
    if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return;
    size_t idx = page_of(va);
    if (buddy_state[idx] != BUDDY_STATE_USED) return;
    size_t o = buddy_order[idx], bytes = order_size(o);
    buddy_unmap(va, bytes);
    buddy_state[idx] = BUDDY_STATE_FREE;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 136) - [public] buddy_free

```
void buddy_free(void *ptr) {
    if (!buddy_initialized || ptr == NULL) return;
    uint64_t va = (uint64_t)(uintptr_t)ptr;
    if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return;
    size_t idx = page_of(va);
    if (buddy_state[idx] != BUDDY_STATE_USED) return;
    size_t o = buddy_order[idx], bytes = order_size(o);
    buddy_unmap(va, bytes);
    buddy_state[idx] = BUDDY_STATE_FREE;
    buddy_order[idx] = (uint8_t)o;
    size_t cur = o;
    while (cur < buddy_max_order) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 153) - [public] fl_remove

```
■ fl_remove(cur, buddy_idx);
■ buddy_state[buddy_idx] = BUDDY_STATE_TAIL;
■ if (buddy_idx < idx) idx = buddy_idx;
■ cur++;
■ buddy_order[idx] = (uint8_t)cur;
■ }
■ fl_push(cur, idx);
■ buddy_used_bytes -= bytes;
■ buddy_free_bytes += bytes;
}
size_t buddy_block_size(void *ptr) {
■ if (!buddy_initialized || ptr == NULL) return 0;
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 163) - [public] buddy_block_size

```
size_t buddy_block_size(void *ptr) {
■ if (!buddy_initialized || ptr == NULL) return 0;
■ uint64_t va = (uint64_t)(uintptr_t)ptr;
■ if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return 0;
■ size_t idx = page_of(va);
■ if (buddy_state[idx] != BUDDY_STATE_USED) return 0;
■ return (size_t)order_size(buddy_order[idx]);
}
size_t buddy_stat_used_bytes(void) { return buddy_used_bytes; }
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 171) - [public] buddy_stat_used_bytes

```
size_t buddy_stat_used_bytes(void) { return buddy_used_bytes; }
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 172) - [public] buddy_stat_free_bytes

```
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 173) - [public] buddy_dump

```
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
}
```

kernel_workspace/kernel/src/mm/alloc/heap/buddy.c (line 175) - [public] kprintf

```
■ kprintf("BUDDY: not initialized\n");
■ return;
■}
■kprintf("BUDDY:\n");
■kprintf("  base: 0x%llx\n", (unsigned long long)buddy_base);
■kprintf("  max order: %llu\n", (unsigned long long)buddy_max_order);
■kprintf("  used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■kprintf("  free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
■}
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 32) - [public] if

```
■ if (!pmm_alloc_frame(&phys)) {
■     for (size_t j = 0; j < i; j++) {
■         uint64_t v = virt + j * ARCH_PAGE_SIZE;
■         uint64_t p = mapping_translate(MAPPING_KERNEL, v);

■         mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 37) - [public] mapping_unmap_range

```
■         mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

    return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                       virt + i * ARCH_PAGE_SIZE,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 39) - [public] if

```
■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

    return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                       virt + i * ARCH_PAGE_SIZE,
■                       phys,
■                       ARCH_PAGE_SIZE,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 40) - [public] pmm_free_frame

```
■         pmm_free_frame(p);
■     }
■ }

    return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                       virt + i * ARCH_PAGE_SIZE,
■                       phys,
■                       ARCH_PAGE_SIZE,
■                       PTE_PRESENT | PTE_WRITABLE | PTE_NX)) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 52) - [public] pmm_free_frame

```
■      pmm_free_frame(phys);

■      for (size_t j = 0; j < i; j++) {
■          uint64_t v = virt + j * ARCH_PAGE_SIZE;
■          uint64_t p = mapping_translate(MAPPING_KERNEL, v);

■          mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■          if (p != UINT64_MAX) {
■              pmm_free_frame(p);
■          }
■      }
      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 58) - [public] mapping_unmap_range

```
■      mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■      if (p != UINT64_MAX) {
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 60) - [public] if

```
■      if (p != UINT64_MAX) {
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
}

      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 61) - [public] pmm_free_frame

```
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
}

static void heap_unmap_cb(uint64_t virt, size_t size)
      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 80) - [public] if

```
■ if (p != UINT64_MAX) {
■     mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 81) - [public] mapping_unmap_range

```
■     mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
■    p[i] = value;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 82) - [public] pmm_free_frame

```
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
■    p[i] = value;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 147) - [public] slab_alloc

```
■     return slab_alloc(size);
■}
#endif
```

```
■return buddy_alloc(size);
}
```

```
void *heap_alloc_aligned(size_t size, size_t align)
{
■if (!heap_initialized || size == 0) {
■    return NULL;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 151) - [public] buddy_alloc

```
    return buddy_alloc(size);
}

void *heap_alloc_aligned(size_t size, size_t align)
{
    if (!heap_initialized || size == 0) {
        return NULL;
    }

    if (!size_is_pow2(align)) {
        return NULL;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 167) - [public] slab_alloc

```
    return slab_alloc(size);
}
#endif

return buddy_alloc_aligned(size, align);
}

void *heap_zalloc(size_t size)
{
    void *ptr = heap_alloc(size);

    if (ptr == NULL) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 171) - [public] buddy_alloc_aligned

```
return buddy_alloc_aligned(size, align);
}

void *heap_zalloc(size_t size)
{
    void *ptr = heap_alloc(size);

    if (ptr == NULL) {
        return NULL;
    }

    heap_memset(ptr, 0, size);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 197) - [public] slab_free

```
    slab_free(ptr);
    return;
}
#endif

if (heap_in_buddy(va)) {
    buddy_free(ptr);
}

size_t heap_usable_size(void *ptr)
{
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 203) - [public] buddy_free

```
    buddy_free(ptr);
}

size_t heap_usable_size(void *ptr)
{
    if (!heap_initialized || ptr == NULL) {
        return 0;
    }

    uint64_t va = (uint64_t)(uintptr_t)ptr;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 217) - [public] slab_usable_size

```
    return slab_usable_size(ptr);
}
#endif

if (heap_in_buddy(va)) {
    return buddy_block_size(ptr);
}

return 0;
}

void heap_dump(void)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 222) - [public] buddy_block_size

```
    return buddy_block_size(ptr);
}

return 0;
}

void heap_dump(void)
{
    if (!heap_initialized) {
        return;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 238) - [public] slab_dump

```
    slab_dump();
}
#endif
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 9) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define SLAB_CACHE_COUNT 8

static const size_t slab_classes[SLAB_CACHE_COUNT] = {
    16, 32, 64, 128, 256, 512, 1024, 2048
};

#define SLAB_MAX_OBJECTS_PER_SLAB 256
#define SLAB_BITMAP_WORDS (SLAB_MAX_OBJECTS_PER_SLAB / 32)

typedef struct slab_desc {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 92) - [public] if

```
■   if ((uint32_t)*p == page) {
■       *p = slab_descs[*p].next_partial;
■       return;
■   }

■   p = &slab_descs[*p].next_partial;
■ }
}

static bool slab_grow(uint32_t cache_id)
{
■ slab_cache_t *c = &slab_caches[cache_id];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 116) - [public] bitmap_free

```
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                         va,
■                         phys,
■                         ARCH_PAGE_SIZE,
■                         PTE_PRESENT | PTE_WRITABLE | PTE_NX)) {
■     pmm_free_frame(phys);
■     bitmap_free(&slab_page_bitmap, page);
■     return false;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 125) - [public] pmm_free_frame

```
■   pmm_free_frame(phys);
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ slab_desc_t *d = &slab_descs[page];

■ d->cache_id = cache_id;
■ d->state = 1;
■ d->free_count = c->objects_per_slab;
■ d->free_head = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 126) - [public] bitmap_free

```
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ slab_desc_t *d = &slab_descs[page];

■ d->cache_id = cache_id;
■ d->state = 1;
■ d->free_count = c->objects_per_slab;
■ d->free_head = 0;

    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 207) - [public] slab_bitmap_clear_all

```
■    slab_bitmap_clear_all(&slab_descs[i]);
■}

■for (size_t i = 0; i < SLAB_CACHE_COUNT; i++) {
■    slab_caches[i].object_size = slab_classes[i];
■    slab_caches[i].objects_per_slab =
■        (uint32_t)(ARCH_PAGE_SIZE / slab_classes[i]);
■    slab_caches[i].partial_head = -1;
■    slab_caches[i].total_slabs = 0;

■    if (slab_caches[i].objects_per_slab > SLAB_MAX_OBJECTS_PER_SLAB) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 218) - [public] if

```
■    if (slab_caches[i].objects_per_slab > SLAB_MAX_OBJECTS_PER_SLAB) {
■        return false;
■    }
■}
```

```
■slab_initialized = true;
```

```
■return true;
■}
```

```
bool slab_ready(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 255) - [public] if

```
■    if (!slab_grow(cache_id)) {
■        slab_unlock();
■        return NULL;
■    }
■}
```

```
■uint32_t page = (uint32_t)c->partial_head;
■slab_desc_t *d = &slab_descs[page];
```

```
■uint32_t idx = d->free_head;
```

```
■uint64_t va = slab_page_va(page);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 256) - [public] slab_unlock

```
■    slab_unlock();
■    return NULL;
■    }
■}
```

```
■uint32_t page = (uint32_t)c->partial_head;
■slab_desc_t *d = &slab_descs[page];
```

```
■uint32_t idx = d->free_head;
```

```
■uint64_t va = slab_page_va(page);
■uint8_t *obj =
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 328) - [public] slab_unlock

```
■    slab_unlock();
■    return;
■}

■slab_bit_clear(d, idx);

■bool was_full = (d->free_count == 0);

■*(uint32_t *) (uintptr_t)va = d->free_head;
■d->free_head = idx;
■d->free_count++;

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 346) - [public] partial_remove

```
■    partial_remove(c, page);

■    uint64_t phys = mapping_translate(MAPPING_KERNEL, slab_va);

■    mapping_unmap_range(MAPPING_KERNEL, slab_va, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 350) - [public] mapping_unmap_range

```
■    mapping_unmap_range(MAPPING_KERNEL, slab_va, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 352) - [public] if

```
■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 353) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 356) - [public] bitmap_free

```
■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

size_t slab_usable_size(void *ptr)
{
■if (!slab_initialized || ptr == NULL) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 400) - [public] if

```
■    if (d->state != 1) {
■        continue;
■    }

■    const slab_cache_t *c = &slab_caches[d->cache_id];

■    used += (size_t)(c->objects_per_slab - d->free_count) *
■        c->object_size;
■}

■return used;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 424) - [public] if

```
■    if (d->state != 1) {
■        continue;
■    }

■    const slab_cache_t *c = &slab_caches[d->cache_id];

■    free_bytes += (size_t)d->free_count * c->object_size;
■}

■return free_bytes;
}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 439) - [public] kprintf

```
■ kprintf("SLAB: not initialized\n");
■ return;
■}

■kprintf("SLAB:\n");

■for (size_t i = 0; i < SLAB_CACHE_COUNT; i++) {
■ kprintf(" size %4u: slabs %u\n",
■         (unsigned int)slab_caches[i].object_size,
■         (unsigned int)slab_caches[i].total_slabs);
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 65) - [public]
bitmap_words_for_bits

```
■return bitmap_words_for_bits(bit_count) * sizeof(uint64_t);
}

static void bitmap_mark_tail_used(bitmap_t *bm)
{
■if (bm->word_count == 0) {
■ return;
■}

■size_t rem = bm->bit_count & 63;

■if (rem == 0) {
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 132) - [public]
bitmap_mark_tail_used

```
■ bitmap_mark_tail_used(bm);
■}
}

void bitmap_set(bitmap_t *bm, size_t bit)
{
■if (bm == NULL || bm->bits == NULL || bit >= bm->bit_count) {
■ return;
■}

■size_t word = bit / 64;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 298) - [public] if

```
■ if (bm->bits[i] != 0) {
■ return false;
■ }
■}

■if ((bm->bits[last_word] & last_mask) != 0) {
■ return false;
■}

■return true;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 329) - [public] if

```
■   if (idx < bm->bit_count) {
■       return idx;
■   }

■   return BITMAP_INVALID;
■}

■for (size_t i = word + 1; i < bm->word_count; i++) {
■   w = bm->bits[i];

■   if (~w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(~w);
■       // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 339) - [public] if

```
■   if (~w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(~w);
■       size_t idx = i * 64 + found_bit;

■       if (idx < bm->bit_count) {
■           return idx;
■       }

■       return BITMAP_INVALID;
■   }
■}

■   // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 343) - [public] if

```
■       if (idx < bm->bit_count) {
■           return idx;
■       }

■       return BITMAP_INVALID;
■   }
■}

■return BITMAP_INVALID;
■}

static size_t find_next_used_bit(const bitmap_t *bm, size_t start, size_t limit)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 377) - [public] if

```
■   if (i == start_word && start_bit != 0) {
■       w &= ~lower_bits_mask(start_bit);
■   }

■   if (i == end_word) {
■       w &= mask_to_bit(end_bit);
■   }

■   if (w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(w);
■       size_t idx = i * 64 + found_bit;

■       // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 381) - [public] if

```
■ if (i == end_word) {
■     w &= mask_to_bit(end_bit);
■ }

■ if (w != 0) {
■     size_t found_bit = (size_t)__builtin_ctzll(w);
■     size_t idx = i * 64 + found_bit;

■     if (idx >= start && idx < limit) {
■         return idx;
■     }
■ }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 385) - [public] if

```
■ if (w != 0) {
■     size_t found_bit = (size_t)__builtin_ctzll(w);
■     size_t idx = i * 64 + found_bit;

■     if (idx >= start && idx < limit) {
■         return idx;
■     }
■ }

■ return BITMAP_INVALID;
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 389) - [public] if

```
■ if (idx >= start && idx < limit) {
■     return idx;
■ }

■ }

■ return BITMAP_INVALID;
}

size_t bitmap_alloc(bitmap_t *bm)
{
■ if (bm == NULL || bm->bits == NULL || bm->bit_count == 0) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 474) - [public] bitmap_alloc

```
■ return bitmap_alloc(bm);
■ }

■ size_t candidate = 0;

■ while (candidate <= bm->bit_count - count) {
■     candidate = align_up_size(candidate, align);

■     if (candidate == BITMAP_INVALID ||
■         candidate > bm->bit_count - count) {
■         break;
■     }
■ }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 489) - [public] if

```
■   if (free_bit == BITMAP_INVALID) {
■       break;
■   }

■   candidate = align_up_size(free_bit, align);

■   if (candidate == BITMAP_INVALID ||
■       candidate > bm->bit_count - count) {
■       break;
■   }

■   if (bitmap_test_range_free(bm, candidate, count)) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 500) - [public] if

```
■   if (bitmap_test_range_free(bm, candidate, count)) {
■       bitmap_set_range(bm, candidate, count);
■       bm->alloc_hint = candidate / 64;
■       return candidate;
■   }

■   size_t used = find_next_used_bit(bm, candidate, candidate + count);

■   if (used == BITMAP_INVALID) {
■       candidate++;
■   } else {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 501) - [public] bitmap_set_range

```
■       bitmap_set_range(bm, candidate, count);
■       bm->alloc_hint = candidate / 64;
■       return candidate;
■   }

■   size_t used = find_next_used_bit(bm, candidate, candidate + count);

■   if (used == BITMAP_INVALID) {
■       candidate++;
■   } else {
■       candidate = used + 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 509) - [public] if

```
■   if (used == BITMAP_INVALID) {
■       candidate++;
■   } else {
■       candidate = used + 1;
■   }
■}

■return BITMAP_INVALID;
}

void bitmap_free(bitmap_t *bm, size_t bit)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 64) - [public] arch_phys_to_virt

```
■return arch_phys_to_virt(frame);  
}
```

```
bool frame_is_valid(frame_t frame)  
{  
■if (!frame_initialized) {  
■    return false;  
■}
```

```
■if (!arch_is_page_aligned(frame)) {  
■    return false;  
■}  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 159) - [public] if

```
■    if (r->type != ARCH_MEM_TYPE_USABLE) {  
■        continue;  
■    }  
  
■    if (!arch_is_page_aligned(r->base) ||  
■        !arch_is_page_aligned(r->len)) {  
■        continue;  
■    }
```

```
■    size_t pfn_start = (size_t)(r->base / ARCH_PAGE_SIZE);  
■    size_t frame_count = (size_t)(r->len / ARCH_PAGE_SIZE);  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 171) - [public] if

```
■    if (frame_count == 0) {  
■        continue;  
■    }  
  
■    bitmap_clear_range(&frame_bitmap, pfn_start, frame_count);  
  
■    if (allocated_frames >= frame_count) {  
■        allocated_frames -= frame_count;  
■    } else {  
■        allocated_frames = 0;  
■    }  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 175) - [public] bitmap_clear_range

```
■    bitmap_clear_range(&frame_bitmap, pfn_start, frame_count);  
  
■    if (allocated_frames >= frame_count) {  
■        allocated_frames -= frame_count;  
■    } else {  
■        allocated_frames = 0;  
■    }  
  
■    dma32_account_free(pfn_start, frame_count);  
■}
```

```
■return true;  
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 177) - [public] if

```
■    if (allocated_frames >= frame_count) {
■        allocated_frames -= frame_count;
■    } else {
■        allocated_frames = 0;
■    }

■    dma32_account_free(pfn_start, frame_count);
■}

■return true;
}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 183) - [public] dma32_account_free

```
■    dma32_account_free(pfn_start, frame_count);
■}

■return true;
}

bool frame_ready(void)
{
■return frame_initialized;
}

bool frame_alloc(frame_t *out)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 230) - [public] frame_free

```
■    frame_free(frame);
■    return false;
■}

■*out = frame;

■return true;
}

bool frame_alloc_contiguous(size_t count,
■                        size_t align_frames,
■                        frame_t *out)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 345) - [public] volatile

```
■__asm__ volatile("" ::: "memory");

■return true;
}

bool frame_alloc_below(size_t count,
■                        size_t align_frames,
■                        uint64_t max_phys,
■                        frame_t *out)
{
■if (!frame_initialized || count == 0 || out == NULL) {
■    return false;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 377) - [public] if

```
■   if (aligned > limit - count) {  
■       break;  
■   }  
  
■   if (bitmap_test_range_free(&frame_bitmap, aligned, count)) {  
■       bitmap_set_range(&frame_bitmap, aligned, count);  
  
■       allocated_frames += count;  
■       dma32_account_alloc(aligned, count);  
  
■       *out = frame_from_pfn(aligned);  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 381) - [public] if

```
■   if (bitmap_test_range_free(&frame_bitmap, aligned, count)) {  
■       bitmap_set_range(&frame_bitmap, aligned, count);  
  
■       allocated_frames += count;  
■       dma32_account_alloc(aligned, count);  
  
■       *out = frame_from_pfn(aligned);  
  
■       return true;  
■   }  
  
■   pfn = aligned + 1;  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 382) - [public] bitmap_set_range

```
■       bitmap_set_range(&frame_bitmap, aligned, count);  
  
■       allocated_frames += count;  
■       dma32_account_alloc(aligned, count);  
  
■       *out = frame_from_pfn(aligned);  
  
■       return true;  
■   }  
  
■   pfn = aligned + 1;  
■ }  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 385) - [public]
dma32_account_alloc

```
■       dma32_account_alloc(aligned, count);  
  
■       *out = frame_from_pfn(aligned);  
  
■       return true;  
■   }  
  
■   pfn = aligned + 1;  
■ }  
  
■ return false;  
}  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 6) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void pmm_panic(const char *msg) __attribute__((noreturn));
static void pmm_panic(const char *msg)
{
    kprintf("pmm panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 8) - [public] pmm_panic

```
static void pmm_panic(const char *msg) __attribute__((noreturn));
static void pmm_panic(const char *msg)
{
    kprintf("pmm panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

    static bool pmm_initialized = false;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 31) - [public] pmm_panic

```
    pmm_panic("pmm_unlock without lock");
}

#define PMM_SIZE_MAX ((size_t)-1)

static size_t pmm_bytes_to_frames(size_t bytes)
{
    if (bytes == 0) {
        return 0;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 80) - [public] if

```
    if (!frame_zero(phys)) {
        return false;
    }

    return true;
}

bool pmm_init(void)
{
    pmm_lock();

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 93) - [public] pmm_unlock

```
■ pmm_unlock();
■ return true;
■}

■if (!arch_memory_ready()) {
■ pmm_unlock();
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 98) - [public] pmm_unlock

```
■ pmm_unlock();
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 99) - [public] kprintf

```
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;

■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 104) - [public] pmm_unlock

```
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;

■pmm_unlock();

■return true;
}

■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 105) - [public] kprintf

```
■    kprintf("pmm_init: frame allocator initialization failed\n");  
■    return false;  
■}
```

```
■pmm_initialized = true;
```

```
■pmm_unlock();
```

```
■return true;
```

```
}
```

```
bool pmm_ready(void)  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 220) - [public] pmm_alloc_frames

```
■return pmm_alloc_frames(frames, out_phys);  
}
```

```
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys)
```

```
{
```

```
■if (!pmm_initialized || out_phys == NULL || bytes == 0) {
```

```
■    return false;
```

```
■}
```

```
■size_t frames = pmm_bytes_to_frames(bytes);
```

```
■if (frames == 0) {
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 236) - [public] pmm_alloc_zero_frame

```
■    return pmm_alloc_zero_frame(out_phys);
```

```
■}
```

```
■pmm_lock();
```

```
■frame_t frame = FRAME_INVALID;
```

```
■bool ok = frame_alloc_contiguous(frames, 1, &frame);
```

```
■pmm_unlock();
```

```
■if (!ok) {
```

```
■    return false;
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 251) - [public] pmm_free_frames

```
■    pmm_free_frames(frame, frames);
```

```
■    return false;
```

```
■}
```

```
■*out_phys = frame;
```

```
■return true;
```

```
}
```

```
bool pmm_alloc_contiguous_bytes(size_t bytes,
```

```
■    size_t align_bytes,
```

```
■    uint64_t *out_phys)
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 275) - [public]

pmm_alloc_frames_aligned

```
■return pmm_alloc_frames_aligned(frames, align_frames, out_phys);
}
```

```
bool pmm_free_frame(uint64_t phys)
```

```
{
■if (!pmm_initialized) {
■    return false;
■}
}
```

```
■pmm_lock();
```

```
■bool ok = frame_free(phys);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 328) - [public] pmm_free_frames

```
■return pmm_free_frames(phys, frames);
}
```

```
size_t pmm_stat_total_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_total();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 337) - [public] frame_total

```
■return frame_total();
}
```

```
size_t pmm_stat_free_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_free_count();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 346) - [public] frame_free_count

```
■return frame_free_count();
}
```

```
size_t pmm_stat_allocated_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_allocated();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 355) - [public] frame_allocated

```
■return frame_allocated();
}

uint64_t pmm_stat_total_bytes(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■uint64_t frames = frame_total();

■if (frames > UINT64_MAX / ARCH_PAGE_SIZE) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 394) - [public]

frame_zone_dma32_total

```
■return frame_zone_dma32_total();
}

size_t pmm_stat_dma32_free_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_dma32_free();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 403) - [public]

frame_zone_dma32_free

```
■return frame_zone_dma32_free();
}

size_t pmm_stat_normal_total_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_normal_total();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 412) - [public]

frame_zone_normal_total

```
■return frame_zone_normal_total();
}

size_t pmm_stat_normal_free_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_normal_free();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 421) - [public]

frame_zone_normal_free

```
    return frame_zone_normal_free();
}

bool pmm_alloc_frames_below(size_t count,
    size_t align_frames,
    uint64_t max_phys,
    uint64_t *out_phys)
{
    if (!pmm_initialized || out_phys == NULL || count == 0) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 452) - [public] kprintf

```
    kprintf("PMM: not initialized\n");
    return;
}

uint64_t total_frames = pmm_stat_total_frames();
uint64_t free_frames = pmm_stat_free_frames();
uint64_t allocated_frames = pmm_stat_allocated_frames();

uint64_t total_mb = pmm_stat_total_bytes() >> 20;
uint64_t free_mb = pmm_stat_free_bytes() >> 20;

kprintf("PMM:\n");
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 490) - [public] if

```
    if (!pmm_alloc_zero_frame(&frames[i])) {
        kprintf("pmm_self_test: single frame alloc failed\n");
        return false;
    }

    if (!arch_is_page_aligned(frames[i])) {
        kprintf("pmm_self_test: frame not aligned\n");
        return false;
    }

    volatile uint64_t *p =
        (volatile uint64_t *)arch_phys_to_virt(frames[i]);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 491) - [public] kprintf

```
    kprintf("pmm_self_test: single frame alloc failed\n");
    return false;
}

    if (!arch_is_page_aligned(frames[i])) {
        kprintf("pmm_self_test: frame not aligned\n");
        return false;
    }

    volatile uint64_t *p =
        (volatile uint64_t *)arch_phys_to_virt(frames[i]);

    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 495) - [public] if

```
■   if (!arch_is_page_aligned(frames[i])) {
■       kprintf("pmm_self_test: frame not aligned\n");
■       return false;
■   }

■   volatile uint64_t *p =
■       (volatile uint64_t *)arch_phys_to_virt(frames[i]);

■   for (size_t j = 0; j < (ARCH_PAGE_SIZE / sizeof(uint64_t)); j++) {
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 496) - [public] kprintf

```
■       kprintf("pmm_self_test: frame not aligned\n");
■       return false;
■   }

■   volatile uint64_t *p =
■       (volatile uint64_t *)arch_phys_to_virt(frames[i]);

■   for (size_t j = 0; j < (ARCH_PAGE_SIZE / sizeof(uint64_t)); j++) {
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 504) - [public] if

```
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■ }

■ for (size_t i = 0; i < 8; i++) {
■     if (!pmm_free_frame(frames[i])) {
■         kprintf("pmm_self_test: single frame free failed\n");
■         return false;
■     }
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 505) - [public] kprintf

```
■       kprintf("pmm_self_test: zero frame not zero\n");
■       return false;
■   }
■ }

■ for (size_t i = 0; i < 8; i++) {
■     if (!pmm_free_frame(frames[i])) {
■         kprintf("pmm_self_test: single frame free failed\n");
■         return false;
■     }
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 512) - [public] if

```
■    if (!pmm_free_frame(frames[i])) {  
■        kprintf("pmm_self_test: single frame free failed\n");  
■        return false;  
■    }  
■}
```

```
■uint64_t contiguous = PMM_INVALID_FRAME;
```

```
■if (!pmm_alloc_frames(16, &contiguous)) {  
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 513) - [public] kprintf

```
■        kprintf("pmm_self_test: single frame free failed\n");  
■        return false;  
■    }  
■}
```

```
■uint64_t contiguous = PMM_INVALID_FRAME;
```

```
■if (!pmm_alloc_frames(16, &contiguous)) {  
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■}  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 522) - [public] kprintf

```
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■}
```

```
■if (!arch_is_page_aligned(contiguous)) {  
■    kprintf("pmm_self_test: contiguous not aligned\n");  
■    return false;  
■}
```

```
■if (!pmm_free_frames(contiguous, 16)) {  
■    kprintf("pmm_self_test: contiguous free failed\n");  
■    return false;  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 527) - [public] kprintf

```
■    kprintf("pmm_self_test: contiguous not aligned\n");  
■    return false;  
■}
```

```
■if (!pmm_free_frames(contiguous, 16)) {  
■    kprintf("pmm_self_test: contiguous free failed\n");  
■    return false;  
■}
```

```
■uint64_t aligned2m = PMM_INVALID_FRAME;
```

```
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 532) - [public] kprintf

```
■ kprintf("pmm_self_test: contiguous free failed\n");
■ return false;
■ }
```

```
■ uint64_t aligned2m = PMM_INVALID_FRAME;
```

```
■ if (!pmm_alloc_frames_aligned(512, 512, &aligned2m)) {
■ kprintf("pmm_self_test: 2M aligned alloc failed\n");
■ return false;
■ }
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 540) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M aligned alloc failed\n");
■ return false;
■ }
```

```
■ uint64_t align2m_mask = (512ULL * ARCH_PAGE_SIZE) - 1;
```

```
■ if ((aligned2m & align2m_mask) != 0) {
■ kprintf("pmm_self_test: 2M alignment failed\n");
■ return false;
■ }
```

```
■ if (!pmm_free_frames(aligned2m, 512)) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 547) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M alignment failed\n");
■ return false;
■ }
```

```
■ if (!pmm_free_frames(aligned2m, 512)) {
■ kprintf("pmm_self_test: 2M aligned free failed\n");
■ return false;
■ }
```

```
■ uint64_t bytes_alloc = PMM_INVALID_FRAME;
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 552) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M aligned free failed\n");
■ return false;
■ }
```

```
■ uint64_t bytes_alloc = PMM_INVALID_FRAME;
```

```
■ if (!pmm_alloc_zero_bytes(10000, &bytes_alloc)) {
■ kprintf("pmm_self_test: bytes alloc failed\n");
■ return false;
■ }
```

```
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 560) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes alloc failed\n");
    ■ return false;
    ■}

    ■size_t bytes_frames = pmm_bytes_to_frames(10000);

    ■volatile uint64_t *p =
    ■ (volatile uint64_t *)arch_phys_to_virt(bytes_alloc);

    ■size_t words = bytes_frames * (ARCH_PAGE_SIZE / sizeof(uint64_t));

    ■for (size_t i = 0; i < words; i++) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 572) - [public] if
    ■ if (p[i] != 0) {
    ■ kprintf("pmm_self_test: bytes alloc not zero\n");
    ■ return false;
    ■ }
    ■}

    ■if (!pmm_free_bytes(bytes_alloc, 10000)) {
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 573) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes alloc not zero\n");
    ■ return false;
    ■ }
    ■}

    ■if (!pmm_free_bytes(bytes_alloc, 10000)) {
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 579) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");

    ■return true;
    }

    #endif

```

kernel_workspace/kernel/src/mm/alloc/special/contiguous.c (line 9) - [public]

size_bytes_to_pages_checked

```
■return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
}

bool contig_alloc(size_t bytes,
■          size_t align,
■          uint64_t *out_phys,
■          void **out_virt)
{
■if (bytes == 0) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 10) - [public]

size_bytes_to_pages_checked

```
■return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
}

bool dma_alloc_coherent(size_t bytes,
■          uint64_t zone_max,
■          uint64_t *out_phys,
■          void **out_virt)
{
■if (bytes == 0 || out_phys == NULL || out_virt == NULL) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 47) - [public] pmm_free_frames

```
■    pmm_free_frames(phys, frames);
■    return false;
■}

■*out_phys = phys;
■*out_virt = (void *)(uintptr_t)virt;

■return true;
}

void dma_free_coherent(uint64_t phys, void *virt, size_t bytes)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 76) - [public] vmm_unmap_device

```
■    vmm_unmap_device((uint64_t)(uintptr_t)virt, len);
■}

■pmm_free_frames(phys, frames);
}

static void clflush_range(void *addr, size_t len)
{
■uintptr_t start = (uintptr_t)addr & ~(uintptr_t)63;
■uintptr_t end = (uintptr_t)addr + len;

■for (uintptr_t a = start; a < end; a += 64) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 88) - [public] volatile

```
■ __asm__ volatile("clflush (%0)" :: "r"(a) : "memory");
■}

■__asm__ volatile("" ::: "memory");
■}

void dma_sync_for_device(void *virt, size_t len)
{
■if (virt == NULL || len == 0) {
■    return;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 91) - [public] volatile

```
■__asm__ volatile("" ::: "memory");
■}

void dma_sync_for_device(void *virt, size_t len)
{
■if (virt == NULL || len == 0) {
■    return;
■}

■clflush_range(virt, len);
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 27) - [public] mapping_init

```
■    mapping_init();
■}

■return mapping_kernel_pml4;
■}

static bool page_range_ok(uint64_t addr, size_t len)
{
■uint64_t start, end;

■if (!range_from_addr_len(addr, (uint64_t)len, ARCH_PAGE_SIZE,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 70) - [public] if

```
■    if (!paging_map_page_in(pml4, v, p, pte_flags)) {
■        for (size_t j = 0; j < mapped; j++) {
■            paging_unmap_page_in(pml4,
■                                virt + (uint64_t)j * ARCH_PAGE_SIZE);
■        }

■        return false;
■    }

■    mapped++;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 102) - [public]
paging_unmap_page_in

```
    paging_unmap_page_in(pml4, virt + (uint64_t)i * ARCH_PAGE_SIZE);
}

return true;
}

bool mapping_protect_range(mapping_space_t space,
    uint64_t virt,
    size_t len,
    uint64_t pte_flags)
{
    if (len == 0) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 128) - [public] if

```
    if (!paging_is_mapped_in(pml4, v)) {
        return false;
    }

    for (size_t i = 0; i < pages; i++) {
        uint64_t v = virt + (uint64_t)i * ARCH_PAGE_SIZE;

        if (!paging_set_flags_in(pml4, v, pte_flags)) {

            return false;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 136) - [public] if

```
    if (!paging_set_flags_in(pml4, v, pte_flags)) {

        return false;
    }

    return true;
}

uint64_t mapping_translate(mapping_space_t space, uint64_t virt)
{
    return paging_translate_in(resolve_space(space), virt);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 147) - [public]
paging_translate_in

```
return paging_translate_in(resolve_space(space), virt);
}

bool mapping_is_mapped(mapping_space_t space, uint64_t virt)
{
    return paging_is_mapped_in(resolve_space(space), virt);
}

static void copy_page_by_phys(uint64_t dst_phys, uint64_t src_phys)
{
    uint64_t *dst = (uint64_t *)arch_phys_to_virt(dst_phys);
    const uint64_t *src = (const uint64_t *)arch_phys_to_virt(src_phys);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 152) - [public] paging_is_mapped_in

```
    return paging_is_mapped_in(resolve_space(space), virt);
}

static void copy_page_by_phys(uint64_t dst_phys, uint64_t src_phys)
{
    uint64_t *dst = (uint64_t *)arch_phys_to_virt(dst_phys);
    const uint64_t *src = (const uint64_t *)arch_phys_to_virt(src_phys);

    for (size_t i = 0; i < ARCH_PAGE_SIZE / sizeof(uint64_t); i++) {
        dst[i] = src[i];
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 164) - [public] volatile

```
    __asm__ volatile("" ::: "memory");
}

bool mapping_copy_range(mapping_space_t dst,
    uint64_t dst_virt,
    mapping_space_t src,
    uint64_t src_virt,
    size_t len)
{
    if (len == 0) {
        return true;
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 199) - [public] copy_page_by_phys

```
    copy_page_by_phys(dp, sp);
}

return true;
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 5) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static page_t *page_descs = NULL;
static size_t page_desc_count = 0;
static bool page_initialized = false;

static size_t page_lock_depth = 0;
static uint64_t page_lock_flags = 0;

static void page_lock(void)
{
    uint64_t flags;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 78) - [public] kprintf

```
    kprintf("page_init: pmm not ready\n");
    return false;
}

const arch_mem_info_t *info = arch_memory_get();

if (info == NULL) {
    kprintf("page_init: no arch mem info\n");
    return false;
}

uint64_t max_pfn = info->max_usable_address / ARCH_PAGE_SIZE;
// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 85) - [public] kprintf

```
■ kprintf("page_init: no arch mem info\n");
■ return false;
■}

■uint64_t max_pfn = info->max_usable_address / ARCH_PAGE_SIZE;

■page_desc_count = (size_t)(max_pfn + 1);

■size_t bytes = page_desc_count * sizeof(page_t);

■kprintf("page_init: max_usable=0x%llx max_addr=0x%llx desc_count=%llu bytes=%llu\n",
■ (unsigned long long)info->max_usable_address,
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 141) - [public] page_unlock

```
■ page_unlock();
■ return NULL;
■}

■page_t *p = &page_descs[page_phys_to_pfn(phys)];

■p->refcount = 1;
■p->type = type;
■p->flags = 0;
■p->reserved = 0;

■page_unlock();
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 168) - [public] page_unlock

```
■ page_unlock();
■ return NULL;
■}

■page_t *p = &page_descs[page_phys_to_pfn(phys)];

■p->refcount = 1;
■p->type = type;
■p->flags = 0;
■p->reserved = 0;

■page_unlock();
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 205) - [public] page_unlock

```
■ page_unlock();
■ return false;
■}

■p->refcount = 1;
■p->type = type;

■page_unlock();

■return true;
}

■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 226) - [public] page_unlock

```
■ page_unlock();
■ return false;
■ }
```

```
■ page->refcount++;
```

```
■ page_unlock();
```

```
■ return true;
}
```

```
bool page_put(page_t *page)
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 246) - [public] page_unlock

```
■ page_unlock();
■ return false;
■ }
```

```
■ page->refcount--;
```

```
■ if (page->refcount != 0) {
■   page_unlock();
■   return false;
■ }
```

```
■ size_t idx = (size_t)(page - page_descs);
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 253) - [public] page_unlock

```
■ page_unlock();
■ return false;
■ }
```

```
■ size_t idx = (size_t)(page - page_descs);
■ uint64_t phys = (uint64_t)idx * ARCH_PAGE_SIZE;
```

```
■ page->type = PAGE_TYPE_FREE;
■ page->flags = 0;
```

```
■ page_unlock();
```

```
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 289) - [public] arch_phys_to_virt

```
■ return arch_phys_to_virt(phys);
}
```

```
size_t page_pfn(const page_t *page)
```

```
{
■ if (!page_ptr_valid(page)) {
■   return (size_t)-1;
■ }
```

```
■ return (size_t)(page - page_descs);
}
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 331) - [public] page_from_phys

```
■return page_from_phys(arch_virt_to_phys(va));  
}
```

```
uint32_t page_refcount(const page_t *page)  
{  
■if (!page_ptr_valid(page)) {  
■    return 0;  
■}  
  
■return page->refcount;  
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 379) - [public] if

```
■    if (page_descs[i].refcount != 0) {  
■        count++;  
■    }  
■}
```

```
■return count;  
}
```

```
void page_dump(void)  
{  
■if (!page_initialized) {  
■    kprintf("PAGE: not initialized\n");  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 390) - [public] kprintf

```
■    kprintf("PAGE: not initialized\n");  
■    return;  
■}  
  
■kprintf("PAGE:\n");  
■kprintf("  total pages: %llu\n", (unsigned long long)page_stat_total());  
■kprintf("  referenced: %llu\n", (unsigned long long)page_stat_referenced());  
}
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 7) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);  
  
#define VMM_VMALLOC_BASE 0xFFFFFA0000000000ULL  
#define VMM_VMALLOC_SIZE (128ULL << 30)  
#define VMM_SIZE_MAX ((size_t)-1)  
  
static bitmap_t vmm_bitmap;  
static bool vmm_initialized = false;  
static size_t vmm_total_pages = 0;  
static size_t vmm_allocated_pages = 0;  
  
static size_t vmm_lock_depth = 0;  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 201) - [public] vmm_unlock

```
■    vmm_unlock();  
■    return false;  
■}  
  
■size_t mapped = 0;  
■bool ok = true;  
  
■for (size_t i = 0; i < pages; i++) {  
■    uint64_t phys = 0;  
  
■    bool got = (flags & VMM_FLAG_ZERO)  
■        ? pmm_alloc_zero_frame(&phys)  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 213) - [public] pmm_alloc_frame

```
■          : pmm_alloc_frame(&phys);

■  if (!got) {
■      ok = false;
■      break;
■  }

■  uint64_t virt = vmm_page_virt(start + i);

■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 215) - [public] if

```
■  if (!got) {
■      ok = false;
■      break;
■  }

■  uint64_t virt = vmm_page_virt(start + i);

■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 222) - [public] if

```
■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }

■  mapped++;
■}

■if (!ok) {
■  for (size_t i = 0; i < mapped; i++) {
■      uint64_t virt = vmm_page_virt(start + i);
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 223) - [public] pmm_free_frame

```
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }

■  mapped++;
■}

■if (!ok) {
■  for (size_t i = 0; i < mapped; i++) {
■      uint64_t virt = vmm_page_virt(start + i);

■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 235) - [public] if

```
■    if (paging_is_mapped(virt)) {
■        uint64_t phys = paging_translate(virt);
■        paging_unmap_page(virt);
■        pmm_free_frame(phys);
■    }

■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 237) - [public] paging_unmap_page

```
■        paging_unmap_page(virt);
■        pmm_free_frame(phys);
■    }

■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 238) - [public] pmm_free_frame

```
■        pmm_free_frame(phys);
■    }

■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 242) - [public] bitmap_free_range

```
■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 243) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 258) - [public] vmm_alloc_aligned

```
■return vmm_alloc_aligned(bytes, ARCH_PAGE_SIZE, flags, out_virt);
}

bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt)
{
■if (!vmm_initialized || out_virt == NULL || len == 0) {
■    return false;
■}

■if (!arch_is_page_aligned(phys) || !arch_is_page_aligned(len)) {
■    return false;
■}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 284) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■size_t mapped = 0;
■bool ok = true;

■for (size_t i = 0; i < pages; i++) {
■    uint64_t virt = vmm_page_virt(start + i);
■    uint64_t frame_phys = phys + (uint64_t)i * ARCH_PAGE_SIZE;

■    if (!paging_map_page(virt, frame_phys, pte)) {
■        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 295) - [public] if

```
■    if (!paging_map_page(virt, frame_phys, pte)) {
■        ok = false;
■        break;
■    }

■    mapped++;
■}

■if (!ok) {
■    for (size_t i = 0; i < mapped; i++) {
■        paging_unmap_page(vmm_page_virt(start + i));
■    }
■    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 305) - [public] paging_unmap_page

```
■    paging_unmap_page(vmm_page_virt(start + i));
■    }

■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 308) - [public] bitmap_free_range

```
■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 309) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 349) - [public] if

```
■    if (paging_is_mapped(v)) {
■        uint64_t phys = paging_translate(v);
■        paging_unmap_page(v);
■        pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 351) - [public] paging_unmap_page

```
■    paging_unmap_page(v);
■    pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 352) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 395) - [public] if

```
■    if (paging_is_mapped(v)) {
■        paging_unmap_page(v);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 396) - [public] paging_unmap_page

```
■    paging_unmap_page(v);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 419) - [public] paging_translate

```
■return paging_translate(virt);
}

size_t vmm_stat_total_pages(void)
{
■if (!vmm_initialized) {
■    return 0;
■}

■return vmm_total_pages;
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 466) - [public] kprintf

```
■    kprintf("VMM: not initialized\n");
■    return;
■}

■kprintf("VMM:\n");
■kprintf("  base: 0x%llx\n", (unsigned long long)VMM_VMALLOC_BASE);
■kprintf("  total pages: %llu\n", (unsigned long long)vmm_stat_total_pages());
■kprintf("  free pages: %llu\n", (unsigned long long)vmm_stat_free_pages());
■kprintf("  allocated pages: %llu\n", (unsigned long long)vmm_stat_allocated_pages());
■kprintf("  total: %llu MiB\n", (unsigned long long)(vmm_stat_total_bytes() >> 20));
■kprintf("  free: %llu MiB\n", (unsigned long long)(vmm_stat_free_bytes() >> 20));
}
```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 115) - [public]

arch_memory_boot_alloc

```
■    arch_memory_boot_alloc(4096, 4096, &phys);

■    uint64_t *t = tbl(phys);

■    for (int i = 0; i < 512; i++) {
■        t[i] = 0;
■    }
■}

■*entry = phys | HW_TABLE;

■return phys;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 139) - [public] tlb_flush_page_addr

```
■    tlb_flush_page_addr(virt);
■    return true;
■}

■uint64_t l2 = ensure_table(&l1t[IDX(virt, L1_SHIFT)], false);
■uint64_t *l2t = tbl(l2);

■uint64_t l3 = ensure_table(&l2t[IDX(virt, L2_SHIFT)], false);
■uint64_t *l3t = tbl(l3);

■l3t[IDX(virt, L3_SHIFT)] = (phys & PAGING_ADDR_MASK) | hw_bits(flags, 3);

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 172) - [public] tlb_flush_page_addr
    ■ tlb_flush_page_addr(virt);
    ■ return true;
    ■}

    ■uint64_t *l2t = tbl(l1t[IDX(virt, L1_SHIFT)] & PAGING_ADDR_MASK);

    ■if (!(l2t[IDX(virt, L2_SHIFT)] & HW_VALID)) {
    ■    return false;
    ■}

    ■if ((l2t[IDX(virt, L2_SHIFT)] & 3) == HW_BLOCK) {
    ■    l2t[IDX(virt, L2_SHIFT)] = 0;
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 184) - [public] tlb_flush_page_addr
    ■ tlb_flush_page_addr(virt);
    ■ return true;
    ■}

    ■uint64_t *l3t = tbl(l2t[IDX(virt, L2_SHIFT)] & PAGING_ADDR_MASK);

    ■if (!(l3t[IDX(virt, L3_SHIFT)] & HW_VALID)) {
    ■    return false;
    ■}

    ■l3t[IDX(virt, L3_SHIFT)] = 0;
    ■tlb_flush_page_addr(virt);
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 216) - [public] return
    ■    return (e1 & 0x0000FFFFFFFFE0000ULL) | (virt & 0x1FFFFFFULL);
    ■}

    ■uint64_t *l2t = tbl(e1 & PAGING_ADDR_MASK);
    ■uint64_t e2 = l2t[IDX(virt, L2_SHIFT)];

    ■if (!(e2 & HW_VALID)) {
    ■    return UINT64_MAX;
    ■}

    ■if ((e2 & 3) == HW_BLOCK) {
    ■    return (e2 & 0x0000FFFFFFFFF0000ULL) | (virt & 0xFFFFFULL);
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 227) - [public] return
    ■    return (e2 & 0x0000FFFFFFFFF0000ULL) | (virt & 0xFFFFFULL);
    ■}

    ■uint64_t *l3t = tbl(e2 & PAGING_ADDR_MASK);
    ■uint64_t e3 = l3t[IDX(virt, L3_SHIFT)];

    ■if (!(e3 & HW_VALID)) {
    ■    return UINT64_MAX;
    ■}

    ■return (e3 & PAGING_ADDR_MASK) | (virt & 0xFFFFFULL);
    }
    // ... (truncated, see source file for more)

```

```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 273) - [public] volatile
    ■__asm__ volatile("mrs %0, ttbr1_el1" : "=r"(cur));

    ■uint64_t *boot_l0 = tbl_early(cur & PAGING_ADDR_MASK);

    ■kernel_l0 = alloc_table_early();

    ■uint64_t *l0 = tbl_early(kernel_l0);

    ■for (int i = 0; i < 512; i++) {
    ■    l0[i] = boot_l0[i];
    ■}

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 297) - [public] if
    ■    if (idx == 0 && pa >= 0x200000 * 512) {
    ■        break;
    ■    }

    ■    l2t[idx] = pa | HW_BLOCK | HW_AF | HW_SH | ATTR_WB | HW_PXN;
    ■}

    ■uint64_t mair = 0x00ULL | (0xFFULL << 8) | (0x44ULL << 16);

    ■__asm__ volatile("msr mair_el1, %0" :: "r"(mair));

    ■uint64_t tcr = (16ULL << 0)
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 306) - [public] volatile
    ■__asm__ volatile("msr mair_el1, %0" :: "r"(mair));

    ■uint64_t tcr = (16ULL << 0)
    ■    | (16ULL << 16)
    ■    | (0b000ULL << 14)
    ■    | (0b100ULL << 30)
    ■    | (0b010ULL << 8)
    ■    | (0b010ULL << 10)
    ■    | (0b110ULL << 12)
    ■    | (0b010ULL << 24)
    ■    | (0b010ULL << 26)
    ■    | (0b110ULL << 28)
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 320) - [public] volatile
    ■__asm__ volatile("msr tcr_el1, %0" :: "r"(tcr));

    ■__asm__ volatile("msr ttbr0_el1, %0" :: "r"(kernel_l0));
    ■__asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    ■__asm__ volatile("isb");
    ■__asm__ volatile("tlbi vmalle1");
    ■__asm__ volatile("dsb sy");
    ■__asm__ volatile("isb");
    ■}

    bool paging_enable_nx(void)
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 322) - [public] volatile

```
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(kernel_l0));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 323) - [public] volatile

```
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 325) - [public] volatile

```
    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 326) - [public] volatile

```
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 327) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    uint64_t v;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 328) - [public] volatile

```
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    uint64_t v;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 340) - [public] volatile

```
    __asm__ volatile("mrs %0, ttbr1_el1" : "=r"(v));

    return v & PAGING_ADDR_MASK;
}

void paging_write_cr3(uint64_t pml4_phys)
{
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 347) - [public] volatile

```
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
}
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 348) - [public] volatile

```
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
}
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 349) - [public] volatile

```
    __asm__ volatile("isb");

    tlb_flush_all();
}
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 18) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 19) - [public] volatile

```
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 20) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    __asm__ volatile("dsb ishst");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 21) - [public] volatile

```
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    __asm__ volatile("dsb ishst");
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 31) - [public] volatile

```
    __asm__ volatile("dsb ishst");
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 32) - [public] volatile

```
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 33) - [public] volatile

```
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    if (pages > 32) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 34) - [public] volatile

```
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    if (pages > 32) {
        tlb_flush_all();
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/tlb.c (line 45) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■for (size_t i = 0; i < pages; i++) {
■ tlb_flush_page_addr(addr + i * 4096);
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/tlb.c (line 50) - [public] tlb_flush_page_addr

```
■ tlb_flush_page_addr(addr + i * 4096);
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
}
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 37) - [public] arch_mem_panic

```
■ arch_mem_panic("bad multiboot2 magic");
■}

■if (mb_info < 8) {
■ arch_mem_panic("bad multiboot2 info address");
■}

■static arch_raw_mem_entry_t raw[ARCH_MAX_MEM_REGIONS];
■size_t n = 0;

■uint32_t total_size = *(const uint32_t *)mb_info;

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 41) - [public] arch_mem_panic

```
■ arch_mem_panic("bad multiboot2 info address");
■}

■static arch_raw_mem_entry_t raw[ARCH_MAX_MEM_REGIONS];
■size_t n = 0;

■uint32_t total_size = *(const uint32_t *)mb_info;

■if (total_size < 8) {
■ arch_mem_panic("multiboot2 info too small");
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 50) - [public] arch_mem_panic

```
■ arch_mem_panic("multiboot2 info too small");
■}

■uintptr_t end_addr = mb_info + total_size;
■uintptr_t tag_addr = (mb_info + 8 + 7) & ~(uintptr_t)7;

■while (tag_addr + 8 <= end_addr) {
■ const struct multiboot_tag *tag =
■ (const struct multiboot_tag *)tag_addr;

■ if (tag->size < 8) {
■ break;
■ // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 60) - [public] if

```
■   if (tag->size < 8) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_END) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 64) - [public] if

```
■   if (tag->type == MULTIBOOT_TAG_TYPE_END) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        uintptr_t entry_addr = (uintptr_t)mmap->entries;
        uintptr_t mmap_end = tag_addr + tag->size;

■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 68) - [public] if

```
■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        uintptr_t entry_addr = (uintptr_t)mmap->entries;
        uintptr_t mmap_end = tag_addr + tag->size;

■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
■           break;
■       }

■       while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 75) - [public] if

```
■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
■           break;
■       }

■       while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
■           const struct multiboot_mmap_entry *e =
■               (const struct multiboot_mmap_entry *)entry_addr;

■           if (n < ARCH_MAX_MEM_REGIONS) {
■               raw[n].base = e->addr;
■               raw[n].len = e->len;
■               raw[n].type = e->type;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 79) - [public] while

```
■ while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
■     const struct multiboot_mmap_entry *e =
■         (const struct multiboot_mmap_entry *)entry_addr;

■     if (n < ARCH_MAX_MEM_REGIONS) {
■         raw[n].base = e->addr;
■         raw[n].len = e->len;
■         raw[n].type = e->type;
■         n++;
■     }

■     entry_addr += mmap->entry_size;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 83) - [public] if

```
■     if (n < ARCH_MAX_MEM_REGIONS) {
■         raw[n].base = e->addr;
■         raw[n].len = e->len;
■         raw[n].type = e->type;
■         n++;
■     }

■     entry_addr += mmap->entry_size;
■ }
■ }

■ tag_addr = (tag_addr + tag->size + 7) & ~(uintptr_t)7;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 106) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void arch_mem_panic(const char *msg) __attribute__((noreturn));
static void arch_mem_panic(const char *msg)
{
■ kprintf("arch/x86_64/memory.c panic: %s\n", msg);

■ for (;;) {
■     __asm__ volatile("cli; hlt");
■ }
}
#define ARCH_U64_MAX        UINT64_MAX
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 108) - [public] arch_mem_panic

```
static void arch_mem_panic(const char *msg) __attribute__((noreturn));
static void arch_mem_panic(const char *msg)
{
■ kprintf("arch/x86_64/memory.c panic: %s\n", msg);

■ for (;;) {
■     __asm__ volatile("cli; hlt");
■ }
}
#define ARCH_U64_MAX        UINT64_MAX
#define ARCH_TMP_REGIONS    (ARCH_MAX_MEM_REGIONS * 3)

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 221) - [public] while

```
■ while (j > 0 && entries[j - 1].base > key.base) {
■     entries[j] = entries[j - 1];
■     j--;
■ }

■ entries[j] = key;
■ }
■ }
```

```
static void map_add(arch_mem_region_t *list,
■     size_t *count,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 244) - [public] if

```
■ if (r.base < last_end) {
■     uint64_t delta = last_end - r.base;

■     if (r.len <= delta) {
■         return;
■     }

■     r.base = last_end;
■     r.len -= delta;
■ }

■
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 247) - [public] if

```
■ if (r.len <= delta) {
■     return;
■ }

■ r.base = last_end;
■ r.len -= delta;
■ }

■
■ if (safe_end(last->base, last->len) == r.base &&
■     last->type == r.type) {
■     last->len += r.len;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 264) - [public] arch_mem_panic

```
■ arch_mem_panic("too many memory regions");
■ }

■ list[(*count)++] = r;
■ }

static void map_reserve_range(uint64_t base, uint64_t len)
{
■ if (len == 0) {
■     return;
■ }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 293) - [public] if

```
■   if (r_end <= base || r.base >= end) {
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
■       continue;
■   }

■
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 294) - [public] map_add

```
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
■       continue;
■   }

■
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 299) - [public] if

```
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■   }

■
■   uint64_t ov_base = u64_max(r.base, base);
■   uint64_t ov_end = u64_min(r_end, end);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 305) - [public] map_add

```
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■   }

■
■   uint64_t ov_base = u64_max(r.base, base);
■   uint64_t ov_end = u64_min(r_end, end);

■   if (ov_end > ov_base) {
■       arch_mem_region_t reserved;
■       reserved.base = ov_base;
■       reserved.len = ov_end - ov_base;
■       reserved.type = ARCH_MEM_TYPE_RESERVED;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 312) - [public] if

```
■   if (ov_end > ov_base) {
■       arch_mem_region_t reserved;
■       reserved.base = ov_base;
■       reserved.len = ov_end - ov_base;
■       reserved.type = ARCH_MEM_TYPE_RESERVED;

■       map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, reserved);
■   }

■
■   if (r_end > end) {
■       arch_mem_region_t after;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 318) - [public] map_add

```
■       map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, reserved);
■   }

■
■   if (r_end > end) {
■       arch_mem_region_t after;
■       after.base = end;
■       after.len = r_end - end;
■       after.type = r.type;

■       map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■   }
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 322) - [public] if

```
■   if (r_end > end) {
■       arch_mem_region_t after;
■       after.base = end;
■       after.len = r_end - end;
■       after.type = r.type;

■       map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■   }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■   arch_mem_panic("memory reserve produced too many regions");
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 328) - [public] map_add

```
■       map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■   }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■   arch_mem_panic("memory reserve produced too many regions");
■}

■mem.count = tcount;

■for (size_t i = 0; i < tcount; i++) {
■   mem.regions[i] = tmp_regions[i];
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 333) - [public] arch_mem_panic

```
■ arch_mem_panic("memory reserve produced too many regions");
■ }

■ mem.count = tcount;

■ for (size_t i = 0; i < tcount; i++) {
■     mem.regions[i] = tmp_regions[i];
■ }
}

static void map_align_usable_regions(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 351) - [public] if

```
■ if (r.type != ARCH_MEM_TYPE_USABLE) {
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, r);
■     continue;
■ }

■ uint64_t aligned_base = align_up_page_safe(r.base);
■ uint64_t aligned_end = align_down_page(r_end);

■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 352) - [public] map_add

```
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, r);
■     continue;
■ }

■ uint64_t aligned_base = align_up_page_safe(r.base);
■ uint64_t aligned_end = align_down_page(r_end);

■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     tiny.type = ARCH_MEM_TYPE_RESERVED;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 359) - [public] if

```
■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     tiny.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, tiny);
■     continue;
■ }

■ if (r.base < aligned_base) {
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 365) - [public] map_add

```
■    map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, tiny);
■    continue;
■ }

■
■ if (r.base < aligned_base) {
■     arch_mem_region_t before;
■     before.base = r.base;
■     before.len = aligned_base - r.base;
■     before.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 370) - [public] if

```
■ if (r.base < aligned_base) {
■     arch_mem_region_t before;
■     before.base = r.base;
■     before.len = aligned_base - r.base;
■     before.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■ }

■
■ {
■     arch_mem_region_t usable;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 376) - [public] map_add

```
■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■ }

■
■ {
■     arch_mem_region_t usable;
■     usable.base = aligned_base;
■     usable.len = aligned_end - aligned_base;
■     usable.type = ARCH_MEM_TYPE_USABLE;

■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, usable);
■ }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 386) - [public] map_add

```
■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, usable);
■ }

■
■ if (aligned_end < r_end) {
■     arch_mem_region_t after;
■     after.base = aligned_end;
■     after.len = r_end - aligned_end;
■     after.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, after);
■ }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 390) - [public] if

```
■    if (aligned_end < r_end) {
■        arch_mem_region_t after;
■        after.base = aligned_end;
■        after.len = r_end - aligned_end;
■        after.type = ARCH_MEM_TYPE_RESERVED;

■        map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■    }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■    arch_mem_panic("memory alignment produced too many regions");
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 396) - [public] map_add

```
■        map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■    }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■    arch_mem_panic("memory alignment produced too many regions");
■}

■mem.count = tcount;

■for (size_t i = 0; i < tcount; i++) {
■    mem.regions[i] = tmp_regions[i];
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 401) - [public] arch_mem_panic

```
■    arch_mem_panic("memory alignment produced too many regions");
■}

■mem.count = tcount;

■for (size_t i = 0; i < tcount; i++) {
■    mem.regions[i] = tmp_regions[i];
■}
}

static void recalc_stats(void)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 421) - [public] if

```
■    if (end > max_address) {
■        max_address = end;
■    }

■    if (r->type == ARCH_MEM_TYPE_USABLE) {
■        if (total_usable > ARCH_U64_MAX - r->len) {
■            total_usable = ARCH_U64_MAX;
■        } else {
■            total_usable += r->len;
■        }
■    }

■    if (end > max_usable_address) {
■        // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 425) - [public] if

```
■ if (r->type == ARCH_MEM_TYPE_USABLE) {
■     if (total_usable > ARCH_U64_MAX - r->len) {
■         total_usable = ARCH_U64_MAX;
■     } else {
■         total_usable += r->len;
■     }

■     if (end > max_usable_address) {
■         max_usable_address = end;
■     }
■ }
■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 426) - [public] if

```
■ if (total_usable > ARCH_U64_MAX - r->len) {
■     total_usable = ARCH_U64_MAX;
■ } else {
■     total_usable += r->len;
■ }

■ if (end > max_usable_address) {
■     max_usable_address = end;
■ }
■ }
■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 432) - [public] if

```
■ if (end > max_usable_address) {
■     max_usable_address = end;
■ }
■ }
■ }

■ mem.total_usable = total_usable;
■ mem.max_address = max_address;
■ mem.max_usable_address = max_usable_address;
■ mem.direct_map_base = ARCH_DIRECT_MAP_BASE;
■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 452) - [public] arch_mem_panic

```
■ arch_mem_panic("arch_memory_init() called twice");
■ }

■ if (entries == NULL || count == 0) {
■     arch_mem_panic("empty memory map");
■ }

■ if (count > ARCH_MAX_MEM_REGIONS) {
■     arch_mem_panic("memory map too large");
■ }

■ mem.count = 0;
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 456) - [public] arch_mem_panic

```
■ arch_mem_panic("empty memory map");
■}

■if (count > ARCH_MAX_MEM_REGIONS) {
■ arch_mem_panic("memory map too large");
■}

■mem.count = 0;

■for (size_t i = 0; i < count; i++) {
■ raw_sorted[i] = entries[i];
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 460) - [public] arch_mem_panic

```
■ arch_mem_panic("memory map too large");
■}

■mem.count = 0;

■for (size_t i = 0; i < count; i++) {
■ raw_sorted[i] = entries[i];
■}

■sort_raw_entries(raw_sorted, count);

■for (size_t i = 0; i < count; i++) {
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 475) - [public] if

```
■ if (base >= end) {
■ continue;
■ }

■ if (mem.count > 0) {
■ const arch_mem_region_t *last = &mem.regions[mem.count - 1];
■ uint64_t last_end = safe_end(last->base, last->len);

■ if (base < last_end) {
■ base = last_end;
■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 479) - [public] if

```
■ if (mem.count > 0) {
■ const arch_mem_region_t *last = &mem.regions[mem.count - 1];
■ uint64_t last_end = safe_end(last->base, last->len);

■ if (base < last_end) {
■ base = last_end;
■ }

■ if (base >= end) {
■ continue;
■ }

■ }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 483) - [public] if

```
■      if (base < last_end) {
■          base = last_end;
■      }

■      if (base >= end) {
■          continue;
■      }
■  }

■      arch_mem_region_t r;
■      r.base = base;
■      r.len = end - base;
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 487) - [public] if

```
■      if (base >= end) {
■          continue;
■      }
■  }

■      arch_mem_region_t r;
■      r.base = base;
■      r.len = end - base;
■      r.type = raw_type_to_arch(raw_sorted[i].type);

■      map_add(mem.regions, &mem.count, ARCH_MAX_MEM_REGIONS, r);
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 497) - [public] map_add

```
■      map_add(mem.regions, &mem.count, ARCH_MAX_MEM_REGIONS, r);
■  }

■  if (mem.count == 0) {
■      arch_mem_panic("no valid memory regions after parsing");
■  }

■  map_reserve_range(0, 0x1000000);

■  if (kernel_phys_end > kernel_phys_start) {
■      map_reserve_range(kernel_phys_start,
■          // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 501) - [public] arch_mem_panic

```
■      arch_mem_panic("no valid memory regions after parsing");
■  }

■  map_reserve_range(0, 0x1000000);

■  if (kernel_phys_end > kernel_phys_start) {
■      map_reserve_range(kernel_phys_start,
■          kernel_phys_end - kernel_phys_start);
■  }

■  if (initrd_phys_end > initrd_phys_start) {
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 571) - [public] if

```
■ if (r->type != ARCH_MEM_TYPE_USABLE) {
■     continue;
■ }

■ uint64_t r_end = safe_end(r->base, r->len);

■ if (base >= r->base && end <= r_end) {
■     return true;
■ }
■}

■return false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 577) - [public] if

```
■ if (base >= r->base && end <= r_end) {
■     return true;
■ }
■}

■return false;
}

void arch_memory_reserve_range(uint64_t base, uint64_t len)
{
■map_reserve_range(base, len);
■recalc_stats();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 619) - [public] if

```
■ if (r->type != ARCH_MEM_TYPE_USABLE) {
■     continue;
■ }

■ uint64_t r_end = safe_end(r->base, r->len);
■ uint64_t candidate = align_up_generic(r->base, align);

■ if (candidate < r->base) {
■     continue;
■ }

■ if (candidate <= r_end && (r_end - candidate) >= len) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 626) - [public] if

```
■ if (candidate < r->base) {
■     continue;
■ }

■ if (candidate <= r_end && (r_end - candidate) >= len) {
■     *out_base = candidate;
■     return true;
■ }
■}

■return false;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 630) - [public] if

```
    if (candidate <= r_end && (r_end - candidate) >= len) {
        *out_base = candidate;
        return true;
    }

return false;
}
```

```
static const char *region_type_name(arch_mem_type_t type)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 6) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void paging_panic(const char *msg) __attribute__((noreturn));
static void paging_panic(const char *msg)
{
    kprintf("arch/x86_64/paging.c panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 8) - [public] paging_panic

```
static void paging_panic(const char *msg) __attribute__((noreturn));
static void paging_panic(const char *msg)
{
    kprintf("arch/x86_64/paging.c panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 11) - [public] kprintf

```
kprintf("arch/x86_64/paging.c panic: %s\n", msg);

for (;;) {
    __asm__ volatile("cli; hlt");
}

}
```

```
static uint64_t boot_phys_offset = 0;
static bool boot_phys_offset_valid = false;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 25) - [public] if

```
    if (!boot_phys_offset_valid) {
        paging_panic("boot physical offset not set");
    }

    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 26) - [public] paging_panic

```
    paging_panic("boot physical offset not set");
}

    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        p[i] = 0;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 29) - [public] return

```
    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        p[i] = 0;
    }

    __asm__ volatile("" ::: "memory");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 40) - [public] volatile

```
    __asm__ volatile("" ::: "memory");
}

static inline size_t pml4_index(uint64_t virt)
{
    return (size_t)((virt >> 39) & 0x1FF);
}

static inline size_t pdpt_index(uint64_t virt)
{
    return (size_t)((virt >> 30) & 0x1FF);
}

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 45) - [public] return
    return (size_t)((virt >> 39) & 0x1FF);
}

static inline size_t pdpt_index(uint64_t virt)
{
    return (size_t)((virt >> 30) & 0x1FF);
}

static inline size_t pd_index(uint64_t virt)
{
    return (size_t)((virt >> 21) & 0x1FF);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 50) - [public] return
    return (size_t)((virt >> 30) & 0x1FF);
}

static inline size_t pd_index(uint64_t virt)
{
    return (size_t)((virt >> 21) & 0x1FF);
}

static inline size_t pt_index(uint64_t virt)
{
    return (size_t)((virt >> 12) & 0x1FF);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 55) - [public] return
    return (size_t)((virt >> 21) & 0x1FF);
}

static inline size_t pt_index(uint64_t virt)
{
    return (size_t)((virt >> 12) & 0x1FF);
}

uint64_t paging_read_cr3(void)
{
    uint64_t cr3;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 60) - [public] return
    return (size_t)((virt >> 12) & 0x1FF);
}

uint64_t paging_read_cr3(void)
{
    uint64_t cr3;

    __asm__ volatile("mov %%cr3, %0" : "=r"(cr3));

    return cr3 & PAGING_ADDR_MASK;
}

// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 67) - [public] volatile

```
__asm__ volatile("mov %%cr3, %0" : "=r"(cr3));
```

```
return cr3 & PAGING_ADDR_MASK;
```

```
}
```

```
void paging_flush_tlb_all(void)
```

```
{
```

```
    tlb_flush_all();
```

```
}
```

```
void paging_flush_page(uint64_t addr)
```

```
{
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 74) - [public] tlb_flush_all

```
    tlb_flush_all();
```

```
}
```

```
void paging_flush_page(uint64_t addr)
```

```
{
```

```
    tlb_flush_page_addr(addr);
```

```
}
```

```
static uint64_t alloc_zeroed_table_page(void)
```

```
{
```

```
    uint64_t phys = 0;
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 79) - [public] tlb_flush_page_addr

```
    tlb_flush_page_addr(addr);
```

```
}
```

```
static uint64_t alloc_zeroed_table_page(void)
```

```
{
```

```
    uint64_t phys = 0;
```

```
/*
```

```
 * Once the physical frame allocator is up, page-table pages must come
```

```
 * from the PMM so that `free_tables`/`paging_aspace_destroy` can hand
```

```
 * them back with `pmm_free_frame` without corrupting the PMM's bookkeeping
```

```
 * (the boot allocator has no "free", so freeing a boot-allocated page
```

```
 * through the PMM double-frees it and later produces a MALFORMED_TABLE).
```

```
 * ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 93) - [public] if

```
/*
```

```
 * Once the physical frame allocator is up, page-table pages must come
```

```
 * from the PMM so that `free_tables`/`paging_aspace_destroy` can hand
```

```
 * them back with `pmm_free_frame` without corrupting the PMM's bookkeeping
```

```
 * (the boot allocator has no "free", so freeing a boot-allocated page
```

```
 * through the PMM double-frees it and later produces a MALFORMED_TABLE).
```

```
*/
```

```
if (pmm_ready()) {
```

```
    if (!pmm_alloc_zero_frame(&phys)) {
```

```
        paging_panic("cannot allocate page table page (pmm)");
```

```
    }
```

```
    return phys;
```

```
}
```

```
if (!arch_memory_boot_alloc(PAGING_PAGE_SIZE,
```

```
    PAGING_PAGE_SIZE,
```

```
    &phys)) {
```

```
    paging_panic("cannot allocate page table page");
```

```
}
```

```
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 94) - [public] if

```
    if (!pmm_alloc_zero_frame(&phys)) {
        paging_panic("cannot allocate page table page (pmm)");
    }
    return phys;
}

if (!arch_memory_boot_alloc(PAGING_PAGE_SIZE,
                            PAGING_PAGE_SIZE,
                            &phys)) {
    paging_panic("cannot allocate page table page");
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 95) - [public] paging_panic

```
    paging_panic("cannot allocate page table page (pmm)");
}
return phys;
}

if (!arch_memory_boot_alloc(PAGING_PAGE_SIZE,
                            PAGING_PAGE_SIZE,
                            &phys)) {
    paging_panic("cannot allocate page table page");
}

zero_page_phys(phys);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 103) - [public] paging_panic

```
    paging_panic("cannot allocate page table page");
}

zero_page_phys(phys);

return phys;
}

static uint64_t ensure_table_entry(uint64_t *table,
                                   size_t index,
                                   uint64_t flags)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 106) - [public] zero_page_phys

```
    zero_page_phys(phys);

    return phys;
}

static uint64_t ensure_table_entry(uint64_t *table,
                                   size_t index,
                                   uint64_t flags)
{
    uint64_t entry = table[index];

    if (entry & PTE_PRESENT) {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 117) - [public] if
    if (entry & PTE_PRESENT) {
        if (entry & PTE_PAGE_SIZE) {
            /*
             * Large page (2MB or 1GB) found where we need a page table.
             * Split it into smaller 4KB page entries.
             */
            uint64_t large_phys = entry & PAGING_ADDR_MASK;
            /* Only keep the real flag bits (low 12 bits + high NX bit), not
             * the physical-address bits that live in the middle of the entry. */
            uint64_t large_flags = entry & ~PAGING_ADDR_MASK;

            /* Allocate a new page table */
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 118) - [public] if
        if (entry & PTE_PAGE_SIZE) {
            /*
             * Large page (2MB or 1GB) found where we need a page table.
             * Split it into smaller 4KB page entries.
             */
            uint64_t large_phys = entry & PAGING_ADDR_MASK;
            /* Only keep the real flag bits (low 12 bits + high NX bit), not
             * the physical-address bits that live in the middle of the entry. */
            uint64_t large_flags = entry & ~PAGING_ADDR_MASK;

            /* Allocate a new page table */
            uint64_t table_phys = alloc_zeroed_table_page();
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 138) - [public] if
            if (i < pages) {
                uint64_t page_phys = large_phys + (uint64_t)i * PAGING_PAGE_SIZE;
                new_table[i] = (page_phys & PAGING_ADDR_MASK) |
                    (large_flags & ~(PTE_PAGE_SIZE));
            } else {
                new_table[i] = 0;
            }
        }

        /*
         * Replace the large page entry with the new page table entry.
         * Preserve the access flags from the large page.
         */
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 152) - [public] if
        if (large_flags & PTE_USER) {
            table_flags |= PTE_USER;
        }
        table[index] = (table_phys & PAGING_ADDR_MASK) | table_flags;

        return table_phys;
    }

    return entry & PAGING_ADDR_MASK;
}

uint64_t table_phys = alloc_zeroed_table_page();
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 171) - [public] if
    if (flags & PTE_USER) {
        table_flags |= PTE_USER;
    }

    table[index] = (table_phys & PAGING_ADDR_MASK) | table_flags;

    return table_phys;
}

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags)
{
    if (!boot_phys_offset_valid) {
        // ... (truncated, see source file for more)
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 182) - [public] if
        if (!boot_phys_offset_valid) {
            return false;
        }

        if ((virt & PAGING_PAGE_MASK) != 0 ||
            (phys & PAGING_PAGE_MASK) != 0) {
            return false;
        }

        /*
         * Tymczasowo nie u■ywamy NX, dopóki EFER.NXE nie jest w■■czony.
         */
        // ... (truncated, see source file for more)
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 214) - [public] paging_flush_page
        paging_flush_page(virt);

        return true;
    }

    static bool map_page_2m(uint64_t virt, uint64_t phys, uint64_t flags)
    {
        if (!boot_phys_offset_valid) {
            return false;
        }

        // ... (truncated, see source file for more)
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 222) - [public] if
        if (!boot_phys_offset_valid) {
            return false;
        }

        if ((virt & PAGING_2M_PAGE_MASK) != 0 ||
            (phys & PAGING_2M_PAGE_MASK) != 0) {
            return false;
        }

        flags &= ~PTE_NX;
        flags &= ~PTE_PAGE_SIZE;

        // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 250) - [public] if

```
/*
 * Je■li istnieje ju■ tablica 4 KiB w tym miejscu,
 * nie nadpisujemy jej 2 MiB stron■.
 */

if ((old & PTE_PRESENT) && !(old & PTE_PAGE_SIZE)) {
    paging_panic("cannot map 2M page over existing 4K table");
}

pd[idx] =
    (phys & PAGING_ADDR_MASK) |
    flags |
    PTE_PRESENT |
    PTE_PAGE_SIZE;

return true;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 251) - [public] paging_panic

```
    paging_panic("cannot map 2M page over existing 4K table");
}

pd[idx] =
    (phys & PAGING_ADDR_MASK) |
    flags |
    PTE_PRESENT |
    PTE_PAGE_SIZE;

return true;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 265) - [public] if

```
if (!boot_phys_offset_valid) {
    return false;
}

if (len == 0) {
    return true;
}

if ((virt & PAGING_PAGE_MASK) != 0 ||
    (phys & PAGING_PAGE_MASK) != 0 ||
    (len & PAGING_PAGE_MASK) != 0) {
    return false;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 269) - [public] if

```
if (len == 0) {
    return true;
}

if ((virt & PAGING_PAGE_MASK) != 0 ||
    (phys & PAGING_PAGE_MASK) != 0 ||
    (len & PAGING_PAGE_MASK) != 0) {
    return false;
}

if (len > UINT64_MAX - virt || len > UINT64_MAX - phys) {
    return false;
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 279) - [public] if
    if (len > UINT64_MAX - virt || len > UINT64_MAX - phys) {
        return false;
    }

    uint64_t virt_end = virt + len;
    uint64_t phys_end = phys + len;

    while (virt < virt_end) {
        uint64_t remaining = virt_end - virt;

        bool can_use_2m =
            (remaining >= PAGING_2M_PAGE_SIZE) &&
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 286) - [public] while
    while (virt < virt_end) {
        uint64_t remaining = virt_end - virt;

        bool can_use_2m =
            (remaining >= PAGING_2M_PAGE_SIZE) &&
            ((virt & PAGING_2M_PAGE_MASK) == 0) &&
            ((phys & PAGING_2M_PAGE_MASK) == 0);

        if (can_use_2m) {
            if (!map_page_2m(virt, phys, flags)) {
                return false;
            }
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 294) - [public] if
    if (can_use_2m) {
        if (!map_page_2m(virt, phys, flags)) {
            return false;
        }

        virt += PAGING_2M_PAGE_SIZE;
        phys += PAGING_2M_PAGE_SIZE;
    } else {
        if (!paging_map_page(virt, phys, flags)) {
            return false;
        }
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 295) - [public] if
    if (!map_page_2m(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_2M_PAGE_SIZE;
    phys += PAGING_2M_PAGE_SIZE;
} else {
    if (!paging_map_page(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_PAGE_SIZE;
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 302) - [public] if

```
    if (!paging_map_page(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_PAGE_SIZE;
    phys += PAGING_PAGE_SIZE;
}

paging_flush_tlb_all();

return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 311) - [public] paging_flush_tlb_all

```
    paging_flush_tlb_all();

    return true;
}
```

void paging_set_boot_phys_offset(uint64_t phys_offset)

```
{
    if ((phys_offset & PAGING_PAGE_MASK) != 0) {
        paging_panic("boot physical offset must be page aligned");
    }

    boot_phys_offset = phys_offset;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 318) - [public] if

```
    if ((phys_offset & PAGING_PAGE_MASK) != 0) {
        paging_panic("boot physical offset must be page aligned");
    }

    boot_phys_offset = phys_offset;
    boot_phys_offset_valid = true;
}

bool paging_boot_phys_offset_valid(void)
{
    return boot_phys_offset_valid;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 319) - [public] paging_panic

```
    paging_panic("boot physical offset must be page aligned");
}

    boot_phys_offset = phys_offset;
    boot_phys_offset_valid = true;
}

bool paging_boot_phys_offset_valid(void)
{
    return boot_phys_offset_valid;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 333) - [public] if

```
if (!arch_memory_ready()) {
    paging_panic("arch memory not initialized");
}

if (!boot_phys_offset_valid) {
    paging_panic("boot physical offset not set");
}

/*
 * Je■li bootloader ju■ mapuje fizyczn■ pami■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 334) - [public] paging_panic

```
    paging_panic("arch memory not initialized");
}

if (!boot_phys_offset_valid) {
    paging_panic("boot physical offset not set");
}

/*
 * Je■li bootloader ju■ mapuje fizyczn■ pami■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */
if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 337) - [public] if

```
if (!boot_phys_offset_valid) {
    paging_panic("boot physical offset not set");
}

/*
 * Je■li bootloader ju■ mapuje fizyczn■ pami■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */
if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
    return;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 338) - [public] paging_panic

```
    paging_panic("boot physical offset not set");
}

/*
 * Je■li bootloader ju■ mapuje fizyczn■ pami■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */
if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
    return;
}

const arch_mem_info_t *info = arch_memory_get();
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 345) - [public] if

```
/*  
 * Jeśli bootloader już mapuje fizyczny pamięć, dokonanie pod  
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robić.  
 */  
  
    if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {  
        return;  
    }  
  
    const arch_mem_info_t *info = arch_memory_get();  
  
    if (info == NULL) {  
        paging_panic("no arch memory info");  
    }  
  
    uint64_t phys_limit = arch_page_align_up(info->max_address);  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 351) - [public] if

```
    if (info == NULL) {  
        paging_panic("no arch memory info");  
    }  
  
    uint64_t phys_limit = arch_page_align_up(info->max_address);  
  
    if (phys_limit == 0 || phys_limit == UINT64_MAX) {  
        paging_panic("invalid physical memory limit");  
    }  
  
    if (!paging_map_range(ARCH_DIRECT_MAP_BASE,  
                          0,  
                          // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 352) - [public] paging_panic

```
        paging_panic("no arch memory info");  
    }  
  
    uint64_t phys_limit = arch_page_align_up(info->max_address);  
  
    if (phys_limit == 0 || phys_limit == UINT64_MAX) {  
        paging_panic("invalid physical memory limit");  
    }  
  
    if (!paging_map_range(ARCH_DIRECT_MAP_BASE,  
                          0,  
                          phys_limit,  
                          // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 357) - [public] if

```
    if (phys_limit == 0 || phys_limit == UINT64_MAX) {  
        paging_panic("invalid physical memory limit");  
    }  
  
    if (!paging_map_range(ARCH_DIRECT_MAP_BASE,  
                          0,  
                          phys_limit,  
                          PAGING_KERNEL_RW)) {  
        paging_panic("failed to create direct map");  
    }  
  
    paging_flush_tlb_all();  
    // ... (truncated, see source file for more)
```



```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 358) - [public] paging_panic
    paging_panic("invalid physical memory limit");
}

    if (!paging_map_range(ARCH_DIRECT_MAP_BASE,
                          0,
                          phys_limit,
                          PAGING_KERNEL_RW)) {
        paging_panic("failed to create direct map");
    }

    paging_flush_tlb_all();
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 365) - [public] paging_panic
    paging_panic("failed to create direct map");
}

    paging_flush_tlb_all();
}

void paging_init(uint64_t phys_offset)
{
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 368) - [public] paging_flush_tlb_all
    paging_flush_tlb_all();
}

void paging_init(uint64_t phys_offset)
{
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 373) - [public]
paging_set_boot_phys_offset
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */

/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 374) - [public]

paging_init_direct_map

```
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */

/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * and returns NULL on the first absent entry.
 * // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 384) - [public] allocated

```
/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * and returns NULL on the first absent entry.
 *
 * Returns NULL on absent entries (create == false), allocation failure,
 * or unexpected large pages.
 */
static uint64_t *resolve_pte(uint64_t pml4_phys, uint64_t virt, bool create)
{
    uint64_t *pml4 = phys_to_ptr(pml4_phys);
    uint64_t *pdpt, *pd, *pt;
    uint64_t entry;
    size_t idx;
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 399) - [public] if

```
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pml4[idx];
    }
    pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    idx = pdpt_index(virt);
    entry = pdpt[idx];
    if (!(entry & PTE_PRESENT)) {
        // ... (truncated, see source file for more)
    }

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 400) - [public] if

```
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pml4[idx];
    }
    pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    idx = pdpt_index(virt);
    entry = pdpt[idx];
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            // ... (truncated, see source file for more)
        }
    }

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 403) - [public] ensure_table_entry
    ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pml4[idx];
}
pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pdpt_index(virt);
entry = pdpt[idx];
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 410) - [public] if
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pdpt[idx];
    }
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 1G large page - not handled here */
    }
    pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 411) - [public] if
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pdpt[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 1G large page - not handled here */
}
pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pd_index(virt);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 414) - [public] ensure_table_entry
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pdpt[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 1G large page - not handled here */
}
pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pd_index(virt);
entry = pd[idx];
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 417) - [public] if
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 1G large page - not handled here */
    }
    pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

    idx = pd_index(virt);
    entry = pd[idx];
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 424) - [public] if
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pd[idx];
    }
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 2M large page - not handled here */
    }
    pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 425) - [public] if
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pd[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 2M large page - not handled here */
}
pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

return &pt[pt_index(virt)];
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 428) - [public] ensure_table_entry
    ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pd[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 2M large page - not handled here */
}
pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

return &pt[pt_index(virt)];
}

/* ----- */
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 431) - [public] if
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 2M large page - not handled here */
    }
    pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    return &pt[pt_index(virt)];
}

/* ----- */
/* Arch-specific operations used by the generic paging layer */
/* ----- */

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 447) - [public] if
    if (!boot_phys_offset_valid) {
        return false;
    }
    if ((virt & PAGING_PAGE_MASK) != 0 || (phys & PAGING_PAGE_MASK) != 0) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;

    pte = resolve_pte(pml4, virt, true);
    if (pte == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 450) - [public] if
    if ((virt & PAGING_PAGE_MASK) != 0 || (phys & PAGING_PAGE_MASK) != 0) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;

    pte = resolve_pte(pml4, virt, true);
    if (pte == NULL) {
        return false;
    }
    *pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 456) - [public] if
    if (pte == NULL) {
        return false;
    }
    *pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    return true;
}

bool paging_unmap_page_in(uint64_t pml4, uint64_t virt)
{
    uint64_t *pte = resolve_pte(pml4, virt, false);
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 461) - [public] paging_flush_page

```
*pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;
```

```
    paging_flush_page(virt);  
    return true;  
}
```

```
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt)
```

```
{  
    uint64_t *pte = resolve_pte(pml4, virt, false);  
  
    if (pte == NULL || !(*pte & PTE_PRESENT)) {  
        return false;  
    }  
    *pte = 0;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 469) - [public] if

```
    if (pte == NULL || !(*pte & PTE_PRESENT)) {  
        return false;  
    }
```

```
    *pte = 0;  
    paging_flush_page(virt);  
    return true;  
}
```

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt)
```

```
{  
    uint64_t *pte = resolve_pte(pml4, virt, false);  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 473) - [public] paging_flush_page

```
*pte = 0;
```

```
    paging_flush_page(virt);  
    return true;  
}
```

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt)
```

```
{  
    uint64_t *pte = resolve_pte(pml4, virt, false);  
  
    if (pte == NULL || !(*pte & PTE_PRESENT)) {  
        return 0;  
    }  
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 481) - [public] if

```
    if (pte == NULL || !(*pte & PTE_PRESENT)) {  
        return 0;  
    }
```

```
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);  
}
```

```
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt)
```

```
{  
    const uint64_t *pte = resolve_pte(pml4, virt, false);  
  
    return pte != NULL && (*pte & PTE_PRESENT) != 0;  
}  
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 484) - [public] return
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);
}

bool paging_is_mapped_in(uint64_t pml4, uint64_t virt)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);

    return pte != NULL && (*pte & PTE_PRESENT) != 0;
}

bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 498) - [public] if
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;
    *pte = (*pte & PAGING_ADDR_MASK) | flags | PTE_PRESENT;
    paging_flush_page(virt);
    return true;
}

bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 503) - [public] paging_flush_page
    *pte = (*pte & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    return true;
}

bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);

    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    if (out_flags != NULL) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 511) - [public] if
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    if (out_flags != NULL) {
        *out_flags = *pte & ~PAGING_ADDR_MASK;
    }
    return true;
}

bool paging_is_mapped(uint64_t virt)
{
    return paging_is_mapped_in(paging_read_cr3(), virt);
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 514) - [public] if

```
    if (out_flags != NULL) {
        *out_flags = *pte & ~PAGING_ADDR_MASK;
    }
    return true;
}

bool paging_is_mapped(uint64_t virt)
{
    return paging_is_mapped_in(paging_read_cr3(), virt);
}

uint64_t paging_translate(uint64_t virt)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 522) - [public] paging_is_mapped_in

```
    return paging_is_mapped_in(paging_read_cr3(), virt);
}

uint64_t paging_translate(uint64_t virt)
{
    return paging_translate_in(paging_read_cr3(), virt);
}

bool paging_unmap_page(uint64_t virt)
{
    return paging_unmap_page_in(paging_read_cr3(), virt);
}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 527) - [public] paging_translate_in

```
    return paging_translate_in(paging_read_cr3(), virt);
}

bool paging_unmap_page(uint64_t virt)
{
    return paging_unmap_page_in(paging_read_cr3(), virt);
}

bool paging_set_flags(uint64_t virt, uint64_t flags)
{
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 532) - [public] paging_unmap_page_in

```
    return paging_unmap_page_in(paging_read_cr3(), virt);
}

bool paging_set_flags(uint64_t virt, uint64_t flags)
{
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}

bool paging_get_flags(uint64_t virt, uint64_t *out_flags)
{
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}
    // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 537) - [public] paging_set_flags_in
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}

bool paging_get_flags(uint64_t virt, uint64_t *out_flags)
{
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}

/* NX is enabled once via the EFER MSR; this is idempotent. */
static bool nx_enabled = false;

void paging_enable_nx(void)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 542) - [public] paging_get_flags_in
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}

/* NX is enabled once via the EFER MSR; this is idempotent. */
static bool nx_enabled = false;

void paging_enable_nx(void)
{
    uint64_t efer;

    if (nx_enabled) {
        return;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 552) - [public] if
    if (nx_enabled) {
        return;
    }
    __asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
    efer |= (1ULL << 11); /* NXE bit */
    __asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
    nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 555) - [public] volatile
    __asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
    efer |= (1ULL << 11); /* NXE bit */
    __asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
    nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
}

void paging_write_cr3(uint64_t pml4_phys)
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 557) - [public] volatile

```
if (nx_enabled) {
    return;
}
__asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
efer |= (1ULL << 11); /* NXE bit */

__asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
}

void paging_write_cr3(uint64_t pml4_phys)
{
    __asm__ volatile("mov %0, %%cr3" : : "r"(pml4_phys) : "memory");
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/paging.c (line 568) - [public] volatile

```
__asm__ volatile("mov %0, %%cr3" : : "r"(pml4_phys) : "memory");
}
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 77) - [public] tlb_asm_invpcid

```
■ tlb_asm_invpcid(INVPCID_ALL, &desc);
■ return;
■}

■tlb_asm_write_cr3(tlb_asm_read_cr3());
}

void tlb_flush_all_including_global(void)
{
    ■if (has_invpcid) {
    ■ tlb_flush_all();
    ■ return;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 87) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■uint64_t cr4 = tlb_asm_read_cr4();

■if (cr4 & CR4_PGE) {
    ■ tlb_asm_write_cr4(cr4 & ~CR4_PGE);
    ■ tlb_asm_write_cr4(cr4);
    ■} else {
    ■ tlb_flush_all();
    ■}
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 94) - [public] tlb_asm_write_cr4

```
■ tlb_asm_write_cr4(cr4 & ~CR4_PGE);
■ tlb_asm_write_cr4(cr4);
    ■} else {
    ■ tlb_flush_all();
    ■}
}

void tlb_flush_page_addr(uint64_t addr)
{
    ■tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 95) - [public] tlb_asm_write_cr4

```
    tlb_asm_write_cr4(cr4);
} else {
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}

void tlb_flush_page(const void *addr)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 97) - [public] tlb_flush_all

```
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}

void tlb_flush_page(const void *addr)
{
    if (addr == NULL) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 109) - [public] tlb_flush_all

```
    tlb_flush_all();
    return;
}

tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    if (pages == 0) {
        return;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/mm/arch/x86_64/tlb.c (line 123) - [public] tlb_flush_all

```
    tlb_flush_all();
    return;
}

uint64_t a = addr & ~TLB_PAGE_MASK;

if (pages > (UINT64_MAX - a) / TLB_PAGE_SIZE) {
    tlb_flush_all();
    return;
}

for (size_t i = 0; i < pages; i++) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 130) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■for (size_t i = 0; i < pages; i++) {
■ tlb_asm_invlpg(a);
■ a += TLB_PAGE_SIZE;
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 135) - [public] tlb_asm_invlpg

```
■ tlb_asm_invlpg(a);
■ a += TLB_PAGE_SIZE;
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■if (addr == NULL) {
■ tlb_flush_all();
■ return;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 143) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
}

void tlb_flush_pcid(uint16_t pcid)
{
■if (has_invpcid) {
■ invpcid_desc_t desc = { pcid, 0 };
■ tlb_asm_invpcid(INVPCID_SINGLE_PCID, &desc);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 154) - [public] tlb_asm_invpcid

```
■ tlb_asm_invpcid(INVPCID_SINGLE_PCID, &desc);
■ return;
■}

■if (pcid == TLB_PCID_KERNEL) {
■ tlb_asm_write_cr3(tlb_asm_read_cr3());
■}
}

void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr)
{
■if (has_invpcid) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 159) - [public] tlb_asm_write_cr3

```
    tlb_asm_write_cr3(tlb_asm_read_cr3());
}

void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr)
{
    if (has_invpcid) {
        invpcid_desc_t desc = { pcid, addr & ~TLB_PAGE_MASK };
        tlb_asm_invpcid(INVPCID_ADDR_IN_PCID, &desc);
        return;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 167) - [public] tlb_asm_invpcid

```
    tlb_asm_invpcid(INVPCID_ADDR_IN_PCID, &desc);
    return;
}

tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}

void tlb_wbinvd(void)
{
    tlb_asm_wbinvd();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 3) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static kcache_t *cache_registry = NULL;
static bool cache_initialized = false;

bool cache_init(void)
{
    if (cache_initialized) {
        return true;
    }

    per_cpu_init();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 10) - [public] if

```
    if (cache_initialized) {
        return true;
    }

    per_cpu_init();

    cache_registry = NULL;
    cache_initialized = true;

    return true;
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 14) - [public] per_cpu_init
    per_cpu_init();

    cache_registry = NULL;
    cache_initialized = true;

    return true;
}

bool cache_ready(void)
{
    return cache_initialized;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 29) - [public] if
    if (c == NULL) {
        return;
    }

    c->next = cache_registry;
    cache_registry = c;
}

kcache_t *cache_find(const char *name)
{
    kcache_t *c = cache_registry;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 41) - [public] while
    while (c != NULL) {
        const char *a = c->name;
        const char *b = name;

        while (*a != '\0' && *a == *b) {
            a++;
            b++;
        }

        if (*a == '\0' && *b == '\0') {
            return c;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 45) - [public] while
        while (*a != '\0' && *a == *b) {
            a++;
            b++;
        }

        if (*a == '\0' && *b == '\0') {
            return c;
        }

        c = c->next;
    }

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 50) - [public] if
    if (*a == '\0' && *b == '\0') {
        return c;
    }

    c = c->next;
}

return NULL;
}

void cache_dump(void)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 62) - [public] if
    if (!cache_initialized) {
        kprintf("CACHE: not initialized\n");
        return;
    }

    kprintf("CACHE:\n");

    kcache_t *c = cache_registry;

    while (c != NULL) {
        kprintf(" %-16s obj=%u free=%u total=%u\n",
            c->name,
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 63) - [public] kprintf
        kprintf("CACHE: not initialized\n");
        return;
    }

    kprintf("CACHE:\n");

    kcache_t *c = cache_registry;

    while (c != NULL) {
        kprintf(" %-16s obj=%u free=%u total=%u\n",
            c->name,
            (unsigned int)c->object_size,
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 67) - [public] kprintf
    kprintf("CACHE:\n");

    kcache_t *c = cache_registry;

    while (c != NULL) {
        kprintf(" %-16s obj=%u free=%u total=%u\n",
            c->name,
            (unsigned int)c->object_size,
            (unsigned int)c->free_count,
            (unsigned int)c->total_objects);

        c = c->next;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/cache/cache.c (line 71) - [public] while
    while (c != NULL) {
        kprintf(" %-16s obj=%u free=%u total=%u\n",
            c->name,
            (unsigned int)c->object_size,
            (unsigned int)c->free_count,
            (unsigned int)c->total_objects);

        c = c->next;
    }
}

```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 82) - [public] buddy_free

```
■ buddy_free(va);
■ return false;
■}

■ch->va = (uint64_t)(uintptr_t)va;
■ch->next = c->chunks;
■c->chunks = ch;

■size_t n = c->chunk_size / c->object_size;

■for (size_t i = 0; i < n; i++) {
■    uint8_t *obj =
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 96) - [public] if

```
■    if (c->ctor != NULL) {
■        c->ctor(obj);
■    }

■    *(void **)obj = c->free_head;
■    c->free_head = obj;
■}

■c->free_count += n;
■c->total_objects += n;

■return true;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 180) - [public] buddy_free

```
■    buddy_free((void *) (uintptr_t)ch->va);
■    heap_free(ch);

■    ch = next;
■}

■c->chunks = NULL;
■c->free_head = NULL;
■c->free_count = 0;
■c->total_objects = 0;

■cache_unlock();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 181) - [public] heap_free

```
■    heap_free(ch);

■    ch = next;
■}

■c->chunks = NULL;
■c->free_head = NULL;
■c->free_count = 0;
■c->total_objects = 0;

■cache_unlock();

    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/cache/object_cache.c (line 208) - [public] cache_unlock

```
■ cache_unlock();
■ return obj;
■}

■if (c->free_head == NULL) {
■    if (!kcache_grow(c)) {
■        cache_unlock();
■        return NULL;
■    }
■}

■void *obj = c->free_head;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 213) - [public] if

```
■    if (!kcache_grow(c)) {
■        cache_unlock();
■        return NULL;
■    }
■}

■void *obj = c->free_head;
■c->free_head = *(void **)obj;
■c->free_count--;

■cache_unlock();

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 214) - [public] cache_unlock

```
■        cache_unlock();
■        return NULL;
■    }
■}

■void *obj = c->free_head;
■c->free_head = *(void **)obj;
■c->free_count--;

■cache_unlock();

■return obj;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 253) - [public] cache_unlock

```
■    cache_unlock();
■    return;
■}

■*(void **)ptr = c->free_head;
■c->free_head = ptr;
■c->free_count++;

■cache_unlock();
}

size_t kcache_object_size(kcache_t *c)
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/core/mm.c (line 15) - [public] kprintf
extern void kprintf(const char *fmt, ...);

static bool mm_initialized = false;

bool mm_init(const mm_boot_params_t *params)
{
    if (mm_initialized) {
        return true;
    }

    if (params == NULL ||
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/mm.c (line 29) - [public] kprintf
    kprintf("mm_init: bad params\n");
    return false;
}

kprintf("mm_init: arch_memory_init...\n");
arch_memory_init(params->memmap,
    params->memmap_count,
    params->kernel_phys_start,
    params->kernel_phys_end,
    params->initrd_phys_start,
    params->initrd_phys_end);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/mm.c (line 46) - [public] kprintf
    kprintf("mm_init: pmm_init FAILED\n");
    return false;
}

kprintf("mm_init: vmm_init...\n");
if (!vmm_init()) {
    kprintf("mm_init: vmm_init FAILED\n");
    return false;
}

kprintf("mm_init: page_init...\n");
if (!page_init()) {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/mm.c (line 52) - [public] kprintf
    kprintf("mm_init: vmm_init FAILED\n");
    return false;
}

kprintf("mm_init: page_init...\n");
if (!page_init()) {
    kprintf("mm_init: page_init FAILED\n");
    return false;
}

mapping_init();

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/core/mm.c (line 58) - [public] kprintf

```
■ kprintf("mm_init: page_init FAILED\n");  
■ return false;  
■ }
```

```
■ mapping_init();
```

```
■ kprintf("mm_init: heap_init...\n");  
■ if (!heap_init()) {  
■ kprintf("mm_init: heap_init FAILED\n");  
■ return false;  
■ }
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 66) - [public] kprintf

```
■ kprintf("mm_init: heap_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: cache_init...\n");  
■ if (!cache_init()) {  
■ kprintf("mm_init: cache_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: paging_subsystem_init...\n");  
■ if (!paging_subsystem_init()) {  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 72) - [public] kprintf

```
■ kprintf("mm_init: cache_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: paging_subsystem_init...\n");  
■ if (!paging_subsystem_init()) {  
■ kprintf("mm_init: paging_subsystem_init FAILED\n");  
■ return false;  
■ }
```

```
■ isolation_init();
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 78) - [public] kprintf

```
■ kprintf("mm_init: paging_subsystem_init FAILED\n");  
■ return false;  
■ }
```

```
■ isolation_init();
```

```
■ kprintf("mm_init: aspace_subsystem_init...\n");  
■ if (!aspace_subsystem_init()) {  
■ kprintf("mm_init: aspace_subsystem_init FAILED\n");  
■ return false;  
■ }
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 86) - [public] kprintf

```
■ kprintf("mm_init: aspace_subsystem_init FAILED\n");
■ return false;
■}

■mm_initialized = true;

■kprintf("mm_init: OK\n");

■return true;
}

bool mm_ready(void)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 104) - [public] pmm_stat_total_bytes

```
■return pmm_stat_total_bytes();
}

uint64_t mm_free_ram(void)
{
■return pmm_stat_free_bytes();
}

void mm_dump(void)
{
■kprintf("MM: ready=%d total=%llu MiB free=%llu MiB\n",
■      (int)mm_initialized,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 109) - [public] pmm_stat_free_bytes

```
■return pmm_stat_free_bytes();
}

void mm_dump(void)
{
■kprintf("MM: ready=%d total=%llu MiB free=%llu MiB\n",
■      (int)mm_initialized,
■      (unsigned long long)(mm_total_ram() >> 20),
■      (unsigned long long)(mm_free_ram() >> 20));

■kalloc_dump();
}
```

kernel_Workspace/kernel/src/mm/core/range.c (line 53) - [public] if

```
■ if (raw_end > UINT64_MAX - pad) {
■     return false;
■ }

■ end = raw_end + pad;
■}

■if (end <= start) {
■     return false;
■}

■if (require_canonical) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/range.c (line 65) - [public] if

```
■ if (!range_is_canonical(start) || !range_is_canonical(end - 1)) {
■     return false;
■ }
■ }

■ if (start < limit_min) {
■     return false;
■ }

■ if (end > limit_max) {
■     return false;
■ }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.c (line 64) - [public] if

```
■ if (region_valid(before)) {
■     out[n++] = before;
■ }
■ }

■ if (cut.end < r.end && n < max_out) {
■     region_t after;

■     after.start = cut.end > r.start ? cut.end : r.start;
■     after.end = r.end;

■     if (region_valid(after)) {
■         // ... (truncated, see source file for more)
■     }
■ }
```

kernel_Workspace/kernel/src/mm/core/region.c (line 75) - [public] if

```
■ if (region_valid(after)) {
■     out[n++] = after;
■ }
■ }

■ return n;
■ }
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 5) - [public] volatile

```
■ __asm__ volatile("pause" ::: "memory");
■ }

static inline uint64_t irq_save_disable(void)
{
■ uint64_t flags;

■ __asm__ volatile(
■     "pushfq\n"
■     "popq %0\n"
■     "cli"
■     : "=r"(flags)
■     // ... (truncated, see source file for more)
■ );
■ }
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 64) - [public] __atomic_load_n

```
■ __atomic_load_n(&lock->owner_cpu, __ATOMIC_ACQUIRE) == me) {
■     lock->depth++;
■     return;
■ }

■ uint32_t ticket = __atomic_fetch_add(&lock->next_ticket, 1,
■                                     __ATOMIC_RELAXED);

■ while (__atomic_load_n(&lock->now_serving, __ATOMIC_ACQUIRE) != ticket) {
■     cpu_pause();
■ }
■ lock->owner_cpu = me;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 73) - [public] cpu_pause

```
■    cpu_pause();
■}
■lock->owner_cpu = me;
■lock->depth = 1;
■lock->saved_flags = flags;
■}

bool smp_lock_release(smp_ticket_lock_t *lock)
{
■if (lock->depth == 0) {
■    return false;
■}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 18) - [public] volatile

```
■__asm__ volatile("inb %%dx, %0" : "=a"(v) : "d"(port));

■return v;
■}

static inline void outb(uint16_t port, uint8_t val)
{
■__asm__ volatile("outb %0, %%dx" : : "a"(val), "d"(port));
■}

static bool uart_ready = false;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 25) - [public] volatile

```
■__asm__ volatile("outb %0, %%dx" : : "a"(val), "d"(port));
■}

static bool uart_ready = false;

static void serial_init(void)
{
■if (uart_ready) {
■    return;
■}

■outb(COM1 + 1, 0x00);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 58) - [public] serial_putc

```
■    serial_putc(*s++);
■}
■}

static void serial_put_uint(uint64_t v, unsigned base, bool upper)
{
■char buf[32];
■int i = 32;

■if (v == 0) {
■    serial_putc('0');
■    return;
■}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 68) - [public] serial_putc

```
■    serial_putc('0');
■    return;
■}

■while (v > 0) {
■    unsigned d = (unsigned)(v % base);

■    buf[--i] = (char)(d < 10 ? '0' + d : (upper ? 'A' : 'a') + d - 10);
■    v /= base;
■}

■while (i < 32) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 80) - [public] serial_putc

```
■    serial_putc(buf[i++]);
■}
}

void kprintf(const char *fmt, ...)
{
■serial_init();

■va_list ap;

■va_start(ap, fmt);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 93) - [public] if

```
■    if (*p != '%') {
■        serial_putc(*p);
■        continue;
■    }

■    p++;
■    if (*p == '\\0') {
■        break;
■    }

■    bool ll = false;
■    bool l = false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 94) - [public] serial_putc

```
■        serial_putc(*p);
■        continue;
■    }

■    p++;
■    if (*p == '\\0') {
■        break;
■    }

■    bool ll = false;
■    bool l = false;
■    bool z = false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 99) - [public] if

```
■   if (*p == '\0') {
■       break;
■   }

■   bool ll = false;
■   bool l = false;
■   bool z = false;

■   while (*p == 'l' || *p == 'z' || *p == 'h') {
■       if (*p == 'l') {
■           if (p[1] == 'l') {
■               ll = true;
■           }
■           // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 107) - [public] while

```
■   while (*p == 'l' || *p == 'z' || *p == 'h') {
■       if (*p == 'l') {
■           if (p[1] == 'l') {
■               ll = true;
■               p++;
■           } else {
■               l = true;
■           }
■       } else if (*p == 'z') {
■           z = true;
■       }
■       p++;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 108) - [public] if

```
■       if (*p == 'l') {
■           if (p[1] == 'l') {
■               ll = true;
■               p++;
■           } else {
■               l = true;
■           }
■       } else if (*p == 'z') {
■           z = true;
■       }
■       p++;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 109) - [public] if

```
■       if (p[1] == 'l') {
■           ll = true;
■           p++;
■       } else {
■           l = true;
■       }
■       } else if (*p == 'z') {
■           z = true;
■       }
■       p++;
■       // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/kprintf.c (line 121) - [public] switch

```
■ switch (*p) {
■ case 's': {
■     const char *s = va_arg(ap, const char *);

■     serial_puts(s ? s : "(null)");
■     break;
■ }
■ case 'c':
■     serial_putc((char)va_arg(ap, int));
■     break;
■ case 'd':
■ case 'i': {
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 125) - [public] serial_puts

```
■     serial_puts(s ? s : "(null)");
■     break;
■ }
■ case 'c':
■     serial_putc((char)va_arg(ap, int));
■     break;
■ case 'd':
■ case 'i': {
■     int64_t v = ll ? (int64_t)va_arg(ap, long long)
■                  : (l ? (int64_t)va_arg(ap, long)
■                  : (int64_t)va_arg(ap, int));

■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 129) - [public] serial_putc

```
■     serial_putc((char)va_arg(ap, int));
■     break;
■ case 'd':
■ case 'i': {
■     int64_t v = ll ? (int64_t)va_arg(ap, long long)
■                  : (l ? (int64_t)va_arg(ap, long)
■                  : (int64_t)va_arg(ap, int));

■     if (v < 0) {
■         serial_putc('-');
■         serial_put_uint((uint64_t)(-v), 10, false);
■     } else {
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 137) - [public] if

```
■     if (v < 0) {
■         serial_putc('-');
■         serial_put_uint((uint64_t)(-v), 10, false);
■     } else {
■         serial_put_uint((uint64_t)v, 10, false);
■     }
■     break;
■ }
■ case 'u': {
■     uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                     : (z ? (uint64_t)va_arg(ap, size_t)
■                     : (l ? (uint64_t)va_arg(ap, unsigned long)
■                     : (uint64_t)va_arg(ap, int)));
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 138) - [public] serial_putc

```
■      serial_putc('-');
■      serial_put_uint((uint64_t)(-v), 10, false);
■    } else {
■      serial_put_uint((uint64_t)v, 10, false);
■    }
■    break;
■  }
■  case 'u': {
■    uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                  : (z ? (uint64_t)va_arg(ap, size_t)
■                  : (l ? (uint64_t)va_arg(ap, unsigned long)
■                  : (uint64_t)va_arg(ap, unsigned int)));
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 139) - [public] serial_put_uint

```
■      serial_put_uint((uint64_t)(-v), 10, false);
■    } else {
■      serial_put_uint((uint64_t)v, 10, false);
■    }
■    break;
■  }
■  case 'u': {
■    uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                  : (z ? (uint64_t)va_arg(ap, size_t)
■                  : (l ? (uint64_t)va_arg(ap, unsigned long)
■                  : (uint64_t)va_arg(ap, unsigned int)));
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 141) - [public] serial_put_uint

```
■      serial_put_uint((uint64_t)v, 10, false);
■    }
■    break;
■  }
■  case 'u': {
■    uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                  : (z ? (uint64_t)va_arg(ap, size_t)
■                  : (l ? (uint64_t)va_arg(ap, unsigned long)
■                  : (uint64_t)va_arg(ap, unsigned int)));
■
■    serial_put_uint(v, 10, false);
■    break;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 151) - [public] serial_put_uint

```
■      serial_put_uint(v, 10, false);
■    break;
■  }
■  case 'x':
■  case 'X': {
■    uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                  : (z ? (uint64_t)va_arg(ap, size_t)
■                  : (l ? (uint64_t)va_arg(ap, unsigned long)
■                  : (uint64_t)va_arg(ap, unsigned int)));
■
■    serial_put_uint(v, 16, *p == 'X');
■    break;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 161) - [public] serial_put_uint

```
■    serial_put_uint(v, 16, *p == 'X');
■    break;
■    }
■    case 'p': {
■        void *ptr = va_arg(ap, void *);

■        serial_puts("0x");
■        serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■        break;
■    }
■    case '%':
■        serial_putc('%');
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 167) - [public] serial_puts

```
■    serial_puts("0x");
■    serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■    break;
■    }
■    case '%':
■        serial_putc('%');
■        break;
■    default:
■        serial_putc('%');
■        serial_putc(*p);
■        break;
■    }
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 168) - [public] serial_put_uint

```
■    serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■    break;
■    }
■    case '%':
■        serial_putc('%');
■        break;
■    default:
■        serial_putc('%');
■        serial_putc(*p);
■        break;
■    }
■}
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 172) - [public] serial_putc

```
■    serial_putc('%');
■    break;
■    default:
■        serial_putc('%');
■        serial_putc(*p);
■        break;
■    }
■}

■va_end(ap);
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/kprintf.c (line 175) - [public] serial_putc

```
■    serial_putc('%');
■    serial_putc(*p);
■    break;
■    }
■}

■va_end(ap);
}

void serial_write_str(const char *s)
{
■serial_init();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 176) - [public] serial_putc

```
■    serial_putc(*p);
■    break;
■    }
■}

■va_end(ap);
}

void serial_write_str(const char *s)
{
■serial_init();
■serial_puts(s);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 68) - [public] if

```
■    if (pml_entry_present(e) && !pml_entry_large(e)) {
■        free_tables(pml_entry_addr(e), level - 1);
■    }
■}

■pmm_free_frame(table_phys);
}

address_space_t *paging_aspace_create(void)
{
■if (!paging_subsystem_ready) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 69) - [public] free_tables

```
■        free_tables(pml_entry_addr(e), level - 1);
■    }
■}

■pmm_free_frame(table_phys);
}

address_space_t *paging_aspace_create(void)
{
■if (!paging_subsystem_ready) {
■    return NULL;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 99) - [public] pmm_free_frame

```
■    pmm_free_frame(pml4_phys);  
■    return NULL;  
■}
```

```
■as->pml4_phys = pml4_phys;  
■as->kernel = false;
```

```
■return as;  
}
```

```
void paging_aspace_destroy(address_space_t *as)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 120) - [public] if

```
■    if (pml_entry_present(e) && !pml_entry_large(e)) {  
■        free_tables(pml_entry_addr(e), PML_LEVEL_PDPT);  
■    }  
■}
```

```
■pmm_free_frame(as->pml4_phys);
```

```
■heap_free(as);  
}
```

```
void paging_aspace_switch(address_space_t *as)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 121) - [public] free_tables

```
■        free_tables(pml_entry_addr(e), PML_LEVEL_PDPT);  
■    }  
■}
```

```
■pmm_free_frame(as->pml4_phys);
```

```
■heap_free(as);  
}
```

```
void paging_aspace_switch(address_space_t *as)  
{  
■if (as == NULL) {  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 173) - [public] if

```
■    if (!paging_map_page_in(pml4, v, p, pte)) {  
■        for (size_t j = 0; j < mapped; j++) {  
■            paging_unmap_page_in(pml4, virt + (uint64_t)j * ARCH_PAGE_SIZE);  
■        }  
■    }  
■}
```

```
■    return false;  
■}
```

```
■    mapped++;  
■}
```

```
■return true;  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 175) - [public] paging_unmap_page_in

```
■         paging_unmap_page_in(pml4, virt + (uint64_t)j * ARCH_PAGE_SIZE);
■     }

■     return false;
■ }

■     mapped++;
■ }

■ return true;
■ }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 207) - [public] if

```
■     if (paging_unmap_page_in(pml4, v)) {
■         tlb_batch_add(&batch, v);
■     }
■ }

■ tlb_batch_commit(&batch);

■ return true;
■ }

bool paging_aspace_protect(address_space_t *as,
■         uint64_t virt,
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 208) - [public] tlb_batch_add

```
■         tlb_batch_add(&batch, v);
■     }
■ }

■ tlb_batch_commit(&batch);

■ return true;
■ }

bool paging_aspace_protect(address_space_t *as,
■         uint64_t virt,
■         size_t len,
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 238) - [public] if

```
■     if (!paging_is_mapped_in(pml4, v)) {
■         return false;
■     }
■ }

■ tlb_batch_t batch;
■ tlb_batch_begin(&batch);

■ for (size_t i = 0; i < pages; i++) {
■     uint64_t v = virt + (uint64_t)i * ARCH_PAGE_SIZE;

■     if (!paging_set_flags_in(pml4, v, pte)) {
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 249) - [public] if

```
■    if (!paging_set_flags_in(pml4, v, pte)) {
■        tlb_batch_commit(&batch);
■        return false;
■    }

■    tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 250) - [public] tlb_batch_commit

```
■        tlb_batch_commit(&batch);
■        return false;
■    }

■    tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.c (line 254) - [public] tlb_batch_add

```
■    tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}

uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt)
{
■if (as == NULL) {
■    return paging_translate_in(paging_kernel_pml4, virt);
■    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 265) - [public] paging_translate_in

```
■    return paging_translate_in(paging_kernel_pml4, virt);
■}

■return paging_translate_in(as->pml4_phys, virt);
}

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot)
{
■if (!paging_subsystem_ready) {
■    return false;
■}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 268) - [public] paging_translate_in
    ■return paging_translate_in(as->pml4_phys, virt);
    ■}

    bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot)
    {
    ■if (!paging_subsystem_ready) {
    ■    return false;
    ■}

    ■return paging_aspace_map((address_space_t *)&(address_space_t){
    ■    .pml4_phys = paging_kernel_pml4,
    ■    .kernel = true,
    ■    // ... (truncated, see source file for more)
    ■});

kernel_Workspace/kernel/src/mm/paging/paging.c (line 277) - [public] paging_aspace_map
    ■return paging_aspace_map((address_space_t *)&(address_space_t){
    ■    .pml4_phys = paging_kernel_pml4,
    ■    .kernel = true,
    ■    },
    ■    virt, phys, len, prot);
    ■}

    bool paging_kernel_unmap(uint64_t virt, size_t len)
    {
    ■if (!paging_subsystem_ready) {
    ■    return false;
    ■}
    ■// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.c (line 290) - [public] paging_aspace_unmap
    ■return paging_aspace_unmap((address_space_t *)&(address_space_t){
    ■    .pml4_phys = paging_kernel_pml4,
    ■    .kernel = true,
    ■    },
    ■    virt, len);
    ■}

kernel_Workspace/kernel/src/mm/paging/tlb.c (line 33) - [public] tlb_flush_all
    ■    tlb_flush_all();
    ■    batch->count = 0;
    ■    batch->full_flush = false;
    ■    return;
    ■}

    ■for (size_t i = 0; i < batch->count; i++) {
    ■    tlb_flush_page_addr(batch->pages[i]);
    ■}

    ■batch->count = 0;
    ■}

kernel_Workspace/kernel/src/mm/paging/tlb.c (line 40) - [public] tlb_flush_page_addr
    ■    tlb_flush_page_addr(batch->pages[i]);
    ■}

    ■batch->count = 0;
    ■}

```


kernel_Workspace/kernel/src/mm/process/address_space.c (line 63) - [public] if

```
■ if (!pmm_alloc_zero_frame(&phys)) {
■     goto rollback;
■ }

■ if (!paging_aspace_map(pa->as, v, phys, ARCH_PAGE_SIZE,
■                         prot | PROT_USER)) {
■     pmm_free_frame(phys);
■     goto rollback;
■ }

■ mapped_end = v + ARCH_PAGE_SIZE;
■ }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 69) - [public] pmm_free_frame

```
■     pmm_free_frame(phys);
■     goto rollback;
■ }

■ mapped_end = v + ARCH_PAGE_SIZE;
■ }

■ return true;

rollback:
■ for (uint64_t v = start; v < mapped_end; v += ARCH_PAGE_SIZE) {
■     uint64_t phys = paging_aspace_translate(pa->as, v);
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 82) - [public]
paging_aspace_unmap

```
■     paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);

■     if (phys != UINT64_MAX) {
■         pmm_free_frame(phys);
■     }
■ }

■ return false;
}

static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 84) - [public] if

```
■     if (phys != UINT64_MAX) {
■         pmm_free_frame(phys);
■     }
■ }

■ return false;
}

static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)
{
■ for (uint64_t v = start; v < end; v += ARCH_PAGE_SIZE) {
■     uint64_t phys = paging_aspace_translate(pa->as, v);
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 85) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

■return false;
}

static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)
{
■for (uint64_t v = start; v < end; v += ARCH_PAGE_SIZE) {
■    uint64_t phys = paging_aspace_translate(pa->as, v);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 97) - [public]
paging_aspace_unmap

```
■    paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 99) - [public] if

```
■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

■while (*p != NULL && (*p)->start < v->start) {
■    p = &(*p)->next;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 100) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

■while (*p != NULL && (*p)->start < v->start) {
■    p = &(*p)->next;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 122) - [public] if

```
■   if (*p == v) {  
■       *p = v->next;  
■       return;  
■   }  
  
■   p = &(*p)->next;  
■}  
}
```

```
vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr)  
{  
■vma_t *v = pa->vmas;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 136) - [public] if

```
■   if (addr >= v->start && addr < v->end) {  
■       return v;  
■   }  
  
■   v = v->next;  
■}  
  
■return NULL;  
}
```

```
static uint64_t find_gap(proc_aspace_t *pa, uint64_t len, uint64_t hint)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 163) - [public] if

```
■   if (v->end <= cand) {  
■       v = v->next;  
■       continue;  
■   }  
  
■   if (cand <= USER_MMAP_MAX - len && cand + len <= v->start) {  
■       return cand;  
■   }  
  
■   cand = arch_page_align_up(v->end);  
  
■   if (cand > USER_MMAP_MAX - len) {  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 168) - [public] if

```
■   if (cand <= USER_MMAP_MAX - len && cand + len <= v->start) {  
■       return cand;  
■   }  
  
■   cand = arch_page_align_up(v->end);  
  
■   if (cand > USER_MMAP_MAX - len) {  
■       return 0;  
■   }  
  
■   v = v->next;  
■}  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 174) - [public] if

```
■    if (cand > USER_MMAP_MAX - len) {  
■        return 0;  
■    }  
  
■    v = v->next;  
■}  
  
■return cand;  
}
```

```
proc_aspace_t *aspace_create(void)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 199) - [public] heap_free

```
■    heap_free(pa);  
■    return NULL;  
■}  
  
■pa->brk_base = USER_BRK_BASE;  
■pa->brk = USER_BRK_BASE;  
■pa->brk_max = USER_BRK_MAX;  
  
■uint64_t stack_start = USER_STACK_TOP - USER_STACK_SIZE;  
  
■vma_t *sv = (vma_t *)heap_alloc(sizeof(vma_t));  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 212) - [public]
paging_aspace_destroy

```
■    paging_aspace_destroy(pa->as);  
■    heap_free(pa);  
■    return NULL;  
■}  
  
■sv->start = stack_start;  
■sv->end = USER_STACK_TOP;  
■sv->prot = PROT_READ | PROT_WRITE;  
■sv->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;  
■sv->next = NULL;  
  
■pa->vmass = sv;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 213) - [public] heap_free

```
■    heap_free(pa);  
■    return NULL;  
■}  
  
■sv->start = stack_start;  
■sv->end = USER_STACK_TOP;  
■sv->prot = PROT_READ | PROT_WRITE;  
■sv->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;  
■sv->next = NULL;  
  
■pa->vmass = sv;  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 226) - [public] heap_free

```
■ heap_free(sv);
■ paging_aspace_destroy(pa->as);
■ heap_free(pa);
■ return NULL;
■ }
```

```
■ return pa;
}
```

```
void aspace_destroy(proc_aspace_t *pa)
```

```
{
■ if (pa == NULL) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 227) - [public]

paging_aspace_destroy

```
■ paging_aspace_destroy(pa->as);
■ heap_free(pa);
■ return NULL;
■ }
```

```
■ return pa;
}
```

```
void aspace_destroy(proc_aspace_t *pa)
```

```
{
■ if (pa == NULL) {
■     return;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 228) - [public] heap_free

```
■ heap_free(pa);
■ return NULL;
■ }
```

```
■ return pa;
}
```

```
void aspace_destroy(proc_aspace_t *pa)
```

```
{
■ if (pa == NULL) {
■     return;
■ }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 246) - [public] unmap_pages_free

```
■ unmap_pages_free(pa, v->start, v->end);
```

```
■ pa->vmass = v->next;
■ heap_free(v);
■ }
```

```
■ paging_aspace_destroy(pa->as);
```

```
■ heap_free(pa);
```

```
■ as_unlock();
```

```
}
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 249) - [public] heap_free

```
■ heap_free(v);
■}

■paging_aspace_destroy(pa->as);

■heap_free(pa);

■as_unlock();
}

address_space_t *aspace_paging_handle(proc_aspace_t *pa)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 281) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■vma_t *v = (vma_t *)heap_alloc(sizeof(vma_t));

■if (v == NULL) {
■    as_unlock();
■    return 0;
■}

■v->start = at;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 288) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■v->start = at;
■v->end = at + bytes;
■v->prot = prot;
■v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■vma_insert(pa, v);

■if (!map_anon_pages(pa, at, at + bytes, prot)) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 300) - [public] vma_remove

```
■ vma_remove(pa, v);
■ heap_free(v);
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 301) - [public] heap_free

```
■ heap_free(v);
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 302) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    ■if (pa == NULL || len == 0) {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 330) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■v->start = a;
■v->end = b;
■v->prot = prot;
■v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■vma_insert(pa, v);

■if (!map_anon_pages(pa, a, b, prot)) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 342) - [public] vma_remove

```
■ vma_remove(pa, v);
■ heap_free(v);
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return a;
}

bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 343) - [public] heap_free

```
■ heap_free(v);
■ as_unlock();
■ return 0;
■ }
```

```
■ as_unlock();
```

```
■ return a;
}
```

```
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 344) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■ }
```

```
■ as_unlock();
```

```
■ return a;
}
```

```
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    ■ if (pa == NULL || len == 0) {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 371) - [public] while

```
■ while (v != NULL) {
■     if (v->start < b && v->end > a) {
■         hit = v;
■         break;
■     }

■     v = v->next;
■ }

■ if (hit == NULL) {
■     break;
■ }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 372) - [public] if

```
■ if (v->start < b && v->end > a) {
■     hit = v;
■     break;
■ }

■ v = v->next;
■ }

■ if (hit == NULL) {
■     break;
■ }

    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/process/address_space.c (line 380) - [public] if

```
■   if (hit == NULL) {
■       break;
■   }

■   uint64_t s = hit->start > a ? hit->start : a;
■   uint64_t e = hit->end < b ? hit->end : b;

■   bool needs_split = (hit->start < s && hit->end > e);
■   vma_t *tail = NULL;

■   if (needs_split) {
■       tail = (vma_t *)heap_alloc(sizeof(vma_t));
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 390) - [public] if

```
■   if (needs_split) {
■       tail = (vma_t *)heap_alloc(sizeof(vma_t));

■       if (tail == NULL) {
■           as_unlock();
■           return false;
■       }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 393) - [public] if

```
■       if (tail == NULL) {
■           as_unlock();
■           return false;
■       }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       tail->start = e;
■       tail->end = hit->end;
■       tail->prot = hit->prot;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 394) - [public] as_unlock

```
■       as_unlock();
■       return false;
■   }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       tail->start = e;
■       tail->end = hit->end;
■       tail->prot = hit->prot;
■       tail->flags = hit->flags;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 399) - [public] unmap_pages_free

```
■ unmap_pages_free(pa, s, e);

■ if (needs_split) {
■     tail->start = e;
■     tail->end = hit->end;
■     tail->prot = hit->prot;
■     tail->flags = hit->flags;
■     tail->next = hit->next;

■     hit->next = tail;
■     hit->end = s;
■ } else if (hit->start < s) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 401) - [public] if

```
■ if (needs_split) {
■     tail->start = e;
■     tail->end = hit->end;
■     tail->prot = hit->prot;
■     tail->flags = hit->flags;
■     tail->next = hit->next;

■     hit->next = tail;
■     hit->end = s;
■ } else if (hit->start < s) {
■     hit->end = s;
■ } else if (hit->end > e) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 415) - [public] vma_remove

```
■ vma_remove(pa, hit);
■ heap_free(hit);
■ }
■}

■as_unlock();

■return true;
}

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
■     uint32_t checked_prot,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 416) - [public] heap_free

```
■ heap_free(hit);
■ }
■}

■as_unlock();

■return true;
}

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
■     uint32_t checked_prot,
■     uint32_t apply_prot,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 445) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■if (allowed != NULL && !allowed(v->prot, checked_prot)) {
■ as_unlock();
■ return false;
■}

■if (!paging_aspace_protect(pa->as, a, b - a, apply_prot)) {
■ as_unlock();
■ return false;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 450) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■if (!paging_aspace_protect(pa->as, a, b - a, apply_prot)) {
■ as_unlock();
■ return false;
■}

■v->prot = apply_prot;

■as_unlock();
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 455) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■v->prot = apply_prot;

■as_unlock();

■return true;
}

bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 468) - [public]

aspace_protect_checked

```
■return aspace_protect_checked(pa, addr, len, prot, prot, NULL);
}

uint64_t aspace_stack_base(void)
{
■return USER_STACK_TOP - USER_STACK_SIZE;
}

uint64_t aspace_reserve_at(proc_aspace_t *pa,
■ uint64_t addr,
■ size_t len,
■ uint32_t flags)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 496) - [public] if

```
■    if (v->start < b && v->end > a) {
■        as_unlock();
■        return 0;
■    }

■    v = v->next;
■}

■vma_t *nv = (vma_t *)heap_alloc(sizeof(vma_t));

■if (nv == NULL) {
■    as_unlock();
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 497) - [public] as_unlock

```
■        as_unlock();
■        return 0;
■    }

■    v = v->next;
■}

■vma_t *nv = (vma_t *)heap_alloc(sizeof(vma_t));

■if (nv == NULL) {
■    as_unlock();
■    return 0;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 507) - [public] as_unlock

```
■    as_unlock();
■    return 0;
■}

■nv->start = a;
■nv->end = b;
■nv->prot = 0;
■nv->flags = flags;

■vma_insert(pa, nv);

■as_unlock();
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 533) - [public] as_unlock

```
■    as_unlock();
■    return cur;
■}

■if (new_brk < pa->brk_base || new_brk > pa->brk_max) {
■    as_unlock();
■    return pa->brk;
■}

■uint64_t nb = arch_page_align_up(new_brk);

■if (nb > pa->brk) {
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 538) - [public] as_unlock

```
■   as_unlock();
■   return pa->brk;
■}

■uint64_t nb = arch_page_align_up(new_brk);

■if (nb > pa->brk) {
■   if (!map_anon_pages(pa, pa->brk, nb, PROT_READ | PROT_WRITE)) {
■       as_unlock();
■       return pa->brk;
■   }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 545) - [public] if

```
■   if (!map_anon_pages(pa, pa->brk, nb, PROT_READ | PROT_WRITE)) {
■       as_unlock();
■       return pa->brk;
■   }

■   vma_t *v = aspace_vma_find(pa, pa->brk_base);

■   if (v == NULL) {
■       v = (vma_t *)heap_alloc(sizeof(vma_t));

■       if (v == NULL) {
■           unmap_pages_free(pa, pa->brk, nb);
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 546) - [public] as_unlock

```
■       as_unlock();
■       return pa->brk;
■   }

■   vma_t *v = aspace_vma_find(pa, pa->brk_base);

■   if (v == NULL) {
■       v = (vma_t *)heap_alloc(sizeof(vma_t));

■       if (v == NULL) {
■           unmap_pages_free(pa, pa->brk, nb);
■           as_unlock();
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 552) - [public] if

```
■   if (v == NULL) {
■       v = (vma_t *)heap_alloc(sizeof(vma_t));

■       if (v == NULL) {
■           unmap_pages_free(pa, pa->brk, nb);
■           as_unlock();
■           return pa->brk;
■       }

■       v->start = pa->brk_base;
■       v->end = nb;
■       v->prot = PROT_READ | PROT_WRITE;
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 555) - [public] if

```
■    if (v == NULL) {
■        unmap_pages_free(pa, pa->brk, nb);
■        as_unlock();
■        return pa->brk;
■    }

■    v->start = pa->brk_base;
■    v->end = nb;
■    v->prot = PROT_READ | PROT_WRITE;
■    v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■    vma_insert(pa, v);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 556) - [public] unmap_pages_free

```
■        unmap_pages_free(pa, pa->brk, nb);
■        as_unlock();
■        return pa->brk;
■    }

■    v->start = pa->brk_base;
■    v->end = nb;
■    v->prot = PROT_READ | PROT_WRITE;
■    v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■    vma_insert(pa, v);
■    } else {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 557) - [public] as_unlock

```
■        as_unlock();
■        return pa->brk;
■    }

■    v->start = pa->brk_base;
■    v->end = nb;
■    v->prot = PROT_READ | PROT_WRITE;
■    v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■    vma_insert(pa, v);
■    } else {
■        v->end = nb;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 566) - [public] vma_insert

```
■    vma_insert(pa, v);
■    } else {
■        v->end = nb;
■    }

■    pa->brk = nb;
■} else if (nb < pa->brk) {
■    aspace_unmap(pa, nb, pa->brk - nb);
■    pa->brk = nb;
■}

■uint64_t cur = pa->brk;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 573) - [public] aspace_unmap

```
■    aspace_unmap(pa, nb, pa->brk - nb);
■    pa->brk = nb;
■}

■uint64_t cur = pa->brk;

■as_unlock();

■return cur;
}
```

kernel_Workspace/kernel/src/mm/process/mmap.c (line 17) - [public] aspace_map_at

```
    return aspace_map_at(pa, addr, len, prot);
}
```

```
return aspace_map_anon(pa, addr, len, prot);
}
```

```
bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/mmap.c (line 20) - [public] aspace_map_anon

```
return aspace_map_anon(pa, addr, len, prot);
}
```

```
bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    return aspace_protect_checked(pa, addr, len,
                                   prot, perm_sanitize(prot),
                                   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/mmap.c (line 25) - [public] aspace_unmap

```
return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    return aspace_protect_checked(pa, addr, len,
                                   prot, perm_sanitize(prot),
                                   perm_mprotect_allowed);
}
```

kernel_Workspace/kernel/src/mm/protection/guard.c (line 6) - [public] aspace_reserve_at

```
return aspace_reserve_at(pa, addr, len, VMA_FLAG_GUARD);
}
```

```
bool guard_user_stack(proc_aspace_t *pa)
{
    uint64_t stack_base = aspace_stack_base();
```

```
    if (stack_base < ARCH_PAGE_SIZE) {
        return false;
    }
```

```
return aspace_reserve_at(pa,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 5) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);
```

```
static void cpuid(uint32_t leaf,
    uint32_t subleaf,
    uint32_t *a,
    uint32_t *b,
    uint32_t *c,
    uint32_t *d)
{
    __asm__ volatile(
        "cpuid"
        : "=a"(*a), "=b"(*b), "=c"(*c), "=d"(*d)
        // ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 24) - [public] volatile
    ■__asm__ volatile("mov %%cr4, %0" : "=r"(v));

    ■return v;
    }

    static void write_cr4(uint64_t v)
    {
    ■__asm__ volatile("mov %0, %%cr4" :: "r"(v) : "memory");
    }

    bool isolation_has_smep(void)
    {
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/protection/isolation.c (line 31) - [public] volatile
    ■__asm__ volatile("mov %0, %%cr4" :: "r"(v) : "memory");
    }

    bool isolation_has_smep(void)
    {
    ■uint32_t a, b, c, d;

    ■cpuid(7, 0, &a, &b, &c, &d);

    ■return (b >> 7) & 1;
    }

        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/protection/isolation.c (line 81) - [public] if
    ■    if (!pml_entry_present(e)) {
    ■        continue;
    ■    }

    ■    if (e & PTE_USER) {
    ■        violations++;
    ■    }

    ■    if (!pml_entry_large(e) && level > PML_LEVEL_PT) {
    ■        violations =
    ■            audit_tables(pml_entry_addr(e), level - 1, violations);
    ■    }
    ■    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/protection/isolation.c (line 85) - [public] if
    ■    if (e & PTE_USER) {
    ■        violations++;
    ■    }

    ■    if (!pml_entry_large(e) && level > PML_LEVEL_PT) {
    ■        violations =
    ■            audit_tables(pml_entry_addr(e), level - 1, violations);
    ■    }
    ■}

    ■return violations;
    }
    // ... (truncated, see source file for more)

```


kernel_workspace/kernel/src/mm/protection/isolation.c (line 89) - [public] if

```
■ if (!pml_entry_large(e) && level > PML_LEVEL_PT) {  
■     violations =  
■         audit_tables(pml_entry_addr(e), level - 1, violations);  
■ }  
■}  
  
■return violations;  
}
```

```
size_t isolation_audit_kernel(void)  
{  
■uint64_t pml4 = paging_read_cr3();  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 91) - [public] audit_tables

```
■         audit_tables(pml_entry_addr(e), level - 1, violations);  
■     }  
■}  
  
■return violations;  
}
```

```
size_t isolation_audit_kernel(void)  
{  
■uint64_t pml4 = paging_read_cr3();  
■uint64_t *t = pml_table_ptr(pml4);  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 108) - [public] if

```
■ if (!pml_entry_present(e)) {  
■     continue;  
■ }  
  
■ if (e & PTE_USER) {  
■     violations++;  
■ }  
  
■ if (!pml_entry_large(e)) {  
■     violations =  
■         audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);  
■ }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 112) - [public] if

```
■ if (e & PTE_USER) {  
■     violations++;  
■ }  
  
■ if (!pml_entry_large(e)) {  
■     violations =  
■         audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);  
■ }  
■}  
  
■return violations;  
}  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 116) - [public] if

```
■    if (!pml_entry_large(e)) {
■        violations =
■            audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);
■    }
■}

■return violations;
}

bool isolation_init(void)
{
■paging_enable_nx();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 118) - [public] audit_tables

```
■        audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);
■    }
■}

■return violations;
}

bool isolation_init(void)
{
■paging_enable_nx();
■isolation_enable_smep();
■isolation_enable_smap();
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 3) - [public] k_audio_play

```
extern int32_t k_audio_play(uint64_t phys, uint32_t len);
extern int32_t k_audio_stop(void);
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
        // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 4) - [public] k_audio_stop

```
extern int32_t k_audio_stop(void);
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }
    // ... (truncated, see source file for more)
```

```

kstd/src/kernel/audio.c (line 5) - [public] k_audio_jack
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    // ... (truncated, see source file for more)
}

kstd/src/kernel/audio.c (line 6) - [public] k_audio_amp
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
    // ... (truncated, see source file for more)
}

kstd/src/kernel/audio.c (line 7) - [public] kvirt_to_phys
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
}
// ... (truncated, see source file for more)

kstd/src/kernel/audio.c (line 13) - [public] if
    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
}

tr_status_t tr_audio_stop(void)
{
    k_audio_stop();
    return TR_OK;
}
// ... (truncated, see source file for more)

```

kstd/src/kernel/audio.c (line 22) - [public] k_audio_stop

```
    k_audio_stop();
    return TR_OK;
}

tr_status_t tr_audio_jack(bool *present)
{
    if (!present) {
        return TR_ERR_INVALID;
    }

    *present = k_audio_jack() == 1;
    return TR_OK;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 28) - [public] if

```
    if (!present) {
        return TR_ERR_INVALID;
    }

    *present = k_audio_jack() == 1;
    return TR_OK;
}

tr_status_t tr_audio_amp(bool on)
{
    k_audio_amp(on ? 1 : 0);
    return TR_OK;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 38) - [public] k_audio_amp

```
    k_audio_amp(on ? 1 : 0);
    return TR_OK;
}
```

kstd/src/kernel/bt.c (line 3) - [public] bt_ready

```
extern bool bt_ready(void);
extern void bt_info(uint8_t *ver, uint8_t *addr);
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 4) - [public] bt_info

```
extern void bt_info(uint8_t *ver, uint8_t *addr);
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    // ... (truncated, see source file for more)
```

```
kstd/src/kernel/bt.c (line 5) - [public] bt_hci_cmd
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    // ... (truncated, see source file for more)
}
```

```
kstd/src/kernel/bt.c (line 6) - [public] bt_event_poll
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
    // ... (truncated, see source file for more)
}
```

```
kstd/src/kernel/bt.c (line 7) - [public] bt_acl_send
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}
// ... (truncated, see source file for more)
```

```
kstd/src/kernel/bt.c (line 8) - [public] bt_acl_recv
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}
// ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 12) - [public] if

```
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}

tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len)
{
    return bt_hci_cmd(opcode, params, len) ? TR_OK : TR_ERR_IO;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 16) - [public] bt_info

```
    bt_info(hci_ver, addr6);
    return TR_OK;
}

tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len)
{
    return bt_hci_cmd(opcode, params, len) ? TR_OK : TR_ERR_IO;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
{
    return bt_event_poll(buf, len) ? TR_OK : TR_ERR_TIMEOUT;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/fs.c (line 3) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern int32_t k_fs_exists(const char *path);

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    int32_t n = k_fs_read(path, buf, cap);

    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        // ... (truncated, see source file for more)
    }
}
```

kstd/src/kernel/fs.c (line 4) - [public] k_fs_exists

```
extern int32_t k_fs_exists(const char *path);

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    int32_t n = k_fs_read(path, buf, cap);

    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        *out = (uint32_t)n;
        // ... (truncated, see source file for more)
    }
}
```

```

kstd/src/kernel/fs.c (line 10) - [public] if
    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        *out = (uint32_t)n;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
    // ... (truncated, see source file for more)

kstd/src/kernel/fs.c (line 14) - [public] if
    if (out) {
        *out = (uint32_t)n;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
{
    return k_fs_exists(path) == 1 ? TR_OK : TR_ERR_NOTFOUND;
}

kstd/src/kernel/input.c (line 3) - [public] k_input_key
extern int32_t k_input_key(void);

int32_t tr_input_key(void)
{
    return k_input_key();
}

kstd/src/kernel/input.c (line 7) - [public] k_input_key
    return k_input_key();
}

kstd/src/kernel/log.c (line 3) - [public] kprintf
extern void kprintf(const char *fmt, ...);

void tr_log(const char *s)
{
    kprintf("%s\n", s);
}

kstd/src/kernel/log.c (line 7) - [public] kprintf
    kprintf("%s\n", s);
}

kstd/src/kernel/mem.c (line 4) - [public] kfree
extern void kfree(void *ptr);
extern bool vmm_map_device(uint64_t phys, unsigned long len, uint64_t *virt);
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    // ... (truncated, see source file for more)
}

```

```

kstd/src/kernel/mem.c (line 5) - [public] vmm_map_device
extern bool vmm_map_device(uint64_t phys, unsigned long len, uint64_t *virt);
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 6) - [public] kvirt_to_phys
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
}
// ... (truncated, see source file for more)

kstd/src/kernel/mem.c (line 10) - [public] kmalloc
return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t v = 0;
    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 16) - [public] kfree
kfree(ptr);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t v = 0;

    if (vmm_map_device(phys, len, &v)) {
        *out_va = (void *)v;
        return TR_OK;
    }

    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 23) - [public] if
if (vmm_map_device(phys, len, &v)) {
    *out_va = (void *)v;
    return TR_OK;
}

return TR_ERR_IO;
}

```


kstd/src/kernel/video.c (line 8) - [public] tr_video_init

```
tr_status_t tr_video_init(void) {
    if (dp_ready()) {
        use_dp = true;
        return TR_OK;
    }
    if (hdmi_ready()) {
        use_dp = false;
        return TR_OK;
    }
    return TR_ERR_IO;
}
```

// ... (truncated, see source file for more)

kstd/src/kernel/video.c (line 9) - [public] if

```
if (dp_ready()) {
    use_dp = true;
    return TR_OK;
}
if (hdmi_ready()) {
    use_dp = false;
    return TR_OK;
}
return TR_ERR_IO;
}
```

```
bool tr_video_is_ready(void) {
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 13) - [public] if

```
if (hdmi_ready()) {
    use_dp = false;
    return TR_OK;
}
return TR_ERR_IO;
}
```

```
bool tr_video_is_ready(void) {
    return use_dp ? dp_ready() : hdmi_ready();
}
```

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 20) - [public] tr_video_is_ready

```
bool tr_video_is_ready(void) {
    return use_dp ? dp_ready() : hdmi_ready();
}
```

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    if (!caps) return TR_ERR_INVALID;

    if (use_dp) {
        dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
        caps->ready = dp_ready();
    } else {
        hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
        // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 24) - [public] tr_video_get_caps

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    if (!caps) return TR_ERR_INVALID;

    if (use_dp) {
        dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
        caps->ready = dp_ready();
    } else {
        hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
        caps->ready = hdmi_ready();
    }
    return TR_OK;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 27) - [public] if

```
if (use_dp) {
    dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
    caps->ready = dp_ready();
} else {
    hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
    caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 28) - [public] dp_caps

```
dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
caps->ready = dp_ready();
} else {
    hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
    caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;

    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 31) - [public] hdmi_caps_raw

```
hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;

    uint64_t seq = 0;
    if (use_dp) {
        seq = dp_submit_fill(color, x, y, w, h);
    }
    // ... (truncated, see source file for more)
```

```

kstd/src/kernel/video.c (line 37) - [public] tr_video_fill
    tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
        if (!tr_video_is_ready()) return TR_ERR_IO;

        uint64_t seq = 0;
        if (use_dp) {
            seq = dp_submit_fill(color, x, y, w, h);
        } else {
            seq = hdmi_submit_fill(color, x, y, w, h);
        }

        return (seq > 0) ? TR_OK : TR_ERR_BUSY;
    }
    // ... (truncated, see source file for more)

kstd/src/kernel/video.c (line 41) - [public] if
    if (use_dp) {
        seq = dp_submit_fill(color, x, y, w, h);
    } else {
        seq = hdmi_submit_fill(color, x, y, w, h);
    }

    return (seq > 0) ? TR_OK : TR_ERR_BUSY;
}

/* ... reszta funkcji mapuje 1:1 na hdmi/dp ... */

kstd/src/user/audio.c (line 7) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static void *bounce = TR_NULL;
static uint64_t bounce_phys = 0;

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            // ... (truncated, see source file for more)
        }
    }
}

kstd/src/user/audio.c (line 8) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static void *bounce = TR_NULL;
static uint64_t bounce_phys = 0;

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            return TR_ERR_NOMEM;
        }
        // ... (truncated, see source file for more)
    }
}

kstd/src/user/audio.c (line 15) - [public] if
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            return TR_ERR_NOMEM;
        }
    }

    uint64_t id = ds_call(SVC_SYS, OP_PAGEPHYS, (uint64_t)bounce, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 18) - [public] if
    if (bounce == TR_NULL) {
        return TR_ERR_NOMEM;
    }

    uint64_t id = ds_call(SVC_SYS, OP_PAGEPHYS, (uint64_t)bounce, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 23) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    bounce_phys = r.arg0;
}

if (len > 4096 * 4) {
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 27) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    bounce_phys = r.arg0;
}

if (len > 4096 * 4) {
    len = 4096 * 4;
}

uint8_t *dst = (uint8_t *)bounce;
// ... (truncated, see source file for more)

kstd/src/user/audio.c (line 34) - [public] if
    if (len > 4096 * 4) {
        len = 4096 * 4;
    }

    uint8_t *dst = (uint8_t *)bounce;
    const uint8_t *src = (const uint8_t *)data;

    for (uint32_t i = 0; i < len; i++) {
        dst[i] = src[i];
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_PLAY, bounce_phys, len, 0);
    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 46) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_audio_stop(void)
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 50) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_audio_stop(void)
{
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
    return TR_OK;
}
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 59) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
    return TR_OK;
}

tr_status_t tr_audio_jack(bool *present)
{
    if (!present) {
        return TR_ERR_INVALID;
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 65) - [public] if
    if (!present) {
        return TR_ERR_INVALID;
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 70) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }

    return TR_ERR_IO;
}

// ... (truncated, see source file for more)

kstd/src/user/audio.c (line 74) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }

    return TR_ERR_IO;
}

tr_status_t tr_audio_amp(bool on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
    return TR_OK;
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 84) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
    return TR_OK;
}

kstd/src/user/bt.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    uint64_t id = ds_call(SVC_BT, BT_INFO, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    uint64_t id = ds_call(SVC_BT, BT_INFO, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        // ... (truncated, see source file for more)
    }
}

```

```

kstd/src/user/bt.c (line 14) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    *hci_ver = (uint8_t)r.arg0;

    for (int i = 0; i < 6; i++) {
        addr6[i] = (uint8_t)(r.arg1 >> (i * 8));
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 18) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    *hci_ver = (uint8_t)r.arg0;

    for (int i = 0; i < 6; i++) {
        addr6[i] = (uint8_t)(r.arg1 >> (i * 8));
    }

    return TR_OK;
}
// ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 40) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 44) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
{
    uint64_t id = ds_call(SVC_BT, BT_EVT, 0, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 54) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint8_t n = (uint8_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 58) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint8_t n = (uint8_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    for (uint8_t i = 0; i < n; i++) {
        dst[i] = scratch[i];
    }

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 83) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_BUSY;
}

tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len)
    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 87) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_BUSY;
}

tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len)
{
    uint64_t id = ds_call(SVC_BT, BT_ACL_IN, 0, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```



```

kstd/src/user/bt.c (line 97) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint16_t n = (uint16_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 101) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint16_t n = (uint16_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    for (uint16_t i = 0; i < n; i++) {
        dst[i] = scratch[i];
    }

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 15) - [public] while
    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }

    scratch[n] = 0;

    uint64_t id = ds_call(SVC_FS, FS_READ, n, cap, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 23) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_NOTFOUND;
    }

    uint32_t got = (uint32_t)r.arg0;

    if (got > cap) {
        got = cap;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 27) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_NOTFOUND;
    }

    uint32_t got = (uint32_t)r.arg0;

    if (got > cap) {
        got = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 33) - [public] if
    if (got > cap) {
        got = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < got; i++) {
        dst[i] = scratch[256 + i];
    }

    if (out) {
        *out = got;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 43) - [public] if
    if (out) {
        *out = got;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        // ... (truncated, see source file for more)

kstd/src/user/fs.c (line 54) - [public] while
    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }

    scratch[n] = 0;

    uint64_t id = ds_call(SVC_FS, FS_EXISTS, n, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

kstd/src/user/fs.c (line 62) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 == 1) {
        return TR_OK;
    }

    return TR_ERR_NOTFOUND;
}

kstd/src/user/fs.c (line 66) - [public] if
    if (ds_take(id, &r) && r.status == 0 && r.arg0 == 1) {
        return TR_OK;
    }

    return TR_ERR_NOTFOUND;
}

kstd/src/user/input.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

int32_t tr_input_key(void)
{
    uint64_t id = ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
        // ... (truncated, see source file for more)
    }
}

```

```

kstd/src/user/input.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

int32_t tr_input_key(void)
{
    uint64_t id = ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }
    // ... (truncated, see source file for more)

kstd/src/user/input.c (line 12) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }

    return -1;
}

kstd/src/user/input.c (line 16) - [public] if
    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }

    return -1;
}

kstd/src/user/log.c (line 10) - [public] while
    while (s[n] && n < DS_SCRATCH_SIZE - 1) {
        scratch[n] = s[n];
        n++;
    }

    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

kstd/src/user/log.c (line 15) - [public] ds_call
    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

kstd/src/user/mem.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static uint64_t next_va = 0x44000000ULL;

void *tr_alloc(uint32_t bytes)
{
    uint32_t pages = (bytes + 4095) / 4096;

    uint64_t id = ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```

```

kstd/src/user/mem.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static uint64_t next_va = 0x44000000ULL;

void *tr_alloc(uint32_t bytes)
{
    uint32_t pages = (bytes + 4095) / 4096;

    uint64_t id = ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
    ds_poll();

    ds_msg_t r;
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 16) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return (void *)r.arg0;
    }

    return TR_NULL;
}

void tr_free(void *ptr, uint32_t bytes)
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 20) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return (void *)r.arg0;
    }

    return TR_NULL;
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    ds_call(SVC_SYS, OP_FREE, (uint64_t)ptr, 0, 0);
}
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 30) - [public] ds_call
    ds_call(SVC_SYS, OP_FREE, (uint64_t)ptr, 0, 0);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t va = next_va;
    next_va += (len + 4095) & ~4095ULL;

    uint64_t id = ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
    ds_poll();

    ds_msg_t r;
    // ... (truncated, see source file for more)

```

```

kstd/src/user/mem.c (line 39) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *out_va = (void *)va;
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

```

```

kstd/src/user/mem.c (line 43) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        *out_va = (void *)va;
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

```

```

kstd/src/user/video.c (line 6) - [public] tr_video_init
/* Zakładamy, że ds_call i ds_take są dostępne z dsclient.c */

tr_status_t tr_video_init(void) {
    /* W user-space init robi kernel, tu tylko sprawdzamy stan */
    return tr_video_is_ready() ? TR_OK : TR_ERR_IO;
}

bool tr_video_is_ready(void) {
    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_CAPS, 0, 0, 0);
    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    // ... (truncated, see source file for more)
}

```

```

kstd/src/user/video.c (line 11) - [public] tr_video_is_ready

bool tr_video_is_ready(void) {
    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_CAPS, 0, 0, 0);
    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    // ... (truncated, see source file for more)
}

```

```

kstd/src/user/video.c (line 13) - [public] ds_poll

    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    // ... (truncated, see source file for more)
}

```

```

kstd/src/user/video.c (line 15) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_FILL, color, arg1, arg2);
    ds_poll();
    // ... (truncated, see source file for more)

```

```

kstd/src/user/video.c (line 21) - [public] tr_video_fill
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_FILL, color, arg1, arg2);
    ds_poll();

    ds_msg_t r;
    if (ds_take(id, &r)) {
        if (r.status == 0) return TR_OK;
        if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/video.c (line 26) - [public] ds_poll
ds_poll();

ds_msg_t r;
if (ds_take(id, &r)) {
    if (r.status == 0) return TR_OK;
    if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
    return TR_ERR_IO;
}
return TR_ERR_TIMEOUT;
}

/* ... reszta funkcji serializuje argumenty do DsMsg ... */

```

```

kstd/src/user/video.c (line 29) - [public] if
    if (ds_take(id, &r)) {
        if (r.status == 0) return TR_OK;
        if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
        return TR_ERR_IO;
    }
    return TR_ERR_TIMEOUT;
}

/* ... reszta funkcji serializuje argumenty do DsMsg ... */

```

```

libs/dsclient.c (line 30) - [public] if
    if (next == d2k->tail) {
        return 0;
    }

    volatile ds_msg_t *m = &d2k_msgs[head];

    m->id = id;
    m->cmd = svc_cmd(class, op);
    m->flags = 1;
    m->arg0 = a0;
    m->arg1 = a1;
    m->arg2 = a2;
    // ... (truncated, see source file for more)

```

```

libs/dsclient.c (line 51) - [public] while
    while (k2d->tail != k2d->head) {
        volatile ds_msg_t *m = &k2d_msgs[k2d->tail];
        ds_msg_t copy = *m;

        k2d->tail = (k2d->tail + 1) % k2d->cap;

        for (int i = 0; i < 16; i++) {
            if (!cache_used[i]) {
                cache[i] = copy;
                cache_used[i] = 1;
                break;
            }
        }
        // ... (truncated, see source file for more)

libs/dsclient.c (line 58) - [public] if
        if (!cache_used[i]) {
            cache[i] = copy;
            cache_used[i] = 1;
            break;
        }
    }
}

int ds_take(uint64_t id, ds_msg_t *out)
{
    for (int i = 0; i < 16; i++) {
        // ... (truncated, see source file for more)

libs/dsclient.c (line 70) - [public] if
        if (cache_used[i] && cache[i].id == id) {
            *out = cache[i];
            cache_used[i] = 0;
            return 1;
        }
    }

    return 0;
}

// ds_init(params_va);
// uint64_t id = ds_call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0);
// ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.c (line 15) - [public] memset
    memset(buffer, (int)0ull, 16);
    buffer[0] = (uint8_t)65ull;
    buffer[1] = (uint8_t)66ull;
    buffer[2] = (uint8_t)67ull;
    r0 = buffer[0];
    r1 = buffer[1];
    r2 = r0 + r1;
    if (r2 != 131ull) goto else_1;
    r0 = 1ull;
    goto end_2;
else_1:
    r0 = 0ull;
    // ... (truncated, see source file for more)

```

C (header)


```

drivers/vgpu/vgpu.h (line 22) - [public] vgpu_init
    bool vgpu_init(uint32_t w, uint32_t h);
    vgpu_info_t vgpu_get_info(void);

    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 23) - [public] vgpu_get_info
    vgpu_info_t vgpu_get_info(void);

    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 25) - [public] vgpu_clear
    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 26) - [public] vgpu_pixel
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 28) - [public] vgpu_surface_create
    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 29) - [public] vgpu_surface_present
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 31) - [public] vgpu_process_ring
    void vgpu_process_ring(volatile void *ring);

#endif

```

kernel_Workspace/kernel/src/battery/battery.h (line 21) - [public] battery_init

```
bool battery_init(void);
bool battery_present(void);
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 22) - [public] battery_present

```
bool battery_present(void);
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 23) - [public] battery_status

```
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 24) - [public] battery_status_packed

```
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 25) - [public] battery_set_threshold

```
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 26) - [public] battery_threshold

```
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 27) - [public] battery_low_pending

```
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 28) - [public] battery_sim_tick

```
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 7) - [public] bool

```
    bool (*present)(void);
    bool (*status)(battery_status_t *out);
} battery_backend_t;

void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 8) - [public] bool

```
    bool (*status)(battery_status_t *out);
} battery_backend_t;

void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 11) - [public] battery_register_backend

```
void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 8) - [public] bt_init

```
bool bt_init(void);
bool bt_ready(void);
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 9) - [public] bt_ready

```
bool bt_ready(void);
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 10) - [public] bt_info

```
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 11) - [public] bt_hci_cmd

```
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 12) - [public] bt_event_poll

```
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 13) - [public] bt_acl_send
    bool bt_acl_send(const uint8_t *data, uint16_t len);
    bool bt_acl_rcv(uint8_t *data, uint16_t cap, uint16_t *len);

    #endif

kernel_Workspace/kernel/src/bluetooth/bt.h (line 14) - [public] bt_acl_rcv
    bool bt_acl_rcv(uint8_t *data, uint16_t cap, uint16_t *len);

    #endif

kernel_Workspace/kernel/src/bluetooth/operation.h (line 11) - [public] bt_op_model_init
    void bt_op_model_init(void);
    bool bt_op_model_run_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

    #endif

kernel_Workspace/kernel/src/bluetooth/operation.h (line 12) - [public] bt_op_model_run_cmd
    bool bt_op_model_run_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 19) - [public] camera_init
    bool camera_init(void);
    bool camera_present(void);
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 20) - [public] camera_present
    bool camera_present(void);
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 21) - [public] camera_caps_get
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 22) - [public] camera_start
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 23) - [public] camera_stop
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

```

```

kernel_Workspace/kernel/src/camera/camera.h (line 24) - [public] camera_frame
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 25) - [public] camera_frame_to_phys
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/operation.h (line 7) - [public] bool
    bool (*present)(void);
    bool (*start)(void);
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 8) - [public] bool
    bool (*start)(void);
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 9) - [public] bool
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 10) - [public] bool
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 13) - [public] camera_register_backend
void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 10) - [public] caps_self_bits
unsigned int caps_self_bits(void);

unsigned int caps_world_bits(unsigned int world_id);

int caps_has(unsigned int world_id, unsigned char cap_id);

int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

int caps_request(unsigned int target, unsigned char cap_id);

int caps_release(unsigned int world_id, unsigned char cap_id);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/caps/caps.h (line 12) - [public] caps_world_bits
    unsigned int caps_world_bits(unsigned int world_id);

    int caps_has(unsigned int world_id, unsigned char cap_id);

    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/caps.h (line 14) - [public] caps_has
    int caps_has(unsigned int world_id, unsigned char cap_id);

    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/caps.h (line 16) - [public] caps_name
    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 18) - [public] caps_request
    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 20) - [public] caps_release
    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 22) - [public] caps_world_count
    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

```

```

kernel_Workspace/kernel/src/caps/caps.h (line 24) - [public] caps_audit_count
    unsigned long long caps_audit_count(void);

    #endif

kernel_Workspace/kernel/src/core-lang/ast.h (line 50) - [public] cl_arena_init
    void cl_arena_init(arena_t *a, uint8_t *buf, size_t cap);
    void *cl_arena_alloc(arena_t *a, size_t n);
    ast_node_t *cl_node(arena_t *ar, uint8_t kind, uint32_t line);

    #endif

kernel_Workspace/kernel/src/core-lang/bridge.h (line 6) - [public] cl_bridge_init
    int cl_bridge_init(cl_vm_t *vm, uint8_t ring);

    int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn);

    #endif

kernel_Workspace/kernel/src/core-lang/bridge.h (line 8) - [public] cl_bridge_add
    int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 16) - [public] cl_lexer_init
    void cl_lexer_init(lexer_t *l, const char *src, size_t len);

    bool cl_lex_next(lexer_t *l, token_t *out);

    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 18) - [public] cl_lex_next
    bool cl_lex_next(lexer_t *l, token_t *out);

    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 20) - [public] cl_lex_all
    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/loader.h (line 15) - [public] cl_save_bc
    size_t cl_save_bc(cl_prog_t *P, uint8_t *buf, size_t cap);
    cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar);

    cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native);

    #endif

kernel_Workspace/kernel/src/core-lang/native.h (line 9) - [public] uint64_t
    /* signature wygenerowanej funkcji:
    uint64_t fn(uint64_t *slots, uint64_t *globals, cl_ext_fn *externs) */

    typedef uint64_t (*cl_native_fn)(uint64_t *, uint64_t *, cl_ext_fn *);

    /* Zwraca bufor z kodem (RW) albo NULL gdy arch nie wspiera subsetu.
    Wywołujący robi go executable (mprotect / mapowanie WX w kernelu). */
    void *cl_native_compile(cl_prog_t *P);
    size_t cl_native_size(void);

    #endif

kernel_Workspace/kernel/src/core-lang/native.h (line 14) - [public] cl_native_size
    size_t cl_native_size(void);

    #endif

```

kernel_Workspace/kernel/src/core-lang/sema.h (line 16) - [public] cl_sema_run

```
int cl_sema_run(sema_ctx_t *s, ast_node_t *prog);

size_t cl_sema_diag_count(const sema_ctx_t *s);
const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i);

#endif
```

kernel_Workspace/kernel/src/core-lang/sema.h (line 18) - [public] cl_sema_diag_count

```
size_t cl_sema_diag_count(const sema_ctx_t *s);
const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 44) - [public] cl_vm_init

```
void cl_vm_init(cl_vm_t *vm, cl_prog_t *prog);
int cl_vm_register_extern(cl_vm_t *vm, const char *name, cl_ext_fn fn);
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 45) - [public] cl_vm_register_extern

```
int cl_vm_register_extern(cl_vm_t *vm, const char *name, cl_ext_fn fn);
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 46) - [public] cl_vm_run

```
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
uint8_t *q, uint8_t *m);
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/vm.h (line 48) - [public] cl_wbytes

```
int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 49) - [public] cl_mask

```
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 50) - [public] cl_add

```
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 51) - [public] cl_sub

```
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 52) - [public] cl_mul

```
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 53) - [public] cl_cmp

```
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

```

kernel_Workspace/kernel/src/core-lang/vm.h (line 54) - [public] cl_shl
    void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
                  uint8_t *q, uint8_t *m);

    #endif

kernel_Workspace/kernel/src/core-lang/vm.h (line 55) - [public] cl_shr
    void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
                  uint8_t *q, uint8_t *m);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 17) - [public] dp_aux_sink_init
    void dp_aux_sink_init(void);
    bool dp_aux_read(uint32_t addr, void *buf, uint32_t len);
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 18) - [public] dp_aux_read
    bool dp_aux_read(uint32_t addr, void *buf, uint32_t len);
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 19) - [public] dp_aux_write
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 20) - [public] dp_aux_read_edid
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/dp.h (line 10) - [public] dp_init
    bool dp_init(void);
    bool dp_ready(void);

    void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

    uint32_t dp_mode_count(void);
    bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                   uint32_t *h, uint32_t *r);
    bool dp_mode_set_by_id(uint32_t id);

    void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/dp.h (line 11) - [public] dp_ready
    bool dp_ready(void);

    void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

    uint32_t dp_mode_count(void);
    bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                   uint32_t *h, uint32_t *r);
    bool dp_mode_set_by_id(uint32_t id);

    void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 13) - [public] dp_link_info
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

uint32_t dp_mode_count(void);
bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                uint32_t *h, uint32_t *r);
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 15) - [public] dp_mode_count
uint32_t dp_mode_count(void);
bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                uint32_t *h, uint32_t *r);
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 18) - [public] dp_mode_set_by_id
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 20) - [public] dp_caps
void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 21) - [public] dp_fb_set
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/link.h (line 7) - [public] dp_link_train
bool dp_link_train(void);
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

#endif
```

```
kernel_Workspace/kernel/src/displayport/link.h (line 8) - [public] dp_link_info
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

#endif
```

```

kernel_Workspace/kernel/src/displayport/operation.h (line 6) - [public] dp_op_set_fb
    void dp_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    void dp_op_state(uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/displayport/operation.h (line 7) - [public] dp_op_state
    void dp_op_state(uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/displayport/operation.h (line 8) - [public] dp_op_fill
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/editor/editor.h (line 21) - [public] editor_run
    int editor_run(const char *path);

    #endif

kernel_Workspace/kernel/src/editor/mouse.h (line 6) - [public] mouse_init
    void mouse_init(void);

    int mouse_poll(int *dx, int *dy, int *dz, int *buttons);

    #endif

kernel_Workspace/kernel/src/editor/mouse.h (line 8) - [public] mouse_poll
    int mouse_poll(int *dx, int *dy, int *dz, int *buttons);

    #endif

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 41) - [public] hdmi_init_with
    bool hdmi_init_with(uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_ready(void);
    void hdmi_caps(hdmi_caps_t *out);
    bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_mode_set_by_id(uint32_t id);
    bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
    uint64_t hdmi_submit(const hdmi_transfer_t *t);
    bool hdmi_poll(uint64_t *out_seq);
    uint32_t hdmi_pending(void);
    uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 42) - [public] hdmi_ready
    bool hdmi_ready(void);
    void hdmi_caps(hdmi_caps_t *out);
    bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_mode_set_by_id(uint32_t id);
    bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
    uint64_t hdmi_submit(const hdmi_transfer_t *t);
    bool hdmi_poll(uint64_t *out_seq);
    uint32_t hdmi_pending(void);
    uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_fb_grant(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 43) - [public] hdmi_caps

```
void hdmi_caps(hdmi_caps_t *out);
bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 44) - [public] hdmi_fb_set

```
bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 45) - [public] hdmi_mode_set_by_id

```
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 46) - [public] hdmi_mode_current_raw

```
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 47) - [public] hdmi_submit

```
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 48) - [public] hdmi_poll

```
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 49) - [public] hdmi_pending

```
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 50) - [public] hdmi_submit_fill

```
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 51) - [public] hdmi_caps_raw

```
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 53) - [public] hdmi_fb_grant

```
bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 54) - [public] hdmi_fb_revoke

```
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 55) - [public] hdmi_fb_granted

```
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 9) - [public] hdmi_iface_acquire

```
bool hdmi_iface_acquire(uint32_t owner);
bool hdmi_iface_release(uint32_t owner);
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 10) - [public] hdmi_iface_release

```
bool hdmi_iface_release(uint32_t owner);
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 11) - [public] hdmi_iface_owner

```
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 12) - [public] hdmi_iface_check

```
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 14) - [public] hdmi_iface_init

```
bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 15) - [public] hdmi_iface_ready

```
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 17) - [public] hdmi_iface_mode_set

```
bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 18) - [public] hdmi_iface_mode_current

```
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

#endif
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 19) - [public] hdmi_iface_mode_at

```
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

#endif
```



```

kernel_Workspace/kernel/src/hdmi/interface.h (line 21) - [public] hdmi_iface_submit_fill
    uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h,
    bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 22) - [public] hdmi_iface_poll
    bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 24) - [public] hdmi_iface_caps
    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 26) - [public] hdmi_iface_fb_grant
    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 27) - [public] hdmi_iface_fb_revoke
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 23) - [public] hdmi_mode_count
    uint32_t hdmi_mode_count(void);
    const hdmi_mode_t *hdmi_mode_at(uint32_t i);
    const hdmi_mode_t *hdmi_mode_find(uint32_t w, uint32_t h, uint32_t refresh);
    const hdmi_mode_t *hdmi_mode_by_id(uint32_t id);
    bool hdmi_mode_valid(const hdmi_mode_t *m);
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 27) - [public] hdmi_mode_valid
    bool hdmi_mode_valid(const hdmi_mode_t *m);
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 28) - [public] hdmi_mode_apply
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

```

kernel_Workspace/kernel/src/hdmi/operation.h (line 7) - [public] hdmi_op_set_mode

```
void hdmi_op_set_mode(const hdmi_mode_t *m);
void hdmi_op_exec(hdmi_transfer_t *t);
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 8) - [public] hdmi_op_exec

```
void hdmi_op_exec(hdmi_transfer_t *t);
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 9) - [public] hdmi_op_state

```
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 10) - [public] hdmi_op_set_fb

```
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 11) - [public] hdmi_op_ready

```
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 13) - [public] kfree

```
void kfree(void *ptr);

size_t kmalloc_usable_size(void *ptr);

void *kalloc_pages(size_t pages);
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 15) - [public] kmalloc_usable_size

```
size_t kmalloc_usable_size(void *ptr);

void *kalloc_pages(size_t pages);
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 18) - [public] kfree_pages
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 22) - [public] kvirt_to_phys
uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 24) - [public] kalloc_dump
void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 9) - [public] dbg_free
void dbg_free(void *ptr);

bool dbg_verify(void *ptr);
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 11) - [public] dbg_verify
bool dbg_verify(void *ptr);
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 12) - [public] dbg_usable_size
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 13) - [public] mm_debug_dump
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 8) - [public] leak_track
bool leak_track(void *ptr, size_t size, uint64_t caller);

bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id);

bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
bool leak_contains(void *ptr);
size_t leak_count(void);

void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 10) - [public] leak_track_id
    bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id);

    bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 12) - [public] leak_untrack
    bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 13) - [public] leak_contains
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 14) - [public] leak_count
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 16) - [public] leak_dump
    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 8) - [public] alloc_stats_note_alloc
    void alloc_stats_note_alloc(size_t size);
    void alloc_stats_note_free(size_t size);

    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 9) - [public] alloc_stats_note_free
    void alloc_stats_note_free(size_t size);

    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif
```

```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 11) - [public] alloc_stats_live_count
    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 12) - [public] alloc_stats_live_bytes
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 13) - [public] alloc_stats_peak_bytes
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 14) - [public]
alloc_stats_total_allocs
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 15) - [public]
alloc_stats_total_frees
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 17) - [public] alloc_stats_dump
    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 8) - [public] bool
typedef bool (*buddy_map_cb)(uint64_t virt, size_t size);
typedef void (*buddy_unmap_cb)(uint64_t virt, size_t size);

bool buddy_init(uint64_t base,
                size_t size,
                buddy_map_cb map_cb,
                buddy_unmap_cb unmap_cb);

bool buddy_ready(void);

void *buddy_alloc(size_t size);
void *buddy_alloc_aligned(size_t size, size_t align);
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 9) - [public] void

```
typedef void (*buddy_unmap_cb)(uint64_t virt, size_t size);
```

```
bool buddy_init(uint64_t base,  
                size_t size,  
                buddy_map_cb map_cb,  
                buddy_unmap_cb unmap_cb);
```

```
bool buddy_ready(void);
```

```
void *buddy_alloc(size_t size);  
void *buddy_alloc_aligned(size_t size, size_t align);  
void buddy_free(void *ptr);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 16) - [public] buddy_ready

```
bool buddy_ready(void);
```

```
void *buddy_alloc(size_t size);  
void *buddy_alloc_aligned(size_t size, size_t align);  
void buddy_free(void *ptr);
```

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 20) - [public] buddy_free

```
void buddy_free(void *ptr);
```

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 22) - [public] buddy_block_size

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 24) - [public] buddy_stat_used_bytes

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 25) - [public] buddy_stat_free_bytes

```
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 27) - [public] buddy_dump

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 8) - [public] heap_init

```
bool heap_init(void);
bool heap_ready(void);

void *heap_alloc(size_t size);
void *heap_alloc_aligned(size_t size, size_t align);
void *heap_zalloc(size_t size);

void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 9) - [public] heap_ready

```
bool heap_ready(void);

void *heap_alloc(size_t size);
void *heap_alloc_aligned(size_t size, size_t align);
void *heap_zalloc(size_t size);

void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 15) - [public] heap_free

```
void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 17) - [public] heap_usable_size

```
size_t heap_usable_size(void *ptr);

void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 19) - [public] heap_dump

```
void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 10) - [public] slab_init

```
bool slab_init(uint64_t base, size_t size);
bool slab_ready(void);

void *slab_alloc(size_t size);
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 11) - [public] slab_ready

```
bool slab_ready(void);

void *slab_alloc(size_t size);
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 14) - [public] slab_free

```
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 16) - [public] slab_usable_size

```
size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 18) - [public] slab_stat_used_bytes

```
size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 19) - [public] slab_stat_free_bytes

```
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 21) - [public] slab_dump

```
void slab_dump(void);

#endif
```


kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 30) - [public]

bitmap_words_for_bits

```
size_t bitmap_words_for_bits(size_t bit_count);
size_t bitmap_bytes_for_bits(size_t bit_count);

void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 31) - [public]

bitmap_bytes_for_bits

```
size_t bitmap_bytes_for_bits(size_t bit_count);

void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 33) - [public] bitmap_init_virt

```
void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 35) - [public] bitmap_init_phys

```
bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 37) - [public] bitmap_fill

```
void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 39) - [public] bitmap_set

```
void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 40) - [public] bitmap_clear

```
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 41) - [public] bitmap_test

```
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 43) - [public] bitmap_fill
    void bitmap_fill(bitmap_t *bm, bool value);

    void bitmap_set(bitmap_t *bm, size_t bit);
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 45) - [public] bitmap_set
    void bitmap_set(bitmap_t *bm, size_t bit);
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 46) - [public] bitmap_clear
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 47) - [public] bitmap_test
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 49) - [public] bitmap_set_range

```
void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

size_t bitmap_alloc(bitmap_t *bm);

size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 50) - [public] bitmap_clear_range

```
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

size_t bitmap_alloc(bitmap_t *bm);

size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

void bitmap_free(bitmap_t *bm, size_t bit);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 52) - [public]

bitmap_test_range_free

```
bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

size_t bitmap_alloc(bitmap_t *bm);

size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

void bitmap_free(bitmap_t *bm, size_t bit);
void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 54) - [public] bitmap_alloc

```
size_t bitmap_alloc(bitmap_t *bm);

size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

void bitmap_free(bitmap_t *bm, size_t bit);
void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

size_t bitmap_count_free(const bitmap_t *bm);

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 57) - [public] bitmap_alloc_from
    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 59) - [public] bitmap_alloc_range
    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 61) - [public] bitmap_free
    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 62) - [public] bitmap_free_range
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 64) - [public] bitmap_count_free
    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 12) - [public] frame_init
    bool frame_init(uint64_t bitmap_phys, size_t bit_count);

    bool frame_init_from_memory(void);

    bool frame_ready(void);

    bool frame_alloc(frame_t *out);
    bool frame_alloc_zero(frame_t *out);

    bool frame_alloc_contiguous(size_t count,
                                size_t align_frames,
                                frame_t *out);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 14) - [public]

frame_init_from_memory

```
bool frame_init_from_memory(void);

bool frame_ready(void);

bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                       size_t align_frames,
                       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 16) - [public] frame_ready

```
bool frame_ready(void);

bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                       size_t align_frames,
                       uint64_t max_phys,
                       frame_t *out);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 18) - [public] frame_alloc

```
bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                       size_t align_frames,
                       uint64_t max_phys,
                       frame_t *out);

bool frame_free(frame_t frame);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 19) - [public] frame_alloc_zero

```
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                       size_t align_frames,
                       uint64_t max_phys,
                       frame_t *out);

bool frame_free(frame_t frame);
bool frame_free_contiguous(frame_t start, size_t count);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 29) - [public] frame_free

```
bool frame_free(frame_t frame);
bool frame_free_contiguous(frame_t start, size_t count);

bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 30) - [public]
frame_free_contiguous

```
bool frame_free_contiguous(frame_t start, size_t count);

bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 32) - [public] frame_is_valid

```
bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 33) - [public] frame_zero

```
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 36) - [public] frame_phys

```
uint64_t frame_phys(frame_t frame);
```

```
size_t frame_to_pfn(frame_t frame);  
frame_t frame_from_pfn(size_t pfn);
```

```
size_t frame_total(void);  
size_t frame_allocated(void);  
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);  
size_t frame_zone_dma32_free(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 38) - [public] frame_to_pfn

```
size_t frame_to_pfn(frame_t frame);  
frame_t frame_from_pfn(size_t pfn);
```

```
size_t frame_total(void);  
size_t frame_allocated(void);  
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);  
size_t frame_zone_dma32_free(void);  
size_t frame_zone_normal_total(void);  
size_t frame_zone_normal_free(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 39) - [public] frame_from_pfn

```
frame_t frame_from_pfn(size_t pfn);
```

```
size_t frame_total(void);  
size_t frame_allocated(void);  
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);  
size_t frame_zone_dma32_free(void);  
size_t frame_zone_normal_total(void);  
size_t frame_zone_normal_free(void);
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 41) - [public] frame_total

```
size_t frame_total(void);  
size_t frame_allocated(void);  
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);  
size_t frame_zone_dma32_free(void);  
size_t frame_zone_normal_total(void);  
size_t frame_zone_normal_free(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 42) - [public] frame_allocated

```
size_t frame_allocated(void);  
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);  
size_t frame_zone_dma32_free(void);  
size_t frame_zone_normal_total(void);  
size_t frame_zone_normal_free(void);
```

```
#endif
```



```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 43) - [public] frame_free_count
    size_t frame_free_count(void);

    size_t frame_zone_dma32_total(void);
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 46) - [public]
frame_zone_dma32_total
    size_t frame_zone_dma32_total(void);
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 47) - [public]
frame_zone_dma32_free
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 48) - [public]
frame_zone_normal_total
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 49) - [public]
frame_zone_normal_free
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 10) - [public] pmm_init
    bool pmm_init(void);

    bool pmm_ready(void);

    bool pmm_alloc_frame(uint64_t *out_phys);

    bool pmm_alloc_zero_frame(uint64_t *out_phys);

    bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

    bool pmm_alloc_frames_aligned(size_t count,
                                   size_t align_frames,
                                   // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 12) - [public] pmm_ready
    bool pmm_ready(void);

    bool pmm_alloc_frame(uint64_t *out_phys);

    bool pmm_alloc_zero_frame(uint64_t *out_phys);

    bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

    bool pmm_alloc_frames_aligned(size_t count,
                                   size_t align_frames,
                                   uint64_t *out_phys);
    bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 14) - [public] pmm_alloc_frame

```
bool pmm_alloc_frame(uint64_t *out_phys);

bool pmm_alloc_zero_frame(uint64_t *out_phys);

bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 16) - [public] pmm_alloc_zero_frame

```
bool pmm_alloc_zero_frame(uint64_t *out_phys);

bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 18) - [public] pmm_alloc_frames

```
bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);

bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 23) - [public] pmm_alloc_bytes

```
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);

bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 24) - [public] pmm_alloc_zero_bytes

```
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);
bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 28) - [public] pmm_free_frame

```
bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 33) - [public] pmm_free_frames

```
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
uint64_t pmm_stat_free_bytes(void);

size_t pmm_stat_dma32_total_frames(void);
size_t pmm_stat_dma32_free_frames(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 34) - [public] pmm_free_bytes

```
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
uint64_t pmm_stat_free_bytes(void);

size_t pmm_stat_dma32_total_frames(void);
size_t pmm_stat_dma32_free_frames(void);
size_t pmm_stat_normal_total_frames(void);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 35) - [public] pmm_stat_total_frames
    size_t pmm_stat_total_frames(void);
    size_t pmm_stat_free_frames(void);
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 36) - [public] pmm_stat_free_frames
    size_t pmm_stat_free_frames(void);
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 37) - [public]
pmm_stat_allocated_frames
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    void pmm_dump(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 39) - [public] pmm_stat_total_bytes
    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    void pmm_dump(void);

#ifdef PMM_DEBUG
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 40) - [public] pmm_stat_free_bytes

```
uint64_t pmm_stat_free_bytes(void);
```

```
size_t pmm_stat_dma32_total_frames(void);  
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 43) - [public]

pmm_stat_dma32_total_frames

```
size_t pmm_stat_dma32_total_frames(void);  
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 44) - [public]

pmm_stat_dma32_free_frames

```
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 45) - [public]

pmm_stat_normal_total_frames

```
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 46) - [public]

pmm_stat_normal_free_frames

```
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 48) - [public] pmm_dump
    void pmm_dump(void);

    #ifdef PMM_DEBUG
    bool pmm_self_test(void);
    #endif

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 51) - [public] pmm_self_test
    bool pmm_self_test(void);
    #endif

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/contiguous.h (line 13) - [public] contig_free
    void contig_free(uint64_t phys, size_t bytes);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 16) - [public] dma_free_coherent
    void dma_free_coherent(uint64_t phys, void *virt, size_t bytes);

    void dma_sync_for_device(void *virt, size_t len);
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 18) - [public] dma_sync_for_device
    void dma_sync_for_device(void *virt, size_t len);
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 19) - [public] dma_sync_for_cpu
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 12) - [public] mapping_init
    void mapping_init(void);
    mapping_space_t mapping_kernel_space(void);

    bool mapping_map_range(mapping_space_t space,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint64_t pte_flags);

    bool mapping_unmap_range(mapping_space_t space,
                             uint64_t virt,
                             size_t len);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 13) - [public]
mapping_kernel_space
    mapping_space_t mapping_kernel_space(void);

    bool mapping_map_range(mapping_space_t space,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint64_t pte_flags);

    bool mapping_unmap_range(mapping_space_t space,
                             uint64_t virt,
                             size_t len);

    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 30) - [public] mapping_translate
uint64_t mapping_translate(mapping_space_t space, uint64_t virt);
bool mapping_is_mapped(mapping_space_t space, uint64_t virt);

bool mapping_copy_range(mapping_space_t dst,
                        uint64_t dst_virt,
                        mapping_space_t src,
                        uint64_t src_virt,
                        size_t len);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 31) - [public] mapping_is_mapped
bool mapping_is_mapped(mapping_space_t space, uint64_t virt);

bool mapping_copy_range(mapping_space_t dst,
                        uint64_t dst_virt,
                        mapping_space_t src,
                        uint64_t src_virt,
                        size_t len);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 22) - [public] page_init
bool page_init(void);
bool page_ready(void);

page_t *page_alloc(uint32_t type);
page_t *page_alloc_zero(uint32_t type);

bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 23) - [public] page_ready
bool page_ready(void);

page_t *page_alloc(uint32_t type);
page_t *page_alloc_zero(uint32_t type);

bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 28) - [public] page_attach
bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 30) - [public] page_get

```
bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 31) - [public] page_put

```
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 33) - [public] page_phys

```
uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 35) - [public] page_pfn

```
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 41) - [public] page_refcount

```
uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```


kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 42) - [public] page_type

```
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 43) - [public] page_set_type

```
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 45) - [public] page_stat_total

```
size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 46) - [public] page_stat_referenced

```
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 48) - [public] page_dump

```
void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 18) - [public] vmm_init

```
bool vmm_init(void);
bool vmm_ready(void);

bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

bool vmm_free(uint64_t virt, size_t bytes);
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 19) - [public] vmm_ready

```
bool vmm_ready(void);

bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

bool vmm_free(uint64_t virt, size_t bytes);
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 21) - [public] vmm_alloc
    bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
    bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 22) - [public] vmm_alloc_aligned
    bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 23) - [public] vmm_map_device
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    uint64_t vmm_stat_free_bytes(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 25) - [public] vmm_free
    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    uint64_t vmm_stat_free_bytes(void);

    void vmm_dump(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 26) - [public] vmm_unmap_device

```
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 28) - [public] vmm_translate

```
uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 30) - [public] vmm_stat_total_pages

```
size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 31) - [public] vmm_stat_free_pages

```
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 32) - [public]

vmm_stat_allocated_pages

```
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 33) - [public] vmm_stat_total_bytes

```
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 34) - [public] vmm_stat_free_bytes
    uint64_t vmm_stat_free_bytes(void);

    void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 36) - [public] vmm_dump
    void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 25) - [public] arch_memory_ready
    bool arch_memory_ready(void);

    bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out);
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 27) - [public]
arch_memory_boot_alloc
    bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out);
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 28) - [public]
arch_memory_reserve_range
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 30) - [public] arch_is_page_aligned
    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 31) - [public] arch_page_align_up
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 32) - [public] arch_page_align_down
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 35) - [public] arch_virt_to_phys
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 37) - [public] arch_memory_max_phys
    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 38) - [public] arch_memory_dump
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 17) - [public] paging_init
    void paging_init(uint64_t boot_phys_offset);

    bool paging_enable_nx(void);

    uint64_t paging_read_cr3(void);
    void paging_write_cr3(uint64_t pml4_phys);

    bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
    bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
    uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 19) - [public] paging_enable_nx
    bool paging_enable_nx(void);

    uint64_t paging_read_cr3(void);
    void paging_write_cr3(uint64_t pml4_phys);

    bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
    bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
    uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 21) - [public] paging_read_cr3

```
uint64_t paging_read_cr3(void);
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 22) - [public] paging_write_cr3

```
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 24) - [public] paging_map_page_in

```
bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 25) - [public] paging_unmap_page_in

```
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 26) - [public] paging_set_flags_in

```
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 27) - [public] paging_is_mapped_in

```
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 28) - [public] paging_translate_in

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 8) - [public] tlb_init

```
bool tlb_init(void);
bool tlb_ready(void);

void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 9) - [public] tlb_ready

```
bool tlb_ready(void);

void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 11) - [public] tlb_flush_all

```
void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 12) - [public]

tlb_flush_all_including_global

```
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 14) - [public] tlb_flush_page

```
void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 15) - [public] tlb_flush_page_addr

```
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 17) - [public] tlb_flush_range

```
void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 18) - [public] tlb_flush_range_addr

```
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 71) - [public] arch_memory_ready

```
bool arch_memory_ready(void);

bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out_base);

const arch_mem_info_t *arch_memory_get(void);
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 73) - [public]

arch_memory_boot_alloc

```
bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out_base);

const arch_mem_info_t *arch_memory_get(void);
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 76) - [public] arch_memory_regions

```
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 77) - [public]

arch_memory_total_usable

```
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 78) - [public]

arch_memory_range_is_usable

```
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 79) - [public]

arch_memory_reserve_range

```
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

static inline uint64_t arch_page_align_up(uint64_t v)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 83) - [public] arch_memory_dump

```
void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

static inline uint64_t arch_page_align_up(uint64_t v)
{
    if (v > UINT64_MAX - ARCH_PAGE_MASK) {
        return UINT64_MAX;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 92) - [public] if

```
    if (v > UINT64_MAX - ARCH_PAGE_MASK) {
        return UINT64_MAX;
    }

    return (v + ARCH_PAGE_MASK) & ~ARCH_PAGE_MASK;
}

static inline bool arch_is_page_aligned(uint64_t v)
{
    return (v & ARCH_PAGE_MASK) == 0;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 107) - [public] return

```
    return (void *)((uintptr_t)(phys + ARCH_DIRECT_MAP_BASE);
}

static inline uint64_t arch_virt_to_phys(const void *virt)
{
    return (uint64_t)((uintptr_t)virt - ARCH_DIRECT_MAP_BASE);
}

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 112) - [public] return

```
    return (uint64_t)((uintptr_t)virt - ARCH_DIRECT_MAP_BASE);
}

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 35) - [public]

paging_set_boot_phys_offset

```
void paging_set_boot_phys_offset(uint64_t phys_offset);

bool paging_boot_phys_offset_valid(void);

void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 37) - [public]

paging_boot_phys_offset_valid

```
bool paging_boot_phys_offset_valid(void);

void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 39) - [public]

paging_init_direct_map

```
void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 41) - [public] paging_init

```
void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 43) - [public] paging_read_cr3

```
uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 44) - [public] paging_flush_tlb_all

```
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 45) - [public] paging_flush_page
    void paging_flush_page(uint64_t addr);

    bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 47) - [public] paging_map_page
    bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 49) - [public] paging_map_range
    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    bool paging_map_page_in(uint64_t pml4,
                           uint64_t virt,
                           // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 51) - [public] paging_translate
    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    bool paging_map_page_in(uint64_t pml4,
                           uint64_t virt,
                           uint64_t phys,
                           uint64_t flags);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 52) - [public] paging_is_mapped

```
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 53) - [public] paging_unmap_page

```
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 54) - [public] paging_set_flags

```
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 55) - [public] paging_get_flags

```
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 56) - [public] paging_enable_nx

```
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 57) - [public] paging_nx_enabled

```
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 63) - [public] paging_unmap_page_in

```
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

uint64_t paging_create_pml4(void);
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 64) - [public] paging_translate_in

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

uint64_t paging_create_pml4(void);
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 65) - [public] paging_is_mapped_in
    bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 66) - [public] paging_set_flags_in
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 67) - [public] paging_get_flags_in
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 69) - [public] paging_create_pml4
    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);

    void paging_enable_smep(void);
    void paging_enable_smap(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 70) - [public] paging_destroy_pml4

```
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 71) - [public] paging_switch_pml4

```
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 73) - [public] paging_map_mmio

```
bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 75) - [public]

paging_enable_write_protect

```
void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 76) - [public]

paging_disable_write_protect

```
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 77) - [public]

paging_write_protect_enabled

```
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 79) - [public] paging_enable_smep

```
void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 80) - [public] paging_enable_smmap

```
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 81) - [public] paging_smep_enabled

```
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 82) - [public] paging_smap_enabled

```
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 84) - [public] paging_pcid_supported

```
bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 85) - [public] paging_enable_pcid

```
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 86) - [public] paging_pcid_enabled

```
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 87) - [public] paging_invpcid

```
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 90) - [public] paging_la57_supported
    bool paging_la57_supported(void);
    void paging_assert_4level_paging(void);
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 91) - [public]
paging_assert_4level_paging
    void paging_assert_4level_paging(void);
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 92) - [public] paging_write_cr3
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 14) - [public] tlb_init
    bool tlb_init(void);
    bool tlb_ready(void);
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 15) - [public] tlb_ready
    bool tlb_ready(void);
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 16) - [public] tlb_has_pcid
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 17) - [public] tlb_has_invpcid
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 19) - [public] tlb_flush_all
    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 20) - [public]
tlb_flush_all_including_global
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 22) - [public] tlb_flush_page
    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 23) - [public] tlb_flush_page_addr
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 25) - [public] tlb_flush_range
    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 26) - [public] tlb_flush_range_addr
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 28) - [public] tlb_flush_pcid
    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 29) - [public] tlb_flush_pcid_addr
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 31) - [public] tlb_wbinvd
    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 32) - [public] tlb_clflush
    void tlb_clflush(uint64_t addr);

#ifdef

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 6) - [public] tlb_asm_invlpg

```
void tlb_asm_invlpg(uint64_t addr);

uint64_t tlb_asm_read_cr3(void);
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 8) - [public] tlb_asm_read_cr3

```
uint64_t tlb_asm_read_cr3(void);
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 9) - [public] tlb_asm_write_cr3

```
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 11) - [public] tlb_asm_read_cr4

```
uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 12) - [public] tlb_asm_write_cr4

```
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 14) - [public] tlb_asm_invpaid

```
void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 16) - [public] tlb_asm_wbinvd
    void tlb_asm_wbinvd(void);
    void tlb_asm_clflush(uint64_t addr);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 17) - [public] tlb_asm_clflush
    void tlb_asm_clflush(uint64_t addr);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 7) - [public] cache_init
    bool cache_init(void);
    bool cache_ready(void);

    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 8) - [public] cache_ready
    bool cache_ready(void);

    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 10) - [public] cache_register
    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 13) - [public] cache_dump
    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 12) - [public] void
typedef void (*kcache_ctor_t)(void *obj);

typedef struct kcache_chunk {
    struct kcache_chunk *next;
    uint64_t va;
} kcache_chunk_t;

typedef struct kcache_mag {
    void *objs[KCACHE_MAG_SIZE];
    size_t count;
} kcache_mag_t;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 47) - [public] kcache_destroy

```
void kcache_destroy(kcache_t *c);

void *kcache_alloc(kcache_t *c);
void *kcache_zalloc(kcache_t *c);
void kcache_free(kcache_t *c, void *ptr);

size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 51) - [public] kcache_free

```
void kcache_free(kcache_t *c, void *ptr);

size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 53) - [public] kcache_object_size

```
size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 54) - [public] kcache_free_count

```
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 55) - [public] kcache_total_count

```
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 12) - [public] per_cpu_init

```
bool per_cpu_init(void);
size_t per_cpu_count(void);
size_t per_cpu_id(void);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 13) - [public] per_cpu_count

```
size_t per_cpu_count(void);
size_t per_cpu_id(void);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 14) - [public] per_cpu_id

```
size_t per_cpu_id(void);

#endif
```


kernel_Workspace/kernel/src/mm/core/address.h (line 10) - [public] addr_is_canonical

```
bool addr_is_canonical(uint64_t a);
bool addr_is_user(uint64_t a);
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/address.h (line 11) - [public] addr_is_user

```
bool addr_is_user(uint64_t a);
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/address.h (line 12) - [public] addr_is_kernel

```
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

kernel_Workspace/kernel/src/mm/core/address.h (line 13) - [public] addr_is_direct_map

```
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

kernel_Workspace/kernel/src/mm/core/address.h (line 15) - [public] addr_align_up

```
uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

```

kernel_Workspace/kernel/src/mm/core/address.h (line 16) - [public] addr_align_down
    uint64_t addr_align_down(uint64_t a, uint64_t align);

    uint64_t addr_phys_to_direct(uint64_t phys);
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/address.h (line 18) - [public] addr_phys_to_direct
    uint64_t addr_phys_to_direct(uint64_t phys);
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/address.h (line 19) - [public] addr_direct_to_phys
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 26) - [public] mm_init
    bool mm_init(const mm_boot_params_t *params);
    bool mm_ready(void);

    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 27) - [public] mm_ready
    bool mm_ready(void);

    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 29) - [public] mm_total_ram
    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 30) - [public] mm_free_ram
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 32) - [public] mm_dump
    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/range.h (line 18) - [public] range_is_canonical
    bool range_is_canonical(uint64_t addr);

#endif

```

kernel_Workspace/kernel/src/mm/core/region.h (line 13) - [public] region_make

```
region_t region_make(uint64_t start, uint64_t len);
bool region_valid(region_t r);
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                       region_t cut,
                       region_t *out,
                       size_t max_out);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.h (line 14) - [public] region_valid

```
bool region_valid(region_t r);
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                       region_t cut,
                       region_t *out,
                       size_t max_out);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.h (line 15) - [public] region_len

```
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                       region_t cut,
                       region_t *out,
                       size_t max_out);

#endif
```

kernel_Workspace/kernel/src/mm/core/region.h (line 17) - [public] region_contains

```
bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                       region_t cut,
                       region_t *out,
                       size_t max_out);

#endif
```

kernel_Workspace/kernel/src/mm/core/region.h (line 18) - [public] region_overlaps

```
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                       region_t cut,
                       region_t *out,
                       size_t max_out);

#endif
```

```

kernel_Workspace/kernel/src/mm/core/region.h (line 19) - [public] region_intersect
    region_t region_intersect(region_t a, region_t b);

    size_t region_subtract(region_t r,
                           region_t cut,
                           region_t *out,
                           size_t max_out);

#endif

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 8) - [public] size_is_pow2
    bool size_is_pow2(size_t v);

    bool size_round_up_pow2(size_t v, size_t *out);

    bool kfree_pages_validate(size_t pages,
                              size_t page_size,
                              size_t usable_bytes,
                              size_t *out_requested_bytes);

    bool size_bytes_to_pages_checked(size_t bytes,
                                     // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 11) - [public] size_round_up_pow2
    bool size_round_up_pow2(size_t v, size_t *out);

    bool kfree_pages_validate(size_t pages,
                              size_t page_size,
                              size_t usable_bytes,
                              size_t *out_requested_bytes);

    bool size_bytes_to_pages_checked(size_t bytes,
                                     size_t page_size,
                                     size_t *out_pages);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 21) - [public] smp_lock_init
    void smp_lock_init(smp_ticket_lock_t *lock);

    void smp_lock_acquire(smp_ticket_lock_t *lock);

    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 24) - [public] smp_lock_acquire
    void smp_lock_acquire(smp_ticket_lock_t *lock);

    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

```

```

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 27) - [public] smp_lock_release
    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 30) - [public] smp_current_cpu_id
    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 19) - [public] paging_subsystem_init
    bool paging_subsystem_init(void);

    address_space_t *paging_aspace_create(void);
    void paging_aspace_destroy(address_space_t *as);
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 22) - [public] paging_aspace_destroy
    void paging_aspace_destroy(address_space_t *as);
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);

    bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 23) - [public] paging_aspace_switch
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);

    bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

    bool paging_aspace_protect(address_space_t *as,
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/paging/paging.h (line 24) - [public] paging_aspace_cr3
uint64_t paging_aspace_cr3(const address_space_t *as);

bool paging_aspace_map(address_space_t *as,
                        uint64_t virt,
                        uint64_t phys,
                        size_t len,
                        uint32_t prot);

bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

bool paging_aspace_protect(address_space_t *as,
                           uint64_t virt,
                           // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 32) - [public] paging_aspace_unmap
bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

bool paging_aspace_protect(address_space_t *as,
                           uint64_t virt,
                           size_t len,
                           uint32_t prot);

uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt);

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 39) - [public] paging_aspace_translate
uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt);

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 41) - [public] paging_kernel_map
bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 42) - [public] paging_kernel_unmap
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 14) - [public] pml_level_shift
uint64_t pml_level_shift(int level);
uint64_t pml_index(uint64_t virt, int level);

bool pml_entry_present(uint64_t entry);
bool pml_entry_large(uint64_t entry);
uint64_t pml_entry_addr(uint64_t entry);
uint64_t *pml_table_ptr(uint64_t table_phys);

#endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 15) - [public] pml_index
uint64_t pml_index(uint64_t virt, int level);

bool pml_entry_present(uint64_t entry);
bool pml_entry_large(uint64_t entry);
uint64_t pml_entry_addr(uint64_t entry);
uint64_t *pml_table_ptr(uint64_t table_phys);

#endif

```

```

kernel_Workspace/kernel/src/mm/paging/pml.h (line 17) - [public] pml_entry_present
    bool pml_entry_present(uint64_t entry);
    bool pml_entry_large(uint64_t entry);
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 18) - [public] pml_entry_large
    bool pml_entry_large(uint64_t entry);
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 19) - [public] pml_entry_addr
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 16) - [public] tlb_batch_begin
    void tlb_batch_begin(tlb_batch_t *batch);
    void tlb_batch_add(tlb_batch_t *batch, uint64_t virt);
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 17) - [public] tlb_batch_add
    void tlb_batch_add(tlb_batch_t *batch, uint64_t virt);
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 18) - [public] tlb_batch_full
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 19) - [public] tlb_batch_commit
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/process/address_space.h (line 30) - [public]
aspace_subsystem_init
    bool aspace_subsystem_init(void);

    proc_aspace_t *aspace_create(void);
    void aspace_destroy(proc_aspace_t *pa);

    address_space_t *aspace_paging_handle(proc_aspace_t *pa);

    uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
    uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
    uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
    bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/process/address_space.h (line 33) - [public] aspace_destroy
void aspace_destroy(proc_aspace_t *pa);

address_space_t *aspace_paging_handle(proc_aspace_t *pa);

uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 37) - [public] aspace_map_anon
uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                           uint32_t checked_prot));

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 38) - [public] aspace_map_at
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                           uint32_t checked_prot));

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 39) - [public] aspace_reserve_at
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                           uint32_t checked_prot));

vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
// ... (truncated, see source file for more)

```



```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 40) - [public] aspace_unmap
    bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

    bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                                uint32_t checked_prot,
                                uint32_t apply_prot,
                                bool (*allowed)(uint32_t old_prot,
                                                  uint32_t checked_prot));

    vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
    uint64_t aspace_stack_base(void);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 41) - [public] aspace_protect
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

    bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                                uint32_t checked_prot,
                                uint32_t apply_prot,
                                bool (*allowed)(uint32_t old_prot,
                                                  uint32_t checked_prot));

    vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
    uint64_t aspace_stack_base(void);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 51) - [public] aspace_stack_base
    uint64_t aspace_stack_base(void);

    uint64_t aspace_brk(proc_aspace_t *pa, uint64_t new_brk);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 53) - [public] aspace_brk
    uint64_t aspace_brk(proc_aspace_t *pa, uint64_t new_brk);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/mmap.h (line 20) - [public] munmap
    bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/mmap.h (line 21) - [public] mprotect
    bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

#endif
```

```
kernel_Workspace/kernel/src/mm/protection/guard.h (line 9) - [public] guard_install
    uint64_t guard_install(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool guard_user_stack(proc_aspace_t *pa);

#endif
```

```
kernel_Workspace/kernel/src/mm/protection/guard.h (line 10) - [public] guard_user_stack
    bool guard_user_stack(proc_aspace_t *pa);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 8) - [public] isolation_init

```
bool isolation_init(void);

bool isolation_has_smep(void);
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 10) - [public] isolation_has_smep

```
bool isolation_has_smep(void);
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 11) - [public] isolation_has_smap

```
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 12) - [public]
isolation_enable_smep

```
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 13) - [public]
isolation_enable_smap

```
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 15) - [public]
isolation_audit_kernel

```
size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 8) - [public] perm_sanitize

```
uint32_t perm_sanitize(uint32_t prot);
bool perm_is_wx(uint32_t prot);
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 9) - [public] perm_is_wx

```
bool perm_is_wx(uint32_t prot);
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 10) - [public] perm_mprotect_allowed

```
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 11) - [public] perm_set_strict_wx

```
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 13) - [public] perm_kernel_default

```
uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 14) - [public] perm_user_default

```
uint32_t perm_user_default(void);

#endif
```

kstd/include/kstd.h (line 12) - [public] tr_log

```
void tr_log(const char *s);

#endif
```

kstd/include/kstd_audio.h (line 7) - [public] tr_free

```
void tr_free(void *ptr, uint32_t bytes);
tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

#endif
```

kstd/include/kstd_audio.h (line 8) - [public] tr_map_mmio

```
tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

#endif
```

kstd/include/kstd_bt.h (line 6) - [public] tr_bt_info

```
tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6);
tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len);
tr_status_t tr_bt_evt(void *buf, uint8_t *len);
tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

#endif
```

```

kstd/include/kstd_bt.h (line 7) - [public] tr_bt_cmd
    tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len);
    tr_status_t tr_bt_evt(void *buf, uint8_t *len);
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 8) - [public] tr_bt_evt
    tr_status_t tr_bt_evt(void *buf, uint8_t *len);
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 9) - [public] tr_bt_acl_send
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 10) - [public] tr_bt_acl_recv
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_fs.h (line 6) - [public] tr_fs_read
    tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out);
    tr_status_t tr_fs_exists(const char *path);

    #endif

kstd/include/kstd_fs.h (line 7) - [public] tr_fs_exists
    tr_status_t tr_fs_exists(const char *path);

    #endif

kstd/include/kstd_input.h (line 6) - [public] tr_input_key
    int32_t tr_input_key(void);

    #endif

kstd/include/kstd_mem.h (line 7) - [public] tr_free
    void tr_free(void *ptr, uint32_t bytes);
    tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

    #endif

kstd/include/kstd_mem.h (line 8) - [public] tr_map_mmio
    tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

    #endif

kstd/include/kstd_video.h (line 22) - [public] tr_video_init
    /* Inicjalizacja i stan */
    tr_status_t tr_video_init(void);
    bool tr_video_is_ready(void);
    tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

    /* Zarz■dzenie trybami */
    uint32_t tr_video_mode_count(void);
    tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
    tr_status_t tr_video_mode_set(uint32_t mode_id);

    /* Operacje na framebufferze */
    tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
        // ... (truncated, see source file for more)

```

kstd/include/kstd_video.h (line 23) - [public] tr_video_is_ready

```
bool tr_video_is_ready(void);
tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

/* Zarządzenie trybami */
uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kstd/include/kstd_video.h (line 24) - [public] tr_video_get_caps

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

/* Zarządzenie trybami */
uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

// ... (truncated, see source file for more)
```

kstd/include/kstd_video.h (line 27) - [public] tr_video_mode_count

```
/* Zarządzenie trybami */

uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 28) - [public] tr_video_mode_get

```
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 29) - [public] tr_video_mode_set

```
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 32) - [public] tr_video_fill

/ Operacje na framebufferze */*

```
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);
```

#endif

libs/dsabi.h (line 93) - [public] volatile

```
__asm__ volatile("int $0x80" :: "a"(1));
}
```

```
typedef struct {
    uint64_t id;
    uint32_t cmd;
    uint32_t flags;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    int32_t status;
    uint32_t pad;
    // ... (truncated, see source file for more)
```

libs/dsabi.h (line 124) - [public] ds_init

```
void ds_init(uint64_t params_va);
uint64_t ds_call(uint32_t cls, uint32_t op,
                uint64_t a0, uint64_t a1, uint64_t a2);
int ds_take(uint64_t id, ds_msg_t *out);
void ds_poll(void);
```

#endif

libs/dsabi.h (line 127) - [public] ds_take

```
int ds_take(uint64_t id, ds_msg_t *out);
void ds_poll(void);
```

#endif

libs/dsabi.h (line 128) - [public] ds_poll

```
void ds_poll(void);
```

#endif

libs/dsasync.h (line 21) - [public] if

```
if (r->done) {
    return 1;
}

ds_poll();

if (r->id != 0 && ds_take(r->id, &r->msg)) {
    r->done = 1;
}

return r->done;
}

// ... (truncated, see source file for more)
```

```

libs/dsasync.h (line 25) - [public] ds_poll
    ds_poll();

    if (r->id != 0 && ds_take(r->id, &r->msg)) {
        r->done = 1;
    }

    return r->done;
}

static inline int ds_req_ok(ds_req_t *r)
{
    return r->done && r->msg.status == 0;
    // ... (truncated, see source file for more)

libs/dsasync.h (line 27) - [public] if
    if (r->id != 0 && ds_take(r->id, &r->msg)) {
        r->done = 1;
    }

    return r->done;
}

static inline int ds_req_ok(ds_req_t *r)
{
    return r->done && r->msg.status == 0;
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 7) - [public] ds_call
    return ds_call(SVC_AUDIO, AUD_PLAY, phys, len, 0);
}

static inline void ds_audio_stop(void)
{
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
}

static inline uint64_t ds_audio_jack_req(void)
{
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 12) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
}

static inline uint64_t ds_audio_jack_req(void)
{
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

static inline void ds_audio_amp(int on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 17) - [public] ds_call
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

static inline void ds_audio_amp(int on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

#endif

```

```

libs/dsaudio.h (line 22) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

#endif

libs/dsinput.h (line 7) - [public] ds_call
    return ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
}

static inline int ds_key_take(uint64_t id, uint8_t *out)
{
    ds_msg_t m;

    if (!ds_take(id, &m) || m.arg0 == 0) {
        return 0;
    }

    *out = (uint8_t)m.arg0;
    // ... (truncated, see source file for more)

libs/dsinput.h (line 14) - [public] if
    if (!ds_take(id, &m) || m.arg0 == 0) {
        return 0;
    }

    *out = (uint8_t)m.arg0;
    return 1;
}

#endif

libs/dslock.h (line 7) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_COUNT, disk, 0, 0);
}

static inline uint64_t ds_blk_read(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_READ, disk, block, 0);
}

static inline uint64_t ds_blk_write(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

// ... (truncated, see source file for more)

libs/dslock.h (line 12) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_READ, disk, block, 0);
}

static inline uint64_t ds_blk_write(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

static inline volatile uint8_t *ds_blk_scratch(void)
{
    return (volatile uint8_t *)DS_SCRATCH_VA;
}

// ... (truncated, see source file for more)

```



```

libs/dslock.h (line 17) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

static inline volatile uint8_t *ds_blk_scratch(void)
{
    return (volatile uint8_t *)DS_SCRATCH_VA;
}

#endif

libs/dslog.h (line 10) - [public] while
    while (s[n] != '\0' && n < DS_SCRATCH_SIZE - 1) {
        scratch[n] = s[n];
        n++;
    }

    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

#endif

libs/dslog.h (line 15) - [public] ds_call
    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

#endif

libs/dsmem.h (line 7) - [public] ds_call
    return ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
}

static inline uint64_t ds_free_pages(uint64_t va)
{
    return ds_call(SVC_SYS, OP_FREE, va, 0, 0);
}

static inline uint64_t ds_map_mmio(uint64_t phys, uint64_t len, uint64_t va)
{
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

// ... (truncated, see source file for more)

libs/dsmem.h (line 12) - [public] ds_call
    return ds_call(SVC_SYS, OP_FREE, va, 0, 0);
}

static inline uint64_t ds_map_mmio(uint64_t phys, uint64_t len, uint64_t va)
{
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

static inline uint64_t ds_page_phys(uint64_t va)
{
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

// ... (truncated, see source file for more)

libs/dsmem.h (line 17) - [public] ds_call
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

static inline uint64_t ds_page_phys(uint64_t va)
{
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

#endif

```

```

libs/dsmem.h (line 22) - [public] ds_call
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

#endif

libs/dsvgpu.h (line 6) - [public] ds_call
extern uint64_t ds_call(uint32_t, uint32_t, uint64_t, uint64_t, uint64_t);
extern void ds_poll(void);
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)
}

libs/dsvgpu.h (line 7) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        // ... (truncated, see source file for more)
    }
}

libs/dsvgpu.h (line 8) - [public] ds_take
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
        // ... (truncated, see source file for more)
    }
}

libs/dsvgpu.h (line 14) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
    }

    *sid = (int32_t)r.arg0;
    *phys = r.arg1;
    return 1;
}

// ... (truncated, see source file for more)

```

```

libs/dsvgpu.h (line 18) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
    }

    *sid = (int32_t)r.arg0;
    *phys = r.arg1;
    return 1;
}

static inline int dsvgpu_present(int32_t sid, uint32_t x, uint32_t y)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_PRESENT,
        // ... (truncated, see source file for more)

libs/dsvgpu.h (line 31) - [public] ds_poll
    ds_poll();

    ds_msg_t r;
    return ds_take(id, &r) && r.status == 0;
}

#endif

libs/dsvideo.h (line 14) - [public] ds_call
    return ds_call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0);
}

static inline int ds_fb_info_take(uint64_t id, ds_fb_t *fb)
{
    ds_msg_t m;

    if (!ds_take(id, &m) || m.status != 0) {
        return 0;
    }

    fb->w = (uint32_t)(m.arg0 >> 16);
    // ... (truncated, see source file for more)

libs/dsvideo.h (line 21) - [public] if
    if (!ds_take(id, &m) || m.status != 0) {
        return 0;
    }

    fb->w = (uint32_t)(m.arg0 >> 16);
    fb->h = (uint32_t)(m.arg0 & 0xFFFF);
    fb->stride = (uint32_t)m.arg1;
    fb->phys = m.arg2;

    return 1;
}

    // ... (truncated, see source file for more)

libs/dsvideo.h (line 35) - [public] ds_call
    ds_call(SVC_VIDEO, VID_FB_TAKEOVER, 0, 0, 0);
}

static inline void ds_fb_release(void)
{
    ds_call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0);
}

static inline void ds_px(volatile uint32_t *fb, uint32_t stride,
    uint32_t x, uint32_t y, uint32_t c)
{
    fb[(uint64_t)y * stride + x] = c;
    // ... (truncated, see source file for more)

```

```

libs/dsvideo.h (line 40) - [public] ds_call
    ds_call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0);
}

static inline void ds_px(volatile uint32_t *fb, uint32_t stride,
                        uint32_t x, uint32_t y, uint32_t c)
{
    fb[(uint64_t)y * stride + x] = c;
}

static inline uint32_t ds_rgb(uint8_t r, uint8_t g, uint8_t b)
{
    return 0xFF000000u | ((uint32_t)r << 16) | ((uint32_t)g << 8) | b;
    // ... (truncated, see source file for more)

```

Nim

kernel_Workspace/kernel/src/nimcore/banner.nim (line 13) - [internal] nim_banner

```

proc nim_banner(buf: ptr uint8, cap: uint32): uint32 {.exportc, cdecl.} =
    var o = 0u32

    for ch in ART:
        if o >= cap:
            break

        buf[o] = uint8(ch)
        o += 1

    return o

```

kernel_Workspace/kernel/src/nimcore/format.nim (line 1) - [internal] nim_u64_to_str

```

proc nim_u64_to_str(v: uint64, base: uint8, buf: ptr uint8, cap: uint32): uint32
    {.exportc, cdecl.} =
        var tmp: array[64, uint8]
        var n = 0u32
        var x = v

        let b = if base < 2: 10u64 else: uint64(base)

        if x == 0:
            tmp[0] = uint8('0')
            n = 1
        else:
            // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/nimcore/format.nim (line 29) - [internal] nim_hex_dump

```

proc nim_hex_dump(src: ptr uint8, len: uint32, buf: ptr uint8, cap: uint32): uint32
    {.exportc, cdecl.} =
        const hexd = "0123456789abcdef"
        var o = 0u32
        var i = 0u32

        while i < len and o + 2 < cap:
            let b = src[i]
            buf[o] = uint8(hexd[int(b shr 4)])
            buf[o + 1] = uint8(hexd[int(b and 15)])
            o += 2
            i += 1
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/nimcore/parse.nim (line 1) - [internal] nim_parse_u64
proc nim_parse_u64(s: ptr uint8, len: uint32, base: uint8, out_v: ptr uint64): uint8
  {.exportc, cdecl.} =
    var v = 0u64
    var i = 0u32
    let b = uint64(base)

    if len == 0: return 0

    if base == 16 and len > 2 and s[0] == uint8('0') and (s[1] == uint8('x') or s[1] == uint8('X')):
      i = 2

    if i >= len: return 0
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 1) - [internal] Rb
type Rb = object
  data: array[1024, uint8]
  head: uint32
  tail: uint32

var key_rb: Rb

proc nim_rb_push(b: uint8): uint8 {.exportc, cdecl.} =
  let next = (key_rb.head + 1) mod 1024
  if next == key_rb.tail: return 0
  key_rb.data[key_rb.head] = b
  key_rb.head = next
  // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 8) - [internal] nim_rb_push
proc nim_rb_push(b: uint8): uint8 {.exportc, cdecl.} =
  let next = (key_rb.head + 1) mod 1024
  if next == key_rb.tail: return 0
  key_rb.data[key_rb.head] = b
  key_rb.head = next
  return 1

proc nim_rb_pop(): int32 {.exportc, cdecl.} =
  if key_rb.head == key_rb.tail: return -1
  let b = key_rb.data[key_rb.tail]
  key_rb.tail = (key_rb.tail + 1) mod 1024
  return int32(b)
  // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 15) - [internal] nim_rb_pop
proc nim_rb_pop(): int32 {.exportc, cdecl.} =
  if key_rb.head == key_rb.tail: return -1
  let b = key_rb.data[key_rb.tail]
  key_rb.tail = (key_rb.tail + 1) mod 1024
  return int32(b)

proc nim_rb_len(): uint32 {.exportc, cdecl.} =
  return (key_rb.head + 1024 - key_rb.tail) mod 1024

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 21) - [internal] nim_rb_len
proc nim_rb_len(): uint32 {.exportc, cdecl.} =
  return (key_rb.head + 1024 - key_rb.tail) mod 1024

```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 1) - [internal] Handler

```
type Handler = proc(arg: ptr uint8, len: uint32): int32 {.cdecl.}

type Cmd = object
  name: array[24, uint8]
  nlen: uint32
  used: bool
  h: Handler

var cmds: array[32, Cmd]

proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 3) - [internal] Cmd

```
type Cmd = object
  name: array[24, uint8]
  nlen: uint32
  used: bool
  h: Handler

var cmds: array[32, Cmd]

proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    if nlen == 0 or nlen >= 24: return 0

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 11) - [internal] nim_shell_register

```
proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    if nlen == 0 or nlen >= 24: return 0

    for i in 0..<32:
      if not cmds[i].used:
        var k = 0u32
        while k < nlen:
          cmds[i].name[k] = name[k]
          k += 1
        cmds[i].nlen = nlen
        cmds[i].used = true
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 28) - [internal] eq_name

```
proc eq_name(c: var Cmd, line: ptr uint8, nlen: uint32): bool =
  if c.nlen != nlen: return false
  var k = 0u32
  while k < nlen:
    if c.name[k] != line[k]: return false
    k += 1
  return true

proc nim_shell_run(line: ptr uint8, len: uint32): int32
  {.exportc, cdecl.} =
    var sp = 0u32
    while sp < len and line[sp] != uint8(' '):
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 36) - [internal] nim_shell_run

```
proc nim_shell_run(line: ptr uint8, len: uint32): int32
  {.exportc, cdecl.} =
    var sp = 0u32
    while sp < len and line[sp] != uint8(' '):
      sp += 1

    if sp == 0: return -1

    var arg: ptr uint8 = nil
    var alen = 0u32

    if sp < len:
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 10) - [internal] nim_policy_log

```
proc nim_policy_log(ring, cls, op, dec: uint8) {.exportc, cdecl.} =
  logbuf[head] = Entry(ring: ring, cls: cls, op: op, dec: dec)
  head = (head + 1) mod 256

  if dec == 1:
    denies += 1

proc nim_policy_denies(): uint64 {.exportc, cdecl.} =
  denies

proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 17) - [internal] nim_policy_denies

```
proc nim_policy_denies(): uint64 {.exportc, cdecl.} =
  denies

proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  {.exportc, cdecl.} =
    if idx >= 256:
      return 0

    let e = logbuf[idx]

    if out_ring != nil: out_ring[] = e.ring
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 20) - [internal] nim_policy_get

```
proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  {.exportc, cdecl.} =
    if idx >= 256:
      return 0

    let e = logbuf[idx]

    if out_ring != nil: out_ring[] = e.ring
    if out_cls != nil: out_cls[] = e.cls
    if out_op != nil: out_op[] = e.op
    if out_dec != nil: out_dec[] = e.dec
      // ... (truncated, see source file for more)
```

Odin

drivers/audiodriver/src/odin/driver.odin (line 8) - [public] Bdl_Entry

```
Bdl_Entry :: struct {
  ■addr: u32,
  ■ctrl: u32,
}

BDL_N :: 8

// mixer (NAM) — 16-bit
nam16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(nam_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 16) - [public] nam16

// mixer (NAM) — 16-bit

```
nam16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(nam_base + u64(off))
}

// bus master — 8/16/32
bm8 :: proc(off: u32) -> ^u8 {
  ■return transmute(^u8)(bm_base + u64(off))
}

bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 21) - [public] bm8

// bus master — 8/16/32

```
bm8 :: proc(off: u32) -> ^u8 {
  ■return transmute(^u8)(bm_base + u64(off))
}

bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

bm32 :: proc(off: u32) -> ^u32 {
  ■return transmute(^u32)(bm_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 25) - [public] bm16

```
bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

bm32 :: proc(off: u32) -> ^u32 {
  ■return transmute(^u32)(bm_base + u64(off))
}

GLOB_CNT  :: 0x2C
GLOB_STA  :: 0x30

PI_BDBAR :: 0x00
// ... (truncated, see source file for more)
```


drivers/audiodriver/src/odin/driver.odin (line 29) - [public] bm32

```
bm32 :: proc(off: u32) -> ^u32 {  
  ■return transmute(^u32)(bm_base + u64(off))  
}
```

```
GLOB_CNT  :: 0x2C  
GLOB_STA  :: 0x30
```

```
PI_BDBAR  :: 0x00  
PI_LVI    :: 0x05  
PI_SR     :: 0x06  
PI_CR     :: 0x0B
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/driver.odin (line 50) - [public] ad_init

```
ad_init :: proc "C" (nam_va: u64, bm_va: u64) -> i32 {  
  ■nam_base = nam_va  
  ■bm_base = bm_va  
  
  ■// cold reset  
  ■mem.volatile_store(bm32(GLOB_CNT), 0x04000000)  
  
  ■for _ in 0..<100000 {  
    ■■if mem.volatile_load(bm32(GLOB_STA)) & 0x04 != 0 {  
      ■■■break  
    }  
  }  
}
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/driver.odin (line 73) - [public] ad_stop

```
ad_stop :: proc "C" () -> i32 {  
  ■mem.volatile_store(bm8(P0_CR), 0)  
  ■mem.volatile_store(bm8(PI_CR), 0)  
  ■return 0  
}  
  
@(export)  
ad_position :: proc "C" () -> u32 {  
  ■return u32(mem.volatile_load(bm16(P0_PICB)))  
}
```

drivers/audiodriver/src/odin/driver.odin (line 80) - [public] ad_position

```
ad_position :: proc "C" () -> u32 {  
  ■return u32(mem.volatile_load(bm16(P0_PICB)))  
}
```

drivers/audiodriver/src/odin/jack.odin (line 8) - [public] ad_jack_present

```
ad_jack_present :: proc "C" () -> i32 {  
  ■pwr := mem.volatile_load(nam16(PWR_CTRL_STA))  
  
  ■// bity ready w powerdown status = tor aktywny  
  ■if pwr & 0x000F != 0 {  
    ■■return 1  
  }  
  
  ■// QEMU i tak nie zdejmuj jacka – fallback  
  ■return 1  
}
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/jack.odin (line 21) - [public] ad_set_amp

```
ad_set_amp :: proc "C" (on: i32) -> i32 {  
  ■if on == 0 {  
    ■■mem.volatile_store(nam16(0x02), 0x8000) // mute  
  } else {  
    ■■mem.volatile_store(nam16(0x02), 0x0000)  
  }  
  ■return 0  
}
```

```

drivers/audiodriver/src/odin/microphone.odin (line 6) - [public] ad_capture
    ad_capture :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
        ■ bdl := transmute(^Bdl_Entry) bdl_va

        ■ n := 0
        ■ off: u64 = 0

        ■ for off < u64(len) && n < BDL_N {
            ■ ■ chunk: u32 = 4096
            ■ ■ rem := u32(u64(len) - off)

            ■ ■ if rem < chunk {
                ■ ■ ■ chunk = rem
                // ... (truncated, see source file for more)

drivers/audiodriver/src/odin/speaker.odin (line 6) - [public] ad_play
    ad_play :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
        ■ bdl := transmute(^Bdl_Entry) bdl_va

        ■ n := 0
        ■ off: u64 = 0

        ■ for off < u64(len) && n < BDL_N {
            ■ ■ chunk: u32 = 4096
            ■ ■ rem := u32(u64(len) - off)

            ■ ■ if rem < chunk {
                ■ ■ ■ chunk = rem
                // ... (truncated, see source file for more)

```

Python

doc.py (line 131) - [public] Definition

```

class Definition:
    path: str          # path relative to the scanned root
    lineno: int
    kind: str          # "function" / "class" / "method" / "type" / "const" / "export"
    name: str
    signature: str      # signature line (short, used in section 1/2)
    is_public: bool
    comment: str = ""   # comment/docstring directly above the definition
    body_snippet: str = "" # first few lines of code (section 3)
    language: str = ""

```

doc.py (line 144) - [public] MarkdownFile

```

class MarkdownFile:
    path: str
    content: str

```

doc.py (line 153) - [public] iter_source_files

```

# ----- #
# Collecting files
# ----- #

def iter_source_files(root: str, exts: set[str] | None, exclude_dirs: set[str]):
    for dirpath, dirnames, filenames in os.walk(root):
        dirnames[:] = [d for d in dirnames if d not in exclude_dirs and not d.startswith(".")
                        or d in (".",)]
        for fn in filenames:
            ext = os.path.splitext(fn)[1].lower()
            full = os.path.join(dirpath, fn)
            rel = os.path.relpath(full, root)
            if ext in MD_EXTS:
                yield rel, full, ext
            elif exts is not None:
                if ext in exts:
                    # ... (truncated, see source file for more)

```

doc.py (line 170) - [public] read_text

```
def read_text(path: str) -> str:
    for enc in ("utf-8", "utf-8-sig", "latin-1"):
        try:
            with open(path, "r", encoding=enc) as f:
                return f.read()
        except (UnicodeDecodeError, LookupError):
            continue
    with open(path, "r", encoding="utf-8", errors="replace") as f:
        return f.read()
```

doc.py (line 185) - [public] extract_preceding_comment

```
# ----- #
# Extracting the comment directly above line `lineno` (1-indexed)
# ----- #

def extract_preceding_comment(lines: list[str], lineno_0based: int, ext: str) -> str:
    prefixes = COMMENT_PREFIXES.get(ext, ("#", "//"))
    collected = []
    i = lineno_0based - 1
    in_block_close = False
    while i >= 0:
        line = lines[i].rstrip("\n")
        stripped = line.strip()
        if stripped == "":
            i -= 1
            if collected:
                # a blank line ends the comment block (unless we haven't
                # ... (truncated, see source file for more)
```

doc.py (line 222) - [public] parse_python

```
# ----- #
# PYTHON - parsed via ast (no code execution)
# ----- #

def parse_python(rel_path: str, full_path: str, source: str) -> list[Definition]:
    defs: list[Definition] = []
    try:
        tree = ast.parse(source, filename=rel_path)
    except SyntaxError:
        return defs

    lines = source.splitlines()

    all_list = None
    for node in ast.walk(tree):
        if isinstance(node, ast.Assign):
            # ... (truncated, see source file for more)
```

doc.py (line 298) - [internal] _rules_js_ts

```
# Each rule: (kind, regex, group_index, decide_public)
# decide_public(match) -> bool, or None when visibility depends on context
# (tracked separately in parse_generic, e.g. Ruby's private/public keyword,
# Ada's "private" section, Odin's @(private) attribute)
```

```
def _rules_js_ts():
    return [
        ("function", re.compile(r"^\s*export\s+(default\s+)?function\s*\*\s*(\w+)"), 2, lambda m: True),
        ("function", re.compile(r"^\s*export\s+(default\s+)?function\s*\*\s*(\w+)"), 2, lambda m: True),
        ("class", re.compile(r"^\s*export\s+(default\s+)?class\s*(\w+)"), 2, lambda m: True),
        ("const", re.compile(r"^\s*export\s+(const|let|var)\s*(\w+)"), 2, lambda m: True),
        ("interface", re.compile(r"^\s*export\s+interface\s*(\w+)"), 1, lambda m: True),
        ("type", re.compile(r"^\s*export\s+type\s*(\w+)"), 1, lambda m: True),
        ("export", re.compile(r"^\s*module\.exports\.(.*)\s*="), 2, lambda m: True),
        ("function", re.compile(r"^\s*(async\s+)?function\s*\*\s*(\w+)"), 2, lambda m: False),
        ("const", re.compile(r"^\s*(const|let|var)\s*(\w+)\s*=\s*(\(|async\s*\(|function)"), 2, lambda m: True),
        ("method", re.compile(r"^\s*(private|)\s*(\w+)\s*\("), 2, lambda m: False),
        # ... (truncated, see source file for more)
```

doc.py (line 313) - [internal] _rules_java_cs

```
def _rules_java_cs():
    return [
        ("class", re.compile(r"^\\s*public\\s+(static\\s+)?(final\\s+)?(abstract\\s+)?(class|interface|enum|record)\\s+(\\w+)\\s*\\(", 2, lambda m: True),
        ("method", re.compile(r"^\\s*public\\s+(static\\s+)?(final\\s+)?[\\w<>\\[\\],\\s]+?\\s(\\w+)\\s*\\(", 2, lambda m: True),
        ("class", re.compile(r"^\\s*(private|protected|internal)\\s+(static\\s+)?(class|interface|enum|record)\\s+(\\w+)\\s*\\(", 2, lambda m: True),
        ("method", re.compile(r"^\\s*(private|protected|internal)\\s+(static\\s+)?[\\w<>\\[\\],\\s]+?\\s(\\w+)\\s*\\(", 2, lambda m: True),
    ]
```

doc.py (line 322) - [internal] _rules_go

```
def _rules_go():
    return [
        ("type", re.compile(r"^\\s*type\\s+([A-Z]\\w*)\\s+(struct|interface)", 1, lambda m: True),
        ("type", re.compile(r"^\\s*type\\s+([a-z_]\\w*)\\s+(struct|interface)", 1, lambda m: False),
        ("function", re.compile(r"^\\s*func\\s+\\([\\^]*\\)\\s*?([A-Z]\\w*)\\s*\\(", 2, lambda m: True),
        ("function", re.compile(r"^\\s*func\\s+\\([\\^]*\\)\\s*?([a-z_]\\w*)\\s*\\(", 2, lambda m: False),
    ]
```

doc.py (line 331) - [internal] _rules_rust

```
def _rules_rust():
    return [
        ("function", re.compile(r"^\\s*pub\\([\\^]*\\)\\s*?\\s+fn\\s+(\\w+)", 2, lambda m: True),
        ("struct", re.compile(r"^\\s*pub\\([\\^]*\\)\\s*?\\s+struct\\s+(\\w+)", 2, lambda m: True),
        ("enum", re.compile(r"^\\s*pub\\([\\^]*\\)\\s*?\\s+enum\\s+(\\w+)", 2, lambda m: True),
        ("trait", re.compile(r"^\\s*pub\\([\\^]*\\)\\s*?\\s+trait\\s+(\\w+)", 2, lambda m: True),
        ("function", re.compile(r"^\\s*fn\\s+(\\w+)", 1, lambda m: False),
        ("struct", re.compile(r"^\\s*struct\\s+(\\w+)", 1, lambda m: False),
    ]
```

doc.py (line 342) - [internal] _rules_c_cpp

```
def _rules_c_cpp():
    return [
        ("function", re.compile(r"^\\s*static\\s+[\\w:~*<>,\\s]+?\\s(\\w+)\\s*\\([\\^;]*\\)\\s*\\{", 1, lambda m: True),
        ("function", re.compile(r"^\\s*[\\w:~*<>,\\s]+?\\s(\\w+)\\s*\\([\\^;]*\\)\\s*\\{", 1, lambda m: True),
        ("declaration", re.compile(r"^\\s*[\\w:~*<>,\\s]+?\\s(\\w+)\\s*\\([\\^;]*\\)\\s*;\"", 1, lambda m: True),
    ]
```

doc.py (line 350) - [internal] _rules_php

```
def _rules_php():
    return [
        ("method", re.compile(r"^\\s*public\\s+(static\\s+)?function\\s+(\\w+)", 2, lambda m: True),
        ("method", re.compile(r"^\\s*(private|protected)\\s+(static\\s+)?function\\s+(\\w+)", 3, lambda m: False),
        ("function", re.compile(r"^\\s*function\\s+(\\w+)", 1, lambda m: True),
        ("class", re.compile(r"^\\s*class\\s+(\\w+)", 1, lambda m: True),
    ]
```

doc.py (line 359) - [internal] _rules_ruby

```
def _rules_ruby():
    return [
        ("method", re.compile(r"^\\s*def\\s+(self\\.)?(\\w+[\\?!]?)", 2, None), # visibility resolved context
        ("class", re.compile(r"^\\s*class\\s+(\\w+)", 1, lambda m: True),
        ("module", re.compile(r"^\\s*module\\s+(\\w+)", 1, lambda m: True),
    ]
```

doc.py (line 367) - [internal] _rules_kotlin_swift

```
def _rules_kotlin_swift():
    return [
        ("function", re.compile(r"^\\s*(public|open)\\s+fun\\s+(\\w+)", 2, lambda m: True),
        ("function", re.compile(r"^\\s*(private|internal|fileprivate)\\s+fun\\s+(\\w+)", 2, lambda m: False),
        ("function", re.compile(r"^\\s*fun\\s+(\\w+)", 1, lambda m: True),
        ("class", re.compile(r"^\\s*(public|open)\\s+class\\s+(\\w+)", 2, lambda m: True),
        ("class", re.compile(r"^\\s*(private|internal|fileprivate)\\s+class\\s+(\\w+)", 2, lambda m: False),
        ("class", re.compile(r"^\\s*class\\s+(\\w+)", 1, lambda m: True),
    ]
```

```

doc.py (line 378) - [internal] _rules_d
def _rules_d():
    return [
        ("class", re.compile(r"^s*(private\s+(class|struct|interface|enum|union)\s+(\w+))", 2, lambda m:
        ("class", re.compile(r"^s*(class|struct|interface|enum|union)\s+(\w+)", 2, lambda m: True),
        ("function", re.compile(r"^s*(private\s+[\w!\[\]\*\<>,\.\s]+?\s(\w+)\s*\([\^;{}]*\)\s*\{", 1, lambda m: True),
        ("function", re.compile(r"^s*[\w!\[\]\*\<>,\.\s]+?\s(\w+)\s*\([\^;{}]*\)\s*\{", 1, lambda m: True)
    ]

doc.py (line 387) - [internal] _rules_nim
def _rules_nim():
    return [
        ("proc", re.compile(r"^s*(proc|func|template|macro|method|iterator)\s+(\w+)\s*", 2, lambda m: True),
        ("proc", re.compile(r"^s*(proc|func|template|macro|method|iterator)\s+(\w+)\s*b", 2, lambda m: False),
        ("type", re.compile(r"^s*type\s+(\w+)\s*", 1, lambda m: True),
        ("type", re.compile(r"^s*type\s+(\w+)\s*b", 1, lambda m: False),
    ]

doc.py (line 399) - [internal] _rules_odin
def _rules_odin():
    # visibility resolved contextually (an @(private) attribute on the line
    # above) - see parse_generic
    return [
        ("proc", re.compile(r"^s*(\w+)\s*::\s*proc\b", 1, None),
        ("struct", re.compile(r"^s*(\w+)\s*::\s*struct\b", 1, None),
        ("enum", re.compile(r"^s*(\w+)\s*::\s*enum\b", 1, None),
        ("union", re.compile(r"^s*(\w+)\s*::\s*union\b", 1, None),
    ]

doc.py (line 410) - [internal] _rules_ada_spec
def _rules_ada_spec():
    # in a .ads file (package specification): visibility depends on whether
    # the declaration comes before or after the "private" section
    return [
        ("package", re.compile(r"^s*package\s+(\w+)", 1, lambda m: True),
        ("procedure", re.compile(r"^s*procedure\s+(\w+)", 1, None),
        ("function", re.compile(r"^s*function\s+(\w+)", 1, None),
    ]

doc.py (line 420) - [internal] _rules_ada_body
def _rules_ada_body():
    # in a .adb file (implementation) - treated as internal detail
    return [
        ("procedure", re.compile(r"^s*procedure\s+(\w+)", 1, lambda m: False),
        ("function", re.compile(r"^s*function\s+(\w+)", 1, lambda m: False),
    ]

doc.py (line 428) - [internal] _rules_asm
def _rules_asm():
    return [
        ("label", re.compile(r"^s*\.\?(global|globl)\s+(\w+)", 2, lambda m: True),
        ("label", re.compile(r"^[A-Za-z_]\w*\s*:", 1, lambda m: False),
    ]

doc.py (line 457) - [public] parse_generic
def parse_generic(rel_path: str, ext: str, source: str, max_body_lines: int) -> list[Definition]:
    rules = RULES_BY_EXT.get(ext)
    if not rules:
        return []
    lines = source.splitlines()
    lang = LANG_BY_EXT.get(ext, ext)
    defs: list[Definition] = []

    ruby_private_mode = False # .rb only
    ada_private_mode = False # .ads only

    for idx, raw_line in enumerate(lines):
        # ... (truncated, see source file for more)

```

doc.py (line 526) - [public] collect

```
# -----#
# Collecting everything
# -----#

def collect(root: str, exts: set[str] | None, exclude_dirs: set[str], max_body_lines: int):
    all_defs: list[Definition] = []
    md_files: list[MarkdownFile] = []

    for rel_path, full_path, ext in iter_source_files(root, exts, exclude_dirs):
        source = read_text(full_path)
        if ext in MD_EXTS:
            md_files.append(MarkdownFile(path=rel_path.replace(os.sep, "/"), content=source))
            continue
        rel_norm = rel_path.replace(os.sep, "/")
        if ext == ".py":
            all_defs.extend(parse_python(rel_norm, full_path, source))
    # ... (truncated, see source file for more)
```

doc.py (line 550) - [public] build_styles

```
# -----#
# Building the PDF
# -----#

def build_styles():
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(
        name="SectionTitle", parent=styles["Heading1"],
        fontSize=18, spaceAfter=14, spaceBefore=6, textColor=colors.HexColor("#1a1a2e"),
    ))
    styles.add(ParagraphStyle(
        name="LangHeading", parent=styles["Heading2"],
        fontSize=13, spaceBefore=14, spaceAfter=6, textColor=colors.HexColor("#16213e"),
    ))
    styles.add(ParagraphStyle(
        name="ItemPath", parent=styles["Normal"],
    ))
    # ... (truncated, see source file for more)
```

doc.py (line 587) - [public] group_by_language

```
def group_by_language(defs: list[Definition]):
    groups: dict[str, list[Definition]] = {}
    for d in defs:
        groups.setdefault(d.language, []).append(d)
    return dict(sorted(groups.items()))
```

doc.py (line 594) - [public] path_line_label

```
def path_line_label(d: Definition) -> str:
    return f"{d.path} (line {d.lineno})"
```

doc.py (line 598) - [public] render_signature_section

```
def render_signature_section(story, styles, title: str, defs: list[Definition], max_items: int):
    story.append(Paragraph(escape(title), styles["SectionTitle"]))
    if not defs:
        story.append(Paragraph("No items in this section.", styles["Normal"]))
        return
    truncated = len(defs) > max_items
    defs = defs[:max_items]
    for lang, items in group_by_language(defs).items():
        story.append(Paragraph(escape(lang), styles["LangHeading"]))
        for d in items:
            block = [
                Paragraph(escape(path_line_label(d)), styles["ItemPath"]),
            ]
    # ... (truncated, see source file for more)
```

```

doc.py (line 622) - [public] render_fragments_section
def render_fragments_section(story, styles, defs: list[Definition], max_items: int):
    story.append(Paragraph("3. Code fragments with comments", styles["SectionTitle"]))
    if not defs:
        story.append(Paragraph("No items.", styles["Normal"]))
        return
    truncated = len(defs) > max_items
    defs = defs[:max_items]
    for lang, items in group_by_language(defs).items():
        story.append(Paragraph(escape(lang), styles["LangHeading"]))
        for d in items:
            block = [Paragraph(escape(path_line_label(d)) + f" - [{ 'public' if d.is_public else 'internal' }]", styles["Normal"])]
            if d.comment:
                # ... (truncated, see source file for more)

doc.py (line 647) - [public] render_markdown_section
def render_markdown_section(story, styles, md_files: list[MarkdownFile]):
    story.append(Paragraph("4. .md files (in order)", styles["SectionTitle"]))
    if not md_files:
        story.append(Paragraph("No .md files in the project.", styles["Normal"]))
        return
    for mf in md_files:
        story.append(Paragraph(escape(mf.path), styles["MdHeading"]))
        content = mf.content
        if len(content) > 20000:
            content = content[:20000] + "\n\n... (truncated, file is very long)"
        story.append(Preformatted(content, styles["CodeBlock"]))

doc.py (line 660) - [internal] _new_doc
def _new_doc(path: str):
    return SimpleDocTemplate(
        path, pagesize=A4,
        leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=16 * mm,
    )

doc.py (line 667) - [internal] _summary_paragraph
def _summary_paragraph(styles, root, public_defs, internal_defs, md_files):
    block = [
        Paragraph("Project technical documentation", styles["Title"]),
        Paragraph(escape(f"Source directory: {os.path.abspath(root)}"), styles["Normal"]),
        Spacer(1, 4),
        Paragraph(
            f"Public API/ABI: {len(public_defs)} | Internal: {len(internal_defs)} | "
            f".md files: {len(md_files)}", styles["Normal"],
        ),
    ]
    return block

doc.py (line 680) - [public] build_pdf
def build_pdf(output_path: str, root: str, public_defs, internal_defs, all_defs_for_fragments,
              md_files, max_items: int, split: bool):
    styles = build_styles()

    if not split:
        doc = _new_doc(output_path)
        story = []
        story += _summary_paragraph(styles, root, public_defs, internal_defs, md_files)
        story.append(PageBreak())
        render_signature_section(story, styles, "1. Public API / ABI", public_defs, max_items)
        story.append(PageBreak())
        render_signature_section(story, styles, "2. Internal functions", internal_defs, max_items)
        # ... (truncated, see source file for more)

```

doc.py (line 734) - [public] main

```
# ----- #
# main
# ----- #

def main():
    parser = argparse.ArgumentParser(description="PDF documentation generator for a whole repo (no compilation)")
    parser.add_argument("root", help="Root directory of the project to scan")
    parser.add_argument("-o", "--output", default="documentation.pdf", help="Output PDF file")
    parser.add_argument("--max-body-lines", type=int, default=12,
                        help="How many lines of code to show in the fragments section (default: 12)")
    parser.add_argument("--max-items-per-section", type=int, default=4000,
                        help="Cap on entries per section (default: 4000)")
    parser.add_argument("--exclude", default="", help="Extra directories to skip, comma-separated")
    parser.add_argument("--ext", default="", help="Restrict to these extensions only, e.g.: .py,.js,.go")
    parser.add_argument("--split", action="store_true",
                        help="Instead of one PDF, generate a separate file per section ")
    # ... (truncated, see source file for more)
```

Rust

c.rs (line 57) - [internal] BlockHeader

```
/* Struktura nagłówka bloku pamięci */
typedef struct BlockHeader {
    size_t size; /* Rozmiar danych użytkownika */
    int is_free; /* Czy blok jest wolny */

    struct BlockHeader* next; /* Wskaźnik do następnego bloku */
    struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
} BlockHeader;

/* Globalna pula pamięci i lista bloków */
static uint8_t memory_pool[MEMORY_POOL_SIZE];
static BlockHeader* free_list = NULL;
static int allocator_initialized = 0;

/* Makro do wyrównywania rozmiaru */
#define ALIGN(size) (((size) + (ALIGNMENT - 1)) & ~(ALIGNMENT - 1))

// ... (truncated, see source file for more)
```

c.rs (line 58) - [internal] BlockHeader

```
/* Struktura nagłówka bloku pamięci */
typedef struct BlockHeader {
    size_t size; /* Rozmiar danych użytkownika */
    int is_free; /* Czy blok jest wolny */
    struct BlockHeader* next; /* Wskaźnik do następnego bloku */

    struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
} BlockHeader;

/* Globalna pula pamięci i lista bloków */
static uint8_t memory_pool[MEMORY_POOL_SIZE];
static BlockHeader* free_list = NULL;
static int allocator_initialized = 0;

/* Makro do wyrównywania rozmiaru */
#define ALIGN(size) (((size) + (ALIGNMENT - 1)) & ~(ALIGNMENT - 1))

/* Inicjalizacja alokatora */
// ... (truncated, see source file for more)
```


comgrub/build.rs (line 1) - [internal] main

```
fn main() {
    nasm_rs::compile_library("boot.asm", &["src/boot.asm"]).unwrap();

    // █cie█ka bezwzgl█dna: `cargo:rustc-link-arg` przekazuje argument do linkera
    // w katalogu docelowym, wi█c relatywna █cie█ka by si█ nie rozwi█za█a.
    let manifest = std::env::var("CARGO_MANIFEST_DIR").unwrap();

    println!("cargo:rustc-link-arg=-T{/linker.ld", manifest);
    println!("cargo:rustc-link-lib=static=boot");

    println!("cargo:rerun-if-changed=src/boot.asm");
    println!("cargo:rerun-if-changed=linker.ld");
    // ... (truncated, see source file for more)
```

comgrub/src/main.rs (line 11) - [internal] kernel_main

```
fn kernel_main(magic: u32, info: *const u8) -> !;
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {
        unsafe { core::arch::asm!("hlt", options(nomem, nostack, preserves_flags)) }
    }
}

/// Punkt wej█cia Rust wywo█ywany z `boot.asm` (`call _rust_entry`).
#[unsafe(no_mangle)]
// ... (truncated, see source file for more)
```

comgrub/src/main.rs (line 15) - [internal] panic

```
fn panic(_info: &PanicInfo) -> ! {
    loop {
        unsafe { core::arch::asm!("hlt", options(nomem, nostack, preserves_flags)) }
    }
}

/// Punkt wej█cia Rust wywo█ywany z `boot.asm` (`call _rust_entry`).
#[unsafe(no_mangle)]
pub extern "C" fn _rust_entry(magic: u32, info: *const u8) -> ! {
    unsafe { kernel_main(magic, info) }
}
```

comlimine/build.rs (line 1) - [internal] main

```
fn main() {
    cc::Build::new()
        .file("src/main.c")
        .include("src")
        .no_default_flags(true)
        .flag("-ffreestanding")
        .flag("-nostdlib")
        .flag("-mno-red-zone")
        .flag("-mno-mmx")
        .flag("-mno-sse")
        .flag("-mno-sse2")
        .compile("limine_entry");
    // ... (truncated, see source file for more)
```

comlimine/src/main.rs (line 1) - [internal] main

```
fn main() {}
```

drivers/amdgpu_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/audiodriver/build.rs (line 12) - [internal] main

```
fn main() {
    let manifest = PathBuf::from(env::var("CARGO_MANIFEST_DIR").unwrap());
    let out = PathBuf::from(env::var("OUT_DIR").unwrap());

    let odin_dir = manifest.join("src/odin");
    let lib_name = "audiodriver_odin";
    let lib_path = out.join(format!("lib{}.a", lib_name));

    println!("cargo:rerun-if-changed=build.rs");
    println!("cargo:rerun-if-changed={}", odin_dir.display());

    let odin = env::var("ODIN").unwrap_or_else(|_| "odin".to_string());
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 4) - [public] JackMgr

```
pub struct JackMgr {
    present: bool,
    amp_on: bool,
}

impl JackMgr {
    pub const fn new() -> Self {
        Self { present: false, amp_on: false }
    }

    pub fn tick(&mut self) {
        let now = ad::jack_present();
        // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 14) - [public] tick

```
pub fn tick(&mut self) {
    let now = ad::jack_present();

    if now != self.present {
        self.present = now;

        if now {
            ds::log::ds_log("jack: present");
            self.set_amp(true);
        } else {
            ds::log::ds_log("jack: gone");
            self.set_amp(false);
        }
    }
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 30) - [public] set_amp

```
pub fn set_amp(&mut self, on: bool) {
    self.amp_on = on;
    ad::set_amp(on);
}

pub fn present(&self) -> bool {
    self.present
}

/// Czy wzmacniacz (amp) jest aktualnie włączony.
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
// ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 35) - [public] present

```
pub fn present(&self) -> bool {
    self.present
}

/// Czy wzmacniacz (amp) jest aktualnie włączony.
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
}
```

drivers/audiodriver/src/jacklib.rs (line 40) - [public] amp_enabled

/// Czy wzmacniacz (amp) jest aktualnie włączony.

```
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
```

drivers/audiodriver/src/lib.rs (line 7) - [internal] ad_init

```
fn ad_init(nam_va: u64, bm_va: u64) -> i32;
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 8) - [internal] ad_play

```
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 9) - [internal] ad_capture

```
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 10) - [internal] ad_stop

```
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 11) - [internal] ad_jack_present

```
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 12) - [internal] ad_set_amp

```
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 13) - [internal] ad_position

```
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    // ... (truncated, see source file for more)
}
```

drivers/audiodriver/src/lib.rs (line 16) - [public] init

```
pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 20) - [public] play
    pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 24) - [public] capture
    pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 28) - [public] stop
    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 32) - [public] jack_present
    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

```
drivers/audiodriver/src/lib.rs (line 36) - [public] set_amp
    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

```
drivers/audiodriver/src/lib.rs (line 40) - [public] position
    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

drivers/audiodriver/src/main.rs (line 8) - [internal] main

```
/// Punkt wejścia binarki pomocniczej.
///
/// Właściwy sterownik działa w driver space przez `ds_entry` (patrz
/// `driverspace/src/main.rs`); ta binarka jest cienkim wrapperem CLI, który
/// pozwala sprawdzić stan gniazda jack z poziomu hosta.
```

```
fn main() {
    let mut mgr = ad::jacklib::JackMgr::new();
    mgr.tick();

    println!(
        "audiodriver: jack present = {}, amp on = {}",
        mgr.present(),
        mgr.amp_enabled()
    );
}
```

drivers/audiodriver/src/tone.rs (line 1) - [public] fill_square

```
pub fn fill_square(buf: &mut [u8], periods: u32) {
    let half = buf.len() as u32 / (periods * 2);

    for i in 0..buf.len() as u32 {
        let v = if (i / half.max(1)) % 2 == 0 { 0xC0 } else { 0x40 };
        buf[i as usize] = v;
    }
}
```

drivers/intelgpu_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/netcam_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/wacomgraphic_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace/src/drivers/storage.rs (line 9) - [public] StorageDrv

```
/// Storage driver stub. It only satisfies the driver registry so that
/// driver space can boot; a real disk driver can slot in later.
```

```
pub struct StorageDrv {
    registered: bool,
}

impl StorageDrv {
    pub const fn new() -> Self {
        Self { registered: false }
    }
}

impl Driver for StorageDrv {
    fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
        // ... (truncated, see source file for more)
    }
}
```

driverspace/src/drivers/storage.rs (line 20) - [internal] init

```
fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
    self.registered = true;
    Ok(())
}
}
```

driverspace/src/main.rs (line 32) - [internal] panic

```
fn panic(_info: &core::panic::PanicInfo) -> ! {  
    loop {}  
}
```

driverspace_workspace/crates/ds-manager/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 4) - [public] RegistryEntry

```
pub struct RegistryEntry {  
    pub vendor_id: u16,  
    pub device_id: u16,  
    pub driver_id: DriverId,  
    pub driver_name: &'static str,  
}  
  
pub struct Registry {  
    entries: &'static [RegistryEntry],  
}  
  
impl Registry {  
    // ... (truncated, see source file for more)
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 11) - [public] Registry

```
pub struct Registry {  
    entries: &'static [RegistryEntry],  
}  
  
impl Registry {  
    pub const fn new(entries: &'static [RegistryEntry]) -> Self {  
        Self { entries }  
    }  
  
    pub fn entries(&self) -> &[RegistryEntry] {  
        self.entries  
    }  
    // ... (truncated, see source file for more)
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 20) - [public] entries

```
pub fn entries(&self) -> &[RegistryEntry] {  
    self.entries  
}  
  
pub fn find(&self, vendor_id: u16, device_id: u16) -> Option<&RegistryEntry> {  
    self.entries.iter().find(|e| e.vendor_id == vendor_id && e.device_id == device_id)  
}  
  
pub static DEFAULT_REGISTRY: Registry = Registry::new(&[  
    RegistryEntry {  
        vendor_id: 0x1234,  
        // ... (truncated, see source file for more)
```

driverspace_workspace/crates/ds-registry/src/database.rs (line 24) - [public] find

```
pub fn find(&self, vendor_id: u16, device_id: u16) -> Option<&RegistryEntry> {  
    self.entries.iter().find(|e| e.vendor_id == vendor_id && e.device_id == device_id)  
}  
  
pub static DEFAULT_REGISTRY: Registry = Registry::new(&[  
    RegistryEntry {  
        vendor_id: 0x1234,  
        device_id: 0x1111,  
        driver_id: 1,  
        driver_name: "qemu-vga",  
    },  
    // ... (truncated, see source file for more)
```

driverspace_workspace/crates/ds-registry/src/matching.rs (line 5) - [public]

find_driver_for_device

```
pub fn find_driver_for_device(vendor_id: u16, device_id: u16) -> Option<DriverId> {
    DEFAULT_REGISTRY.find(vendor_id, device_id).map(|entry| entry.driver_id)
}

/// Zwraca nazwę sterownika dla danego ID.
pub fn get_driver_name(driver_id: DriverId) -> Option<'static str> {
    DEFAULT_REGISTRY.entries().iter()
        .find(|e| e.driver_id == driver_id)
        .map(|e| e.driver_name)
}
```

driverspace_workspace/crates/ds-registry/src/matching.rs (line 10) - [public] get_driver_name

/// Zwraca nazwę sterownika dla danego ID.

```
pub fn get_driver_name(driver_id: DriverId) -> Option<'static str> {
    DEFAULT_REGISTRY.entries().iter()
        .find(|e| e.driver_id == driver_id)
        .map(|e| e.driver_name)
}
```

driverspace_workspace/crates/ds-registry/src/state.rs (line 5) - [public] DeviceStatus

```
pub enum DeviceStatus {
    /// Urządzenie wykryte, ale sterownik jeszcze nie załadowany.
    Discovered,
    /// Sterownik załadowany i zainicjalizowany.
    Attached,
    /// Urządzenie odłączane (hotunplug lub błąd).
    Detached,
    /// Wystąpił błąd krytyczny (sterownik się wysypał, sprzęt nie odpowiada).
    Error,
}

/// Struktura przechowująca stan konkretnego urządzenia.
// ... (truncated, see source file for more)
```

driverspace_workspace/crates/ds-registry/src/state.rs (line 17) - [public] DeviceState

/// Struktura przechowująca stan konkretnego urządzenia.

```
pub struct DeviceState {
    pub vendor_id: u16,
    pub device_id: u16,
    pub status: DeviceStatus,
    pub driver_id: Option<u32>, // ID sterownika, który obsługuje to urządzenie
}

impl DeviceState {
    pub const fn new(vendor_id: u16, device_id: u16) -> Self {
        Self {
            vendor_id,
            device_id,
        }
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/crates/ds-registry/src/state.rs (line 34) - [public] set_attached

```
pub fn set_attached(&mut self, driver_id: u32) {
    self.status = DeviceStatus::Attached;
    self.driver_id = Some(driver_id);
}

pub fn set_detached(&mut self) {
    self.status = DeviceStatus::Detached;
    self.driver_id = None;
}

pub fn set_error(&mut self) {
    self.status = DeviceStatus::Error;
    // ... (truncated, see source file for more)
}
```


driverspace_workspace/crates/ds-registry/src/state.rs (line 39) - [public] set_detached

```
pub fn set_detached(&mut self) {
    self.status = DeviceStatus::Detached;
    self.driver_id = None;
}

pub fn set_error(&mut self) {
    self.status = DeviceStatus::Error;
}
}
```

driverspace_workspace/crates/ds-registry/src/state.rs (line 44) - [public] set_error

```
pub fn set_error(&mut self) {
    self.status = DeviceStatus::Error;
}
}
```

driverspace_workspace/drivers/amdgpu-driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace_workspace/drivers/audiodriver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace_workspace/drivers/intelgpu-driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace_workspace/drivers/netcam_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 42) - [internal] VgpuInfo

```
struct VgpuInfo {
    fb: *mut u32,
    width: u32,
    height: u32,
    bpp: u32,
    stride: u32,
}

#[no_mangle]
pub extern "C" fn driver_main() {
    ds_info!("VGPU", "Virtual GPU driver starting...");

    // ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 67) - [internal] vgpu_init

```
fn vgpu_init(w: u32, h: u32) -> bool {
    unsafe {
        if G_READY {
            return true;
        }
    }

    ds_info!("VGPU", "Searching for VGA device...");

    let bdf = match find_vga_device() {
        Some(bdf) => bdf,
        None => {
            // ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 138) - [internal] find_vga_device

```
fn find_vga_device() -> Option<u64> {
    let mut msg = kapi_abi::DsMsg::new(DsCmd::PciFind as u32);
    msg.arg0 = PCI_CLASS_VGA as u64;

    let reply = kapi_syscall::sys_ipc_call(DsCmd::PciFind, PCI_CLASS_VGA as u64, 0, 0);

    if reply.is_ok() && reply.arg0 != 0 {
        Some(reply.arg0)
    } else {
        None
    }
}

// ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 151) - [internal] read_pci_config

```
fn read_pci_config(bdf: u64, offset: u32) -> u32 {
    let reply = kapi_syscall::sys_ipc_call(DsCmd::PciRead, bdf, offset as u64, 0);

    if reply.is_ok() {
        reply.arg0 as u32
    } else {
        0xFFFFFFFF
    }
}

fn read_pci_bar(bdf: u64, bar: u32) -> u64 {
    let offset = 0x10 + (bar * 4);
    // ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 161) - [internal] read_pci_bar

```
fn read_pci_bar(bdf: u64, bar: u32) -> u64 {
    let offset = 0x10 + (bar * 4);
    let val = read_pci_config(bdf, offset);

    if val == 0 || val == 0xFFFFFFFF {
        return 0;
    }

    (val & 0xFFFFFFFF0) as u64
}

fn enable_pci_device(bdf: u64) {
    // ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 172) - [internal] enable_pci_device

```
fn enable_pci_device(bdf: u64) {
    let cmd_offset = 0x04;
    let mut cmd = read_pci_config(bdf, cmd_offset);
    cmd |= 0x07;

    let mut msg = kapi_abi::DsMsg::new(DsCmd::PciWrite as u32);
    msg.arg0 = bdf;
    msg.arg1 = cmd_offset as u64;
    msg.arg2 = cmd as u64;

    kapi_syscall::sys_ipc_call(DsCmd::PciWrite, bdf, cmd_offset as u64, cmd as u64);
}

// ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 185) - [internal] vbe_write

```
fn vbe_write(idx: u16, val: u16) {
    unsafe {
        core::arch::asm!("out dx, ax", in("dx") VBE_INDEX_PORT, in("ax") idx, options(nomem, nostack, preserve_flags),
        core::arch::asm!("out dx, ax", in("dx") VBE_DATA_PORT, in("ax") val, options(nomem, nostack, preserve_flags));
    }
}

fn vgpu_clear(color: u32) {
    unsafe {
        if !G_READY {
            return;
        }
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 192) - [internal] vgpu_clear

```
fn vgpu_clear(color: u32) {
    unsafe {
        if !G_READY {
            return;
        }

        let fb = G_INFO.fb;
        let n = (G_INFO.width * G_INFO.height) as usize;

        for i in 0..n {
            core::ptr::write_volatile(fb.add(i), color);
        }
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 207) - [internal] vgpu_pixel

```
fn vgpu_pixel(x: u32, y: u32, color: u32) {
    unsafe {
        if !G_READY || x >= G_INFO.width || y >= G_INFO.height {
            return;
        }

        let offset = (y * G_INFO.stride + x) as usize;
        core::ptr::write_volatile(G_INFO.fb.add(offset), color);
    }
}

#[panic_handler]
// ... (truncated, see source file for more)
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 219) - [internal] panic

```
fn panic(_info: &core::panic::PanicInfo) -> ! {
    ds_error!("VGPU", "PANIC!");
    loop {
        kapi_syscall::sys_yield();
    }
}
```

driverspace_workspace/drivers/wacomgraphic_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace_workspace/lib/ds-formal-ffi/build.rs (line 10) - [internal] main

```
fn main() {
    // ■cie■ka do projektu Ada SPARK
    let spark_project_dir = PathBuf::from("../..formal/ds-spark-core");

    println!("cargo:rerun-if-changed=../..formal/ds-spark-core/src");

    let gprbuild_available = Command::new("gprbuild")
        .arg("--version")
        .status()
        .map(|s| s.success())
        .unwrap_or(false);

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 7) - [public] ds_spark_safe_add_32

// Odpowiednik: function Safe_Add_32 (A, B : UInt32) return UInt32

```
pub fn ds_spark_safe_add_32(a: u32, b: u32) -> u32;

// Odpowiednik: function Safe_Array_Index ...
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
}

// Bezpieczne wrappery dla reszty kodu Rust w workspace
pub mod math {
    use super::*;

    /// Bezpieczne dodawanie. W trybie debug Rust i tak to sprawdzi,
    /// ale w release mode (gdzie overflow to undefined behavior w C/Ada),
    /// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 10) - [public]

ds_spark_safe_array_index

// Odpowiednik: function Safe_Array_Index ...

```
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
}

// Bezpieczne wrappery dla reszty kodu Rust w workspace
pub mod math {
    use super::*;

    /// Bezpieczne dodawanie. W trybie debug Rust i tak to sprawdzi,
    /// ale w release mode (gdzie overflow to undefined behavior w C/Ada),
    /// ta funkcja ma matematyczny dowód z SPARK, ■e nigdy nie overflownie.
    #[inline(always)]
    pub fn safe_add(a: u32, b: u32) -> u32 {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 21) - [public] safe_add

```
pub fn safe_add(a: u32, b: u32) -> u32 {
    // W produkcji wo■amy udowodniony kod z Ady
    unsafe { ds_spark_safe_add_32(a, b) }
}

unsafe extern "C" {
    pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
    pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
}

pub mod pmm {
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 28) - [public]

ds_spark_buddy_get_buddy

```
pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;

}

pub mod pmm {
    use super::*;
    #[inline(always)]
    pub fn get_buddy(addr: u64, exp: u64) -> u64 {
        // Wywołanie kodu, który ma MATEMATYCZNY DOWÓD poprawności
        unsafe { ds_spark_buddy_get_buddy(addr, exp) }
    }
}
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 29) - [public] ds_spark_safe_dma_end

```
pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;

}

pub mod pmm {
    use super::*;
    #[inline(always)]
    pub fn get_buddy(addr: u64, exp: u64) -> u64 {
        // Wywołanie kodu, który ma MATEMATYCZNY DOWÓD poprawności
        unsafe { ds_spark_buddy_get_buddy(addr, exp) }
    }
}
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 35) - [public] get_buddy

```
pub fn get_buddy(addr: u64, exp: u64) -> u64 {
    // Wywołanie kodu, który ma MATEMATYCZNY DOWÓD poprawności
    unsafe { ds_spark_buddy_get_buddy(addr, exp) }
}

}
```

driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 1) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 10) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}

}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 7) - [public] RequestState

```
pub enum RequestState {
    Pending = 0,
    InProgress = 1,
    Completed = 2,
    Failed = 3,
}

/// High-level I/O request with metadata.
pub struct IoRequest {
    pub id: u64,
    pub lba: u64,
    pub block_count: u32,
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 15) - [public] IoRequest

/// High-level I/O request with metadata.

```
pub struct IoRequest {
    pub id: u64,
    pub lba: u64,
    pub block_count: u32,
    pub buffer_ptr: *mut u8,
    pub buffer_len: usize,
    pub is_write: bool,
    pub priority: u8,
    state: AtomicU32,
    pub callback: Option<fn(&IoRequest)>,
}
```

// ... (truncated, see source file for more)

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 28) - [public] new

```
pub fn new(id: u64, lba: u64, block_count: u32, buffer: *mut u8, len: usize, is_write: bool) -> Self {
    Self {
        id,
        lba,
        block_count,
        buffer_ptr: buffer,
        buffer_len: len,
        is_write,
        priority: 0,
        state: AtomicU32::new(RequestState::Pending as u32),
        callback: None,
    }
}
```

// ... (truncated, see source file for more)

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 42) - [public] state

```
pub fn state(&self) -> RequestState {
    match self.state.load(Ordering::Acquire) {
        0 => RequestState::Pending,
        1 => RequestState::InProgress,
        2 => RequestState::Completed,
        3 => RequestState::Failed,
        _ => RequestState::Failed,
    }
}
```

```
pub fn mark_in_progress(&self) {
    self.state.store(RequestState::InProgress as u32, Ordering::Release);
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 52) - [public] mark_in_progress

```
pub fn mark_in_progress(&self) {
    self.state.store(RequestState::InProgress as u32, Ordering::Release);
}
```

```
pub fn mark_completed(&self) {
    self.state.store(RequestState::Completed as u32, Ordering::Release);
    if let Some(cb) = self.callback {
        cb(self);
    }
}
```

```
pub fn mark_failed(&self) {
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 56) - [public] mark_completed

```
pub fn mark_completed(&self) {
    self.state.store(RequestState::Completed as u32, Ordering::Release);
    if let Some(cb) = self.callback {
        cb(self);
    }
}

pub fn mark_failed(&self) {
    self.state.store(RequestState::Failed as u32, Ordering::Release);
}
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 63) - [public] mark_failed

```
pub fn mark_failed(&self) {
    self.state.store(RequestState::Failed as u32, Ordering::Release);
}
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 6) - [public] BlockRequest

/// Represents a single I/O request to a block device.

```
pub struct BlockRequest {
    pub lba: u64,
    pub block_count: u32,
    pub buffer_ptr: *mut u8,
    pub buffer_len: usize,
    pub is_write: bool,
}
```

/// Core trait for all block devices.

/// Drivers must implement this to integrate with the block subsystem.

```
pub trait BlockDevice {
    /// Read blocks from the device starting at LBA.
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 16) - [public] BlockDevice

/// Core trait for all block devices.

/// Drivers must implement this to integrate with the block subsystem.

```
pub trait BlockDevice {
    /// Read blocks from the device starting at LBA.
    fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;

    /// Write blocks to the device starting at LBA.
    fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

    /// Flush internal caches to physical media.
    fn flush(&mut self) -> Result<(), DsError>;

    /// Get device geometry (block size, total blocks).
    fn geometry(&self) -> BlockGeometry;
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 18) - [internal] read_blocks

/// Read blocks from the device starting at LBA.

```
fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;

/// Write blocks to the device starting at LBA.
fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

/// Flush internal caches to physical media.
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 21) - [internal] write_blocks

/// Write blocks to the device starting at LBA.

```
fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

/// Flush internal caches to physical media.
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 24) - [internal] flush

/// Flush internal caches to physical media.

```
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 27) - [internal] geometry

/// Get device geometry (block size, total blocks).

```
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}

}

/// Physical geometry of a block device.
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 30) - [internal] supports_trim

/// Check if device supports TRIM/discard.

```
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}

}

/// Physical geometry of a block device.
#[derive(Debug, Clone, Copy)]
pub struct BlockGeometry {
    pub block_size: u32,
    // ... (truncated, see source file for more)
```


driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 33) - [internal] submit_request

/// Optional: Submit async request (for advanced schedulers).

```
    fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
        Err(DsError::Unknown)
    }
}

/// Physical geometry of a block device.
#[derive(Debug, Clone, Copy)]
pub struct BlockGeometry {
    pub block_size: u32,
    pub total_blocks: u64,
    pub max_transfer_blocks: u32,
}

// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 40) - [public] BlockGeometry

```
pub struct BlockGeometry {
    pub block_size: u32,
    pub total_blocks: u64,
    pub max_transfer_blocks: u32,
}

impl BlockGeometry {
    pub fn total_bytes(&self) -> u64 {
        self.total_blocks * self.block_size as u64
    }
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 47) - [public] total_bytes

```
    pub fn total_bytes(&self) -> u64 {
        self.total_blocks * self.block_size as u64
    }
}
```

driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 1) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 10) - [internal] it_works

```
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}
```

driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 1) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 10) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 6) - [public] ManagerClient

```
pub struct ManagerClient {
    manager_ep: Handle,
}

impl ManagerClient {
    pub const fn new(ep: Handle) -> Self {
        Self { manager_ep: ep }
    }

    pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
        let reply = sys_ipc_call(DsCmd::MemMapMmio, phys_base, size, 0);
        if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 15) - [public] request_mmio

```
pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::MemMapMmio, phys_base, size, 0);
    if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
}

pub fn bind_irq(&self, irq_num: u32) -> Result<(), DsError> {
    let reply = sys_ipc_call(DsCmd::BindIrq, irq_num as u64, 0, 0);
    if reply.is_ok() { Ok(()) } else { Err(reply.error_status()) }
}

pub fn alloc_dma(&self, size: u64, flags: u32) -> Result<(u64, u64), DsError> {
    let reply = sys_ipc_call(DsCmd::MemAllocDma, size, flags as u64, 0);
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 20) - [public] bind_irq

```
pub fn bind_irq(&self, irq_num: u32) -> Result<(), DsError> {
    let reply = sys_ipc_call(DsCmd::BindIrq, irq_num as u64, 0, 0);
    if reply.is_ok() { Ok(()) } else { Err(reply.error_status()) }
}

pub fn alloc_dma(&self, size: u64, flags: u32) -> Result<(u64, u64), DsError> {
    let reply = sys_ipc_call(DsCmd::MemAllocDma, size, flags as u64, 0);
    if reply.is_ok() { Ok((reply.arg0, reply.arg1)) } else { Err(reply.error_status()) }
}

pub fn pci_find(&self, class_code: u32) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PciFind, class_code as u64, 0, 0);
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 25) - [public] alloc_dma

```
pub fn alloc_dma(&self, size: u64, flags: u32) -> Result<(u64, u64), DsError> {
    let reply = sys_ipc_call(DsCmd::MemAllocDma, size, flags as u64, 0);
    if reply.is_ok() { Ok((reply.arg0, reply.arg1)) } else { Err(reply.error_status()) }
}

pub fn pci_find(&self, class_code: u32) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PciFind, class_code as u64, 0, 0);
    if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
}

pub fn get_page_phys(&self, virt_addr: u64) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PagePhys, virt_addr, 0, 0);
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 30) - [public] pci_find

```
pub fn pci_find(&self, class_code: u32) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PciFind, class_code as u64, 0, 0);
    if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
}

pub fn get_page_phys(&self, virt_addr: u64) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PagePhys, virt_addr, 0, 0);
    if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
}
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 35) - [public] get_page_phys

```
pub fn get_page_phys(&self, virt_addr: u64) -> Result<u64, DsError> {
    let reply = sys_ipc_call(DsCmd::PagePhys, virt_addr, 0, 0);
    if reply.is_ok() { Ok(reply.arg0) } else { Err(reply.error_status()) }
}
}
```

driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 10) - [public] MessageDispatcher

```
pub struct MessageDispatcher {
    handlers: [Option<HandlerFn>; 64],
}

impl MessageDispatcher {
    pub const fn new() -> Self {
        Self {
            handlers: [None; 64],
        }
    }

    pub fn register(&mut self, opcode: Opcode, handler: HandlerFn) -> Result<(), DsError> {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 21) - [public] register

```
pub fn register(&mut self, opcode: Opcode, handler: HandlerFn) -> Result<(), DsError> {
    let idx = (opcode as u32 & 0x3F) as usize;
    if self.handlers[idx].is_some() {
        return Err(DsError::DeviceBusy);
    }
    self.handlers[idx] = Some(handler);
    Ok(())
}

pub fn dispatch(&self, sender: Handle, opcode_raw: u32, payload_ptr: *const u8, payload_len: u32) ->
    let opcode = Opcode::from_u32(opcode_raw).ok_or(DsError::InvalidMessage?);
    let idx = (opcode_raw & 0x3F) as usize;
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 30) - [public] dispatch

```
pub fn dispatch(&self, sender: Handle, opcode_raw: u32, payload_ptr: *const u8, payload_len: u32) ->
    let opcode = Opcode::from_u32(opcode_raw).ok_or(DsError::InvalidMessage)?;
    let idx = (opcode_raw & 0x3F) as usize;

    if let Some(handler) = self.handlers[idx] {
        handler(sender, payload_ptr, payload_len)
    } else {
        ds_error!("Unhandled opcode: {:?}", opcode);
        Err(DsError::InvalidMessage)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 7) - [public] LogBuffer

```
pub struct LogBuffer {
    buf: [u8; MAX_LOG_LINE_LEN],
    pos: usize,
}

impl LogBuffer {
    pub const fn new() -> Self {
        Self {
            buf: [0; MAX_LOG_LINE_LEN],
            pos: 0,
        }
    }
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 20) - [public] as_bytes

```
pub fn as_bytes(&self) -> &[u8] {
    &self.buf[..self.pos]
}

pub fn reset(&mut self) {
    self.pos = 0;
}

impl Write for LogBuffer {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        let bytes = s.as_bytes();
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 24) - [public] reset

```
pub fn reset(&mut self) {
    self.pos = 0;
}

impl Write for LogBuffer {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        let bytes = s.as_bytes();
        let remaining = self.buf.len() - self.pos;
        let copy_len = if bytes.len() > remaining { remaining } else { bytes.len() };

        self.buf[self.pos..self.pos + copy_len].copy_from_slice(&bytes[..copy_len]);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 30) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> fmt::Result {
    let bytes = s.as_bytes();
    let remaining = self.buf.len() - self.pos;
    let copy_len = if bytes.len() > remaining { remaining } else { bytes.len() };

    self.buf[self.pos..self.pos + copy_len].copy_from_slice(&bytes[..copy_len]);
    self.pos += copy_len;
    Ok(())
}
```

driverspace_workspace/lib/ds-log/src/levels.rs (line 6) - [public] LogLevel

```
pub enum LogLevel {
    Trace = 0,
    Info = 1,
    Warn = 2,
    Error = 3,
    Fatal = 4,
}

impl LogLevel {
    pub const fn as_str(&self) -> &'static str {
        match self {
            Self::Trace => "TRACE",
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 15) - [public] EndpointRef

```
pub struct EndpointRef;

impl EndpointRef {
    pub const fn new() -> Self { Self }

    pub fn is_valid(&self) -> bool {
        MANAGER_ENDPOINT_RAW.load(Ordering::Acquire) != 0
    }

    /// Handle endpointu Managera (0 = niezarejestrowany).
    pub fn handle(&self) -> u32 {
        MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
    }
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 20) - [public] is_valid

```
pub fn is_valid(&self) -> bool {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire) != 0
}

/// Handle endpointu Managera (0 = niezarejestrowany).
pub fn handle(&self) -> u32 {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
}

pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 25) - [public] handle

/// Handle endpointu Managera (0 = niezarejestrowany).

```
pub fn handle(&self) -> u32 {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
}

pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}

pub static MANAGER_ENDPOINT: EndpointRef = EndpointRef::new();
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 29) - [public] set

```
pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}

pub static MANAGER_ENDPOINT: EndpointRef = EndpointRef::new();
```

driverspace_workspace/lib/ds-log/src/transport.rs (line 8) - [internal] uart_putchar

```
fn uart_putchar(c: u8) {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::asm!("out dx, al", in("dx") 0x3F8u16, in("al") c, options(nomem, nostack, preserves_flags))
    }
}

pub fn emit_log(level: LogLevel, module: &str, buf: &LogBuffer) {
    let bytes = buf.as_bytes();

    for &b in bytes {
        uart_putchar(b);
    }
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-log/src/transport.rs (line 15) - [public] emit_log

```
pub fn emit_log(level: LogLevel, module: &str, buf: &LogBuffer) {
    let bytes = buf.as_bytes();

    for &b in bytes {
        uart_putchar(b);
    }
    uart_putchar(b'\n');

    let _ = sys_ipc_call(
        DsCmd::Log,
        bytes.as_ptr() as u64,
        bytes.len() as u64,
    );
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 5) - [public] BuddyAllocator

```
pub struct BuddyAllocator {
    base_addr: usize,
    total_pages: usize,
    free_lists: [usize; MAX_ORDER + 1],
}

impl BuddyAllocator {
    pub const fn new(base_addr: usize, total_pages: usize) -> Self {
        Self {
            base_addr,
            total_pages,
            free_lists: [0; MAX_ORDER + 1],
        }
    }
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 20) - [public] alloc_pages

```
pub fn alloc_pages(&mut self, order: usize) -> Option<usize> {
    if order > MAX_ORDER {
        return None;
    }
    // TODO: Implement actual buddy allocation (find free block, split if necessary)
    Some(self.base_addr)
}

pub fn free_pages(&mut self, addr: usize, order: usize) {
    // TODO: Implement buddy merge logic
}

}
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 28) - [public] free_pages

```
pub fn free_pages(&mut self, addr: usize, order: usize) {
    // TODO: Implement buddy merge logic
}

}
```

driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 6) - [public] DriverHeap

```
pub struct DriverHeap {
    buddy: BuddyAllocator,
    // TODO: Add array of SlabAllocator for sizes: 16, 32, 64, 128, 256, 512, 1024
}

impl DriverHeap {
    pub const fn new() -> Self {
        Self {
            buddy: BuddyAllocator::new(0, 0),
        }
    }

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 18) - [public] init

```
pub fn init(&mut self, base_addr: usize, total_pages: usize) {
    self.buddy = BuddyAllocator::new(base_addr, total_pages);
}

unsafe impl GlobalAlloc for DriverHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        ds_error!("DriverHeap::alloc called before full initialization or unimplemented");
        null_mut()
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 1) - [public] SlabAllocator

```
pub struct SlabAllocator {
    object_size: usize,
    free_head: usize,
}

impl SlabAllocator {
    pub const fn new(object_size: usize) -> Self {
        Self {
            object_size,
            free_head: 0,
        }
    }

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 14) - [public] alloc

```
pub fn alloc(&mut self) -> Option<usize> {
    // TODO: Pop address from free_head linked list
    Some(0)
}

pub fn free(&mut self, ptr: usize) {
    // TODO: Push address to free_head linked list
}

}
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 19) - [public] free

```
pub fn free(&mut self, ptr: usize) {
    // TODO: Push address to free_head linked list
}

}
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 7) - [public] DmaFlags

```
pub struct DmaFlags(pub u32);

impl DmaFlags {
    pub const COHERENT: Self = Self(1 << 0);
    pub const HIGH_MEM: Self = Self(1 << 1);
    pub const CONTIGUOUS: Self = Self(1 << 2);
}

pub struct DmaBuffer {
    pub phys_addr: u64,
    pub virt_addr: u64,
    pub size: u64,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 15) - [public] DmaBuffer

```
pub struct DmaBuffer {
    pub phys_addr: u64,
    pub virt_addr: u64,
    pub size: u64,
    pub handle: Handle,
}

impl DmaBuffer {
    pub fn allocate(manager_ep: Handle, size: u64, flags: DmaFlags) -> Result<Self, DsError> {
        let reply = sys_ipc_call(DsCmd::MemAllocDma, size, flags.0 as u64, 0);
        if reply.is_ok() {
            Ok(Self { phys_addr: reply.arg0, virt_addr: reply.arg1, size, handle: manager_ep })
        }
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 23) - [public] allocate

```
pub fn allocate(manager_ep: Handle, size: u64, flags: DmaFlags) -> Result<Self, DsError> {
    let reply = sys_ipc_call(DsCmd::MemAllocDma, size, flags.0 as u64, 0);
    if reply.is_ok() {
        Ok(Self { phys_addr: reply.arg0, virt_addr: reply.arg1, size, handle: manager_ep })
    } else {
        Err(reply.error_status())
    }
}

impl Drop for DmaBuffer {
    fn drop(&mut self) {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 34) - [internal] drop

```
fn drop(&mut self) {
    let _ = sys_ipc_call(DsCmd::MemFreeDma, self.phys_addr, self.size, 0);
}

}
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 6) - [public] MmioRegion

```
pub struct MmioRegion {
    pub phys_base: u64,
    pub virt_base: u64,
    pub size: u64,
    pub handle: Handle,
}

impl MmioRegion {
    pub fn map(manager_ep: Handle, phys_base: u64, size: u64) -> Result<Self, DsError> {
        let reply = sys_ipc_call(DsCmd::MemMapMmio, phys_base, size, 0);
        if reply.is_ok() {
            Ok(Self { phys_base, virt_base: reply.arg0, size, handle: manager_ep })
        }
        // ... (truncated, see source file for more)
```



```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 14) - [public] map
    pub fn map(manager_ep: Handle, phys_base: u64, size: u64) -> Result<Self, DsError> {
        let reply = sys_ipc_call(DsCmd::MemMapMmio, phys_base, size, 0);
        if reply.is_ok() {
            Ok(Self { phys_base, virt_base: reply.arg0, size, handle: manager_ep })
        } else {
            Err(reply.error_status())
        }
    }
}

impl Drop for MmioRegion {
    fn drop(&mut self) {
        // ... (truncated, see source file for more)
    }
}

```

```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 25) - [internal] drop
    fn drop(&mut self) {
        let _ = sys_ipc_call(DsCmd::MemUnmapMmio, self.virt_base, self.size, 0);
    }
}

pub struct Volatile<T>(pub T);

impl<T: Copy> Volatile<T> {
    #[inline(always)]
    pub fn read(&self) -> T {
        unsafe { core::ptr::read_volatile(&self.0) }
    }
    // ... (truncated, see source file for more)
}

```

```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 30) - [public] Volatile
pub struct Volatile<T>(pub T);

impl<T: Copy> Volatile<T> {
    #[inline(always)]
    pub fn read(&self) -> T {
        unsafe { core::ptr::read_volatile(&self.0) }
    }

    #[inline(always)]
    pub fn write(&mut self, value: T) {
        unsafe { core::ptr::write_volatile(&mut self.0, value); }
    }
    // ... (truncated, see source file for more)
}

```

```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 34) - [public] read
    pub fn read(&self) -> T {
        unsafe { core::ptr::read_volatile(&self.0) }
    }

    #[inline(always)]
    pub fn write(&mut self, value: T) {
        unsafe { core::ptr::write_volatile(&mut self.0, value); }
    }
}

```

```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 39) - [public] write
    pub fn write(&mut self, value: T) {
        unsafe { core::ptr::write_volatile(&mut self.0, value); }
    }
}

```

driverspace_workspace/lib/ds-mem/src/vmm.rs (line 6) - [public] map_memory

```
pub fn map_memory(
    _manager_ep: Handle,
    phys_addr: u64,
    _virt_addr: u64,
    size: u64,
    flags: u64,
) -> Result<(), DsError> {
    let reply = sys_ipc_call(DsCmd::MemMapMmio, phys_addr, size, flags);
    if reply.is_ok() { Ok(()) } else { Err(reply.error_status()) }
}

pub fn unmap_memory(_manager_ep: Handle, virt_addr: u64, size: u64) -> Result<(), DsError> {
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-mem/src/vmm.rs (line 17) - [public] unmap_memory

```
pub fn unmap_memory(_manager_ep: Handle, virt_addr: u64, size: u64) -> Result<(), DsError> {
    let reply = sys_ipc_call(DsCmd::MemUnmapMmio, virt_addr, size, 0);
    if reply.is_ok() { Ok(()) } else { Err(reply.error_status()) }
}
```

driverspace_workspace/lib/kapi-abi/build.rs (line 9) - [internal] main

```
/// Build script dla `kapi-abi`.
///
/// Crate jest w 100% `no_std` i nie generuje kodu — skrypt istnieje tylko po to,
/// żeby Cargo miało poprawny entry point (pusty `build.rs` powoduje błąd
/// "main` function not found in crate `build_script_build`").
///
/// Gdy pojawi się generowanie nagłówek C/Odin z typów ABI, kod trafi tutaj.

fn main() {
    println!("cargo:rerun-if-changed=src");
}
```

driverspace_workspace/lib/kapi-abi/src/errors.rs (line 7) - [public] DsError

```
pub enum DsError {
    /// Success
    Success = 0,

    /// Unknown
    Unknown = 1,

    /// InvalidHandle
    InvalidHandle = 2,

    /// OutOfMemory
    OutOfMemory = 3,
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/kapi-abi/src/errors.rs (line 44) - [public] from_u32

```
/// u32 to DsError mapping
pub fn from_u32(val: u32) -> Self {
    match val {
        0 => Self::Success,
        1 => Self::Unknown,
        2 => Self::InvalidHandle,
        3 => Self::OutOfMemory,
        4 => Self::PermissionDenied,
        5 => Self::DeviceNotFound,
        6 => Self::DeviceBusy,
        7 => Self::InvalidMessage,
        8 => Self::BufferTooSmall,
        9 => Self::DeviceFault,
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 5) - [public] DsCmd

```
pub enum DsCmd {
    None = 0,
    Init = 1,
    Caps = 2,
    Ready = 3,
    RegisterDriver = 4,
    AttachDevice = 5,
    DetachDevice = 6,
    MapDeviceMemory = 7,
    BindIrq = 8,
    Shutdown = 9,
    Log = 10,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 30) - [public] from_u32

```
pub fn from_u32(val: u32) -> Option<Self> {
    match val {
        0 => Some(Self::None),
        1 => Some(Self::Init),
        2 => Some(Self::Caps),
        3 => Some(Self::Ready),
        4 => Some(Self::RegisterDriver),
        5 => Some(Self::AttachDevice),
        6 => Some(Self::DetachDevice),
        7 => Some(Self::MapDeviceMemory),
        8 => Some(Self::BindIrq),
        9 => Some(Self::Shutdown),
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 10) - [public] BusType

```
pub enum BusType {
    Pci = 0,
    Usb = 1,
    Acpi = 2,
    Platform = 3, // ■ UART, ■■ Framebuffer
    Unknown = 0xFF,
}

bitflags! {
    /// ■■■■/■■■■
    #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
    #[repr(C)]
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 22) - [public] DeviceFlags

```
pub struct DeviceFlags: u32 {
    /// ■■■■ DMA (Direct Memory Access)
    const DMA_CAPABLE = 1 << 0;
    /// ■■■■■■■■ (Bus Mastering)
    const BUS_MASTER = 1 << 1;
    /// ■■■■■■■■
    const HOTPLUGGABLE = 1 << 2;
    /// ■■■■■■■■■■
    const LOW_POWER = 1 << 3;
}

// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/payloads/dev.rs (line 38) - [public] DevAttachPayload

```
pub struct DevAttachPayload {  
    /// Device ID  
    pub device_id: u64,  
  
    /// Bus Type  
    pub bus_type: BusType,  
  
    /// DMA, Flags  
    pub flags: DeviceFlags,  
  
    /// ID (Vendor ID) - PCI 0x1002 (AMD)  
    pub vendor_id: u32,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/payloads/irq.rs (line 6) - [public] IrqBindPayload

```
pub struct IrqBindPayload {  
    /// IRQ / MSI vector  
    pub irq_number: u32,  
  
    /// Flags  
    pub flags: u32,  
  
    /// Padding  
    pub _pad: u32,  
}
```

driverspace_workspace/lib/kapi-abi/src/payloads/mem.rs (line 5) - [public] DmaAllocPayload

```
pub struct DmaAllocPayload {  
    pub size: u64,  
    pub flags: u32,  
    pub _pad: u32,  
}  
  
#[repr(C)]  
#[derive(Debug, Clone, Copy)]  
pub struct MmioMapPayload {  
    pub phys_base: u64,  
    pub size: u64,  
}
```

driverspace_workspace/lib/kapi-abi/src/payloads/mem.rs (line 13) - [public] MmioMapPayload

```
pub struct MmioMapPayload {  
    pub phys_base: u64,  
    pub size: u64,  
}
```

driverspace_workspace/lib/kapi-abi/src/payloads/sys.rs (line 9) - [public] LogPayload

```
pub struct LogPayload {  
    /// ds-log` LogLevel`  
    pub level: u8,  
  
    /// Module Name  
    pub module_len: u16,  
  
    /// Message  
    pub msg_len: u16,  
  
    /// Padding  
    pub _pad: u32,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 5) - [public] Handle

```
pub struct Handle(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(i32)]
pub enum Status {
    Ok = 0,
    InvalidArg = -1,
    PermissionDenied = -2,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 9) - [public] CapId

```
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(i32)]
pub enum Status {
    Ok = 0,
    InvalidArg = -1,
    PermissionDenied = -2,
    NotFound = -3,
    OutOfMemory = -4,
    Busy = -5,
    IOError = -6,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 13) - [public] Status

```
pub enum Status {
    Ok = 0,
    InvalidArg = -1,
    PermissionDenied = -2,
    NotFound = -3,
    OutOfMemory = -4,
    Busy = -5,
    IOError = -6,
}

impl From<i32> for Status {
    fn from(val: i32) -> Self {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 24) - [internal] from

```
fn from(val: i32) -> Self {
    match val {
        0 => Self::Ok,
        -1 => Self::InvalidArg,
        -2 => Self::PermissionDenied,
        -3 => Self::NotFound,
        -4 => Self::OutOfMemory,
        -5 => Self::Busy,
        -6 => Self::IoError,
        _ => Self::IoError,
    }
}
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 41) - [public] PhysAddr

```
pub struct PhysAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
pub struct VirtAddr(pub u64);
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 45) - [public] VirtAddr

```
pub struct VirtAddr(pub u64);
```

driverspace_workspace/lib/kapi-abi/src/wire.rs (line 6) - [public] DsMsg

```
pub struct DsMsg {
    pub id: u64,
    pub cmd: u32,
    pub flags: u32,
    pub arg0: u64,
    pub arg1: u64,
    pub arg2: u64,
    pub status: i32,
    pub pad: u32,
}

impl DsMsg {
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/wire.rs (line 31) - [public] is_ok

```
pub fn is_ok(&self) -> bool {
    self.status == 0
}

pub fn error_status(&self) -> crate::DsError {
    crate::DsError::from_u32(self.status as u32)
}
}
```

driverspace_workspace/lib/kapi-abi/src/wire.rs (line 35) - [public] error_status

```
pub fn error_status(&self) -> crate::DsError {
    crate::DsError::from_u32(self.status as u32)
}
}
```

driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 5) - [public] RingBuffer

```
pub struct RingBuffer {
    head: AtomicU64,
    tail: AtomicU64,
    capacity: u32,
    _pad: u32,
}

impl RingBuffer {
    pub unsafe fn from_ptr(ptr: *mut u8) -> &'static mut Self {
        unsafe { &mut *(ptr as *mut Self) }
    }

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 17) - [internal] slot_ptr

```
fn slot_ptr(&self, index: u64) -> *mut DsMsg {
    let base = self as *const _ as *const u8;
    let offset = core::mem::size_of::<Self>()
        + (index as usize % self.capacity as usize) * core::mem::size_of::<DsMsg>();
    unsafe { base.add(offset) as *mut DsMsg }
}

pub fn push(&self, msg: &DsMsg) -> bool {
    let head = self.head.load(Ordering::Acquire);
    let tail = self.tail.load(Ordering::Acquire);

    if head - tail >= self.capacity as u64 {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 24) - [public] push

```
pub fn push(&self, msg: &DsMsg) -> bool {
    let head = self.head.load(Ordering::Acquire);
    let tail = self.tail.load(Ordering::Acquire);

    if head - tail >= self.capacity as u64 {
        return false;
    }

    unsafe {
        self.slot_ptr(head).write(*msg);
    }

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/channel.rs (line 40) - [public] pop

```
pub fn pop(&self) -> Option<DsMsg> {
    let head = self.head.load(Ordering::Acquire);
    let tail = self.tail.load(Ordering::Acquire);

    if tail >= head {
        return None;
    }

    let msg = unsafe { self.slot_ptr(tail).read() };
    self.tail.store(tail + 1, Ordering::Release);
    Some(msg)
}

// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 21) - [public] sys_ipc_call

```
pub fn sys_ipc_call(cmd: DsCmd, arg0: u64, arg1: u64, arg2: u64) -> DsMsg {
    let mut msg = DsMsg::new(cmd as u32);
    msg.arg0 = arg0;
    msg.arg1 = arg1;
    msg.arg2 = arg2;

    unsafe {
        while !(*K2D_RING).push(&msg) {
            sys_yield();
        }
    }

    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 40) - [public] sys_poll

```
pub fn sys_poll() {
    unsafe {
        while (*D2K_RING).pop().is_none() {
            sys_yield();
        }
    }
}

pub fn sys_yield() {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::asm!("pause", options(nomem, nostack, preserves_flags));
    }
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 48) - [public] sys_yield

```
pub fn sys_yield() {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::asm!("pause", options(nomem, nostack, preserves_flags));
    }
}
```

driverspace_workspace/src/drivers/vgpu.rs (line 8) - [public] init

```
pub fn init() -> Result<(), Status> {
    let pci_msg = crate::ipc::Message {
        cmd: 0x20,
        arg0: VENDOR_ID as u64,
        arg1: DEVICE_ID as u64,
    };

    crate::ipc::send(pci_msg)?;

    match crate::ipc::receive() {
        Some(reply) if reply.cmd == 0 => {
            let bar0_phys = reply.arg0;
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/src/ipc.rs (line 7) - [public] Message

```
pub struct Message {
    pub cmd: u32,
    pub arg0: u64,
    pub arg1: u64,
}

static RING_BUFFER: [AtomicU64; 1024] = {
    const INIT: AtomicU64 = AtomicU64::new(0);
    [INIT; 1024]
};

static HEAD: AtomicU64 = AtomicU64::new(0);
// ... (truncated, see source file for more)
```

driverspace_workspace/src/ipc.rs (line 21) - [public] send

```
pub fn send(msg: Message) -> Result<(), Status> {
    let head = HEAD.load(Ordering::Acquire);
    let tail = TAIL.load(Ordering::Acquire);

    if (head + 1) % 1024 == tail {
        return Err(Status::Busy);
    }

    let idx = (head * 2) as usize;
    RING_BUFFER[idx].store(msg.cmd as u64, Ordering::Release);
    RING_BUFFER[idx + 1].store(msg.arg0, Ordering::Release);

    // ... (truncated, see source file for more)
}
```

driverspace_workspace/src/ipc.rs (line 38) - [public] receive

```
pub fn receive() -> Option<Message> {
    let head = HEAD.load(Ordering::Acquire);
    let tail = TAIL.load(Ordering::Acquire);

    if head == tail {
        return None;
    }

    let idx = (tail * 2) as usize;
    let cmd = RING_BUFFER[idx].load(Ordering::Acquire) as u32;
    let arg0 = RING_BUFFER[idx + 1].load(Ordering::Acquire);

    // ... (truncated, see source file for more)
}
```

driverspace_workspace/src/ipc.rs (line 55) - [public] reply

```
pub fn reply(status: Status, data: u64) {
    let msg = Message {
        cmd: status as u32,
        arg0: data,
        arg1: 0,
    };
    send(msg).ok();
}
```


driverspace_workspace/src/memory.rs (line 5) - [public] alloc

```
pub fn alloc(size: usize) -> Result<*mut u8, Status> {
    let msg = crate::ipc::Message {
        cmd: 0x10,
        arg0: size as u64,
        arg1: 0,
    };

    crate::ipc::send(msg)?;

    match crate::ipc::receive() {
        Some(reply) if reply.cmd == 0 => {
            Ok(reply.arg0 as *mut u8)
        }
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/src/memory.rs (line 23) - [public] free

```
pub fn free(ptr: *mut u8, size: usize) {
    let msg = crate::ipc::Message {
        cmd: 0x11,
        arg0: ptr as u64,
        arg1: size as u64,
    };

    crate::ipc::send(msg).ok();
}
```

driverspace_workspace/src/self_test.rs (line 6) - [public] run_all

```
pub fn run_all() -> Result<usize, Status> {
    let mut passed = 0;

    test_ipc_ring()?;
    passed += 1;

    test_memory_alloc()?;
    passed += 1;

    test_capability_check()?;
    passed += 1;

    // ... (truncated, see source file for more)
}
```

driverspace_workspace/src/self_test.rs (line 21) - [internal] test_ipc_ring

```
fn test_ipc_ring() -> Result<(), Status> {
    let msg = ipc::Message {
        cmd: 0xFF,
        arg0: 0x12345678,
        arg1: 0xABCDEF00,
    };

    ipc::send(msg)?;

    let received = ipc::receive().ok_or(Status::IoError)?;

    if received.cmd != msg.cmd || received.arg0 != msg.arg0 {
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/src/self_test.rs (line 39) - [internal] test_memory_alloc

```
fn test_memory_alloc() -> Result<(), Status> {
    let ptr = memory::alloc(4096)?;

    if ptr.is_null() {
        return Err(Status::OutOfMemory);
    }

    unsafe {
        core::ptr::write_bytes(ptr, 0xAA, 4096);

        for i in 0..4096 {
            if core::ptr::read(ptr.add(i)) != 0xAA {
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

driverspace_workspace/src/self_test.rs (line 62) - [internal] test_capability_check

```
fn test_capability_check() -> Result<(), Status> {
    let caps = crate::capabilities::current();

    if !caps.has(crate::capabilities::Capability::MMIO) {
        return Err(Status::PermissionDenied);
    }

    Ok(())
}
```

driverspacelib/src/abi.rs (line 53) - [public] DsCmd

```
pub enum DsCmd {
    None = 0,
    Init = 1,
    Caps = 2,
    Ready = 3,
    RegisterDriver = 4,
    AttachDevice = 5,
    DetachDevice = 6,
    MapDeviceMemory = 7,
    BindIrq = 8,
    Shutdown = 9,
    Log = 10,
    // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 83) - [public] DsRing

```
pub struct DsRing {
    pub head: u64,
    pub tail: u64,
    pub cap: u64,
}

pub const DS_RING_HDR_SIZE: usize = core::mem::size_of::<DsRing>();

/// A single message exchanged between kernel and driver space.
#[repr(C)]
#[derive(Clone, Copy, Default)]
pub struct DsMsg {
    // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 94) - [public] DsMsg

```
pub struct DsMsg {
    pub id: u64,
    pub cmd: u32,
    pub flags: u32,
    pub arg0: u64,
    pub arg1: u64,
    pub arg2: u64,
    pub status: i32,
    pub pad: u32,
}

pub const DS_MSG_SIZE: usize = core::mem::size_of::<DsMsg>();
// ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 110) - [public] DsInitParams

```
pub struct DsInitParams {
    pub magic: u64,
    pub version: u32,
    pub pad: u32,
    pub k2d_va: u64,
    pub d2k_va: u64,
    pub ring_cap: u64,
    pub ds_va_base: u64,
    pub ds_va_size: u64,
}

/// Library-side error type for failed driver-space calls.
// ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 123) - [public] DsError

```
pub enum DsError {
    Timeout,
    QueueFull,
    BadStatus(i32),
    NoAspace,
    NoMemory,
    NotPrepared,
}

impl From<i32> for DsError {
    fn from(v: i32) -> Self {
        DsError::BadStatus(v)
        // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 133) - [internal] from

```
fn from(v: i32) -> Self {
    DsError::BadStatus(v)
}
```

driverspacelib/src/audio.rs (line 4) - [public] AudioBars

```
pub struct AudioBars {
    pub nam_phys: u64,
    pub bm_phys: u64,
}

pub fn info_req() -> u64 {
    request(DsCmd::AudioInfo, 0, 0, 0)
}

pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 9) - [public] info_req

```
pub fn info_req() -> u64 {
    request(DsCmd::AudioInfo, 0, 0, 0)
}

pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(AudioBars { nam_phys: r.arg0, bm_phys: r.arg1 })
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 13) - [public] info_take

```
pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(AudioBars { nam_phys: r.arg0, bm_phys: r.arg1 })
}

pub fn page_phys_req(va: u64) -> u64 {
    request(DsCmd::PagePhys, va, 0, 0)
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 23) - [public] page_phys_req

```
pub fn page_phys_req(va: u64) -> u64 {
    request(DsCmd::PagePhys, va, 0, 0)
}

pub fn page_phys_take(id: u64) -> Option<u64> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(r.arg0)
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 27) - [public] page_phys_take

```
pub fn page_phys_take(id: u64) -> Option<u64> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(r.arg0)
}
```

driverspacelib/src/driver.rs (line 6) - [public] DeviceInfo

```
pub struct DeviceInfo {
    pub device_id: u64,
    pub vendor_id: u64,
    pub class_code: u64,
}

/// Any driver that lives in driver space and is registered with the kernel
/// driver-space manager.
pub trait Driver {
    fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/driver.rs (line 14) - [public] Driver

```
/// Any driver that lives in driver space and is registered with the kernel
/// driver-space manager.

pub trait Driver {
    fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/driver.rs (line 15) - [internal] init

```
fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/input.rs (line 4) - [public] key_req

```
pub fn key_req() -> u64 {
    svc::call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0)
}

pub fn key_take(id: u64) -> Option<u8> {
    let r = svc::take(id)?;

    if r.arg0 == 0 { None } else { Some(r.arg0 as u8) }
}
```

driverspacelib/src/input.rs (line 8) - [public] key_take

```
pub fn key_take(id: u64) -> Option<u8> {
    let r = svc::take(id)?;

    if r.arg0 == 0 { None } else { Some(r.arg0 as u8) }
}
```

driverspacelib/src/jack.rs (line 4) - [public] JackInfo

```
pub struct JackInfo {
    pub present: bool,
    pub amp_on: bool,
}

pub fn query_req() -> u64 {
    request(DsCmd::JackQuery, 0, 0, 0)
}

pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    // ... (truncated, see source file for more)
```

driverspacelib/src/jack.rs (line 9) - [public] query_req

```
pub fn query_req() -> u64 {
    request(DsCmd::JackQuery, 0, 0, 0)
}

pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    Some(JackInfo {
        present: r.arg0 & 1 != 0,
        amp_on: r.arg0 & 2 != 0,
    })
}

// ... (truncated, see source file for more)
```

driverspacelib/src/jack.rs (line 13) - [public] query_take

```
pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    Some(JackInfo {
        present: r.arg0 & 1 != 0,
        amp_on: r.arg0 & 2 != 0,
    })
}

pub fn set_amp(on: bool) {
    request(DsCmd::JackSetAmp, on as u64, 0, 0);
}

// ... (truncated, see source file for more)
```

driverspacelib/src/jack.rs (line 22) - [public] set_amp

```
pub fn set_amp(on: bool) {
    request(DsCmd::JackSetAmp, on as u64, 0, 0);
}

pub fn play(va: u64, len: u32) -> u64 {
    request(DsCmd::AudioPlay, va, len as u64, 0)
}

pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/jack.rs (line 26) - [public] play

```
pub fn play(va: u64, len: u32) -> u64 {
    request(DsCmd::AudioPlay, va, len as u64, 0)
}

pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/jack.rs (line 30) - [public] stop

```
pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/log.rs (line 10) - [public] ds_log

/// Wysyła tekst do logu jądrowa (komenda `DsCmd::Log`).

```
pub fn ds_log(msg: &str) {
    let _ = request(DsCmd::Log, msg.as_ptr() as u64, msg.len() as u64, 0);
}

/// To samo, ale dla surowego wskaźnika i długości (np. bufora bajtów).
pub fn ds_log_raw(ptr: *const u8, len: usize) {
    let _ = request(DsCmd::Log, ptr as u64, len as u64, 0);
}
```

driverspacelib/src/log.rs (line 15) - [public] ds_log_raw

/// To samo, ale dla surowego wskaźnika i długości (np. bufora bajtów).

```
pub fn ds_log_raw(ptr: *const u8, len: usize) {
    let _ = request(DsCmd::Log, ptr as u64, len as u64, 0);
}
```

driverspacelib/src/runtime.rs (line 38) - [public] init_once

/// Reads the boot parameters block and points our ring views at the

/// kernel-provided buffers.

```
pub fn init_once(params_va: u64) {
    unsafe {
        let p = &*(params_va as *const DsInitParams);

        K2D = p.k2d_va as *mut DsRing;
        D2K = p.d2k_va as *mut DsRing;
        K2D_MSGS = (p.k2d_va + DS_RING_HDR_SIZE as u64) as *mut DsMsg;
        D2K_MSGS = (p.d2k_va + DS_RING_HDR_SIZE as u64) as *mut DsMsg;
    }
}

/// Registers a driver (by calling its `Driver::init` hook) and notifies
// ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 51) - [public] register

/// Registers a driver (by calling its `Driver::init` hook) and notifies

/// the kernel that this driver space is now managed.

```
pub fn register<D: Driver>(drv: &mut D) {
    let _ = drv.init(&DeviceInfo::default());
    let _ = request(DsCmd::RegisterDriver, 0, 0, 0);
}

unsafe fn push_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    let ring = D2K;
    if ring.is_null() {
        return 0;
    }

    let id = NEXT_ID;
    // ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 94) - [public] request

/// Sends a driver-space command to the kernel. Returns the id used later

/// with [take_resp].

```
pub fn request(cmd: DsCmd, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd as u32, a0, a1, a2) }
}

/// Sends a raw (service-class packed) command to the kernel.
pub fn request_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd, a0, a1, a2) }
}

/// Drains pending kernel->driver responses into the local cache.
pub fn tick() {
    unsafe {
        // ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 99) - [public] request_raw

/// Sends a raw (service-class packed) command to the kernel.

```
pub fn request_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd, a0, a1, a2) }
}
```

/// Drains pending kernel->driver responses into the local cache.

```
pub fn tick() {
    unsafe {
        let ring = K2D;
        if ring.is_null() {
            return;
        }
    }
}
```

// ... (truncated, see source file for more)

driverspacelib/src/runtime.rs (line 104) - [public] tick

/// Drains pending kernel->driver responses into the local cache.

```
pub fn tick() {
    unsafe {
        let ring = K2D;
        if ring.is_null() {
            return;
        }

        while (*ring).tail != (*ring).head {
            let slot = K2D_MSGS.add((*ring).tail as usize);
            let copy = *slot;
            (*ring).tail = ((*ring).tail + 1) % (*ring).cap;
        }
    }
}
```

// ... (truncated, see source file for more)

driverspacelib/src/runtime.rs (line 128) - [public] take_resp

/// Returns (and consumes) the response with the given request id.

```
pub fn take_resp(id: u64) -> Option<DsMsg> {
    unsafe {
        for i in 0..RESP_CACHE {
            if CACHE_USED[i] && CACHE_MSG[i].id == id {
                let m = CACHE_MSG[i];
                CACHE_USED[i] = false;
                return Some(m);
            }
        }
    }
    None
}
```

// ... (truncated, see source file for more)

driverspacelib/src/svc.rs (line 4) - [public] call

```
pub fn call(class: u32, op: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    request_raw(svc_cmd(class, op), a0, a1, a2)
}
```

```
pub fn take(id: u64) -> Option<DsMsg> {
    take_resp(id)
}
```

driverspacelib/src/svc.rs (line 8) - [public] take

```
pub fn take(id: u64) -> Option<DsMsg> {
    take_resp(id)
}
```

driverspacelib/src/video.rs (line 4) - [public] FbInfo

```
pub struct FbInfo {
    pub width: u32,
    pub height: u32,
    pub stride: u32,
    pub phys: u64,
}

pub fn fb_info_req() -> u64 {
    svc::call(SVC_VIDEO0, VID_FB_INFO, 0, 0, 0)
}

pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 11) - [public] fb_info_req

```
pub fn fb_info_req() -> u64 {
    svc::call(SVC_VIDEO0, VID_FB_INFO, 0, 0, 0)
}

pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    let r = svc::take(id)?;

    Some(FbInfo {
        width: (r.arg0 >> 16) as u32,
        height: (r.arg0 & 0xFFFF) as u32,
        stride: r.arg1 as u32,
        phys: r.arg2,
    })
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 15) - [public] fb_info_take

```
pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    let r = svc::take(id)?;

    Some(FbInfo {
        width: (r.arg0 >> 16) as u32,
        height: (r.arg0 & 0xFFFF) as u32,
        stride: r.arg1 as u32,
        phys: r.arg2,
    })
}

pub fn takeover() -> u64 {
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 26) - [public] takeover

```
pub fn takeover() -> u64 {
    svc::call(SVC_VIDEO0, VID_FB_TAKEOVER, 0, 0, 0)
}

pub fn release() -> u64 {
    svc::call(SVC_VIDEO0, VID_FB_RELEASE, 0, 0, 0)
}
```

driverspacelib/src/video.rs (line 30) - [public] release

```
pub fn release() -> u64 {
    svc::call(SVC_VIDEO0, VID_FB_RELEASE, 0, 0, 0)
}
```



```

gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 6) - [public] GfxContext
pub struct GfxContext {
    server_endpoint: u64,
}

impl GfxContext {
    pub fn init() -> Self {
        Self { server_endpoint: 0x8000 }
    }

    pub fn create_surface(&self, width: u32, height: u32) -> Option<SurfaceId> {
        let config = SurfaceConfig {
            width,
            // ... (truncated, see source file for more)
        }
    }
}

gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 11) - [public] init
pub fn init() -> Self {
    Self { server_endpoint: 0x8000 }
}

pub fn create_surface(&self, width: u32, height: u32) -> Option<SurfaceId> {
    let config = SurfaceConfig {
        width,
        height,
        format: PixelFormat::XRGB8888,
        flags: gfx_protocol::surfaces::SURFACE_FLAG_VISIBLE,
    };

    // ... (truncated, see source file for more)
}

gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 15) - [public] create_surface
pub fn create_surface(&self, width: u32, height: u32) -> Option<SurfaceId> {
    let config = SurfaceConfig {
        width,
        height,
        format: PixelFormat::XRGB8888,
        flags: gfx_protocol::surfaces::SURFACE_FLAG_VISIBLE,
    };

    let mut msg = GfxIpcMsg::default();
    msg.cmd = GfxCmd::CreateSurface as u32;
    msg.arg0 = &config as *const _ as u64;

    // ... (truncated, see source file for more)
}

gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 35) - [public] commit
pub fn commit(&self, surface: SurfaceId) {
    let mut msg = GfxIpcMsg::default();
    msg.cmd = GfxCmd::Commit as u32;
    msg.arg0 = surface.0 as u64;
    self.send_ipc(msg);
}

fn send_ipc(&self, msg: GfxIpcMsg) -> GfxIpcMsg {
    // Stub: Wywołanie syscalla jdra (np. int 0x80 lub SVC)
    // przekazujące wiadomość do gfx-server
    GfxIpcMsg::default()
}

// ... (truncated, see source file for more)

gfx_protocol_workspace/crates/gfx-api/src/lib.rs (line 42) - [internal] send_ipc
fn send_ipc(&self, msg: GfxIpcMsg) -> GfxIpcMsg {
    // Stub: Wywołanie syscalla jdra (np. int 0x80 lub SVC)
    // przekazujące wiadomość do gfx-server
    GfxIpcMsg::default()
}
}

```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 4) - [internal] Surface

```
struct Surface {
    id: u32,
    x: i32,
    y: i32,
    width: u32,
    height: u32,
    z_order: i32,
    buffer: *mut u32,
}

pub struct DisplayServer {
    pub width: u32,
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 14) - [public] DisplayServer

```
pub struct DisplayServer {
    pub width: u32,
    pub height: u32,
    fb: *mut u32,
    surfaces: [Option<Surface>; 64],
    next_id: u32,
}

impl DisplayServer {
    pub fn new(width: u32, height: u32) -> Self {
        let fb_size = (width * height * 4) as usize;
        let layout = core::alloc::Layout::from_size_align(fb_size, 4096).unwrap();
        // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 23) - [public] new

```
pub fn new(width: u32, height: u32) -> Self {
    let fb_size = (width * height * 4) as usize;
    let layout = core::alloc::Layout::from_size_align(fb_size, 4096).unwrap();
    let fb = unsafe { kstd_alloc::KernelHeap.alloc(layout) } as *mut u32;

    Self { width, height, fb, surfaces: core::array::from_fn(|_| None), next_id: 1 }
}

pub fn create_surface(&mut self, width: u32, height: u32) -> u32 {
    let id = self.next_id;
    self.next_id += 1;

    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 31) - [public] create_surface

```
pub fn create_surface(&mut self, width: u32, height: u32) -> u32 {
    let id = self.next_id;
    self.next_id += 1;

    let buf_size = (width * height * 4) as usize;
    let layout = core::alloc::Layout::from_size_align(buf_size, 4096).unwrap();
    let buffer = unsafe { kstd_alloc::KernelHeap.alloc(layout) } as *mut u32;

    for slot in self.surfaces.iter_mut() {
        if slot.is_none() {
            *slot = Some(Surface { id, x: 0, y: 0, width, height, z_order: 0, buffer });
            break;
        }
    }
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 48) - [public] set_position

```
pub fn set_position(&mut self, id: u32, x: i32, y: i32) {
    for slot in self-surfaces.iter_mut() {
        if let Some(s) = slot {
            if s.id == id { s.x = x; s.y = y; break; }
        }
    }
}

pub fn commit(&mut self, _id: u32) {}

pub fn composite(&mut self) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 56) - [public] commit

```
pub fn commit(&mut self, _id: u32) {}

pub fn composite(&mut self) {
    unsafe {
        for i in 0..(self.width * self.height) {
            self.fb.add(i as usize).write_volatile(0xFF101020);
        }

        for slot in self-surfaces.iter() {
            if let Some(s) = slot {
                if s.buffer.is_null() { continue; }
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 58) - [public] composite

```
pub fn composite(&mut self) {
    unsafe {
        for i in 0..(self.width * self.height) {
            self.fb.add(i as usize).write_volatile(0xFF101020);
        }

        for slot in self-surfaces.iter() {
            if let Some(s) = slot {
                if s.buffer.is_null() { continue; }
                unsafe {
                    for row in 0..s.height {
                        // ... (truncated, see source file for more)
                    }
                }
            }
        }
    }
}
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 84) - [public]

framebuffer_ptr

```
pub fn framebuffer_ptr(&self) -> *const u32 { self.fb }
pub fn fb_size(&self) -> usize { (self.width * self.height * 4) as usize }
}
```

gfx_protocol_workspace/crates/gfx-server/src/compositor.rs (line 85) - [public] fb_size

```
pub fn fb_size(&self) -> usize { (self.width * self.height * 4) as usize }
}
```

gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 5) - [public] GpuBridge

```
pub struct GpuBridge {
    fb_phys: u64,
    fb_virt: u64,
    fb_size: usize,
    width: u32,
    height: u32,
}

impl GpuBridge {
    pub fn new() -> Self {
        Self { fb_phys: 0, fb_virt: 0, fb_size: 0, width: 0, height: 0 }
    }
    // ... (truncated, see source file for more)
}
```

gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 14) - [public] new

```
pub fn new() -> Self {
    Self { fb_phys: 0, fb_virt: 0, fb_size: 0, width: 0, height: 0 }
}

pub fn init(&mut self) -> bool {
    ds_info!("GPU-Bridge", "Requesting framebuffer info from vgpu via DS IPC");

    let reply = kapi_syscall::sys_ipc_call(DsCmd::VideoFbInfo, 0, 0, 0);
    if !reply.is_ok() {
        ds_error!("GPU-Bridge", "Failed to get FB info from vgpu");
        return false;
    }
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 18) - [public] init

```
pub fn init(&mut self) -> bool {
    ds_info!("GPU-Bridge", "Requesting framebuffer info from vgpu via DS IPC");

    let reply = kapi_syscall::sys_ipc_call(DsCmd::VideoFbInfo, 0, 0, 0);
    if !reply.is_ok() {
        ds_error!("GPU-Bridge", "Failed to get FB info from vgpu");
        return false;
    }

    self.fb_phys = reply.arg0;
    self.width = reply.arg1 as u32;
    self.height = reply.arg2 as u32;
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/crates/gfx-server/src/gpu_bridge.rs (line 47) - [public] present

```
pub fn present(&self, src_buffer: *const u32, size: usize) {
    if self.fb_virt == 0 { return; }
    unsafe {
        core::ptr::copy_nonoverlapping(
            src_buffer,
            self.fb_virt as *mut u32,
            size / 4,
        );
    }
}
```

gfx_protocol_workspace/crates/gfx-server/src/main.rs (line 49) - [internal] wait_for_ipc

```
fn wait_for_ipc() -> GfxIpcMsg {
    // Block on kernel IPC ring (k2d) for messages from user-space apps
    GfxIpcMsg::default()
}

fn reply(cmd: u32, status: i32, data: u64) {
    // Push reply to kernel IPC ring (d2k)
}
```

gfx_protocol_workspace/crates/gfx-server/src/main.rs (line 54) - [internal] reply

```
fn reply(cmd: u32, status: i32, data: u64) {
    // Push reply to kernel IPC ring (d2k)
}
```

gfx_protocol_workspace/lib/gfx-protocol/src/commands.rs (line 7) - [public] GfxCmd

```
pub enum GfxCmd {
    CreateSurface = 0x01,
    DestroySurface = 0x02,
    ResizeSurface = 0x03,

    AllocBuffer = 0x10,
    AttachBuffer = 0x11,
    Commit = 0x12,

    SetPosition = 0x20,
    SetZOrder = 0x21,

    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/lib/gfx-protocol/src/commands.rs (line 24) - [public] GfxIpcMsg

```
pub struct GfxIpcMsg {
    pub cmd: u32,
    pub status: i32,
    pub arg0: u64,
    pub arg1: u64,
    pub arg2: u64,
}
```

gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 5) - [public] SurfaceId

```
pub struct SurfaceId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
pub struct BufferId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u32)]
pub enum PixelFormat {
    XRGB8888 = 0,
    ARGB8888 = 1,
}

// ... (truncated, see source file for more)
```

gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 9) - [public] BufferId

```
pub struct BufferId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u32)]
pub enum PixelFormat {
    XRGB8888 = 0,
    ARGB8888 = 1,
}

#[derive(Debug, Clone, Copy)]
#[repr(C)]
pub struct SurfaceConfig {
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 13) - [public] PixelFormat

```
pub enum PixelFormat {
    XRGB8888 = 0,
    ARGB8888 = 1,
}

#[derive(Debug, Clone, Copy)]
#[repr(C)]
pub struct SurfaceConfig {
    pub width: u32,
    pub height: u32,
    pub format: PixelFormat,
    pub flags: u32,
    // ... (truncated, see source file for more)
```

gfx_protocol_workspace/lib/gfx-protocol/src/surfaces.rs (line 20) - [public] SurfaceConfig

```
pub struct SurfaceConfig {
    pub width: u32,
    pub height: u32,
    pub format: PixelFormat,
    pub flags: u32,
}

pub const SURFACE_FLAG_VISIBLE: u32 = 1 << 0;
pub const SURFACE_FLAG_FULLSCREEN: u32 = 1 << 1;
```

kernel-drivers/audio/src/lib.rs (line 9) - [internal] ad_init

```
fn ad_init(base: u32) -> bool;
fn ad_jack_present() -> bool;
fn ad_set_amp(on: bool);
fn ad_play_phys(phys: u64, len: u32) -> bool;
fn ad_stop();
}
```

```
static mut AMP_ON: bool = false;
static mut PRESENT: bool = false;
```

```
pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
    // ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 10) - [internal] ad_jack_present

```
fn ad_jack_present() -> bool;
fn ad_set_amp(on: bool);
fn ad_play_phys(phys: u64, len: u32) -> bool;
fn ad_stop();
}
```

```
static mut AMP_ON: bool = false;
static mut PRESENT: bool = false;
```

```
pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
}

// ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 11) - [internal] ad_set_amp

```
fn ad_set_amp(on: bool);
fn ad_play_phys(phys: u64, len: u32) -> bool;
fn ad_stop();
}
```

```
static mut AMP_ON: bool = false;
static mut PRESENT: bool = false;
```

```
pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
}

// ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 12) - [internal] ad_play_phys

```
fn ad_play_phys(phys: u64, len: u32) -> bool;
fn ad_stop();
}
```

```
static mut AMP_ON: bool = false;
static mut PRESENT: bool = false;
```

```
pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
}

pub fn poll_jack() -> bool {
    // ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 13) - [internal] ad_stop

```
fn ad_stop();
}

static mut AMP_ON: bool = false;
static mut PRESENT: bool = false;

pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
}

pub fn poll_jack() -> bool {
    let now = unsafe { ad_jack_present() };
    // ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 19) - [public] init

```
pub fn init(base: u32) -> bool {
    unsafe { ad_init(base) }
}

pub fn poll_jack() -> bool {
    let now = unsafe { ad_jack_present() };

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 23) - [public] poll_jack

```
pub fn poll_jack() -> bool {
    let now = unsafe { ad_jack_present() };

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }

    false
}
// ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 36) - [public] query

```
pub fn query() -> u32 {
    unsafe { (PRESENT as u32) | ((AMP_ON as u32) << 1) }
}

pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    unsafe { ad_set_amp(on); }
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    unsafe { ad_play_phys(phys, len) }
}
// ... (truncated, see source file for more)
```

kernel-drivers/audio/src/lib.rs (line 40) - [public] set_amp

```
pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    unsafe { ad_set_amp(on); }
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    unsafe { ad_play_phys(phys, len) }
}

pub fn stop() {
    unsafe { ad_stop(); }
}
```

kernel-drivers/audio/src/lib.rs (line 45) - [public] play_phys

```
pub fn play_phys(phys: u64, len: u32) -> bool {
    unsafe { ad_play_phys(phys, len) }
}

pub fn stop() {
    unsafe { ad_stop(); }
}
```

kernel-drivers/audio/src/lib.rs (line 49) - [public] stop

```
pub fn stop() {
    unsafe { ad_stop(); }
}
```

kernel-drivers/battery/src/lib.rs (line 6) - [internal] battery_init

```
fn battery_init() -> bool;
}

pub fn init() -> bool {
    unsafe { battery_init() }
}
```

kernel-drivers/battery/src/lib.rs (line 9) - [public] init

```
pub fn init() -> bool {
    unsafe { battery_init() }
}
```

kernel-drivers/bluetooth/src/lib.rs (line 7) - [internal] bt_init

```
fn bt_init() -> bool;
fn bt_ready() -> bool;
}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel-drivers/bluetooth/src/lib.rs (line 8) - [internal] bt_ready

```
fn bt_ready() -> bool;
}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```


kernel-drivers/bluetooth/src/lib.rs (line 11) - [public] init

```
pub fn init() -> bool {  
    unsafe { bt_init() }  
}  
  
pub fn ready() -> bool {  
    unsafe { bt_ready() }  
}
```

kernel-drivers/bluetooth/src/lib.rs (line 15) - [public] ready

```
pub fn ready() -> bool {  
    unsafe { bt_ready() }  
}
```

kernel-drivers/camera/src/lib.rs (line 6) - [internal] camera_init

```
fn camera_init() -> bool;  
  
pub fn init() -> bool {  
    unsafe { camera_init() }  
}
```

kernel-drivers/camera/src/lib.rs (line 9) - [public] init

```
pub fn init() -> bool {  
    unsafe { camera_init() }  
}
```

kernel-drivers/displayport/src/lib.rs (line 6) - [internal] dp_init

```
fn dp_init() -> bool;  
fn dp_ready() -> bool;  
  
pub fn init() -> bool {  
    unsafe { dp_init() }  
}  
  
pub fn ready() -> bool {  
    unsafe { dp_ready() }  
}
```

kernel-drivers/displayport/src/lib.rs (line 7) - [internal] dp_ready

```
fn dp_ready() -> bool;  
  
pub fn init() -> bool {  
    unsafe { dp_init() }  
}  
  
pub fn ready() -> bool {  
    unsafe { dp_ready() }  
}
```

kernel-drivers/displayport/src/lib.rs (line 10) - [public] init

```
pub fn init() -> bool {  
    unsafe { dp_init() }  
}  
  
pub fn ready() -> bool {  
    unsafe { dp_ready() }  
}
```

kernel-drivers/displayport/src/lib.rs (line 14) - [public] ready

```
pub fn ready() -> bool {  
    unsafe { dp_ready() }  
}
```

kernel-drivers/gfx/src/lib.rs (line 9) - [public] FramebufferInfo

```
pub struct FramebufferInfo {
    pub width: u32,
    pub height: u32,
    pub stride: u32,
    pub phys: u64,
}

pub fn fb_info() -> Option<FramebufferInfo> {
    let (mut w, mut h, mut s, mut p) = (0u32, 0u32, 0u32, 0u64);

    unsafe {
        if hw_sys::fb_info(&mut w, &mut h, &mut s, &mut p) {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel-drivers/gfx/src/lib.rs (line 16) - [public] fb_info

```
pub fn fb_info() -> Option<FramebufferInfo> {
    let (mut w, mut h, mut s, mut p) = (0u32, 0u32, 0u32, 0u64);

    unsafe {
        if hw_sys::fb_info(&mut w, &mut h, &mut s, &mut p) {
            Some(FramebufferInfo { width: w, height: h, stride: s, phys: p })
        } else {
            None
        }
    }
}
```

kernel-drivers/hdmi/src/lib.rs (line 6) - [internal] hdmi_init_with

```
fn hdmi_init_with(fb_phys: u64, w: u32, h: u32, stride: u32) -> bool;
fn hdmi_ready() -> bool;

pub fn init() -> bool {
    let (mut w, mut h, mut stride, mut phys) = (0u32, 0u32, 0u32, 0u64);

    unsafe {
        if !hw_sys::fb_info(&mut w, &mut h, &mut stride, &mut phys) {
            return false;
        }
    }

    // ... (truncated, see source file for more)
}
```

kernel-drivers/hdmi/src/lib.rs (line 7) - [internal] hdmi_ready

```
fn hdmi_ready() -> bool;

pub fn init() -> bool {
    let (mut w, mut h, mut stride, mut phys) = (0u32, 0u32, 0u32, 0u64);

    unsafe {
        if !hw_sys::fb_info(&mut w, &mut h, &mut stride, &mut phys) {
            return false;
        }
    }

    hdmi_init_with(phys, w, h, stride)
    // ... (truncated, see source file for more)
}
```

kernel-drivers/hdmi/src/lib.rs (line 10) - [public] init

```
pub fn init() -> bool {
    let (mut w, mut h, mut stride, mut phys) = (0u32, 0u32, 0u32, 0u64);

    unsafe {
        if !hw_sys::fb_info(&mut w, &mut h, &mut stride, &mut phys) {
            return false;
        }

        hdmi_init_with(phys, w, h, stride)
    }
}

// ... (truncated, see source file for more)
```

kernel-drivers/hdmi/src/lib.rs (line 22) - [public] ready

```
pub fn ready() -> bool {
    unsafe { hdmi_ready() }
}
```

kernel-drivers/hw-sys/src/lib.rs (line 13) - [public] kprintf

```
/// printf-style logging (mirrors the kernel `kprintf`).

pub fn kprintf(fmt: *const u8, ...);

/// Allocate a DMA-coherent buffer; returns phys + virt.
pub fn dma_alloc_coherent(
    bytes: usize,
    zone_max: u64,
    out_phys: *mut u64,
    out_virt: *mut *mut u8,
) -> bool;

pub fn dma_free_coherent(phys: u64, virt: *mut u8, bytes: usize);

// ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 16) - [public] dma_alloc_coherent

```
/// Allocate a DMA-coherent buffer; returns phys + virt.

pub fn dma_alloc_coherent(
    bytes: usize,
    zone_max: u64,
    out_phys: *mut u64,
    out_virt: *mut *mut u8,
) -> bool;

pub fn dma_free_coherent(phys: u64, virt: *mut u8, bytes: usize);

/// Map device MMIO into the higher half; returns a virtual address.
pub fn mmio_map(phys: u64, len: usize, out_virt: *mut u64) -> bool;

// ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 23) - [public] dma_free_coherent

```
pub fn dma_free_coherent(phys: u64, virt: *mut u8, bytes: usize);

/// Map device MMIO into the higher half; returns a virtual address.
pub fn mmio_map(phys: u64, len: usize, out_virt: *mut u64) -> bool;

pub fn mmio_unmap(virt: u64, len: usize);

/// Is `virt` actually present in the current page table?
pub fn paging_is_mapped(virt: u64) -> bool;

/// Query the primary framebuffer (used by `hdmi`/`gfx`).
pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32, out_phys: *mut u64) -> bool;
// ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 26) - [public] mmio_map

/// Map device MMIO into the higher half; returns a virtual address.

```
pub fn mmio_map(phys: u64, len: usize, out_virt: *mut u64) -> bool;

pub fn mmio_unmap(virt: u64, len: usize);

/// Is `virt` actually present in the current page table?
pub fn paging_is_mapped(virt: u64) -> bool;

/// Query the primary framebuffer (used by `hdmî`/`gfx`).
pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32, out_phys: *mut u64) -> bool;
}

pub mod log {
    // ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 28) - [public] mmio_unmap

```
pub fn mmio_unmap(virt: u64, len: usize);

/// Is `virt` actually present in the current page table?
pub fn paging_is_mapped(virt: u64) -> bool;

/// Query the primary framebuffer (used by `hdmî`/`gfx`).
pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32, out_phys: *mut u64) -> bool;
}

pub mod log {
    /// Print a plain (already formatted) message, `%s`-style.
    pub fn info(msg: &str) {
        // ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 31) - [public] paging_is_mapped

/// Is `virt` actually present in the current page table?

```
pub fn paging_is_mapped(virt: u64) -> bool;

/// Query the primary framebuffer (used by `hdmî`/`gfx`).
pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32, out_phys: *mut u64) -> bool;
}

pub mod log {
    /// Print a plain (already formatted) message, `%s`-style.
    pub fn info(msg: &str) {
        let mut buf = [0u8; 256];
        let len = msg.len().min(255);
        buf[..len].copy_from_slice(&msg.as_bytes()[..len]);
        // ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 34) - [public] fb_info

/// Query the primary framebuffer (used by `hdmî`/`gfx`).

```
pub fn fb_info(out_w: *mut u32, out_h: *mut u32, out_stride: *mut u32, out_phys: *mut u64) -> bool;
}

pub mod log {
    /// Print a plain (already formatted) message, `%s`-style.
    pub fn info(msg: &str) {
        let mut buf = [0u8; 256];
        let len = msg.len().min(255);
        buf[..len].copy_from_slice(&msg.as_bytes()[..len]);
        buf[len] = 0;
        unsafe { super::kprintf(b"%s\n\0".as_ptr(), buf.as_ptr()); }
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/hw-sys/src/lib.rs (line 39) - [public] info

/// Print a plain (already formatted) message, `%s`-style.

```
pub fn info(msg: &str) {
    let mut buf = [0u8; 256];
    let len = msg.len().min(255);
    buf[..len].copy_from_slice(&msg.as_bytes()[..len]);
    buf[len] = 0;
    unsafe { super::kprintf(b"%s\n\0".as_ptr(), buf.as_ptr()); }
}
```

kernel-drivers/pcie/src/lib.rs (line 20) - [public] PciDev

```
pub struct PciDev {
    pub bus: u8,
    pub dev: u8,
    pub func: u8,
}
```

/// A fully-probed device (as returned by [`enumerate`]).

#[derive(Debug, Clone, Copy)]

```
pub struct PciDevice {
    pub address: PciDev,
    pub vendor_id: u16,
    pub device_id: u16,
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 28) - [public] PciDevice

```
pub struct PciDevice {
    pub address: PciDev,
    pub vendor_id: u16,
    pub device_id: u16,
    pub class_code: u8,
    pub subclass: u8,
    pub prog_if: u8,
    pub header_type: u8,
}
```

```
// -----
// Port I/O (config mechanism #1)
// ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 54) - [internal] cfg_addr

```
fn cfg_addr(bus: u32, dev: u32, func: u32, off: u32) -> u32 {
    0x8000_0000 | (bus << 16) | (dev << 11) | (func << 8) | (off & 0xFC)
}
```

```
// -----
// Config-space access
// -----
```

```
pub fn read32(d: PciDev, off: u32) -> u32 {
    unsafe {
        outl(0xCF8, cfg_addr(d.bus as u32, d.dev as u32, d.func as u32, off));
        inl(0xCFC)
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 62) - [public] read32

```
// -----  
// Config-space access  
// -----  
  
pub fn read32(d: PciDev, off: u32) -> u32 {  
    unsafe {  
        outl(0xCF8, cfg_addr(d.bus as u32, d.dev as u32, d.func as u32, off));  
        inl(0xCFC)  
    }  
}  
  
pub fn write32(d: PciDev, off: u32, v: u32) {  
    unsafe {  
        outl(0xCF8, cfg_addr(d.bus as u32, d.dev as u32, d.func as u32, off));  
        outl(0xCFC, v);  
    }  
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 69) - [public] write32

```
pub fn write32(d: PciDev, off: u32, v: u32) {  
    unsafe {  
        outl(0xCF8, cfg_addr(d.bus as u32, d.dev as u32, d.func as u32, off));  
        outl(0xCFC, v);  
    }  
}  
  
pub fn read16(d: PciDev, off: u32) -> u16 {  
    (read32(d, off) >> ((off & 2) * 8)) as u16  
}  
  
pub fn read8(d: PciDev, off: u32) -> u8 {  
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 76) - [public] read16

```
pub fn read16(d: PciDev, off: u32) -> u16 {  
    (read32(d, off) >> ((off & 2) * 8)) as u16  
}  
  
pub fn read8(d: PciDev, off: u32) -> u8 {  
    (read32(d, off) >> ((off & 3) * 8)) as u8  
}  
  
impl PciDev {  
    pub fn vendor(self) -> u16 {  
        read16(self, 0x00)  
    }  
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 80) - [public] read8

```
pub fn read8(d: PciDev, off: u32) -> u8 {  
    (read32(d, off) >> ((off & 3) * 8)) as u8  
}  
  
impl PciDev {  
    pub fn vendor(self) -> u16 {  
        read16(self, 0x00)  
    }  
  
    pub fn device_id(self) -> u16 {  
        read16(self, 0x02)  
    }  
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 85) - [public] vendor

```
pub fn vendor(self) -> u16 {
    read16(self, 0x00)
}

pub fn device_id(self) -> u16 {
    read16(self, 0x02)
}

pub fn class(self) -> u8 {
    (read32(self, 0x08) >> 24) as u8
}

// ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 89) - [public] device_id

```
pub fn device_id(self) -> u16 {
    read16(self, 0x02)
}

pub fn class(self) -> u8 {
    (read32(self, 0x08) >> 24) as u8
}

pub fn subclass(self) -> u8 {
    (read32(self, 0x08) >> 16) as u8
}

// ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 93) - [public] class

```
pub fn class(self) -> u8 {
    (read32(self, 0x08) >> 24) as u8
}

pub fn subclass(self) -> u8 {
    (read32(self, 0x08) >> 16) as u8
}

pub fn prog_if(self) -> u8 {
    (read32(self, 0x08) >> 8) as u8
}

// ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 97) - [public] subclass

```
pub fn subclass(self) -> u8 {
    (read32(self, 0x08) >> 16) as u8
}

pub fn prog_if(self) -> u8 {
    (read32(self, 0x08) >> 8) as u8
}

/// Read BAR `idx`, handling both 32-bit and 64-bit BARs.
pub fn bar(self, idx: u32) -> u64 {
    let lo = read32(self, 0x10 + idx * 4) as u64;
    if lo & 0x04 != 0 {
// ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 101) - [public] prog_if

```
pub fn prog_if(self) -> u8 {
    (read32(self, 0x08) >> 8) as u8
}

/// Read BAR `idx`, handling both 32-bit and 64-bit BARs.
pub fn bar(self, idx: u32) -> u64 {
    let lo = read32(self, 0x10 + idx * 4) as u64;
    if lo & 0x04 != 0 {
        let hi = read32(self, 0x10 + (idx + 1) * 4) as u64;
        (hi << 32) | (lo & 0xFFFF_FFF0)
    } else {
        lo & 0xFFFF_FFF0
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 106) - [public] bar

/// Read BAR `idx`, handling both 32-bit and 64-bit BARs.

```
pub fn bar(self, idx: u32) -> u64 {
    let lo = read32(self, 0x10 + idx * 4) as u64;
    if lo & 0x04 != 0 {
        let hi = read32(self, 0x10 + (idx + 1) * 4) as u64;
        (hi << 32) | (lo & 0xFFFF_FFF0)
    } else {
        lo & 0xFFFF_FFF0
    }
}

/// Enable memory-space access and bus mastering.
pub fn enable_mmio(self) {
    // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 117) - [public] enable_mmio

/// Enable memory-space access and bus mastering.

```
pub fn enable_mmio(self) {
    let cmd = read16(self, 0x04) as u32;
    write32(self, 0x04, cmd | 0x0006);
}
}
```

/// Find the first device matching class/subclass/prog-if.

```
pub fn find_class(class: u8, subclass: u8, prog_if: u8) -> Option<PciDev> {
    for bus in 0u32..256 {
        for dev in 0u32..32 {
            let mut func = 0u32;
            let d = PciDev { bus: bus as u8, dev: dev as u8, func: 0 };
            // ... (truncated, see source file for more)
```

kernel-drivers/pcie/src/lib.rs (line 124) - [public] find_class

/// Find the first device matching class/subclass/prog-if.

```
pub fn find_class(class: u8, subclass: u8, prog_if: u8) -> Option<PciDev> {
    for bus in 0u32..256 {
        for dev in 0u32..32 {
            let mut func = 0u32;
            let d = PciDev { bus: bus as u8, dev: dev as u8, func: 0 };
            if d.vendor() == 0xFFFF {
                continue;
            }
            let multifunction = (read8(d, 0x0E) & 0x80) != 0;
            while func < 8 {
                let f = PciDev { bus: bus as u8, dev: dev as u8, func: func as u8 };
                if f.vendor() != 0xFFFF
            }
            // ... (truncated, see source file for more)
```



```
kernel-drivers/pcie/src/lib.rs (line 152) - [internal] probe
fn probe(bus: u8, dev: u8, func: u8) -> Option<PciDevice> {
    let address = PciDev { bus, dev, func };
    let vendor_id = read16(address, 0x00);
    if vendor_id == 0xFFFF {
        return None;
    }
    Some(PciDevice {
        address,
        vendor_id,
        device_id: read16(address, 0x02),
        class_code: read8(address, 0x0B),
        subclass: read8(address, 0x0A),
    })
    // ... (truncated, see source file for more)
```

```
kernel-drivers/pcie/src/lib.rs (line 170) - [public] enumerate
/// Enumerate the whole bus into a `Vec` of probed devices.

pub fn enumerate() -> Vec<PciDevice> {
    let mut devices = Vec::new();
    for bus in 0u8..=255 {
        for dev in 0u8..32 {
            if let Some(d0) = probe(bus, dev, 0) {
                let multifunction = d0.header_type & 0x80 != 0;
                devices.push(d0);
                if multifunction {
                    for func in 1u8..8 {
                        if let Some(dn) = probe(bus, dev, func) {
                            devices.push(dn);
                        }
                    }
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel-drivers/usb/src/class/hid/keyboard.rs (line 18) - [public] key_to_ascii
pub fn key_to_ascii(key: u8, shift: bool) -> Option<u8> {
    match key {
        0x04..=0x1D => {
            let t = if shift { KEY_SHIFT } else { KEY_NORMAL };
            Some(t[key as usize])
        }
        0x1E..=0x26 => {
            let t = if shift { NUM_SHIFT } else { NUM_NORMAL };
            Some(t[(key - 0x1E) as usize])
        }
        0x27 => {
            let t = if shift { NUM_SHIFT } else { NUM_NORMAL };
            Some(t[key as usize])
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel-drivers/usb/src/class/hid/keyboard.rs (line 44) - [public] push_char
pub fn push_char(c: u8) {
    unsafe {
        let next = (IN_HEAD + 1) % 64;

        if next != IN_TAIL {
            INBUF[IN_HEAD] = c;
            IN_HEAD = next;
        }
    }
}

pub fn take_char() -> Option<u8> {
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/hid/keyboard.rs (line 55) - [public] take_char

```
pub fn take_char() -> Option<u8> {
    unsafe {
        if IN_HEAD == IN_TAIL {
            return None;
        }

        let c = INBUF[IN_TAIL];
        IN_TAIL = (IN_TAIL + 1) % 64;
        Some(c)
    }
}
```

kernel-drivers/usb/src/class/hid/mod.rs (line 16) - [public] HidKeyboard

```
pub struct HidKeyboard {
    pub slot: u8,
    pub ep_idx: u32,
    ring: TransferRing,
    data: DmaBuf,
    prev: [u8; 6],
}
```

```
static mut KEYS: [Option<HidKeyboard>; 2] = [None, None];
```

```
fn submit(x: &mut Xhci, kb: &mut HidKeyboard) {
    kb.ring.enqueue(Trb::normal(kb.data.phys, 8));
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/hid/mod.rs (line 26) - [internal] submit

```
fn submit(x: &mut Xhci, kb: &mut HidKeyboard) {
    kb.ring.enqueue(Trb::normal(kb.data.phys, 8));
    x.regs.doorbell(kb.slot as u32, kb.ep_idx, 0);
}
```

```
pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool, UsbError> {
    let mut iface = 0u8;
    let mut found = false;

    for i in 0..dev.iface_count {
        let f = &dev.ifaces[i];

        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/hid/mod.rs (line 31) - [public] attach

```
pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool, UsbError> {
    let mut iface = 0u8;
    let mut found = false;

    for i in 0..dev.iface_count {
        let f = &dev.ifaces[i];

        if f.class == 3 && f.subclass == 1 && f.protocol == 1 {
            iface = f.number;
            found = true;
            break;
        }
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/hid/mod.rs (line 105) - [public] poll

```
pub fn poll(x: &mut Xhci) {
    while let Some(t) = x.ev.pending() {
        let t = t;
        x.ev.pop();
        x.regs.rt_write(super::super::host::xhci::init::RT_ERDP, x.ev.erdp() as u32);

        if t.typ() != TRB_TRANSFER_EVENT {
            continue;
        }

        let slot = t.slot_id();
        let ep = t.ep_id();
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/class/hid/report.rs (line 1) - [public] BootReport

```
pub struct BootReport {
    pub shift: bool,
    pub keys: [u8; 6],
}

pub fn parse_boot_keyboard(report: &[u8]) -> Option<BootReport> {
    if report.len() < 8 {
        return None;
    }

    let mut keys = [0u8; 6];
    keys.copy_from_slice(&report[2..8]);
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/class/hid/report.rs (line 6) - [public] parse_boot_keyboard

```
pub fn parse_boot_keyboard(report: &[u8]) -> Option<BootReport> {
    if report.len() < 8 {
        return None;
    }

    let mut keys = [0u8; 6];
    keys.copy_from_slice(&report[2..8]);

    Some(BootReport {
        shift: report[0] & 0x22 != 0,
        keys,
    })
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 6) - [public] DriverError

```
pub enum DriverError {
    Io,
}

pub trait BlockDevice {
    fn name(&self) -> &'static str;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 10) - [public] BlockDevice

```
pub trait BlockDevice {
    fn name(&self) -> &'static str;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 11) - [internal] name

```
    fn name(&self) -> &'static str;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }

    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 12) - [internal] block_size

```
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }

    fn block_size(&self) -> usize {
        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 13) - [internal] block_count

```
    fn block_count(&self) -> u64;
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }

    fn block_size(&self) -> usize {
        self.block_len as usize
    }
    // ... (truncated, see source file for more)
```

```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 14) - [internal] read_block
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }

    fn block_size(&self) -> usize {
        self.block_len as usize
    }
    // ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/blockdev.rs (line 15) - [internal] write_block
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError>;
}

impl BlockDevice for UsbMass {
    fn name(&self) -> &'static str {
        "usb0"
    }

    fn block_size(&self) -> usize {
        self.block_len as usize
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/blockdev.rs (line 19) - [internal] name
    fn name(&self) -> &'static str {
        "usb0"
    }

    fn block_size(&self) -> usize {
        self.block_len as usize
    }

    fn block_count(&self) -> u64 {
        self.sectors as u64
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/blockdev.rs (line 23) - [internal] block_size
    fn block_size(&self) -> usize {
        self.block_len as usize
    }

    fn block_count(&self) -> u64 {
        self.sectors as u64
    }

    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError> {
        let m = unsafe { super::MASS0.as_mut().ok_or(DriverError::Io)? };
        super::with_controller(|x| {
            // ... (truncated, see source file for more)

```

```

kernel-drivers/usb/src/class/mass/blockdev.rs (line 27) - [internal] block_count
    fn block_count(&self) -> u64 {
        self.sectors as u64
    }

    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError> {
        let m = unsafe { super::MASS0.as_mut().ok_or(DriverError::Io)? };

        super::with_controller(|x| {
            scsi::read10(x, m, block as u32, 1, m.data.phys)
        }).map_err(|_| DriverError::Io)?;

        unsafe {
            // ... (truncated, see source file for more)
        }
    }

kernel-drivers/usb/src/class/mass/blockdev.rs (line 31) - [internal] read_block
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), DriverError> {
        let m = unsafe { super::MASS0.as_mut().ok_or(DriverError::Io)? };

        super::with_controller(|x| {
            scsi::read10(x, m, block as u32, 1, m.data.phys)
        }).map_err(|_| DriverError::Io)?;

        unsafe {
            core::ptr::copy_nonoverlapping(m.data.virt, buf.as_mut_ptr(),
                                           m.block_len as usize);
        }

        // ... (truncated, see source file for more)
    }

kernel-drivers/usb/src/class/mass/blockdev.rs (line 46) - [internal] write_block
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), DriverError> {
        let m = unsafe { super::MASS0.as_mut().ok_or(DriverError::Io)? };

        unsafe {
            core::ptr::copy_nonoverlapping(buf.as_ptr(), m.data.virt,
                                           m.block_len as usize);
        }

        super::with_controller(|x| {
            scsi::write10(x, m, block as u32, 1, m.data.phys)
        }).map_err(|_| DriverError::Io)?;

        // ... (truncated, see source file for more)
    }

kernel-drivers/usb/src/class/mass/mod.rs (line 18) - [public] UsbMass
    pub struct UsbMass {
        pub slot: u8,
        pub out_idx: u32,
        pub in_idx: u32,
        pub out_ring: TransferRing,
        pub in_ring: TransferRing,
        pub data: DmaBuf,
        pub cmd: DmaBuf,
        pub tag: u32,
        pub sectors: u32,
        pub block_len: u32,
    }

    // ... (truncated, see source file for more)

```

```

kernel-drivers/usb/src/class/mass/mod.rs (line 32) - [public] bulk_out
pub fn bulk_out(&mut self, x: &mut Xhci, phys: u64, len: u32) -> Result<(), UsbError> {
    self.out_ring.enqueue(Trb::normal(phys, len));
    x.regs.doorbell(self.slot as u32, self.out_idx, 0);

    let cc = control::wait_transfer_ep(x, self.slot, self.out_idx as u8)?;

    if cc != CC_SUCCESS {
        return Err(UsbError::Transfer(cc));
    }

    Ok(())
}
// ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/mod.rs (line 45) - [public] bulk_in
pub fn bulk_in(&mut self, x: &mut Xhci, phys: u64, len: u32) -> Result<(), UsbError> {
    self.in_ring.enqueue(Trb::normal(phys, len));
    x.regs.doorbell(self.slot as u32, self.in_idx, 0);

    let cc = control::wait_transfer_ep(x, self.slot, self.in_idx as u8)?;

    if cc != CC_SUCCESS {
        return Err(UsbError::Transfer(cc));
    }

    Ok(())
}
// ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/mod.rs (line 58) - [public] scsi_cmd
pub fn scsi_cmd(&mut self, x: &mut Xhci, cdb: &[u8],
    data_phys: u64, data_len: u32, dir_in: bool) -> Result<(), UsbError> {
    self.tag += 1;

    let mut cbw = [0u8; 31];

    cbw[0..4].copy_from_slice(&CBW_SIG.to_le_bytes());
    cbw[4..8].copy_from_slice(&self.tag.to_le_bytes());
    cbw[8..12].copy_from_slice(&data_len.to_le_bytes());
    cbw[12] = if dir_in { 0x80 } else { 0x00 };
    cbw[13] = 0;
    cbw[14] = cdb.len() as u8;
    // ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/mod.rs (line 102) - [public] attach
pub fn attach(x: &mut Xhci, dev: &mut UsbDevice) -> Result<bool, UsbError> {
    let mut found = false;

    for i in 0..dev.iface_count {
        let f = &dev.ifaces[i];

        if f.class == 8 && f.subclass == 6 && f.protocol == 0x50 {
            found = true;
            break;
        }
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/class/mass/mod.rs (line 182) - [public] mass0
/// Borrow the attached mass-storage device (implements [blockdev::BlockDevice]).
pub fn mass0() -> Option<&'static mut UsbMass> {
    unsafe { MASS0.as_mut() }
}

pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> R {
    let x = unsafe { &mut *XHCI_PTR };
    f(x)
}

```

kernel-drivers/usb/src/class/mass/mod.rs (line 186) - [public] with_controller

```
pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> R {  
    let x = unsafe { &mut *XHCI_PTR };  
    f(x)  
}
```

kernel-drivers/usb/src/class/mass/scsi.rs (line 5) - [public] read_capacity

```
pub fn read_capacity(x: &mut Xhci, m: &mut UsbMass) -> Result<(u32, u32), UsbError> {  
    let cdb = [0x25u8, 0, 0, 0, 0, 0, 0, 0, 0, 0];  
  
    m.scsi_cmd(x, &cdb, m.data.phys, 8, true)?;  
  
    unsafe {  
        let p = m.data.virt;  
  
        let last = u32::from_be_bytes([*p.add(0), *p.add(1), *p.add(2), *p.add(3)]);  
        let blen = u32::from_be_bytes([*p.add(4), *p.add(5), *p.add(6), *p.add(7)]);  
  
        Ok((last + 1, blen))  
    }  
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/scsi.rs (line 20) - [public] read10

```
pub fn read10(x: &mut Xhci, m: &mut UsbMass,  
             lba: u32, count: u16, phys: u64) -> Result<(), UsbError> {  
    let cdb = [  
        0x28, 0,  
        (lba >> 24) as u8, (lba >> 16) as u8, (lba >> 8) as u8, lba as u8,  
        0,  
        (count >> 8) as u8, count as u8,  
        0,  
    ];  
  
    m.scsi_cmd(x, &cdb, phys, count as u32 * m.block_len, true)  
}  
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/class/mass/scsi.rs (line 33) - [public] write10

```
pub fn write10(x: &mut Xhci, m: &mut UsbMass,  
              lba: u32, count: u16, phys: u64) -> Result<(), UsbError> {  
    let cdb = [  
        0x2A, 0,  
        (lba >> 24) as u8, (lba >> 16) as u8, (lba >> 8) as u8, lba as u8,  
        0,  
        (count >> 8) as u8, count as u8,  
        0,  
    ];  
  
    m.scsi_cmd(x, &cdb, phys, count as u32 * m.block_len, false)  
}
```

kernel-drivers/usb/src/class/mod.rs (line 6) - [public] ClassDriver

```
pub trait ClassDriver {  
    fn probe(&self, dev: &UsbDevice) -> bool;  
}
```

kernel-drivers/usb/src/class/mod.rs (line 7) - [internal] probe

```
fn probe(&self, dev: &UsbDevice) -> bool;  
}
```


kernel-drivers/usb/src/core/descriptor.rs (line 2) - [public] DeviceDesc

```
pub struct DeviceDesc {
    pub max_packet0: u8,
    pub vendor: u16,
    pub product: u16,
    pub class: u8,
    pub subclass: u8,
    pub protocol: u8,
    pub num_configs: u8,
}

#[derive(Clone, Copy, Default)]
pub struct InterfaceDesc {
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/descriptor.rs (line 13) - [public] InterfaceDesc

```
pub struct InterfaceDesc {
    pub number: u8,
    pub class: u8,
    pub subclass: u8,
    pub protocol: u8,
    pub num_eps: u8,
}

#[derive(Clone, Copy, Default)]
pub struct EndpointDesc {
    pub address: u8,
    pub attributes: u8,
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/descriptor.rs (line 22) - [public] EndpointDesc

```
pub struct EndpointDesc {
    pub address: u8,
    pub attributes: u8,
    pub max_packet: u16,
    pub interval: u8,
}

pub fn parse_device(buf: &[u8]) -> Option<DeviceDesc> {
    if buf.len() < 18 || buf[1] != 1 {
        return None;
    }

    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/descriptor.rs (line 29) - [public] parse_device

```
pub fn parse_device(buf: &[u8]) -> Option<DeviceDesc> {
    if buf.len() < 18 || buf[1] != 1 {
        return None;
    }

    Some(DeviceDesc {
        max_packet0: buf[7],
        vendor: u16::from_le_bytes([buf[8], buf[9]]),
        product: u16::from_le_bytes([buf[10], buf[11]]),
        class: buf[4],
        subclass: buf[5],
        protocol: buf[6],
        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/descriptor.rs (line 45) - [public] parse_config

```
pub fn parse_config(buf: &[u8],
                    ifaces: &mut [InterfaceDesc],
                    eps: &mut [EndpointDesc])
    -> Option<(u8, u16, usize, usize)> {
    if buf.len() < 9 || buf[1] != 2 {
        return None;
    }

    let total = u16::from_le_bytes([buf[2], buf[3]]);
    let config_value = buf[5];

    let mut off = 9usize;
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/device.rs (line 7) - [public] UsbDevice

```
pub struct UsbDevice {
    pub slot: u8,
    pub port: u32,
    pub speed: u32,
    pub ep0_mps: u16,
    pub desc: DeviceDesc,
    pub config_value: u8,
    pub ifaces: [InterfaceDesc; 4],
    pub iface_count: usize,
    pub eps: [EndpointDesc; 8],
    pub ep_count: usize,
    pub ctx: Contexts,
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/device.rs (line 24) - [public] new

```
pub fn new(slot: u8, speed: u32, ctx_size: usize) -> Result<Self, UsbError> {
    Ok(Self {
        slot,
        port: 0,
        speed,
        ep0_mps: 8,
        desc: DeviceDesc::default(),
        config_value: 0,
        ifaces: [InterfaceDesc::default(); 4],
        iface_count: 0,
        eps: [EndpointDesc::default(); 8],
        ep_count: 0,
        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/enumerate.rs (line 11) - [public] enumerate

```
pub fn enumerate(x: &mut Xhci, port: u32) -> Result<UsbDevice, UsbError> {
    let slot = x.command(Trb::enable_slot())?.slot_id();
    let speed = x.regs.port_speed(port);

    let mut dev = UsbDevice::new(slot, speed, x.ctx_size)?;

    dev.port = port;

    let mps = default_ep0_mps(speed);
    dev.ep0_mps = mps;

    dev.ctx.setup_address_device(speed, port, mps, dev.ep0.phys());
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/core/speed.rs (line 6) - [public] default_ep0_mps

```
pub fn default_ep0_mps(speed: u32) -> u16 {
    match speed {
        SPEED_HIGH => 64,
        SPEED_SUPER => 512,
        _ => 8,
    }
}

pub const EP_CONTROL: u8 = 0;
pub const EP_ISO: u8 = 1;
pub const EP_BULK: u8 = 2;
pub const EP_INTERRUPT: u8 = 3;
```

kernel-drivers/usb/src/dma.rs (line 4) - [public] DmaBuf

/// A DMA-coherent buffer (physical + virtual address).

```
pub struct DmaBuf {
    pub phys: u64,
    pub virt: *mut u8,
    pub len: usize,
}

impl DmaBuf {
    pub fn new(len: usize) -> Result<Self, UsbError> {
        let mut phys = 0u64;
        let mut virt: *mut u8 = core::ptr::null_mut();

        let ok = unsafe { hw_sys::dma_alloc_coherent(len, 0xFFFF_FFFF, &mut phys, &mut virt) };
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/dma.rs (line 11) - [public] new

```
pub fn new(len: usize) -> Result<Self, UsbError> {
    let mut phys = 0u64;
    let mut virt: *mut u8 = core::ptr::null_mut();

    let ok = unsafe { hw_sys::dma_alloc_coherent(len, 0xFFFF_FFFF, &mut phys, &mut virt) };

    if !ok || virt.is_null() {
        return Err(UsbError::Invalid);
    }

    Ok(Self { phys, virt, len })
}
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/dma.rs (line 24) - [public] zero

```
pub fn zero(&mut self) {
    unsafe { core::ptr::write_bytes(self.virt, 0, self.len) }
}

impl Drop for DmaBuf {
    fn drop(&mut self) {
        unsafe { hw_sys::dma_free_coherent(self.phys, self.virt, self.len) };
    }
}
```

kernel-drivers/usb/src/dma.rs (line 30) - [internal] drop

```
fn drop(&mut self) {
    unsafe { hw_sys::dma_free_coherent(self.phys, self.virt, self.len) };
}
}
```

kernel-drivers/usb/src/host/xhci/context.rs (line 4) - [public] Contexts

```
pub struct Contexts {
    pub input: DmaBuf,
    pub output: DmaBuf,
    pub ctx_size: usize,
}

impl Contexts {
    pub fn setup_configure_ep(&mut self, speed: u32, port: u32,
                             epidx: u32, ep_type: u32, mps: u16,
                             interval: u32, ring_phys: u64) {
        self.input.zero();
        self.in_add_drop(1 | (1 << epidx), 0);
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/context.rs (line 11) - [public] setup_configure_ep

```
pub fn setup_configure_ep(&mut self, speed: u32, port: u32,
                           epidx: u32, ep_type: u32, mps: u16,
                           interval: u32, ring_phys: u64) {
    self.input.zero();
    self.in_add_drop(1 | (1 << epidx), 0);

    let s = self.slot_ptr();

    unsafe {
        s.add(0).write_volatile((speed << 20) | (2 << 27));
        s.add(1).write_volatile(port << 16);
    }
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/host/xhci/context.rs (line 37) - [public] setup_configure_bulk_pair

```
pub fn setup_configure_bulk_pair(&mut self, speed: u32, port: u32,
                                  out_idx: u32, in_idx: u32,
                                  mps_out: u16, mps_in: u16,
                                  ring_out: u64, ring_in: u64) {
    self.input.zero();
    self.in_add_drop(1 | (1 << out_idx) | (1 << in_idx), 0);

    let s = self.slot_ptr();

    unsafe {
        s.add(0).write_volatile((speed << 20) | (3 << 27));
        s.add(1).write_volatile(port << 16);
    }
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/host/xhci/context.rs (line 71) - [public] new

```
pub fn new(ctx_size: usize) -> Result<Self, UsbError> {
    let mut input = DmaBuf::new(ctx_size * 3)?;
    let output = DmaBuf::new(ctx_size * 2)?;

    input.zero();

    Ok(Self { input, output, ctx_size })
}

fn in_add_drop(&mut self, add: u32, drop: u32) {
    let p = self.input.virt as *mut u32;

    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/host/xhci/context.rs (line 80) - [internal] in_add_drop

```
fn in_add_drop(&mut self, add: u32, drop: u32) {
    let p = self.input.virt as *mut u32;

    unsafe {
        p.add(0).write_volatile(drop);
        p.add(1).write_volatile(add);
    }
}

fn slot_ptr(&mut self) -> *mut u32 {
    unsafe { self.input.virt.add(self.ctx_size) as *mut u32 }
}
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/context.rs (line 89) - [internal] slot_ptr

```
fn slot_ptr(&mut self) -> *mut u32 {
    unsafe { self.input.virt.add(self.ctx_size) as *mut u32 }
}

fn ep0_ptr(&mut self) -> *mut u32 {
    unsafe { self.input.virt.add(self.ctx_size * 2) as *mut u32 }
}

pub fn setup_address_device(&mut self, speed: u32, port: u32,
                           mps: u16, ring_phys: u64) {
    self.input.zero();
    self.in_add_drop(3, 0);
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/context.rs (line 93) - [internal] ep0_ptr

```
fn ep0_ptr(&mut self) -> *mut u32 {
    unsafe { self.input.virt.add(self.ctx_size * 2) as *mut u32 }
}

pub fn setup_address_device(&mut self, speed: u32, port: u32,
                           mps: u16, ring_phys: u64) {
    self.input.zero();
    self.in_add_drop(3, 0);

    let s = self.slot_ptr();

    unsafe {
        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/context.rs (line 97) - [public] setup_address_device

```
pub fn setup_address_device(&mut self, speed: u32, port: u32,
                           mps: u16, ring_phys: u64) {
    self.input.zero();
    self.in_add_drop(3, 0);

    let s = self.slot_ptr();

    unsafe {
        s.add(0).write_volatile((speed << 20) | (1 << 27));
        s.add(1).write_volatile(port << 16);
    }

    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/context.rs (line 120) - [public] setup_evaluate_mps

```
pub fn setup_evaluate_mps(&mut self, mps: u16) {
    self.input.zero();
    self.in_add_drop(2, 0);

    let e = self.ep0_ptr();

    unsafe {
        e.add(1).write_volatile((mps as u32) << 16);
    }
}
```

kernel-drivers/usb/src/host/xhci/control.rs (line 6) - [public] wait_transfer

```
pub fn wait_transfer(x: &mut Xhci, slot: u8) -> Result<u8, UsbError> {
    for _ in 0..2_000_000 {
        if let Some(t) = x.ev.pending() {
            let t = t;
            x.ev.pop();
            super::init::rt_write64(&x.regs, super::init::RT_ERDP, x.ev.erdp());

            if t.typ() == TRB_TRANSFER_EVENT && t.slot_id() == slot {
                return Ok(t.completion_code());
            }

            continue;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/control.rs (line 26) - [public] wait_transfer_ep

```
pub fn wait_transfer_ep(x: &mut Xhci, slot: u8, ep: u8) -> Result<u8, UsbError> {
    for _ in 0..2_000_000 {
        if let Some(t) = x.ev.pending() {
            let t = t;
            x.ev.pop();
            super::init::rt_write64(&x.regs, super::init::RT_ERDP, x.ev.erdp());

            if t.typ() == TRB_TRANSFER_EVENT && t.slot_id() == slot && t.ep_id() == ep {
                return Ok(t.completion_code());
            }

            continue;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/control.rs (line 46) - [public] control

```
pub fn control(x: &mut Xhci,
    dev: &mut UsbDevice,
    setup: u64,
    trt: u32,
    data: Option<(&mut [u8], usize)>,
    out_len: usize) -> Result<usize, UsbError> {
    dev.ep0.enqueue(Trb::setup_stage(setup, trt));

    let (dir_in, len) = match &data {
        Some( (_, l)) => (trt == 3, *l),
        None => (false, 0),
    };
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/host/xhci/control.rs (line 94) - [public] control_in

```
pub fn control_in(x: &mut Xhci, dev: &mut UsbDevice,
    req: u8, value: u16, index: u16,
    buf: &mut [u8]) -> Result<usize, UsbError> {
    let len = buf.len();
    let setup = pack_setup(0x80, req, value, index, len as u16);
    control(x, dev, setup, 3, Some((buf, len)), 0)
}

pub fn control_out(x: &mut Xhci, dev: &mut UsbDevice,
    req: u8, value: u16, index: u16,
    data: &[u8]) -> Result<usize, UsbError> {
    let setup = pack_setup(0x00, req, value, index, data.len() as u16);
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/control.rs (line 102) - [public] control_out

```
pub fn control_out(x: &mut Xhci, dev: &mut UsbDevice,
    req: u8, value: u16, index: u16,
    data: &[u8]) -> Result<usize, UsbError> {
    let setup = pack_setup(0x00, req, value, index, data.len() as u16);

    if data.is_empty() {
        control(x, dev, setup, 0, None, 0)
    } else {
        let mut tmp = [0u8; 512];
        tmp[..data.len()].copy_from_slice(data);
        control(x, dev, setup, 2, Some((&mut tmp, data.len()))), 0)
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/event.rs (line 6) - [public] drain_events

```
pub fn drain_events(&mut self) {
    while let Some(t) = self.ev.pending() {
        let t = t;
        self.ev.pop();
        super::init::rt_write64(&self.regs, super::init::RT_ERDP, self.ev.erdp());

        match t.typ() {
            TRB_PORT_STATUS => unsafe {
                kprintf(b"usb: port status change, port=%d\n\0".as_ptr(),
                    (t.param >> 24) as u32);
            },
            TRB_HC_EVENT => unsafe {
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

kernel-drivers/usb/src/host/xhci/init.rs (line 14) - [public] Xhci

```
pub struct Xhci {
    pub regs: XhciRegs,
    pub cmd: CmdRing,
    pub ev: EventRing,
    pub dcbaa: DmaBuf,
    pub ctx_size: usize,
    pub slots: u32,
    pub ports: u32,
}

fn spin_wait<F: Fn() -> bool>(f: F) -> Result<(), UsbError> {
    for _ in 0..2_000_000 {
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/init.rs (line 24) - [internal] spin_wait

```
fn spin_wait<F: Fn() -> bool>(f: F) -> Result<(), UsbError> {
    for _ in 0..2_000_000 {
        if f() {
            return Ok(());
        }

        core::hint::spin_loop();
    }

    Err(UsbError::Timeout)
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/init.rs (line 36) - [internal] op_write64

```
fn op_write64(regs: &XhciRegs, off: usize, v: u64) {
    regs.op_write(off, (v & 0xFFFF_FFFF) as u32);
    regs.op_write(off + 4, (v >> 32) as u32);
}

pub(super) fn rt_write64(regs: &XhciRegs, off: usize, v: u64) {
    regs.rt_write(off, (v & 0xFFFF_FFFF) as u32);
    regs.rt_write(off + 4, (v >> 32) as u32);
}

pub fn init(regs: XhciRegs) -> Result<Xhci, UsbError> {
    let cmd0 = regs.op_read(OP_USBCMD);
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/init.rs (line 41) - [public] rt_write64

```
pub(super) fn rt_write64(regs: &XhciRegs, off: usize, v: u64) {
    regs.rt_write(off, (v & 0xFFFF_FFFF) as u32);
    regs.rt_write(off + 4, (v >> 32) as u32);
}

pub fn init(regs: XhciRegs) -> Result<Xhci, UsbError> {
    let cmd0 = regs.op_read(OP_USBCMD);

    if cmd0 & CMD_RS != 0 {
        regs.op_write(OP_USBCMD, cmd0 & !CMD_RS);
        spin_wait(|| regs.op_read(OP_USBSTS) & STS_HCH != 0)?;
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/init.rs (line 46) - [public] init

```
pub fn init(regs: XhciRegs) -> Result<Xhci, UsbError> {
    let cmd0 = regs.op_read(OP_USBCMD);

    if cmd0 & CMD_RS != 0 {
        regs.op_write(OP_USBCMD, cmd0 & !CMD_RS);
        spin_wait(|| regs.op_read(OP_USBSTS) & STS_HCH != 0)?;
    }

    regs.op_write(OP_USBCMD, CMD_HCRST);

    spin_wait(|| regs.op_read(OP_USBCMD) & CMD_HCRST == 0)?;
    spin_wait(|| regs.op_read(OP_USBSTS) & STS_CNR == 0)?;
    // ... (truncated, see source file for more)
```


kernel-drivers/usb/src/host/xhci/init.rs (line 100) - [public] command

```
pub fn command(&mut self, trb: Trb) -> Result<Trb, UsbError> {
    self.cmd.enqueue(trb);
    self.regs.doorbell(0, 0, 0);

    for _ in 0..2_000_000 {
        if let Some(t) = self.ev.pending() {
            let t = t;
            self.ev.pop();
            rt_write64(&self.regs, RT_ERDP, self.ev.erdp());

            if t.typ() == TRB_CMD_COMPLETION {
                if t.completion_code() == CC_SUCCESS {
                    // ... (truncated, see source file for more)
                }
            }
        }
    }
}
```

kernel-drivers/usb/src/host/xhci/init.rs (line 127) - [public] scan_ports

```
pub fn scan_ports(&mut self) {
    for p in 1..=self.ports {
        let sc = self.regs.port_sc(p);

        if sc & PORTSC_CCS == 0 {
            continue;
        }

        let speed = (sc & PORTSC_SPEED) >> 10;

        unsafe {
            kprintf(b"usb: port %d connected, speed=%d\n\0".as_ptr(), p, speed);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/init.rs (line 170) - [internal] attach_port

```
fn attach_port(&mut self, p: u32) {
    let mut dev = match crate::core::enumerate::enumerate(self, p) {
        Ok(d) => d,
        Err(_) => {
            unsafe {
                kprintf(b"usb: port %d enumeration failed\n\0".as_ptr(), p);
            }
            return;
        }
    };
    use crate::class::{hid, mass};
    // ... (truncated, see source file for more)
}
```

kernel-drivers/usb/src/host/xhci/regs.rs (line 35) - [public] XhciRegs

```
pub struct XhciRegs {
    base: *mut u8,
    pub cap_len: usize,
    pub db_off: usize,
    pub rt_off: usize,
    pub max_slots: u32,
    pub max_intrs: u32,
    pub max_ports: u32,
    pub addr64: bool,
    pub csz64: bool,
}

// ... (truncated, see source file for more)
```

```

kernel-drivers/usb/src/host/xhci/regs.rs (line 48) - [public] new
pub fn new(phys: u64) -> Result<Self, UsbError> {
    let ptr = if phys >= 0xFFFF800000000000 {
        phys
    } else {
        let mut virt = 0u64;

        if !unsafe { hw_sys::mmio_map(phys, MAP_SIZE, &mut virt) } {
            return Err(UsbError::MapFailed);
        }

        // The runtime can report success yet leave the MMIO unmapped;
        // reading it would page-fault, so verify.
        // ... (truncated, see source file for more)
}

kernel-drivers/usb/src/host/xhci/regs.rs (line 90) - [internal] op
fn op(&self) -> *mut u8 {
    unsafe { self.base.add(self.cap_len) }
}

fn rt(&self) -> *mut u8 {
    unsafe { self.base.add(self.rt_off) }
}

fn db(&self) -> *mut u8 {
    unsafe { self.base.add(self.db_off) }
}

// ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/regs.rs (line 94) - [internal] rt
fn rt(&self) -> *mut u8 {
    unsafe { self.base.add(self.rt_off) }
}

fn db(&self) -> *mut u8 {
    unsafe { self.base.add(self.db_off) }
}

pub fn op_read(&self, off: usize) -> u32 {
    unsafe { r32(self.op().add(off)) }
}

// ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/regs.rs (line 98) - [internal] db
fn db(&self) -> *mut u8 {
    unsafe { self.base.add(self.db_off) }
}

pub fn op_read(&self, off: usize) -> u32 {
    unsafe { r32(self.op().add(off)) }
}

pub fn op_write(&self, off: usize, v: u32) {
    unsafe { w32(self.op().add(off), v) }
}

// ... (truncated, see source file for more)

```

```

kernel-drivers/usb/src/host/xhci/regs.rs (line 102) - [public] op_read
    pub fn op_read(&self, off: usize) -> u32 {
        unsafe { r32(self.op().add(off)) }
    }

    pub fn op_write(&self, off: usize, v: u32) {
        unsafe { w32(self.op().add(off), v) }
    }

    pub fn rt_read(&self, off: usize) -> u32 {
        unsafe { r32(self.rt().add(off)) }
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/regs.rs (line 106) - [public] op_write
    pub fn op_write(&self, off: usize, v: u32) {
        unsafe { w32(self.op().add(off), v) }
    }

    pub fn rt_read(&self, off: usize) -> u32 {
        unsafe { r32(self.rt().add(off)) }
    }

    pub fn rt_write(&self, off: usize, v: u32) {
        unsafe { w32(self.rt().add(off), v) }
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/regs.rs (line 110) - [public] rt_read
    pub fn rt_read(&self, off: usize) -> u32 {
        unsafe { r32(self.rt().add(off)) }
    }

    pub fn rt_write(&self, off: usize, v: u32) {
        unsafe { w32(self.rt().add(off), v) }
    }

    pub fn doorbell(&self, slot: u32, target: u32, task: u32) {
        unsafe {
            w32(self.db().add(slot as usize * 4),
                (target & 0xFF) | ((task & 0xFFFF) << 16));
        }
        // ... (truncated, see source file for more)
    }

kernel-drivers/usb/src/host/xhci/regs.rs (line 114) - [public] rt_write
    pub fn rt_write(&self, off: usize, v: u32) {
        unsafe { w32(self.rt().add(off), v) }
    }

    pub fn doorbell(&self, slot: u32, target: u32, task: u32) {
        unsafe {
            w32(self.db().add(slot as usize * 4),
                (target & 0xFF) | ((task & 0xFFFF) << 16));
        }
    }

    pub fn port_sc(&self, port: u32) -> u32 {
        // ... (truncated, see source file for more)
    }

```

kernel-drivers/usb/src/host/xhci/regs.rs (line 118) - [public] doorbell

```
pub fn doorbell(&self, slot: u32, target: u32, task: u32) {
    unsafe {
        w32(self.db().add(slot as usize * 4),
            (target & 0xFF) | ((task & 0xFFFF) << 16));
    }
}

pub fn port_sc(&self, port: u32) -> u32 {
    self.op_read(OP_PORTSC + (port as usize - 1) * 0x10)
}

pub fn port_sc_write(&self, port: u32, v: u32) {
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/regs.rs (line 125) - [public] port_sc

```
pub fn port_sc(&self, port: u32) -> u32 {
    self.op_read(OP_PORTSC + (port as usize - 1) * 0x10)
}

pub fn port_sc_write(&self, port: u32, v: u32) {
    self.op_write(OP_PORTSC + (port as usize - 1) * 0x10, v);
}

pub fn port_speed(&self, port: u32) -> u32 {
    (self.port_sc(port) & PORTSC_SPEED) >> 10
}
}
```

kernel-drivers/usb/src/host/xhci/regs.rs (line 129) - [public] port_sc_write

```
pub fn port_sc_write(&self, port: u32, v: u32) {
    self.op_write(OP_PORTSC + (port as usize - 1) * 0x10, v);
}

pub fn port_speed(&self, port: u32) -> u32 {
    (self.port_sc(port) & PORTSC_SPEED) >> 10
}
}
```

kernel-drivers/usb/src/host/xhci/regs.rs (line 133) - [public] port_speed

```
pub fn port_speed(&self, port: u32) -> u32 {
    (self.port_sc(port) & PORTSC_SPEED) >> 10
}
}
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 5) - [public] CmdRing

```
pub struct CmdRing {
    buf: DmaBuf,
    len: usize,
    enqueue: usize,
    cycle: bool,
}

impl CmdRing {
    pub fn new(count: usize) -> Result<Self, UsbError> {
        let len = count + 1;
        let buf = DmaBuf::new(len * 16)?;

        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 13) - [public] new

```
pub fn new(count: usize) -> Result<Self, UsbError> {
    let len = count + 1;
    let buf = DmaBuf::new(len * 16)?;

    unsafe {
        (buf.virt as *mut Trb).add(count).write_volatile(Trb::link(buf.phys));
    }

    Ok(Self { buf, len, enqueue: 0, cycle: true })
}

pub fn phys(&self) -> u64 {
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 24) - [public] phys

```
pub fn phys(&self) -> u64 {
    self.buf.phys
}

pub fn enqueue(&mut self, mut trb: Trb) {
    if self.enqueue == self.len - 1 {
        self.enqueue = 0;
        self.cycle = !self.cycle;
    }

    if self.cycle {
        trb.control |= 1;
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 28) - [public] enqueue

```
pub fn enqueue(&mut self, mut trb: Trb) {
    if self.enqueue == self.len - 1 {
        self.enqueue = 0;
        self.cycle = !self.cycle;
    }

    if self.cycle {
        trb.control |= 1;
    } else {
        trb.control &= !1;
    }

    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 48) - [public] EventRing

```
pub struct EventRing {
    buf: DmaBuf,
    erst: DmaBuf,
    len: usize,
    dequeue: usize,
    cycle: bool,
}

pub struct TransferRing {
    buf: DmaBuf,
    len: usize,
    enqueue: usize,
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 56) - [public] TransferRing

```
pub struct TransferRing {
    buf: DmaBuf,
    len: usize,
    enqueue: usize,
    cycle: bool,
}

impl TransferRing {
    pub fn new(count: usize) -> Result<Self, UsbError> {
        let len = count + 1;
        let buf = DmaBuf::new(len * 16)?;

        // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 64) - [public] new

```
pub fn new(count: usize) -> Result<Self, UsbError> {
    let len = count + 1;
    let buf = DmaBuf::new(len * 16)?;

    unsafe {
        (buf.virt as *mut Trb).add(count).write_volatile(Trb::link(buf.phys));
    }

    Ok(Self { buf, len, enqueue: 0, cycle: true })
}

pub fn phys(&self) -> u64 {
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 75) - [public] phys

```
pub fn phys(&self) -> u64 {
    self.buf.phys
}

pub fn enqueue(&mut self, mut trb: Trb) {
    if self.enqueue == self.len - 1 {
        self.enqueue = 0;
        self.cycle = !self.cycle;
    }

    if self.cycle {
        trb.control |= 1;
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 79) - [public] enqueue

```
pub fn enqueue(&mut self, mut trb: Trb) {
    if self.enqueue == self.len - 1 {
        self.enqueue = 0;
        self.cycle = !self.cycle;
    }

    if self.cycle {
        trb.control |= 1;
    } else {
        trb.control &= !1;
    }

    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 100) - [public] new

```
pub fn new(count: usize) -> Result<Self, UsbError> {
    let buf = DmaBuf::new(count * 16)?;
    let erst = DmaBuf::new(16)?;

    unsafe {
        let e = erst.virt as *mut u64;
        e.add(0).write_volatile(buf.phys);
        e.add(1).write_volatile(count as u64);
    }

    Ok(Self { buf, erst, len: count, dequeue: 0, cycle: true })
}
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 113) - [public] erst_phys

```
pub fn erst_phys(&self) -> u64 {
    self.erst.phys
}

pub fn pending(&self) -> Option<Trb> {
    let t = unsafe {
        (self.buf.virt as *const Trb).add(self.dequeue).read_volatile()
    };

    if t.cycle() == self.cycle {
        Some(t)
    } else {
        None
    }
}
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 117) - [public] pending

```
pub fn pending(&self) -> Option<Trb> {
    let t = unsafe {
        (self.buf.virt as *const Trb).add(self.dequeue).read_volatile()
    };

    if t.cycle() == self.cycle {
        Some(t)
    } else {
        None
    }
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 129) - [public] pop

```
pub fn pop(&mut self) {
    self.dequeue += 1;

    if self.dequeue == self.len {
        self.dequeue = 0;
        self.cycle = !self.cycle;
    }
}

pub fn erdp(&self) -> u64 {
    (self.buf.phys + (self.dequeue as u64) * 16) & 0xFFFF_FFFF_FFFF_FF00
}
// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/ring.rs (line 138) - [public] erdp

```
pub fn erdp(&self) -> u64 {
    (self.buf.phys + (self.dequeue as u64) * 16) & 0xFFFF_FFFF_FFFF_FF00
}
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 29) - [public] Trb

```
pub struct Trb {
    pub param: u64,
    pub status: u32,
    pub control: u32,
}

impl Trb {
    pub fn typ(&self) -> u32 {
        (self.control >> 10) & 0x3F
    }

    pub fn cycle(&self) -> bool {
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 36) - [public] typ

```
pub fn typ(&self) -> u32 {
    (self.control >> 10) & 0x3F
}

pub fn cycle(&self) -> bool {
    self.control & 1 != 0
}

pub fn completion_code(&self) -> u8 {
    (self.status >> 24) as u8
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 40) - [public] cycle

```
pub fn cycle(&self) -> bool {
    self.control & 1 != 0
}

pub fn completion_code(&self) -> u8 {
    (self.status >> 24) as u8
}

pub fn slot_id(&self) -> u8 {
    (self.control >> 24) as u8
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 44) - [public] completion_code

```
pub fn completion_code(&self) -> u8 {
    (self.status >> 24) as u8
}

pub fn slot_id(&self) -> u8 {
    (self.control >> 24) as u8
}

pub fn ep_id(&self) -> u8 {
    ((self.control >> 16) & 0x1F) as u8
}

// ... (truncated, see source file for more)
```


kernel-drivers/usb/src/host/xhci/trb.rs (line 48) - [public] slot_id

```
pub fn slot_id(&self) -> u8 {
    (self.control >> 24) as u8
}

pub fn ep_id(&self) -> u8 {
    ((self.control >> 16) & 0x1F) as u8
}

pub fn transfer_len(&self) -> u32 {
    self.status & 0x00FF_FFFF
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 52) - [public] ep_id

```
pub fn ep_id(&self) -> u8 {
    ((self.control >> 16) & 0x1F) as u8
}

pub fn transfer_len(&self) -> u32 {
    self.status & 0x00FF_FFFF
}

pub fn link(addr: u64) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: 0,
    }
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 56) - [public] transfer_len

```
pub fn transfer_len(&self) -> u32 {
    self.status & 0x00FF_FFFF
}

pub fn link(addr: u64) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: 0,
        control: (TRB_LINK << 10) | (1 << 1),
    }
}

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 60) - [public] link

```
pub fn link(addr: u64) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: 0,
        control: (TRB_LINK << 10) | (1 << 1),
    }
}

pub fn enable_slot() -> Self {
    Trb { control: TRB_ENABLE_SLOT << 10, ..Default::default() }
}

// ... (truncated, see source file for more)
```

```

kernel-drivers/usb/src/host/xhci/trb.rs (line 68) - [public] enable_slot
    pub fn enable_slot() -> Self {
        Trb { control: TRB_ENABLE_SLOT << 10, ..Default::default() }
    }

    pub fn address_device(slot: u8, ctx_phys: u64) -> Self {
        Trb {
            param: ctx_phys & ADDR_MASK,
            status: 0,
            control: (TRB_ADDRESS_DEVICE << 10) | ((slot as u32) << 24),
        }
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/trb.rs (line 72) - [public] address_device
    pub fn address_device(slot: u8, ctx_phys: u64) -> Self {
        Trb {
            param: ctx_phys & ADDR_MASK,
            status: 0,
            control: (TRB_ADDRESS_DEVICE << 10) | ((slot as u32) << 24),
        }
    }

    pub fn configure_ep(slot: u8, ctx_phys: u64) -> Self {
        Trb {
            param: ctx_phys & ADDR_MASK,
            status: 0,
            // ... (truncated, see source file for more)
        }
    }

kernel-drivers/usb/src/host/xhci/trb.rs (line 80) - [public] configure_ep
    pub fn configure_ep(slot: u8, ctx_phys: u64) -> Self {
        Trb {
            param: ctx_phys & ADDR_MASK,
            status: 0,
            control: (TRB_CONFIGURE_EP << 10) | ((slot as u32) << 24),
        }
    }

    pub fn noop_cmd() -> Self {
        Trb { control: TRB_NOOP_CMD << 10, ..Default::default() }
    }

    // ... (truncated, see source file for more)

kernel-drivers/usb/src/host/xhci/trb.rs (line 88) - [public] noop_cmd
    pub fn noop_cmd() -> Self {
        Trb { control: TRB_NOOP_CMD << 10, ..Default::default() }
    }

    pub fn setup_stage(raw_setup: u64, trt: u32) -> Self {
        Trb {
            param: raw_setup,
            status: 0,
            control: (TRB_SETUP << 10) | (1 << 6) | ((trt & 3) << 16),
        }
    }

    // ... (truncated, see source file for more)

```

kernel-drivers/usb/src/host/xhci/trb.rs (line 92) - [public] setup_stage

```
pub fn setup_stage(raw_setup: u64, trt: u32) -> Self {
    Trb {
        param: raw_setup,
        status: 0,
        control: (TRB_SETUP << 10) | (1 << 6) | ((trt & 3) << 16),
    }
}

pub fn data_stage(addr: u64, len: u32, dir_in: bool) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: len & 0xFFFF,
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 100) - [public] data_stage

```
pub fn data_stage(addr: u64, len: u32, dir_in: bool) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: len & 0xFFFF,
        control: (TRB_DATA << 10) | ((dir_in as u32) << 16),
    }
}

pub fn status_stage(dir_in: bool) -> Self {
    Trb {
        param: 0,
        status: 0,
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 108) - [public] status_stage

```
pub fn status_stage(dir_in: bool) -> Self {
    Trb {
        param: 0,
        status: 0,
        control: (TRB_STATUS << 10) | ((dir_in as u32) << 16) | (1 << 5),
    }
}

pub fn normal(addr: u64, len: u32) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: len & 0xFFFF,
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 116) - [public] normal

```
pub fn normal(addr: u64, len: u32) -> Self {
    Trb {
        param: addr & ADDR_MASK,
        status: len & 0xFFFF,
        control: (TRB_NORMAL << 10) | (1 << 5),
    }
}

pub fn evaluate_ctx(slot: u8, ctx_phys: u64) -> Self {
    Trb {
        param: ctx_phys & ADDR_MASK,
        status: 0,
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 124) - [public] evaluate_ctx

```
pub fn evaluate_ctx(slot: u8, ctx_phys: u64) -> Self {
    Trb {
        param: ctx_phys & ADDR_MASK,
        status: 0,
        control: (TRB_EVALUATE_CTX << 10) | ((slot as u32) << 24),
    }
}

pub fn pack_setup(bm_request: u8, b_request: u8, value: u16,
    index: u16, length: u16) -> u64 {
    (bm_request as u64)
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/host/xhci/trb.rs (line 133) - [public] pack_setup

```
pub fn pack_setup(bm_request: u8, b_request: u8, value: u16,
    index: u16, length: u16) -> u64 {
    (bm_request as u64)
    | ((b_request as u64) << 8)
    | ((value as u64) << 16)
    | ((index as u64) << 32)
    | ((length as u64) << 48)
}
```

kernel-drivers/usb/src/lib.rs (line 23) - [public] UsbError

```
pub enum UsbError {
    NoController,
    MapFailed,
    Timeout,
    NotReady,
    BadDescriptor,
    Invalid,
    Transfer(u8),
}

static mut CONTROLLER: Option<Xhci> = None;

// ... (truncated, see source file for more)
```

kernel-drivers/usb/src/lib.rs (line 36) - [public] init

/// Probe the PCI bus for an xHCI controller and bring it online.

```
pub fn init() -> Result<(), UsbError> {
    if unsafe { CONTROLLER.is_some() } {
        return Ok(());
    }

    let xhci_pci = pci_glue::find_xhci()?;
    hw_sys::log::info("usb: xHCI controller found");

    let regs = host::xhci::regs::XhciRegs::new(xhci_pci.bar0_phys?);

    let mut xhci = host::xhci::init::init(regs)?;
    xhci.scan_ports();
    // ... (truncated, see source file for more)
```

kernel-drivers/usb/src/lib.rs (line 57) - [public] poll

/// Poll attached HID devices and drain controller events.

```
pub fn poll() {
    if let Some(x) = unsafe { CONTROLLER.as_mut() } {
        class::hid::poll(x);
    }
}

/// Borrow the active controller (when it is online).
pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> Option<R> {
    unsafe { CONTROLLER.as_mut().map(f) }
}
```

kernel-drivers/usb/src/lib.rs (line 64) - [public] with_controller

/// Borrow the active controller (when it is online).

```
pub fn with_controller<F: FnOnce(&mut Xhci) -> R, R>(f: F) -> Option<R> {
    unsafe { CONTROLLER.as_mut().map(f) }
}
```

kernel-drivers/usb/src/pci_glue.rs (line 8) - [public] XhciPci

```
pub struct XhciPci {
    pub dev: PciDev,
    pub bar0_phys: u64,
}

pub fn find_xhci() -> Result<XhciPci, UsbError> {
    let dev = pcie::find_class(XHCI_CLASS, XHCI_SUBCLASS, XHCI_PROGIF)
        .ok_or(UsbError::NoController)?;

    let bar0 = dev.bar(0);

    if bar0 == 0 {
        // ... (truncated, see source file for more)
    }
}
```

kernel-drivers/usb/src/pci_glue.rs (line 13) - [public] find_xhci

```
pub fn find_xhci() -> Result<XhciPci, UsbError> {
    let dev = pcie::find_class(XHCI_CLASS, XHCI_SUBCLASS, XHCI_PROGIF)
        .ok_or(UsbError::NoController)?;

    let bar0 = dev.bar(0);

    if bar0 == 0 {
        return Err(UsbError::NoController);
    }

    dev.enable_mmio();

    // ... (truncated, see source file for more)
}
```

kernel_Workspace/C_base/kc-abi-menager/src/main.rs (line 2) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

kernel_Workspace/C_base/kc-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/C_base/kc-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}

}
```

kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/Odin_base/odin-bin-loader/src/main.rs (line 2) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

kernel_Workspace/base/kw-C-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-C-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_workspace/base/kw-base/src/api/dispatch.rs (line 21) - [public] dispatch

// Main syscall dispatcher. Called from the trampoline/assembly entry point.

// Returns i64: positive for success/data, negative for KernelError codes.

```
pub fn dispatch(num: u64, a0: u64, a1: u64, a2: u64, a3: u64) -> i64 {
    let res: Result<i64, KernelError> = match num {
        SYS_READ => fs_ops::read(a0 as u32, a1 as *mut u8, a2 as usize).map(|v| v as i64),
        SYS_WRITE => fs_ops::write(a0 as u32, a1 as *const u8, a2 as usize).map(|v| v as i64),
        SYS_OPEN => fs_ops::open(a0 as *const u8, a1 as u32).map(|v| v as i64),
        SYS_CLOSE => fs_ops::close(a0 as u32).map(|_| 0),

        SYS_MMAP => memory::mmap(a0, a1 as usize, a2 as u32, a3 as u32).map(|v| v as i64),
        SYS_MUNMAP => memory::munmap(a0, a1 as usize).map(|_| 0),

        SYS_SPAWN => process::spawn(a0 as u32, a1, a2).map(|v| v as i64),
        SYS_EXIT => { process::exit(a0 as i32); },
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 4) - [public] open

```
pub fn open(path_ptr: *const u8, flags: u32) -> KResult<u32> {
    if path_ptr.is_null() { return Err(KernelError::BadAddress); }
    let path = unsafe { core::ffi::CStr::from_ptr(path_ptr as *const i8) }
        .to_str().map_err(|_| KernelError::InvalidArg)?;
    let handle = Vfs::open(path, flags)?;
    Ok(handle.0)
}

pub fn read(fd: u32, buf: *mut u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts_mut(buf, len) };
    Vfs::read(Handle(fd as i32), slice)
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 12) - [public] read

```
pub fn read(fd: u32, buf: *mut u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts_mut(buf, len) };
    Vfs::read(Handle(fd as i32), slice)
}

pub fn write(fd: u32, buf: *const u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts(buf, len) };
    Vfs::write(Handle(fd as i32), slice)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 18) - [public] write

```
pub fn write(fd: u32, buf: *const u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts(buf, len) };
    Vfs::write(Handle(fd as i32), slice)
}

pub fn close(fd: u32) -> KResult<()> {
    extern "C" { fn k_fs_close(fd: i32) -> i32; }
    let rc = unsafe { k_fs_close(fd as i32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(()) }
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 24) - [public] close

```
pub fn close(fd: u32) -> KResult<()> {
    extern "C" { fn k_fs_close(fd: i32) -> i32; }
    let rc = unsafe { k_fs_close(fd as i32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(()) }
}
```

kernel_workspace/base/kw-base/src/api/memory.rs (line 8) - [public] mmap

```
pub fn mmap(addr: u64, len: usize, prot: u32, _flags: u32) -> KResult<u64> {
    if len == 0 { return Err(KernelError::InvalidArg); }

    let mut vmm_flags = 0;
    if prot & PROT_READ != 0 { vmm_flags |= vmm::VMM_FLAG_READ; }
    if prot & PROT_WRITE != 0 { vmm_flags |= vmm::VMM_FLAG_WRITE; }
    if prot & PROT_EXEC != 0 { vmm_flags |= vmm::VMM_FLAG_EXEC; }

    let virt = vmm::alloc(len, vmm_flags).map_err(KernelError::from)?;
    Ok(virt.0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/memory.rs (line 20) - [public] munmap

```
pub fn munmap(addr: u64, len: usize) -> KResult<()> {
    if len == 0 { return Err(KernelError::InvalidArg); }
    vmm::free(kstd_base::VirtAddr(addr), len).map_err(KernelError::from)
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 4) - [internal] k_net_socket

```
fn k_net_socket(domain: u32, sock_type: u32, proto: u32) -> i32;
fn k_net_bind(fd: i32, addr: *const u8, len: u32) -> i32;
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 5) - [internal] k_net_bind

```
fn k_net_bind(fd: i32, addr: *const u8, len: u32) -> i32;
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 6) - [internal] k_net_send

```
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    // ... (truncated, see source file for more)
```


kernel_workspace/base/kw-base/src/api/net_ops.rs (line 7) - [internal] k_net_recv

```
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;
}

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 10) - [public] socket

```
pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 15) - [public] send

```
pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_recv(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 21) - [public] recv

```
pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_recv(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
```

kernel_workspace/base/kw-base/src/api/process.rs (line 4) - [internal] k_proc_spawn

```
fn k_proc_spawn(world_id: u32, entry: u64, stack: u64) -> i64;
fn k_proc_exit(code: i32);
fn k_proc_yield();
fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 5) - [internal] k_proc_exit

```
fn k_proc_exit(code: i32);
fn k_proc_yield();
fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 6) - [internal] k_proc_yield

```
fn k_proc_yield();
fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 7) - [internal] k_proc_get_pid

```
fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 10) - [public] spawn

```
pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}

pub fn yield_now() {
    unsafe { k_proc_yield(); }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/process.rs (line 15) - [public] exit

```
pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}

pub fn yield_now() {
    unsafe { k_proc_yield(); }
}

pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_workspace/base/kw-base/src/api/process.rs (line 20) - [public] yield_now

```
pub fn yield_now() {
    unsafe { k_proc_yield(); }
}

pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_workspace/base/kw-base/src/api/process.rs (line 24) - [public] get_pid

```
pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 4) - [public] disable

```
pub fn disable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("cli", options(nomem, nostack)); }
}

#[inline(always)]
pub fn enable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("sti", options(nomem, nostack)); }
}

#[inline(always)]
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 10) - [public] enable

```
pub fn enable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("sti", options(nomem, nostack)); }
}

#[inline(always)]
pub fn halt() {
    unsafe { core::arch::asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 16) - [public] halt

```
pub fn halt() {
    unsafe { core::arch::asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    #[cfg(target_arch = "x86_64")]
    {
        let flags: u64;
        unsafe { core::arch::asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
        (flags & (1 << 9)) != 0
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 21) - [public] are_enabled

```
pub fn are_enabled() -> bool {
    #[cfg(target_arch = "x86_64")]
    {
        let flags: u64;
        unsafe { core::arch::asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
        (flags & (1 << 9)) != 0
    }
    #[cfg(not(target_arch = "x86_64"))]
    { false }
}

pub struct InterruptGuard {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 32) - [public] InterruptGuard

```
pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        let was_enabled = are_enabled();
        disable();
        Self { was_enabled }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 37) - [public] disable

```
pub fn disable() -> Self {
    let was_enabled = are_enabled();
    disable();
    Self { was_enabled }
}

impl Drop for InterruptGuard {
    fn drop(&mut self) {
        if self.was_enabled { enable(); }
    }
}
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 45) - [internal] drop

```
fn drop(&mut self) {
    if self.was_enabled { enable(); }
}

}
```

kernel_Workspace/base/kw-base/src/arch/registers.rs (line 3) - [public] CpuContext

```
pub struct CpuContext {
    pub r15: u64, pub r14: u64, pub r13: u64, pub r12: u64,
    pub rbp: u64, pub rbx: u64,
    pub r11: u64, pub r10: u64, pub r9: u64, pub r8: u64,
    pub rax: u64, pub rcx: u64, pub rdx: u64, pub rsi: u64,
    pub rdi: u64,
    pub rip: u64,
    pub cs: u64,
    pub rflags: u64,
    pub rsp: u64,
    pub ss: u64,
}
```

kernel_Workspace/base/kw-base/src/arch/x86_64.rs (line 30) - [public] read_cr3

```
pub fn read_cr3() -> u64 {
    let val: u64;
    unsafe { asm!("mov {}, cr3", out(reg) val, options(nomem, nostack, preserves_flags)); }
    val
}

#[inline(always)]
pub fn write_cr3(val: u64) {
    unsafe { asm!("mov cr3, {}", in(reg) val, options(nomem, nostack, preserves_flags)); }
}
```

kernel_Workspace/base/kw-base/src/arch/x86_64.rs (line 37) - [public] write_cr3

```
pub fn write_cr3(val: u64) {
    unsafe { asm!("mov cr3, {}", in(reg) val, options(nomem, nostack, preserves_flags)); }
}
```

kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 5) - [public] init_serial

```
pub fn init_serial() {
    unsafe {
        outb(COM1 + 1, 0x00); // Disable interrupts
        outb(COM1 + 3, 0x80); // Enable DLAB
        outb(COM1 + 0, 0x03); // 38400 baud
        outb(COM1 + 1, 0x00);
        outb(COM1 + 3, 0x03); // 8 bits, no parity, one stop bit
        outb(COM1 + 2, 0xC7); // Enable FIFO
        outb(COM1 + 4, 0x0B); // IRQs enabled, RTS/DSR set
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/early_print.rs (line 17) - [internal]

is_transmitter_empty

```
fn is_transmitter_empty() -> bool {
    unsafe { (inb(COM1 + 5) & 0x20) != 0 }
}

pub fn print_char(c: u8) {
    while !is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(COM1, c); }
}

pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 21) - [public] print_char

```
pub fn print_char(c: u8) {
    while !is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(COM1, c); }
}

pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
        print_char(b);
    }
}
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 26) - [public] print_str

```
pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
        print_char(b);
    }
}
```

kernel_workspace/base/kw-base/src/boot/memory_detect.rs (line 5) - [public] MemoryRegion

```
pub struct MemoryRegion {
    pub base: PhysAddr,
    pub size: Size,
    pub is_usable: bool,
}

pub fn extract_memory_map(info: &BootInfo) -> (usize, [MemoryRegion; 32]) {
    let mut regions = [MemoryRegion { base: PhysAddr(0), size: Size(0), is_usable: false }; 32];
    let mut count = 0;

    let tag_ptr = info.find_tag(TAG_TYPE_MMAP);
    if tag_ptr.is_null() { return (0, regions); }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/memory_detect.rs (line 11) - [public]

extract_memory_map

```
pub fn extract_memory_map(info: &BootInfo) -> (usize, [MemoryRegion; 32]) {
    let mut regions = [MemoryRegion { base: PhysAddr(0), size: Size(0), is_usable: false }; 32];
    let mut count = 0;

    let tag_ptr = info.find_tag(TAG_TYPE_MMAP);
    if tag_ptr.is_null() { return (0, regions); }

    let mmap = unsafe { &*(tag_ptr as *const MmapTag) };
    let mut entry_ptr = unsafe { (tag_ptr as *const u8).add(core::mem::size_of::<MmapTag>()) };
    let end_ptr = unsafe { (tag_ptr as *const u8).add(mmap.tag.size as usize) };

    while entry_ptr < end_ptr && count < 32 {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 6) - [public] Multiboot2Info

```
pub struct Multiboot2Info {
    pub total_size: u32,
    pub reserved: u32,
}

#[repr(C)]
pub struct Multiboot2Tag {
    pub tag_type: u32,
    pub size: u32,
}

pub const TAG_TYPE_END: u32 = 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 12) - [public] Multiboot2Tag

```
pub struct Multiboot2Tag {
    pub tag_type: u32,
    pub size: u32,
}

pub const TAG_TYPE_END: u32 = 0;
pub const TAG_TYPE_CMDLINE: u32 = 1;
pub const TAG_TYPE_BOOTLOADER: u32 = 2;
pub const TAG_TYPE_MODULE: u32 = 3;
pub const TAG_TYPE_BASIC_MEMINFO: u32 = 4;
pub const TAG_TYPE_BOOTDEV: u32 = 5;
pub const TAG_TYPE_MMAP: u32 = 6;
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 27) - [public] MmapTag

```
pub struct MmapTag {
    pub tag: Multiboot2Tag,
    pub entry_size: u32,
    pub entry_version: u32,
}

#[repr(C)]
pub struct MmapEntry {
    pub base_addr: u64,
    pub length: u64,
    pub entry_type: u32,
    pub reserved: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 34) - [public] MmapEntry

```
pub struct MmapEntry {
    pub base_addr: u64,
    pub length: u64,
    pub entry_type: u32,
    pub reserved: u32,
}

pub struct BootInfo {
    pub ptr: *const Multiboot2Info,
}

unsafe impl Send for BootInfo {}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 41) - [public] BootInfo

```
pub struct BootInfo {
    pub ptr: *const Multiboot2Info,
}

unsafe impl Send for BootInfo {}
unsafe impl Sync for BootInfo {}

impl BootInfo {
    pub unsafe fn from_ptr(ptr: *const u8) -> Self {
        Self { ptr: ptr as *const Multiboot2Info }
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 53) - [public] first_tag

```
pub fn first_tag(&self) -> *const Multiboot2Tag {
    unsafe { (self.ptr as *const u8).add(8) as *const Multiboot2Tag }
}

pub fn next_tag(tag: *const Multiboot2Tag) -> *const Multiboot2Tag {
    unsafe {
        let addr = tag as usize;
        let next = (addr + (*tag).size as usize + 7) & !7;
        next as *const Multiboot2Tag
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 57) - [public] next_tag

```
pub fn next_tag(tag: *const Multiboot2Tag) -> *const Multiboot2Tag {
    unsafe {
        let addr = tag as usize;
        let next = (addr + (*tag).size as usize + 7) & !7;
        next as *const Multiboot2Tag
    }
}

pub fn find_tag(&self, tag_type: u32) -> *const Multiboot2Tag {
    let mut tag = self.first_tag();
    unsafe {
        while (*tag).tag_type != TAG_TYPE_END {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/base/kw-base/src/boot/multiboot.rs (line 65) - [public] find_tag

```
pub fn find_tag(&self, tag_type: u32) -> *const Multiboot2Tag {
    let mut tag = self.first_tag();
    unsafe {
        while (*tag).tag_type != TAG_TYPE_END {
            if (*tag).tag_type == tag_type { return tag; }
            tag = Self::next_tag(tag);
        }
    }
    core::ptr::null()
}
```

kernel_Workspace/base/kw-base/src/boot/stack.rs (line 2) - [public] BootStack

```
pub struct BootStack {
    pub data: [u8; 65536],
}

impl BootStack {
    pub const fn new() -> Self {
        Self { data: [0; 65536] }
    }

    pub fn top(&self) -> u64 {
        (self.data.as_ptr() as u64) + (self.data.len() as u64)
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/boot/stack.rs (line 11) - [public] top

```
pub fn top(&self) -> u64 {
    (self.data.as_ptr() as u64) + (self.data.len() as u64)
}

}
```


kernel_Workspace/base/kw-base/src/core/error.rs (line 5) - [public] KernelError

```
pub enum KernelError {
    Ok = 0,
    NoMemory = -1,
    InvalidArg = -2,
    NotFound = -3,
    AlreadyExists = -4,
    PermissionDenied = -5,
    Busy = -6,
    Timeout = -7,
    NotSupported = -8,
    IOError = -9,
    BadAddress = -10,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/error.rs (line 65) - [internal] from

```
fn from(s: kstd_base::Status) -> Self {
    match s {
        kstd_base::Status::Success => Self::Ok,
        kstd_base::Status::OutOfMemory => Self::NoMemory,
        kstd_base::Status::InvalidArgument => Self::InvalidArg,
        kstd_base::Status::NotFound => Self::NotFound,
        kstd_base::Status::AlreadyExists => Self::AlreadyExists,
        kstd_base::Status::PermissionDenied => Self::PermissionDenied,
        kstd_base::Status::Busy => Self::Busy,
        kstd_base::Status::TimedOut => Self::Timeout,
        kstd_base::Status::NotSupported => Self::NotSupported,
        kstd_base::Status::IOError => Self::IOError,
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/error.rs (line 84) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    write!(f, "{:?}", self)
}
}
```

kernel_Workspace/base/kw-base/src/core/result.rs (line 5) - [public] IntoKResult

```
pub trait IntoKResult<T> {
    fn into_kresult(self) -> KResult<T>;
}

impl<T> IntoKResult<T> for Option<T> {
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/result.rs (line 6) - [internal] into_kresult

```
fn into_kresult(self) -> KResult<T>;
}

impl<T> IntoKResult<T> for Option<T> {
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/result.rs (line 10) - [internal] into_kresult

```
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
    }
}
```

kernel_workspace/base/kw-base/src/core/result.rs (line 16) - [internal] into_kresult

```
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
    }
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 4) - [public] BlockDevice

```
pub trait BlockDevice {
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), KernelError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 5) - [internal] read_block

```
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), KernelError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 6) - [internal] write_block

```
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 7) - [internal] block_size

```
fn block_size(&self) -> usize;
fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 8) - [internal] block_count

```
fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 11) - [public] CharDevice

```
pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 12) - [internal] read

```
fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 13) - [internal] write

```
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 16) - [public] NetworkDevice

```
pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 17) - [internal] send

```
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 18) - [internal] recv

```
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 19) - [internal] mac_address

```
fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 22) - [public] FileSystem

```
pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 23) - [internal] open

```
fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
fn close(&self, handle: Handle) -> Result<(), KernelError>;
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 24) - [internal] close

```
fn close(&self, handle: Handle) -> Result<(), KernelError>;
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 25) - [internal] read

```
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 26) - [internal] write

```
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 27) - [internal] stat

```
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    pub ctime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 31) - [public] FileStat

```
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    pub ctime: u64,
    pub is_dir: bool,
}

pub trait Scheduler {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 42) - [public] Scheduler

```
pub trait Scheduler {
    fn schedule(&mut self) -> Option<TaskId>;
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 43) - [internal] schedule

```
    fn schedule(&mut self) -> Option<TaskId>;
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 44) - [internal] enqueue

```
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 45) - [internal] dequeue

```
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 46) - [internal] yield_current

```
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 49) - [public] InterruptController

```
pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 50) - [internal] enable_irq

```
fn enable_irq(&self, irq: u32);
fn disable_irq(&self, irq: u32);
fn ack_irq(&self, irq: u32);
fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 51) - [internal] disable_irq

```
fn disable_irq(&self, irq: u32);
fn ack_irq(&self, irq: u32);
fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 52) - [internal] ack_irq

```
fn ack_irq(&self, irq: u32);
fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 53) - [internal] end_of_interrupt

```
fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/types.rs (line 5) - [public] Pid

```
pub struct Pid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Tid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/core/types.rs (line 9) - [public] Tid

```
pub struct Tid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    pub const STDIN: Self = Self(0);
    pub const STDOUT: Self = Self(1);
    pub const STDERR: Self = Self(2);

    // ... (truncated, see source file for more)
}
```


kernel_workspace/base/kw-base/src/core/types.rs (line 13) - [public] Fd

```
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    pub const STDIN: Self = Self(0);
    pub const STDOUT: Self = Self(1);
    pub const STDERR: Self = Self(2);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 >= 0
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/types.rs (line 29) - [public] PrivilegeLevel

```
pub enum PrivilegeLevel {
    Kernel = 0,
    Driver = 1,
    System = 2,
    User = 3,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum Arch {
    X86_64 = 0,
    AArch64 = 1,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/types.rs (line 38) - [public] Arch

```
pub enum Arch {
    X86_64 = 0,
    AArch64 = 1,
    Riscv64 = 2,
}
```

kernel_workspace/base/kw-base/src/crypto/ciphers.rs (line 4) - [internal] k_aes256_enc

```
fn k_aes256_enc(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
fn k_aes256_dec(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
}

pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/ciphers.rs (line 5) - [internal] k_aes256_dec

```
fn k_aes256_dec(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
}

pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 8) - [public] BlockCipher

```
pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 13) - [internal] encrypt

```
    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            // ... (truncated, see source file for more)
        }
    }
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 14) - [internal] decrypt

```
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            return Err(KernelError::InvalidArg);
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 17) - [public] Aes256Cbc

```
pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            return Err(KernelError::InvalidArg);
        }
        if data.len() % Self::BLOCK_LEN != 0 {
            return Err(KernelError::InvalidArg);
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 23) - [internal] encrypt

```
fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
    if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
        return Err(KernelError::InvalidArg);
    }
    if data.len() % Self::BLOCK_LEN != 0 {
        return Err(KernelError::InvalidArg);
    }
    if unsafe { k_aes256_enc(key.as_ptr(), iv.as_ptr(), data.as_mut_ptr(), data.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 37) - [internal] decrypt

```
fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
    if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
        return Err(KernelError::InvalidArg);
    }
    if data.len() % Self::BLOCK_LEN != 0 {
        return Err(KernelError::InvalidArg);
    }
    if unsafe { k_aes256_dec(key.as_ptr(), iv.as_ptr(), data.as_mut_ptr(), data.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 4) - [internal] k_sha256

```
fn k_sha256(data: *const u8, len: usize, out: *mut u8) -> bool;
fn k_sha512(data: *const u8, len: usize, out: *mut u8) -> bool;
}

pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ hashes.rs (line 5) - [internal] k_sha512

```
fn k_sha512(data: *const u8, len: usize, out: *mut u8) -> bool;
}

pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 8) - [public] Hasher

```
pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 10) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 13) - [public] Sha256

```
pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 16) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
    if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
    if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}

pub struct Sha512;
impl Hasher for Sha512 {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 26) - [public] Sha512

```
pub struct Sha512;
impl Hasher for Sha512 {
    const OUTPUT_LEN: usize = 64;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha512(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 29) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
    if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
    if unsafe { k_sha512(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 4) - [public] SymmetricKey

```
pub struct SymmetricKey {
    inner: SecureBuffer,
}

impl SymmetricKey {
    pub fn generate(len: usize) -> KResult<Self> {
        let mut buf = SecureBuffer::new(len).ok_or(KernelError::NoMemory)?;
        super::random::fill_random(buf.as_mut_slice())?;
        Ok(Self { inner: buf })
    }

    pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 9) - [public] generate

```
pub fn generate(len: usize) -> KResult<Self> {
    let mut buf = SecureBuffer::new(len).ok_or(KernelError::NoMemory)?;
    super::random::fill_random(buf.as_mut_slice())?;
    Ok(Self { inner: buf })
}

pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
    let mut buf = SecureBuffer::new(bytes.len()).ok_or(KernelError::NoMemory)?;
    buf.as_mut_slice().copy_from_slice(bytes);
    Ok(Self { inner: buf })
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 15) - [public] from_bytes

```
pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
    let mut buf = SecureBuffer::new(bytes.len()).ok_or(KernelError::NoMemory)?;
    buf.as_mut_slice().copy_from_slice(bytes);
    Ok(Self { inner: buf })
}

pub fn as_slice(&self) -> &[u8] {
    self.inner.as_slice()
}

}
```

kernel_Workspace/base/kw-base/src/crypto/keys.rs (line 21) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {
    self.inner.as_slice()
}
```

kernel_Workspace/base/kw-base/src/crypto/random.rs (line 4) - [internal] k_get_random

```
fn k_get_random(buf: *mut u8, len: usize) -> bool;
fn k_get_hw_random(buf: *mut u8, len: usize) -> bool;

pub fn fill_random(buf: &mut [u8]) -> KResult<()> {
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/random.rs (line 5) - [internal] k_get_hw_random

```
fn k_get_hw_random(buf: *mut u8, len: usize) -> bool;

pub fn fill_random(buf: &mut [u8]) -> KResult<()> {
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}

pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/random.rs (line 8) - [public] fill_random

```
pub fn fill_random(buf: &mut [u8]) -> KResult<()> {
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}

pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {
    if unsafe { k_get_hw_random(buf.as_mut_ptr(), buf.len()) } {
        Ok(())
    } else {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/random.rs (line 16) - [public] fill_hw_random

```
pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {
    if unsafe { k_get_hw_random(buf.as_mut_ptr(), buf.len()) } {
        Ok(())
    } else {
        Err(KernelError::NotSupported)
    }
}

pub fn next_u32() -> KResult<u32> {
    let mut val = 0u32;
    fill_random(unsafe { core::slice::from_raw_parts_mut(&mut val as *mut u32 as *mut u8, 4) })?;
    Ok(val)
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 24) - [public] next_u32

```
pub fn next_u32() -> KResult<u32> {
    let mut val = 0u32;
    fill_random(unsafe { core::slice::from_raw_parts_mut(&mut val as *mut u32 as *mut u8, 4) })?;
    Ok(val)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 5) - [public] secure_zero

*// Securely zero out memory. Uses volatile writes to prevent the compiler
// from optimizing the memset away (crucial for wiping keys).*

```
pub fn secure_zero(ptr: *mut u8, len: usize) {
    unsafe {
        for i in 0..len {
            ptr::write_volatile(ptr.add(i), 0);
        }
        core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::SeqCst);
    }
}

pub struct SecureBuffer {
    ptr: *mut u8,
    len: usize,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 14) - [public] SecureBuffer

```
pub struct SecureBuffer {
    ptr: *mut u8,
    len: usize,
}

impl SecureBuffer {
    pub fn new(len: usize) -> Option<Self> {
        let layout = core::alloc::Layout::from_size_align(len, 8).ok()?;
        let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.ok()?;
        secure_zero(ptr, len);
        Some(Self { ptr, len })
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 20) - [public] new

```
pub fn new(len: usize) -> Option<Self> {
    let layout = core::alloc::Layout::from_size_align(len, 8).ok()?;
    let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.ok()?;
    secure_zero(ptr, len);
    Some(Self { ptr, len })
}

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.ptr, self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 27) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.ptr, self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr, self.len) }
}

impl Drop for SecureBuffer {
    fn drop(&mut self) {
        secure_zero(self.ptr, self.len);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 31) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr, self.len) }
}

impl Drop for SecureBuffer {
    fn drop(&mut self) {
        secure_zero(self.ptr, self.len);
        let layout = core::alloc::Layout::from_size_align(self.len, 8).unwrap();
        unsafe { kstd_alloc::KernelHeap.dealloc(self.ptr, layout); }
    }
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 37) - [internal] drop

```
fn drop(&mut self) {
    secure_zero(self.ptr, self.len);
    let layout = core::alloc::Layout::from_size_align(self.len, 8).unwrap();
    unsafe { kstd_alloc::KernelHeap.dealloc(self.ptr, layout); }
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 7) - [public] Ps2Controller

```
pub struct Ps2Controller;

impl Ps2Controller {
    pub fn init() {
        // Disable devices
        unsafe {
            outb(PS2_CMD, 0xAD);
            outb(PS2_CMD, 0xA7);

            // Flush output buffer
            while (inb(PS2_STATUS) & 0x01) != 0 { inb(PS2_DATA); }

            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 10) - [public] init

```
pub fn init() {
    // Disable devices
    unsafe {
        outb(PS2_CMD, 0xAD);
        outb(PS2_CMD, 0xA7);

        // Flush output buffer
        while (inb(PS2_STATUS) & 0x01) != 0 { inb(PS2_DATA); }

        // Enable devices
        outb(PS2_CMD, 0xAE);
        outb(PS2_CMD, 0xA8);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 25) - [public] read_scancode

```
pub fn read_scancode() -> Option<u8> {
    unsafe {
        if (inb(PS2_STATUS) & 0x01) != 0 {
            Some(inb(PS2_DATA))
        } else {
            None
        }
    }
}
```


kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 4) - [public] Uart16550

```
pub struct Uart16550 {
    port: u16,
}

impl Uart16550 {
    pub const COM1: u16 = 0x3F8;

    pub fn new(port: u16) -> Self {
        let mut uart = Self { port };
        uart.init();
        uart
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 11) - [public] new

```
pub fn new(port: u16) -> Self {
    let mut uart = Self { port };
    uart.init();
    uart
}

fn init(&mut self) {
    unsafe {
        outb(self.port + 1, 0x00); // Disable interrupts
        outb(self.port + 3, 0x80); // Enable DLAB
        outb(self.port + 0, 0x03); // 38400 baud
        outb(self.port + 1, 0x00);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 17) - [internal] init

```
fn init(&mut self) {
    unsafe {
        outb(self.port + 1, 0x00); // Disable interrupts
        outb(self.port + 3, 0x80); // Enable DLAB
        outb(self.port + 0, 0x03); // 38400 baud
        outb(self.port + 1, 0x00);
        outb(self.port + 3, 0x03); // 8 bits, no parity, one stop bit
        outb(self.port + 2, 0xC7); // Enable FIFO
        outb(self.port + 4, 0x0B); // IRQs enabled, RTS/DSR set
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 29) - [internal] is_transmitter_empty

```
fn is_transmitter_empty(&self) -> bool {
    unsafe { (inb(self.port + 5) & 0x20) != 0 }
}

pub fn write_byte(&mut self, b: u8) {
    while !self.is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(self.port, b); }
}

impl Write for Uart16550 {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 33) - [public] write_byte

```
pub fn write_byte(&mut self, b: u8) {
    while !self.is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(self.port, b); }
}

impl Write for Uart16550 {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.bytes() {
            if b == b'\n' { self.write_byte(b'\r'); }
            self.write_byte(b);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 40) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    for b in s.bytes() {
        if b == b'\n' { self.write_byte(b'\r'); }
        self.write_byte(b);
    }
    Ok(())
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 5) - [public] FileType

```
pub enum FileType { Regular, Directory, CharDevice, BlockDevice, Pipe, Socket }

pub trait Inode {
    fn stat(&self) -> KResult<FileStat>;
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 7) - [public] Inode

```
pub trait Inode {
    fn stat(&self) -> KResult<FileStat>;
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 8) - [internal] stat

```
fn stat(&self) -> KResult<FileStat>;
fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 9) - [internal] read

```
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 10) - [internal] write

```
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 11) - [internal] lookup

```
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<()>;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 14) - [public] File

```
pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<()>;
}

pub struct Vfs;
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 15) - [internal] inode

```
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 16) - [internal] flags

```
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 19) - [public] FileSystem

```
pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 20) - [internal] name

```
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 21) - [internal] root_inode

```
fn root_inode(&self) -> Handle;
fn sync(&self) -> KResult<()>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 22) - [internal] sync

```
fn sync(&self) -> KResult<()>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 25) - [public] Vfs

```
pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
    }

    pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
        extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 28) - [public] open

```
pub fn open(path: &str, flags: u32) -> KResult<Handle> {
    // Route to legacy kernel VFS via FFI if needed, or internal registry
    extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
    let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
}

pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
    extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
    let rc = unsafe { k_fs_read(fd.0 as i32, buf.as_mut_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/fs/vfs.rs (line 35) - [public] read

```
pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
    extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
    let rc = unsafe { k_fs_read(fd.0 as i32, buf.as_mut_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn write(fd: Handle, buf: &[u8]) -> KResult<usize> {
    extern "C" { fn k_fs_write(fd: i32, buf: *const u8, len: u32) -> i32; }
    let rc = unsafe { k_fs_write(fd.0 as i32, buf.as_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
}
```

kernel_workspace/base/kw-base/src/fs/vfs.rs (line 41) - [public] write

```
pub fn write(fd: Handle, buf: &[u8]) -> KResult<usize> {
    extern "C" { fn k_fs_write(fd: i32, buf: *const u8, len: u32) -> i32; }
    let rc = unsafe { k_fs_write(fd.0 as i32, buf.as_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 4) - [internal] k_acpi_get_rsdp

```
fn k_acpi_get_rsdp() -> *const u8;
fn k_acpi_find_table(sig: *const u8, idx: u32) -> *const u8;
}

pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 5) - [internal] k_acpi_find_table

```
fn k_acpi_find_table(sig: *const u8, idx: u32) -> *const u8;
}

pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 8) - [public] get_rsdp

```
pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 13) - [public] find_table

```
pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 5) - [public] DmaBuffer

```
pub struct DmaBuffer {
    frames: ContiguousFrames,
    virt: VirtAddr,
}

impl DmaBuffer {
    pub fn allocate(pages: usize) -> KResult<Self> {
        let frames = ContiguousFrames::alloc(pages)?;
        let virt = gluecore::bridge::mm::vmm::alloc(
            frames.byte_size(),
            gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE | gluecore::bridge::mm::vmm::VMM_FLAG_EXEC,
        ).map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 11) - [public] allocate

```
pub fn allocate(pages: usize) -> KResult<Self> {
    let frames = ContiguousFrames::alloc(pages)?;
    let virt = gluecore::bridge::mm::vmm::alloc(
        frames.byte_size(),
        gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE | gluecore::bridge::mm::vmm::VMM_FLAG_EXEC,
    ).map_err(KernelError::from)?;

    Ok(Self { frames, virt })
}

pub fn phys_addr(&self) -> PhysAddr { self.frames.phys_addr() }
pub fn virt_addr(&self) -> VirtAddr { self.virt }
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 21) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr { self.frames.phys_addr() }
pub fn virt_addr(&self) -> VirtAddr { self.virt }
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 22) - [public] virt_addr

```
pub fn virt_addr(&self) -> VirtAddr { self.virt }
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 23) - [public] size

```
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 25) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 29) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 7) - [public] PciAddress

```
pub struct PciAddress {
    pub bus: u8,
    pub device: u8,
    pub function: u8,
}

impl PciAddress {
    pub const fn new(bus: u8, device: u8, function: u8) -> Self {
        Self { bus, device, function }
    }

    fn config_addr(self, offset: u8) -> u32 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 18) - [internal] config_addr

```
fn config_addr(self, offset: u8) -> u32 {
    0x8000_0000
    | ((self.bus as u32) << 16)
    | ((self.device as u32 & 0x1F) << 11)
    | ((self.function as u32 & 0x07) << 8)
    | (offset as u32 & 0xFC)
}
}
```

```
#[derive(Debug, Clone, Copy)]
pub struct PciDevice {
    pub addr: PciAddress,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 28) - [public] PciDevice

```
pub struct PciDevice {
    pub addr: PciAddress,
    pub vendor_id: u16,
    pub device_id: u16,
    pub class_code: u8,
    pub subclass: u8,
}

pub fn read_config32(addr: PciAddress, offset: u8) -> u32 {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        inl(PCI_CONFIG_DATA)
    }
    // ... (truncated, see source file for more)
}
```


kernel_workspace/base/kw-base/src/hal/pci.rs (line 36) - [public] read_config32

```
pub fn read_config32(addr: PciAddress, offset: u8) -> u32 {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        inl(PCI_CONFIG_DATA)
    }
}

pub fn write_config32(addr: PciAddress, offset: u8, value: u32) {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        outl(PCI_CONFIG_DATA, value);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 43) - [public] write_config32

```
pub fn write_config32(addr: PciAddress, offset: u8, value: u32) {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        outl(PCI_CONFIG_DATA, value);
    }
}

pub fn scan_bus() -> kstd_data::Vec<PciDevice> {
    let mut devices = kstd_data::Vec::new();
    for bus in 0..=255 {
        for dev in 0..32 {
            let addr = PciAddress::new(bus, dev, 0);
            // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 50) - [public] scan_bus

```
pub fn scan_bus() -> kstd_data::Vec<PciDevice> {
    let mut devices = kstd_data::Vec::new();
    for bus in 0..=255 {
        for dev in 0..32 {
            let addr = PciAddress::new(bus, dev, 0);
            let id = read_config32(addr, 0);
            let vendor = id as u16;
            if vendor == 0xFFFF { continue; }

            devices.push(PciDevice {
                addr,
                vendor_id: vendor,
            });
            // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/timers.rs (line 4) - [internal] k_timer_get_ticks

```
fn k_timer_get_ticks() -> u64;
fn k_timer_get_frequency() -> u64;
fn k_timer_sleep_ms(ms: u32);
fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 5) - [internal] k_timer_get_frequency
    fn k_timer_get_frequency() -> u64;
    fn k_timer_sleep_ms(ms: u32);
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 6) - [internal] k_timer_sleep_ms
    fn k_timer_sleep_ms(ms: u32);
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 7) - [internal] k_timer_set_oneshot
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 10) - [public] get_ticks
pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

// ... (truncated, see source file for more)

```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 14) - [public] get_frequency

```
pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 18) - [public] sleep_ms

```
pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 22) - [public] set_oneshot

```
pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 4) - [internal] k_usb_init

```
fn k_usb_init() -> bool;
fn k_usb_enum_devices() -> u32;
fn k_usb_get_device_count() -> u32;

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 5) - [internal] k_usb_enum_devices

```
fn k_usb_enum_devices() -> u32;
fn k_usb_get_device_count() -> u32;

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 6) - [internal] k_usb_get_device_count
    fn k_usb_get_device_count() -> u32;
}

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

pub fn enumerate_devices() -> KResult<u32> {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 9) - [public] init
pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

pub fn enumerate_devices() -> KResult<u32> {
    let count = unsafe { k_usb_enum_devices() };
    if count == 0 && unsafe { k_usb_get_device_count() } == 0 {
        Err(KernelError::NotFound)
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 17) - [public] enumerate_devices
pub fn enumerate_devices() -> KResult<u32> {
    let count = unsafe { k_usb_enum_devices() };
    if count == 0 && unsafe { k_usb_get_device_count() } == 0 {
        Err(KernelError::NotFound)
    } else {
        Ok(count)
    }
}

kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 4) - [internal] k_ipc_send
    fn k_ipc_send(dst: u32, a0: u64, a1: u64) -> i32;
    fn k_ipc_recv(a0: *mut u64, a1: *mut u64) -> i32;
}

#[derive(Debug, Clone, Copy, Default)]
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 5) - [internal] k_ipc_recv
    fn k_ipc_recv(a0: *mut u64, a1: *mut u64) -> i32;
}

#[derive(Debug, Clone, Copy, Default)]
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    // ... (truncated, see source file for more)
}

```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 9) - [public] IpcMessage

```
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn recv() -> KResult<IpcMessage> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 15) - [public] send

```
pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn recv() -> KResult<IpcMessage> {
    let mut a0: u64 = 0;
    let mut a1: u64 = 0;
    let from = unsafe { k_ipc_recv(&mut a0, &mut a1) };
    if from < 0 {
        Err(KernelError::from_code(from))
    } else {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 20) - [public] recv

```
pub fn recv() -> KResult<IpcMessage> {
    let mut a0: u64 = 0;
    let mut a1: u64 = 0;
    let from = unsafe { k_ipc_recv(&mut a0, &mut a1) };
    if from < 0 {
        Err(KernelError::from_code(from))
    } else {
        Ok(IpcMessage { from: from as u32, arg0: a0, arg1: a1 })
    }
}
```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 5) - [public] SharedMemory

```
pub struct SharedMemory {
    frames: ContiguousFrames,
    virt: VirtAddr,
}

impl SharedMemory {
    pub fn allocate(pages: usize) -> KResult<Self> {
        let frames = ContiguousFrames::alloc(pages)?;
        let virt = gluecore::bridge::mm::vmm::alloc(
            frames.byte_size(),
            gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE
        ).map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 11) - [public] allocate

```
pub fn allocate(pages: usize) -> KResult<Self> {
    let frames = ContiguousFrames::alloc(pages)?;
    let virt = gluecore::bridge::mm::vmm::alloc(
        frames.byte_size(),
        gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE
    ).map_err(KernelError::from)?;

    for i in 0..pages {
        let phys = PhysAddr(frames.phys_addr().0 + (i as u64 * 4096));
        let v = VirtAddr(virt.0 + (i as u64 * 4096));
        paging::map_page(v, phys, paging::PAGE_PRESENT | paging::PAGE_WRITABLE)?;
    }
    // ... (truncated, see source file for more)
```

```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 27) - [public] virt_addr
    pub fn virt_addr(&self) -> VirtAddr { self.virt }
    pub fn size(&self) -> usize { self.frames.byte_size() }
}

impl Drop for SharedMemory {
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 28) - [public] size
    pub fn size(&self) -> usize { self.frames.byte_size() }
}

impl Drop for SharedMemory {
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 32) - [internal] drop
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 4) - [public] ContiguousFrames
pub struct ContiguousFrames {
    addr: PhysAddr,
    count: usize,
}

impl ContiguousFrames {
    pub fn alloc(count: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames(count).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 10) - [public] alloc
    pub fn alloc(count: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames(count).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames_aligned(count, align_frames).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn phys_addr(&self) -> PhysAddr { self.addr }
    pub fn frame_count(&self) -> usize { self.count }
    // ... (truncated, see source file for more)
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 15) - [public] alloc_aligned
    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames_aligned(count, align_frames).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn phys_addr(&self) -> PhysAddr { self.addr }
    pub fn frame_count(&self) -> usize { self.count }
    pub fn byte_size(&self) -> usize { self.count * 4096 }

    pub fn into_raw(self) -> (PhysAddr, usize) {
        let addr = self.addr;
        let count = self.count;
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 20) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr { self.addr }
pub fn frame_count(&self) -> usize { self.count }
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 21) - [public] frame_count

```
pub fn frame_count(&self) -> usize { self.count }
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 22) - [public] byte_size

```
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 24) - [public] into_raw

```
pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
}
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 37) - [internal] drop

```
fn drop(&mut self) {
    let _ = pmm::free_frames(self.addr, self.count);
}
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 4) - [public] Frame

```
pub struct Frame {
    addr: PhysAddr,
}

impl Frame {
    pub fn alloc() -> KResult<Self> {
        let addr = pmm::alloc_frame().map_err(KernelError::from)?;
        Ok(Self { addr })
    }

    pub fn alloc_zeroed() -> KResult<Self> {
        let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 9) - [public] alloc

```
pub fn alloc() -> KResult<Self> {
    let addr = pmm::alloc_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn alloc_zeroed() -> KResult<Self> {
    let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn phys_addr(&self) -> PhysAddr {
    self.addr
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 14) - [public] alloc_zeroed

```
pub fn alloc_zeroed() -> KResult<Self> {
    let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn phys_addr(&self) -> PhysAddr {
    self.addr
}

pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 19) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr {
    self.addr
}

pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    addr
}

pub unsafe fn from_raw(addr: PhysAddr) -> Self {
    Self { addr }
    // ... (truncated, see source file for more)
}
```


kernel_workspace/base/kw-base/src/mem/frame.rs (line 23) - [public] into_raw

```
pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    addr
}

pub unsafe fn from_raw(addr: PhysAddr) -> Self {
    Self { addr }
}

impl Drop for Frame {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/frame.rs (line 35) - [internal] drop

```
fn drop(&mut self) {
    let _ = pmm::free_frame(self.addr);
}
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 4) - [public] alloc

```
pub fn alloc<T>() -> KResult<*mut T> {
    let layout = Layout::new(core::mem::size_of::<T>(), core::mem::align_of::<T>());
    let ptr = KernelHeap.alloc(layout).map_err(KernelError::from)?;
    Ok(ptr as *mut T)
}

pub fn alloc_zeroed<T>() -> KResult<*mut T> {
    let ptr = alloc::<T>()?;
    unsafe { core::ptr::write_bytes(ptr, 0, 1); }
    Ok(ptr)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 10) - [public] alloc_zeroed

```
pub fn alloc_zeroed<T>() -> KResult<*mut T> {
    let ptr = alloc::<T>()?;
    unsafe { core::ptr::write_bytes(ptr, 0, 1); }
    Ok(ptr)
}

pub unsafe fn free<T>(ptr: *mut T) {
    let layout = Layout::new(core::mem::size_of::<T>(), core::mem::align_of::<T>());
    KernelHeap.dealloc(ptr as *mut u8, layout);
}

pub fn alloc_slice<T>(len: usize) -> KResult<*mut T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 21) - [public] alloc_slice

```
pub fn alloc_slice<T>(len: usize) -> KResult<*mut T> {
    let layout = Layout::new(core::mem::size_of::<T>() * len, core::mem::align_of::<T>());
    let ptr = KernelHeap.alloc(layout).map_err(KernelError::from)?;
    Ok(ptr as *mut T)
}

pub unsafe fn free_slice<T>(ptr: *mut T, len: usize) {
    let layout = Layout::new(core::mem::size_of::<T>() * len, core::mem::align_of::<T>());
    KernelHeap.dealloc(ptr as *mut u8, layout);
}
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 14) - [internal] paging_map_page

```
fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
fn paging_unmap_page(virt: u64) -> bool;
}

pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 15) - [internal] paging_unmap_page

```
fn paging_unmap_page(virt: u64) -> bool;
}

pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 18) - [public] map_page

```
pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    if unsafe { paging_unmap_page(virt.0) } {
        Ok(())
    } else {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 26) - [public] unmap_page

```
pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    if unsafe { paging_unmap_page(virt.0) } {
        Ok(())
    } else {
        Err(KernelError::InvalidArg)
    }
}

pub fn flush_tlb_page(virt: VirtAddr) {
    context::flush_page(virt);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 34) - [public] flush_tlb_page

```
pub fn flush_tlb_page(virt: VirtAddr) {
    context::flush_page(virt);
}

pub fn flush_tlb_all() {
    context::flush_tlb_all();
}
```

```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 38) - [public] flush_tlb_all
    pub fn flush_tlb_all() {
        context::flush_tlb_all();
    }

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 4) - [internal] slab_cache_create
    fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
    fn slab_cache_destroy(cache: *mut u8);
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}
// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 5) - [internal] slab_cache_destroy
    fn slab_cache_destroy(cache: *mut u8);
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 6) - [internal] slab_alloc
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 7) - [internal] slab_free
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}

```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 10) - [public] SlabCache

```
pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 18) - [public] new

```
pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
    let c_name = name.as_ptr();
    let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
    if cache.is_null() {
        Err(KernelError::NoMemory)
    } else {
        Ok(Self { inner: cache })
    }
}

pub fn alloc<T>(&self) -> KResult<*mut T> {
    let ptr = unsafe { slab_alloc(self.inner) };
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 28) - [public] alloc

```
pub fn alloc<T>(&self) -> KResult<*mut T> {
    let ptr = unsafe { slab_alloc(self.inner) };
    if ptr.is_null() {
        Err(KernelError::NoMemory)
    } else {
        Ok(ptr as *mut T)
    }
}

pub fn free<T>(&self, ptr: *mut T) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr as *mut u8) };
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 37) - [public] free

```
pub fn free<T>(&self, ptr: *mut T) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr as *mut u8) };
    }
}

impl Drop for SlabCache {
    fn drop(&mut self) {
        if !self.inner.is_null() {
            unsafe { slab_cache_destroy(self.inner) };
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 45) - [internal] drop

```
fn drop(&mut self) {
    if !self.inner.is_null() {
        unsafe { slab_cache_destroy(self.inner) };
    }
}
```

kernel_workspace/base/kw-base/src/sync/atomic.rs (line 5) - [public] compiler_fence

```
pub fn compiler_fence() {
    core::sync::atomic::compiler_fence(Ordering::SeqCst);
}

#[inline(always)]
pub fn memory_barrier() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("mfence", options(nostack, preserves_flags)); }
}
```

kernel_workspace/base/kw-base/src/sync/atomic.rs (line 10) - [public] memory_barrier

```
pub fn memory_barrier() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("mfence", options(nostack, preserves_flags)); }
}
```

kernel_workspace/base/kw-base/src/sync/futex.rs (line 4) - [internal] k_futex_wait

```
fn k_futex_wait(uaddr: *const u32, expected_val: u32, timeout_ns: u64) -> i32;
fn k_futex_wake(uaddr: *const u32, wake_count: u32) -> i32;

pub const FUTEX_WAIT: u32 = 0;
pub const FUTEX_WAKE: u32 = 1;

pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/futex.rs (line 5) - [internal] k_futex_wake

```
fn k_futex_wake(uaddr: *const u32, wake_count: u32) -> i32;

pub const FUTEX_WAIT: u32 = 0;
pub const FUTEX_WAKE: u32 = 1;

pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/futex.rs (line 11) - [public] wait

```
pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    let rc = unsafe { k_futex_wake(uaddr, count) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}
```

kernel_workspace/base/kw-base/src/sync/futex.rs (line 16) - [public] wake

```
pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    let rc = unsafe { k_futex_wake(uaddr, count) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}
```

kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 11) - [public] Mutex

// A sleeping mutex. Uses atomics for the fast path.

// In a full implementation, the slow path would enqueue the thread

// into a WaitQueue and call the scheduler. Here we use yield as a placeholder.

```
pub struct Mutex<T> {
    state: AtomicU8,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for Mutex<T> {}
unsafe impl<T: Send> Send for Mutex<T> {}

impl<T> Mutex<T> {
    pub const fn new(data: T) -> Self {
        Self {
            state: AtomicU8::new(UNLOCKED),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 27) - [public] lock

```
pub fn lock(&self) -> MutexGuard<'_, T> {
    loop {
        if self.state.compare_exchange_weak(
            UNLOCKED, LOCKED, Ordering::Acquire, Ordering::Relaxed
        ).is_ok() {
            return MutexGuard { lock: self };
        }
        // Slow path: yield CPU instead of busy-spinning
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 39) - [public] try_lock

```
pub fn try_lock(&self) -> Option<MutexGuard<'_, T>> {
    if self.state.compare_exchange(
        UNLOCKED, LOCKED, Ordering::Acquire, Ordering::Relaxed
    ).is_ok() {
        Some(MutexGuard { lock: self })
    } else {
        None
    }
}

fn unlock(&self) {
    self.state.store(UNLOCKED, Ordering::Release);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/sync/mutex.rs (line 49) - [internal] unlock

```
fn unlock(&self) {
    self.state.store(UNLOCKED, Ordering::Release);
    // TODO: Wake up one thread from the waitqueue if any
}

pub struct MutexGuard<'a, T> {
    lock: &'a Mutex<T>,
}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 55) - [public] MutexGuard

```
pub struct MutexGuard<'a, T> {
    lock: &'a Mutex<T>,
}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 61) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 65) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 69) - [internal] drop

```
fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 5) - [public] RwLock

```
pub struct RwLock<T> {
    // -1 = write locked, >0 = number of readers, 0 = unlocked
    state: AtomicIsz,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send + Sync> Sync for RwLock<T> {}
unsafe impl<T: Send> Send for RwLock<T> {}

impl<T> RwLock<T> {
    pub const fn new(data: T) -> Self {
        Self { state: AtomicIsz::new(0), data: UnsafeCell::new(data) }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 19) - [public] read

```
pub fn read(&self) -> RwLockReadGuard<'_, T> {
    loop {
        let val = self.state.load(Ordering::Relaxed);
        if val >= 0 {
            if self.state.compare_exchange_weak(
                val, val + 1, Ordering::Acquire, Ordering::Relaxed
            ).is_ok() {
                return RwLockReadGuard { lock: self };
            }
        }
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 33) - [public] write

```
pub fn write(&self) -> RwLockWriteGuard<'_, T> {
    loop {
        if self.state.compare_exchange_weak(
            0, -1, Ordering::Acquire, Ordering::Relaxed
        ).is_ok() {
            return RwLockWriteGuard { lock: self };
        }
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 45) - [public] RwLockReadGuard

```
pub struct RwLockReadGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> Drop for RwLockReadGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 48) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> Drop for RwLockReadGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 51) - [internal] drop

```
fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    // ... (truncated, see source file for more)
```


kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 54) - [public] RwLockWriteGuard

```
pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 57) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 60) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 63) - [internal] drop

```
fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 3) - [public] Semaphore

```
pub struct Semaphore {
    count: AtomicI32,
}

impl Semaphore {
    pub const fn new(initial: i32) -> Self {
        Self { count: AtomicI32::new(initial) }
    }

    pub fn acquire(&self) {
        loop {
            let val = self.count.load(Ordering::Relaxed);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 12) - [public] acquire

```
pub fn acquire(&self) {
    loop {
        let val = self.count.load(Ordering::Relaxed);
        if val > 0 {
            if self.count.compare_exchange_weak(
                val, val - 1, Ordering::Acquire, Ordering::Relaxed
            ).is_ok() {
                return;
            }
        }
        crate::task::scheduler::yield_now();
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 26) - [public] try_acquire

```
pub fn try_acquire(&self) -> bool {
    let val = self.count.load(Ordering::Relaxed);
    if val > 0 {
        self.count.compare_exchange(
            val, val - 1, Ordering::Acquire, Ordering::Relaxed
        ).is_ok()
    } else {
        false
    }
}

pub fn release(&self) {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 37) - [public] release

```
pub fn release(&self) {
    self.count.fetch_add(1, Ordering::Release);
    // TODO: Wake up one thread from the waitqueue
}
}
```

kernel_workspace/base/kw-base/src/task/context.rs (line 6) - [public] context_switch

```
pub fn context_switch(old_ctx: *mut TaskContext, new_ctx: *mut TaskContext);
}

pub unsafe fn switch(old: &mut TaskContext, new: &TaskContext) {
    context_switch(old, new as *const _ as *mut _);
}
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 6) - [public] ProcessState

```
pub enum ProcessState {
    Created,
    Running,
    Sleeping,
    Zombie,
    Dead,
}

pub struct Process {
    pub pid: Pid,
    pub world_id: WorldId,
    pub state: ProcessState,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/process.rs (line 14) - [public] Process

```
pub struct Process {
    pub pid: Pid,
    pub world_id: WorldId,
    pub state: ProcessState,
    pub cr3_phys: u64,
}

impl Process {
    pub fn new(pid: Pid, world_id: WorldId) -> KResult<Self> {
        let cr3 = pmm::alloc_zero_frame().map_err(KernelError::from)?;
        Ok(Self {
            pid,
            // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/process.rs (line 22) - [public] new

```
pub fn new(pid: Pid, world_id: WorldId) -> KResult<Self> {
    let cr3 = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self {
        pid,
        world_id,
        state: ProcessState::Created,
        cr3_phys: cr3.0,
    })
}

pub fn transition(&mut self, new_state: ProcessState) -> KResult<()> {
    match (self.state, new_state) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 32) - [public] transition

```
pub fn transition(&mut self, new_state: ProcessState) -> KResult<()> {
    match (self.state, new_state) {
        (ProcessState::Created, ProcessState::Running) |
        (ProcessState::Running, ProcessState::Sleeping) |
        (ProcessState::Sleeping, ProcessState::Running) |
        (ProcessState::Running, ProcessState::Zombie) |
        (ProcessState::Zombie, ProcessState::Dead) => {
            self.state = new_state;
            Ok(())
        }
        _ => Err(KernelError::InvalidArg),
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 48) - [internal] drop

```
fn drop(&mut self) {
    if self.cr3_phys != 0 {
        let _ = pmm::free_frame(kstd_base::PhysAddr(self.cr3_phys));
    }
}
}
```

kernel_workspace/base/kw-base/src/task/queue.rs (line 5) - [public] WaitQueue

```
pub struct WaitQueue {
    lock: SpinLock<()>,
    head: ListHead,
}

impl WaitQueue {
    pub const fn new() -> Self {
        Self {
            lock: SpinLock::new(),
            head: ListHead::new(),
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/task/queue.rs (line 18) - [public] init

```
pub fn init(&mut self) {
    self.head.init();
}

pub fn sleep(&self, thread: &mut Thread) {
    let _guard = self.lock.lock();
    thread.state = super::thread::ThreadState::Blocked;
    // In a real implementation, we would link thread's list_node into self.head
    // and then call context::switch() to the scheduler.
}

pub fn wake_one(&self) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 22) - [public] sleep

```
pub fn sleep(&self, thread: &mut Thread) {
    let _guard = self.lock.lock();
    thread.state = super::thread::ThreadState::Blocked;
    // In a real implementation, we would link thread's list_node into self.head
    // and then call context::switch() to the scheduler.
}

pub fn wake_one(&self) {
    let _guard = self.lock.lock();
    if self.head.is_empty() { return; }
    // In a real implementation, we would pop the first thread from self.head,
    // set its state to Ready, and add it to the scheduler runqueue.
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 29) - [public] wake_one

```
pub fn wake_one(&self) {
    let _guard = self.lock.lock();
    if self.head.is_empty() { return; }
    // In a real implementation, we would pop the first thread from self.head,
    // set its state to Ready, and add it to the scheduler runqueue.
}

pub fn wake_all(&self) {
    let _guard = self.lock.lock();
    // Wake all threads in the list
}
}
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 36) - [public] wake_all

```
pub fn wake_all(&self) {
    let _guard = self.lock.lock();
    // Wake all threads in the list
}
}
```

kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 4) - [public] yield_now

```
pub fn yield_now() {
    unsafe {
        core::arch::asm!("int 0x30", options(nomem, nostack)); // Trigger scheduler IPI/interrupt
    }
}

pub fn halt_cpu() {
    loop {
        crate::arch::interrupts::enable();
        crate::arch::interrupts::halt();
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 10) - [public] halt_cpu

```
pub fn halt_cpu() {
    loop {
        crate::arch::interrupts::enable();
        crate::arch::interrupts::halt();
    }
}

pub fn set_current_priority(prio: i32) -> KResult<()> {
    if prio < -20 || prio > 19 {
        return Err(KernelError::InvalidArg);
    }
    // Hook into legacy kernel scheduler via gluecore if needed
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/scheduler.rs (line 17) - [public] set_current_priority

```
pub fn set_current_priority(prio: i32) -> KResult<()> {
    if prio < -20 || prio > 19 {
        return Err(KernelError::InvalidArg);
    }
    // Hook into legacy kernel scheduler via gluecore if needed
    Ok(())
}
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 5) - [public] ThreadState

```
pub enum ThreadState {
    Ready,
    Running,
    Blocked,
    Terminated,
}

pub struct Thread {
    pub tid: Tid,
    pub state: ThreadState,
    pub context: TaskContext,
    pub kernel_stack_top: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 12) - [public] Thread

```
pub struct Thread {
    pub tid: Tid,
    pub state: ThreadState,
    pub context: TaskContext,
    pub kernel_stack_top: u64,
}

impl Thread {
    pub fn new(tid: Tid, entry_point: u64, stack_top: u64) -> Self {
        let mut ctx = TaskContext::default();
        ctx.rip = entry_point;
        ctx.rsp = stack_top;
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 20) - [public] new

```
pub fn new(tid: Tid, entry_point: u64, stack_top: u64) -> Self {
    let mut ctx = TaskContext::default();
    ctx.rip = entry_point;
    ctx.rsp = stack_top;
    ctx.rflags = 0x202; // Interrupts enabled

    Self {
        tid,
        state: ThreadState::Ready,
        context: ctx,
        kernel_stack_top: stack_top,
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/bitmap.rs (line 3) - [public] Bitmap

```
pub struct Bitmap {
    data: *mut u64,
    bits: usize,
    owned: bool,
}

impl Bitmap {
    pub fn new_external(ptr: *mut u64, bits: usize) -> Self {
        Self { data: ptr, bits, owned: false }
    }

    pub fn words_needed(bits: usize) -> usize {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 10) - [public] new_external
    pub fn new_external(ptr: *mut u64, bits: usize) -> Self {
        Self { data: ptr, bits, owned: false }
    }

    pub fn words_needed(bits: usize) -> usize {
        (bits + 63) / 64
    }

    #[inline]
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 14) - [public] words_needed
    pub fn words_needed(bits: usize) -> usize {
        (bits + 63) / 64
    }

    #[inline]
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }

    #[inline]
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 19) - [internal] word_idx
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }

    #[inline]
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }

    pub fn set(&mut self, bit: usize) {
        if bit < self.bits {
            unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 24) - [internal] bit_mask
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }

    pub fn set(&mut self, bit: usize) {
        if bit < self.bits {
            unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
        }
    }

    pub fn clear(&mut self, bit: usize) {
        if bit < self.bits {
            // ... (truncated, see source file for more)
        }
    }

```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 28) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) &= !self.bit_mask(bit); }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 34) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) &= !self.bit_mask(bit); }
    }
}

pub fn test(&self, bit: usize) -> bool {
    if bit >= self.bits { return false; }
    unsafe { (*self.data.add(self.word_idx(bit)) & self.bit_mask(bit)) != 0 }
}

pub fn find_first_zero(&self) -> Option<usize> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 40) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    if bit >= self.bits { return false; }
    unsafe { (*self.data.add(self.word_idx(bit)) & self.bit_mask(bit)) != 0 }
}

pub fn find_first_zero(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != u64::MAX {
            let bit = (!word).trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 45) - [public] find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != u64::MAX {
            let bit = (!word).trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 57) - [public] find_first_set

```
pub fn find_first_set(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != 0 {
            let bit = word.trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 69) - [public] alloc_first_zero

```
pub fn alloc_first_zero(&mut self) -> Option<usize> {
    let idx = self.find_first_zero()?;
    self.set(idx);
    Some(idx)
}

pub fn count_set(&self) -> usize {
    let mut count = 0;
    for i in 0..Self::words_needed(self.bits) {
        count += unsafe { (*self.data.add(i)).count_ones() as usize };
    }
    count
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 75) - [public] count_set

```
pub fn count_set(&self) -> usize {
    let mut count = 0;
    for i in 0..Self::words_needed(self.bits) {
        count += unsafe { (*self.data.add(i)).count_ones() as usize };
    }
    count
}

pub fn count_free(&self) -> usize {
    self.bits - self.count_set()
}

}
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 83) - [public] count_free

```
pub fn count_free(&self) -> usize {
    self.bits - self.count_set()
}

}
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 9) - [internal] Node

```
struct Node<K, V> {
    keys: [mem::MaybeUninit<K>; MAX_KEYS],
    vals: [mem::MaybeUninit<V>; MAX_KEYS],
    children: [*mut Node<K, V>; MAX_KEYS + 1],
    len: usize,
    leaf: bool,
}

impl<K: Ord, V> Node<K, V> {
    fn new_leaf() -> *mut Self {
        let layout = Layout::new(mem::size_of::<Self>(), mem::align_of::<Self>());
        let ptr = KernelHeap.alloc(layout).unwrap() as *mut Self;
        // ... (truncated, see source file for more)
    }
}
```


kernel_workspace/base/kw-base/src/utils/btree.rs (line 18) - [internal] new_leaf

```
fn new_leaf() -> *mut Self {
    let layout = Layout::new(mem::size_of::<Self>(), mem::align_of::<Self>());
    let ptr = KernelHeap.alloc(layout).unwrap() as *mut Self;
    unsafe {
        (*ptr).len = 0;
        (*ptr).leaf = true;
        (*ptr).children = [core::ptr::null_mut(); MAX_KEYS + 1];
    }
    ptr
}

fn new_internal() -> *mut Self {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/btree.rs (line 29) - [internal] new_internal

```
fn new_internal() -> *mut Self {
    let ptr = Self::new_leaf();
    unsafe { (*ptr).leaf = false; }
    ptr
}

pub struct BTreeMap<K, V> {
    root: *mut Node<K, V>,
    len: usize,
}
```

// ... (truncated, see source file for more)

kernel_workspace/base/kw-base/src/utils/btree.rs (line 36) - [public] BTreeMap

```
pub struct BTreeMap<K, V> {
    root: *mut Node<K, V>,
    len: usize,
}

unsafe impl<K: Send, V: Send> Send for BTreeMap<K, V> {}
unsafe impl<K: Sync, V: Sync> Sync for BTreeMap<K, V> {}

impl<K: Ord, V> BTreeMap<K, V> {
    pub fn new() -> Self {
        Self { root: core::ptr::null_mut(), len: 0 }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/btree.rs (line 45) - [public] new

```
pub fn new() -> Self {
    Self { root: core::ptr::null_mut(), len: 0 }
}

pub fn insert(&mut self, key: K, val: V) {
    if self.root.is_null() {
        self.root = Node::new_leaf();
    }

    let r = self.root;
    if unsafe { (*r).len == MAX_KEYS } {
        let s = Node::new_internal();
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 49) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.root.is_null() {
        self.root = Node::new_leaf();
    }

    let r = self.root;
    if unsafe { (*r).len == MAX_KEYS } {
        let s = Node::new_internal();
        unsafe {
            (*s).children[0] = r;
            self.split_child(s, 0);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 69) - [internal] split_child

```
fn split_child(&mut self, parent: *mut Node<K, V>, i: usize) {
    let child = unsafe { (*parent).children[i] };
    let new_node = if unsafe { (*child).leaf } { Node::new_leaf() } else { Node::new_internal() };
    let mid = MIN_KEYS;

    unsafe {
        for j in 0..MIN_KEYS {
            (*new_node).keys[j] = mem::MaybeUninit::new((*child).keys[mid + 1 + j].assume_init_read());
            (*new_node).vals[j] = mem::MaybeUninit::new((*child).vals[mid + 1 + j].assume_init_read());
        }
        if !(*child).leaf {
            for j in 0..B {
                // ... (truncated, see source file for more)
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 102) - [internal] insert_non_full

```
fn insert_non_full(&mut self, node: *mut Node<K, V>, key: K, val: V) {
    let mut i = unsafe { (*node).len };
    unsafe {
        if (*node).leaf {
            while i > 0 && key < (*node).keys[i - 1].assume_init_ref() {
                (*node).keys[i] = mem::MaybeUninit::new((*node).keys[i - 1].assume_init_read());
                (*node).vals[i] = mem::MaybeUninit::new((*node).vals[i - 1].assume_init_read());
                i -= 1;
            }
            (*node).keys[i] = mem::MaybeUninit::new(key);
            (*node).vals[i] = mem::MaybeUninit::new(val);
            (*node).len += 1;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 129) - [public] get

```
pub fn get(&self, key: &K) -> Option<V> {
    if self.root.is_null() { return None; }
    let mut node = self.root;
    loop {
        let n = unsafe { &*node };
        let mut i = 0;
        while i < n.len && *key > *unsafe { n.keys[i].assume_init_ref() } {
            i += 1;
        }
        if i < n.len && *key == *unsafe { n.keys[i].assume_init_ref() } {
            return Some(unsafe { n.vals[i].assume_init_ref() });
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 146) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 147) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 6) - [internal] FnvHasher

```
struct FnvHasher(u64);
impl Hasher for FnvHasher {
    fn finish(&self) -> u64 { self.0 }
    fn write(&mut self, bytes: &[u8]) {
        for &b in bytes {
            self.0 ^= b as u64;
            self.0 = self.0.wrapping_mul(0x1000000001b3);
        }
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 8) - [internal] finish

```
fn finish(&self) -> u64 { self.0 }
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 9) - [internal] write

```
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    h.finish() as usize
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 17) - [internal] hash_key

```
fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    h.finish() as usize
}

enum Slot<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    slots: *mut Slot<K, V>,
    cap: usize,
    len: usize,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 25) - [public] HashMap

```
pub struct HashMap<K, V> {
    slots: *mut Slot<K, V>,
    cap: usize,
    len: usize,
}

unsafe impl<K: Send, V: Send> Send for HashMap<K, V> {}
unsafe impl<K: Sync, V: Sync> Sync for HashMap<K, V> {}

impl<K: Hash + Eq, V> HashMap<K, V> {
    pub fn new() -> Self {
        Self { slots: core::ptr::null_mut(), cap: 0, len: 0 }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 35) - [public] new

```
pub fn new() -> Self {
    Self { slots: core::ptr::null_mut(), cap: 0, len: 0 }
}

fn grow(&mut self, new_cap: usize) {
    let layout = Layout::new(new_cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
    let new_slots = KernelHeap.alloc(layout).unwrap() as *mut Slot<K, V>;
    for i in 0..new_cap {
        unsafe { new_slots.add(i).write(Slot::Empty); }
    }

    let old_slots = self.slots;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 39) - [internal] grow

```
fn grow(&mut self, new_cap: usize) {
    let layout = Layout::new(new_cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
    let new_slots = KernelHeap.alloc(layout).unwrap() as *mut Slot<K, V>;
    for i in 0..new_cap {
        unsafe { new_slots.add(i).write(Slot::Empty); }
    }

    let old_slots = self.slots;
    let old_cap = self.cap;
    self.slots = new_slots;
    self.cap = new_cap;
    self.len = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 64) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.cap == 0 || self.len * 4 >= self.cap * 3 {
        self.grow(if self.cap == 0 { 8 } else { self.cap * 2 });
    }
    let h = hash_key(&key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty | Slot::Tombstone => {
                *slot = Slot::Occupied(key, val);
                self.len += 1;
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 87) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &*self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.cap; }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 101) - [public] get_mut

```
pub fn get_mut(&mut self, key: &K) -> Option<&mut V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.cap; }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 115) - [public] remove

```
pub fn remove(&mut self, key: &K) -> Option<V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, _) if *k == *key => {
                let old = mem::replace(slot, Slot::Tombstone);
                self.len -= 1;
                if let Slot::Occupied(_, v) = old { return Some(v); }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 133) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}

impl<K, V> Drop for HashMap<K, V> {
    fn drop(&mut self) {
        if !self.slots.is_null() {
            for i in 0..self.cap {
                unsafe { self.slots.add(i).drop_in_place(); }
            }
            let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
            KernelHeap.dealloc(self.slots as *mut u8, layout);
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 134) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}

impl<K, V> Drop for HashMap<K, V> {
    fn drop(&mut self) {
        if !self.slots.is_null() {
            for i in 0..self.cap {
                unsafe { self.slots.add(i).drop_in_place(); }
            }
            let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>(),
                KernelHeap.dealloc(self.slots as *mut u8, layout));
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 138) - [internal] drop

```
fn drop(&mut self) {
    if !self.slots.is_null() {
        for i in 0..self.cap {
            unsafe { self.slots.add(i).drop_in_place(); }
        }
        let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>(),
            KernelHeap.dealloc(self.slots as *mut u8, layout));
    }
}
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 1) - [public] RingBuffer

```
pub struct RingBuffer<T, const N: usize> {
    data: [core::mem::MaybeUninit<T>; N],
    head: usize,
    tail: usize,
    count: usize,
}

impl<T, const N: usize> RingBuffer<T, N> {
    pub const fn new() -> Self {
        Self {
            data: unsafe { core::mem::MaybeUninit::uninit().assume_init() },
            head: 0,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 18) - [public] push

```
pub fn push(&mut self, val: T) -> bool {
    if self.count == N { return false; }
    self.data[self.head] = core::mem::MaybeUninit::new(val);
    self.head = (self.head + 1) % N;
    self.count += 1;
    true
}

pub fn pop(&mut self) -> Option<T> {
    if self.count == 0 { return None; }
    let val = unsafe { self.data[self.tail].as_ptr().read() };
    self.tail = (self.tail + 1) % N;
    // ... (truncated, see source file for more)
}
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 26) - [public] pop

```
pub fn pop(&mut self) -> Option<T> {
    if self.count == 0 { return None; }
    let val = unsafe { self.data[self.tail].as_ptr().read() };
    self.tail = (self.tail + 1) % N;
    self.count -= 1;
    Some(val)
}

pub fn peek(&self) -> Option<&T> {
    if self.count == 0 { return None; }
    Some(unsafe { self.data[self.tail].assume_init_ref() })
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 34) - [public] peek

```
pub fn peek(&self) -> Option<T> {
    if self.count == 0 { return None; }
    Some(unsafe { self.data[self.tail].assume_init_ref() })
}

pub fn len(&self) -> usize { self.count }
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 39) - [public] len

```
pub fn len(&self) -> usize { self.count }
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 40) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 41) - [public] is_full

```
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 42) - [public] capacity

```
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 46) - [internal] drop

```
fn drop(&mut self) {
    while self.pop().is_some() {}
}
}
```

kernel_workspace/base/kw-lib/src/kernel/syscall.rs (line 7) - [public] syscall0

```
pub fn syscall0(num: u64) -> i64 {
    let ret: i64;
    unsafe {
        core::arch::asm!(
            "int 0x80",
            in("rax") num,
            out("rax") ret,
            options(nostack, preserves_flags)
        );
    }
    ret
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/kernel/syscall.rs (line 22) - [public] syscall3

```
pub fn syscall3(num: u64, a1: u64, a2: u64, a3: u64) -> i64 {
    let ret: i64;
    unsafe {
        core::arch::asm!(
            "int 0x80",
            in("rax") num,
            in("rdi") a1,
            in("rsi") a2,
            in("rdx") a3,
            out("rax") ret,
            options(nostack, preserves_flags)
        );
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 3) - [public] ticks

// High-level time API for libraries and drivers.

```
pub fn ticks() -> u64 {
    kw_base::hal::timers::get_ticks()
}

pub fn frequency() -> u64 {
    kw_base::hal::timers::get_frequency()
}

pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 7) - [public] frequency

```
pub fn frequency() -> u64 {
    kw_base::hal::timers::get_frequency()
}

pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
    // ... (truncated, see source file for more)
```


kernel_workspace/base/kw-lib/src/kernel/time.rs (line 11) - [public] sleep_ms

```
pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
}
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 15) - [public] sleep_ticks

```
pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
}
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 8) - [internal] SlabPage

```
struct SlabPage {
    next: *mut SlabPage,
    free_head: *mut u8,
    in_use: u32,
}

pub struct SlabCache {
    obj_size: usize,
    align: usize,
    objs_per_page: usize,
    partial_pages: *mut SlabPage,
    full_pages: *mut SlabPage,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 14) - [public] SlabCache

```
pub struct SlabCache {
    obj_size: usize,
    align: usize,
    objs_per_page: usize,
    partial_pages: *mut SlabPage,
    full_pages: *mut SlabPage,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 26) - [public] new

```
pub fn new(obj_size: usize, align: usize) -> Self {
    let actual_size = if obj_size < core::mem::size_of::<mut u8>() {
        core::mem::size_of::<mut u8>()
    } else {
        obj_size
    };

    let aligned_size = (actual_size + align - 1) & !(align - 1);
    let objs_per_page = (4096 - core::mem::size_of::<SlabPage>()) / aligned_size;

    Self {
        obj_size: aligned_size,
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 45) - [internal] alloc_page

```
fn alloc_page(&self) -> KResult<*mut SlabPage> {
    let layout = core::alloc::Layout::from_size_align(4096, 4096).unwrap();
    let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.map_err(|_| KernelError::NoMemory)?;

    let page = ptr as *mut SlabPage;
    unsafe {
        (*page).next = core::ptr::null_mut();
        (*page).in_use = 0;

        // Build free list inside the page
        let data_start = ptr.add(core::mem::size_of::<SlabPage>());
        let mut current = data_start;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 69) - [public] alloc

```
pub fn alloc(&mut self) -> KResult<*mut u8> {
    if self.partial_pages.is_null() {
        let page = self.alloc_page()?;
        unsafe {
            (*page).next = self.partial_pages;
            self.partial_pages = page;
        }
    }

    let page = self.partial_pages;
    let obj = unsafe { (*page).free_head };

    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 99) - [public] free

```
pub fn free(&mut self, obj: *mut u8) {
    // Find the page this object belongs to
    let page_addr = (obj as usize) & !(4096 - 1);
    let page = page_addr as *mut SlabPage;

    unsafe {
        *(obj as *mut *mut u8) = (*page).free_head;
        (*page).free_head = obj;
        (*page).in_use -= 1;

        if (*page).in_use == 0 {
            // TODO: Remove from partial/full list and free page
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 9) - [internal] FreeList

```
struct FreeList {
    head: u64, // Physical address of the first free block, 0 if empty
}

impl FreeList {
    const fn new() -> Self { Self { head: 0 } }
}

pub struct BuddyAllocator {
    base_phys: PhysAddr,
    total_frames: usize,
    free_lists: [FreeList; MAX_ORDER],
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 17) - [public] BuddyAllocator

```
pub struct BuddyAllocator {
    base_phys: PhysAddr,
    total_frames: usize,
    free_lists: [FreeList; MAX_ORDER],
    // Bitmap to track if a block is split or free
    // For simplicity, we use a basic tracking mechanism here.
    // In a full implementation, this would be a per-order bitmap.
    split_bitmap: *mut u64,
}

unsafe impl Send for BuddyAllocator {}
unsafe impl Sync for BuddyAllocator {}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 40) - [public] init

```
pub fn init(&mut self, base: PhysAddr, frames: usize, bitmap_ptr: *mut u64) {
    self.base_phys = base;
    self.total_frames = frames;
    self.split_bitmap = bitmap_ptr;

    // Clear bitmap
    let words = kw_base::utils::Bitmap::words_needed(frames);
    unsafe {
        core::ptr::write_bytes(bitmap_ptr, 0, words * 8);
    }

    // Add all frames to the highest possible order blocks
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 78) - [internal] frame_to_phys

```
fn frame_to_phys(&self, frame: usize) -> PhysAddr {
    PhysAddr(self.base_phys.0 + (frame as u64 * 4096))
}

fn phys_to_frame(&self, phys: PhysAddr) -> usize {
    ((phys.0 - self.base_phys.0) / 4096) as usize
}

fn free_block(&mut self, frame: usize, order: usize) {
    // Mark as not split in bitmap (simplified logic)
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);
    bitmap.clear(frame);
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 82) - [internal] phys_to_frame

```
fn phys_to_frame(&self, phys: PhysAddr) -> usize {
    ((phys.0 - self.base_phys.0) / 4096) as usize
}

fn free_block(&mut self, frame: usize, order: usize) {
    // Mark as not split in bitmap (simplified logic)
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);
    bitmap.clear(frame);

    // Push to free list
    let phys = self.frame_to_phys(frame);
    unsafe {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 86) - [internal] free_block

```
fn free_block(&mut self, frame: usize, order: usize) {  
    // Mark as not split in bitmap (simplified logic)  
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);  
    bitmap.clear(frame);  
  
    // Push to free list  
    let phys = self.frame_to_phys(frame);  
    unsafe {  
        let ptr = phys.0 as *mut u64;  
        *ptr = self.free_lists[order].head;  
    }  
    self.free_lists[order].head = phys.0;  
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 100) - [public] alloc

```
pub fn alloc(&mut self, order: usize) -> KResult<PhysAddr> {  
    if order >= MAX_ORDER { return Err(KernelError::InvalidArg); }  
  
    // Find the smallest available order >= requested  
    let mut current_order = order;  
    while current_order < MAX_ORDER && self.free_lists[current_order].head == 0 {  
        current_order += 1;  
    }  
  
    if current_order == MAX_ORDER {  
        return Err(KernelError::NoMemory);  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 133) - [public] free

```
pub fn free(&mut self, phys: PhysAddr, order: usize) {  
    let mut frame = self.phys_to_frame(phys);  
    let mut current_order = order;  
  
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);  
  
    // Merge with buddy if possible  
    while current_order < MAX_ORDER - 1 {  
        let buddy_frame = frame ^ (1 << current_order);  
        if buddy_frame >= self.total_frames { break; }  
  
        // Check if buddy is free and of the same order  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 7) - [public] AddressSpace

```
pub struct AddressSpace {  
    pub cr3_phys: u64,  
    vma_head: *mut Vma,  
    vma_count: usize,  
}  
  
unsafe impl Send for AddressSpace {}  
unsafe impl Sync for AddressSpace {}  
  
impl AddressSpace {  
    pub fn new(cr3: u64) -> Self {  
        Self {  
            // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 17) - [public] new

```
pub fn new(cr3: u64) -> Self {
    Self {
        cr3_phys: cr3,
        vma_head: core::ptr::null_mut(),
        vma_count: 0,
    }
}

pub fn map_region(&mut self, start: VirtAddr, size: usize, flags: u32, vma_type: VmaType) -> KResult<Vma> {
    let end = VirtAddr(start.0 + size as u64);

    // Check for overlaps
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 25) - [public] map_region

```
pub fn map_region(&mut self, start: VirtAddr, size: usize, flags: u32, vma_type: VmaType) -> KResult<Vma> {
    let end = VirtAddr(start.0 + size as u64);

    // Check for overlaps
    let mut current = self.vma_head;
    while !current.is_null() {
        let vma = unsafe { &*current };
        if vma.overlaps(start, end) {
            return Err(KernelError::AlreadyExists);
        }
        current = vma.next;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 53) - [public] find_vma

```
pub fn find_vma(&self, addr: VirtAddr) -> Option<&Vma> {
    let mut current = self.vma_head;
    while !current.is_null() {
        let vma = unsafe { &*current };
        if vma.contains(addr) {
            return Some(vma);
        }
        current = vma.next;
    }
    None
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 67) - [internal] drop

```
fn drop(&mut self) {
    let mut current = self.vma_head;
    while !current.is_null() {
        let next = unsafe { (*current).next };
        let layout = core::alloc::Layout::new::(<Vma>());
        unsafe { kstd_alloc::KernelHeap.dealloc(current as *mut u8, layout); }
        current = next;
    }
}
}
```

kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 6) - [public] VmaType

```
pub enum VmaType {
    Anonymous,
    FileBacked,
    DeviceMapped,
    Stack,
    Heap,
}

pub struct Vma {
    pub start: VirtAddr,
    pub end: VirtAddr,
    pub flags: u32, // PROT_READ | PROT_WRITE | PROT_EXEC
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-libs/src/mem/virtual/vma.rs (line 14) - [public] Vma

```
pub struct Vma {
    pub start: VirtAddr,
    pub end: VirtAddr,
    pub flags: u32, // PROT_READ | PROT_WRITE | PROT_EXEC
    pub vma_type: VmaType,
    pub file_offset: u64, // If FileBacked
    pub next: *mut Vma,    // For linked list in AddressSpace
}

impl Vma {
    pub fn new(start: VirtAddr, end: VirtAddr, flags: u32, vma_type: VmaType) -> Self {
        Self {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-libs/src/mem/virtual/vma.rs (line 24) - [public] new

```
pub fn new(start: VirtAddr, end: VirtAddr, flags: u32, vma_type: VmaType) -> Self {
    Self {
        start,
        end,
        flags,
        vma_type,
        file_offset: 0,
        next: core::ptr::null_mut(),
    }
}

pub fn size(&self) -> usize {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-libs/src/mem/virtual/vma.rs (line 35) - [public] size

```
pub fn size(&self) -> usize {
    (self.end.0 - self.start.0) as usize
}

pub fn contains(&self, addr: VirtAddr) -> bool {
    addr.0 >= self.start.0 && addr.0 < self.end.0
}

pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-libs/src/mem/virtual/vma.rs (line 39) - [public] contains

```
pub fn contains(&self, addr: VirtAddr) -> bool {
    addr.0 >= self.start.0 && addr.0 < self.end.0
}

pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-libs/src/mem/virtual/vma.rs (line 43) - [public] overlaps

```
pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 20) - [public] StrBuffer

// Simple fixed-capacity string buffer for formatting paths/logs without heap allocation.

```
pub struct StrBuffer<const N: usize> {
    buf: [u8; N],
    len: usize,
}

impl<const N: usize> StrBuffer<N> {
    pub const fn new() -> Self {
        Self { buf: [0; N], len: 0 }
    }

    pub fn push_str(&mut self, s: &str) -> bool {
        let bytes = s.as_bytes();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 30) - [public] push_str

```
pub fn push_str(&mut self, s: &str) -> bool {
    let bytes = s.as_bytes();
    if self.len + bytes.len() > N { return false; }
    self.buf[self.len..self.len + bytes.len()].copy_from_slice(bytes);
    self.len += bytes.len();
    true
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(&self.buf[..self.len]) }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 38) - [public] as_str

```
pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(&self.buf[..self.len]) }
}

pub fn as_cstr_ptr(&mut self) -> *const u8 {
    if self.len < N {
        self.buf[self.len] = 0;
    }
    self.buf.as_ptr()
}

}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 42) - [public] as_cstr_ptr

```
pub fn as_cstr_ptr(&mut self) -> *const u8 {
    if self.len < N {
        self.buf[self.len] = 0;
    }
    self.buf.as_ptr()
}

}
```

kernel_workspace/base/kw-odin-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-odin-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {  
    let result = add(2, 2);  
    assert_eq!(result, 4);  
}
```

kernel_workspace/core/gluecore/build.rs (line 1) - [internal] main

```
fn main() {}
```

kernel_workspace/core/gluecore/src/bindings.rs (line 6) - [public] CPhysRange

```
pub struct CPhysRange {  
    pub start: u64,  
    pub end: u64,  
}  
  
impl From<CPhysRange> for (PhysAddr, usize) {  
    fn from(val: CPhysRange) -> Self {  
        (PhysAddr(val.start), (val.end - val.start) as usize)  
    }  
}
```

kernel_workspace/core/gluecore/src/bindings.rs (line 12) - [internal] from

```
fn from(val: CPhysRange) -> Self {  
    (PhysAddr(val.start), (val.end - val.start) as usize)  
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 4) - [internal] boot_get_memory_map

```
fn boot_get_memory_map(out_ptr: *mut u64, out_size: *mut usize) -> bool;  
fn boot_get_cmdline(out_ptr: *mut u64, out_len: *mut usize) -> bool;  
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) -> bool;  
  
pub struct BootMemoryMap {  
    pub ptr: u64,  
    pub size: usize,  
}  
  
pub fn get_memory_map() -> Option<BootMemoryMap> {  
    let mut ptr: u64 = 0;  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/boot.rs (line 5) - [internal] boot_get_cmdline

```
fn boot_get_cmdline(out_ptr: *mut u64, out_len: *mut usize) -> bool;  
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) -> bool;  
  
pub struct BootMemoryMap {  
    pub ptr: u64,  
    pub size: usize,  
}  
  
pub fn get_memory_map() -> Option<BootMemoryMap> {  
    let mut ptr: u64 = 0;  
    let mut size: usize = 0;  
    // ... (truncated, see source file for more)
```


kernel_workspace/core/gluecore/src/boot.rs (line 6) - [internal] boot_get_framebuffer

```
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) ->
}

pub struct BootMemoryMap {
    pub ptr: u64,
    pub size: usize,
}

pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        // ... (truncated, see source file for more)
    }
```

kernel_workspace/core/gluecore/src/boot.rs (line 9) - [public] BootMemoryMap

```
pub struct BootMemoryMap {
    pub ptr: u64,
    pub size: usize,
}

pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        Some(BootMemoryMap { ptr, size })
    } else {
        None
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 14) - [public] get_memory_map

```
pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        Some(BootMemoryMap { ptr, size })
    } else {
        None
    }
}

pub struct BootFramebuffer {
    pub phys: PhysAddr,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 24) - [public] BootFramebuffer

```
pub struct BootFramebuffer {
    pub phys: PhysAddr,
    pub width: u32,
    pub height: u32,
    pub pitch: u32,
}

pub fn get_framebuffer() -> Option<BootFramebuffer> {
    let mut phys: u64 = 0;
    let mut w: u32 = 0;
    let mut h: u32 = 0;
    let mut pitch: u32 = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 31) - [public] get_framebuffer

```
pub fn get_framebuffer() -> Option<BootFramebuffer> {
    let mut phys: u64 = 0;
    let mut w: u32 = 0;
    let mut h: u32 = 0;
    let mut pitch: u32 = 0;
    if unsafe { boot_get_framebuffer(&mut phys, &mut w, &mut h, &mut pitch) } {
        Some(BootFramebuffer { phys: PhysAddr(phys), width: w, height: h, pitch })
    } else {
        None
    }
}
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 6) - [public] has_cap

```
pub fn has_cap(world_id: u32, cap: CapId) -> bool {
    unsafe { extern_block::caps_has(world_id, cap.0 as u8) == 1 }
}

pub fn request_cap(target_world: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_request(target_world, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::PermissionDenied)
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 10) - [public] request_cap

```
pub fn request_cap(target_world: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_request(target_world, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::PermissionDenied)
    }
}

pub fn release_cap(world_id: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_release(world_id, cap.0 as u8) };
    if rc == 0 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 19) - [public] release_cap

```
pub fn release_cap(world_id: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_release(world_id, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 7) - [public] alloc_frame

/// Allocates a single physical frame (4KB).

```
pub fn alloc_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frame(&mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Allocates a single zeroed physical frame.

```
pub fn alloc_zero_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 17) - [public] alloc_zero_frame

/// Allocates a single zeroed physical frame.

```
pub fn alloc_zero_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_zero_frame(&mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Allocates contiguous physical frames.
pub fn alloc_frames(count: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 27) - [public] alloc_frames

/// Allocates contiguous physical frames.

```
pub fn alloc_frames(count: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frames(count, &mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Allocates contiguous physical frames with a specific alignment (in frames).
pub fn alloc_frames_aligned(count: usize, align_frames: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 37) - [public] alloc_frames_aligned

/// Allocates contiguous physical frames with a specific alignment (in frames).

```
pub fn alloc_frames_aligned(count: usize, align_frames: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frames_aligned(count, align_frames, &mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Frees a single physical frame.
pub fn free_frame(addr: PhysAddr) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frame(addr.0) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 47) - [public] free_frame

/// Frees a single physical frame.

```
pub fn free_frame(addr: PhysAddr) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frame(addr.0) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

/// Frees contiguous physical frames.
pub fn free_frames(addr: PhysAddr, count: usize) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frames(addr.0, count) } {
        Ok(())
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 56) - [public] free_frames

/// Frees contiguous physical frames.

```
pub fn free_frames(addr: PhysAddr, count: usize) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frames(addr.0, count) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 14) - [public] alloc

/// Allocates virtual memory and maps it to physical frames.

```
pub fn alloc(bytes: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_alloc(bytes, flags, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Allocates virtual memory with a specific alignment.

```
pub fn alloc_aligned(bytes: usize, align: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 24) - [public] alloc_aligned

/// Allocates virtual memory with a specific alignment.

```
pub fn alloc_aligned(bytes: usize, align: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_alloc_aligned(bytes, align, flags, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Frees previously allocated virtual memory.

```
pub fn free(addr: VirtAddr, bytes: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_free(addr.0, bytes) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 34) - [public] free

/// Frees previously allocated virtual memory.

```
pub fn free(addr: VirtAddr, bytes: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_free(addr.0, bytes) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

/// Maps a physical device region (MMIO) into the kernel virtual space.

```
pub fn map_device(phys: PhysAddr, len: usize) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_map_device(phys.0, len, &mut virt) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 43) - [public] map_device

/// Maps a physical device region (MMIO) into the kernel virtual space.

```
pub fn map_device(phys: PhysAddr, len: usize) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_map_device(phys.0, len, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::BadAddress)
    }
}

/// Unmaps a device region.
pub fn unmap_device(addr: VirtAddr, len: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_unmap_device(addr.0, len) } {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 53) - [public] unmap_device

/// Unmaps a device region.

```
pub fn unmap_device(addr: VirtAddr, len: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_unmap_device(addr.0, len) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

/// Translates a kernel virtual address to a physical address.
pub fn virt_to_phys(vaddr: VirtAddr) -> PhysAddr {
    let ptr = vaddr.0 as *const u8;
    PhysAddr(unsafe { extern_block::kvirt_to_phys(ptr) })
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 62) - [public] virt_to_phys

/// Translates a kernel virtual address to a physical address.

```
pub fn virt_to_phys(vaddr: VirtAddr) -> PhysAddr {
    let ptr = vaddr.0 as *const u8;
    PhysAddr(unsafe { extern_block::kvirt_to_phys(ptr) })
}
```

kernel_workspace/core/gluecore/src/context.rs (line 4) - [internal] paging_read_cr3

```
fn paging_read_cr3() -> u64;
fn paging_write_cr3(pml4_phys: u64);
fn paging_flush_tlb_all();
fn paging_flush_page(addr: u64);
fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/context.rs (line 5) - [internal] paging_write_cr3

```
fn paging_write_cr3(pml4_phys: u64);
fn paging_flush_tlb_all();
fn paging_flush_page(addr: u64);
fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
    // ... (truncated, see source file for more)
}
```

```

kernel_Workspace/core/gluecore/src/context.rs (line 6) - [internal] paging_flush_tlb_all
    fn paging_flush_tlb_all();
    fn paging_flush_page(addr: u64);
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 7) - [internal] paging_flush_page
    fn paging_flush_page(addr: u64);
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 8) - [internal]
context_switch_save_restore
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    // ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 11) - [public] read_cr3
pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

// ... (truncated, see source file for more)

```

kernel_workspace/core/gluecore/src/context.rs (line 15) - [public] write_cr3

```
pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/context.rs (line 19) - [public] flush_tlb_all

```
pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

pub unsafe fn switch_context(old_ctx: *mut u8, new_ctx: *mut u8) {
    context_switch_save_restore(old_ctx, new_ctx);
}
```

kernel_workspace/core/gluecore/src/context.rs (line 23) - [public] flush_page

```
pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

pub unsafe fn switch_context(old_ctx: *mut u8, new_ctx: *mut u8) {
    context_switch_save_restore(old_ctx, new_ctx);
}
```

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 8) - [public] pmm_alloc_frame

// --- Physical Memory Manager (PMM) ---

```
pub fn pmm_alloc_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_zero_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 9) - [public]

pmm_alloc_zero_frame

```
pub fn pmm_alloc_zero_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
// ... (truncated, see source file for more)
```

```

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 10) - [public] pmm_alloc_frames
    pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
    pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
    pub fn pmm_free_frame(phys: u64) -> bool;
    pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

    // --- Virtual Memory Manager (VMM) ---
    pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_free(virt: u64, bytes: usize) -> bool;
    pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
    pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

    // ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 11) - [public]
pmm_alloc_frames_aligned
    pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
    pub fn pmm_free_frame(phys: u64) -> bool;
    pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

    // --- Virtual Memory Manager (VMM) ---
    pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_free(virt: u64, bytes: usize) -> bool;
    pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
    pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

    // --- Kernel Heap ---
    // ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 12) - [public] pmm_free_frame
    pub fn pmm_free_frame(phys: u64) -> bool;
    pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

    // --- Virtual Memory Manager (VMM) ---
    pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_free(virt: u64, bytes: usize) -> bool;
    pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
    pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

    // --- Kernel Heap ---
    pub fn kmalloc(size: usize) -> *mut u8;
    // ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 13) - [public] pmm_free_frames
    pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

    // --- Virtual Memory Manager (VMM) ---
    pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
    pub fn vmm_free(virt: u64, bytes: usize) -> bool;
    pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
    pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

    // --- Kernel Heap ---
    pub fn kmalloc(size: usize) -> *mut u8;
    pub fn kzalloc(size: usize) -> *mut u8;
    // ... (truncated, see source file for more)

```


kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 16) - [public] vmm_alloc

// --- Virtual Memory Manager (VMM) ---

```
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 17) - [public]

vmm_alloc_aligned

```
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 18) - [public] vmm_free

```
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 19) - [public] vmm_map_device

```
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;
```

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 20) - [public] vmm_unmap_device

```
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// --- Kernel Heap ---
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 23) - [public] kmalloc

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 24) - [public] kzalloc

```
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 25) - [public] krealloc

```
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 26) - [public] kfree

```
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 27) - [public] kalloc_pages

```
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 28) - [public] kfree_pages

```
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 29) - [public] kvirt_to_phys

```
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 32) - [public] paging_read_cr3

```
// --- Paging / Address Spaces ---

pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 33) - [public] paging_write_cr3
    pub fn paging_write_cr3(pml4_phys: u64);
    pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 34) - [public] paging_map_page
    pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 35) - [public]
paging_unmap_page
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 36) - [public]
paging_flush_tlb_all
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 37) - [public]
paging_flush_page
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/interrupts.rs (line 4) - [public] disable
    pub fn disable() {
        unsafe { asm!("cli", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn enable() {
        unsafe { asm!("sti", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn halt() {
        unsafe { asm!("hlt", options(nomem, nostack)); }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/core/gluecore/src/interrupts.rs (line 9) - [public] enable
    pub fn enable() {
        unsafe { asm!("sti", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn halt() {
        unsafe { asm!("hlt", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn are_enabled() -> bool {
        let flags: u64;
        // ... (truncated, see source file for more)
    }

```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 14) - [public] halt

```
pub fn halt() {
    unsafe { asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    let flags: u64;
    unsafe { asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
    (flags & (1 << 9)) != 0
}

pub struct InterruptGuard {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 19) - [public] are_enabled

```
pub fn are_enabled() -> bool {
    let flags: u64;
    unsafe { asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
    (flags & (1 << 9)) != 0
}

pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 25) - [public] InterruptGuard

```
pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        let was_enabled = are_enabled();
        disable();
        Self { was_enabled }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 30) - [public] disable

```
pub fn disable() -> Self {
    let was_enabled = are_enabled();
    disable();
    Self { was_enabled }
}

impl Drop for InterruptGuard {
    fn drop(&mut self) {
        if self.was_enabled {
            enable();
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 38) - [internal] drop

```
fn drop(&mut self) {
    if self.was_enabled {
        enable();
    }
}
```

```
kernel_Workspace/core/gluecore/src/syscalls.rs (line 4) - [internal] syscall_dispatch
    fn syscall_dispatch(num: u64, arg1: u64, arg2: u64, arg3: u64, arg4: u64) -> i64;
}

pub fn invoke(num: u64, args: [u64; 4]) -> Result<u64, Status> {
    let ret = unsafe {
        syscall_dispatch(num, args[0], args[1], args[2], args[3])
    };

    if ret < 0 {
        Err(Status::from((-ret) as u32))
    } else {
        Ok(ret as u64)
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_Workspace/core/gluecore/src/syscalls.rs (line 7) - [public] invoke
pub fn invoke(num: u64, args: [u64; 4]) -> Result<u64, Status> {
    let ret = unsafe {
        syscall_dispatch(num, args[0], args[1], args[2], args[3])
    };

    if ret < 0 {
        Err(Status::from((-ret) as u32))
    } else {
        Ok(ret as u64)
    }
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 5) - [internal] kmalloc
    fn kmalloc(size: usize) -> *mut u8;
    fn kcalloc(size: usize) -> *mut u8;
    fn kcalloc(n: usize, size: usize) -> *mut u8;
    fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
    fn kfree(ptr: *mut u8);
    fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 6) - [internal] kcalloc
    fn kcalloc(size: usize) -> *mut u8;
    fn kcalloc(n: usize, size: usize) -> *mut u8;
    fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
    fn kfree(ptr: *mut u8);
    fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 7) - [internal] kcalloc

```
fn kcalloc(n: usize, size: usize) -> *mut u8;
fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 8) - [internal] krealloc

```
fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 9) - [internal] kfree

```
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 10) - [internal] kmalloc_aligned

```
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        };
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 13) - [public] KernelHeap

```
pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        };
        if ptr.is_null() {
            Err(Status::OutOfMemory)
        } else {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 16) - [internal] alloc

```
fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
    let ptr = if layout.align() > 8 {
        unsafe { kmalloc_aligned(layout.size(), layout.align()) }
    } else {
        unsafe { kmalloc(layout.size()) }
    };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(ptr)
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 29) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { kfree(ptr) };
    }
}

fn realloc(&self, ptr: *mut u8, _old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = unsafe { krealloc(ptr, new_size) };
    if new_ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(new_ptr)
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 35) - [internal] realloc

```
fn realloc(&self, ptr: *mut u8, _old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = unsafe { krealloc(ptr, new_size) };
    if new_ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(new_ptr)
    }
}

pub fn alloc_zeroed(size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kzalloc(size) };
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 45) - [public] alloc_zeroed

```
pub fn alloc_zeroed(size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kzalloc(size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}

pub fn alloc_array(n: usize, elem_size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kcalloc(n, elem_size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}
```



```
kernel_workspace/core/kstd_alloc/src/heap.rs (line 50) - [public] alloc_array
pub fn alloc_array(n: usize, elem_size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kcalloc(n, elem_size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}
```

```
kernel_workspace/core/kstd_alloc/src/lib.rs (line 15) - [internal] oom
fn oom() -> ! {
    loop {
        core::hint::spin_loop();
    }
}
```

```
pub struct KernelAllocator;

unsafe impl GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: CoreLayout) -> *mut u8 {
        let l = Layout::new(layout.size(), layout.align());
        match KernelHeap.alloc(l) {
            // ... (truncated, see source file for more)
        }
    }
}
```

```
kernel_workspace/core/kstd_alloc/src/lib.rs (line 21) - [public] KernelAllocator
pub struct KernelAllocator;

unsafe impl GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: CoreLayout) -> *mut u8 {
        let l = Layout::new(layout.size(), layout.align());
        match KernelHeap.alloc(l) {
            Ok(ptr) => ptr,
            Err(_) => oom(),
        }
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: CoreLayout) {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_workspace/core/kstd_alloc/src/omm.rs (line 5) - [public] OomHandler
pub trait OomHandler: Fn() -> ! + Send + Sync {}
impl<T: Fn() -> ! + Send + Sync> OomHandler for T {}

static mut HANDLER: Option<&'static dyn OomHandler> = None;

pub fn set_handler(handler: &'static dyn OomHandler) {
    unsafe { HANDLER = Some(handler) };
}

pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/core/kstd_alloc/src/omm.rs (line 10) - [public] set_handler
pub fn set_handler(handler: &'static dyn OomHandler) {
    unsafe { HANDLER = Some(handler) };
}

pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    unsafe {
        if let Some(h) = HANDLER {
            h();
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_alloc/src/omm.rs (line 14) - [public] invoke

```
pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    unsafe {
        if let Some(h) = HANDLER {
            h();
        }
    }
    panic!("OOM");
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 5) - [internal] kalloc_pages

```
fn kalloc_pages(order: usize) -> *mut u8;
fn kfree_pages(ptr: *mut u8, pages: usize);

pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 6) - [internal] kfree_pages

```
fn kfree_pages(ptr: *mut u8, pages: usize);

pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 9) - [public] alloc_pages

```
pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

pub fn free_pages(addr: VirtAddr, order: usize) {
    let pages = 1 << order;
    unsafe { kfree_pages(addr.0 as *mut u8, pages) };
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 18) - [public] free_pages

```
pub fn free_pages(addr: VirtAddr, order: usize) {
    let pages = 1 << order;
    unsafe { kfree_pages(addr.0 as *mut u8, pages) };
}

pub fn alloc_single_page() -> Result<VirtAddr, Status> {
    alloc_pages(0)
}

pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 23) - [public] alloc_single_page

```
pub fn alloc_single_page() -> Result<VirtAddr, Status> {
    alloc_pages(0)
}

pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 27) - [public] free_single_page

```
pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 31) - [public] pages_for_size

```
pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/slab.rs (line 5) - [internal] slab_alloc

```
fn slab_alloc(cache: *mut u8) -> *mut u8;
fn slab_free(cache: *mut u8, ptr: *mut u8);
fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/slab.rs (line 6) - [internal] slab_free

```
fn slab_free(cache: *mut u8, ptr: *mut u8);
fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 7) - [internal] slab_cache_create
    fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
    fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 8) - [internal] slab_cache_destroy
    fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 11) - [public] SlabCache
pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 19) - [public] new
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            Err(Status::OutOfMemory)
        } else {
            Ok(Self { inner: cache })
        }
    }
}

impl Allocator for SlabCache {
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 31) - [internal] alloc

```
fn alloc(&self, _layout: Layout) -> Result<*mut u8, Status> {
    let ptr = unsafe { slab_alloc(self.inner) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}

fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr) };
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 36) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr) };
    }
}

impl Drop for SlabCache {
    fn drop(&mut self) {
        if !self.inner.is_null() {
            unsafe { slab_cache_destroy(self.inner) };
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 44) - [internal] drop

```
fn drop(&mut self) {
    if !self.inner.is_null() {
        unsafe { slab_cache_destroy(self.inner) };
    }
}

}
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 3) - [public] Allocator

```
pub trait Allocator {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status>;
    fn dealloc(&self, ptr: *mut u8, layout: Layout);
    fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
        let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
        if !ptr.is_null() {
            unsafe {
                kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
            }
            self.dealloc(ptr, old);
        }
        Ok(new_ptr)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 4) - [internal] alloc

```
fn alloc(&self, layout: Layout) -> Result<*mut u8, Status>;
fn dealloc(&self, ptr: *mut u8, layout: Layout);
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 5) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, layout: Layout);
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 6) - [internal] realloc

```
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 19) - [public] Layout

```
pub struct Layout {
    size: usize,
    align: usize,
}

impl Layout {
    pub const fn new(size: usize, align: usize) -> Self {
        Self { size, align }
    }

    pub const fn from_size(size: usize) -> Self {
        Self { size, align: 8 }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 41) - [public] padded

```
pub fn padded(&self) -> usize {
    (self.size + self.align - 1) & !(self.align - 1)
}
}
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 3) - [public] Status

```
pub enum Status {
    Success = 0,
    OutOfMemory = 1,
    InvalidArgument = 2,
    PermissionDenied = 3,
    NotFound = 4,
    AlreadyExists = 5,
    Busy = 6,
    TimedOut = 7,
    NotSupported = 8,
    IOError = 9,
    BadAddress = 10,
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 20) - [internal] from

```
fn from(val: u32) -> Self {
    match val {
        0 => Self::Success,
        1 => Self::OutOfMemory,
        2 => Self::InvalidArgument,
        3 => Self::PermissionDenied,
        4 => Self::NotFound,
        5 => Self::AlreadyExists,
        6 => Self::Busy,
        7 => Self::TimedOut,
        8 => Self::NotSupported,
        9 => Self::IOError,
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 40) - [internal] from

```
fn from(val: Status) -> Self {
    val as u32
}

impl Status {
    #[inline]
    pub const fn is_ok(self) -> bool {
        matches!(self, Self::Success)
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/intrinsics.rs (line 12) - [public] memory_barrier

```
pub fn memory_barrier() {
    core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::SeqCst);
}

#[inline(always)]
pub unsafe fn memcpy(dst: *mut u8, src: *const u8, n: usize) {
    core::ptr::copy_nonoverlapping(src, dst, n);
}

#[inline(always)]
pub unsafe fn memset(dst: *mut u8, val: u8, n: usize) {
    core::ptr::write_bytes(dst, val, n);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/markers.rs (line 3) - [public] SendSync

```
pub struct SendSync<T>(PhantomData<T>);
unsafe impl<T> Send for SendSync<T> {}
unsafe impl<T> Sync for SendSync<T> {}

pub struct Immutable<T>(PhantomData<T>);
impl<T> Immutable<T> {
    pub const fn new() -> Self {
        Self(PhantomData)
    }
}

pub struct Unpin<T>(PhantomData<T>);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/markers.rs (line 7) - [public] Immutable

```
pub struct Immutable<T>(PhantomData<T>);
impl<T> Immutable<T> {
    pub const fn new() -> Self {
        Self(PhantomData)
    }
}

pub struct Unpin<T>(PhantomData<T>);
impl<T> core::marker::Unpin for Unpin<T> {}
```

kernel_workspace/core/kstd_base/src/markers.rs (line 14) - [public] Unpin

```
pub struct Unpin<T>(PhantomData<T>);
impl<T> core::marker::Unpin for Unpin<T> {}
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 7) - [public] Ring

```
pub enum Ring {
    Kernel = 0,
    Driver = 1,
    User    = 3,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
pub struct ProtFlags(pub u32);

impl ProtFlags {
    pub const NONE: Self = Self(0);
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 15) - [public] ProtFlags

```
pub struct ProtFlags(pub u32);

impl ProtFlags {
    pub const NONE: Self = Self(0);
    pub const READ: Self = Self(1 << 0);
    pub const WRITE: Self = Self(1 << 1);
    pub const EXEC: Self = Self(1 << 2);
    pub const USER: Self = Self(1 << 3);
    pub const DEVICE: Self = Self(1 << 4);

    #[inline]
    pub const fn contains(self, other: Self) -> bool {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 39) - [internal] bitor

```
fn bitor(self, rhs: Self) -> Self {
    self.union(rhs)
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
pub struct MapFlags(pub u32);

impl MapFlags {
    pub const NONE: Self = Self(0);
    pub const FIXED: Self = Self(1 << 0);
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 46) - [public] MapFlags

```
pub struct MapFlags(pub u32);

impl MapFlags {
    pub const NONE: Self = Self(0);
    pub const FIXED: Self = Self(1 << 0);
    pub const ANONYMOUS: Self = Self(1 << 1);
    pub const SHARED: Self = Self(1 << 2);
    pub const PRIVATE: Self = Self(1 << 3);
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(transparent)]
// ... (truncated, see source file for more)
```


kernel_workspace/core/kstd_base/src/primitives.rs (line 58) - [public] IrqFlags

```
pub struct IrqFlags(pub u32);

impl IrqFlags {
    pub const NONE: Self = Self(0);
    pub const LEVEL_TRIGGERED: Self = Self(1 << 0);
    pub const ACTIVE_LOW: Self = Self(1 << 1);
    pub const SHARED: Self = Self(1 << 2);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 3) - [public] PageAlign

```
pub trait PageAlign {
    fn align_up(self) -> Self;
    fn align_down(self) -> Self;
    fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 4) - [internal] align_up

```
fn align_up(self) -> Self;
fn align_down(self) -> Self;
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 5) - [internal] align_down

```
fn align_down(self) -> Self;
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    fn align_down(self) -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 6) - [internal] is_aligned

```
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    fn align_down(self) -> Self {
        self & !(PAGE_SIZE as u64 - 1)
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 11) - [internal] align_up

```
fn align_up(self) -> Self {
    (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn align_down(self) -> Self {
    self & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 16) - [internal] align_down

```
fn align_down(self) -> Self {
    self & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
}

impl PageAlign for usize {
    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 21) - [internal] is_aligned

```
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
}

impl PageAlign for usize {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE - 1) & !(PAGE_SIZE - 1)
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 28) - [internal] align_up

```
fn align_up(self) -> Self {
    (self + PAGE_SIZE - 1) & !(PAGE_SIZE - 1)
}

#[inline]
fn align_down(self) -> Self {
    self & !(PAGE_SIZE - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 33) - [internal] align_down

```
fn align_down(self) -> Self {
    self & !(PAGE_SIZE - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}

pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 38) - [internal] is_aligned

```
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}

pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    fn write_u8(&self, port: u16, val: u8);
    fn read_u16(&self, port: u16) -> u16;
    fn write_u16(&self, port: u16, val: u16);
    fn read_u32(&self, port: u16) -> u32;
    fn write_u32(&self, port: u16, val: u32);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 43) - [public] IoPort

```
pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    fn write_u8(&self, port: u16, val: u8);
    fn read_u16(&self, port: u16) -> u16;
    fn write_u16(&self, port: u16, val: u16);
    fn read_u32(&self, port: u16) -> u32;
    fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 44) - [internal] read_u8

```
fn read_u8(&self, port: u16) -> u8;
fn write_u8(&self, port: u16, val: u8);
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 45) - [internal] write_u8

```
fn write_u8(&self, port: u16, val: u8);
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 46) - [internal] read_u16

```
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 47) - [internal] write_u16

```
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 48) - [internal] read_u32

```
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 49) - [internal] write_u32

```
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 52) - [public] MmioRegion

```
pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 53) - [internal] read_volatile_u32

```
fn read_volatile_u32(&self, offset: usize) -> u32;
fn write_volatile_u32(&self, offset: usize, val: u32);
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 54) - [internal] write_volatile_u32

```
fn write_volatile_u32(&self, offset: usize, val: u32);
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 55) - [internal] read_volatile_u64

```
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 56) - [internal] write_volatile_u64

```
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/types.rs (line 3) - [public] PhysAddr

```
pub struct PhysAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct VirtAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
#[repr(transparent)]
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/types.rs (line 7) - [public] VirtAddr

```
pub struct VirtAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
#[repr(transparent)]
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 != Self::INVALID.0
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_base/src/types.rs (line 11) - [public] Handle

```
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 != Self::INVALID.0
    }
}

impl Default for Handle {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 23) - [internal] default

```
fn default() -> Self {
    Self::INVALID
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct WorldId(pub u32);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 30) - [public] CapId

```
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct WorldId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 34) - [public] WorldId

```
pub struct WorldId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/types.rs (line 38) - [public] TaskId

```
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    pub const fn is_aligned_to(self, align: u64) -> bool {
        self.0 & (align - 1) == 0
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/types.rs (line 42) - [public] Size

```
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    pub const fn is_aligned_to(self, align: u64) -> bool {
        self.0 & (align - 1) == 0
    }

    #[inline]
    pub const fn page_offset(self) -> u64 {
        self.0 & 0xFFF
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/panic.rs (line 5) - [internal] write_all

```
fn write_all(serial: &mut kstd_io::Serial, vga: &mut kstd_io::VgaConsole, s: &str) {
    let _ = serial.write_str(s);
    let _ = vga.write_str(s);
}
```

```
pub fn panic(info: &PanicInfo) -> ! {
    let mut serial = kstd_io::Serial;
    let mut vga = kstd_io::VgaConsole;

    write_all(&mut serial, &mut vga, "\n!!! KERNEL PANIC !!!\n");

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/panic.rs (line 11) - [public] panic

```
pub fn panic(info: &PanicInfo) -> ! {
    let mut serial = kstd_io::Serial;
    let mut vga = kstd_io::VgaConsole;

    write_all(&mut serial, &mut vga, "\n!!! KERNEL PANIC !!!\n");

    if let Some(location) = info.location() {
        let _ = core::write!(FmtWriter(&mut serial), "At {}:{}:{}\n", location.file(), location.line(), 1);
        let _ = core::write!(FmtWriter(&mut vga), "At {}:{}:{}\n", location.file(), location.line(), loca
    }

    {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 34) - [internal] FmtWriter

```
struct FmtWriter<'a, W: Write>(&'a mut W);

impl<'a, W: Write> core::fmt::Write for FmtWriter<'a, W> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        let _ = self.0.write_str(s);
        Ok(())
    }
}

// Render the panic message into a fixed buffer without using alloc.
// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 37) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> core::fmt::Result {
    let _ = self.0.write_str(s);
    Ok(())
}

// Render the panic message into a fixed buffer without using alloc.
// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    use core::fmt::Write as _;
    let mut s = kstd_data::string::String::new();
    let _ = core::write!(FmtMsg(&mut s), "{}", message);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 45) - [internal] heapless_msg

// Render the panic message into a fixed buffer without using alloc.

// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.

```
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    use core::fmt::Write as _;
    let mut s = kstd_data::string::String::new();
    let _ = core::write!(FmtMsg(&mut s), "{}", message);
    s
}

struct FmtMsg<'a>(&'a mut kstd_data::string::String);

impl<'a> core::fmt::Write for FmtMsg<'a> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        self.0.push_str(s);
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 52) - [internal] FmtMsg

```
struct FmtMsg<'a>(&'a mut kstd_data::string::String);

impl<'a> core::fmt::Write for FmtMsg<'a> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        self.0.push_str(s);
        Ok(())
    }
}
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 55) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> core::fmt::Result {
    self.0.push_str(s);
    Ok(())
}
```


kernel_workspace/core/kstd_core/src/sync.rs (line 5) - [public] SpinLock

```
pub struct SpinLock<T> {
    locked: AtomicBool,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for SpinLock<T> {}
unsafe impl<T: Send> Send for SpinLock<T> {}

impl<T> SpinLock<T> {
    pub const fn new(data: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/core/kstd_core/src/sync.rs (line 21) - [public] lock

```
pub fn lock(&self) -> SpinLockGuard<'_, T> {
    while self.locked.compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed).is_err() {
        while self.locked.load(Ordering::Relaxed) {
            core::hint::spin_loop();
        }
    }
    SpinLockGuard { lock: self }
}

pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
    if self.locked.compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed).is_ok() {
        Some(SpinLockGuard { lock: self })
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_core/src/sync.rs (line 30) - [public] try_lock

```
pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
    if self.locked.compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed).is_ok() {
        Some(SpinLockGuard { lock: self })
    } else {
        None
    }
}

pub struct SpinLockGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/sync.rs (line 39) - [public] SpinLockGuard

```
pub struct SpinLockGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> Deref for SpinLockGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.lock.data.get() }
    }
}

impl<T> DerefMut for SpinLockGuard<'_, T> {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 45) - [internal] deref

```
    fn deref(&self) -> &T {
        unsafe { &*self.lock.data.get() }
    }
}

impl<T> DerefMut for SpinLockGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }
    }
}

impl<T> Drop for SpinLockGuard<'_, T> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 51) - [internal] deref_mut

```
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }
    }
}

impl<T> Drop for SpinLockGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 57) - [internal] drop

```
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 4) - [internal] sched_create_task

```
    fn sched_create_task(world_id: u32, flags: u32, out_task_id: *mut u64) -> bool;
    fn sched_destroy_task(task_id: u64) -> bool;
    fn sched_yield() -> bool;
    fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/task.rs (line 5) - [internal] sched_destroy_task

```
    fn sched_destroy_task(task_id: u64) -> bool;
    fn sched_yield() -> bool;
    fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/task.rs (line 6) - [internal] sched_yield

```
fn sched_yield() -> bool;
fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 7) - [internal] sched_get_current_task

```
fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        if unsafe { sched_create_task(world.0, flags, &mut id) } {
            Ok(Self { id: TaskId(id), world })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 10) - [public] Task

```
pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        if unsafe { sched_create_task(world.0, flags, &mut id) } {
            Ok(Self { id: TaskId(id), world })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 16) - [public] create

```
pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
    let mut id: u64 = 0;
    if unsafe { sched_create_task(world.0, flags, &mut id) } {
        Ok(Self { id: TaskId(id), world })
    } else {
        Err(Status::OutOfMemory)
    }
}

pub fn destroy(&self) -> Result<(), Status> {
    if unsafe { sched_destroy_task(self.id.0) } {
        Ok(())
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_core/src/task.rs (line 25) - [public] destroy

```
pub fn destroy(&self) -> Result<(), Status> {
    if unsafe { sched_destroy_task(self.id.0) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

pub fn current() -> TaskId {
    TaskId(unsafe { sched_get_current_task() })
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/task.rs (line 33) - [public] current

```
pub fn current() -> TaskId {
    TaskId(unsafe { sched_get_current_task() })
}

pub fn yield_now() {
    unsafe { sched_yield(); }
}
}
```

kernel_workspace/core/kstd_core/src/task.rs (line 37) - [public] yield_now

```
pub fn yield_now() {
    unsafe { sched_yield(); }
}
}
```

kernel_workspace/core/kstd_core/src/thread.rs (line 4) - [internal] thread_create

```
fn thread_create(task_id: u64, entry: u64, stack_ptr: u64, arg: u64, out_tid: *mut u64) -> bool;
fn thread_exit() -> !;
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_core/src/thread.rs (line 5) - [internal] thread_exit

```
fn thread_exit() -> !;
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 6) - [internal] thread_get_current_id

```
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 9) - [public] Thread

```
pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
            Ok(Self { id: tid, task })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 15) - [public] spawn

```
pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
    let mut tid: u64 = 0;
    if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
        Ok(Self { id: tid, task })
    } else {
        Err(Status::OutOfMemory)
    }
}

pub fn exit() -> ! {
    unsafe { thread_exit(); }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 24) - [public] exit

```
pub fn exit() -> ! {
    unsafe { thread_exit(); }
}

pub fn current_id() -> u64 {
    unsafe { thread_get_current_id() }
}
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 28) - [public] current_id

```
pub fn current_id() -> u64 {
    unsafe { thread_get_current_id() }
}
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 2) - [internal] k_tick

```
fn k_tick() -> u64;
}

#[inline(always)]
pub fn kernel_tick() -> u64 {
    unsafe { k_tick() }
}

#[inline(always)]
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 6) - [public] kernel_tick

```
pub fn kernel_tick() -> u64 {
    unsafe { k_tick() }
}

#[inline(always)]
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::x86_64::_rdtsc()
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 11) - [public] rdtsc

```
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::x86_64::_rdtsc()
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        kernel_tick()
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 4) - [public] ListHead

```
pub struct ListHead {
    next: *mut ListHead,
    prev: *mut ListHead,
}

impl ListHead {
    pub const fn new() -> Self {
        Self { next: ptr::null_mut(), prev: ptr::null_mut() }
    }

    pub fn init(&mut self) {
        self.next = self as *mut _;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 14) - [public] init

```
pub fn init(&mut self) {
    self.next = self as *mut _;
    self.prev = self as *mut _;
}

pub fn add(&mut self, new_node: *mut ListHead) {
    unsafe {
        let next = self.next;
        (*new_node).next = next;
        (*new_node).prev = self as *mut _;
        (*next).prev = new_node;
        self.next = new_node;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 19) - [public] add

```
pub fn add(&mut self, new_node: *mut ListHead) {
    unsafe {
        let next = self.next;
        (*new_node).next = next;
        (*new_node).prev = self as *mut _;
        (*next).prev = new_node;
        self.next = new_node;
    }
}

pub fn add_tail(&mut self, new_node: *mut ListHead) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 29) - [public] add_tail

```
pub fn add_tail(&mut self, new_node: *mut ListHead) {
    unsafe {
        let prev = self.prev;
        (*new_node).next = self as *mut _;
        (*new_node).prev = prev;
        (*prev).next = new_node;
        self.prev = new_node;
    }
}

pub fn del(&mut self) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 39) - [public] del

```
pub fn del(&mut self) {
    unsafe {
        let prev = self.prev;
        let next = self.next;
        (*prev).next = next;
        (*next).prev = prev;
        self.init();
    }
}

pub fn is_empty(&self) -> bool {
    self.next as *const _ == self as *const _
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 49) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.next as *const _ == self as *const _
}

#[macro_export]
macro_rules! list_entry {
    ($ptr:expr, $ty:ty, $field:ident) => {
        ($ptr as usize - $crate::offset_of!($ty, $field)) as *mut $ty
    };
}
```

kernel_Workspace/core/kstd_data/src/map.rs (line 4) - [internal] FnvHasher

```
struct FnvHasher(u64);
impl Hasher for FnvHasher {
    fn finish(&self) -> u64 { self.0 }
    fn write(&mut self, bytes: &[u8]) {
        for &b in bytes {
            self.0 ^= b as u64;
            self.0 = self.0.wrapping_mul(0x1000000001b3);
        }
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 6) - [internal] finish

```
fn finish(&self) -> u64 { self.0 }
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 7) - [internal] write

```
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    buckets: Vec<Bucket<K, V>>,
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 17) - [public] HashMap

```
pub struct HashMap<K, V> {
    buckets: Vec<Bucket<K, V>>,
    len: usize,
}

impl<K: Hash + Eq, V> HashMap<K, V> {
    pub fn new() -> Self {
        Self { buckets: Vec::new(), len: 0 }
    }

    fn hash_key(key: &K) -> usize {
        let mut hasher = FnvHasher(0xcbf29ce484222325);
        // ... (truncated, see source file for more)
```


kernel_Workspace/core/kstd_data/src/map.rs (line 23) - [public] new

```
pub fn new() -> Self {
    Self { buckets: Vec::new(), len: 0 }
}

fn hash_key(key: &K) -> usize {
    let mut hasher = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut hasher);
    hasher.finish() as usize
}

fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 27) - [internal] hash_key

```
fn hash_key(key: &K) -> usize {
    let mut hasher = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut hasher);
    hasher.finish() as usize
}

fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 33) - [internal] idx

```
fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }

    let hash = Self::hash_key(&key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
// ... (truncated, see source file for more)

```

kernel_Workspace/core/kstd_data/src/map.rs (line 35) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }

    let hash = Self::hash_key(&key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty | Bucket::Tombstone => {
                self.buckets[idx] = Bucket::Occupied(key, val);
            }
// ... (truncated, see source file for more)

```

kernel_Workspace/core/kstd_data/src/map.rs (line 59) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    if self.buckets.is_empty() { return None; }
    let hash = Self::hash_key(key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty => return None,
            Bucket::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.buckets.len(); }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 73) - [public] remove

```
pub fn remove(&mut self, key: &K) -> Option<V> {
    if self.buckets.is_empty() { return None; }
    let hash = Self::hash_key(key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty => return None,
            Bucket::Occupied(k, _) if *k == *key => {
                let old = core::mem::replace(&mut self.buckets[idx], Bucket::Tombstone);
                self.len -= 1;
                if let Bucket::Occupied(_, v) = old { return Some(v); }
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 91) - [internal] resize

```
fn resize(&mut self, new_cap: usize) {
    let old_buckets = core::mem::replace(&mut self.buckets, Vec::with_capacity(new_cap));
    for _ in 0..new_cap { self.buckets.push(Bucket::Empty); }

    self.len = 0;
    for b in old_buckets.as_slice() {
        if let Bucket::Occupied(k, v) = b {
            let k = unsafe { core::ptr::read(k) };
            let v = unsafe { core::ptr::read(v) };
            self.insert(k, v);
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 107) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/core/kstd_data/src/map.rs (line 108) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 4) - [public] String

```
pub struct String { vec: Vec<u8> }

impl String {
    pub const fn new() -> Self { Self { vec: Vec::new() } }

    pub fn from_str(s: &str) -> Self {
        let mut v = Vec::with_capacity(s.len());
        v.extend_from_slice(s.as_bytes());
        Self { vec: v }
    }

    pub fn push_str(&mut self, s: &str) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 9) - [public] from_str

```
pub fn from_str(s: &str) -> Self {
    let mut v = Vec::with_capacity(s.len());
    v.extend_from_slice(s.as_bytes());
    Self { vec: v }
}

pub fn push_str(&mut self, s: &str) {
    self.vec.extend_from_slice(s.as_bytes());
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 15) - [public] push_str

```
pub fn push_str(&mut self, s: &str) {
    self.vec.extend_from_slice(s.as_bytes());
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}

pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 19) - [public] as_str

```
pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}

pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

impl core::ops::Deref for String {
    type Target = str;
    fn deref(&self) -> &str { self.as_str() }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 23) - [public] len

```
pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

impl core::ops::Deref for String {
    type Target = str;
    fn deref(&self) -> &str { self.as_str() }
}

impl fmt::Write for String {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.push_str(s);
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 24) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

impl core::ops::Deref for String {
    type Target = str;
    fn deref(&self) -> &str { self.as_str() }
}

impl fmt::Write for String {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.push_str(s);
        Ok(())
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 29) - [internal] deref

```
fn deref(&self) -> &str { self.as_str() }
}

impl fmt::Write for String {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.push_str(s);
        Ok(())
    }
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 33) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> fmt::Result {
    self.push_str(s);
    Ok(())
}
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 3) - [internal] Node

```
struct Node<K, V> {
    key: K, val: V,
    left: usize, right: usize,
    height: i32,
}

pub struct TreeMap<K, V> {
    nodes: Vec<Node<K, V>>,
    free_list: Vec<usize>,
    root: usize,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 9) - [public] TreeMap

```
pub struct TreeMap<K, V> {
    nodes: Vec<Node<K, V>>,
    free_list: Vec<usize>,
    root: usize,
}

impl<K: Ord, V> TreeMap<K, V> {
    pub fn new() -> Self {
        Self { nodes: Vec::new(), free_list: Vec::new(), root: 0 }
    }

    fn alloc_node(&mut self, key: K, val: V) -> usize {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 16) - [public] new

```
pub fn new() -> Self {
    Self { nodes: Vec::new(), free_list: Vec::new(), root: 0 }
}

fn alloc_node(&mut self, key: K, val: V) -> usize {
    if let Some(idx) = self.free_list.pop() {
        self.nodes[idx - 1] = Node { key, val, left: 0, right: 0, height: 1 };
        idx
    } else {
        self.nodes.push(Node { key, val, left: 0, right: 0, height: 1 });
        self.nodes.len()
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 20) - [internal] alloc_node

```
fn alloc_node(&mut self, key: K, val: V) -> usize {
    if let Some(idx) = self.free_list.pop() {
        self.nodes[idx - 1] = Node { key, val, left: 0, right: 0, height: 1 };
        idx
    } else {
        self.nodes.push(Node { key, val, left: 0, right: 0, height: 1 });
        self.nodes.len()
    }
}

fn free_node(&mut self, idx: usize) {
    self.free_list.push(idx);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 30) - [internal] free_node

```
fn free_node(&mut self, idx: usize) {
    self.free_list.push(idx);
}

fn height(&self, idx: usize) -> i32 {
    if idx == 0 { 0 } else { self.nodes[idx - 1].height }
}

fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 34) - [internal] height

```
fn height(&self, idx: usize) -> i32 {
    if idx == 0 { 0 } else { self.nodes[idx - 1].height }
}

fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}

fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 38) - [internal] update_height

```
fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}

fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
    else { self.height(self.nodes[idx - 1].left) - self.height(self.nodes[idx - 1].right) }
}

fn rotate_right(&mut self, y: usize) -> usize {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 44) - [internal] balance_factor

```
fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
    else { self.height(self.nodes[idx - 1].left) - self.height(self.nodes[idx - 1].right) }
}

fn rotate_right(&mut self, y: usize) -> usize {
    let x = self.nodes[y - 1].left;
    let t2 = self.nodes[x - 1].right;
    self.nodes[x - 1].right = y;
    self.nodes[y - 1].left = t2;
    self.update_height(y);
    self.update_height(x);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 49) - [internal] rotate_right

```
fn rotate_right(&mut self, y: usize) -> usize {
    let x = self.nodes[y - 1].left;
    let t2 = self.nodes[x - 1].right;
    self.nodes[x - 1].right = y;
    self.nodes[y - 1].left = t2;
    self.update_height(y);
    self.update_height(x);
    x
}

fn rotate_left(&mut self, x: usize) -> usize {
    let y = self.nodes[x - 1].right;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 59) - [internal] rotate_left

```
fn rotate_left(&mut self, x: usize) -> usize {
    let y = self.nodes[x - 1].right;
    let t2 = self.nodes[y - 1].left;
    self.nodes[y - 1].left = x;
    self.nodes[x - 1].right = t2;
    self.update_height(x);
    self.update_height(y);
    y
}

fn balance(&mut self, idx: usize) -> usize {
    self.update_height(idx);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 69) - [internal] balance

```
fn balance(&mut self, idx: usize) -> usize {
    self.update_height(idx);
    let bf = self.balance_factor(idx);
    if bf > 1 {
        if self.balance_factor(self.nodes[idx - 1].left) < 0 {
            let l = self.nodes[idx - 1].left;
            self.nodes[idx - 1].left = self.rotate_left(l);
        }
        return self.rotate_right(idx);
    }
    if bf < -1 {
        if self.balance_factor(self.nodes[idx - 1].right) > 0 {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 89) - [internal] insert_rec

```
fn insert_rec(&mut self, idx: usize, key: K, val: V) -> usize {
    if idx == 0 { return self.alloc_node(key, val); }

    let i = idx - 1;
    if key < self.nodes[i].key {
        self.nodes[i].left = self.insert_rec(self.nodes[i].left, key, val);
    } else if key > self.nodes[i].key {
        self.nodes[i].right = self.insert_rec(self.nodes[i].right, key, val);
    } else {
        self.nodes[i].val = val;
        return idx;
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 104) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    self.root = self.insert_rec(self.root, key, val);
}

pub fn get(&self, key: &K) -> Option<&V> {
    let mut idx = self.root;
    while idx != 0 {
        let i = idx - 1;
        if *key < self.nodes[i].key { idx = self.nodes[i].left; }
        else if *key > self.nodes[i].key { idx = self.nodes[i].right; }
        else { return Some(&self.nodes[i].val); }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 108) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    let mut idx = self.root;
    while idx != 0 {
        let i = idx - 1;
        if *key < self.nodes[i].key { idx = self.nodes[i].left; }
        else if *key > self.nodes[i].key { idx = self.nodes[i].right; }
        else { return Some(&self.nodes[i].val); }
    }
    None
}

pub fn len(&self) -> usize { self.nodes.len() - self.free_list.len() }
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 119) - [public] len

```
pub fn len(&self) -> usize { self.nodes.len() - self.free_list.len() }
pub fn is_empty(&self) -> bool { self.root == 0 }
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 120) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.root == 0 }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 4) - [public] Vec

```
pub struct Vec<T> {
    ptr: NonNull<T>,
    len: usize,
    cap: usize,
}

unsafe impl<T: Send> Send for Vec<T> {}
unsafe impl<T: Sync> Sync for Vec<T> {}

impl<T> Vec<T> {
    pub const fn new() -> Self {
        Self { ptr: NonNull::dangling(), len: 0, cap: 0 }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 18) - [public] with_capacity

```
pub fn with_capacity(cap: usize) -> Self {
    let mut v = Self::new();
    if cap > 0 { v.grow_to(cap); }
    v
}

fn grow_to(&mut self, new_cap: usize) {
    let elem_size = mem::size_of::<T>();
    let align = mem::align_of::<T>();
    let new_ptr = if self.cap == 0 {
        KernelHeap.alloc(Layout::new(new_cap * elem_size, align)).unwrap() as *mut T
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 24) - [internal] grow_to

```
fn grow_to(&mut self, new_cap: usize) {
    let elem_size = mem::size_of::<T>();
    let align = mem::align_of::<T>();
    let new_ptr = if self.cap == 0 {
        KernelHeap.alloc(Layout::new(new_cap * elem_size, align)).unwrap() as *mut T
    } else {
        KernelHeap.realloc(
            self.ptr.as_ptr() as *mut u8,
            Layout::new(self.cap * elem_size, align),
            new_cap * elem_size
        ).unwrap() as *mut T
    };
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 40) - [public] push

```
pub fn push(&mut self, val: T) {
    if self.len == self.cap {
        let new_cap = if self.cap == 0 { 4 } else { self.cap * 2 };
        self.grow_to(new_cap);
    }
    unsafe { ptr::write(self.ptr.as_ptr().add(self.len), val); }
    self.len += 1;
}

pub fn pop(&mut self) -> Option<T> {
    if self.len == 0 { None }
    else {
        // ... (truncated, see source file for more)
    }
}
```


kernel_Workspace/core/kstd_data/src/vec.rs (line 49) - [public] pop

```
pub fn pop(&mut self) -> Option<T> {
    if self.len == 0 { None }
    else {
        self.len -= 1;
        Some(unsafe { ptr::read(self.ptr.as_ptr().add(self.len)) })
    }
}

pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 57) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 58) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 59) - [public] capacity

```
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_data/src/vec.rs (line 61) - [public] as_slice

```
pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    pub fn extend_from_slice(&mut self, slice: &[T]) {
        let new_len = self.len + slice.len();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_data/src/vec.rs (line 65) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    pub fn extend_from_slice(&mut self, slice: &[T]) {
        let new_len = self.len + slice.len();
        if new_len > self.cap {
            let mut new_cap = if self.cap == 0 { 4 } else { self.cap };
            while new_cap < new_len { new_cap *= 2; }
            self.grow_to(new_cap);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_data/src/vec.rs (line 71) - [public] extend_from_slice

```
pub fn extend_from_slice(&mut self, slice: &[T]) {
    let new_len = self.len + slice.len();
    if new_len > self.cap {
        let mut new_cap = if self.cap == 0 { 4 } else { self.cap };
        while new_cap < new_len { new_cap *= 2; }
        self.grow_to(new_cap);
    }
    unsafe {
        ptr::copy_nonoverlapping(slice.as_ptr(), self.ptr.as_ptr().add(self.len), slice.len());
    }
    self.len = new_len;
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_data/src/vec.rs (line 87) - [internal] deref

```
fn deref(&self) -> &[T] { self.as_slice() }

impl<T> DerefMut for Vec<T> {
    fn deref_mut(&mut self) -> &mut [T] { self.as_mut_slice() }
}

impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.cap > 0 {
            unsafe {
                for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 91) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut [T] { self.as_mut_slice() }
}

impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.cap > 0 {
            unsafe {
                for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
                let layout = Layout::new(self.cap * mem::size_of::<T>(), mem::align_of::<T>());
                KernelHeap.dealloc(self.ptr.as_ptr() as *mut u8, layout);
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 95) - [internal] drop

```
fn drop(&mut self) {
    if self.cap > 0 {
        unsafe {
            for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
            let layout = Layout::new(self.cap * mem::size_of::<T>(), mem::align_of::<T>());
            KernelHeap.dealloc(self.ptr.as_ptr() as *mut u8, layout);
        }
    }
}
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 1) - [public] RingBuffer

```
pub struct RingBuffer<const N: usize> {
    data: [u8; N],
    head: usize,
    tail: usize,
}

impl<const N: usize> RingBuffer<N> {
    pub const fn new() -> Self {
        Self { data: [0; N], head: 0, tail: 0 }
    }

    pub fn push(&mut self, b: u8) -> bool {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 12) - [public] push

```
pub fn push(&mut self, b: u8) -> bool {
    let next_head = (self.head + 1) % N;
    if next_head == self.tail { return false; }
    self.data[self.head] = b;
    self.head = next_head;
    true
}

pub fn pop(&mut self) -> Option<u8> {
    if self.head == self.tail { return None; }
    let b = self.data[self.tail];
    self.tail = (self.tail + 1) % N;
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 20) - [public] pop

```
pub fn pop(&mut self) -> Option<u8> {
    if self.head == self.tail { return None; }
    let b = self.data[self.tail];
    self.tail = (self.tail + 1) % N;
    Some(b)
}

pub fn is_empty(&self) -> bool { self.head == self.tail }
pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/core/kstd_io/src/buffer.rs (line 27) - [public] is_empty
    pub fn is_empty(&self) -> bool { self.head == self.tail }
    pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
    pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}
```

```
impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
// ... (truncated, see source file for more)
```

```
kernel_workspace/core/kstd_io/src/buffer.rs (line 28) - [public] is_full
    pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
    pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}
```

```
impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
```

```
kernel_workspace/core/kstd_io/src/buffer.rs (line 29) - [public] clear
    pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}
```

```
impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
```

```
kernel_workspace/core/kstd_io/src/buffer.rs (line 33) - [internal] write_str
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
```

```
kernel_workspace/core/kstd_io/src/console.rs (line 4) - [internal] kprintf
    fn kprintf(fmt: *const u8, ...);
    fn serial_write_str(s: *const u8);
}
```

```
pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 5) - [internal] serial_write_str

```
fn serial_write_str(s: *const u8);
}

pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 8) - [public] Serial

```
pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
            null_terminated[len] = 0;
            serial_write_str(null_terminated.as_ptr());
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 10) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    let bytes = s.as_bytes();
    if bytes.is_empty() { return Ok(()); }
    unsafe {
        let mut null_terminated = [0u8; 4096];
        let len = bytes.len().min(4095);
        core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
        null_terminated[len] = 0;
        serial_write_str(null_terminated.as_ptr());
    }
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/console.rs (line 24) - [public] VgaConsole

```
pub struct VgaConsole;
impl Write for VgaConsole {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let fmt = b"%s\0".as_ptr();
        let mut null_terminated = [0u8; 4096];
        let len = s.as_bytes().len().min(4095);
        unsafe {
            core::ptr::copy_nonoverlapping(s.as_ptr(), null_terminated.as_mut_ptr(), len);
            null_terminated[len] = 0;
            kprintf(fmt, null_terminated.as_ptr());
        }
        Ok(())
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 26) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    let fmt = b"%s\0".as_ptr();
    let mut null_terminated = [0u8; 4096];
    let len = s.as_bytes().len().min(4095);
    unsafe {
        core::ptr::copy_nonoverlapping(s.as_ptr(), null_terminated.as_mut_ptr(), len);
        null_terminated[len] = 0;
        kprintf(fmt, null_terminated.as_ptr());
    }
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 2) - [internal] k_input_key

```
fn k_input_key() -> i32;
fn k_input_keycode() -> u32;
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 3) - [internal] k_input_keycode

```
fn k_input_keycode() -> u32;
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 4) - [internal] kbuf_pop

```
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 7) - [public] raw_scancode

```
pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
}

pub fn buffered_byte() -> Option<u8> {
    let b = unsafe { kbuf_pop() };
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 12) - [public] cooked_char

```
pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
}

pub fn buffered_byte() -> Option<u8> {
    let b = unsafe { kbuf_pop() };
    if b < 0 { None } else { Some(b as u8) }
}

pub struct RawInput;
impl crate::traits::Read for RawInput {
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 17) - [public] buffered_byte

```
pub fn buffered_byte() -> Option<u8> {
    let b = unsafe { kbuf_pop() };
    if b < 0 { None } else { Some(b as u8) }
}

pub struct RawInput;
impl crate::traits::Read for RawInput {
    fn read_byte(&mut self) -> Option<u8> {
        raw_scancode().map(|c| c as u8)
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 22) - [public] RawInput

```
pub struct RawInput;
impl crate::traits::Read for RawInput {
    fn read_byte(&mut self) -> Option<u8> {
        raw_scancode().map(|c| c as u8)
    }
}

pub struct CookedInput;
impl crate::traits::Read for CookedInput {
    fn read_byte(&mut self) -> Option<u8> {
        cooked_char()
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 24) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8> {
    raw_scancode().map(|c| c as u8)
}

pub struct CookedInput;
impl crate::traits::Read for CookedInput {
    fn read_byte(&mut self) -> Option<u8> {
        cooked_char()
    }
}
```

kernel_workspace/core/kstd_io/src/input.rs (line 29) - [public] CookedInput

```
pub struct CookedInput;
impl crate::traits::Read for CookedInput {
    fn read_byte(&mut self) -> Option<u8> {
        cooked_char()
    }
}
```

kernel_workspace/core/kstd_io/src/input.rs (line 31) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8> {
    cooked_char()
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 1) - [public] Write

```
pub trait Write {
    fn write_str(&mut self, s: &str) -> Result<(), ()>;

    fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
        let mut buf = [0u8; 64];
        let mut i = 64;
        if val == 0 {
            i -= 1;
            buf[i] = b'0';
        } else {
            while val > 0 {
                i -= 1;
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 2) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()>;

fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
    let mut buf = [0u8; 64];
    let mut i = 64;
    if val == 0 {
        i -= 1;
        buf[i] = b'0';
    } else {
        while val > 0 {
            i -= 1;
            let rem = (val % base as u64) as u8;
            buf[i] = if rem < 10 { b'0' + rem } else { b'a' + rem - 10 };
            val /= base as u64;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 4) - [internal] write_u64

```
fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
    let mut buf = [0u8; 64];
    let mut i = 64;
    if val == 0 {
        i -= 1;
        buf[i] = b'0';
    } else {
        while val > 0 {
            i -= 1;
            let rem = (val % base as u64) as u8;
            buf[i] = if rem < 10 { b'0' + rem } else { b'a' + rem - 10 };
            val /= base as u64;
        }
        // ... (truncated, see source file for more)
    }
}
```


kernel_workspace/core/kstd_io/src/traits.rs (line 22) - [public] Read

```
pub trait Read {  
    fn read_byte(&mut self) -> Option<u8>;  
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 23) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8>;  
}
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 14) - [public] PartitionKind

```
pub enum PartitionKind {  
    KernelCore    = 0,  
    DriverSpace   = 1,  
    UserSpace     = 3,  
    LinuxCompat   = 4,  
    WindowsCompat = 5,  
}  
  
#[derive(Debug, Clone, Copy, PartialEq, Eq)]  
#[repr(u8)]  
pub enum AbiFlavor {  
    NativeTrangorge,  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 24) - [public] AbiFlavor

```
pub enum AbiFlavor {  
    NativeTrangorge,  
    PosixLinux,  
    Win32Nt,  
    DriverSpaceRpc,  
}  
  
#[derive(Debug, Clone, Copy)]  
pub struct SandboxRules {  
    pub virt_start: VirtAddr,  
    pub virt_end: VirtAddr,  
    pub phys_base: PhysAddr,  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 32) - [public] SandboxRules

```
pub struct SandboxRules {  
    pub virt_start: VirtAddr,  
    pub virt_end: VirtAddr,  
    pub phys_base: PhysAddr,  
    pub mem_size: Size,  
    pub max_ipc_ports: u32,  
    pub max_open_handles: u32,  
    pub allowed_irq_mask: u64,  
    pub can_access_hw_direct: bool,  
    pub can_map_arbitrary_phys: bool,  
}  
  
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 53) - [public] OsWorkspace

// Represents an isolated execution environment.

// Binds a World/Process to its memory sandbox, capabilities, and ABI translation hooks.

```
pub struct OsWorkspace {  
    pub id: WorldId,  
    pub kind: PartitionKind,  
    pub abi: AbiFlavor,  
    pub asid: u16,  
    pub cr3_phys: u64,  
    pub rules: SandboxRules,  
    pub syscall_entry: u64,  
    pub abi_translate_hook: u64,  
}  
  
impl OsWorkspace {  
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 81) - [public] check_memory_access

```
pub fn check_memory_access(&self, addr: VirtAddr, size: usize) -> bool {
    let end = addr.0.saturating_add(size as u64);
    end <= self.rules.virt_end.0 && addr.0 >= self.rules.virt_start.0
}
```

kernel_Workspace/core/linux_abi/src/caps.rs (line 15) - [public] requires_native_cap

```
pub fn requires_native_cap(linux_cap: u32) -> Option<kstd_base::CapId> {
    match linux_cap {
        CAP_SYS_ADMIN | CAP_SYS_MODULE => Some(kstd_base::CapId(0x01)), // Native Root/Kernel
        CAP_NET_RAW | CAP_NET_BIND_SERVICE => Some(kstd_base::CapId(0x10)), // Native Net
        CAP_SYS_RAWIO => Some(kstd_base::CapId(0x20)), // Native HW/IO
        _ => None, // Mapped via standard user sandbox rules
    }
}
```

kernel_Workspace/core/linux_abi/src/errcodes.rs (line 25) - [public] status_to_errno

```
pub fn status_to_errno(s: Status) -> u32 {
    match s {
        Status::Success => 0,
        Status::OutOfMemory => ENOMEM,
        Status::InvalidArgument => EINVAL,
        Status::PermissionDenied => EACCES,
        Status::NotFound => ENOENT,
        Status::AlreadyExists => EEXIST,
        Status::Busy => EBUSY,
        Status::TimedOut => EAGAIN,
        Status::NotSupported => ENOSYS,
        Status::IOError => EIO,
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/linux_abi/src/errcodes.rs (line 44) - [public] errno_to_status

```
pub fn errno_to_status(e: u32) -> Status {
    match e {
        0 => Status::Success,
        ENOMEM => Status::OutOfMemory,
        EINVAL => Status::InvalidArgument,
        EACCES | EPERM => Status::PermissionDenied,
        ENOENT => Status::NotFound,
        EEXIST => Status::AlreadyExists,
        EBUSY => Status::Busy,
        EAGAIN => Status::TimedOut,
        ENOSYS => Status::NotSupported,
        EIO => Status::IOError,
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/linux_abi/src/ipc.rs (line 4) - [public] IpcPerm

```
pub struct IpcPerm {
    pub key: u32,
    pub uid: u32,
    pub gid: u32,
    pub cuid: u32,
    pub cgid: u32,
    pub mode: u16,
    pub __seq: u16,
}

#[repr(C)]
#[derive(Debug, Clone, Copy)]
// ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi/src/ipc.rs (line 16) - [public] ShmidDs

```
pub struct ShmidDs {
    pub shm_perm: IpcPerm,
    pub shm_segsz: usize,
    pub shm_atime: u64,
    pub shm_dtime: u64,
    pub shm_ctime: u64,
    pub shm_cpid: u32,
    pub shm_lpid: u32,
    pub shm_nattch: u64,
}
```

kernel_workspace/core/linux_abi/src/sysnums.rs (line 25) - [public] map_syscall_to_native_op

```
pub fn map_syscall_to_native_op(sys_num: u64) -> Option<u32> {
    match sys_num {
        SYS_READ | SYS_WRITE => Some(0x1001), // Native FS/IO opcode
        SYS_MMAP | SYS_MUNMAP | SYS_BRK => Some(0x2001), // Native MM opcode
        SYS_GETPID => Some(0x3001), // Native Proc opcode
        SYS_EXIT | SYS_EXIT_GROUP => Some(0x3002),
        SYS_SOCKET | SYS_CONNECT => Some(0x4001), // Native Net opcode
        _ => None, // Unhandled, requires full POSIX shim
    }
}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 4) - [public] UserContext

```
pub struct UserContext {
    pub r15: u64, pub r14: u64, pub r13: u64, pub r12: u64,
    pub rbp: u64, pub rbx: u64,
    pub r11: u64, pub r10: u64, pub r9: u64, pub r8: u64,
    pub rax: u64, pub rcx: u64, pub rdx: u64, pub rsi: u64,
    pub rdi: u64,
    pub orig_rax: u64,
    pub rip: u64,
    pub cs: u64,
    pub eflags: u64,
    pub rsp: u64,
    pub ss: u64,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 21) - [public] syscall_args

```
pub fn syscall_args(&self) -> (u64, u64, u64, u64, u64, u64) {
    (self.rdi, self.rsi, self.rdx, self.r10, self.r8, self.r9)
}

// Set syscall return value
#[inline]
pub fn set_return(&mut self, val: u64) {
    self.rax = val;
}

}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 27) - [public] set_return

```
pub fn set_return(&mut self, val: u64) {
    self.rax = val;
}

}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 5) - [public] LinuxMsgBuf

```
pub struct LinuxMsgBuf {
    pub mtype: u64,
    // Followed by variable length payload in actual usage
}

// Native equivalent of msgsnd/msgrcv via TrangoOS Ports
pub struct PosixMqShim {
    pub native_port: Handle,
}

impl PosixMqShim {
    // Conceptual mapping: Linux mq -> Native Port Send
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 11) - [public] PosixMqShim

// Native equivalent of msgsnd/msgrcv via TrangoOS Ports

```
pub struct PosixMqShim {
    pub native_port: Handle,
}

impl PosixMqShim {
    // Conceptual mapping: Linux mq -> Native Port Send
    pub fn send_msg(&self, _mtype: u64, _payload: &[u8]) -> Result<(), Status> {
        // TODO: Route to native IPC port using kstd_os_workspace rules
        Err(Status::NotSupported)
    }

    // Conceptual mapping: Linux mq -> Native Port Recv
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 17) - [public] send_msg

// Conceptual mapping: Linux mq -> Native Port Send

```
pub fn send_msg(&self, _mtype: u64, _payload: &[u8]) -> Result<(), Status> {
    // TODO: Route to native IPC port using kstd_os_workspace rules
    Err(Status::NotSupported)
}

// Conceptual mapping: Linux mq -> Native Port Recv
pub fn recv_msg(&self, _out: &mut [u8]) -> Result<u64, Status> {
    // TODO: Block/Wait on native IPC port
    Err(Status::NotSupported)
}
}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 23) - [public] recv_msg

// Conceptual mapping: Linux mq -> Native Port Recv

```
pub fn recv_msg(&self, _out: &mut [u8]) -> Result<u64, Status> {
    // TODO: Block/Wait on native IPC port
    Err(Status::NotSupported)
}
}
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 15) - [public] SockAddrIn

```
pub struct SockAddrIn {
    pub sin_family: u16,
    pub sin_port: u16, // Network byte order
    pub sin_addr: Ipv4Addr,
    pub sin_zero: [u8; 8],
}

#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct SockAddrIn6 {
    pub sin6_family: u16,
    pub sin6_port: u16,
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 24) - [public] SockAddrIn6

```
pub struct SockAddrIn6 {
    pub sin6_family: u16,
    pub sin6_port: u16,
    pub sin6_flowinfo: u32,
    pub sin6_addr: Ipv6Addr,
    pub sin6_scope_id: u32,
}

// Translate Linux socket type to native TrangoOS IPC/Net channel kind
#[inline]
pub fn map_socket_type(family: u16, sock_type: u16) -> Option<u8> {
    match (family, sock_type) {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 34) - [public] map_socket_type

```
pub fn map_socket_type(family: u16, sock_type: u16) -> Option<u8> {
    match (family, sock_type) {
        (AF_INET, SOCK_STREAM) => Some(0x01), // Native TCP Channel
        (AF_INET, SOCK_DGRAM)  => Some(0x02), // Native UDP Port
        (AF_UNIX, SOCK_STREAM) => Some(0x10), // Native Local Pipe/Channel
        _ => None,
    }
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 4) - [public] LinuxDevice

// Linux Device Model abstraction (maps to native DriverSpace registry)

```
pub struct LinuxDevice {
    pub name: [u8; 32],
    pub bus_id: [u8; 32],
    pub driver_data: u64,
}

pub trait LinuxDriver {
    fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
    fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
// ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 10) - [public] LinuxDriver

```
pub trait LinuxDriver {
    fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
    fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 11) - [internal] probe

```
fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 12) - [internal] remove

```
fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 17) - [public] register_linux_driver

```
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 4) - [public] DmaDirection

```
pub enum DmaDirection {
    ToDevice = 1,
    FromDevice = 2,
    Bidirectional = 0,
}

// Linux DMA API shim.
// Under the hood, this must request physical grants from the kernel via DriverSpace IPC.
pub struct DmaMapping {
    pub virt: VirtAddr,
    pub phys: PhysAddr,
    pub size: Size,
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 12) - [public] DmaMapping

// Linux DMA API shim.

// Under the hood, this must request physical grants from the kernel via DriverSpace IPC.

```
pub struct DmaMapping {
    pub virt: VirtAddr,
    pub phys: PhysAddr,
    pub size: Size,
}

impl DmaMapping {
    // Equivalent to dma_map_single
    pub fn map_single(_cpu_addr: VirtAddr, _size: Size, _dir: DmaDirection) -> Result<PhysAddr, Status> {
        // TODO: Call native DMA grant IPC
        Err(Status::NotSupported)
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 20) - [public] map_single

// Equivalent to dma_map_single

```
pub fn map_single(_cpu_addr: VirtAddr, _size: Size, _dir: DmaDirection) -> Result<PhysAddr, Status> {
    // TODO: Call native DMA grant IPC
    Err(Status::NotSupported)
}

// Equivalent to dma_unmap_single
pub fn unmap_single(_phys: PhysAddr, _size: Size, _dir: DmaDirection) {
    // TODO: Revoke DMA grant IPC
}
}
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 26) - [public] unmap_single

// Equivalent to dma_unmap_single

```
pub fn unmap_single(_phys: PhysAddr, _size: Size, _dir: DmaDirection) {
    // TODO: Revoke DMA grant IPC
}
}
```

kernel_workspace/core/windows-com/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/windows-com/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/kernel-bin/build.rs (line 13) - [internal] main

```
// kernel_Workspace/kernel-bin/build.rs
//
// NOTE: `src/boot.asm` and `linker.ld` are deliberately NOT wired into the
// build. The `_start` of the bootimage comes from `bootloader::entry_point!`
// (src/main.rs) and the linker script is supplied by the `bootloader` crate
// itself; assembling the old asm entry as well would define the entry symbols
// twice and passing `-T linker.ld` would override the bootloader layout
// (bootimage then bails out with a build failure).
//
// The files are kept as reference for a future multiboot/long-mode bootstrap
// (for the day we drop bootloader 0.9).
```

```
fn main() {
    println!("cargo:rerun-if-changed=src/boot.asm");
    println!("cargo:rerun-if-changed=linker.ld");
}
```

kernel_Workspace/kernel-bin/src/lib.rs (line 13) - [public] announce_boot

/// Serial banner emitted right before the legacy kernel takes over.

```
pub fn announce_boot() {
    let mut serial = kstd_io::Serial;
    serial
        .write_str("\n[Kernel-Bin] starting: handing over to the legacy kernel\n")
        .ok();
}

/// Handoff target for the legacy kernel.
///
/// `kernel::kernel_main` ends with `extern "C" { fn kernel_bin_entry(*const u8) -> ! }`,
/// so this symbol has to exist in the bootimage. The bootloader `BootInfo`
/// pointer is not used yet: the new layer currently only owns the idle loop.
/// ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/lib.rs (line 36) - [public] kernel_ipc_loop

/// Idle loop of the new entry layer (IPC rings are driven from interrupts).

```
pub fn kernel_ipc_loop() -> ! {
    #[cfg(target_arch = "x86_64")]
    loop {
        unsafe { core::arch::asm!("hlt") }
    }

    #[cfg(not(target_arch = "x86_64"))]
    loop {
        core::hint::spin_loop();
    }
}
```

kernel_Workspace/kernel-bin/src/main.rs (line 17) - [internal] kernel_main

```
fn kernel_main(boot_info: &'static BootInfo) -> ! {
    // New layer: banner + plumbing that is linked into the image before the
    // legacy kernel takes over.
    kernel_bin::announce_boot();

    // Legacy kernel: full boot (arch, mm, gfx, drivers, fs, tests, terminal).
    // It never returns (`kernel::terminal::run() -> !`).
    kernel::kernel_main(boot_info)
}
```

kernel_workspace/kernel/build.rs (line 6) - [internal] collect_c_files

```
fn collect_c_files(dir: &Path, out: &mut Vec<PathBuf>) {
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let path = entry.path();
            if path.is_dir() {
                let name = path.file_name().and_then(|n| n.to_str());
                if matches!(name, Some("linuxcom") | Some("wincom") | Some("aarch64") | Some("arm64") | S
                    continue;
            }
            collect_c_files(&path, out);
        } else if matches!(path.extension().and_then(|e| e.to_str()), Some("c") | Some("s")) {
            out.push(path);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/build.rs (line 23) - [internal] clang_target

```
fn clang_target(arch: &str) -> &'static str {
    match arch {
        "x86_64" => "x86_64-unknown-none-elf",
        "riscv64" => "riscv64gc-unknown-none-elf",
        other => panic!(
            "cannot build the C bridge for unsupported target architecture: {other}"
        ),
    }
}

fn main() {
    let manifest_dir = env::var("CARGO_MANIFEST_DIR").expect("CARGO_MANIFEST_DIR");
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/build.rs (line 33) - [internal] main

```
fn main() {
    let manifest_dir = env::var("CARGO_MANIFEST_DIR").expect("CARGO_MANIFEST_DIR");
    let out_dir = env::var("OUT_DIR").expect("OUT_DIR");
    let src_dir = Path::new(&manifest_dir).join("src");

    println!("cargo:rerun-if-changed=src");

    let arch = env::var("CARGO_CFG_TARGET_ARCH").unwrap_or_else(|_| "x86_64".to_string());

    match arch.as_str() {
        "x86_64" => {
            let llvm_target = clang_target("x86_64");
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/arch/mod.rs (line 13) - [public] init

```
pub fn init() {
    imp::init();
}

pub fn hlt_loop() -> ! {
    imp::hlt_loop()
}

pub fn now() -> u64 {
    imp::now()
    // ... (truncated, see source file for more)
}
```


kernel_workspace/kernel/src/arch/mod.rs (line 18) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    imp::hlt_loop()
}

pub fn now() -> u64 {
    imp::now()
}

pub fn current_cpu() -> usize {
    imp::current_cpu()
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/mod.rs (line 23) - [public] now

```
pub fn now() -> u64 {
    imp::now()
}

pub fn current_cpu() -> usize {
    imp::current_cpu()
}
```

kernel_workspace/kernel/src/arch/mod.rs (line 28) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    imp::current_cpu()
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 9) - [internal] HeapSpace

```
struct HeapSpace([u8; HEAP_SIZE]);

static HEAP_MEM: HeapSpace = HeapSpace([0; HEAP_SIZE]);

struct Bump {
    next: usize,
    end: usize,
}

static BUMP: Mutex<Bump> = Mutex::new(Bump { next: 0, end: 0 });

pub fn heap_init() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 13) - [internal] Bump

```
struct Bump {
    next: usize,
    end: usize,
}

static BUMP: Mutex<Bump> = Mutex::new(Bump { next: 0, end: 0 });

pub fn heap_init() {
    let base = &HEAP_MEM as *const HeapSpace as usize;
    let mut b = BUMP.lock();
    b.next = base;
    b.end = base + HEAP_SIZE;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 20) - [public] heap_init

```
pub fn heap_init() {
    let base = &HEAP_MEM as *const HeapSpace as usize;
    let mut b = BUMP.lock();
    b.next = base;
    b.end = base + HEAP_SIZE;
}

pub struct KernelHeap;

unsafe impl GlobalAlloc for KernelHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let mut b = BUMP.lock();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 27) - [public] KernelHeap

```
pub struct KernelHeap;

unsafe impl GlobalAlloc for KernelHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let mut b = BUMP.lock();
        let align = layout.align().max(core::mem::size_of::<usize>());
        let cur = (b.next + align - 1) & !(align - 1);
        match cur.checked_add(layout.size()) {
            Some(new_next) if new_next <= b.end => {
                b.next = new_next;
                cur as *mut u8
            }
            _ => {
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 51) - [public] now

```
pub fn now() -> u64 {
    let t: u64;
    unsafe {
        core::arch::asm!("rdtime {0}", out(reg) t, options(nomem, nostack));
    }
    t
}

#[panic_handler]
fn riscv_panic(info: &PanicInfo) -> ! {
    crate::serial::write_str("\n[PANIC] ");
    crate::serial::print_args(format_args!("{}", info));
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 60) - [internal] riscv_panic

```
fn riscv_panic(info: &PanicInfo) -> ! {
    crate::serial::write_str("\n[PANIC] ");
    crate::serial::print_args(format_args!("{}", info));
    crate::hlt_loop()
}

pub fn init() {
    early_uart("[C]");
    heap_init();
    early_uart("[D]");
    crate::serial::init();
    early_uart("[E]");
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 66) - [public] init

```
pub fn init() {
    early_uart("[C]");
    heap_init();
    early_uart("[D]");
    crate::serial::init();
    early_uart("[E]");
}

pub fn current_cpu() -> usize {
    0
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 74) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    0
}

pub fn hlt_loop() -> ! {
    loop {
        unsafe {
            core::arch::asm!("wfi", options(nomem, nostack, preserves_flags));
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 78) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    loop {
        unsafe {
            core::arch::asm!("wfi", options(nomem, nostack, preserves_flags));
        }
    }
}

pub fn early_uart(s: &str) {
    const THR: *mut u8 = 0x1000_0000 as *mut u8;
    const LSR: *const u8 = 0x1000_0005 as *const u8;
    for b in s.bytes() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 86) - [public] early_uart

```
pub fn early_uart(s: &str) {
    const THR: *mut u8 = 0x1000_0000 as *mut u8;
    const LSR: *const u8 = 0x1000_0005 as *const u8;
    for b in s.bytes() {
        unsafe {
            while LSR.read_volatile() & 0x20 == 0 {}
            THR.write_volatile(b);
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 5) - [internal] x86_panic

```
fn x86_panic(info: &PanicInfo) -> ! {
    crate::gfx::panic_screen::show(info)
}

pub fn init() {
    crate::serial::init();
    crate::gdt::init();
    crate::interrupts::init_idt();
    unsafe { crate::interrupts::PICS.lock().initialize() };
    x86_64::instructions::interrupts::enable();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 9) - [public] init

```
pub fn init() {
    crate::serial::init();
    crate::gdt::init();
    crate::interrupts::init_idt();
    unsafe { crate::interrupts::PICS.lock().initialize() };
    x86_64::instructions::interrupts::enable();
}

pub fn hlt_loop() -> ! {
    loop {
        x86_64::instructions::hlt();
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 17) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    loop {
        x86_64::instructions::hlt();
    }
}

pub fn current_cpu() -> usize {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        use core::arch::x86_64::__cpuid;
        (__cpuid(1).ebx >> 24) as usize
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 23) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        use core::arch::x86_64::__cpuid;
        (__cpuid(1).ebx >> 24) as usize
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        0
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 35) - [public] now

```
pub fn now() -> u64 {
    unsafe { core::arch::x86_64::_rdtsc() }
}
```

// NOTE: the bootloader entry point (`\_start`) is defined by the *binary* crate
// (`kernel-bin/src/main.rs` via `bootloader::entry\_point!`), which then calls
// `crate::kernel\_main`. A library must not define `\_start` as well, otherwise
// rustc fails with "symbol `\_start` is already defined" (two `entry\_point!`
// expansions in the same crate) or the linker reports a duplicate symbol when
// both the library and the binary end up in the same image.

kernel_workspace/kernel/src/audio/jack.rs (line 6) - [public] init

```
pub fn init(base: u32) -> bool {
    hal::init(base)
}

pub fn poll_jack() -> bool {
    let now = hal::jack_present();

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/audio/jack.rs (line 10) - [public] poll_jack

```
pub fn poll_jack() -> bool {
    let now = hal::jack_present();

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }

    false
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/audio/jack.rs (line 23) - [public] query

```
pub fn query() -> u32 {
    unsafe {
        (PRESENT as u32) | ((AMP_ON as u32) << 1)
    }
}

pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    hal::set_amp(on);
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/audio/jack.rs (line 29) - [public] set_amp

```
pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    hal::set_amp(on);
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    hal::play_phys(phys, len)
}

pub fn stop() {
    hal::stop();
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 34) - [public] play_phys

```
pub fn play_phys(phys: u64, len: u32) -> bool {
    hal::play_phys(phys, len)
}

pub fn stop() {
    hal::stop();
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 38) - [public] stop

```
pub fn stop() {  
    hal::stop();  
}
```

kernel_workspace/kernel/src/battery/aut.rs (line 4) - [public] authorize

```
pub fn authorize(ring: u8, op: u8) -> bool {  
    if op == BATT_THRESH as u8 && ring >= 3 {  
        return false;  
    }  
  
    true  
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 5) - [internal] battery_status_packed

```
fn battery_status_packed(a0: *mut u64, a1: *mut u64, a2: *mut u64) -> bool;  
fn battery_set_threshold(pct: u32) -> bool;  
}
```

```
pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {  
    if !aut::authorize(ring, op as u8) {  
        return -1;  
    }  
  
    match op {  
        x if x == aut::BATT_STATUS => {  
            let mut a0 = 0u64;  
            // ... (truncated, see source file for more)  
        }  
    }  
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 6) - [internal] battery_set_threshold

```
fn battery_set_threshold(pct: u32) -> bool;  
}  
  
pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {  
    if !aut::authorize(ring, op as u8) {  
        return -1;  
    }  
  
    match op {  
        x if x == aut::BATT_STATUS => {  
            let mut a0 = 0u64;  
            let mut a1 = 0u64;  
            // ... (truncated, see source file for more)  
        }  
    }  
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 9) - [public] batt_call

```
pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {  
    if !aut::authorize(ring, op as u8) {  
        return -1;  
    }  
  
    match op {  
        x if x == aut::BATT_STATUS => {  
            let mut a0 = 0u64;  
            let mut a1 = 0u64;  
            let mut a2 = 0u64;  
  
            let ok = unsafe { battery_status_packed(&mut a0, &mut a1, &mut a2) };  
            // ... (truncated, see source file for more)  
        }  
    }  
}
```

kernel_workspace/kernel/src/battery/init.rs (line 2) - [internal] battery_init

```
fn battery_init() -> bool;  
}  
  
pub fn init() -> bool {  
    unsafe { battery_init() }  
}
```

kernel_workspace/kernel/src/battery/init.rs (line 5) - [public] init

```
pub fn init() -> bool {  
    unsafe { battery_init() }  
}
```

kernel_Workspace/kernel/src/bluetooth/aut.rs (line 8) - [public] authorize

```
pub fn authorize(ring: u8, op: u32) -> bool {
    if op == BT_INIT {
        return ring == 0;
    }

    if ring == 0 {
        return true;
    }

    if op == BT_CMD || op == BT_ACL_OUT {
        return ring < 3;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 7) - [internal] bt_init

```
fn bt_init() -> bool;
fn bt_ready() -> bool;
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 8) - [internal] bt_ready

```
fn bt_ready() -> bool;
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 9) - [internal] bt_info

```
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 10) - [internal] bt_hci_cmd

```
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;
```

```
pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 11) - [internal] bt_event_poll

```
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;
```

```
pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 12) - [internal] bt_acl_send

```
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 13) - [internal] bt_acl_recv

```
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }
    // ... (truncated, see source file for more)
}
```


kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 20) - [public] bt_call

```
pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }

    if op == aut::BT_INIT {
        return if unsafe { bt_init() } { 0 } else { -2 };
    }

    if !unsafe { bt_ready() } {
        return -2;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 2) - [internal] bt_init

```
fn bt_init() -> bool;
fn bt_ready() -> bool;

}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 3) - [internal] bt_ready

```
fn bt_ready() -> bool;

}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 6) - [public] init

```
pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 10) - [public] ready

```
pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/camera/aut.rs (line 6) - [public] authorize

```
pub fn authorize(ring: u8, op: u8) -> bool {
    if ring == 0 {
        return true;
    }

    if op == CAM_START as u8 || op == CAM_STOP as u8 || op == CAM_FRAME as u8 {
        if ring >= 3 {
            return false;
        }
    }

    true
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/camera/bridge.rs (line 5) - [internal] camera_caps_get_w
    fn camera_caps_get_w(w: *mut u32, h: *mut u32, fmt: *mut u32, fps: *mut u32) -> bool;
    fn camera_start() -> bool;
    fn camera_stop() -> bool;
    fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/camera/bridge.rs (line 6) - [internal] camera_start
    fn camera_start() -> bool;
    fn camera_stop() -> bool;
    fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/camera/bridge.rs (line 7) - [internal] camera_stop
    fn camera_stop() -> bool;
    fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/camera/bridge.rs (line 8) - [internal] camera_frame_to_phys
    fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }
    // ... (truncated, see source file for more)
}

```

```

kernel_workspace/kernel/src/camera/bridge.rs (line 11) - [internal] grant_phys
    fn grant_phys(va: u64) -> Option<u64> {
        crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
    }

    pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
        if !aut::authorize(ring, op as u8) {
            return -1;
        }

        match op {
            x if x == aut::CAM_CAPS => {
                let mut w = 0u32;
                // ... (truncated, see source file for more)
            }
        }
    }

kernel_workspace/kernel/src/camera/bridge.rs (line 15) - [public] cam_call
    pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
        if !aut::authorize(ring, op as u8) {
            return -1;
        }

        match op {
            x if x == aut::CAM_CAPS => {
                let mut w = 0u32;
                let mut h = 0u32;
                let mut fmt = 0u32;
                let mut fps = 0u32;

                // ... (truncated, see source file for more)
            }
        }
    }

kernel_workspace/kernel/src/camera/init.rs (line 2) - [internal] camera_init
    fn camera_init() -> bool;
}

    pub fn init() -> bool {
        unsafe { camera_init() }
    }

kernel_workspace/kernel/src/camera/init.rs (line 5) - [public] init
    pub fn init() -> bool {
        unsafe { camera_init() }
    }

kernel_workspace/kernel/src/caps/audit.rs (line 6) - [public] EventKind
    pub enum EventKind {
        CheckOk,
        CheckDeny,
        Grant,
        Revoke,
        Register,
        Unregister,
    }

    #[derive(Debug, Clone, Copy)]
    pub struct AuditEvent {
        pub seq: u64,
        // ... (truncated, see source file for more)
    }

```

kernel_workspace/kernel/src/caps/audit.rs (line 16) - [public] AuditEvent

```
pub struct AuditEvent {
    pub seq: u64,
    pub tick: u64,
    pub world_id: u32,
    pub target_world: u32,
    pub cap: Capability,
    pub kind: EventKind,
}

const AUDIT_CAP: usize = 2048;

struct AuditInner {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 27) - [internal] AuditInner

```
struct AuditInner {
    buf: [Option<AuditEvent>; AUDIT_CAP],
    head: usize,
    seq: u64,
    total: u64,
}

static AUDIT: Mutex<AuditInner> = Mutex::new(AuditInner {
    buf: [None; AUDIT_CAP],
    head: 0,
    seq: 0,
    total: 0,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 41) - [public] init_audit_log

```
pub fn init_audit_log() -> Result<(), &'static str> {
    let mut a = AUDIT.lock();
    a.head = 0;
    a.seq = 0;
    a.total = 0;
    for i in 0..AUDIT_CAP {
        a.buf[i] = None;
    }
    Ok(())
}

fn now_tick() -> u64 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 52) - [internal] now_tick

```
fn now_tick() -> u64 {
    crate::arch::now()
}

fn push(kind: EventKind, world: u32, target: u32, cap: Capability) {
    let mut a = AUDIT.lock();
    let idx = a.head as usize % AUDIT_CAP;
    let ev = AuditEvent {
        seq: a.seq,
        tick: now_tick(),
        world_id: world,
        target_world: target,
        // ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/caps/audit.rs (line 56) - [internal] push
    fn push(kind: EventKind, world: u32, target: u32, cap: Capability) {
        let mut a = AUDIT.lock();
        let idx = a.head as usize % AUDIT_CAP;
        let ev = AuditEvent {
            seq: a.seq,
            tick: now_tick(),
            world_id: world,
            target_world: target,
            cap,
            kind,
        };
        a.buf[idx] = Some(ev);
        // ... (truncated, see source file for more)
    }

kernel_workspace/kernel/src/caps/audit.rs (line 73) - [public] log_check
    pub fn log_check(world: u32, cap: Capability, ok: bool) {
        push(if ok { EventKind::CheckOk } else { EventKind::CheckDeny },
            world, world, cap);
    }

    pub fn log_grant(granter: u32, target: u32, cap: Capability, ok: bool) {
        if ok {
            push(EventKind::Grant, granter, target, cap);
        }
    }

    pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
        // ... (truncated, see source file for more)
    }

kernel_workspace/kernel/src/caps/audit.rs (line 78) - [public] log_grant
    pub fn log_grant(granter: u32, target: u32, cap: Capability, ok: bool) {
        if ok {
            push(EventKind::Grant, granter, target, cap);
        }
    }

    pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
        if ok {
            push(EventKind::Revoke, world, world, cap);
        }
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/caps/audit.rs (line 84) - [public] log_revoke
    pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
        if ok {
            push(EventKind::Revoke, world, world, cap);
        }
    }

    pub fn log_register(world: u32) {
        push(EventKind::Register, world, world, Capability::User);
    }

    pub fn log_unregister(world: u32) {
        push(EventKind::Unregister, world, world, Capability::User);
        // ... (truncated, see source file for more)
    }

```

kernel_workspace/kernel/src/caps/audit.rs (line 90) - [public] log_register

```
pub fn log_register(world: u32) {
    push(EventKind::Register, world, world, Capability::User);
}

pub fn log_unregister(world: u32) {
    push(EventKind::Unregister, world, world, Capability::User);
}

pub fn count() -> usize {
    AUDIT.lock().total as usize
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 94) - [public] log_unregister

```
pub fn log_unregister(world: u32) {
    push(EventKind::Unregister, world, world, Capability::User);
}

pub fn count() -> usize {
    AUDIT.lock().total as usize
}

pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 98) - [public] count

```
pub fn count() -> usize {
    AUDIT.lock().total as usize
}

pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    let mut idx = if a.head == 0 { AUDIT_CAP - 1 } else { a.head - 1 };

    for _ in 0..n {
        if let Some(ev) = a.buf[idx] {
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 102) - [public] recent

```
pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    let mut idx = if a.head == 0 { AUDIT_CAP - 1 } else { a.head - 1 };

    for _ in 0..n {
        if let Some(ev) = a.buf[idx] {
            out.push(ev);
        }
        if idx == 0 { idx = AUDIT_CAP - 1; } else { idx -= 1; }
        if out.len() >= a.total as usize { break; }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 119) - [public] by_world

```
pub fn by_world(world: u32, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.world_id == world || e.target_world == world)
        .take(limit)
        .collect()
}

pub fn by_kind(kind: EventKind, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.kind == kind)
        .take(limit)
        .collect()
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 126) - [public] by_kind

```
pub fn by_kind(kind: EventKind, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.kind == kind)
        .take(limit)
        .collect()
}
```

```
pub fn deny_count() -> usize {
    by_kind(EventKind::CheckDeny, AUDIT_CAP).len()
}
```

```
#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 133) - [public] deny_count

```
pub fn deny_count() -> usize {
    by_kind(EventKind::CheckDeny, AUDIT_CAP).len()
}
```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_audit() {
        init_audit_log().unwrap();

        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 142) - [internal] test_audit

```
fn test_audit() {
    init_audit_log().unwrap();

    log_check(1, Capability::User, true);
    log_check(1, Capability::Driver, false);
    log_grant(0, 1, Capability::User, true);

    assert_eq!(count(), 3);
    assert_eq!(deny_count(), 1);

    let r = recent(3);
    assert_eq!(r.len(), 3);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/check.rs (line 10) - [public] kernel_world_id

```
pub fn kernel_world_id() -> u32 {
    KERNEL_WORLD.load(Ordering::Relaxed)
}

pub fn set_kernel_world_id(id: u32) {
    KERNEL_WORLD.store(id, Ordering::Relaxed);
}

pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/check.rs (line 14) - [public] set_kernel_world_id

```
pub fn set_kernel_world_id(id: u32) {
    KERNEL_WORLD.store(id, Ordering::Relaxed);
}

pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/check.rs (line 18) - [public] current_world_id_pub

```
pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

fn current_world_id() -> u32 {
    current_world_id_pub()
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/check.rs (line 22) - [public] set_current_world

```
pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

fn current_world_id() -> u32 {
    current_world_id_pub()
}

pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/caps/check.rs (line 26) - [internal] current_world_id

```
fn current_world_id() -> u32 {
    current_world_id_pub()
}

pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 30) - [public] has_cap

```
pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 35) - [public] world_has_cap

```
pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    if store::world_has_cap(wid, cap) {
        audit::log_check(wid, cap, true);
        Ok(())
    } else {
        audit::log_check(wid, cap, false);
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 39) - [public] require_cap

```
pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    if store::world_has_cap(wid, cap) {
        audit::log_check(wid, cap, true);
        Ok(())
    } else {
        audit::log_check(wid, cap, false);
        Err(CapabilityError {
            required: cap,
            world_id: Some(wid),
        })
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 54) - [public] require_world_cap

```
pub fn require_world_cap(world_id: u32, cap: Capability) -> CapResult<()> {
    if store::world_has_cap(world_id, cap) {
        audit::log_check(world_id, cap, true);
        Ok(())
    } else {
        audit::log_check(world_id, cap, false);
        Err(CapabilityError {
            required: cap,
            world_id: Some(world_id),
        })
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 67) - [public] require_caps

```
pub fn require_caps(caps: &[Capability]) -> CapResult<()> {
    for &cap in caps {
        require_cap(cap)?;
    }
    Ok(())
}

pub fn with_cap<T, F>(cap: Capability, f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_cap(cap)?;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/check.rs (line 74) - [public] with_cap

```
pub fn with_cap<T, F>(cap: Capability, f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_cap(cap)?;
    Ok(f())
}

pub fn with_caps<T, F>(caps: &[Capability], f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/check.rs (line 82) - [public] with_caps

```
pub fn with_caps<T, F>(caps: &[Capability], f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_caps(caps)?;
    Ok(f())
}

#[macro_export]
macro_rules! require_cap {
    ($cap:expr) => {
        $crate::caps::check::require_cap($cap)?
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 104) - [public] assert_cap

```
pub fn assert_cap(cap: Capability) {
    if !has_cap(cap) {
        panic!("CAPABILITY VIOLATION: {} required", cap.name());
    }
}

#[inline(always)]
pub fn fast_check(cap: Capability) -> bool {
    store::world_has_cap(current_world_id(), cap)
}

pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 111) - [public] fast_check

```
pub fn fast_check(cap: Capability) -> bool {
    store::world_has_cap(current_world_id(), cap)
}

pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
where
    F: FnOnce() -> T,
{
    if has_cap(cap) {
        f()
    } else {
        default
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/check.rs (line 115) - [public] if_cap

```
pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
where
    F: FnOnce() -> T,
{
    if has_cap(cap) {
        f()
    } else {
        default
    }
}

pub struct TemporaryCaps {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/check.rs (line 126) - [public] TemporaryCaps

```
pub struct TemporaryCaps {
    world_id: u32,
    original: CapabilitySet,
}

impl TemporaryCaps {
    pub fn enter(extra: CapabilitySet) -> CapResult<Self> {
        let wid = current_world_id();
        let original = store::get_world_caps(wid)
            .map_err(|_| CapabilityError { required: Capability::User, world_id: Some(wid) })?;

        require_cap(Capability::Admin)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/check.rs (line 132) - [public] enter

```
pub fn enter(extra: CapabilitySet) -> CapResult<Self> {
    let wid = current_world_id();
    let original = store::get_world_caps(wid)
        .map_err(|_| CapabilityError { required: Capability::User, world_id: Some(wid) })?;

    require_cap(Capability::Admin)?;

    store::set_world_caps(wid, original.union(extra))
        .map_err(|_| CapabilityError { required: Capability::Admin, world_id: Some(wid) })?;

    Ok(Self { world_id: wid, original })
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 147) - [internal] drop

```
fn drop(&mut self) {
    let _ = store::set_world_caps(self.world_id, self.original);
}
}
```

```
#[cfg(test)]
mod tests {
    use super::*;
    use crate::caps::sets;

    #[test]
    fn test_require() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/check.rs (line 158) - [internal] test_require

```
fn test_require() {
    crate::caps::init().unwrap();

    let wid = store::register_world(None, sets::presets::standard_user()).unwrap();

    assert!(store::world_has_cap(wid, Capability::User));
    assert!(!store::world_has_cap(wid, Capability::Driver));
}
}
```

kernel_workspace/kernel/src/caps/defaults.rs (line 6) - [public] default_for_ring

```
pub fn default_for_ring(ring: u8) -> CapabilitySet {
    match ring {
        0 => sets::presets::kernel(),
        1 => sets::presets::driver(),
        2 => sets::presets::privileged_user(),
        _ => sets::presets::standard_user(),
    }
}

pub fn install_defaults() -> Result<(), &'static str> {
    let id = store::register_world(None, sets::presets::kernel())?;
    crate::caps::check::set_kernel_world_id(id);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/defaults.rs (line 15) - [public] install_defaults

```
pub fn install_defaults() -> Result<(), &'static str> {
    let id = store::register_world(None, sets::presets::kernel())?;
    crate::caps::check::set_kernel_world_id(id);
    audit::log_register(id);
    Ok(())
}

pub fn register_for_ring(ring: u8, parent: Option<u32>) -> Result<u32, &'static str> {
    let set = default_for_ring(ring);
    let wid = store::register_world(parent, set)?;
    audit::log_register(wid);
    Ok(wid)
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/caps/defaults.rs (line 22) - [public] register_for_ring
pub fn register_for_ring(ring: u8, parent: Option<u32>) -> Result<u32, &'static str> {
    let set = default_for_ring(ring);
    let wid = store::register_world(parent, set)?;
    audit::log_register(wid);
    Ok(wid)
}
```

```
kernel_workspace/kernel/src/caps/export.rs (line 7) - [internal] cap_from_id
```

```
fn cap_from_id(id: u8) -> Option<Capability> {
    Capability::iter_all().find(|c| c.id() == id)
}

#[no_mangle]
pub extern "C" fn caps_self_bits() -> u32 {
    let wid = check::current_world_id_pub();
    store::get_world_caps(wid).map(|c| c.bits()).unwrap_or(0)
}

#[no_mangle]
pub extern "C" fn caps_world_bits(world_id: u32) -> u32 {
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/caps/grant.rs (line 18) - [public] grant_cap
```

```
pub fn grant_cap(granter: u32, target: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(granter, cap) {
        audit::log_grant(granter, target, cap, false);
        return Err(CapabilityError { required: cap, world_id: Some(granter) });
    }

    store::add_world_cap(target, cap)
        .map_err(|_| CapabilityError { required: cap, world_id: Some(target) })?;

    audit::log_grant(granter, target, cap, true);
    Ok(())
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/caps/grant.rs (line 31) - [public] delegate_caps
```

```
pub fn delegate_caps(granter: u32, target: u32, caps: CapabilitySet) -> CapResult<()> {
    for cap in caps.iter() {
        grant_cap(granter, target, cap)?;
    }
    Ok(())
}

pub fn inherit_set(parent_world: u32) -> CapabilitySet {
    let parent = match store::get_world_caps(parent_world) {
        Ok(c) => c,
        Err(_) => CapabilitySet::empty(),
    };
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/caps/grant.rs (line 38) - [public] inherit_set
```

```
pub fn inherit_set(parent_world: u32) -> CapabilitySet {
    let parent = match store::get_world_caps(parent_world) {
        Ok(c) => c,
        Err(_) => CapabilitySet::empty(),
    };

    let mut child = parent;
    for &cap in NON_INHERITABLE {
        child = child.remove(cap);
    }
    child
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 51) - [public] spawn_child

```
pub fn spawn_child(parent_world: u32) -> Result<u32, &'static str> {  
    let child_set = inherit_set(parent_world);  
    store::register_world(Some(parent_world), child_set)  
}
```

```
#[derive(Clone, Copy)]  
struct TempGrant {  
    world_id: u32,  
    cap: Capability,  
    expires_tick: u64,  
    active: bool,  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 58) - [internal] TempGrant

```
struct TempGrant {  
    world_id: u32,  
    cap: Capability,  
    expires_tick: u64,  
    active: bool,  
}
```

```
const MAX_TEMP: usize = 128;
```

```
static TEMP: Mutex<[TempGrant; MAX_TEMP]> = Mutex::new([TempGrant {  
    world_id: 0,  
    cap: Capability::User,  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 74) - [internal] now_tick

```
fn now_tick() -> u64 {  
    crate::arch::now()  
}
```

```
pub fn grant_temporary(granter: u32, target: u32, cap: Capability,  
                       ttl_ticks: u64) -> CapResult<()>  
{  
    grant_cap(granter, target, cap)?;  
  
    let mut t = TEMP.lock();  
    let slot = t.iter_mut().find(|g| !g.active)  
        .ok_or(CapabilityError { required: cap, world_id: Some(target) })?;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 78) - [public] grant_temporary

```
pub fn grant_temporary(granter: u32, target: u32, cap: Capability,  
                       ttl_ticks: u64) -> CapResult<()>  
{  
    grant_cap(granter, target, cap)?;  
  
    let mut t = TEMP.lock();  
    let slot = t.iter_mut().find(|g| !g.active)  
        .ok_or(CapabilityError { required: cap, world_id: Some(target) })?;  
  
    *slot = TempGrant {  
        world_id: target,  
        cap,  
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 97) - [public] prune_expired

```
pub fn prune_expired() {
    let t_now = now_tick();
    let mut t = TEMP.lock();

    for g in t.iter_mut() {
        if g.active && t_now >= g.expires_tick {
            let _ = store::remove_world_cap(g.world_id, g.cap);
            audit::log_revoke(g.world_id, g.cap, true);
            g.active = false;
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 110) - [public] is_temporary

```
pub fn is_temporary(world_id: u32, cap: Capability) -> bool {
    let t = TEMP.lock();
    t.iter().any(|g| g.active && g.world_id == world_id && g.cap == cap)
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::caps::sets;

    #[test]
    fn test_inherit() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/grant.rs (line 121) - [internal] test_inherit

```
fn test_inherit() {
    crate::caps::init().unwrap();

    let parent = store::register_world(None, sets::presets::privileged_user()).unwrap();
    let child_set = inherit_set(parent);

    assert!(child_set.has(Capability::User));
    assert!(child_set.has(Capability::Spawn));
    assert!(!child_set.has(Capability::Admin));
    assert!(!child_set.has(Capability::SyscallAll));
}

}
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 5) - [public] parent

```
pub fn parent(cap: Capability) -> Option<Capability> {
    Some(match cap {
        Capability::Root => return None,

        Capability::Ring0 => Capability::Root,
        Capability::Driver => Capability::Ring0,
        Capability::User => Capability::Root,
        Capability::Admin => Capability::Ring0,

        Capability::PhysAlloc => Capability::Driver,
        Capability::VirtMap => Capability::Driver,
        Capability::Dma => Capability::Driver,
        // ... (truncated, see source file for more)
    })
}
```

kernel_Workspace/kernel/src/caps/hierarchy.rs (line 47) - [public] implies

```
pub fn implies(held: Capability, required: Capability) -> bool {
    if held == required {
        return true;
    }

    let mut cur = required;
    while let Some(p) = parent(cur) {
        if p == held {
            return true;
        }
        cur = p;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/hierarchy.rs (line 62) - [public] set_implies

```
pub fn set_implies(held: CapabilitySet, required: Capability) -> bool {
    for cap in held.iter() {
        if implies(cap, required) {
            return true;
        }
    }
    false
}
```

```
pub fn expand_hierarchy(set: CapabilitySet) -> CapabilitySet {
    let mut result = set;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/hierarchy.rs (line 71) - [public] expand_hierarchy

```
pub fn expand_hierarchy(set: CapabilitySet) -> CapabilitySet {
    let mut result = set;

    for cap in set.iter() {
        let mut cur = cap;
        while let Some(p) = parent(cur) {
            result = result.add(p);
            cur = p;
        }
    }

    result
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/hierarchy.rs (line 85) - [public] path_to_root

```
pub fn path_to_root(cap: Capability) -> Vec<Capability> {
    let mut path = vec![cap];
    let mut cur = cap;
    while let Some(p) = parent(cur) {
        path.push(p);
        cur = p;
    }
    path.reverse();
    path
}

pub fn depth(cap: Capability) -> usize {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/caps/hierarchy.rs (line 96) - [public] depth

```
pub fn depth(cap: Capability) -> usize {
    let mut d = 0;
    let mut cur = cap;
    while let Some(p) = parent(cur) {
        d += 1;
        cur = p;
    }
    d
}

pub fn subtree(root: Capability) -> Vec<Capability> {
    let mut result = vec![root];
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 106) - [public] subtree

```
pub fn subtree(root: Capability) -> Vec<Capability> {
    let mut result = vec![root];
    let mut i = 0;

    while i < result.len() {
        let cur = result[i];
        for cap in Capability::iter_all() {
            if parent(cap) == Some(cur) && !result.contains(&cap) {
                result.push(cap);
            }
        }
        i += 1;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 123) - [public] can_delegate

```
pub fn can_delegate(holder: CapabilitySet, candidate: Capability) -> bool {
    set_implies(holder, candidate)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_hierarchy() {
        assert!(implies(Capability::Root, Capability::Driver));
        assert!(implies(Capability::Driver, Capability::DevPci));
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 132) - [internal] test_hierarchy

```
fn test_hierarchy() {
    assert!(implies(Capability::Root, Capability::Driver));
    assert!(implies(Capability::Driver, Capability::DevPci));
    assert!(!implies(Capability::DevPci, Capability::Driver));

    let driver_set = CapabilitySet::single(Capability::Driver);
    assert!(set_implies(driver_set, Capability::DevPci));
    assert!(!set_implies(driver_set, Capability::Admin));

    let expanded = expand_hierarchy(CapabilitySet::single(Capability::DevPci));
    assert!(expanded.has(Capability::DevPci));
    assert!(expanded.has(Capability::Driver));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 149) - [internal] test_depth

```
fn test_depth() {
    assert_eq!(depth(Capability::Root), 0);
    assert_eq!(depth(Capability::Driver), 2);
    assert_eq!(depth(Capability::DevPci), 3);
}
}
```

kernel_workspace/kernel/src/caps/mod.rs (line 31) - [public] init

```
pub fn init() -> Result<(), &'static str> {
    store::init_store()?;
    audit::init_audit_log()?;
    defaults::install_defaults()?;
    Ok(())
}

pub fn snapshot() -> SystemSnapshot {
    SystemSnapshot {
        global_caps: store::global_caps(),
        revoked: revoke::revoked_list(),
        audit_count: audit::count(),
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/mod.rs (line 38) - [public] snapshot

```
pub fn snapshot() -> SystemSnapshot {
    SystemSnapshot {
        global_caps: store::global_caps(),
        revoked: revoke::revoked_list(),
        audit_count: audit::count(),
        world_count: store::world_count(),
    }
}

#[derive(Debug, Clone)]
pub struct SystemSnapshot {
    pub global_caps: CapabilitySet,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/mod.rs (line 48) - [public] SystemSnapshot

```
pub struct SystemSnapshot {
    pub global_caps: CapabilitySet,
    pub revoked: Vec<Capability>,
    pub audit_count: usize,
    pub world_count: usize,
}

pub fn self_test() -> crate::testing::TestResult {
    let mut ok = true;

    let mut chk = |step: u32, cond: bool| -> bool {
        if !cond {
            // ... (truncated, see source file for more)
        }
    };
}
```

kernel_workspace/kernel/src/caps/mod.rs (line 55) - [public] self_test

```
pub fn self_test() -> crate::testing::TestResult {
    let mut ok = true;

    let mut chk = |step: u32, cond: bool| -> bool {
        if !cond {
            println!("[caps] self_test step {} FAILED", step);
        }
        cond
    };

    let a = CapabilitySet::empty().add(Capability::User);
    let b = CapabilitySet::single(Capability::Driver);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/policy.rs (line 9) - [public] set_hook

```
pub fn set_hook(h: PolicyHook) {
    let raw: *mut () = unsafe { core::mem::transmute(h as PolicyHook) };
    HOOK.store(raw, Ordering::Relaxed);
}

fn policy_allows(world: u32, cap: Capability) -> bool {
    let raw = HOOK.load(Ordering::Relaxed);
    if raw.is_null() {
        return true;
    }
    let hook: PolicyHook = unsafe { core::mem::transmute(raw) };
    hook(world, cap)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 14) - [internal] policy_allows

```
fn policy_allows(world: u32, cap: Capability) -> bool {
    let raw = HOOK.load(Ordering::Relaxed);
    if raw.is_null() {
        return true;
    }
    let hook: PolicyHook = unsafe { core::mem::transmute(raw) };
    hook(world, cap)
}

pub fn enforce(world: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 23) - [public] enforce

```
pub fn enforce(world: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }

    if !policy_allows(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }

    Ok(())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 35) - [public] enforce_self

```
pub fn enforce_self(cap: Capability) -> CapResult<()> {
    let wid = super::check::current_world_id_pub();
    enforce(wid, cap)
}

pub fn allowed(world: u32, cap: Capability) -> bool {
    enforce(world, cap).is_ok()
}
```

kernel_workspace/kernel/src/caps/policy.rs (line 40) - [public] allowed

```
pub fn allowed(world: u32, cap: Capability) -> bool {
    enforce(world, cap).is_ok()
}
```

```
kernel_workspace/kernel/src/caps/revoke.rs (line 7) - [public] revoke_from_world
pub fn revoke_from_world(world_id: u32, cap: Capability) -> CapResult<()> {
    store::remove_world_cap(world_id, cap)
        .map_err(|_| CapabilityError { required: cap, world_id: Some(world_id) })?;

    audit::log_revoke(world_id, cap, true);
    Ok(())
}

pub fn revoke_subtree(world_id: u32, cap: Capability) -> CapResult<()> {
    for c in hierarchy::subtree(cap) {
        let _ = store::remove_world_cap(world_id, c);
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/caps/revoke.rs (line 15) - [public] revoke_subtree
pub fn revoke_subtree(world_id: u32, cap: Capability) -> CapResult<()> {
    for c in hierarchy::subtree(cap) {
        let _ = store::remove_world_cap(world_id, c);
    }
    audit::log_revoke(world_id, cap, true);
    Ok(())
}

pub fn revoke_global(cap: Capability) {
    store::add_global_revoked(cap);
    audit::log_revoke(0, cap, true);
}
// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/caps/revoke.rs (line 23) - [public] revoke_global
pub fn revoke_global(cap: Capability) {
    store::add_global_revoked(cap);
    audit::log_revoke(0, cap, true);
}

pub fn restore_global(cap: Capability) {
    store::remove_global_revoked(cap);
}

pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/caps/revoke.rs (line 28) - [public] restore_global
pub fn restore_global(cap: Capability) {
    store::remove_global_revoked(cap);
}

pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();

    let revoked = CapabilitySet::all().diff(store::global_caps());
    for cap in revoked.iter() {
        out.push(cap);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/revoke.rs (line 32) - [public] revoked_list

```
pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();

    let revoked = CapabilitySet::all().diff(store::global_caps());
    for cap in revoked.iter() {
        out.push(cap);
    }
    out
}

pub fn is_globally_revoked(cap: Capability) -> bool {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 43) - [public] is_globally_revoked

```
pub fn is_globally_revoked(cap: Capability) -> bool {
    !store::global_caps().has(cap)
}

pub fn lockdown(world_id: u32) {
    let keep = CapabilitySet::empty()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv);

    for cap in Capability::iter_all() {
        if !keep.has(cap) {
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 47) - [public] lockdown

```
pub fn lockdown(world_id: u32) {
    let keep = CapabilitySet::empty()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv);

    for cap in Capability::iter_all() {
        if !keep.has(cap) {
            let _ = store::remove_world_cap(world_id, cap);
        }
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 68) - [internal] test_revoke

```
fn test_revoke() {
    crate::caps::init().unwrap();

    let wid = store::register_world(None, sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(wid, Capability::DevPci));

    revoke_from_world(wid, Capability::Driver).unwrap();

    assert!(!store::world_has_cap(wid, Capability::Driver));
}
}
```

kernel_Workspace/kernel/src/caps/sets.rs (line 4) - [public] CapSetBuilder

```
pub struct CapSetBuilder {
    set: CapabilitySet,
}

impl CapSetBuilder {
    pub fn new() -> Self {
        Self { set: CapabilitySet::empty() }
    }

    pub fn empty() -> Self {
        Self::new()
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 9) - [public] new

```
pub fn new() -> Self {
    Self { set: CapabilitySet::empty() }
}

pub fn empty() -> Self {
    Self::new()
}

pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 13) - [public] empty

```
pub fn empty() -> Self {
    Self::new()
}

pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}

pub fn add_many(mut self, caps: &[Capability]) -> Self {
    for &cap in caps {
        self.set = self.set.add(cap);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 17) - [public] add

```
pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}

pub fn add_many(mut self, caps: &[Capability]) -> Self {
    for &cap in caps {
        self.set = self.set.add(cap);
    }
    self
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/caps/sets.rs (line 22) - [public] add_many
    pub fn add_many(mut self, caps: &[Capability]) -> Self {
        for &cap in caps {
            self.set = self.set.add(cap);
        }
        self
    }

    pub fn add_category(mut self, cat: super::types::CapCategory) -> Self {
        for cap in Capability::iter_all() {
            if cap.category() == cat {
                self.set = self.set.add(cap);
            }
        }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/caps/sets.rs (line 29) - [public] add_category
    pub fn add_category(mut self, cat: super::types::CapCategory) -> Self {
        for cap in Capability::iter_all() {
            if cap.category() == cat {
                self.set = self.set.add(cap);
            }
        }
        self
    }

    pub fn remove(mut self, cap: Capability) -> Self {
        self.set = self.set.remove(cap);
        self
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/sets.rs (line 38) - [public] remove
    pub fn remove(mut self, cap: Capability) -> Self {
        self.set = self.set.remove(cap);
        self
    }

    pub fn with_hierarchy(self) -> Self {
        Self { set: hierarchy::expand_hierarchy(self.set) }
    }

    pub fn build(self) -> CapabilitySet {
        self.set
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/sets.rs (line 43) - [public] with_hierarchy
    pub fn with_hierarchy(self) -> Self {
        Self { set: hierarchy::expand_hierarchy(self.set) }
    }

    pub fn build(self) -> CapabilitySet {
        self.set
    }
}

pub mod presets {
    use super::*;

    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/caps/sets.rs (line 47) - [public] build

```
pub fn build(self) -> CapabilitySet {
    self.set
}

pub mod presets {
    use super::*;

    pub fn minimal_user() -> CapabilitySet {
        CapSetBuilder::new()
            .add(Capability::User)
            .add(Capability::Mmap)
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 55) - [public] minimal_user

```
pub fn minimal_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::Mmap)
        .add(Capability::IpcSend)
        .add(Capability::IpcRecv)
        .add(Capability::FsRead)
        .add(Capability::Spawn)
        .add(Capability::SyscallRestricted)
        .build()
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 67) - [public] standard_user

```
pub fn standard_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::Mmap)
        .add(Capability::Protect)
        .add(Capability::Spawn)
        .add(Capability::Kill)
        .add(Capability::IpcSend)
        .add(Capability::IpcRecv)
        .add(Capability::IpcBroadcast)
        .add(Capability::FsRead)
        .add(Capability::FsWrite)
        // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 84) - [public] privileged_user

```
pub fn privileged_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add_many(&[
            Capability::Mmap, Capability::Protect, Capability::Spawn,
            Capability::Kill, Capability::IpcSend, Capability::IpcRecv,
            Capability::IpcBroadcast, Capability::IpcPrivileged,
            Capability::FsRead, Capability::FsWrite,
            Capability::FsCreate, Capability::FsMount,
            Capability::Sched, Capability::SyscallAll,
        ])
        .build()
    // ... (truncated, see source file for more)
}
```


kernel_workspace/kernel/src/caps/sets.rs (line 98) - [public] driver

```
pub fn driver() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::Driver)
        .add_many(&[
            Capability::PhysAlloc, Capability::VirtMap, Capability::Dma,
            Capability::DevPci, Capability::DevPort,
            Capability::DevMmio, Capability::DevIrq,
            Capability::IpcSend, Capability::IpcRecv,
            Capability::IpcPrivileged, Capability::SyscallAll,
        ])
        .build()
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 111) - [public] kernel

```
pub fn kernel() -> CapabilitySet {
    CapabilitySet::all()
}

pub fn sandbox() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv)
        .add(Capability::SyscallRestricted)
        .build()
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 115) - [public] sandbox

```
pub fn sandbox() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv)
        .add(Capability::SyscallRestricted)
        .build()
}

pub fn validate_hierarchy(set: CapabilitySet) -> Result<(), &'static str> {
    for cap in set.iter() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 125) - [public] validate_hierarchy

```
pub fn validate_hierarchy(set: CapabilitySet) -> Result<(), &'static str> {
    for cap in set.iter() {
        let mut cur = cap;
        while let Some(p) = hierarchy::parent(cur) {
            if p != Capability::Root && !set.has(p) {
                return Err("hierarchy inconsistency: child without parent");
            }
            cur = p;
            if cur == Capability::Root {
                break;
            }
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 141) - [public] effective

```
pub fn effective(set: CapabilitySet) -> CapabilitySet {
    hierarchy::expand_hierarchy(set)
}

pub fn effective_diff(a: CapabilitySet, b: CapabilitySet) -> CapabilitySet {
    let a_eff = effective(a);
    let b_eff = effective(b);
    a_eff.diff(b_eff)
}

pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 145) - [public] effective_diff

```
pub fn effective_diff(a: CapabilitySet, b: CapabilitySet) -> CapabilitySet {
    let a_eff = effective(a);
    let b_eff = effective(b);
    a_eff.diff(b_eff)
}

pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    let sup_eff = effective(superset);
    sup_eff.contains(sub_eff)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 151) - [public] is_subset_with_hierarchy

```
pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    let sup_eff = effective(superset);
    sup_eff.contains(sub_eff)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_presets() {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 162) - [internal] test_presets

```
fn test_presets() {
    let user = presets::minimal_user();
    assert!(user.has(Capability::User));
    assert!(!user.has(Capability::Driver));
    assert!(!user.has(Capability::DevPci));

    let drv = presets::driver();
    assert!(drv.has(Capability::Driver));
    assert!(drv.has(Capability::DevPci));
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 174) - [internal] test_effective

```
fn test_effective() {
    let set = CapabilitySet::single(Capability::DevPci);
    let eff = effective(set);

    assert!(eff.has(Capability::DevPci));
    assert!(eff.has(Capability::Driver));
    assert!(eff.has(Capability::Ring0));
    assert!(eff.has(Capability::Root));
}
}
```

kernel_workspace/kernel/src/caps/store.rs (line 9) - [public] WorldCaps

```
pub struct WorldCaps {
    pub world_id: u32,
    pub caps: CapabilitySet,
    pub inherited_from: Option<u32>,
    pub revokable: bool,
}

static STORE: Mutex<StoreInner> = Mutex::new(StoreInner {
    worlds: [None; MAX_WORLDS],
    global_revoked: CapabilitySet::empty(),
    next_id: 1,
});
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 22) - [internal] StoreInner

```
struct StoreInner {
    worlds: [Option<WorldCaps>; MAX_WORLDS],
    global_revoked: CapabilitySet,
    next_id: u32,
}

pub fn init_store() -> Result<(), &'static str> {
    let mut s = STORE.lock();
    for i in 0..MAX_WORLDS {
        s.worlds[i] = None;
    }
    Ok(())
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 28) - [public] init_store

```
pub fn init_store() -> Result<(), &'static str> {
    let mut s = STORE.lock();
    for i in 0..MAX_WORLDS {
        s.worlds[i] = None;
    }
    Ok(())
}

pub fn global_caps() -> CapabilitySet {
    let s = STORE.lock();
    CapabilitySet::all().diff(s.global_revoked)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 36) - [public] global_caps

```
pub fn global_caps() -> CapabilitySet {
    let s = STORE.lock();
    CapabilitySet::all().diff(s.global_revoked)
}

pub fn register_world(parent_world: Option<u32>, initial: CapabilitySet) -> Result<u32, &'static str> {
    let mut s = STORE.lock();

    if let Some(pwid) = parent_world {
        let parent_caps = find_world_inner(&s, pwid)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 41) - [public] register_world

```
pub fn register_world(parent_world: Option<u32>, initial: CapabilitySet) -> Result<u32, &'static str> {
    let mut s = STORE.lock();

    if let Some(pwid) = parent_world {
        let parent_caps = find_world_inner(&s, pwid)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        for cap in initial.iter() {
            if !hierarchy::set_implies(parent_eff, cap) {
                return Err("child capability exceeds parent");
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 71) - [internal] find_world_inner

```
fn find_world_inner(s: &StoreInner, world_id: u32) -> Result<WorldCaps, &'static str> {
    for w in &s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                return Ok(*wc);
            }
        }
    }
    Err("world not found")
}
```

```
pub fn get_world_caps(world_id: u32) -> Result<CapabilitySet, &'static str> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 82) - [public] get_world_caps

```
pub fn get_world_caps(world_id: u32) -> Result<CapabilitySet, &'static str> {
    let s = STORE.lock();
    let wc = find_world_inner(&s, world_id)?;
    Ok(wc.caps.diff(s.global_revoked))
}
```

```
pub fn set_world_caps(world_id: u32, new_caps: CapabilitySet) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    let wc = find_world_inner(&s, world_id)?;
    if let Some(parent_id) = wc.inherited_from {
        let parent_caps = find_world_inner(&s, parent_id)?;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 88) - [public] set_world_caps

```
pub fn set_world_caps(world_id: u32, new_caps: CapabilitySet) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    let wc = find_world_inner(&s, world_id)?;
    if let Some(parent_id) = wc.inherited_from {
        let parent_caps = find_world_inner(&s, parent_id)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        for cap in new_caps.iter() {
            if !hierarchy::set_implies(parent_eff, cap) {
                return Err("cannot grant capability exceeding parent");
            }
        }
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/caps/store.rs (line 115) - [public] add_world_cap
pub fn add_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let current = get_world_caps(world_id)?;
    set_world_caps(world_id, current.add(cap))
}

pub fn remove_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                if !wc.revokable {
                    // ... (truncated, see source file for more)
                }
            }
        }
    }
}

kernel_Workspace/kernel/src/caps/store.rs (line 120) - [public] remove_world_cap
pub fn remove_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                if !wc.revokable {
                    return Err("world capabilities are not revokable");
                }
                wc.caps = wc.caps.remove(cap);
                return Ok(());
            }
        }
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/caps/store.rs (line 138) - [public] unregister_world
pub fn unregister_world(world_id: u32) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                *w = None;
                return Ok(());
            }
        }
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/caps/store.rs (line 153) - [public] world_has_cap
pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    let s = STORE.lock();
    if let Ok(wc) = find_world_inner(&s, world_id) {
        let eff = wc.caps.diff(s.global_revoked);
        hierarchy::set_implies(eff, cap)
    } else {
        false
    }
}

pub fn world_count() -> usize {
    let s = STORE.lock();
    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/caps/store.rs (line 163) - [public] world_count

```
pub fn world_count() -> usize {
    let s = STORE.lock();
    s.worlds.iter().filter(|w| w.is_some()).count()
}

pub fn iter_worlds<F: FnMut(u32, CapabilitySet)>(mut f: F) {
    let s = STORE.lock();
    for w in &s.worlds {
        if let Some(wc) = w {
            f(wc.world_id, wc.caps);
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 168) - [public] iter_worlds

```
pub fn iter_worlds<F: FnMut(u32, CapabilitySet)>(mut f: F) {
    let s = STORE.lock();
    for w in &s.worlds {
        if let Some(wc) = w {
            f(wc.world_id, wc.caps);
        }
    }
}

pub fn add_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.add(cap);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 177) - [public] add_global_revoked

```
pub fn add_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.add(cap);
}

pub fn remove_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.remove(cap);
}

#[cfg(test)]
mod tests {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 182) - [public] remove_global_revoked

```
pub fn remove_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.remove(cap);
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_world_registration() {
        init_store().unwrap();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/store.rs (line 192) - [internal] test_world_registration

```
fn test_world_registration() {
    init_store().unwrap();

    let wid = register_world(None, sets::presets::standard_user()).unwrap();
    assert!(world_has_cap(wid, Capability::User));
    assert!(!world_has_cap(wid, Capability::Driver));

    let child_result = register_world(Some(wid),
        CapabilitySet::single(Capability::DevPci));
    assert!(child_result.is_err());
}
}
```

kernel_workspace/kernel/src/caps/syscalls.rs (line 13) - [internal] cap_from_id

```
fn cap_from_id(id: u8) -> Option<Capability> {
    Capability::iter_all().find(|c| c.id() == id)
}

pub fn cap_syscall(num: u64, a0: u64, a1: u64, a2: u64) -> u64 {
    let wid = check::current_world_id_pub();

    match num {
        SYS_CAP_QUERY => {
            match cap_from_id(a0 as u8) {
                Some(cap) => store::world_has_cap(wid, cap) as u64,
                None => 0,
            }
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/caps/syscalls.rs (line 17) - [public] cap_syscall

```
pub fn cap_syscall(num: u64, a0: u64, a1: u64, a2: u64) -> u64 {
    let wid = check::current_world_id_pub();

    match num {
        SYS_CAP_QUERY => {
            match cap_from_id(a0 as u8) {
                Some(cap) => store::world_has_cap(wid, cap) as u64,
                None => 0,
            }
        }
        SYS_CAP_REQUEST => {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/caps/tests.rs (line 8) - [internal] test_full_flow

```
fn test_full_flow() {
    init().unwrap();

    let kernel = crate::caps::check::kernel_world_id();

    let drv = store::register_world(Some(kernel), sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(drv, Capability::DevPci));
    assert!(!store::world_has_cap(drv, Capability::Admin));

    let usr = store::register_world(Some(kernel), sets::presets::standard_user()).unwrap();
    assert!(store::world_has_cap(usr, Capability::FsRead));
    assert!(!store::world_has_cap(usr, Capability::DevPci));
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/tests.rs (line 29) - [internal] test_hierarchy_enforcement

```
fn test_hierarchy_enforcement() {
    init().unwrap();

    let parent = store::register_world(None, sets::presets::standard_user()).unwrap();

    let bad = store::register_world(Some(parent),
        CapabilitySet::single(Capability::DevPci));
    assert!(bad.is_err());
}

#[test]
fn test_global_revocation() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/tests.rs (line 40) - [internal] test_global_revocation

```
fn test_global_revocation() {
    init().unwrap();

    let wid = store::register_world(None, sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(wid, Capability::Dma));

    revoke::revoke_global(Capability::Dma);
    assert!(!store::world_has_cap(wid, Capability::Dma));

    revoke::restore_global(Capability::Dma);
    assert!(store::world_has_cap(wid, Capability::Dma));
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/tests.rs (line 54) - [internal] test_audit_trail

```
fn test_audit_trail() {
    init().unwrap();

    let wid = store::register_world(None, sets::presets::standard_user()).unwrap();

    store::world_has_cap(wid, Capability::User);
    assert!(audit::count() > 0);
}
```

kernel_workspace/kernel/src/caps/types.rs (line 10) - [public] Capability

```
pub enum Capability {
    Root = 0,
    Ring0 = 1,
    Driver = 2,
    User = 3,
    Admin = 4,

    PhysAlloc = 5,
    VirtMap = 6,
    Dma = 7,
    Mmap = 8,
    Protect = 9,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 99) - [public] iter_all

```
pub fn iter_all() -> impl Iterator<Item = Capability> {
    [
        Self::Root, Self::Ring0, Self::Driver, Self::User, Self::Admin,
        Self::PhysAlloc, Self::VirtMap, Self::Dma, Self::Mmap,
        Self::Protect, Self::HugePage,
        Self::Spawn, Self::Kill, Self::Debug, Self::Ptrace, Self::Sched,
        Self::IpcSend, Self::IpcRecv, Self::IpcBroadcast, Self::IpcPrivileged,
        Self::FsRead, Self::FsWrite, Self::FsCreate, Self::FsMount,
        Self::DevPci, Self::DevPort, Self::DevMmio, Self::DevIrq,
        Self::SyscallAll, Self::SyscallRestricted,
    ].into_iter()
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/caps/types.rs (line 112) - [public] category

```
pub fn category(self) -> CapCategory {
    match self as u8 {
        0..=4 => CapCategory::Ring,
        5..=10 => CapCategory::Memory,
        11..=15 => CapCategory::Process,
        16..=19 => CapCategory::Ipc,
        20..=23 => CapCategory::Fs,
        24..=27 => CapCategory::Device,
        28..=29 => CapCategory::Syscall,
        _ => CapCategory::Unknown,
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 127) - [public] CapCategory

```
pub enum CapCategory {
    Ring,
    Memory,
    Process,
    Ipc,
    Fs,
    Device,
    Syscall,
    Unknown,
}

impl CapCategory {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 154) - [public] CapabilitySet

```
pub struct CapabilitySet {
    bits: u32,
}

impl CapabilitySet {
    pub const fn empty() -> Self {
        Self { bits: 0 }
    }

    pub const fn all() -> Self {
        Self { bits: 0x3FFF_FFFF }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 215) - [public] iter

```
pub fn iter(self) -> impl Iterator<Item = Capability> {
    Capability::iter_all().filter(move |&c| self.has(c))
}

impl fmt::Debug for CapabilitySet {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "CapSet[")?;
        let mut first = true;
        for cap in self.iter() {
            if !first { write!(f, ",")?; }
            write!(f, "{}", cap.name())?;
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 221) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    write!(f, "CapSet[")?;
    let mut first = true;
    for cap in self.iter() {
        if !first { write!(f, ",")?; }
        write!(f, "{}", cap.name())?;
        first = false;
    }
    write!(f, "]")
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 234) - [public] CapabilityError

```
pub struct CapabilityError {
    pub required: Capability,
    pub world_id: Option<u32>,
}

impl fmt::Display for CapabilityError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self.world_id {
            Some(wid) => write!(f, "capability {} required (world {})",
                                self.required.name(), wid),
            None => write!(f, "capability {} required", self.required.name()),
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 240) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self.world_id {
        Some(wid) => write!(f, "capability {} required (world {})",
                            self.required.name(), wid),
        None => write!(f, "capability {} required", self.required.name()),
    }
}

pub type CapResult<T> = Result<T, CapabilityError>;

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 256) - [internal] test_set_operations

```
fn test_set_operations() {
    let s1 = CapabilitySet::empty().add(Capability::Root);
    let s2 = CapabilitySet::empty().add(Capability::Driver);

    assert!(s1.has(Capability::Root));
    assert!(!s1.has(Capability::Driver));
    assert_eq!(s1.union(s2).count(), 2);
    assert_eq!(s1.intersect(s2).count(), 0);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 4) - [public] CpuEntry

```
pub struct CpuEntry {
    pub apic_id: u32,
    pub enabled: bool,
}

pub struct IoApic {
    pub id: u8,
    pub address: u32,
    pub gsi_base: u32,
}

pub struct MadtInfo {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 9) - [public] IoApic

```
pub struct IoApic {
    pub id: u8,
    pub address: u32,
    pub gsi_base: u32,
}

pub struct MadtInfo {
    pub lapic_base: u64,
    pub cpus: Vec<CpuEntry>,
    pub io_apics: Vec<IoApic>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 15) - [public] MadtInfo

```
pub struct MadtInfo {
    pub lapic_base: u64,
    pub cpus: Vec<CpuEntry>,
    pub io_apics: Vec<IoApic>,
}

pub struct FadtInfo {
    pub pm1a_cnt_blk: u32,
}

pub struct Rsdp {
    pub revision: u8,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 21) - [public] FadtInfo

```
pub struct FadtInfo {
    pub pm1a_cnt_blk: u32,
}

pub struct Rsdp {
    pub revision: u8,
    pub rsdt_addr: u32,
    pub xsdt_addr: u64,
}

const RSDP_SIGNATURE: &[u8; 8] = b"RSD PTR ";
const MADT_SIGNATURE: &[u8; 4] = b"APIC";
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 25) - [public] Rsdp

```
pub struct Rsdp {
    pub revision: u8,
    pub rsdt_addr: u32,
    pub xsdt_addr: u64,
}

const RSDP_SIGNATURE: &[u8; 8] = b"RSD PTR ";
const MADT_SIGNATURE: &[u8; 4] = b"APIC";
const FADT_SIGNATURE: &[u8; 4] = b"FACP";

unsafe fn phys_read<T: Copy>(phys_offset: u64, phys: u64) -> T {
    ptr::read_unaligned((phys_offset + phys) as *const T)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 43) - [internal] checksum

```
fn checksum(bytes: &[u8]) -> bool {
    bytes.iter().fold(0u8, |acc, &b| acc.wrapping_add(b)) == 0
}

unsafe fn check_rsdp(phys_offset: u64, addr: u64) -> Option<u64> {
    if phys_slice(phys_offset, addr, 8) != RSDP_SIGNATURE {
        return None;
    }
    if !checksum(phys_slice(phys_offset, addr, 20)) {
        return None;
    }
    Some(addr)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 57) - [public] find_rsdp

```
pub fn find_rsdp(phys_offset: u64) -> Option<u64> {
    unsafe {
        let ebda_seg = u16::from_le_bytes([
            phys_read::<u8>(phys_offset, 0x40E),
            phys_read::<u8>(phys_offset, 0x40F),
        ]);
        if ebda_seg != 0 {
            let ebda = (ebda_seg as u64) << 4;
            let mut addr = ebda;
            while addr < ebda + 0x400 {
                if let Some(rsdp) = check_rsdp(phys_offset, addr) {
                    return Some(rsdp);
                }
            }
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 20) - [internal] x2apic_msr

```
fn x2apic_msr(reg: u32) -> u32 {
    0x800 + (reg >> 4)
}

pub fn init(base_phys: u64) -> bool {
    unsafe {
        let msr = Msr::new(APIC_BASE_MSR).read();
        if msr & (1 << 10) != 0 {
            X2APIC = true;
            return true;
        }
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 24) - [public] init

```
pub fn init(base_phys: u64) -> bool {
    unsafe {
        let msr = Msr::new(APIC_BASE_MSR).read();
        if msr & (1 << 10) != 0 {
            X2APIC = true;
            return true;
        }
    }

    let base = if base_phys >= 0xFFFFF80000000000 {
        base_phys
    } else {
        match crate::mm::virt::map_device(base_phys, 0x1000) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 46) - [public] is_x2apic

```
pub fn is_x2apic() -> bool {
    unsafe { X2APIC }
}

pub fn read(reg: u32) -> u32 {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).read() as u32
        } else {
            ptr::read_volatile((LAPIC_BASE as *const u8).add(reg as usize) as *const u32)
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 50) - [public] read

```
pub fn read(reg: u32) -> u32 {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).read() as u32
        } else {
            ptr::read_volatile((LAPIC_BASE as *const u8).add(reg as usize) as *const u32)
        }
    }
}

pub fn write(reg: u32, val: u32) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 60) - [public] write

```
pub fn write(reg: u32, val: u32) {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).write(val as u64);
        } else {
            ptr::write_volatile((LAPIC_BASE as *mut u8).add(reg as usize) as *mut u32, val);
        }
    }
}

pub fn id() -> u32 {
    if unsafe { X2APIC } {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 70) - [public] id

```
pub fn id() -> u32 {
    if unsafe { X2APIC } {
        read(REG_ID)
    } else {
        read(REG_ID) >> 24
    }
}

pub fn version() -> u32 {
    read(REG_VERSION)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 78) - [public] version

```
pub fn version() -> u32 {
    read(REG_VERSION)
}

pub fn enable_bsp() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 0x700);
}

pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 82) - [public] enable_bsp

```
pub fn enable_bsp() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 0x700);
}

pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    write(REG_LVT_LINT1, 1 << 16);
}
```

```
pub fn eoi() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 87) - [public] enable_ap

```
pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    write(REG_LVT_LINT1, 1 << 16);
}

pub fn eoi() {
    write(REG_EOI, 0);
}

pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 93) - [public] eoi

```
pub fn eoi() {
    write(REG_EOI, 0);
}

pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        if X2APIC {
            let val = ((dest_apic_id as u64) << 32) | icr as u64;
            Msr::new(x2apic_msr(REG_ICR0)).write(val);
        } else {
            write(REG_ICR1, dest_apic_id << 24);
            write(REG_ICR0, icr);
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 97) - [public] send_ipi

```
pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        if X2APIC {
            let val = ((dest_apic_id as u64) << 32) | icr as u64;
            Msr::new(x2apic_msr(REG_ICR0)).write(val);
        } else {
            write(REG_ICR1, dest_apic_id << 24);
            write(REG_ICR0, icr);
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 109) - [public] send_init_ipi

```
pub fn send_init_ipi() {
    send_ipi((5 << 8) | (1 << 14) | (1 << 15) | (3 << 18), 0);

    send_ipi((5 << 8) | (3 << 18), 0);
}

pub fn send_startup_ipi(apic_id: u32, vector: u8) {
    send_ipi((6 << 8) | (vector as u32), apic_id);
}
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 116) - [public] send_startup_ipi

```
pub fn send_startup_ipi(apic_id: u32, vector: u8) {
    send_ipi((6 << 8) | (vector as u32), apic_id);
}
```

kernel_workspace/kernel/src/cpu/mod.rs (line 18) - [public] init_riscv

```
pub fn init_riscv() {
    riscv::init();
}

#[cfg(target_arch = "x86_64")]
pub fn total_cpus() -> u32 {
    smp::total_cpus()
}

#[cfg(target_arch = "riscv64")]
pub fn total_cpus() -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/mod.rs (line 24) - [public] total_cpus

```
pub fn total_cpus() -> u32 {
    smp::total_cpus()
}

#[cfg(target_arch = "riscv64")]
pub fn total_cpus() -> u32 {
    1
}

//.
//.
```

kernel_workspace/kernel/src/cpu/mod.rs (line 29) - [public] total_cpus

```
pub fn total_cpus() -> u32 {
    1
}

//.
//.
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 7) - [public] init

```
pub fn init() {  
  
}  
  
pub fn current_hart() -> u64 {  
    0  
}  
  
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 12) - [public] current_hart

```
pub fn current_hart() -> u64 {  
    0  
}  
  
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
}
```

```
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 17) - [public] poweroff

```
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
}  
  
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
}  
  
fn system_reset(reset_type: u32) -> ! {  
    let err: i64;  
    let _val: i64;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 22) - [public] reboot

```
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
}  
  
fn system_reset(reset_type: u32) -> ! {  
    let err: i64;  
    let _val: i64;  
    unsafe {  
        core::arch::asm!(  
            "ecall",  
            inlateout("a0") reset_type as usize => err,  
            inlateout("a1") 0usize => _val,  
            // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/riscv.rs (line 26) - [internal] system_reset

```
fn system_reset(reset_type: u32) -> ! {
    let err: i64;
    let _val: i64;
    unsafe {
        core::arch::asm!(
            "ecall",
            inlateout("a0") reset_type as usize => err,
            inlateout("a1") 0usize => _val,
            in("a2") 0usize,
            in("a6") SBI_FID_SYSTEM_RESET,
            in("a7") SBI_EID_SRST,
            options(nomem, nostack)
        );
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 639) - [public]
get_switch_count

```
pub fn get_switch_count() -> u64 {
    SWITCH_COUNT.load(Ordering::Relaxed)
}

pub fn get_fpu_switch_count() -> u64 {
    FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
}

pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 643) - [public]
get_fpu_switch_count

```
pub fn get_fpu_switch_count() -> u64 {
    FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
}

pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

pub fn disable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 647) - [public]
enable_lazy_fpu

```
pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

pub fn disable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
}

pub fn is_lazy_fpu_enabled() -> bool {
    LAZY_FPU_ENABLED.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 651) - [public]
disable_lazy_fpu
    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        if !FPU_AVAILABLE {
            return;
        }
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 655) - [public]
is_lazy_fpu_enabled
    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        if !FPU_AVAILABLE {
            return;
        }

        enable_fpu();

        if !LAST_FPU_TASK.is_null() {
            // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 692) - [public]
satp_mode_name
    pub fn satp_mode_name(satp: u64) -> &'static str {
        let mode = (satp >> SATP_MODE_SHIFT) & 0xF;
        match mode {
            SATP_MODE_BARE => "Bare",
            SATP_MODE_SV39 => "Sv39",
            SATP_MODE_SV48 => "Sv48",
            SATP_MODE_SV57 => "Sv57",
            _ => "Unknown",
        }
    }

    pub fn satp_asid(satp: u64) -> u64 {
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 703) - [public]
satp_asid
    pub fn satp_asid(satp: u64) -> u64 {
        (satp >> 44) & ((1 << SATP_ASID_BITS) - 1)
    }

    pub fn satp_ppn(satp: u64) -> u64 {
        satp & SATP_PPN_MASK
    }

#[cfg(test)]
mod tests {
    use super::*;

    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 707) - [public]

satp_ppn

```
pub fn satp_ppn(satp: u64) -> u64 {
    satp & SATP_PPN_MASK
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_fpu_save_area_size() {
        assert_eq!(FPU_SAVE_SIZE, 256);
        assert!(FPU_SAVE_SIZE + 8 <= 512);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 716) - [internal]

test_fpu_save_area_size

```
fn test_fpu_save_area_size() {
    assert_eq!(FPU_SAVE_SIZE, 256);
    assert!(FPU_SAVE_SIZE + 8 <= 512);
}

#[test]
fn test_satp_constants() {
    assert_eq!(SATP_MODE_SHIFT, 60);
    assert_eq!(SATP_ASID_BITS, 16);
    assert_eq!(SATP_MODE_SV39, 8);
    assert_eq!(SATP_MODE_SV48, 9);
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 722) - [internal]

test_satp_constants

```
fn test_satp_constants() {
    assert_eq!(SATP_MODE_SHIFT, 60);
    assert_eq!(SATP_ASID_BITS, 16);
    assert_eq!(SATP_MODE_SV39, 8);
    assert_eq!(SATP_MODE_SV48, 9);
}

#[test]
fn test_sstatus_fs_mask() {
    assert_eq!(SSTATUS_FS_MASK, 0x3 << 13);
    assert_eq!(SSTATUS_FS_OFF, 0);
    assert_eq!(SSTATUS_FS_INITIAL, 1);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 730) - [internal]

test_sstatus_fs_mask

```
fn test_sstatus_fs_mask() {
    assert_eq!(SSTATUS_FS_MASK, 0x3 << 13);
    assert_eq!(SSTATUS_FS_OFF, 0);
    assert_eq!(SSTATUS_FS_INITIAL, 1);
    assert_eq!(SSTATUS_FS_CLEAN, 2);
    assert_eq!(SSTATUS_FS_DIRTY, 3);
}

#[test]
fn test_satp_mode_name() {
    assert_eq!(satp_mode_name(SATP_MODE_BARE << SATP_MODE_SHIFT), "Bare");
    assert_eq!(satp_mode_name(SATP_MODE_SV39 << SATP_MODE_SHIFT), "Sv39");
    // ... (truncated, see source file for more)
}
```

```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 739) - [internal]
test_satp_mode_name
    fn test_satp_mode_name() {
        assert_eq!(satp_mode_name(SATP_MODE_BARE << SATP_MODE_SHIFT), "Bare");
        assert_eq!(satp_mode_name(SATP_MODE_SV39 << SATP_MODE_SHIFT), "Sv39");
        assert_eq!(satp_mode_name(SATP_MODE_SV48 << SATP_MODE_SHIFT), "Sv48");
        assert_eq!(satp_mode_name(SATP_MODE_SV57 << SATP_MODE_SHIFT), "Sv57");
    }
}

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 45) - [internal]
XSaveArea
    struct XSaveArea {
        data: [u8; XSAVE_AREA_SIZE],
    }

    impl XSaveArea {
        const fn new() -> Self {
            Self { data: [0; XSAVE_AREA_SIZE] }
        }
    }

    #[repr(C)]
    struct ExtendedContext {
        // ... (truncated, see source file for more)
    }

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 56) - [internal]
ExtendedContext
    struct ExtendedContext {
        xsave_area: XSaveArea,
        xcomp_bv: u64,
        header_bv: u64,
    }

    #[inline(always)]
    unsafe fn read_cr3() -> u64 {
        let cr3: u64;
        asm!("mov {0}, cr3", out(reg) cr3, options(nomem, nostack, preserves_flags));
        cr3
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 458) - [internal]
TssStruct
    struct TssStruct {
        reserved0: u32,
        rsp0: u64,
        rsp1: u64,
        rsp2: u64,
        reserved1: u64,
        ist1: u64,
        ist2: u64,
        ist3: u64,
        ist4: u64,
        ist5: u64,
        ist6: u64,
        // ... (truncated, see source file for more)
    }

```

```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 731) - [public]
get_switch_count
    pub fn get_switch_count() -> u64 {
        SWITCH_COUNT.load(Ordering::Relaxed)
    }

    pub fn get_fpu_switch_count() -> u64 {
        FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
    }

    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 735) - [public]
get_fpu_switch_count
    pub fn get_fpu_switch_count() -> u64 {
        FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
    }

    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 739) - [public]
enable_lazy_fpu
    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 743) - [public]
disable_lazy_fpu
    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        clts();

        if !LAST_FPU_TASK.is_null() {
            // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 747) - [public]
is_lazy_fpu_enabled

```
pub fn is_lazy_fpu_enabled() -> bool {
    LAZY_FPU_ENABLED.load(Ordering::Relaxed)
}

pub unsafe fn kernel_fpu_begin() {
    clts();

    if !LAST_FPU_TASK.is_null() {
        save_fpu_state(LAST_FPU_TASK);
        LAST_FPU_TASK = core::ptr::null_mut();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 769) - [internal]
test_xsave_area_alignment

```
fn test_xsave_area_alignment() {
    assert_eq!(core::mem::align_of::<XSaveArea>(), 64);
}

#[test]
fn test_msr_constants() {
    assert_eq!(MSR_FS_BASE, 0xC0000100);
    assert_eq!(MSR_GS_BASE, 0xC0000101);
    assert_eq!(MSR_KERNEL_GS_BASE, 0xC0000102);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 774) - [internal]
test_msr_constants

```
fn test_msr_constants() {
    assert_eq!(MSR_FS_BASE, 0xC0000100);
    assert_eq!(MSR_GS_BASE, 0xC0000101);
    assert_eq!(MSR_KERNEL_GS_BASE, 0xC0000102);
}

#[test]
fn test_cr3_masks() {
    assert_eq!(CR3_PCID_MASK, 0xFFF);
    assert_eq!(CR3_NO_FLUSH, 1 << 63);
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 781) - [internal]
test_cr3_masks

```
fn test_cr3_masks() {
    assert_eq!(CR3_PCID_MASK, 0xFFF);
    assert_eq!(CR3_NO_FLUSH, 1 << 63);
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 49) - [public] register_runqueue

```
pub fn register_runqueue(cpu: u32, rq: *mut RunQueue) {
    if (cpu as usize) < MAX_CPUS {
        RQ_REGISTRY[cpu as usize].store(rq, Ordering::Release);
    }
}

pub fn register_apic_id(cpu: u32, apic_id: u32) {
    if (cpu as usize) < MAX_CPUS {
        APIC_ID_TABLE[cpu as usize].store(apic_id, Ordering::Release);
    }
}

// ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 55) - [public] register_apic_id
pub fn register_apic_id(cpu: u32, apic_id: u32) {
    if (cpu as usize) < MAX_CPUS {
        APIC_ID_TABLE[cpu as usize].store(apic_id, Ordering::Release);
    }
}

pub fn snapshot_registry() -> [*mut RunQueue; MAX_CPUS] {
    let mut out = [ptr::null_mut(); MAX_CPUS];
    for i in 0..MAX_CPUS {
        out[i] = RQ_REGISTRY[i].load(Ordering::Acquire);
    }
    out
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 61) - [public] snapshot_registry
pub fn snapshot_registry() -> [*mut RunQueue; MAX_CPUS] {
    let mut out = [ptr::null_mut(); MAX_CPUS];
    for i in 0..MAX_CPUS {
        out[i] = RQ_REGISTRY[i].load(Ordering::Acquire);
    }
    out
}

fn cpu_to_apic_id(cpu: u32) -> Option<u32> {
    if (cpu as usize) >= MAX_CPUS {
        return None;
    }
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 69) - [internal] cpu_to_apic_id
fn cpu_to_apic_id(cpu: u32) -> Option<u32> {
    if (cpu as usize) >= MAX_CPUS {
        return None;
    }
    let id = APIC_ID_TABLE[cpu as usize].load(Ordering::Acquire);
    if id == u32::MAX {
        None
    } else {
        Some(id)
    }
}

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 81) - [public] set_tsc_frequency
pub fn set_tsc_frequency(hz: u64) {
    TSC_HZ.store(hz, Ordering::Release);
}

pub fn tsc_frequency() -> u64 {
    TSC_HZ.load(Ordering::Acquire)
}

pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 85) - [public] tsc_frequency

```
pub fn tsc_frequency() -> u64 {
    TSC_HZ.load(Ordering::Acquire)
}

pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// -----
// Port I/O – uprzywilejowane, więc wyliczenie prawdziwe na jędrzym x86_64
// -----

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 89) - [public] booted_cpu_count

```
pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// -----
// Port I/O – uprzywilejowane, więc wyliczenie prawdziwe na jędrzym x86_64
// -----

#[cfg(all(target_arch = "x86_64", not(test)))]
unsafe fn out_u8(port: u16, val: u8) {
    core::arch::asm!("out dx, al", in("dx") port, in("al") val, options(nomem, nostack, preserves_flags))
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 175) - [internal] ns_from_tsc

```
fn ns_from_tsc(tsc_delta: u64, hz: u64) -> u64 {
    if hz == 0 {
        0
    } else {
        ((tsc_delta as u128) * 1_000_000_000u128 / hz as u128) as u64
    }
}

pub fn now_ns() -> u64 {
    ns_from_tsc(unsafe { rdtsc() }, tsc_frequency())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 183) - [public] now_ns

```
pub fn now_ns() -> u64 {
    ns_from_tsc(unsafe { rdtsc() }, tsc_frequency())
}

/// Kalibruje TSC względem kanału 2 PIT (jak w prawdziwym jądrze – PIT
/// jest wolniejszy, ale zawsze obecny). Wymaga wyliczonych przerwań.
pub unsafe fn calibrate_tsc_via_pit(calib_ms: u32) -> u64 {
    const PIT_GATE_PORT: u16 = 0x61;
    const PIT_CMD_PORT: u16 = 0x43;
    const PIT_CH2_DATA_PORT: u16 = 0x42;

    let count = ((PIT_FREQUENCY_HZ as u64 * calib_ms as u64) / 1000) as u16;
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 217) - [internal]

timer_count_for_hz

```
fn timer_count_for_hz(hz: u64) -> u32 {
    if hz == 0 {
        1_000_000
    } else {
        cmp_max_1((hz / TICK_HZ) as u32)
    }
}

fn cmp_max_1(v: u32) -> u32 {
    if v == 0 {
        1
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 225) - [internal] cmp_max_1

```
fn cmp_max_1(v: u32) -> u32 {
    if v == 0 {
        1
    } else {
        v
    }
}

fn default_timer_count() -> u32 {
    timer_count_for_hz(tsc_frequency())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 233) - [internal]

default_timer_count

```
fn default_timer_count() -> u32 {
    timer_count_for_hz(tsc_frequency())
}

// -----
// Local APIC MMIO – nieodzwzorowane w przestrzeni testowej, więc stub
// dla `test`/architektur innych niż x86_64
// -----

#[cfg(all(target_arch = "x86_64", not(test)))]
unsafe fn apic_write(reg: usize, value: u32) {
    ptr::write_volatile((LOCAL_APIC_BASE + reg) as *mut u32, value);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 270) - [public] current_cpu_id

```
pub fn current_cpu_id() -> u32 {
    unsafe {
        let apic_id = apic_read(APIC_REG_ID) >> 24;
        for cpu in 0..MAX_CPUS as u32 {
            if cpu_to_apic_id(cpu) == Some(apic_id) {
                return cpu;
            }
        }
        CPU_NONE
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 338) - [public] IdtEntry

```
pub struct IdtEntry {
    offset_low: u16,
    selector: u16,
    ist: u8,
    type_attr: u8,
    offset_mid: u16,
    offset_high: u32,
    zero: u32,
}

impl IdtEntry {
    pub const fn missing() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 353) - [public] set

```
pub fn set(&mut self, handler: usize, selector: u16, ist: u8, type_attr: u8) {
    self.offset_low = (handler & 0xFFFF) as u16;
    self.offset_mid = ((handler >> 16) & 0xFFFF) as u16;
    self.offset_high = ((handler >> 32) & 0xFFFF_FFFF) as u32;
    self.selector = selector;
    self.ist = ist & 0x7;
    self.type_attr = type_attr;
    self.zero = 0;
}

pub fn is_present(&self) -> bool {
    self.type_attr & 0x80 != 0
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 363) - [public] is_present

```
pub fn is_present(&self) -> bool {
    self.type_attr & 0x80 != 0
}

#[repr(C, packed)]
struct IdtDescriptor {
    limit: u16,
    base: u64,
}
```

```
static mut IDT: [IdtEntry; IDT_SIZE] = [IdtEntry::missing(); IDT_SIZE];
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 369) - [internal] IdtDescriptor

```
struct IdtDescriptor {
    limit: u16,
    base: u64,
}

static mut IDT: [IdtEntry; IDT_SIZE] = [IdtEntry::missing(); IDT_SIZE];

pub unsafe fn install_gate(vector: u8, handler: usize, selector: u16, ist: u8, type_attr: u8) {
    let entry_ptr = ptr::addr_of_mut!(IDT[vector as usize]);
    (*entry_ptr).set(handler, selector, ist, type_attr);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 437) - [internal]

arch_context_switch

```
fn arch_context_switch(prev_ctx: *mut CpuContext, next_ctx: *const CpuContext);
}

pub unsafe fn context_switch(rq: &mut RunQueue) {
    let flags = rq.lock.lock_irqsave();
    let (prev, next) = rq.schedule();
    rq.lock.unlock_irqrestore(flags);

    if core::ptr::eq(prev, next) {
        return;
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 462) - [internal] maybe_balance

```
fn maybe_balance(cpu: u32, registry: &[*mut RunQueue; MAX_CPUS]) {
    let rq_ptr = registry[cpu as usize];
    if rq_ptr.is_null() {
        return;
    }
    unsafe {
        let rq = &*rq_ptr;
        let now = rq.clock.load(Ordering::Relaxed);
        let next = rq.next_balance.load(Ordering::Relaxed);
        if now >= next {
            rq.next_balance.store(now + BALANCE_INTERVAL_NS, Ordering::Relaxed);
            smp::load_balance(registry);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 587) - [internal]

registry_defaults_to_null_for_untouched_cpu

```
fn registry_defaults_to_null_for_untouched_cpu() {
    let snap = snapshot_registry();
    assert!(snap[MAX_CPUS - 1].is_null());
}

#[test]
fn apic_id_table_defaults_to_none_then_roundtrips() {
    assert_eq!(cpu_to_apic_id(2), None);
    register_apic_id(2, 9);
    assert_eq!(cpu_to_apic_id(2), Some(9));
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 593) - [internal]

apic_id_table_defaults_to_none_then_roundtrips

```
fn apic_id_table_defaults_to_none_then_roundtrips() {
    assert_eq!(cpu_to_apic_id(2), None);
    register_apic_id(2, 9);
    assert_eq!(cpu_to_apic_id(2), Some(9));
}

#[test]
fn register_runqueue_ignores_out_of_range_cpu() {
    register_runqueue(MAX_CPUS as u32 + 10, ptr::null_mut());
}

#[test]
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 600) - [internal]
register_runqueue_ignores_out_of_range_cpu

    fn register_runqueue_ignores_out_of_range_cpu() {
        register_runqueue(MAX_CPUS as u32 + 10, ptr::null_mut());
    }

#[test]
fn ns_from_tsc_is_zero_without_calibration() {
    assert_eq!(ns_from_tsc(1_000_000, 0), 0);
}

#[test]
fn ns_from_tsc_scales_correctly() {
    // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 605) - [internal]
ns_from_tsc_is_zero_without_calibration

    fn ns_from_tsc_is_zero_without_calibration() {
        assert_eq!(ns_from_tsc(1_000_000, 0), 0);
    }

#[test]
fn ns_from_tsc_scales_correctly() {
    // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
    assert_eq!(ns_from_tsc(1_000_000, 1_000_000_000), 1_000_000);
    // 2 GHz zegar: te same cykle to po nowa czasu.
    assert_eq!(ns_from_tsc(1_000_000, 2_000_000_000), 500_000);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 610) - [internal]
ns_from_tsc_scales_correctly

    fn ns_from_tsc_scales_correctly() {
        // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
        assert_eq!(ns_from_tsc(1_000_000, 1_000_000_000), 1_000_000);
        // 2 GHz zegar: te same cykle to po nowa czasu.
        assert_eq!(ns_from_tsc(1_000_000, 2_000_000_000), 500_000);
    }

#[test]
fn timer_count_for_hz_has_sane_fallback() {
    assert_eq!(timer_count_for_hz(0), 1_000_000);
    assert_eq!(timer_count_for_hz(1_000_000_000), 1_000_000);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 618) - [internal]
timer_count_for_hz_has_sane_fallback

    fn timer_count_for_hz_has_sane_fallback() {
        assert_eq!(timer_count_for_hz(0), 1_000_000);
        assert_eq!(timer_count_for_hz(1_000_000_000), 1_000_000);
    }

#[test]
fn timer_count_for_hz_never_returns_zero() {
    assert_eq!(timer_count_for_hz(TICK_HZ / 2), 1);
}

#[test]
fn tlb_pending_flag_sets_and_clears() {
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 624) - [internal]

timer_count_for_hz_never_returns_zero

```
fn timer_count_for_hz_never_returns_zero() {
    assert_eq!(timer_count_for_hz(TICK_HZ / 2), 1);
}

#[test]
fn tlb_pending_flag_sets_and_clears() {
    let idx = MAX_CPUS - 2;
    TLB_SHOOTDOWN_PENDING[idx].store(true, Ordering::Release);
    assert!(TLB_SHOOTDOWN_PENDING[idx].swap(false, Ordering::AcqRel));
    assert!(TLB_SHOOTDOWN_PENDING[idx].load(Ordering::Acquire));
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 629) - [internal]

tlb_pending_flag_sets_and_clears

```
fn tlb_pending_flag_sets_and_clears() {
    let idx = MAX_CPUS - 2;
    TLB_SHOOTDOWN_PENDING[idx].store(true, Ordering::Release);
    assert!(TLB_SHOOTDOWN_PENDING[idx].swap(false, Ordering::AcqRel));
    assert!(TLB_SHOOTDOWN_PENDING[idx].load(Ordering::Acquire));
}

#[test]
fn idt_entry_encodes_and_reports_present() {
    let mut e = IdtEntry::missing();
    assert!(e.is_present());
    e.set(0xDEAD_BEEF_1234, 0x08, 1, GATE_INTERRUPT);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 637) - [internal]

idt_entry_encodes_and_reports_present

```
fn idt_entry_encodes_and_reports_present() {
    let mut e = IdtEntry::missing();
    assert!(e.is_present());
    e.set(0xDEAD_BEEF_1234, 0x08, 1, GATE_INTERRUPT);
    assert!(e.is_present());
}

#[test]
fn idt_entry_missing_is_not_present() {
    assert!(IdtEntry::missing().is_present());
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 645) - [internal]

idt_entry_missing_is_not_present

```
fn idt_entry_missing_is_not_present() {
    assert!(IdtEntry::missing().is_present());
}

#[cfg(target_arch = "x86_64")]
#[test]
fn cpuid_leaf_zero_reports_a_nonzero_max_leaf() {
    unsafe {
        let (eax, _, _, _) = cpuid(0);
        assert!(eax >= 1);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 651) - [internal]

cpuid_leaf_zero_reports_a_nonzero_max_leaf

```
fn cpuid_leaf_zero_reports_a_nonzero_max_leaf() {
    unsafe {
        let (eax, _, _, _) = cpuid(0);
        assert!(eax >= 1);
    }
}

#[cfg(target_arch = "x86_64")]
#[test]
fn rdtsc_does_not_go_backwards_across_two_reads() {
    unsafe {
        let a = rdtsc();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 660) - [internal]

rdtsc_does_not_go_backwards_across_two_reads

```
fn rdtsc_does_not_go_backwards_across_two_reads() {
    unsafe {
        let a = rdtsc();
        let b = rdtsc();
        assert!(b >= a);
    }
}

#[cfg(not(target_arch = "x86_64"))]
#[test]
fn cpuid_and_rdtsc_are_zero_off_x86_64() {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 670) - [internal]

cpuid_and_rdtsc_are_zero_off_x86_64

```
fn cpuid_and_rdtsc_are_zero_off_x86_64() {
    unsafe {
        assert_eq!(cpuid(0), (0, 0, 0, 0));
        assert_eq!(rdtsc(), 0);
    }
}

#[test]
fn set_and_read_tsc_frequency_is_stable_within_this_test() {
    set_tsc_frequency(3_000_000_000);
    assert_eq!(tsc_frequency(), 3_000_000_000);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 678) - [internal]

set_and_read_tsc_frequency_is_stable_within_this_test

```
fn set_and_read_tsc_frequency_is_stable_within_this_test() {
    set_tsc_frequency(3_000_000_000);
    assert_eq!(tsc_frequency(), 3_000_000_000);
}

#[test]
fn booted_cpu_count_starts_at_or_above_zero() {
    assert!(booted_cpu_count() < u32::MAX);
}
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 684) - [internal]

booted_cpu_count_starts_at_or_above_zero

```
fn booted_cpu_count_starts_at_or_above_zero() {
    assert!(booted_cpu_count() < u32::MAX);
}
}
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 12) - [public] SchedClassOps

```
pub struct SchedClassOps {
    pub name: &'static str,
    pub class: SchedClass,
    pub next: Option<&'static SchedClassOps>,

    pub enqueue_task: unsafe fn(&mut RunQueue, *mut TaskStruct, EnqueueFlags),
    pub dequeue_task: unsafe fn(&mut RunQueue, *mut TaskStruct, DequeueFlags),
    pub yield_task: unsafe fn(&mut RunQueue, *mut TaskStruct),
    pub pick_next_task: unsafe fn(&mut RunQueue) -> *mut TaskStruct,
    pub check_preempt_curr: unsafe fn(&RunQueue, *mut TaskStruct) -> bool,
    pub task_tick: unsafe fn(&mut RunQueue, *mut TaskStruct),
    pub charge_exec: unsafe fn(&mut RunQueue, *mut TaskStruct, u64, u64),
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 336) - [public] class_of

```
// -----
// Dispatch
// -----

pub fn class_of(class: SchedClass) -> &'static SchedClassOps {
    match class {
        SchedClass::Stop => &STOP_SCHED_CLASS,
        SchedClass::Deadline => &DL_SCHED_CLASS,
        SchedClass::RealTime => &RT_SCHED_CLASS,
        SchedClass::Fair => &FAIR_SCHED_CLASS,
        SchedClass::Idle => &IDLE_SCHED_CLASS,
    }
}

pub unsafe fn pick_next(rq: &mut RunQueue) -> *mut TaskStruct {
    let mut cursor: &'static SchedClassOps = &STOP_SCHED_CLASS;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 514) - [internal] make_idle

```
fn make_idle(pid: TaskId) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 520) - [internal] make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 526) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    assert_eq!(STOP_SCHED_CLASS.class, SchedClass::Stop);
    assert!(core::ptr::eq(STOP_SCHED_CLASS.next.unwrap(), &DL_SCHED_CLASS));
    assert_eq!(DL_SCHED_CLASS.class, SchedClass::Deadline);
    assert!(core::ptr::eq(DL_SCHED_CLASS.next.unwrap(), &RT_SCHED_CLASS));
    assert_eq!(RT_SCHED_CLASS.class, SchedClass::RealTime);
    assert!(core::ptr::eq(RT_SCHED_CLASS.next.unwrap(), &FAIR_SCHED_CLASS));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 531) - [internal]

chain_is_ordered_stop_dl_rt_fair_idle

```
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    assert_eq!(STOP_SCHED_CLASS.class, SchedClass::Stop);
    assert!(core::ptr::eq(STOP_SCHED_CLASS.next.unwrap(), &DL_SCHED_CLASS));
    assert_eq!(DL_SCHED_CLASS.class, SchedClass::Deadline);
    assert!(core::ptr::eq(DL_SCHED_CLASS.next.unwrap(), &RT_SCHED_CLASS));
    assert_eq!(RT_SCHED_CLASS.class, SchedClass::RealTime);
    assert!(core::ptr::eq(RT_SCHED_CLASS.next.unwrap(), &FAIR_SCHED_CLASS));
    assert_eq!(FAIR_SCHED_CLASS.class, SchedClass::Fair);
    assert!(core::ptr::eq(FAIR_SCHED_CLASS.next.unwrap(), &IDLE_SCHED_CLASS));
    assert_eq!(IDLE_SCHED_CLASS.class, SchedClass::Idle);
    assert!(IDLE_SCHED_CLASS.next.is_none());
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 545) - [internal]

class_of_maps_every_variant_correctly

```
fn class_of_maps_every_variant_correctly() {
    assert!(core::ptr::eq(class_of(SchedClass::Stop), &STOP_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Deadline), &DL_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::RealTime), &RT_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Fair), &FAIR_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Idle), &IDLE_SCHED_CLASS));
}

#[test]
fn pick_next_walks_the_chain_in_priority_order() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 554) - [internal]

pick_next_walks_the_chain_in_priority_order

```
fn pick_next_walks_the_chain_in_priority_order() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
    rt_task.prio = 10;
    let mut dl_task = make_task(3, SchedPolicy::Deadline, 0);
    dl_task.dl.dl_runtime = 1000;
    dl_task.dl.dl_deadline = 100_000;
    dl_task.dl.dl_period = 100_000;
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 596) - [internal]

rt_get_rr_interval_is_nonzero_only_for_round_robin

```
fn rt_get_rr_interval_is_nonzero_only_for_round_robin() {
    let fifo = make_task(1, SchedPolicy::Fifo, 0);
    let rr = make_task(2, SchedPolicy::RoundRobin, 0);
    unsafe {
        assert_eq!(get_rr_interval(&fifo as *const _), 0);
        assert!(get_rr_interval(&rr as *const _) > 0);
    }
}

#[test]
fn fair_and_deadline_and_idle_have_zero_rr_interval() {
    let fair = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 606) - [internal]

fair_and_deadline_and_idle_have_zero_rr_interval

```
fn fair_and_deadline_and_idle_have_zero_rr_interval() {
    let fair = make_task(1, SchedPolicy::Normal, 0);
    let dl = make_task(2, SchedPolicy::Deadline, 0);
    let idle = make_idle(3);
    unsafe {
        assert_eq!(get_rr_interval(&fair as *const _), 0);
        assert_eq!(get_rr_interval(&dl as *const _), 0);
        assert_eq!(get_rr_interval(&idle as *const _), 0);
    }
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 618) - [internal]

check_preempt_cross_class_ignores_intra_class_ops

```
fn check_preempt_cross_class_ignores_intra_class_ops() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
    rt_task.prio = 5;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 639) - [internal]

switched_to_reschedules_when_new_class_outranks_current

```
fn switched_to_reschedules_when_new_class_outranks_current() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut promoted = make_task(2, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fair_task));
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 664) - [internal]
change_task_class_moves_task_between_underlying_queues

```
fn change_task_class_moves_task_between_underlying_queues() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        assert_eq!(rq.fair.nr_running, 1);
        assert_eq!(rq.rt.nr_running, 0);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 688) - [internal]
change_task_class_on_non_queued_task_does_not_touch_queues

```
fn change_task_class_on_non_queued_task_does_not_touch_queues() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        change_task_class(&mut rq, ptr_of(&mut t), SchedPolicy::Fifo);
        assert_eq!(rq.fair.nr_running, 0);
        assert_eq!(rq.rt.nr_running, 0);
        assert!(!(*ptr_of(&mut t)).se.on_rq);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 706) - [internal]
rt_task_tick_only_requeues_round_robin_on_slice_expiry

```
fn rt_task_tick_only_requeues_round_robin_on_slice_expiry() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fifo_task = make_task(1, SchedPolicy::Fifo, 0);
    fifo_task.prio = 10;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fifo_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fifo_task));
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 727) - [internal]
rt_task_tick_marks_resched_when_round_robin_slice_expires

```
fn rt_task_tick_marks_resched_when_round_robin_slice_expires() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut rr_task = make_task(1, SchedPolicy::RoundRobin, 0);
    rr_task.prio = 10;
    rr_task.rt.time_slice = 0;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut rr_task), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 750) - [internal]

dl_task_tick_dequeues_when_throttled

```
fn dl_task_tick_dequeues_when_throttled() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 1000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 776) - [internal]

fair_charge_dispatches_to_underlying_rqfair

```
fn fair_charge_dispatches_to_underlying_rqfair() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        let before = (*ptr_of(&mut t)).se.vruntime;
        charge(&mut rq, ptr_of(&mut t), 1_000_000, 1_000_000);
        assert!((*ptr_of(&mut t)).se.vruntime > before);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 27) - [public] Bitmap

```
pub struct Bitmap<const WORDS: usize> {
    words: [u64; WORDS],
}

impl<const WORDS: usize> Bitmap<WORDS> {
    pub const CAPACITY: usize = WORDS * BITS_PER_WORD;

    pub const fn new() -> Self {
        Self { words: [0; WORDS] }
    }

    pub const fn filled() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 43) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

#[inline]
pub fn clear(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 50) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

#[inline]
pub fn toggle(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] ^= mask_of(bit);
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 57) - [public] toggle

```
pub fn toggle(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] ^= mask_of(bit);
    }
}

#[inline]
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn set_range(&mut self, start: usize, end: usize) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 64) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn set_range(&mut self, start: usize, end: usize) {
    let end = end.min(Self::CAPACITY);
    let mut bit = start;
    while bit < end {
        self.set(bit);
        bit += 1;
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 68) - [public] set_range

```
pub fn set_range(&mut self, start: usize, end: usize) {
    let end = end.min(Self::CAPACITY);
    let mut bit = start;
    while bit < end {
        self.set(bit);
        bit += 1;
    }
}

pub fn is_empty(&self) -> bool {
    self.words.iter().all(|w| *w == 0)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 77) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.words.iter().all(|w| *w == 0)
}

pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| *w == u64::MAX)
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 81) - [public] is_full

```
pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| *w == u64::MAX)
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 85) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 89) - [public] find_first_set

```
pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 98) - [public]

find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < Self::CAPACITY {
                return Some(bit);
            }
        }
        return None;
    }
}
None
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 111) - [public]

find_next_set

```
pub fn find_next_set(&self, after: usize) -> Option<usize> {
    let mut bit = after + 1;
    while bit < Self::CAPACITY {
        let idx = word_of(bit);
        let shift = bit % BITS_PER_WORD;
        let masked = self.words[idx] & (u64::MAX << shift);
        if masked != 0 {
            let found = idx * BITS_PER_WORD + masked.trailing_zeros() as usize;
            return if found < Self::CAPACITY { Some(found) } else { None };
        }
        bit = (idx + 1) * BITS_PER_WORD;
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 126) - [public] and

```
pub fn and(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & other.words[i];
    }
    out
}

pub fn or(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] | other.words[i];
    }
    out
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 134) - [public] or

```
pub fn or(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] | other.words[i];
    }
    out
}

pub fn xor(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] ^ other.words[i];
    }
    out
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 142) - [public] xor

```
pub fn xor(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] ^ other.words[i];
    }
    out
}

pub fn andnot(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & !other.words[i];
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 150) - [public] andnot

```
pub fn andnot(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & !other.words[i];
    }
    out
}

pub fn intersects(&self, other: &Self) -> bool {
    self.words.iter().zip(other.words.iter()).any(|(a, b)| a & b != 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 158) - [public] intersects

```
pub fn intersects(&self, other: &Self) -> bool {
    self.words.iter().zip(other.words.iter()).any(|(a, b)| a & b != 0)
}

pub fn iter(&self) -> BitmapIter<'_> {
    BitmapIter { words: &self.words, next_bit: 0, capacity: Self::CAPACITY }
}

impl<const WORDS: usize> Default for Bitmap<WORDS> {
    fn default() -> Self {
        Self::new()
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 162) - [public] iter

```
pub fn iter(&self) -> BitmapIter<'_> {
    BitmapIter { words: &self.words, next_bit: 0, capacity: Self::CAPACITY }
}

impl<const WORDS: usize> Default for Bitmap<WORDS> {
    fn default() -> Self {
        Self::new()
    }
}

pub struct BitmapIter<'a> {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 168) - [internal]

default

```
    fn default() -> Self {
        Self::new()
    }
}

pub struct BitmapIter<'a> {
    words: &'a [u64],
    next_bit: usize,
    capacity: usize,
}

impl<'a> Iterator for BitmapIter<'a> {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 173) - [public]

BitmapIter

```
pub struct BitmapIter<'a> {
    words: &'a [u64],
    next_bit: usize,
    capacity: usize,
}

impl<'a> Iterator for BitmapIter<'a> {
    type Item = usize;

    fn next(&mut self) -> Option<usize> {
        while self.next_bit < self.capacity {
            let bit = self.next_bit;
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 182) - [internal] next

```
    fn next(&mut self) -> Option<usize> {
        while self.next_bit < self.capacity {
            let bit = self.next_bit;
            self.next_bit += 1;
            if self.words[word_of(bit)] & mask_of(bit) != 0 {
                return Some(bit);
            }
        }
        None
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 200) - [public]

BitmapSlice

```
// -----
// BitmapSlice — widok nad `&mut [u64]` o rozmiarze nieznanym w
// czasie kompilacji (np. maski o rozmiarze zależnym od liczby CPU
// wykrytej w ACPI/MADT).
// -----

pub struct BitmapSlice<'a> {
    words: &'a mut [u64],
    nbits: usize,
}

impl<'a> BitmapSlice<'a> {
    pub fn new(words: &'a mut [u64], nbits: usize) -> Self {
        debug_assert!(words.len() >= words_for_bits(nbits));
        Self { words, nbits }
    }

    pub fn capacity(&self) -> usize {
        // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 206) - [public] new

```
pub fn new(words: &'a mut [u64], nbits: usize) -> Self {
    debug_assert!(words.len() >= words_for_bits(nbits));
    Self { words, nbits }
}

pub fn capacity(&self) -> usize {
    self.nbits
}

pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 211) - [public] capacity

```
pub fn capacity(&self) -> usize {
    self.nbits
}

pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 215) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 221) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

pub fn test(&self, bit: usize) -> bool {
    bit < self.nbits && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 227) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < self.nbits && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 231) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < self.nbits {
                return Some(bit);
            }
        }
        return None;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 235) - [public]
find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < self.nbits {
                return Some(bit);
            }
        }
        return None;
    }
}
None
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 248) - [public]
clear_all

```
pub fn clear_all(&mut self) {
    for w in self.words.iter_mut() {
        *w = 0;
    }
}

// -----
// AtomicBitmap<WORDS> – bezpieczna współbieżnie mapa bitowa, w tym
// bez-blokadowy przydział pierwszego wolnego bitu (test-and-set przez
// CAS) – używana m.in. przez alokator PID w `core.rs`.
// -----
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 263) - [public]

AtomicBitmap

```
pub struct AtomicBitmap<const WORDS: usize> {
    words: [AtomicU64; WORDS],
}

impl<const WORDS: usize> AtomicBitmap<WORDS> {
    pub const CAPACITY: usize = WORDS * BITS_PER_WORD;

    pub const fn new() -> Self {
        Self { words: [ZERO_WORD; WORDS] }
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 275) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)].load(Ordering::Acquire) & mask_of(bit) != 0
}

/// Ustawia bit atomowo, zwraca POPRZEDNI stan (true = już by ustawiony).
pub fn test_and_set(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return true;
    }
    let prev = self.words[word_of(bit)].fetch_or(mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 280) - [public]

test_and_set

/// Ustawia bit atomowo, zwraca POPRZEDNI stan (true = już by ustawiony).

```
pub fn test_and_set(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return true;
    }
    let prev = self.words[word_of(bit)].fetch_or(mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}

/// Czy ci bit atomowo, zwraca POPRZEDNI stan (true = by ustawiony).
pub fn test_and_clear(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 289) - [public]

test_and_clear

/// Czy ci bit atomowo, zwraca POPRZEDNI stan (true = by ustawiony).

```
pub fn test_and_clear(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return false;
    }
    let prev = self.words[word_of(bit)].fetch_and(!mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.load(Ordering::Relaxed).count_ones()).sum()
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 297) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.load(Ordering::Relaxed).count_ones()).sum()
}

pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| w.load(Ordering::Relaxed) == u64::MAX)
}

/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny wzięło ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 301) - [public] is_full

```
pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| w.load(Ordering::Relaxed) == u64::MAX)
}

/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny wzięło ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// przechodzi dalej, żeby nie pominąć bitów zwolnionych w
/// międzyczasie.
pub fn find_first_zero_and_set(&self) -> Option<usize> {
    for idx in 0..WORDS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 311) - [public]

find_first_zero_and_set

```
/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny wzięło ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// przechodzi dalej, żeby nie pominąć bitów zwolnionych w
/// międzyczasie.
```

```
pub fn find_first_zero_and_set(&self) -> Option<usize> {
    for idx in 0..WORDS {
        loop {
            let cur = self.words[idx].load(Ordering::Acquire);
            if cur == u64::MAX {
                break;
            }
            let bit_in_word = (!cur).trailing_zeros() as usize;
            let global_bit = idx * BITS_PER_WORD + bit_in_word;
            if global_bit >= Self::CAPACITY {
                return None;
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 340) - [internal]

default

```
fn default() -> Self {
    Self::new()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn words_for_bits_rounds_up() {
        assert_eq!(words_for_bits(1), 1);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 350) - [internal]

words_for_bits_rounds_up

```
fn words_for_bits_rounds_up() {
    assert_eq!(words_for_bits(1), 1);
    assert_eq!(words_for_bits(64), 1);
    assert_eq!(words_for_bits(65), 2);
    assert_eq!(words_for_bits(128), 2);
}

#[test]
fn set_clear_test_roundtrip() {
    let mut b: Bitmap<2> = Bitmap::new();
    assert!(b.test(70));
    b.set(70);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 358) - [internal]

set_clear_test_roundtrip

```
fn set_clear_test_roundtrip() {
    let mut b: Bitmap<2> = Bitmap::new();
    assert!(b.test(70));
    b.set(70);
    assert!(b.test(70));
    assert!(b.test(69));
    b.clear(70);
    assert!(b.test(70));
}

#[test]
fn out_of_range_access_is_a_safe_noop() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 369) - [internal]

out_of_range_access_is_a_safe_noop

```
fn out_of_range_access_is_a_safe_noop() {
    let mut b: Bitmap<1> = Bitmap::new();
    b.set(1000);
    assert!(b.test(1000));
    assert!(b.is_empty());
}

#[test]
fn weight_counts_set_bits_across_words() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(0);
    b.set(63);
    b.set(64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 377) - [internal]

weight_counts_set_bits_across_words

```
fn weight_counts_set_bits_across_words() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(0);
    b.set(63);
    b.set(64);
    b.set(127);
    assert_eq!(b.weight(), 4);
}

#[test]
fn find_first_set_crosses_word_boundary() {
    let mut b: Bitmap<3> = Bitmap::new();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 387) - [internal]

find_first_set_crosses_word_boundary

```
fn find_first_set_crosses_word_boundary() {
    let mut b: Bitmap<3> = Bitmap::new();
    b.set(130);
    assert_eq!(b.find_first_set(), Some(130));
}

#[test]
fn find_first_zero_skips_full_words() {
    let mut b: Bitmap<2> = Bitmap::filled();
    b.clear(70);
    assert_eq!(b.find_first_zero(), Some(70));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 394) - [internal]

find_first_zero_skips_full_words

```
fn find_first_zero_skips_full_words() {
    let mut b: Bitmap<2> = Bitmap::filled();
    b.clear(70);
    assert_eq!(b.find_first_zero(), Some(70));
}

#[test]
fn find_first_zero_none_when_full() {
    let b: Bitmap<2> = Bitmap::filled();
    assert_eq!(b.find_first_zero(), None);
    assert!(b.is_full());
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 401) - [internal]

find_first_zero_none_when_full

```
fn find_first_zero_none_when_full() {
    let b: Bitmap<2> = Bitmap::filled();
    assert_eq!(b.find_first_zero(), None);
    assert!(b.is_full());
}

#[test]
fn find_next_set_after_given_bit() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(5);
    b.set(80);
    assert_eq!(b.find_next_set(5), Some(80));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 408) - [internal]

find_next_set_after_given_bit

```
fn find_next_set_after_given_bit() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(5);
    b.set(80);
    assert_eq!(b.find_next_set(5), Some(80));
    assert_eq!(b.find_next_set(80), None);
}

#[test]
fn set_range_sets_contiguous_span() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set_range(60, 70);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 417) - [internal]

set_range_sets_contiguous_span

```
fn set_range_sets_contiguous_span() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set_range(60, 70);
    for i in 60..70 {
        assert!(b.test(i), "bit {i} powinien by■ ustawiony");
    }
    assert!(!b.test(59));
    assert!(!b.test(70));
    assert_eq!(b.weight(), 10);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 429) - [internal]

boolean_ops_behave_as_expected

```
fn boolean_ops_behave_as_expected() {
    let mut a: Bitmap<1> = Bitmap::new();
    let mut b: Bitmap<1> = Bitmap::new();
    a.set(1);
    a.set(2);
    b.set(2);
    b.set(3);
    assert_eq!(a.and(&b).weight(), 1);
    assert!(a.and(&b).test(2));
    assert_eq!(a.or(&b).weight(), 3);
    assert_eq!(a.xor(&b).weight(), 2);
    assert_eq!(a.andnot(&b).weight(), 1);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 446) - [internal]

iter_yields_set_bits_in_order

```
fn iter_yields_set_bits_in_order() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(3);
    b.set(64);
    b.set(100);
    let collected: [usize; 3] = {
        let mut it = b.iter();
        [it.next().unwrap(), it.next().unwrap(), it.next().unwrap()]
    };
    assert_eq!(collected, [3, 64, 100]);
    assert!(b.iter().nth(3).is_none());
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 460) - [internal]

slice_bitmap_respects_logical_bit_count_not_word_count

```
fn slice_bitmap_respects_logical_bit_count_not_word_count() {
    let mut storage = [0u64; 2];
    let mut view = BitmapSlice::new(&mut storage, 70);
    assert_eq!(view.capacity(), 70);
    view.set(69);
    assert!(view.test(69));
    view.set(100); // poza `nbits`, mimo ■e mie■ci si■ w drugim s■owie
    assert!(!view.test(100));
    assert_eq!(view.find_first_zero(), Some(0));
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 472) - [internal]

slice_bitmap_clear_all_resets_backing_storage

```
fn slice_bitmap_clear_all_resets_backing_storage() {
    let mut storage = [u64::MAX; 1];
    let mut view = BitmapSlice::new(&mut storage, 64);
    assert_eq!(view.weight(), 64);
    view.clear_all();
    assert_eq!(view.weight(), 0);
}

#[test]
fn atomic_bitmap_test_and_set_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(5));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 481) - [internal]

atomic_bitmap_test_and_set_reports_previous_state

```
fn atomic_bitmap_test_and_set_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(5));
    assert!(bm.test(5));
    assert!(bm.test_and_set(5));
}

#[test]
fn atomic_bitmap_test_and_clear_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    bm.test_and_set(9);
    assert!(bm.test_and_clear(9));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 489) - [internal]

atomic_bitmap_test_and_clear_reports_previous_state

```
fn atomic_bitmap_test_and_clear_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    bm.test_and_set(9);
    assert!(bm.test_and_clear(9));
    assert!(bm.test(9));
    assert!(bm.test_and_clear(9));
}

#[test]
fn atomic_bitmap_find_first_zero_and_set_claims_sequentially() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 498) - [internal]

atomic_bitmap_find_first_zero_and_set_claims_sequentially

```
fn atomic_bitmap_find_first_zero_and_set_claims_sequentially() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
    assert_eq!(bm.find_first_zero_and_set(), Some(1));
    bm.test_and_clear(0);
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
}

#[test]
fn atomic_bitmap_find_first_zero_and_set_returns_none_when_full() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    for _ in 0..64 {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 507) - [internal]

atomic_bitmap_find_first_zero_and_set_returns_none_when_full

```
fn atomic_bitmap_find_first_zero_and_set_returns_none_when_full() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    for _ in 0..64 {
        assert!(bm.find_first_zero_and_set().is_some());
    }
    assert!(bm.is_full());
    assert_eq!(bm.find_first_zero_and_set(), None);
}

#[test]
fn atomic_bitmap_weight_matches_manual_count() {
    let bm: AtomicBitmap<2> = AtomicBitmap::new();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 517) - [internal]

atomic_bitmap_weight_matches_manual_count

```
fn atomic_bitmap_weight_matches_manual_count() {
    let bm: AtomicBitmap<2> = AtomicBitmap::new();
    bm.test_and_set(0);
    bm.test_and_set(64);
    bm.test_and_set(127);
    assert_eq!(bm.weight(), 3);
}

#[test]
fn atomic_bitmap_out_of_range_set_is_harmless() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(1000));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 526) - [internal]

atomic_bitmap_out_of_range_set_is_harmless

```
fn atomic_bitmap_out_of_range_set_is_harmless() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(1000));
    assert!(!bm.test_and_clear(1000));
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 17) - [public]

AtomicCpuMask

```
pub struct AtomicCpuMask {
    bits: [AtomicU64; WORDS],
}

impl AtomicCpuMask {
    #[inline]
    pub const fn new() -> Self {
        const INIT: AtomicU64 = AtomicU64::new(0);
        Self { bits: [INIT; WORDS] }
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 29) - [public] set

```
pub fn set(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_or(1u64 << (idx % 64), Ordering::Relaxed);
    }
}

#[inline]
pub fn clear(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_and(!(1u64 << (idx % 64)), Ordering::Relaxed);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 37) - [public] clear

```
pub fn clear(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_and(!(1u64 << (idx % 64)), Ordering::Relaxed);
    }
}

#[inline]
pub fn test(&self, cpu: u32) -> bool {
    let idx = cpu as usize;
    idx < MAX_CPUS && (self.bits[idx / 64].load(Ordering::Acquire) & (1u64 << (idx % 64))) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 45) - [public] test

```
pub fn test(&self, cpu: u32) -> bool {
    let idx = cpu as usize;
    idx < MAX_CPUS && (self.bits[idx / 64].load(Ordering::Acquire) & (1u64 << (idx % 64))) != 0
}

#[inline]
pub fn snapshot(&self) -> CpuMask {
    let mut m = CpuMask::empty();
    for cpu in 0..MAX_CPUS as u32 {
        if self.test(cpu) {
            m.set(cpu);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 51) - [public] snapshot

```
pub fn snapshot(&self) -> CpuMask {
    let mut m = CpuMask::empty();
    for cpu in 0..MAX_CPUS as u32 {
        if self.test(cpu) {
            m.set(cpu);
        }
    }
    m
}

static POSSIBLE: AtomicCpuMask = AtomicCpuMask::new();
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 74) - [public]

mark_possible

```
pub fn mark_possible(cpu: u32) {
    POSSIBLE.set(cpu);
}

#[inline]
pub fn mark_present(cpu: u32) {
    PRESENT.set(cpu);
}

#[inline]
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 79) - [public]

mark_present

```
pub fn mark_present(cpu: u32) {
    PRESENT.set(cpu);
}

#[inline]
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    ACTIVE.set(cpu);
}

#[inline]
pub fn mark_offline(cpu: u32) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 84) - [public]

mark_online

```
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    ACTIVE.set(cpu);
}

#[inline]
pub fn mark_offline(cpu: u32) {
    ACTIVE.clear(cpu);
    ONLINE.clear(cpu);
}

#[inline]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 90) - [public]

mark_offline

```
pub fn mark_offline(cpu: u32) {
    ACTIVE.clear(cpu);
    ONLINE.clear(cpu);
}

#[inline]
pub fn is_online(cpu: u32) -> bool {
    ONLINE.test(cpu)
}

#[inline]
pub fn online_mask() -> CpuMask {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 96) - [public]

is_online

```
pub fn is_online(cpu: u32) -> bool {
    ONLINE.test(cpu)
}

#[inline]
pub fn online_mask() -> CpuMask {
    ONLINE.snapshot()
}

#[inline]
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 101) - [public]

online_mask

```
pub fn online_mask() -> CpuMask {
    ONLINE.snapshot()
}

#[inline]
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
}

#[inline]
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 106) - [public]

active_mask

```
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
}

#[inline]
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        PACKAGE_ID[cpu as usize].store(pkg, Ordering::Release);
        CORE_ID[cpu as usize].store(core, Ordering::Release);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 111) - [public]

set_topology

```
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        PACKAGE_ID[cpu as usize].store(pkg, Ordering::Release);
        CORE_ID[cpu as usize].store(core, Ordering::Release);
    }
}

#[inline]
pub fn sibling_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    let core = CORE_ID[cpu as usize].load(Ordering::Acquire);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 119) - [public]

sibling_mask

```
pub fn sibling_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    let core = CORE_ID[cpu as usize].load(Ordering::Acquire);
    if pkg == NO_TOPO || core == NO_TOPO {
        return out;
    }

    for c in online_mask().iter() {
        let c_idx = c as usize;
        if PACKAGE_ID[c_idx].load(Ordering::Acquire) == pkg && CORE_ID[c_idx].load(Ordering::Acquire) ==
            out.set(c);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 137) - [public]
package_mask

```
pub fn package_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    if pkg == NO_TOPO {
        return out;
    }

    for c in online_mask().iter() {
        let c_idx = c as usize;
        if PACKAGE_ID[c_idx].load(Ordering::Acquire) == pkg {
            out.set(c);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 20) - [internal]
plist_prio

```
fn plist_prio(node: *mut TaskStruct) -> i32 {
    unsafe { *plist_prio_slot(node) }
}

/// Odtwarza `TaskStruct` z wzmiana `plist_node`.
///
/// Przechodzi przez `TaskStruct::container_of`, więc w trybie debug
/// wynik jest zweryfikowany round-tripem – błądny `PLIST_NODE_OFFSET`
/// (np. po zmianie layoutu `TaskStruct` bez aktualizacji stałej)
/// wywala się jako panika w testach, a nie jako ciche uszkodzenie
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 44) - [public] PList

```
pub struct PList {
    pub head: ListHead,
}

impl PList {
    pub const fn new() -> Self {
        Self { head: ListHead::new() }
    }

    pub fn is_empty(&self) -> bool {
        self.head.is_empty()
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 53) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.head.is_empty()
}

/// Wstawia `task` z priorytetem `priority`, zachowując porządek
/// rosnący po poziomach oraz FIFO w obrębie tego samego priorytetu.
///
/// # Bezpieczeństwo
/// `task` musi być przypięty (nie przenoszony w pamięci) przez
/// cały czas obecności na liście, i nie może już być na niej
/// obecny (podwójny insert bez remove uszkodzi listę – patrz
/// `debug_assert` poniżej, który to wyłapuje w testach).
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 152) - [public] first

```
/// Zadanie o najwyższym priorytecie (head listy poziomów),
/// pierwsze w kolejności FIFO na tym poziomie. O(1).

pub fn first(&self) -> *mut TaskStruct {
    if self.head.is_empty() { return ptr::null_mut(); }
    unsafe { task_from_node(self.head.next) }
}

/// Zadanie o najniższym priorytecie (ostatni poziom), ostatnie
/// w kolejności FIFO na tym poziomie.
///
/// O(1) — poprzednio ta funkcja mogła degenerować się do O(N)
/// przy wielu zadaniach na tym samym (najniższym) priorytecie,
/// bo szła liniowo przez `same_prio.prev` licząc na to, że
/// wskaźnik "prev" head'a wskazuje bezpośrednio na ogon (co nie
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 167) - [public] last

```
/// Zadanie o najniższym priorytecie (ostatni poziom), ostatnie
/// w kolejności FIFO na tym poziomie.
///
/// O(1) — poprzednio ta funkcja mogła degenerować się do O(N)
/// przy wielu zadaniach na tym samym (najniższym) priorytecie,
/// bo szła liniowo przez `same_prio.prev` licząc na to, że
/// wskaźnik "prev" head'a wskazuje bezpośrednio na ogon (co nie
/// było gwarantowane przez ówczesny `insert`). Teraz `insert`
/// utrzymuje ten inwariant jawnie: `same_prio.prev` head'a
/// zawsze wskazuje ogon FIFO, więc odczyt jest bezpośredni.

pub fn last(&self) -> *mut TaskStruct {
    if self.head.is_empty() { return ptr::null_mut(); }
    unsafe {
        let last_level = task_from_node(self.head.prev);
        let same_prio = plist_same_prio(last_level);
        if (*same_prio).is_empty() {
            last_level
        } else {
            task_from_same_prio((*same_prio).prev)
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 23) - [internal]
rb_parent

```
fn rb_parent(node: *mut TaskStruct) -> *mut TaskStruct {
    if node.is_null() { return ptr::null_mut(); }
    unsafe { (*rb_parent_color_slot(node) & !1) as *mut TaskStruct }
}

#[inline(always)]
fn rb_color(node: *mut TaskStruct) -> usize {
    if node.is_null() { return RB_BLACK; }
    unsafe { *rb_parent_color_slot(node) & 1 }
}

#[inline(always)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 29) - [internal]
rb_color

```
fn rb_color(node: *mut TaskStruct) -> usize {
    if node.is_null() { return RB_BLACK; }
    unsafe { *rb_parent_color_slot(node) & 1 }
}

#[inline(always)]
fn rb_is_red(node: *mut TaskStruct) -> bool {
    !node.is_null() && rb_color(node) == RB_RED
}

#[inline(always)]
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 35) - [internal]
rb_is_red

```
fn rb_is_red(node: *mut TaskStruct) -> bool {
    !node.is_null() && rb_color(node) == RB_RED
}

#[inline(always)]
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        *rb_parent_color_slot(node) = (parent as usize & !1) | color;
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 40) - [internal]
rb_set_parent_color

```
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        *rb_parent_color_slot(node) = (parent as usize & !1) | color;
    }
}

#[inline(always)]
fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
    let color = rb_color(node);
    rb_set_parent_color(node, parent, color);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 48) - [internal]
rb_set_parent

```
fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
    let color = rb_color(node);
    rb_set_parent_color(node, parent, color);
}

#[inline(always)]
fn rb_set_color(node: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        let slot = rb_parent_color_slot(node);
        *slot = (*slot & !1) | color;
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 54) - [internal]

rb_set_color

```
fn rb_set_color(node: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        let slot = rb_parent_color_slot(node);
        *slot = (*slot & !1) | color;
    }
}

pub struct RbTree {
    pub root: *mut TaskStruct,
    pub leftmost: *mut TaskStruct,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 62) - [public] RbTree

```
pub struct RbTree {
    pub root: *mut TaskStruct,
    pub leftmost: *mut TaskStruct,
}

impl RbTree {
    pub const fn new() -> Self {
        Self { root: ptr::null_mut(), leftmost: ptr::null_mut() }
    }

    pub unsafe fn insert<F>(&mut self, node: *mut TaskStruct, mut less: F)
    where
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 116) - [internal]

rb_insert_fixup

```
fn rb_insert_fixup(&mut self, mut node: *mut TaskStruct) {
    unsafe {
        while rb_is_red(rb_parent(node)) {
            let parent = rb_parent(node);
            let grandparent = rb_parent(parent);
            debug_assert(!grandparent.is_null(), "czerwony rodzic implikuje istnienie czarnego dziadka");

            if parent == *rb_left(grandparent) {
                let uncle = *rb_right(grandparent);
                if rb_is_red(uncle) {
                    rb_set_color(parent, RB_BLACK);
                    rb_set_color(uncle, RB_BLACK);
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 4) - [public] RtArray

```
pub struct RtArray {
    pub queue: [ListHead; MAX_RT_PRIO as usize],
    pub bitmap: [u64; 2],
    pub nr_running: usize,
}

impl RtArray {
    pub const fn new() -> Self {
        Self {
            queue: [ListHead::new(); MAX_RT_PRIO as usize],
            bitmap: [0, 0],
            nr_running: 0,
        }
    }
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 20) - [public] set_bit

```
pub fn set_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] |= 1u64 << bit;
}

#[inline(always)]
pub fn clear_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] &= !(1u64 << bit);
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 27) - [public] clear_bit

```
pub fn clear_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] &= !(1u64 << bit);
}

#[inline(always)]
fn bit_is_set(&self, prio: usize) -> bool {
    let word = prio / 64;
    let bit = prio % 64;
    (self.bitmap[word] & (1u64 << bit)) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 34) - [internal] bit_is_set

```
fn bit_is_set(&self, prio: usize) -> bool {
    let word = prio / 64;
    let bit = prio % 64;
    (self.bitmap[word] & (1u64 << bit)) != 0
}

#[inline(always)]
pub fn highest_prio(&self) -> Option<usize> {
    if self.bitmap[0] != 0 {
        Some(self.bitmap[0].trailing_zeros() as usize)
    } else if self.bitmap[1] != 0 {
        Some(64 + self.bitmap[1].trailing_zeros() as usize)
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 41) - [public] highest_prio

```
pub fn highest_prio(&self) -> Option<usize> {
    if self.bitmap[0] != 0 {
        Some(self.bitmap[0].trailing_zeros() as usize)
    } else if self.bitmap[1] != 0 {
        Some(64 + self.bitmap[1].trailing_zeros() as usize)
    } else {
        None
    }
}

/// Dodaje `task` do kolejki jego priorytetu i ustawia bit
/// w bitmapie. 0(1).
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 149) - [public]

active_levels

```
/// Liczba unikalnych aktywnych poziomów priorytetu (nie mylić
/// z `nr_running`, które liczy wszystkie zakolejkowane zadania).
///
/// Dopisane, bo bez tego nie da się odróżnić "jeden proces o
/// wysokim priorytecie" od "wiele procesów rozłożonych na wiele
/// priorytetów" bez przechodzenia całej bitmapy ręcznie za każdym
/// razem, gdy taka informacja jest potrzebna (np. w heurystykach
/// load-balancingu między CPU).

pub fn active_levels(&self) -> u32 {
    self.bitmap[0].count_ones() + self.bitmap[1].count_ones()
}

/// Odtwarza `TaskStruct` z wzmiana `rt.run_list`.
///
/// Przechodzi przez `TaskStruct::container_of`, więc w trybie debug
/// wynik jest zweryfikowany round-tripem (patrz dokumentacja
/// `container_of` w `task.rs`) – spójnie z tym, jak `plist.rs`
/// odtwarza `TaskStruct` ze swoich wzmianek.
#[inline(always)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 15) - [public]

PidAllocator

```
pub struct PidAllocator {
    bitmap: [AtomicU64; PID_BITMAP_WORDS],
    next_hint: AtomicU32,
    allocated: AtomicU32,
}

impl PidAllocator {
    pub const fn new() -> Self {
        const ZERO: AtomicU64 = AtomicU64::new(0);
        Self {
            bitmap: [ZERO; PID_BITMAP_WORDS],
            next_hint: AtomicU32::new(RESERVED_PIDS),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 31) - [public] alloc

```
pub fn alloc(&self) -> Option<TaskId> {
    let hint = self.next_hint.load(Ordering::Relaxed) as usize;
    let start_idx = hint / 64;

    for offset in 0..PID_BITMAP_WORDS {
        let idx = (start_idx + offset) % PID_BITMAP_WORDS;
        let word = self.bitmap[idx].load(Ordering::Relaxed);
        if word == u64::MAX { continue; }

        let bit = (!word).trailing_zeros();
        let pid = (idx * 64 + bit as usize) as u32;
        if pid >= MAX_PID { continue; }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 54) - [public] free

```
pub fn free(&self, pid: TaskId) {
    let pid = pid as u32;
    if pid >= MAX_PID { return; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    if self.bitmap[idx].fetch_and(!bit, Ordering::AcqRel) & bit != 0 {
        self.allocated.fetch_sub(1, Ordering::Relaxed);
    }
}

pub fn reserve(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 64) - [public] reserve

```
pub fn reserve(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    if pid >= MAX_PID { return false; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    if self.bitmap[idx].fetch_or(bit, Ordering::AcqRel) & bit == 0 {
        self.allocated.fetch_add(1, Ordering::Relaxed);
        true
    } else {
        false
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 77) - [public] is_used

```
pub fn is_used(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    if pid >= MAX_PID { return true; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    (self.bitmap[idx].load(Ordering::Acquire) & bit) != 0
}

pub fn count_allocated(&self) -> u32 {
    self.allocated.load(Ordering::Relaxed)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 85) - [public] count_allocated

```
pub fn count_allocated(&self) -> u32 {
    self.allocated.load(Ordering::Relaxed)
}

<<<<<<< HEAD
pub const MAX_TASKS: usize = 65536;
const TGID_SLOTS: usize = MAX_TASKS / 4;

pub struct TaskTable {
    pid_slots: [AtomicPtr<TaskStruct>; MAX_TASKS],
    tgid_slots: [AtomicPtr<TaskStruct>; TGID_SLOTS],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 94) - [public] TaskTable

```
pub struct TaskTable {
    pid_slots: [AtomicPtr<TaskStruct>; MAX_TASKS],
    tgid_slots: [AtomicPtr<TaskStruct>; TGID_SLOTS],
    count: AtomicU32,
    =====
    #[repr(C)]
    pub struct RtFields {
        pub rt_priority: u32,
        pub run_list: ListHead,
    }
    >>>>>>> da9f7b9 (wip: opis zmian)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 100) - [public] RtFields

```
pub struct RtFields {
    pub rt_priority: u32,
    pub run_list: ListHead,
    >>>>>> da9f7b9 (wip: opis zmian)
}
```

```
impl TaskTable {
    pub const fn new() -> Self {
        const NULL_PTR: AtomicPtr<TaskStruct> = AtomicPtr::new(ptr::null_mut());
        Self {
            pid_slots: [NULL_PTR; MAX_TASKS],
            tgid_slots: [NULL_PTR; TGID_SLOTS],
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 116) - [internal] hash_pid

```
fn hash_pid(pid: TaskId) -> usize { (pid as usize) % MAX_TASKS }
fn hash_tgid(tgid: TaskId) -> usize { (tgid as usize) % TGID_SLOTS }

pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 117) - [internal]
hash_tgid

```
fn hash_tgid(tgid: TaskId) -> usize { (tgid as usize) % TGID_SLOTS }

pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        if self.pid_slots[idx].compare_exchange(
            expected, task, Ordering::AcqRel, Ordering::Acquire
        ).is_ok() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 119) - [public] insert

```
pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        if self.pid_slots[idx].compare_exchange(
            expected, task, Ordering::AcqRel, Ordering::Acquire
        ).is_ok() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 154) - [public] remove

```
pub fn remove(&self, pid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let slot = self.pid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }

        if unsafe { (*slot).pid == pid } {
            if self.pid_slots[idx].compare_exchange(
                slot, ptr::null_mut(), Ordering::AcqRel, Ordering::Acquire
            ).is_ok() {
                self.count.fetch_sub(1, Ordering::Relaxed);
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 185) - [public] lookup_pid

```
pub fn lookup_pid(&self, pid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let slot = self.pid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }
        if unsafe { (*slot).pid == pid } { return slot; }
    }
    ptr::null_mut()
}

pub fn lookup_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 196) - [public]
lookup_tgid

```
pub fn lookup_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_tgid(tgid);
    for offset in 0..TGID_SLOTS {
        let idx = (start + offset) % TGID_SLOTS;
        let slot = self.tgid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }
        if unsafe { (*slot).tgid == tgid && (*slot).pid == tgid } { return slot; }
    }
    ptr::null_mut()
}

pub fn count(&self) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 207) - [public] count

```
pub fn count(&self) -> u32 {
    self.count.load(Ordering::Relaxed)
}

unsafe fn task_from_list_node(node: *mut ListHead) -> *mut TaskStruct {
    let dummy = MaybeUninit::uninit();
    let base = dummy.as_ptr() as *mut u8;
    let field = core::ptr::addr_of!((*dummy.as_ptr()).thread_group) as *mut u8;
    let offset = field.offset_from(base) as usize;
    (node as *mut u8).sub(offset) as *mut TaskStruct
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 221) - [public]

TaskIterator

```
pub struct TaskIterator<'a> {
    table: &'a TaskTable,
    index: usize,
    =====
impl TaskStruct {
    pub const RB_LEFT_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_left);
    pub const RB_RIGHT_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_right);
    pub const RB_PARENT_COLOR_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_parent_color);

    pub const PLIST_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_prio);
    pub const PLIST_SAME_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_same_prio);
    pub const PLIST_NODE_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_node);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 279) - [public] new

```
pub fn new(table: &'a TaskTable) -> Self {
    Self { table, index: 0 }
}

impl<'a> Iterator for TaskIterator<'a> {
    type Item = *mut TaskStruct;

    fn next(&mut self) -> Option<Self::Item> {
        while self.index < MAX_TASKS {
            let slot = self.table.pid_slots[self.index].load(Ordering::Acquire);
            self.index += 1;
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 287) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    while self.index < MAX_TASKS {
        let slot = self.table.pid_slots[self.index].load(Ordering::Acquire);
        self.index += 1;
        if !slot.is_null() {
            return Some(slot);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 299) - [public]

ThreadGroupIterator

```
pub struct ThreadGroupIterator {
    current: *mut TaskStruct,
    head: *mut ListHead,
}

impl ThreadGroupIterator {
    pub unsafe fn new(leader: *mut TaskStruct) -> Self {
        if leader.is_null() {
            return Self { current: ptr::null_mut(), head: ptr::null_mut() };
        }
        let head = &(*leader).thread_group as *const ListHead as *mut ListHead;
        let first = (*head).next;
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 319) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    if self.current.is_null() || self.head.is_null() { return None; }
    let result = self.current;
    let list_node = &(*self.current).thread_group as *const ListHead as *mut ListHead;
    let next_node = (*list_node).next;

    if next_node == self.head || next_node.is_null() {
        self.current = ptr::null_mut();
    } else {
        self.current = task_from_list_node(next_node);
    }
    Some(result)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 334) - [public]

ChildIterator

```
pub struct ChildIterator {
    current: *mut TaskStruct,
}

impl ChildIterator {
    pub unsafe fn new(parent: *mut TaskStruct) -> Self {
        if parent.is_null() {
            return Self { current: ptr::null_mut() };
        }
        Self { current: (*parent).children }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 350) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    if self.current.is_null() { return None; }
    let result = self.current;
    self.current = (*self.current).sibling;
    Some(result)
}

pub struct DescendantIterator {
    stack: Vec<*mut TaskStruct>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 358) - [public]

DescendantIterator

```
pub struct DescendantIterator {
    stack: Vec<*mut TaskStruct>,
}

impl DescendantIterator {
    pub unsafe fn new(root: *mut TaskStruct) -> Self {
        let mut stack = Vec::new();
        if !root.is_null() {
            stack.push(root);
        }
        Self { stack }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 375) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    let current = self.stack.pop()?;
    let mut child = (*current).children;
    while !child.is_null() {
        self.stack.push(child);
        child = (*child).sibling;
    }
    Some(current)
}
```

```
pub struct ProcessGroup {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 386) - [public]
ProcessGroup

```
pub struct ProcessGroup {
    pub pgid: TaskId,
    pub leader: AtomicPtr<TaskStruct>,
    pub session: AtomicPtr<Session>,
    pub tasks: ListHead,
    pub lock: SpinLock,
    pub refcount: AtomicU32,
    pub is_orphaned: AtomicBool,
}
```

```
impl ProcessGroup {
    pub fn new(pgid: TaskId, leader: *mut TaskStruct) -> Self {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 397) - [public] new

```
pub fn new(pgid: TaskId, leader: *mut TaskStruct) -> Self {
    let mut pg = Self {
        pgid,
        leader: AtomicPtr::new(leader),
        session: AtomicPtr::new(ptr::null_mut()),
        tasks: ListHead::new(),
        lock: SpinLock::new(),
        refcount: AtomicU32::new(1),
        is_orphaned: AtomicBool::new(false),
    };
    pg.tasks.init();
    pg
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 427) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.refcount.load(Ordering::Relaxed) <= 1
}

pub unsafe fn send_signal(&self, sig: u8) {
    let mut node = self.tasks.next;
    while node != &self.tasks as *const ListHead as *mut ListHead && !node.is_null() {
        let task = task_from_list_node(node);
        if !task.is_null() {
            (*task).sig.raise(sig);
        }
        node = (*node).next;
    }
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 443) - [public] Session

```
pub struct Session {
    pub sid: TaskId,
    pub leader: AtomicPtr<TaskStruct>,
    pub groups: ListHead,
    pub lock: SpinLock,
    pub refcount: AtomicU32,
    pub foreground_pgid: AtomicU32,
}

impl Session {
    pub fn new(sid: TaskId, leader: *mut TaskStruct) -> Self {
        let mut s = Self {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 453) - [public] new

```
pub fn new(sid: TaskId, leader: *mut TaskStruct) -> Self {
    let mut s = Self {
        sid,
        leader: AtomicPtr::new(leader),
        groups: ListHead::new(),
        lock: SpinLock::new(),
        refcount: AtomicU32::new(1),
        foreground_pgid: AtomicU32::new(0),
    };
    s.groups.init();
    s
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 473) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.refcount.load(Ordering::Relaxed) <= 1
}

pub fn set_foreground(&self, pgid: TaskId) {
    self.foreground_pgid.store(pgid as u32, Ordering::Release);
}

pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 477) - [public] set_foreground

```
pub fn set_foreground(&self, pgid: TaskId) {
    self.foreground_pgid.store(pgid as u32, Ordering::Release);
}

pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}
}

pub const MAX_UID: usize = 65536;

pub struct UidTaskCount {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 481) - [public]

get_foreground

```
pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}

pub const MAX_UID: usize = 65536;

pub struct UidTaskCount {
    counts: [AtomicU32; MAX_UID],
}

impl UidTaskCount {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 488) - [public]

UidTaskCount

```
pub struct UidTaskCount {
    counts: [AtomicU32; MAX_UID],
}

impl UidTaskCount {
    pub const fn new() -> Self {
        const ZERO: AtomicU32 = AtomicU32::new(0);
        Self { counts: [ZERO; MAX_UID] }
    }

    pub fn inc(&self, uid: u32) -> u32 {
        let idx = (uid as usize) % MAX_UID;
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 498) - [public] inc

```
pub fn inc(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_add(1, Ordering::Relaxed) + 1
}

pub fn dec(&self, uid: u32) {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_update(Ordering::Relaxed, Ordering::Relaxed, |v| {
        if v > 0 { Some(v - 1) } else { Some(0) }
    }).ok();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 503) - [public] dec

```
pub fn dec(&self, uid: u32) {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_update(Ordering::Relaxed, Ordering::Relaxed, |v| {
        if v > 0 { Some(v - 1) } else { Some(0) }
    }).ok();
}

pub fn get(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 510) - [public] get

```
pub fn get(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].load(Ordering::Relaxed)
}
}
```

```
pub enum TaskRegistryError {
    PidExhausted,
    PidAlreadyUsed,
    TableFull,
    InvalidTask,
    NprocExceeded,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 516) - [public]

TaskRegistryError

```
pub enum TaskRegistryError {
    PidExhausted,
    PidAlreadyUsed,
    TableFull,
    InvalidTask,
    NprocExceeded,
}
```

```
pub struct TaskRegistry {
    pub pids: PidAllocator,
    pub table: TaskTable,
    pub init_task: AtomicPtr<TaskStruct>,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 524) - [public]

TaskRegistry

```
pub struct TaskRegistry {
    pub pids: PidAllocator,
    pub table: TaskTable,
    pub init_task: AtomicPtr<TaskStruct>,
    pub total_forks: AtomicU64,
    pub total_exits: AtomicU64,
    pub global_lock: SpinLock,
    pub uid_counts: UidTaskCount,
}
```

```
impl TaskRegistry {
    pub const fn new() -> Self {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 585) - [public]

find_by_pid

```
pub fn find_by_pid(&self, pid: TaskId) -> *mut TaskStruct {
    self.table.lookup_pid(pid)
}

pub fn find_by_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    self.table.lookup_tgid(tgid)
}

pub unsafe fn link_hierarchy(&self, parent: *mut TaskStruct, child: *mut TaskStruct) {
    if parent.is_null() || child.is_null() { return; }
    let _guard = self.global_lock.lock_guard();

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 589) - [public]

find_by_tgid

```
pub fn find_by_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    self.table.lookup_tgid(tgid)
}

pub unsafe fn link_hierarchy(&self, parent: *mut TaskStruct, child: *mut TaskStruct) {
    if parent.is_null() || child.is_null() { return; }
    let _guard = self.global_lock.lock_guard();

    (*child).parent = parent;
    (*child).ppid = (*parent).pid;
    (*child).sibling = (*parent).children;
    (*parent).children = child;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 685) - [public] iter

```
pub fn iter(&self) -> TaskIterator {
    TaskIterator::new(&self.table)
}

pub unsafe fn send_signal_to_group(&self, pgid: TaskId, sig: u8) {
    let leader = self.find_by_tgid(pgid);
    if leader.is_null() { return; }

    for task_ptr in ThreadGroupIterator::new(leader) {
        if !task_ptr.is_null() {
            (*task_ptr).sig.raise(sig);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 700) - [public]

set_init_task

```
pub fn set_init_task(&self, task: *mut TaskStruct) {
    self.init_task.store(task, Ordering::Release);
}

pub fn stats(&self) -> (u32, u64, u64) {
    (
        self.table.count(),
        self.total_forks.load(Ordering::Relaxed),
        self.total_exits.load(Ordering::Relaxed),
    )
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 704) - [public] stats

```
pub fn stats(&self) -> (u32, u64, u64) {
    (
        self.table.count(),
        self.total_forks.load(Ordering::Relaxed),
        self.total_exits.load(Ordering::Relaxed),
    )
}

pub struct PidNamespace {
    pub id: u32,
    pub level: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 713) - [public]

PidNamespace

```
pub struct PidNamespace {
    pub id: u32,
    pub level: u32,
    pub parent: *mut PidNamespace,
    pub registry: TaskRegistry,
    pub lock: SpinLock,
}

impl PidNamespace {
    pub const fn new(id: u32, level: u32, parent: *mut PidNamespace) -> Self {
        Self {
            id,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 22) - [public] alloc_pid

```
pub fn alloc_pid() -> Option<TaskId> {
    PID_BITMAP.find_first_zero_and_set().map(|b| b as TaskId)
}

pub fn free_pid(pid: TaskId) {
    if (pid as usize) < PID_MAX {
        PID_BITMAP.test_and_clear(pid as usize);
    }
}
```

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 26) - [public] free_pid

```
pub fn free_pid(pid: TaskId) {
    if (pid as usize) < PID_MAX {
        PID_BITMAP.test_and_clear(pid as usize);
    }
}
```

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
```

```
pub fn reserve_pid(pid: TaskId) -> bool {
    if (pid as usize) >= PID_MAX {
        return false;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 34) - [public] reserve_pid

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
```

```
pub fn reserve_pid(pid: TaskId) -> bool {
    if (pid as usize) >= PID_MAX {
        return false;
    }
    !PID_BITMAP.test_and_set(pid as usize)
}

pub fn pid_in_use(pid: TaskId) -> bool {
    (pid as usize) < PID_MAX && PID_BITMAP.test(pid as usize)
}
```

```
// -----
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 41) - [public] pid_in_use

```
pub fn pid_in_use(pid: TaskId) -> bool {
    (pid as usize) < PID_MAX && PID_BITMAP.test(pid as usize)
}

// -----
// Rozruch CPU
// -----

pub unsafe fn boot_bsp(cpu: u32, apic_id: u32, idle: *mut TaskStruct, rq: &mut RunQueue) {
    reserve_pid((*idle).pid);
    rq.bind_idle_task();
    arch_hooks::register_apic_id(cpu, apic_id);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 219) - [public]

sys_sched_get_priority_max

```
pub fn sys_sched_get_priority_max(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        MAX_RT_PRIO - 1
    } else {
        0
    }
}

pub fn sys_sched_get_priority_min(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        0
    } else {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 227) - [public]

sys_sched_get_priority_min

```
pub fn sys_sched_get_priority_min(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        0
    } else {
        0
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::cpu::scheduler::entities::task::TaskStructT;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 240) - [internal] make_idle

```
fn make_idle(pid: TaskId) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 246) - [internal] make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStructT) -> *mut TaskStructT {
    t as *mut TaskStructT
}

#[test]
fn two_consecutive_allocations_are_distinct() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 252) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStructT) -> *mut TaskStructT {
    t as *mut TaskStructT
}

#[test]
fn two_consecutive_allocations_are_distinct() {
    let a = alloc_pid().unwrap();
    let b = alloc_pid().unwrap();
    assert_ne!(a, b);
    free_pid(a);
    free_pid(b);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 257) - [internal]
two_consecutive_allocations_are_distinct

```
fn two_consecutive_allocations_are_distinct() {
    let a = alloc_pid().unwrap();
    let b = alloc_pid().unwrap();
    assert_ne!(a, b);
    free_pid(a);
    free_pid(b);
}

#[test]
fn reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid() {
    let pid = 25_000;
    assert!(reserve_pid(pid));
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 266) - [internal]
reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid

```
fn reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid() {
    let pid = 25_000;
    assert!(reserve_pid(pid));
    assert!(!reserve_pid(pid));
    assert!(pid_in_use(pid));
    free_pid(pid);
    assert!(!pid_in_use(pid));
    assert!(reserve_pid(pid));
    free_pid(pid);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 278) - [internal]

reserve_pid_rejects_out_of_range

```
fn reserve_pid_rejects_out_of_range() {
    assert!(!reserve_pid(PID_MAX as TaskId + 1));
}

#[test]
fn boot_bsp_registers_apic_and_runqueue() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(200, ptr_of(&mut idle));
    unsafe {
        boot_bsp(200, 42, ptr_of(&mut idle), &mut rq);
    }
    assert!(core::ptr::eq(arch_hooks::snapshot_registry()[200], &rq as *const RunQueue as *mut RunQueue));
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 283) - [internal]

boot_bsp_registers_apic_and_runqueue

```
fn boot_bsp_registers_apic_and_runqueue() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(200, ptr_of(&mut idle));
    unsafe {
        boot_bsp(200, 42, ptr_of(&mut idle), &mut rq);
    }
    assert!(core::ptr::eq(arch_hooks::snapshot_registry()[200], &rq as *const RunQueue as *mut RunQueue));
    arch_hooks::register_runqueue(200, core::ptr::null_mut());
}

#[test]
fn wake_up_new_task_enqueues_and_marks_runnable() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 294) - [internal]

wake_up_new_task_enqueues_and_marks_runnable

```
fn wake_up_new_task_enqueues_and_marks_runnable() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut child = make_task(90_001, SchedPolicy::Normal, 0);
    child.set_state(TaskState::Interruptible).unwrap();

    let registry: [*mut RunQueue; 1] = [&mut rq as *mut RunQueue];
    unsafe {
        wake_up_new_task(ptr_of(&mut child), &registry).unwrap();
        assert_eq!((*ptr_of(&mut child)).state(), TaskState::Runnable);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 312) - [internal]

exit_bookkeeping_dequeues_and_marks_zombie

```
fn exit_bookkeeping_dequeues_and_marks_zombie() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        class::enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq.lock.unlock_irqrestore(flags);

        let returned_rq = exit_bookkeeping(ptr_of(&mut t));
    }
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 332) - [internal]
exit_bookkeeping_on_unqueued_task_returns_null_rq

```
fn exit_bookkeeping_on_unqueued_task_returns_null_rq() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let rq_ptr = exit_bookkeeping(ptr_of(&mut t));
        assert!(rq_ptr.is_null());
        assert_eq!((*ptr_of(&mut t)).state(), TaskState::Zombie);
    }
}

#[test]
fn reap_zombie_destroys_and_frees_pid() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 342) - [internal]
reap_zombie_destroys_and_frees_pid

```
fn reap_zombie_destroys_and_frees_pid() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    let pid = 26_000;
    t.pid = pid;
    assert!(reserve_pid(pid));

    unsafe {
        let _ = t.set_state(TaskState::Zombie);
        reap_zombie(ptr_of(&mut t));
    }

    assert_eq!(t.state(), TaskState::Dead);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 358) - [internal]
maybe_reschedule_is_noop_when_not_needed

```
fn maybe_reschedule_is_noop_when_not_needed() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let registry: [*mut RunQueue; 1] = [&mut rq as *mut RunQueue];
    unsafe {
        maybe_reschedule(0, &registry);
    }
}

#[test]
fn sys_nice_clamps_to_valid_range() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 369) - [internal]
sys_nice_clamps_to_valid_range

```
fn sys_nice_clamps_to_valid_range() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        sys_nice(ptr_of(&mut t), 100).unwrap();
        assert_eq!(weight_to_nice((*ptr_of(&mut t)).se.weight), NICE_MAX);
        sys_nice(ptr_of(&mut t), -100).unwrap();
        assert_eq!(weight_to_nice((*ptr_of(&mut t)).se.weight), NICE_MIN);
    }
}

#[test]
fn sys_nice_rejects_non_fair_policy() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 380) - [internal]

sys_nice_rejects_non_fair_policy

```
fn sys_nice_rejects_non_fair_policy() {
    let mut t = make_task(1, SchedPolicy::Fifo, 0);
    unsafe {
        assert!(sys_nice(ptr_of(&mut t), 1).is_err());
    }
}
```

#[test]

```
fn sys_sched_setscheduler_moves_task_to_new_class() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 388) - [internal]

sys_sched_setscheduler_moves_task_to_new_class

```
fn sys_sched_setscheduler_moves_task_to_new_class() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        class::enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq.lock.unlock_irqrestore(flags);

        sys_sched_setscheduler(&mut rq, ptr_of(&mut t), SchedPolicy::Fifo);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 407) - [internal]

sys_sched_rr_get_interval_matches_class_dispatch

```
fn sys_sched_rr_get_interval_matches_class_dispatch() {
    let fifo = make_task(1, SchedPolicy::Fifo, 0);
    let rr = make_task(2, SchedPolicy::RoundRobin, 0);
    unsafe {
        assert_eq!(sys_sched_rr_get_interval(&fifo as *const _), 0);
        assert!(sys_sched_rr_get_interval(&rr as *const _) > 0);
    }
}
```

#[test]

```
fn sys_sched_affinity_roundtrips() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 417) - [internal]

sys_sched_affinity_roundtrips

```
fn sys_sched_affinity_roundtrips() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let mask = CpuMask::single(3);
        sys_sched_setaffinity(ptr_of(&mut t), mask).unwrap();
        assert_eq!(sys_sched_getaffinity(ptr_of(&mut t)), mask);
    }
}
```

#[test]

```
fn priority_range_helpers_match_posix_expectations() {
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Fifo), 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 427) - [internal]

priority_range_helpers_match_posix_expectations

```
fn priority_range_helpers_match_posix_expectations() {
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Fifo), 0);
    assert_eq!(sys_sched_get_priority_max(SchedPolicy::Fifo), MAX_RT_PRIO - 1);
    assert_eq!(sys_sched_get_priority_max(SchedPolicy::Normal), 0);
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Normal), 0);
}
}
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 1) - [public] Tracepoint

```
pub struct Tracepoint {}

pub struct TracePointId {}

pub struct Tracepointkind{}

pub struct tracepoints {
    // za chwile
}

pub struct TraceEvent{

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 3) - [public]

TracePointId

```
pub struct TracePointId {}

pub struct Tracepointkind{}

pub struct tracepoints {
    // za chwile
}

pub struct TraceEvent{

}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 5) - [public]

Tracepointkind

```
pub struct Tracepointkind{}

pub struct tracepoints {
    // za chwile
}

pub struct TraceEvent{

}

pub enum TraceData{

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 7) - [public] tracepoints

```
pub struct tracepoints {
    // za chwile
}

pub struct TraceEvent{

}

pub enum TraceData{

}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 11) - [public] TraceEvent
pub struct TraceEvent{
}

pub enum TraceData{
}

pub struct ContextSwichTraceData{
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 15) - [public] TraceData
pub enum TraceData{
}

pub struct ContextSwichTraceData{
}

pub struct TraceContext{
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 19) - [public]
ContextSwichTraceData
pub struct ContextSwichTraceData{
}

pub struct TraceContext{
}

pub struct TracepointRegistry{
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 23) - [public]
TraceContext
pub struct TraceContext{
}

pub struct TracepointRegistry{
}

pub struct RegisteredTracepoint{}

pub struct TraceBuffer{}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 27) - [public]

TracepointRegistry

```
pub struct TracepointRegistry{  
  
}  
  
pub struct RegisteredTracepoint{  
  
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 31) - [public]

RegisteredTracepoint

```
pub struct RegisteredTracepoint{  
  
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 33) - [public]

TraceBuffer

```
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 35) - [public]

PerCpuTracebuffer

```
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 19) - [public]

EnqueueFlags

```
pub struct EnqueueFlags: u32 {  
    const ENQUEUE_RESTORE    = 1 << 0;  
    const ENQUEUE_WAKEUP     = 1 << 1;  
    const ENQUEUE_NEW        = 1 << 2;  
    const ENQUEUE_MIGRATED   = 1 << 3;  
    const ENQUEUE_HEAD       = 1 << 4;  
}  
  
bitflags::bitflags! {  
    #[derive(Debug, Clone, Copy, PartialEq, Eq)]  
    pub struct DequeueFlags: u32 {  
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 30) - [public]

DequeueFlags

```
pub struct DequeueFlags: u32 {  
    const DEQUEUE_SLEEP      = 1 << 0;  
    const DEQUEUE_SAVE       = 1 << 1;  
    const DEQUEUE_MIGRATING  = 1 << 2;  
}  
  
pub const MIN_GRANULARITY_NS: u64 = 750_000; // 0.75 ms  
pub const TARGET_LATENCY_NS: u64 = 6_000_000; // 6 ms  
pub const BALANCE_INTERVAL_NS: u64 = 4_000_000;  
pub const IMBALANCE_THRESHOLD: u32 = 2;  
  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 189) - [internal]

insert_fixup

```
fn insert_fixup(root: &mut *mut TaskStruct, mut z: *mut TaskStruct) {
    unsafe {
        while color_of(parent_of(z)) == RB_RED {
            let zp = parent_of(z);
            let zpp = parent_of(zp);
            debug_assert!(!zpp.is_null(), "rodzic czerwonego w zla nie moze by korzeniem");
            if zp == left_of(zpp) {
                let y = right_of(zpp);
                if color_of(y) == RB_RED {
                    set_color(zp, RB_BLACK);
                    set_color(y, RB_BLACK);
                    set_color(zpp, RB_RED);
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 240) - [public] insert

```
pub fn insert(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct, node: *mut TaskStruct, key: i32) {
    unsafe {
        debug_assert!(!node.is_null());
        (*node).se.rb_left = ptr::null_mut();
        (*node).se.rb_right = ptr::null_mut();
        (*node).se.rb_parent_color = rb_make_parent_color(ptr::null_mut(), RB_RED);

        let node_key = key_of(node);
        let mut parent: *mut TaskStruct = ptr::null_mut();
        let mut cur = *root;
        let mut is_leftmost = true;

        while !cur.is_null() {
            if node_key < key_of(cur) {
                is_leftmost = true;
                parent = cur;
                cur = (*cur).se.rb_left;
            } else {
                is_leftmost = false;
                parent = cur;
                cur = (*cur).se.rb_right;
            }
        }

        if is_leftmost {
            (*parent).se.rb_left = node;
        } else {
            (*parent).se.rb_right = node;
        }
        (*node).se.rb_parent_color = rb_make_parent_color(ptr::null_mut(), RB_RED);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 293) - [internal]

delete_fixup

```
fn delete_fixup(root: &mut *mut TaskStruct, mut x: *mut TaskStruct, mut x_parent: *mut TaskStruct) {
    unsafe {
        while x != *root && color_of(x) == RB_BLACK {
            if x_parent.is_null() {
                break;
            }
            if x == left_of(x_parent) {
                let mut w = right_of(x_parent);
                if color_of(w) == RB_RED {
                    set_color(w, RB_BLACK);
                    set_color(x_parent, RB_RED);
                    rotate_left(root, x_parent);
                }
            } else {
                let mut w = left_of(x_parent);
                if color_of(w) == RB_RED {
                    set_color(w, RB_BLACK);
                    set_color(x_parent, RB_RED);
                    rotate_right(root, x_parent);
                }
            }
            x = x_parent;
            x_parent = parent_of(x);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 357) - [public] delete

```
pub fn delete(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct, z: *mut TaskStruct) {
    unsafe {
        debug_assert!(!z.is_null());
        let mut y = z;
        let mut y_original_color = color_of(y);
        let x: *mut TaskStruct;
        let x_parent: *mut TaskStruct;

        if left_of(z).is_null() {
            x = right_of(z);
            x_parent = parent_of(z);
            transplant(root, z, right_of(z));
        } else {
            x = left_of(z);
            x_parent = parent_of(x);
            if right_of(x) == z {
                transplant(root, x, z);
                x = right_of(x);
            } else {
                transplant(root, x, left_of(x));
                x = left_of(x);
            }
            x_parent = parent_of(x);
        }
        (*x).se.rb_parent_color = y_original_color;
    }
    // ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 423) - [internal] check
fn check(node: *mut TaskStruct, key_of: KeyOf) -> Result<usize, &'static str> {
    unsafe {
        if node.is_null() {
            return Ok(1);
        }
        if color_of(node) == RB_RED {
            if color_of(left_of(node)) == RB_RED || color_of(right_of(node)) == RB_RED {
                return Err("czerwony w████ze████ ma czerwone dziecko");
            }
        }
        if !left_of(node).is_null() && key_of(left_of(node)) > key_of(node) {
            return Err("naruszony porz████dek BST (lewe dziecko)");
        }
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 452) - [internal]
fair_key_of
```

```
fn fair_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).se.vruntime }
}

#[repr(C)]
pub struct RqFair {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub nr_running: u32,
    pub load_weight: u64,
    pub min_vruntime: u64,
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 457) - [public] RqFair
```

```
pub struct RqFair {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub nr_running: u32,
    pub load_weight: u64,
    pub min_vruntime: u64,
}

impl Default for RqFair {
    fn default() -> Self {
        Self {
            root: ptr::null_mut(),
        }
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 466) - [internal] default
```

```
fn default() -> Self {
    Self {
        root: ptr::null_mut(),
        leftmost: ptr::null_mut(),
        nr_running: 0,
        load_weight: 0,
        min_vruntime: 0,
    }
}

impl RqFair {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 478) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.root.is_null()
}

pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 482) - [public] leftmost

```
pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
            && !flags.contains(EnqueueFlags::ENQUEUE_MIGRATED)
        {
            se.vruntime = cmp::max(se.vruntime, self.min_vruntime);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 486) - [internal]

place_entity

```
fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
            && !flags.contains(EnqueueFlags::ENQUEUE_MIGRATED)
        {
            se.vruntime = cmp::max(se.vruntime, self.min_vruntime);
        }
    }
}

/// Naprawa wady #2: prawdziwe wstawienie do drzewa RB (nie
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 499) - [public] enqueue

/// Naprawa wady #2: prawdziwe wstawienie do drzewa RB (nie

/// pojedynczy wskaźnik udajemy drzewo).

```
pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert(!task.is_null());
    self.place_entity(task, flags);
    rbtree::insert(&mut self.root, &mut self.leftmost, task, fair_key_of);
    unsafe {
        (*task).se.on_rq = true;
        self.load_weight = self.load_weight.saturating_add((*task).se.weight);
    }
    self.nr_running += 1;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 510) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {
    debug_assert(!task.is_null());
    rbtree::delete(&mut self.root, &mut self.leftmost, task);
    unsafe {
        (*task).se.on_rq = false;
        self.load_weight = self.load_weight.saturating_sub((*task).se.weight);
    }
    self.nr_running = self.nr_running.saturating_sub(1);
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 520) - [public] pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn charge_exec(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) -> u64 {
    unsafe {
        let se = &mut (*task).se;
        let delta_vruntime = calc_delta_fair(delta_exec, se.weight, se.inv_weight);
        se.vruntime = se.vruntime.saturating_add(delta_vruntime);
        se.load.accumulate(now, delta_exec, se.weight, true);

        self.min_vruntime = cmp::max(self.min_vruntime, se.vruntime);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 524) - [public] charge_exec

```
pub fn charge_exec(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) -> u64 {
    unsafe {
        let se = &mut (*task).se;
        let delta_vruntime = calc_delta_fair(delta_exec, se.weight, se.inv_weight);
        se.vruntime = se.vruntime.saturating_add(delta_vruntime);
        se.load.accumulate(now, delta_exec, se.weight, true);

        self.min_vruntime = cmp::max(self.min_vruntime, se.vruntime);
        if !self.leftmost.is_null() {
            let leftmost_key = fair_key_of(self.leftmost);
            self.min_vruntime = cmp::min(self.min_vruntime, cmp::max(leftmost_key, self.min_vruntime));
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 540) - [public] should_preempt

```
pub fn should_preempt(&self, current: *const TaskStruct, candidate: *const TaskStruct) -> bool {
    unsafe {
        if !fair_has_priority(&*candidate, &*current) {
            return false;
        }
        let vdiff = (*current).se.vruntime.saturating_sub((*candidate).se.vruntime);
        let ran_for = (*current).se.sum_exec_runtime.saturating_sub((*current).se.prev_sum_exec_runtime);
        vdiff > 0 && ran_for >= MIN_GRANULARITY_NS || vdiff > TARGET_LATENCY_NS
    }
}

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 552) - [public]

rb_invariants_ok

```
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, fair_key_of).is_ok() }
}

#[cfg(test)]
pub fn rb_count(&self) -> usize {
    unsafe { rbtree::count(self.root) }
}

fn dl_key_of(node: *const TaskStruct) -> u64 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 557) - [public] rb_count

```
pub fn rb_count(&self) -> usize {
    unsafe { rbtree::count(self.root) }
}

fn dl_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).dl.deadline }
}

#[repr(C)]
pub struct RqDl {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 563) - [internal]

dl_key_of

```
fn dl_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).dl.deadline }
}

#[repr(C)]
pub struct RqDl {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub dl_nr_running: u32,
    pub running_bw: u64,
    pub max_bw: u64,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 568) - [public] RqDl

```
pub struct RqDl {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub dl_nr_running: u32,
    pub running_bw: u64,
    pub max_bw: u64,
}

pub const DL_BW_SCALE: u64 = 1 << 20;

impl Default for RqDl {
    fn default() -> Self {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 579) - [internal] default

```
fn default() -> Self {
    Self {
        root: ptr::null_mut(),
        leftmost: ptr::null_mut(),
        dl_nr_running: 0,
        running_bw: 0,
        max_bw: (DL_BW_SCALE * 95) / 100,
    }
}

impl RqDl {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 591) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.root.is_null()
}

pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 595) - [public] leftmost

```
pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    let bw = (dl_runtime.saturating_mul(DL_BW_SCALE)) / dl_period;
    self.running_bw.saturating_add(bw) <= self.max_bw
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 599) - [public] admission_control

```
pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    let bw = (dl_runtime.saturating_mul(DL_BW_SCALE)) / dl_period;
    self.running_bw.saturating_add(bw) <= self.max_bw
}

fn task_bw(task: *const TaskStruct) -> u64 {
    unsafe {
        let dl = &(*task).dl;
        if dl.dl_period == 0 {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 607) - [internal] task_bw

```
fn task_bw(task: *const TaskStruct) -> u64 {
    unsafe {
        let dl = &(*task).dl;
        if dl.dl_period == 0 {
            0
        } else {
            (dl.dl_runtime.saturating_mul(DL_BW_SCALE)) / dl.dl_period
        }
    }
}

pub fn enqueue(&mut self, task: *mut TaskStruct, now: u64, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 618) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct, now: u64, flags: EnqueueFlags) {
    debug_assert(!task.is_null());
    unsafe {
        let dl = &mut (*task).dl;
        if flags.contains(EnqueueFlags::ENQUEUE_NEW) {
            // Pierwsze wejście do systemu: zacznij nowy okres CBS
            // teraz, z pełnym budżetem.
            dl.runtime = dl.dl_runtime as i64;
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.replenish_at = now.saturating_add(dl.dl_period);
            self.running_bw = self.running_bw.saturating_add(Self::task_bw(task));
        } else if dl.throttled {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 639) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct, removing_permanently: bool) {
    debug_assert(!task.is_null());
    rbtree::delete(&mut self.root, &mut self.leftmost, task);
    unsafe { (*task).se.on_rq = false };
    self.dl_nr_running = self.dl_nr_running.saturating_sub(1);
    if removing_permanently {
        self.running_bw = self.running_bw.saturating_sub(Self::task_bw(task));
    }
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 649) - [public]

pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn update_curr(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) {
    unsafe {
        let dl = &mut (*task).dl;
        dl.runtime -= delta_exec as i64;

        if now >= dl.deadline {
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.runtime = dl.dl_runtime as i64;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 653) - [public]
update_curr

```
pub fn update_curr(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) {
    unsafe {
        let dl = &mut (*task).dl;
        dl.runtime -= delta_exec as i64;

        if now >= dl.deadline {
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.runtime = dl.dl_runtime as i64;
            dl.throttled = false;
            (*task).flags.fetch_remove(TaskFlags::PF_DL_THROTTLED);
        } else if dl.runtime <= 0 {
            dl.throttled = true;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 670) - [public]
should_preempt

```
pub fn should_preempt(&self, current: *const TaskStruct, candidate: *const TaskStruct) -> bool {
    unsafe { deadline_has_priority(&*candidate, &*current) }
}

#[cfg(test)]
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, dl_key_of).is_ok() }
}
}
```

```
const RT_BITMAP_WORDS: usize = (MAX_RT_PRIO as usize + 63) / 64;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 675) - [public]
rb_invariants_ok

```
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, dl_key_of).is_ok() }
}
}
```

```
const RT_BITMAP_WORDS: usize = (MAX_RT_PRIO as usize + 63) / 64;
```

```
#[repr(C)]
pub struct RqRt {
    active: [ListHead; MAX_RT_PRIO as usize],
    bitmap: [u64; RT_BITMAP_WORDS],
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 684) - [public] RqRt

```
pub struct RqRt {
    active: [ListHead; MAX_RT_PRIO as usize],
    bitmap: [u64; RT_BITMAP_WORDS],
    pub nr_running: u32,
    pub rt_nr_migratory: u32,
    pub overloaded: bool,
    highest_prio: i32,
}

impl RqRt {
    pub fn new() -> Self {
        const INIT: ListHead = ListHead { prev: ptr::null_mut(), next: ptr::null_mut() };
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 694) - [public] new

```
pub fn new() -> Self {
    const INIT: ListHead = ListHead { prev: ptr::null_mut(), next: ptr::null_mut() };
    let mut rq = Self {
        active: [INIT; MAX_RT_PRIO as usize],
        bitmap: [0; RT_BITMAP_WORDS],
        nr_running: 0,
        rt_nr_migratory: 0,
        overloaded: false,
        highest_prio: MAX_RT_PRIO,
    };
    for head in rq.active.iter_mut() {
        head.init();
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 710) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.nr_running == 0
}

fn set_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] |= 1u64 << (prio % 64);
}

fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 714) - [internal] set_bit

```
fn set_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] |= 1u64 << (prio % 64);
}

fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

fn sched_find_first_bit(&self) -> Option<usize> {
    for (word_idx, word) in self.bitmap.iter().enumerate() {
        if *word != 0 {
            let bit = word.trailing_zeros() as usize;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 718) - [internal]

clear_bit

```
fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

fn sched_find_first_bit(&self) -> Option<usize> {
    for (word_idx, word) in self.bitmap.iter().enumerate() {
        if *word != 0 {
            let bit = word.trailing_zeros() as usize;
            let prio = word_idx * 64 + bit;
            if prio < MAX_RT_PRIO as usize {
                return Some(prio);
            }
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 722) - [internal]
sched_find_first_bit

```
fn sched_find_first_bit(&self) -> Option<usize> {  
    for (word_idx, word) in self.bitmap.iter().enumerate() {  
        if *word != 0 {  
            let bit = word.trailing_zeros() as usize;  
            let prio = word_idx * 64 + bit;  
            if prio < MAX_RT_PRIO as usize {  
                return Some(prio);  
            }  
        }  
    }  
    None  
}  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 735) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {  
    debug_assert!(!task.is_null());  
    unsafe {  
        let prio = (*task).prio;  
        debug_assert!(prio >= 0 && prio < MAX_RT_PRIO, "prio RT poza zakresem 0..MAX_RT_PRIO");  
        let prio = prio as usize;  
  
        debug_assert!(  
            core::ptr::eq((*task).rt.owner, task),  
            "RtSchedEntity.owner musi wskazywać na właściwą cielę – zapomniano TaskStruct::init?"  
        );  
  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 769) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {  
    debug_assert!(!task.is_null());  
    unsafe {  
        let prio = (*task).rt.queued_prio as usize;  
        debug_assert!(prio < MAX_RT_PRIO as usize, "dequeue zadania, które nie było zakolejkowane");  
  
        (*task).rt.rt_list.remove();  
        (*task).se.on_rq = false;  
        (*task).rt.queued_prio = MAX_RT_PRIO as u32;  
  
        if self.active[prio].is_empty() {  
            self.clear_bit(prio);  
        }  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 793) - [public]

pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {  
    match self.sched_find_first_bit() {  
        Some(prio) => unsafe {  
            let head = &self.active[prio] as *const ListHead as *mut ListHead;  
            let front = (*head).next;  
            if front.is_null() || core::ptr::eq(front, head) {  
                ptr::null_mut()  
            } else {  
                let rt_entity = front as *mut super::task::RtSchedEntity;  
                (*rt_entity).owner  
            }  
        },  
        None => {  
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 809) - [public] requeue

```
pub fn requeue(&mut self, task: *mut TaskStruct) {
    debug_assert!(!task.is_null());
    unsafe {
        let prio = (*task).rt.queued_prio as usize;
        debug_assert!(prio < MAX_RT_PRI0 as usize);
        (*task).rt.rt_list.remove();
        let node_ptr = &mut (*task).rt.rt_list as *mut ListHead;
        self.active[prio].insert_before(node_ptr);
    }
}

pub fn highest_priority(&self) -> i32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 820) - [public] highest_priority

```
pub fn highest_priority(&self) -> i32 {
    self.highest_prio
}

#[repr(C)]
#[derive(Default)]
pub struct RqStop {
    task: *mut TaskStruct,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 828) - [public] RqStop

```
pub struct RqStop {
    task: *mut TaskStruct,
}

impl RqStop {
    pub fn is_empty(&self) -> bool {
        self.task.is_null()
    }

    pub fn enqueue(&mut self, task: *mut TaskStruct) {
        debug_assert!(self.task.is_null(), "co najwyżej jedno zadanie Stop naraz na CPU");
        unsafe { (*task).se.on_rq = true };
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 833) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.task.is_null()
}

pub fn enqueue(&mut self, task: *mut TaskStruct) {
    debug_assert!(self.task.is_null(), "co najwyżej jedno zadanie Stop naraz na CPU");
    unsafe { (*task).se.on_rq = true };
    self.task = task;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 837) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct) {
    debug_assert!(self.task.is_null(), "co najwyzej jedno zadanie Stop naraz na CPU");
    unsafe { (*task).se.on_rq = true };
    self.task = task;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        unsafe { (*task).se.on_rq = false };
        self.task = ptr::null_mut();
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 843) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        unsafe { (*task).se.on_rq = false };
        self.task = ptr::null_mut();
    }
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.task
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 850) - [public]

pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.task
}
}
```

#[repr(C)]

```
pub struct RunQueue {
    pub lock: SpinLock,
    pub cpu: u32,
    pub online: AtomicBool,
    pub clock: AtomicU64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 857) - [public] RunQueue

```
pub struct RunQueue {
    pub lock: SpinLock,
    pub cpu: u32,
    pub online: AtomicBool,
    pub clock: AtomicU64,
    pub clock_task: AtomicU64,

    pub fair: RqFair,
    pub rt: RqRt,
    pub dl: RqDl,
    pub stop: RqStop,

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 882) - [public] new

```
pub fn new(cpu: u32, idle_task: *mut TaskStruct) -> Self {
    assert!(!idle_task.is_null(), "RunQueue::new wymaga poprawnego zadania idle");
    unsafe {
        debug_assert!(
            (*idle_task).sched_class == SchedClass::Idle,
            "zadanie przekazane jako idle musi mieć sched_class == Idle"
        );
    }
    let rq = Self {
        lock: SpinLock::new(),
        cpu,
        online: AtomicBool::new(true),
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 914) - [public]

bind_idle_task

```
pub fn bind_idle_task(&self) {
    unsafe {
        (*self.idle).set_rq_ptr(self as *const RunQueue as *mut RunQueue as *mut core::ffi::c_void);
    }
}

fn now(&self) -> u64 {
    self.clock.load(Ordering::Relaxed)
}

pub unsafe fn advance_clock(&self, delta_ns: u64) {
    debug_assert!(self.lock.is_locked(), "advance_clock wymaga self.lock");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 920) - [internal] now

```
fn now(&self) -> u64 {
    self.clock.load(Ordering::Relaxed)
}

pub unsafe fn advance_clock(&self, delta_ns: u64) {
    debug_assert!(self.lock.is_locked(), "advance_clock wymaga self.lock");
    self.clock.fetch_add(delta_ns, Ordering::Relaxed);
    self.clock_task.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn current(&self) -> *mut TaskStruct {
    self.curr.load(Ordering::Acquire)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 930) - [public] current

```
pub fn current(&self) -> *mut TaskStruct {
    self.curr.load(Ordering::Acquire)
}

fn set_current(&self, task: *mut TaskStruct) {
    self.curr.store(task, Ordering::Release);
}

pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 934) - [internal]

set_current

```
fn set_current(&self, task: *mut TaskStruct) {
    self.curr.store(task, Ordering::Release);
}

pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 938) - [public]

nr_running

```
pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert!(task.is_null(), "enqueue_task: task == null");
    debug_assert!(self.lock.is_locked(), "enqueue_task wymaga self.lock");
    debug_assert!(
        !(*task).se.on_rq,
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 942) - [public] is_idle

```
pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert!(task.is_null(), "enqueue_task: task == null");
    debug_assert!(self.lock.is_locked(), "enqueue_task wymaga self.lock");
    debug_assert!(
        !(*task).se.on_rq,
        "enqueue_task: zadanie jest już zakolejkowane (podwójny enqueue)"
    );

    let now = self.now();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1001) - [internal]

nr_uninterruptible_dec_if

```
fn nr_uninterruptible_dec_if(&self, task: *mut TaskStruct) {
    unsafe {
        if (*task).state() == TaskState::Runnable {
            let prev = self.nr_uninterruptible.load(Ordering::Relaxed);
            if prev > 0 {
                self.nr_uninterruptible.fetch_sub(1, Ordering::Relaxed);
            }
        }
    }
}

pub unsafe fn activate_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1202) - [internal]

fair_max_vruntime

```
fn fair_max_vruntime(&self) -> Option<u64> {
    unsafe {
        let max_node = rbtree::subtree_max(self.fair_root_for_test());
        if max_node.is_null() {
            None
        } else {
            Some((*max_node).se.vruntime)
        }
    }
}

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1214) - [public]

fair_root_for_test

```
pub fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

#[cfg(not(test))]
fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1219) - [internal]

fair_root_for_test

```
fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}

pub unsafe fn schedule_tail(&mut self, prev: *mut TaskStruct) {
    if prev.is_null() {
        return;
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1223) - [internal]

fair_root_internal

```
fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}

pub unsafe fn schedule_tail(&mut self, prev: *mut TaskStruct) {
    if prev.is_null() {
        return;
    }
    if (*prev).is_zombie() {
        (*prev).on_cpu.store(false, Ordering::Release);
    } else if (*prev).se.on_rq {
        (*prev).record_involuntary_switch();
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1246) - [internal] fmt

```
fn fmt(&self, f: &mut core::fmt::Formatter<'>) -> core::fmt::Result {
    f.debug_struct("RunQueue")
        .field("cpu", &self.cpu)
        .field("nr_running", &self.nr_running.load(Ordering::Relaxed))
        .field("fair_nr", &self.fair.nr_running)
        .field("rt_nr", &self.rt.nr_running)
        .field("dl_nr", &self.dl.dl_nr_running)
        .field("clock", &self.clock.load(Ordering::Relaxed))
        .finish()
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1294) - [internal]
can_migrate_task

```
fn can_migrate_task(task: *const TaskStruct, dst_cpu: u32) -> bool {
    unsafe {
        if (*task).flags.contains(TaskFlags::PF_NO_MIGRATE) {
            return false;
        }
        if (*task).on_cpu.load(Ordering::Relaxed) {
            // Fizycznie wykonywane teraz na jakimś CPU – przez ten cznik
            // kontekstu jest w toku, nie ruszamy.
            return false;
        }
        (*task).se.cpus_allowed.is_set(dst_cpu)
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1489) - [internal]
make_idle

```
fn make_idle(pid: TaskId, cpu: u32) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    let _ = cpu;
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1496) - [internal]
make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1502) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    const N: usize = 40;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut seed: u64 = 88172645463325252;
    let mut vruntimes = [0u64; N];
    for i in 0..N {
        seed ^= seed << 13;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1507) - [internal]

fair_tree_maintains_rb_invariants_under_pseudorandom_inserts

```
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    const N: usize = 40;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut seed: u64 = 88172645463325252;
    let mut vruntimes = [0u64; N];
    for i in 0..N {
        seed ^= seed << 13;
        seed ^= seed >> 7;
        seed ^= seed << 17;
        vruntimes[i] = seed % 1_000_000;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1537) - [internal]

fair_tree_inorder_traversal_is_sorted_by_vruntime

```
fn fair_tree_inorder_traversal_is_sorted_by_vruntime() {
    const N: usize = 25;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut fair = RqFair::default();
    for i in 0..N {
        tasks[i].init_test_stub(i as u64, SchedPolicy::Normal, 0);
        tasks[i].se.vruntime = ((N - i) * 1000) as u64;
        let p = ptr_of(&mut tasks[i]);
        fair.enqueue(p, EnqueueFlags::ENQUEUE_RESTORE);
    }
    let mut collected = [0u64; N];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1566) - [internal]

fair_tree_survives_random_removals_keeping_invariants

```
fn fair_tree_survives_random_removals_keeping_invariants() {
    const N: usize = 30;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut fair = RqFair::default();
    let mut ptrs: [*mut TaskStruct; N] = [ptr::null_mut(); N];
    for i in 0..N {
        tasks[i].init_test_stub(i as u64, SchedPolicy::Normal, 0);
        tasks[i].se.vruntime = (i as u64) * 137 % 5000;
        ptrs[i] = ptr_of(&mut tasks[i]);
        fair.enqueue(ptrs[i], EnqueueFlags::ENQUEUE_RESTORE);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1588) - [internal]
deadline_tree_orders_by_earliest_deadline

```
fn deadline_tree_orders_by_earliest_deadline() {
    let mut a = make_task(1, SchedPolicy::Deadline, 0);
    let mut b = make_task(2, SchedPolicy::Deadline, 0);
    let mut c = make_task(3, SchedPolicy::Deadline, 0);
    a.dl.deadline = 5000;
    b.dl.deadline = 1000;
    c.dl.deadline = 3000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut a), 0, EnqueueFlags::ENQUEUE_RESTORE);
    dl.enqueue(ptr_of(&mut b), 0, EnqueueFlags::ENQUEUE_RESTORE);
    dl.enqueue(ptr_of(&mut c), 0, EnqueueFlags::ENQUEUE_RESTORE);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1609) - [internal]
rt_queue_fifo_order_at_same_priority_without_cycles

```
fn rt_queue_fifo_order_at_same_priority_without_cycles() {
    let mut a = make_task(1, SchedPolicy::Fifo, 0);
    let mut b = make_task(2, SchedPolicy::Fifo, 0);
    let mut c = make_task(3, SchedPolicy::Fifo, 0);
    // Ta sama priorytetowa "szufladka".
    a.prio = 10;
    b.prio = 10;
    c.prio = 10;
    a.rt.rt_priority = 10;
    b.rt.rt_priority = 10;
    c.rt.rt_priority = 10;

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1648) - [internal]
rt_queue_picks_highest_priority_across_levels

```
fn rt_queue_picks_highest_priority_across_levels() {
    let mut low = make_task(1, SchedPolicy::Fifo, 0);
    let mut high = make_task(2, SchedPolicy::Fifo, 0);
    low.prio = 50;
    low.rt.rt_priority = 50;
    high.prio = 5;
    high.rt.rt_priority = 5;

    let mut rt = RqRt::new();
    rt.enqueue(ptr_of(&mut low), EnqueueFlags::ENQUEUE_RESTORE);
    rt.enqueue(ptr_of(&mut high), EnqueueFlags::ENQUEUE_RESTORE);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1667) - [internal]
rt_queue_requeue_moves_task_to_back_of_its_level

```
fn rt_queue_requeue_moves_task_to_back_of_its_level() {
    let mut a = make_task(1, SchedPolicy::RoundRobin, 0);
    let mut b = make_task(2, SchedPolicy::RoundRobin, 0);
    a.prio = 20;
    a.rt.rt_priority = 20;
    b.prio = 20;
    b.rt.rt_priority = 20;

    let mut rt = RqRt::new();
    rt.enqueue(ptr_of(&mut a), EnqueueFlags::ENQUEUE_RESTORE);
    rt.enqueue(ptr_of(&mut b), EnqueueFlags::ENQUEUE_RESTORE);

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1689) - [internal]
pick_next_task_respects_stop_over_deadline_over_rt_over_fair_over_idle
    fn pick_next_task_respects_stop_over_deadline_over_rt_over_fair_over_idle() {
        let mut idle = make_idle(0, 0);
        let mut rq = RunQueue::new(0, ptr_of(&mut idle));
        rq.bind_idle_task();

        let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
        let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
        rt_task.prio = 10;
        rt_task.rt.rt_priority = 10;
        let mut dl_task = make_task(3, SchedPolicy::Deadline, 0);
        dl_task.dl.dl_runtime = 1000;
        dl_task.dl.dl_deadline = 100_000;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1739) - [internal]
pick_next_task_never_returns_null_even_when_fully_empty
    fn pick_next_task_never_returns_null_even_when_fully_empty() {
        let mut idle = make_idle(0, 1);
        let mut rq = RunQueue::new(1, ptr_of(&mut idle));
        rq.bind_idle_task();
        unsafe {
            let flags = rq.lock.lock_irqsave();
            let picked = rq.pick_next_task();
            assert(!picked.is_null());
            assert!(core::ptr::eq(picked, ptr_of(&mut idle)));
            rq.lock.unlock_irqrestore(flags);
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1754) - [internal]
deadline_task_is_picked_even_before_its_deadline_has_passed
    fn deadline_task_is_picked_even_before_its_deadline_has_passed() {
        let mut idle = make_idle(0, 2);
        let mut rq = RunQueue::new(2, ptr_of(&mut idle));
        rq.bind_idle_task();

        let mut rt_task = make_task(1, SchedPolicy::Fifo, 0);
        rt_task.prio = 0; // najwyższy możliwy priorytet RT
        rt_task.rt.rt_priority = 99;
        let mut dl_task = make_task(2, SchedPolicy::Deadline, 0);
        dl_task.dl.dl_runtime = 1_000;
        dl_task.dl.dl_deadline = 1_000_000_000; // bardzo odległy termin
        dl_task.dl.dl_period = 1_000_000_000;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1783) - [internal]
min_vruntime_never_decreases_across_updates
    fn min_vruntime_never_decreases_across_updates() {
        let mut t = make_task(1, SchedPolicy::Normal, 0);
        let mut fair = RqFair::default();
        fair.enqueue(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);

        let m1 = fair.charge_exec(ptr_of(&mut t), 1_000_000, 1_000_000);
        let m2 = fair.charge_exec(ptr_of(&mut t), 2_000_000, 3_000_000);
        assert!(m2 >= m1);

        let m3 = fair.charge_exec(ptr_of(&mut t), 0, 3_000_000);
        assert!(m3 >= m2);
    }
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1797) - [internal]
newly_woken_task_does_not_start_before_min_vruntime

```
fn newly_woken_task_does_not_start_before_min_vruntime() {
    let mut veteran = make_task(1, SchedPolicy::Normal, 0);
    let mut newcomer = make_task(2, SchedPolicy::Normal, 0);

    let mut fair = RqFair::default();
    fair.enqueue(ptr_of(&mut veteran), EnqueueFlags::ENQUEUE_NEW);
    fair.charge_exec(ptr_of(&mut veteran), 50_000_000, 50_000_000);

    newcomer.se.vruntime = 0;
    fair.enqueue(ptr_of(&mut newcomer), EnqueueFlags::ENQUEUE_WAKEUP);

    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1817) - [internal]
higher_class_always_preempts_lower_class

```
fn higher_class_always_preempts_lower_class() {
    let mut idle = make_idle(0, 3);
    let mut rq = RunQueue::new(3, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let flags = rq.lock.lock_irqsave();
        rq.enqueue_task(ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fair_task));
        assert!(ptr_of(&mut fair_task).needs_resched());
    }

    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1842) - [internal]
fair_preemption_requires_minimum_granularity

```
fn fair_preemption_requires_minimum_granularity() {
    let idle_owner_vrt = 0u64;
    let _ = idle_owner_vrt;
    let mut current = make_task(1, SchedPolicy::Normal, 0);
    let mut candidate = make_task(2, SchedPolicy::Normal, 0);
    current.se.vruntime = 1000;
    current.se.sum_exec_runtime = 100; // dopiero co zacz██, poni██ej MIN_GRANULARITY_NS
    current.se.prev_sum_exec_runtime = 0;
    candidate.se.vruntime = 0;

    let fair = RqFair::default();
    assert!(!fair.should_preempt(&current as *const _, &candidate as *const _));
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1860) - [internal]
deadline_preempts_via_earliest_deadline_first

```
fn deadline_preempts_via_earliest_deadline_first() {
    let mut current = make_task(1, SchedPolicy::Deadline, 0);
    let mut candidate = make_task(2, SchedPolicy::Deadline, 0);
    current.dl.deadline = 5000;
    candidate.dl.deadline = 1000;

    let dl = RqDl::default();
    assert!(dl.should_preempt(&current as *const _, &candidate as *const _));
    candidate.dl.deadline = 9000;
    assert!(!dl.should_preempt(&current as *const _, &candidate as *const _));
}

// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1874) - [internal]
deadline_task_gets_throttled_when_budget_is_exhausted

```
fn deadline_task_gets_throttled_when_budget_is_exhausted() {
    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 10_000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut t), 0, EnqueueFlags::ENQUEUE_NEW);
    assert!(!t.dl.throttled);

    dl.update_curr(ptr_of(&mut t), 15_000, 20_000);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1892) - [internal]
deadline_task_is_replenished_after_period_rollover

```
fn deadline_task_is_replenished_after_period_rollover() {
    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 10_000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut t), 0, EnqueueFlags::ENQUEUE_NEW);
    assert_eq!(t.dl.deadline, 100_000);
    dl.update_curr(ptr_of(&mut t), 10_000, 100_000);

    assert!(!t.dl.throttled, "po zakończeniu okresu budżetu musi zostać odnowiony");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1909) - [internal]
deadline_admission_control_rejects_overcommitted_bandwidth

```
fn deadline_admission_control_rejects_overcommitted_bandwidth() {
    let mut dl = RqDl::default();
    assert!(dl.admission_control(60_000, 100_000));

    let mut a = make_task(1, SchedPolicy::Deadline, 0);
    a.dl.dl_runtime = 60_000;
    a.dl.dl_period = 100_000;
    dl.enqueue(ptr_of(&mut a), 0, EnqueueFlags::ENQUEUE_NEW);
    assert!(!dl.admission_control(60_000, 100_000));
    // Ale 30% wciśnięcie się mieści (60% + 30% = 90% <= 95%).
    assert!(dl.admission_control(30_000, 100_000));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1924) - [internal]
activate_and_deactivate_round_trip_updates_bookkeeping

```
fn activate_and_deactivate_round_trip_updates_bookkeeping() {
    let mut idle = make_idle(0, 4);
    let mut rq = RunQueue::new(4, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        assert_eq!(rq.nr_running(), 0);

        rq.activate_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        assert_eq!(rq.nr_running(), 1);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1951) - [internal]
double_enqueue_is_rejected_in_debug_builds

```
fn double_enqueue_is_rejected_in_debug_builds() {  
    let mut idle = make_idle(0, 5);  
    let mut rq = RunQueue::new(5, ptr_of(&mut idle));  
    rq.bind_idle_task();  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);  
        assert!((*ptr_of(&mut t)).se.on_rq);  
        rq.lock.unlock_irqrestore(flags);  
    }  
}  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1966) - [internal]
select_task_rq_prefers_idle_cpu_with_cache_affinity

```
fn select_task_rq_prefers_idle_cpu_with_cache_affinity() {  
    let mut idle0 = make_idle(100, 0);  
    let mut idle1 = make_idle(101, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    t.se.last_cpu = 1;  
  
    let registry: [*mut RunQueue; 2] = [&mut rq0 as *mut RunQueue, &mut rq1 as *mut RunQueue];  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1985) - [internal]
select_task_rq_picks_least_loaded_when_no_affinity_hit

```
fn select_task_rq_picks_least_loaded_when_no_affinity_hit() {  
    let mut idle0 = make_idle(100, 0);  
    let mut idle1 = make_idle(101, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut filler = make_task(2, SchedPolicy::Normal, 0);  
    unsafe {  
        let f0 = rq0.lock.lock_irqsave();  
        rq0.enqueue_task(ptr_of(&mut filler), EnqueueFlags::ENQUEUE_NEW);  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2011) - [internal]
wake_up_process_transitions_state_and_enqueues_remotely

```
fn wake_up_process_transitions_state_and_enqueues_remotely() {  
    let mut idle0 = make_idle(100, 0);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    rq0.bind_idle_task();  
  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    t.set_state(TaskState::Uninterruptible).unwrap();  
    t.se.last_cpu = CPU_NONE;  
  
    let registry: [*mut RunQueue; 1] = [&mut rq0 as *mut RunQueue];  
    unsafe {  
        smp::wake_up_process(ptr_of(&mut t), &registry).unwrap();  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2030) - [internal]
wake_up_process_is_idempotent_when_already_queued

```
fn wake_up_process_is_idempotent_when_already_queued() {
    let mut idle0 = make_idle(100, 0);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    rq0.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    let registry: [*mut RunQueue; 1] = [&mut rq0 as *mut RunQueue];
    unsafe {
        let flags = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq0.lock.unlock_irqrestore(flags);
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2049) - [internal]
idle_balance_pulls_one_task_from_busy_cpu

```
fn idle_balance_pulls_one_task_from_busy_cpu() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);
    let mut t2 = make_task(3, SchedPolicy::Normal, 0);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2081) - [internal]
idle_balance_does_nothing_when_balanced

```
fn idle_balance_does_nothing_when_balanced() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t0), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2105) - [internal]
migrated_task_gets_updated_cpu_and_rq_pointer

```
fn migrated_task_gets_updated_cpu_and_rq_pointer() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2140) - [internal]
no_migration_respects_cpus_allowed_mask

```
fn no_migration_respects_cpus_allowed_mask() {  
    let mut idle0 = make_idle(200, 0);  
    let mut idle1 = make_idle(201, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut t0 = make_task(1, SchedPolicy::Normal, 0);  
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);  
    t0.se.cpus_allowed = CpuMask::single(0);  
    t1.se.cpus_allowed = CpuMask::single(0);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2169) - [internal]
yield_task_pushes_vruntime_to_back_of_fair_queue

```
fn yield_task_pushes_vruntime_to_back_of_fair_queue() {  
    let mut idle = make_idle(0, 6);  
    let mut rq = RunQueue::new(6, ptr_of(&mut idle));  
    rq.bind_idle_task();  
  
    let mut a = make_task(1, SchedPolicy::Normal, 0);  
    let mut b = make_task(2, SchedPolicy::Normal, 0);  
    a.se.vruntime = 100;  
    b.se.vruntime = 5000;  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2192) - [internal]
schedule_tail_records_voluntary_vs_involuntary_switch

```
fn schedule_tail_records_voluntary_vs_involuntary_switch() {  
    let mut idle = make_idle(0, 7);  
    let mut rq = RunQueue::new(7, ptr_of(&mut idle));  
    rq.bind_idle_task();  
  
    let mut sleeper = make_task(1, SchedPolicy::Normal, 0);  
    let mut preempted = make_task(2, SchedPolicy::Normal, 0);  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut sleeper), EnqueueFlags::ENQUEUE_NEW);  
        rq.enqueue_task(ptr_of(&mut preempted), EnqueueFlags::ENQUEUE_NEW);  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2218) - [internal]
clock_advances_and_update_curr_charges_real_delta

```
fn clock_advances_and_update_curr_charges_real_delta() {  
    let mut idle = make_idle(0, 8);  
    let mut rq = RunQueue::new(8, ptr_of(&mut idle));  
    rq.bind_idle_task();  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);  
        rq.set_curr_task(ptr_of(&mut t));  
  
        assert_eq!(rq.clock.load(Ordering::Relaxed), 0);  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2243) - [internal]
update_curr_is_a_noop_on_idle_task

```
fn update_curr_is_a_noop_on_idle_task() {
    let mut idle = make_idle(0, 9);
    let mut rq = RunQueue::new(9, ptr_of(&mut idle));
    rq.bind_idle_task();
    unsafe {
        let flags = rq.lock.lock_irqsave();
        rq.advance_clock(1_000_000);
        rq.update_curr(); // curr == idle, nie powinno panikowa■ ani nic zmieni■
        rq.lock.unlock_irqrestore(flags);
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2257) - [internal]
lower_nice_task_accumulates_vruntime_slower

```
fn lower_nice_task_accumulates_vruntime_slower() {
    let mut nice_low = make_task(1, SchedPolicy::Normal, -10); // wysoki priorytet
    let mut nice_high = make_task(2, SchedPolicy::Normal, 10); // niski priorytet
    assert!(nice_to_weight(-10) > nice_to_weight(10));

    let mut fair = RqFair::default();
    fair.enqueue(ptr_of(&mut nice_low), EnqueueFlags::ENQUEUE_NEW);
    fair.enqueue(ptr_of(&mut nice_high), EnqueueFlags::ENQUEUE_NEW);

    fair.charge_exec(ptr_of(&mut nice_low), 10_000_000, 10_000_000);
    fair.charge_exec(ptr_of(&mut nice_high), 10_000_000, 10_000_000);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2281) - [internal]
load_balance_moves_task_between_two_uneven_queues

```
fn load_balance_moves_task_between_two_uneven_queues() {
    let mut idle0 = make_idle(0, 0);
    let mut idle1 = make_idle(0, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut tasks: [TaskStruct; 4] = core::array::from_fn(|_| TaskStruct::blank());
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        for (i, t) in tasks.iter_mut().enumerate() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2310) - [internal]
load_balance_is_noop_below_imbalance_threshold

```
fn load_balance_is_noop_below_imbalance_threshold() {
    let mut idle0 = make_idle(0, 0);
    let mut idle1 = make_idle(0, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t0), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 21) - [public]

account_switch

// Wygodne funkcje globalne (dla kompatybilności i szybkiego użycia)

```
pub fn account_switch() {
    TOTAL_SWITCHES.fetch_add(1, Ordering::Relaxed);
}

pub fn account_voluntary_switch(task: &mut TaskStruct) {
    TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_voluntary_switches = task.stats.nr_voluntary_switches.saturating_add(1);
}

pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 25) - [public]

account_voluntary_switch

```
pub fn account_voluntary_switch(task: &mut TaskStruct) {
    TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_voluntary_switches = task.stats.nr_voluntary_switches.saturating_add(1);
}

pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 30) - [public]

account_involuntary_switch

```
pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_migrations = task.stats.nr_migrations.saturating_add(1);
    task.se.nr_migrations = task.se.nr_migrations.saturating_add(1);
}

pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 35) - [public]

account_migration

```
pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_migrations = task.stats.nr_migrations.saturating_add(1);
    task.se.nr_migrations = task.se.nr_migrations.saturating_add(1);
}

pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.utime = task.stats.utime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 41) - [public]

account_user_time

```
pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.uptime = task.stats.uptime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

pub fn account_system_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.stime = task.stats.stime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 48) - [public]

account_system_time

```
pub fn account_system_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.stime = task.stats.stime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

pub fn account_idle_time(delta_ns: u64) {
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn account_iowait_time(delta_ns: u64) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 55) - [public]

account_idle_time

```
pub fn account_idle_time(delta_ns: u64) {
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn account_iowait_time(delta_ns: u64) {
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

// Gettery globalne
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 59) - [public]

account_iowait_time

```
pub fn account_iowait_time(delta_ns: u64) {
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

// Gettery globalne
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 64) - [public]

total_switches

// Gettery globalne

```
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 65) - [public]

total_voluntary_switches

```
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 66) - [public]

total_involuntary_switches

```
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

pub struct RqStats {

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 67) - [public]

total_migrations

```
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

pub struct RqStats {

/// Liczba przebiegów kontekstu na tym CPU.

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 68) - [public]

total_user_time_ns

```
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 69) - [public]

total_system_time_ns

```
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 70) - [public]

total_idle_time_ns

```
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 71) - [public]

total_iowait_time_ns

```
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 72) - [public]
total_running_time_ns
    pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    pub nr_involuntary_switches: u64,
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 77) - [public] RqStats
```

```
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    pub nr_involuntary_switches: u64,
    /// Liczba migracji zadań z/do tego CPU.
    pub nr_migrations: u64,
    /// Czas spędzony w trybie użytkownika (ns).
    pub user_time_ns: u64,
    /// Czas spędzony w trybie jądra (ns).
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 126) - [public]
```

```
record_switch
    pub fn record_switch(&mut self, voluntary: bool) {
        self.nr_switches += 1;
        if voluntary {
            self.nr_voluntary_switches += 1;
        } else {
            self.nr_involuntary_switches += 1;
        }
        // Aktualizuj globalne liczniki
        TOTAL_SWITCHES.fetch_add(1, Ordering::Relaxed);
        if voluntary {
            TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
        } else {
            // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 143) - [public]
```

```
record_migration
    pub fn record_migration(&mut self) {
        self.nr_migrations += 1;
        TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    }

    #[inline]
    pub fn add_user_time(&mut self, delta_ns: u64) {
        self.user_time_ns += delta_ns;
        self.running_time_ns += delta_ns;
        TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
        TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 149) - [public]

add_user_time

```
pub fn add_user_time(&mut self, delta_ns: u64) {
    self.user_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_system_time(&mut self, delta_ns: u64) {
    self.system_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 157) - [public]

add_system_time

```
pub fn add_system_time(&mut self, delta_ns: u64) {
    self.system_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_idle_time(&mut self, delta_ns: u64) {
    self.idle_time_ns += delta_ns;
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 165) - [public]

add_idle_time

```
pub fn add_idle_time(&mut self, delta_ns: u64) {
    self.idle_time_ns += delta_ns;
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_iowait_time(&mut self, delta_ns: u64) {
    self.iowait_time_ns += delta_ns;
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 171) - [public]

add_iowait_time

```
pub fn add_iowait_time(&mut self, delta_ns: u64) {
    self.iowait_time_ns += delta_ns;
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn set_nr_running(&mut self, nr: u32) {
    self.nr_running = nr;
}

#[inline]
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 177) - [public]

set_nr_running

```
pub fn set_nr_running(&mut self, nr: u32) {
    self.nr_running = nr;
}

#[inline]
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    self.nr_uninterruptible = nr;
}

#[inline]
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 182) - [public]

set_nr_uninterruptible

```
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    self.nr_uninterruptible = nr;
}

#[inline]
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    self.sum_weight = sum_weight;
}

/// Oblicza przybliżone obciążenie CPU (0-100).
pub fn load_percent(&self) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 187) - [public]

update_load_stats

```
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    self.sum_weight = sum_weight;
}

/// Oblicza przybliżone obciążenie CPU (0-100).
pub fn load_percent(&self) -> u32 {
    if self.running_time_ns == 0 {
        return 0;
    }
    let total = self.running_time_ns + self.idle_time_ns;
    if total == 0 {
        return 0;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 193) - [public] load_percent

/// Oblicza przybliżone obciążenie CPU (0-100).

```
pub fn load_percent(&self) -> u32 {
    if self.running_time_ns == 0 {
        return 0;
    }
    let total = self.running_time_ns + self.idle_time_ns;
    if total == 0 {
        return 0;
    }
    ((self.running_time_ns * 100) / total) as u32
}

pub fn load_avg_1min(&self) -> f32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 204) - [public]

```
load_avg_1min
    pub fn load_avg_1min(&self) -> f32 {
        self.load_percent() as f32 / 100.0
    }
}

// -----

/// Prosty histogram czasów wykonania (w ns) w skali logarytmicznej.
#[derive(Debug, Clone, Copy)]
pub struct ExecTimeHistogram {
    // 16 przedziałów: 0-1us, 1-2us, 2-4us, 4-8us, ... >8ms
    buckets: [u64; 16],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 213) - [public]

```
ExecTimeHistogram
    pub struct ExecTimeHistogram {
        // 16 przedziałów: 0-1us, 1-2us, 2-4us, 4-8us, ... >8ms
        buckets: [u64; 16],
        total_samples: u64,
        total_time_ns: u64,
    }

    impl ExecTimeHistogram {
        pub const fn new() -> Self {
            Self {
                buckets: [0; 16],
                total_samples: 0,
                // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 229) - [public] add_sample

```
    pub fn add_sample(&mut self, exec_ns: u64) {
        self.total_samples += 1;
        self.total_time_ns += exec_ns;

        // Wyznacz indeks na podstawie log2, z dolnym progiem 1024 ns
        let mut idx = 0;
        let mut threshold = 1024u64; // 1us
        let mut upper = 2048u64;
        for i in 0..15 {
            if exec_ns < upper {
                idx = i;
                break;
            }
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 251) - [public] average_ns

```
    pub fn average_ns(&self) -> f32 {
        if self.total_samples == 0 {
            0.0
        } else {
            self.total_time_ns as f32 / self.total_samples as f32
        }
    }

    pub fn total_samples(&self) -> u64 {
        self.total_samples
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 259) - [public]

```
total_samples
    pub fn total_samples(&self) -> u64 {
        self.total_samples
    }
}

impl Default for ExecTimeHistogram {
    fn default() -> Self {
        Self::new()
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 265) - [internal] default

```
fn default() -> Self {
    Self::new()
}

/// Zbiorcza struktura do raportowania wszystkich statystyk systemu.
#[derive(Debug, Default, Clone)]
pub struct SystemStats {
    pub total_switches: u64,
    pub total_voluntary_switches: u64,
    pub total_involuntary_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 273) - [public] SystemStats

```
pub struct SystemStats {
    pub total_switches: u64,
    pub total_voluntary_switches: u64,
    pub total_involuntary_switches: u64,
    pub total_migrations: u64,
    pub total_user_time_ns: u64,
    pub total_system_time_ns: u64,
    pub total_idle_time_ns: u64,
    pub total_iowait_time_ns: u64,
    pub total_running_time_ns: u64,
    pub per_cpu: Vec<CpuStatsSnapshot>,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 287) - [public]

```
CpuStatsSnapshot

pub struct CpuStatsSnapshot {
    pub cpu: u32,
    pub nr_switches: u64,
    pub user_time_ns: u64,
    pub system_time_ns: u64,
    pub idle_time_ns: u64,
    pub iowait_time_ns: u64,
    pub running_time_ns: u64,
    pub load_percent: u32,
}

pub fn collect_system_stats() -> SystemStats {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 298) - [public]

collect_system_stats

```
pub fn collect_system_stats() -> SystemStats {
    SystemStats {
        total_switches: TOTAL_SWITCHES.load(Ordering::Relaxed),
        total_voluntary_switches: TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed),
        total_involuntary_switches: TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed),
        total_migrations: TOTAL_MIGRATIONS.load(Ordering::Relaxed),
        total_user_time_ns: TOTAL_USER_TIME_NS.load(Ordering::Relaxed),
        total_system_time_ns: TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed),
        total_idle_time_ns: TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed),
        total_iowait_time_ns: TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed),
        total_running_time_ns: TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed),
        per_cpu: Vec::new(), // W praktyce wypelnia z tablicy CPU
    } // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 315) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    writeln!(f, "CPU stats:");
    writeln!(f, "  switches: {} (voluntary {}, involuntary {})",
        self.nr_switches, self.nr_voluntary_switches, self.nr_involuntary_switches);
    writeln!(f, "  migrations: {}", self.nr_migrations);
    writeln!(f, "  user time: {} ns", self.user_time_ns);
    writeln!(f, "  system time: {} ns", self.system_time_ns);
    writeln!(f, "  idle time: {} ns", self.idle_time_ns);
    writeln!(f, "  iowait time: {} ns", self.iowait_time_ns);
    writeln!(f, "  running time: {} ns", self.running_time_ns);
    writeln!(f, "  load: {}%", self.load_percent());
    writeln!(f, "  nr_running: {}, nr_uninterruptible: {}",
        self.nr_running, self.nr_uninterruptible);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 332) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    writeln!(f, "System-wide scheduler stats:");
    writeln!(f, "  total switches: {}", self.total_switches);
    writeln!(f, "  voluntary: {}", self.total_voluntary_switches);
    writeln!(f, "  involuntary: {}", self.total_involuntary_switches);
    writeln!(f, "  migrations: {}", self.total_migrations);
    writeln!(f, "  user time: {} ns", self.total_user_time_ns);
    writeln!(f, "  system time: {} ns", self.total_system_time_ns);
    writeln!(f, "  idle time: {} ns", self.total_idle_time_ns);
    writeln!(f, "  iowait time: {} ns", self.total_iowait_time_ns);
    writeln!(f, "  running time: {} ns", self.total_running_time_ns);
    if !self.per_cpu.is_empty() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 44) - [public] TaskFlags

```
pub struct TaskFlags: u32 {
    const PF_KTHREAD = 1 << 0;
    const PF_EXITING = 1 << 1;
    const PF_EXITPIDONE = 1 << 2;
    const PF_FORKNOEXEC = 1 << 3;
    const PF_WQ_WORKER = 1 << 4;
    const PF_NO_SETAFFINITY = 1 << 5;
    const PF_IDLE = 1 << 6;
    const PF_MEMALLOC = 1 << 7;
    const PF_FROZEN = 1 << 8;
    const PF_SUPERPRIV = 1 << 9;
    const PF_DUMPCORE = 1 << 10;
    // ... (truncated, see source file for more)
}
```


kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 73) - [public]

AtomicTaskFlags

```
pub struct AtomicTaskFlags(AtomicU32);

impl AtomicTaskFlags {
    pub const fn new(flags: TaskFlags) -> Self {
        Self(AtomicU32::new(flags.bits()))
    }

    pub const fn empty() -> Self {
        Self::new(TaskFlags::empty())
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 85) - [public] load

```
pub fn load(&self, order: Ordering) -> TaskFlags {
    TaskFlags::from_bits_truncate(self.0.load(order))
}

#[inline]
pub fn store(&self, flags: TaskFlags, order: Ordering) {
    self.0.store(flags.bits(), order);
}

#[inline]
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 90) - [public] store

```
pub fn store(&self, flags: TaskFlags, order: Ordering) {
    self.0.store(flags.bits(), order);
}

#[inline]
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

/// Czy❏ci bity `flags`, zwraca poprzedni❏ warto❏.
#[inline]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 95) - [public] fetch_insert

```
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

/// Czy❏ci bity `flags`, zwraca poprzedni❏ warto❏.
#[inline]
pub fn fetch_remove(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_and(!flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 102) - [public] fetch_remove
    pub fn fetch_remove(&self, flags: TaskFlags) -> TaskFlags {
        let prev = self.0.fetch_and(!flags.bits(), Ordering::AcqRel);
        TaskFlags::from_bits_truncate(prev)
    }

    #[inline]
    pub fn contains(&self, flags: TaskFlags) -> bool {
        self.load(Ordering::Acquire).contains(flags)
    }

    #[inline]
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 108) - [public] contains
    pub fn contains(&self, flags: TaskFlags) -> bool {
        self.load(Ordering::Acquire).contains(flags)
    }

    #[inline]
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        let prev = self.fetch_insert(flags);
        !prev.contains(flags)
    }
}

impl Clone for AtomicTaskFlags {
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 113) - [public] test_and_set
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        let prev = self.fetch_insert(flags);
        !prev.contains(flags)
    }
}

impl Clone for AtomicTaskFlags {
    fn clone(&self) -> Self {
        Self::new(self.load(Ordering::Acquire))
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 120) - [internal] clone
    fn clone(&self) -> Self {
        Self::new(self.load(Ordering::Acquire))
    }
}

#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum TaskState {
    Runnable = 0,
    Interruptible = 1,
    Uninterruptible = 2,
    Stopped = 3,
    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 127) - [public] TaskState

```
pub enum TaskState {
    Runnable = 0,
    Interruptible = 1,
    Uninterruptible = 2,
    Stopped = 3,
    Traced = 4,
    Zombie = 5,
    Dead = 6,
    Idle = 7,
}

impl TaskState {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 200) - [internal] default

```
fn default() -> Self {
    TaskState::Dead
}

#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SchedPolicy {
    Normal = 0,
    Batch = 1,
    Idle = 2,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 208) - [public] SchedPolicy

```
pub enum SchedPolicy {
    Normal = 0,
    Batch = 1,
    Idle = 2,
    Fifo = 3,
    RoundRobin = 4,
    Deadline = 5,
    Stop = 6,
}

impl SchedPolicy {
    pub const fn is_realtime(self) -> bool {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 230) - [public] SchedClass

```
pub enum SchedClass {
    Idle = 0,
    Fair = 1,
    RealTime = 2,
    Deadline = 3,
    Stop = 4,
}

impl From<SchedPolicy> for SchedClass {
    fn from(policy: SchedPolicy) -> Self {
        match policy {
            SchedPolicy::Idle => SchedClass::Idle,
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 239) - [internal] from

```
fn from(policy: SchedPolicy) -> Self {
    match policy {
        SchedPolicy::Idle => SchedClass::Idle,
        SchedPolicy::Normal | SchedPolicy::Batch => SchedClass::Fair,
        SchedPolicy::Fifo | SchedPolicy::RoundRobin => SchedClass::RealTime,
        SchedPolicy::Deadline => SchedClass::Deadline,
        SchedPolicy::Stop => SchedClass::Stop,
    }
}
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 253) - [public] CpuMask

```
pub struct CpuMask {
    bits: [u64; CPUMASK_WORDS],
}

impl CpuMask {
    pub const fn empty() -> Self {
        Self { bits: [0; CPUMASK_WORDS] }
    }

    pub const fn all() -> Self {
        Self { bits: [u64::MAX; CPUMASK_WORDS] }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 266) - [public] single

```
pub fn single(cpu: u32) -> Self {
    let mut m = Self::empty();
    m.set(cpu);
    m
}

pub fn first_n(n: u32) -> Self {
    let mut m = Self::empty();
    let n = cmp::min(n as usize, MAX_CPUS);
    for cpu in 0..n {
        m.set(cpu as u32);
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 272) - [public] first_n

```
pub fn first_n(n: u32) -> Self {
    let mut m = Self::empty();
    let n = cmp::min(n as usize, MAX_CPUS);
    for cpu in 0..n {
        m.set(cpu as u32);
    }
    m
}

pub fn set(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 281) - [public] set

```
pub fn set(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] |= 1u64 << (cpu % 64);
    }
}

pub fn clear(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] &= !(1u64 << (cpu % 64));
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 288) - [public] clear

```
pub fn clear(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] &= !(1u64 << (cpu % 64));
    }
}

pub fn is_set(&self, cpu: u32) -> bool {
    let cpu = cpu as usize;
    cpu < MAX_CPUS && (self.bits[cpu / 64] & (1u64 << (cpu % 64))) != 0
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 295) - [public] is_set

```
pub fn is_set(&self, cpu: u32) -> bool {
    let cpu = cpu as usize;
    cpu < MAX_CPUS && (self.bits[cpu / 64] & (1u64 << (cpu % 64))) != 0
}

pub fn is_empty(&self) -> bool {
    self.bits.iter().all(|w| *w == 0)
}

pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 300) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.bits.iter().all(|w| *w == 0)
}

pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}

pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 304) - [public] count

```
pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}

pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 308) - [public] intersects

```
pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    out
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 312) - [public] and

```
pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    out
}

pub fn or(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] | other.bits[i];
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 320) - [public] or

```
pub fn or(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] | other.bits[i];
    }
    out
}

pub fn first(&self) -> Option<u32> {
    for (word_idx, word) in self.bits.iter().enumerate() {
        if *word != 0 {
            return Some((word_idx * 64 + word.trailing_zeros() as usize) as u32);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 328) - [public] first

```
pub fn first(&self) -> Option<u32> {
    for (word_idx, word) in self.bits.iter().enumerate() {
        if *word != 0 {
            return Some((word_idx * 64 + word.trailing_zeros() as usize) as u32);
        }
    }
    None
}

pub fn next_after(&self, after: u32) -> Option<u32> {
    let start = after.wrapping_add(1);
    for cpu in start..MAX_CPUS as u32 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 337) - [public] next_after

```
pub fn next_after(&self, after: u32) -> Option<u32> {
    let start = after.wrapping_add(1);
    for cpu in start..MAX_CPUS as u32 {
        if self.is_set(cpu) {
            return Some(cpu);
        }
    }
    for cpu in 0..=after.min(MAX_CPUS as u32 - 1) {
        if self.is_set(cpu) {
            return Some(cpu);
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 352) - [public] iter

```
pub fn iter(&self) -> CpuMaskIter {
    CpuMaskIter { mask: *self, next_cpu: 0 }
}

impl Default for CpuMask {
    fn default() -> Self {
        CpuMask::empty()
    }
}

pub struct CpuMaskIter {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 358) - [internal] default

```
fn default() -> Self {
    CpuMask::empty()
}

pub struct CpuMaskIter {
    mask: CpuMask,
    next_cpu: usize,
}

impl Iterator for CpuMaskIter {
    type Item = u32;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 363) - [public] CpuMaskIter

```
pub struct CpuMaskIter {
    mask: CpuMask,
    next_cpu: usize,
}

impl Iterator for CpuMaskIter {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        while self.next_cpu < MAX_CPUS {
            let cpu = self.next_cpu as u32;
            self.next_cpu += 1;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 371) - [internal] next

```
fn next(&mut self) -> Option<u32> {
    while self.next_cpu < MAX_CPUS {
        let cpu = self.next_cpu as u32;
        self.next_cpu += 1;
        if self.mask.is_set(cpu) {
            return Some(cpu);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 406) - [public] nice_to_weight

```
pub fn nice_to_weight(nice: i8) -> u64 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WEIGHT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn nice_to_wmult(nice: i8) -> u32 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WMULT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 411) - [public] nice_to_wmult

```
pub fn nice_to_wmult(nice: i8) -> u32 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WMULT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    let mut best_diff = u64::MAX;
    for (idx, w) in NICE_TO_WEIGHT_TABLE.iter().enumerate() {
        let diff = if *w > weight { *w - weight } else { weight - *w };
        if diff < best_diff {
            best_diff = diff;
        }
    }
    // ... (truncated, see source file for more)
}
```


kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 416) - [public]

weight_to_nice

```
pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    let mut best_diff = u64::MAX;
    for (idx, w) in NICE_TO_WEIGHT_TABLE.iter().enumerate() {
        let diff = if *w > weight { *w - weight } else { weight - *w };
        if diff < best_diff {
            best_diff = diff;
            best_idx = idx;
        }
    }
    NICE_MIN + best_idx as i8
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 431) - [public]

calc_delta_fair

```
pub fn calc_delta_fair(delta_exec: u64, weight: u64, inv_weight: u32) -> u64 {
    if weight == NICE_0_LOAD {
        return delta_exec;
    }
    let product = (delta_exec as u128) * (inv_weight as u128);
    (product >> WMULT_SHIFT) as u64
}
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct InterruptFrame {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 442) - [public]

InterruptFrame

```
pub struct InterruptFrame {
    pub rip: u64,
    pub cs: u64,
    pub rflags: u64,
    pub rsp: u64,
    pub ss: u64,
}

#[repr(C, align(16))]
#[derive(Clone, Copy)]
pub struct FxSaveArea {
    pub bytes: [u8; 512],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 452) - [public] FxSaveArea

```
pub struct FxSaveArea {
    pub bytes: [u8; 512],
}

impl FxSaveArea {
    pub const fn zeroed() -> Self {
        Self { bytes: [0u8; 512] }
    }
}

impl Default for FxSaveArea {
    fn default() -> Self {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 463) - [internal] default
    fn default() -> Self {
        Self::zeroed()
    }
}

impl fmt::Debug for FxSaveArea {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "FxSaveArea(512 B @ {:p})", self.bytes.as_ptr())
    }
}

#[repr(C)]
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 469) - [internal] fmt
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "FxSaveArea(512 B @ {:p})", self.bytes.as_ptr())
    }
}

#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct CpuContext {
    pub rsp: u64,
    pub rdi: u64,
    pub rbx: u64,
    pub rbp: u64,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 476) - [public] CpuContext
pub struct CpuContext {
    pub rsp: u64,
    pub rdi: u64,
    pub rbx: u64,
    pub rbp: u64,
    pub r12: u64,
    pub r13: u64,
    pub r14: u64,
    pub r15: u64,
    pub rip: u64,
    pub fs_base: u64,
    pub gs_base: u64,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 493) - [internal] default
    fn default() -> Self {
        Self {
            rsp: 0,
            rdi: 0,
            rbx: 0,
            rbp: 0,
            r12: 0,
            r13: 0,
            r14: 0,
            r15: 0,
            rip: 0,
            fs_base: 0,
            // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 548) - [public]

xsave_area_size

```
pub fn xsave_area_size() -> usize {
    core::mem::size_of::<FxSaveArea>()
}

pub const RB_RED: usize = 0;
pub const RB_BLACK: usize = 1;

#[inline]
pub fn rb_parent(pc: usize) -> *mut TaskStruct {
    (pc & !1) as *mut TaskStruct
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 557) - [public] rb_parent

```
pub fn rb_parent(pc: usize) -> *mut TaskStruct {
    (pc & !1) as *mut TaskStruct
}

#[inline]
pub fn rb_color(pc: usize) -> usize {
    pc & 1
}

#[inline]
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 562) - [public] rb_color

```
pub fn rb_color(pc: usize) -> usize {
    pc & 1
}

#[inline]
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
}

#[inline]
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 567) - [public] rb_is_red

```
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
}

#[inline]
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
}

#[inline]
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 572) - [public] rb_is_black

```
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
}

#[inline]
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
}

#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct LoadAvg {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 577) - [public]
rb_make_parent_color

```
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
}

#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct LoadAvg {
    pub last_update_time: u64,
    pub load_sum: u64,
    pub load_avg: u32,
    pub util_sum: u64,
    pub util_avg: u32,
    // ... (truncated, see source file for more)
```

(List truncated to 4000 entries to keep the document size manageable.)

4. .md files (in order)

README.md

```
# TrangorgeOS

# TrangorgeOS

> A modern, high-performance bare-metal operating system built ground-up on a custom **Separated Tri-Partition Architecture**

---

## Overview & Philosophy

TrangorgeOS is built with a strict **from-scratch philosophy** (~35,000+ lines of code, actively expanding)

The project operates under an intensive development cycle aimed at producing a stable, bare-metal kernel

---

## Visuals & Screenshots

| Kernel Loading & Memory Allocation Testing | Kernel Base Resolution | Kernel in 1080p |
|:---:|:---:|:---:|
| ![Kernel Loading](kernelloading.png) | ![Kernel Base Res](kernel_loader_base_res.png) | ![Kernel 1080p](kernel_1080p.png) |

---

## Architectural Model: Separated Tri-Partition Architecture

TrangorgeOS decouples driver functionality and system privilege into 4 distinct operational layers (3 main components):

...

+-----+
|                                     |
|             USERSPACE              |
| - Applications & High-Level Libraries |
|                                     |
+-----+
|             USER DRIVER SPACE (UDS) |
| - High-risk / Peripherals (Fault-isolated, strict boundary) |
|                                     |
+-----+
|             DRIVER SPACE (DS)       |
| * Dynamic Drivers   : GPU & complex hardware |
| * Static Drivers    : Init & single-action hardware setup |
| * Library Drivers   : Inter-driver interface providers |
|                                     |
+-----+
|             KERNEL CORE / DRIVERS   |
| - Trusted Core Drivers (Network, USB stack base, Core MM) |
| - Fine-grained Kernel Policy Enforcement & Data Flow Control |
|                                     |
+-----+

...

1. **Kernel Core & Trusted Drivers:** Holds only maximum-trust drivers (e.g., base network, USB stack core)
2. **Driver Space (DS):** Modular driver execution environment with strict error margins:
   - **Dynamic Drivers:** Handle complex, stateful hardware (e.g., GPU control).
   - **Static Drivers:** Perform hardware initialization or non-exporting, single-purpose setup.
   - **Library Drivers:** Expose specialized interfaces (e.g., PCI, HDMI control) for other drivers to control hardware.
3. **User Driver Space (UDS):** High-level peripheral drivers isolated at a safe distance from the core to prevent system crashes.
4. **Userspace & User Library Drivers:** Top-level space housing application runtime and user-level libraries.

---

## Current Subsystems & Refactoring Status

> **Notice:** The kernel is currently undergoing a major refactoring phase (modernization and safety audit)

- **Memory Management (MM):** Fully custom memory management subsystem and dynamic allocator written in C
- **Network & USB:** Network driver base operational; USB stack operating under a controlled bare-metal environment
- **Input & Display:** Built-in BIOS PS/2 fallback support; active development on modern USB input. Kernel
```

```

- **Multiprocessing & Interrupts:** SMP multi-core initialization active. APIC/IDT interrupt architecture
- **Binary Support:** Native execution support for `ELF` and custom `.bin` binaries with custom IPC inter

---

## Custom Versioning System

TrangorgeOS uses an in-house versioning scheme reflecting the precise state of development:

 $\text{\$}\text{\$}\text{\text{Major}}\text{\$}\text{\$} . \text{\text{Changes}} . \text{\text{FixesPerChange}} . \text{\text{Iteration}}\text{\text{State}}\text{\$}\text{\$}$ 

*Example: `v0.182.7.1a`*
- **`0` (Major):** Pre-release major build.
- **`182` (Changes):** Total cumulative feature changes introduced.
- **`7` (Fixes):** Bugfixes applied for the current change.
- **`1` (Iteration):** Single target architecture fully active.
- **`a` (State):** Alpha phase (`a` = initial work  $\rightarrow$  `b` = stabilization  $\rightarrow$  `g` =

### Scale Milestones:
- **Alpha:** Pre-release scale (up to ~60,000 LOC), active architectural building.
- **Beta:** System runs stably on physical hardware.
- **Gamma:** Final stabilization phase prior to Release `v1.0`.

---

## Git Branch Model & Workflow

The repository relies on a strict multi-tier branch hierarchy:

| Branch | Purpose |
|:---|:---|
| `init` | Conceptual structure, architecture blueprints, no active codebase. |
| `new` | Experimental features and isolated proof-of-concept tests. |
| `unstable` | Primary active development branch. |
| `stabilizing` | Refactoring, bug-hunting, and code modernizing before release. |
| `stable` | Tested, incremental updates with guaranteed stability. |
| `main` | Production showcase branch. |

---

## Build

The project builds as two Cargo workspaces:

- **`kernel_workspace`** – the kernel and its bootable image (`kernel-bin`).
- **`kernel-drivers`** – the exported hardware-driver libraries (`pcie`, `usb`, `audio`, ...), extracted f

Build orchestration is handled with [Just](https://github.com/casey/just) (`justfile`):

```sh
just # build everything (driver libraries -> kernel bootimage)
just drivers # build the driver libraries only
just kernel # build the kernel bootimage
just run # build + run the kernel in QEMU
just check # cargo check (no linking)
```

> **Note:** the experimental `ctrlfile` build tool was **excluded from the repository** for now – it is r

---

## Project Roadmap

- [x] **Current Stage (`stabilizing` / Alpha v0.182.x):** Deep refactoring of MM allocator, scheduler mod
- [ ] **Milestone 1 (~1.5 Months):** Complete memory allocator refactoring, lock down current subsystem r
- [ ] **Milestone 2 (~4 Months):** Mature active drivers, write dedicated GPU/display drivers, initiate m
- [ ] **Milestone 3 (~1 Year):** Complete full bare-metal testing on physical hardware, establish stable

---



<i>TrangorgeOS – Building bare-metal systems from scratch, one commit at a time.</i>


```

User space contains applications, libraries, tools, and peripheral drivers that do not need direct access

5. Driver-space communication

Driver-space communication uses two ring buffers. A message contains an identifier, command, flags, three

```
```text
Driver or user-facing component
 |
 v
 driverspacelib API
 |
 v
 d2k / k2d ring messages
 |
 v
Kernel-side service dispatcher
 |
 +--> physical memory and page mapping
 +--> PCI / USB / device registry
 +--> MMIO and DMA permissions
 +--> block and audio services
...
```
```

The intended rule is that a driver should not duplicate kernel mechanisms, while the kernel should not ne

6. Kernel subsystem map

| Subsystem | Main location | Responsibility |
|-----------------------------|--|---|
| Entry and diagnostics | `kernel/src/main.rs`, `serial.rs`, `vga_buffer.rs`, `testing.rs` | Boot-time er |
| ABI | `kernel/src/abi` | Allocation, memory, filesystem, and public API contracts. |
| CPU and interrupts | `cpu/`, `gdt.rs`, `interrupts.rs` | ACPI, LAPIC, GDT, trampolines, CPU state, and |
| Memory | `mm/`, `allocator/` | Physical and virtual memory, paging, heap, cache, protection, DMA, and a |
| Filesystem | `fs/` | MBR, disk access, ATA PIO, block drivers, FAT32, EXT4, TFS, and initialization. |
| Hardware drivers | `drivers/`, `pci.rs` | PCI, USB, HID, mass storage, host controllers, and glue code. |
| Graphics and terminal | `gfx/`, `terminal/`, `vga_buffer.rs` | Framebuffer, VGA, fonts, console, termin |
| Networking | `nic/` | Ethernet, packets, protocols, device abstractions, and VirtIO networking. |
| Installation and updates | `ctrlinstall/` | Repositories, indexes, manifests, dependency resolution, tr |
| Driver-space initialization | `driverspaceinit/` | Address-space setup, shared memory, initialization, |

The memory subsystem bootstrap order is explicitly visible in `mm\_init()`: `arch\_memory\_init` → `paging_

7. Memory and allocator architecture

The memory subsystem is divided into physical memory, virtual memory, heap allocation, caching, DMA/conti

```
```text
Rust and C allocation APIs
 |
 v
Allocation API and FFI bridge
 |
 +--> physical memory: bitmap, frames, PMM
 +--> virtual memory: VMM, mapping, pages, paging, TLB
 +--> heap: buddy allocator, optional slab allocator
 +--> special memory: contiguous allocation and DMA
 +--> cache: object cache and per-CPU support
 +--> protection: guard, isolation, permissions
 +--> process spaces: address_space and mmap
...
```
```

7.1. Initialization

`mm\_init()` validates boot memory-map parameters and initializes the memory stack in a deliberate order.

7.2. Active heap path

The active heap path uses the buddy allocator. `heap.c` reserves a 256 MiB buddy region and maps physical

The buddy implementation tracks allocation orders, free lists, mapped pages, used/free statistics, and bu

7.3. Required allocator documentation

The next allocator specification should define ownership, alignment, blocking behavior, interrupt-context

8. Repository map

| Directory | Role |
|--------------------------|--|
| kernel/ | Main kernel and its memory, filesystem, CPU, networking, ABI, and hardware layers. |
| drivers/ | Packages for concrete device drivers such as GPU, audio, camera, and Wacom. |
| driverspace/ | Driver-space executable or runtime and its public API. |
| driverspacelib/ | Shared driver-space ABI, runtime, logging, memory, ring, audio, and block helpers. |
| trangorgelibc/ | Client-side library and syscall/ABI-facing facilities. |
| comgrub/ | GRUB boot component. |
| comlimine/ | Limine boot component. |
| iso-builder/ | ISO-image construction tool. |
| triang-lang/ | Lexer, parser, AST, semantic analysis, IR, and C/assembly code generation. |
| mp4_to_bmp/ | Auxiliary media-conversion tool. |
| targets/ | Build-target configuration. |
| Docker files and scripts | Reproducible build and run environment. |

9. Build and boot components

The repository contains both `comgrub` and `comlimine`, indicating two boot-component paths. The project

10. Current risks and open questions

The driver-space mechanism is present, but its long-term ABI and lifecycle policy are not yet fully speci

The public source also requires a careful compile check around `kernel/src/driverspaceinit/init/init.rs`:

Architecture support should be tracked per target. Listing ARM64 and RISC-V on the website does not by it

11. Summary

TrangorgeOS has a clear architectural direction: a hybrid kernel with a three-way driver split, a layered

The highest-value next steps are to stabilize the driver-space ABI, make the allocator contract explicit,

References

[1]: <https://github.com/CTRL-F-org3/TrangorgeOS/tree/unstable> "TrangorgeOS – unstable branch"

[2]: <https://trangorgeos.website/> "Official TrangorgeOS website"

[3]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/driverspaceinit/init/init.rs> "Tr

[4]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/driverspaceinit/init/service.rs>

[5]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/alloc/heap/heap.c> "TrangorgeO

[6]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/alloc/api/alloc.c> "TrangorgeO

[7]: <https://github.com/CTRL-F-org3/TrangorgeOS/blob/unstable/kernel/src/mm/core/mm.c> "TrangorgeOS – memo

TrangorgeOS — TODO.md

TrangorgeOS – TODO

> **Purpose:** This file tracks the remaining work required to move TrangorgeOS from the current `unstable`
>
> **Status convention:** `TODO` means not started or not verified; `WIP` means partially implemented; `BL`
>
> **Priority convention:** `P0` blocks reliable boot or basic correctness; `P1` is required for the next

0. Current snapshot

| Area | Current state | Priority |
|---------------------------------|---|----------|
| Kernel boot and x86_64 baseline | Present, but requires repeatable build and boot verification | P0 |
| Memory subsystem | Layered implementation exists; initialization order is defined in <code>mm_init()</code> | P0 |
| Heap allocator | Buddy path is active; slab implementation exists but <code>HEAP_USE_SLAB</code> is <code>0</code> | P1 |
| Driver space | Separate address space, shared rings, initialization parameters, and services are present | |
| Driver-space ABI | Implemented enough for experiments, not yet documented as stable | P0 |
| Filesystem | FAT32/EXT4 and additional filesystem code are present; correctness and integration tests a | |
| USB/PCI | Significant infrastructure exists; device coverage and hardware validation are incomplete | P1 |
| Networking | NIC/VirtIO structures exist; network-driver work is explicitly a project priority | P1 |

| ARM64 and RISC-V | Public target goals; feature parity and build/boot status must be established | P2 |
| Package manager | Kernel-side `ctrlinstall` modules exist; end-to-end package workflow needs completion |
| Toolchain and ecosystem | `triang-lang`, libraries, ISO tooling, and auxiliary tools exist at different levels |
| Release process | No published release; stable branch criteria are not yet formalized | P1 |

1. P0 – immediate correctness blockers

1.1 Repair and compile-check driver-space initialization

Status: TODO

Area: `kernel/src/driverspaceinit/init/init.rs`

The visible implementation creates a `Driverspace` structure with an address space, two ring physical address spaces, and a scratch page.

Tasks:

- Add or remove scratch-page state consistently in the `Driverspace` structure and initialization path.
- Ensure the scratch page is allocated, zeroed, mapped, and released exactly once.
- Ensure `scratch\_view()` cannot return a pointer after the driver-space instance has been destroyed.
- Ensure every failure after partial allocation releases previously allocated frames and address-space mappings.
- Compile the kernel after the fix with the repository's documented target.

Acceptance criteria: The driver-space initialization module compiles without undefined fields or unreleased resources.

1.2 Establish a reproducible baseline build

Status: TODO

Area: root Docker scripts, `kernel/`, `targets/`

Document and verify the one canonical build command. The repository contains Docker scripts, target configuration, and build scripts.

Tasks:

- Verify the required Rust toolchain and external binaries.
- Run the Docker build from a clean environment.
- Record the produced kernel, image, and ISO paths.
- Run the image in an emulator and capture serial output.
- Add a short "Build and run" section to the root `README.md`.
- Make the build fail on warnings or errors that affect the boot artifact, where practical.

Acceptance criteria: A new contributor can clone the repository, run the documented commands, produce a reproducible build, and boot the image.

1.3 Define the driver-space ABI versioning rules

Status: TODO

Area: `kernel/src/driverspaceinit/abi/`, `driverspacelib/src/`

The protocol already contains `DsMsg`, ring structures, magic/version fields, and command identifiers. It also defines how to handle unknown commands and malformed messages.

Tasks:

- Document the binary layout, field widths, alignment, endianness, and reserved fields of `DsMsg` and ring structures.
- Define the meaning of every command and every argument.
- Define error/status codes and whether negative values are stable ABI values.
- Define how unknown commands, malformed messages, full rings, and stale IDs are handled.
- Define the compatibility policy for `DS\_MAGIC`, `DS\_VERSION`, and command additions.
- Add compile-time size/alignment assertions on both kernel and driver-space sides.
- Add a protocol version document and a small ABI conformance test.

Acceptance criteria: The same ABI document can be used to implement a driver without reading kernel source code.

1.4 Make driver-space isolation explicit and testable

Status: TODO

Area: `kernel/src/driverspaceinit/`, `kernel/src/mm/protection/`, `kernel/src/mm/process/`

The kernel creates a separate address space and maps shared pages into it. The security and failure-isolation requirements are:

Tasks:

- Test that driver space cannot read arbitrary kernel virtual addresses.
- Test that driver-space mappings are limited to explicitly granted RAM and MMIO regions.
- Validate permissions for shared rings, initialization parameters, RAM grants, and MMIO grants.

- Reject overlapping, unaligned, out-of-range, and double-freed grants.
- Add a driver crash or invalid-command test and verify that kernel execution continues.
- Define whether and how a failed driver can be restarted or detached.

****Acceptance criteria:**** Negative tests demonstrate that unauthorized mappings and malformed service requests are rejected.

2. P1 – kernel and hardware reliability

2.1 Complete the allocator specification and test suite

****Status:**** WIP

****Area:**** `kernel/src/mm/alloc/`

The active heap path uses buddy allocation. Slab code is present but disabled with `HEAP\_USE\_SLAB 0`. The code is not tested.

****Tasks:****

- Document physical-frame, virtual-mapping, heap, buddy, slab, DMA, and contiguous-memory responsibilities.
- Add tests for zero-size allocations, alignment, large allocations, `krealloc`, double free, invalid free, and overlapping.
- Test buddy split and coalescing across every supported order.
- Test page mapping rollback when a physical-frame allocation fails partway through.
- Verify `heap\_usable\_size()` for active and invalid pointers.
- Decide whether slab should remain disabled, be enabled for small objects, or be redesigned.
- Add allocator statistics and leak checks to a repeatable debug test.
- Define which allocator functions are safe during interrupt context and early boot.

****Acceptance criteria:**** The allocator passes deterministic unit/stress tests and its public contract describes its behavior.

2.2 Finish filesystem correctness and integration tests

****Status:**** WIP

****Area:**** `kernel/src/fs/`

The tree contains MBR, disk, ATA PIO, block-driver, FAT32, EXT4, TFS, and filesystem initialization code.

****Tasks:****

- Define the supported FAT32 and EXT4 feature subset.
- Test mounting, directory creation, file creation, reads, writes, truncation, append, rename, delete, and unlink.
- Test unaligned reads/writes and operations spanning multiple blocks.
- Test corrupted superblocks, invalid MBR data, short reads, failed writes, and unexpected device removal.
- Define and test filesystem locking and concurrent access rules.
- Separate experimental TFS behavior from the stable filesystem API.
- Add image-based regression fixtures for each supported filesystem.

****Acceptance criteria:**** Filesystem tests pass on generated disk images and data remains correct after unmounting.

2.3 Stabilize PCI and USB discovery

****Status:**** WIP

****Area:**** `kernel/src/drivers/`, `kernel/src/pci.rs`

USB core, EHCI/XHCI, HID, CDC, and mass-storage structures exist. The next step is reliable enumeration, and then driver development.

****Tasks:****

- Document PCI configuration access and BAR discovery.
- Verify MMIO BAR validation and alignment.
- Complete USB device, configuration, interface, endpoint, and speed handling.
- Add enumeration tests with recorded or emulated descriptor sets.
- Test device removal, reset, stalled endpoints, transfer timeouts, and controller errors.
- Define which USB classes remain in kernel space and which move to driver space.
- Connect mass-storage discovery to the block-device registry and filesystem layer.

****Acceptance criteria:**** A supported USB storage device and a supported HID device can be enumerated repeatedly.

2.4 Build a functional network-driver milestone

****Status:**** WIP

****Area:**** `kernel/src/nic/`

The website identifies network-driver development as an important contribution area. The repository contains a list of tasks.

****Tasks:****

- Define the minimum supported NIC and VirtIO feature set.
- Complete RX/TX queue setup, descriptor ownership, interrupt or polling behavior, and recovery.
- Add packet-buffer ownership and lifetime rules.
- Implement and test a minimal Ethernet/ARP/IPv4 path, or explicitly document the current scope.
- Add checksum, MTU, malformed-packet, and queue-exhaustion tests.
- Add an emulator-based network test with captured packets.
- Define how network drivers are exposed to driver space.

****Acceptance criteria:**** The selected NIC can transmit and receive validated packets in an emulator or test environment.

2.5 Define and test audio driver integration

****Status:**** WIP

****Area:**** `kernel/src/audio/`, `drivers/audiodriver/`, `driverspacelib/src/audio.rs`

Audio code spans kernel services, driver-space helpers, JACK-related bindings, and driver code. The ownership is split between kernel and driver space.

****Tasks:****

- Define which audio operations are kernel services and which belong to the audio driver.
- Document buffer ownership, physical-page grants, sample format, rate, channel count, and duration units.
- Test `AudioInfo`, `AudioPlay`, `AudioStop`, and amplifier controls with invalid buffers and lengths.
- Verify that audio buffers cannot be used after revocation.
- Decide whether JACK integration is a development dependency or part of the target runtime.
- Add a minimal deterministic tone test in an emulator or supported host environment.

****Acceptance criteria:**** Audio requests use a documented ABI, invalid grants are rejected, and the driver can be built against the documented ABI.

2.6 Formalize bootloader and ISO output

****Status:**** TODO

****Area:**** `comgrub/`, `comlimine/`, `iso-builder/`

The project contains GRUB and Limine-related boot components and a separate ISO builder. Their supported output formats are documented in the respective READMEs.

****Tasks:****

- Choose the primary supported boot path for the next milestone.
- Document the difference between GRUB and Limine images.
- Verify linker scripts, kernel load addresses, entry points, memory-map handoff, and initrd placement.
- Add ISO-content validation before boot.
- Add emulator smoke tests for each supported boot path.
- Record serial and graphical output expected at each milestone.

****Acceptance criteria:**** The canonical ISO boots through the documented bootloader and fails with a useful error message if it fails.

3. P1 – code quality and integration

3.1 Define the kernel ABI boundary

****Status:**** TODO

****Area:**** `kernel/src/abi/`, `trangorgelibc/src/abi/`

The kernel ABI and client-side library must agree on types, error values, syscall numbers, pointer rules, and other details.

****Tasks:****

- Inventory every public ABI type and function.
- Define integer widths, structure layout, alignment, ownership, and lifetime rules.
- Define pointer validation and user/kernel buffer-copy rules.
- Synchronize error handling between kernel and `trangorgelibc`.
- Add generated or checked bindings where possible.
- Add ABI compatibility tests.

****Acceptance criteria:**** A client library can be rebuilt against the documented ABI and pass layout and binding tests.

3.2 Define driver lifecycle and registration

****Status:**** TODO

****Area:**** `driverspace/src/`, `driverspacelib/src/driver.rs`, `kernel/src/driverspaceinit/`

The project needs one lifecycle model for discovery, initialization, registration, attachment, running, and shutdown.

Tasks:

- Define driver states and allowed transitions.
- Define device identity, vendor/class fields, capabilities, and ownership.
- Complete driver registration and device attachment commands or remove unused commands from the public API.
- Define resource acquisition and release for IRQs, MMIO, DMA, and shared memory.
- Define shutdown ordering and timeout handling.
- Add a mock driver used in CI to exercise the complete lifecycle.

Acceptance criteria: A mock driver can register, attach to a device, request resources, handle a request, and shutdown.

3.3 Reduce duplicated or experimental paths

Status: TODO

Area: whole repository

Several components appear to contain experimental or partially connected paths. These should be clearly marked as experimental.

Tasks:

- Mark unused driver initialization helpers and experimental commands.
- Remove stale generated output from source-controlled directories where appropriate.
- Document intentionally empty or placeholder modules.
- Run formatting and static checks for Rust, C, Zig, and assembly sources where tools are available.
- Remove IDE metadata from production source directories unless intentionally retained.
- Record known warnings and decide whether each is acceptable or must be fixed.

Acceptance criteria: The repository has a documented list of experimental components, no unexplained experimental components.

4. P2 – architecture portability

4.1 Separate architecture-independent and architecture-specific code

Status: TODO

Area: `kernel/src/mm/arch/x86\_64/`, CPU, paging, boot, targets

The current memory code visibly contains x86_64-specific implementations. Portability requires clear interfaces and architecture-independent code.

Tasks:

- Define architecture-independent interfaces for page tables, TLB operations, interrupts, timers, CPU state, and other hardware-specific components.
- Move x86_64-only constants and operations behind architecture modules.
- Define target-specific linker scripts and boot parameters.
- Add architecture capability tables for page size, address width, cache behavior, and DMA constraints.
- Add compile-only CI checks for ARM64 and RISC-V targets.

Acceptance criteria: Architecture-independent kernel code compiles without importing x86_64 implementation-specific code.

4.2 ARM64 milestone

Status: BLOCKED by architecture abstraction work

- Define the ARM64 target specification.
- Implement or stub the boot entry and exception vector.
- Implement page-table and TLB operations.
- Implement interrupt-controller integration.
- Port the memory bootstrap and serial diagnostics.
- Boot a minimal ARM64 image in an emulator.

Acceptance criteria: A documented ARM64 smoke test reaches kernel initialization and reports memory-sizes.

4.3 RISC-V milestone

Status: BLOCKED by architecture abstraction work

- Define the RISC-V target specification.
- Implement boot entry, trap handling, paging mode, and timer setup.
- Port serial diagnostics and memory initialization.
- Boot a minimal RISC-V image in an emulator.

Acceptance criteria: A documented RISC-V smoke test reaches kernel initialization and reports memory-sizes.

5. P2 – package manager and user ecosystem

5.1 Complete `ctrlinstall`

****Status:**** WIP

****Area:**** `kernel/src/ctrlinstall/`

The repository contains repository fetching, indexes, manifests, dependency resolution, transactions, dif

****Tasks:****

- Define repository and package formats.
- Define signatures, trust roots, hashes, and rollback behavior.
- Complete dependency resolution and conflict diagnostics.
- Test interrupted transactions and power-loss recovery.
- Add install, remove, update, search, and list interfaces.
- Document how driver packages and kernel-compatible packages are separated.

****Acceptance criteria:**** A package can be fetched, verified, installed, listed, upgraded, rolled back, an

5.2 Stabilize `trangorgellibc`

****Status:**** TODO

- Inventory implemented versus placeholder modules.
- Define syscall wrappers and error semantics.
- Add memory, I/O, synchronization, and ABI tests.
- Document which interfaces are safe for applications and which are experimental.

5.3 Advance `triang-lang`

****Status:**** WIP

- Document the language syntax and project format.
- Define the AST and IR stability policy.
- Add parser and semantic-analysis tests.
- Add deterministic code-generation tests for C and assembly.
- Define the x86_64 output ABI.
- Decide which language features are required for system tools.

6. P3 – tools, documentation, and community

6.1 Documentation structure

****Status:**** WIP

- Move the architecture documents into a stable `docs/` directory.
- Add a contributor guide and development-environment guide.
- Add a boot-flow diagram and driver-space sequence diagram.
- Add per-subsystem pages for memory, filesystem, USB, network, ABI, and package management.
- Keep English as the reference language or define an explicit bilingual-document policy.
- Add a changelog policy for ABI and boot-image changes.

6.2 Code editor and developer tools

****Status:**** TODO

- Define the dedicated editor's scope and minimum viable feature set.
- Decide whether the editor is part of the OS image or a host-side development tool.
- Document project files, build integration, syntax support, and debugger/serial integration.
- Connect editor workflows to `triang-lang` and package-management tools.

6.3 Contribution and release process

****Status:**** TODO

- Define branch policy for `main`, `unstable`, and the future stable branch.
- Define release criteria and a versioning scheme.
- Add CI for formatting, compilation, ABI tests, allocator tests, filesystem-image tests, and emulator bo
- Add issue templates for kernel bugs, driver bugs, build failures, and documentation gaps.
- Document the AGPLv3 licensing requirements and contribution expectations.

7. Suggested milestone order

The recommended order is intentionally dependency-driven:

1. Repair and compile-check driver-space initialization.
2. Establish a reproducible x86_64 build, ISO, and emulator boot test.
3. Freeze and test the driver-space ABI at an experimental version.
4. Add isolation and invalid-request tests.
5. Finish allocator, filesystem, USB, and network regre

... (truncated, file is very long)

TrangorgeOS — dokumentacja architektury.markdown

TrangorgeOS – dokumentacja architektury

> **Status dokumentu:** wersja robocza 0.1
> **Zakres:** architektura systemu, podzia jdra, pam i alokator, model sterowników oraz mapa repozy
> **ród:** drzewo projektu przekazane przez autora, kod ga zi `unstable` oraz oficjalna strona proje

1. Cel projektu

TrangorgeOS jest systemem operacyjnym budowanym od podstaw. Projekt ma docelowo obsugiwa kilka archite

Kod systemu jest wielojzyczny, ale naley rozróni stan biecy od planów. Publiczna ga `unstable

Dokumentacja ma dwa cele. Po pierwsze, ma by **map orientacyjn dla autora**, który rozwija wiele pods

2. Stan projektu

Według opisu autora jdro jest ju zaimplementowane, system plików dziaa, cho nadal wymaga poprawek, a

Ponijsza tabela rozdziela informacje potwierdzone przez autora, twierdzenia oficjalnej strony oraz fakty

| Obszar | Potwierdzone | Wniosek ze struktury |
|------------------|---|--|
| Architektura CPU | x86_64, póniej ARM64 i RISC-V | Obecne artefakty wskazuj przede wszystkim na x86_ |
| Typ jdra | Hybrydowe | Wydzielenie `driverspace` i `driverspacelib` wspiera taki model. |
| Jzyki | Rust w kernelu; strona wymienia Ada/SPARK, C i jzyk własny jako planowany ekosystem | GitHub |
| System plików | Dziaa, ale ma problemy | W `kernel/src/fs` znajduj si obsługa dysku, MBR, FAT32, ex |
| Alokator | Około 8 tys. linii według autora | Kod potwierdza stert opart obecnie na buddy allocatorz |
| Sterowniki | Rozwijane, jeszcze nieukończone | Repozytorium zawiera sterowniki GPU, audio, kamery, Wac |

3. Model wysokopoziomowy

System można obecnie opisać jako kilka współpracujących warstw:

```
```text
+-----+
| Programy, narzdzia i biblioteki użytkowe |
| triang-lang, trangorgelibc, mp4_to_bmp, narzdzia ISO |
+-----+
| Driver space | Biblioteki i kontrakty ABI |
| driverspace | driverspacelib, trangorgelibc |
| rejestracja i wykonywanie | syscall, typy, pam, I/O |
| sterowników | |
+-----+
| Kernel space | CPU, MM, FS, NIC, USB, PCI, |
| podstawowe mechanizmy | przerwania, grafika, terminal |
+-----+
| Sprt i firmware | x86_64; w przyszłości ARM64, |
| | RISC-V |
+-----+
```
```

Diagram jest zgodny z publicznym opisem projektu [2], natomiast szczegóły komunikacji s ju czciowo v

4. Trójpodzia sterowników

Oficjalna strona definiuje trójpodzia jako **kernel space**, **driver space** i **user space** [2]. Kod

1. **Kernel space** – kod, który musi dziaa z najwyższymi uprawnieniami i bezpośrednim dostpem do me
2. **Driver space** – wydzielone środowisko dla sterowników, uruchamiane i obsługiwane przez mechanizmy
3. **User space / warstwa kliencka** – programy i biblioteki korzystajce z usług sterowników przez stab

4.1. Przepływ danych sterownika

W aktualnym kodzie komunikacja kernel ↔ driver space odbywa się przez dwa pierścienie: `k2d` (kernel-to-driver)

Przykładowy przepływ powinien wyglądać następująco:

```
```text
Program lub biblioteka
 |
 v
ABI / syscall / publiczne API
 |
 v
driverspacelib: typy, runtime, logowanie, ring buffer, audio, block
 |
 v
Driver space: API, rejestracja, sesja sterownika, komunikacja
 |
 v
Kernel glue: PCI/USB/DMA/przerwania/pamięć
 |
 v
Kontroler i urządzenie sprzętowe
```
```

Najważniejsza zasada architektoniczna brzmi: **sterownik nie powinien powielać logiki jądra, a jądro nie powinno powielać sterownika

4.2. Elementy modelu widoczne w repozytorium

| Element | Lokalizacja | Rola |
|----------------------------------|------------------------------|---|
| API driver space | `driverspace/src/api.rs` | Publiczne punkty wejścia i wyjścia |
| Biblioteka driver space | `driverspacelib/src` | ABI, runtime, logowanie, pamięć, ring buffer, audio, block |
| Inicjalizacja po stronie kernela | `kernel/src/driverspaceinit` | Wejście, komunikacja, komendy inicjalizacji |
| Pakiety sterowników | `drivers/*` | Konkretne sterowniki GPU, audio, kamery i urządzenia wskazujących |
| Interfejsy klienta | `trangorgelibc/src` | Typy ABI, błądy, tabela kernela, syscall oraz warstwy I/O |

5. Architektura jądra

Główny kod jądra znajduje się w `kernel/src`. Z punktu widzenia odpowiedzialności można go podzielić na podsystemy.

| Podsystem | Katalogi / pliki | Odpowiedzialności |
|-----------------------------|---|---|
| Punkt wejścia i diagnostyka | `main.rs`, `serial.rs`, `vga_buffer.rs`, `testing.rs` | Start systemu, wyjście |
| ABI | `abi/` | Publiczne kontrakty alokacji, pamięci, systemu plików i API |
| CPU i przerwania | `cpu/`, `gdt.rs`, `interrupts.rs` | ACPI, LAPIC, GDT, trampoliny, wejścia przerwań |
| Pamięć | `mm/`, `allocator/` | Pamięć fizyczna i wirtualna, mapowanie, stronicowanie, ochrona, cache |
| Sterowniki sprzętowe | `drivers/`, `pci.rs` | PCI, USB, urządzenia HID, mass storage, host controllers |
| System plików | `fs/` | MBR, dyski, ATA PIO, warstwa blokowa, FAT32, ext4, TFS i inicjalizacja FS |
| Grafika i konsola | `gfx/`, `terminal/`, `vga_buffer.rs` | Framebuffer, VGA, font, konsola, terminal oraz |
| Sieć | `nic/` | Ethernet, pakiety, protokoły, urządzenia i VirtIO network queues |
| Instalacja i aktualizacje | `ctrlinstall/` | Repozytorium, indeksy, manifesty, rozwiązywanie zależności |
| Driver space | `driverspaceinit/` | Uruchamianie, komunikacja i usługi dla wydzielonej przestrzeni sterownika |
| Audio | `audio/` | Integracja audio po stronie kernela, w szczególności warstwa jack |

5.1. Zasady zależności

Zależności powinny płynąć od warstw wyższych do niższych przez jawne interfejsy, a nie przez przypadkowe

Warto traktować katalogi `abi`, `driverspaceinit`, `driverspacelib` i `trangorgelibc` jako **granice architektury

6. Alokator i podsystem pamięci

Kod `kernel/src/mm` implementuje wielowarstwowy podsystem pamięci, a nie pojedynczą funkcję `alloc`. Roz

```
```text
Warstwy pamięci TrangorgeOS

API Rust / C i mostek FFI
 |
 v
mm/api.rs, mm/ffi.rs, mm/mm_bridge.rs
 |
 v
```
```



```

+--> pami██ fizyczna: pmm, bitmap, frame
|
+--> pami██ wirtualna: vmm, mapping, page, paging, tlb
|
+--> sterta: heap, buddy, slab
|
+--> pami██ specjalna: contiguous, dma
|
+--> cache i lokalno██ CPU: cache, object_cache, per_cpu
|
+--> przestrzenie procesów i ochrona: address_space, mmap,
guard, isolation, permissions
...

```

6.1. Pami██ fizyczna

Warstwa ``mm/alloc/physical`` zawiera bitmapę, ramki i PMM. Jej zadaniem jest śledzenie dostępnych jednostek.

6.2. Pami██ wirtualna i stronicowanie

Warstwy ``virtual``, ``paging``, ``arch/x86_64`` oraz ``process`` wskazują na rozdzielenie mapowania adresów, tabeli.

6.3. Sterta, buddy i slab

W bibliotece `ga██zi`` aktywnie ścieżką sterty jest `**buddy allocator**`. ``heap_init()`` zawsze inicjuje.

Kod ``buddy.c`` potwierdza przydział bloków według rzędu, listy wolnych bloków, mapowanie stron oraz koalescencję.

Praktyczny podział odpowiedzialności powinien być następujący:

| Komponent | Zadanie |
|------------------------|--|
| PMM / bitmap / frame | Przydział ramek fizycznych. |
| Buddy | Zarządzanie większymi, potęgami obszarów pamięci i ewentualna koalescencja bloków. |
| Heap | Wystawienie ogólnego interfejsu alokacji sterty. |
| Slab | Wydajny przydział obiektów o określonym rozmiarze. |
| DMA / contiguous | Przydział obszarów spełniających wymagania sprzętu. |
| Cache / per-CPU | Ograniczenie kosztu blokad i poprawa lokalności. |
| Protection / isolation | Kontrola praw dostępu oraz izolacja przestrzeni. |

6.4. Kontrakt, który warto dopisać do kodu

Dokumentacja alokatora powinna w przyszłości odpowiedzieć na sześć pytań: kto jest właścicielem przydziału?

7. Sterowniki sprzętowe

Katalog ``drivers`` zawiera osobne pakiety dla różnych klas urządzeń. Aktualna mapa wygląda następująco.

| Pakiet | Przeznaczenie |
|------------------------------------|---|
| <code>`amdgpu_driver`</code> | Sterownik GPU AMD. |
| <code>`intelgpu_driver`</code> | Sterownik GPU Intel. |
| <code>`netcam_driver`</code> | Sterownik kamery sieciowej lub urządzenia tego typu. |
| <code>`wacomgraphic_driver`</code> | Sterownik urządzenia graficznego Wacom. |
| <code>`audiodriver`</code> | Sterownik audio; zawiera Rust, integrację jack oraz moduły Odin dla mikrofonu i głośnika. |

W jądrze znajdują się ponadto ogólne elementy obsługi USB i PCI. USB jest podzielone na warstwy core, host.

Warstwa sieciowa ``kernel/src/nic`` zawiera abstrakcje urządzenia i sterownika, Ethernet, pakiety, protokoły.

8. Boot, budowanie i narzędzia

Projekt zawiera dwa widoczne warianty elementu startowego: ``comgrub`` oraz ``comlimine``. Pierwszy zawiera

``iso-builder`` jest osobnym narzędziem do tworzenia obrazu ISO. Składa się z modułów Oberonowych (``Mod``).

``targets/x86_64-unknown-none`` oraz ``kernel/x86_64-kernel.json`` wskazują na obecny profil bare-metal dla x86_64.

``triang-lang`` jest osobnym kompilatorem lub językiem projektowym. Zawiera lexer, parser, AST, analizę semantyki.

9. Mapa repozytorium

| Katalog | Znaczenie |
|---------|-----------|
|---------|-----------|

```

|---|---|
| `kernel/` | Główne jądro systemu, jego ABI, pamięć, FS, CPU, sieć i sterowniki. |
| `drivers/` | Osobne pakiety konkretnych sterowników urządzeń. |
| `driverspace/` | Program lub środowisko driver space oraz jego API. |
| `driverspacelib/` | Biblioteka wspólna dla sterowników działających w driver space. |
| `trangorgelib/` | Biblioteka klienta i warstwa ABI/syscall. |
| `comgrub/` | Wariant komponentu startowego z GRUB. |
| `comlimine/` | Wariant komponentu startowego z Limine. |
| `iso-builder/` | Narzędzie tworzące obraz ISO. |
| `triang-lang/` | Język/kompilator z generatorami C i ASM. |
| `mp4_to_bmp/` | Narzędzie pomocnicze do konwersji MP4 do BMP. |
| `targets/` | Konfiguracje targetów kompilacji. |
| `Dockerfile` i skrypty Docker | Powtarzalne środowisko budowania i uruchamiania. |

```

10. Zasady utrzymania dokumentacji

Każdy nowy podsystem powinien mieć krótki opis odpowiedzialności, publiczne wejście, zależności oraz in

Warto prowadzić dokumentację w układzie odpowiadającym repozytorium, ale nie kopiować bezpośrednio całej

Dla sterownika przydatny jest stały szablon:

```

Pole	Treść do uzupełnienia
Nazwa	Nazwa urządzenia i pakietu.
Przestrzeń wykonania	Kernel space, driver space albo user space.
Magistrała	PCI, USB, VirtIO lub inna.
Zależności	Pamięć, DMA, przerwania, kolejki, ABI.
Punkt inicjalizacji	Funkcja lub komenda uruchamiająca sterownik.
API	Operacje udostępniane klientom.
Ograniczenia	Brakujące funkcje, problemy i zagrożenia sprzętowe.
Status	Projektowany, eksperymentalny, działający lub stabilny.

```

11. Otwarte pytania techniczne

Poniższe kwestie powinny zostać doprecyzowane w następnej wersji dokumentacji, ponieważ nie da się ich

1. Jakie są docelowe granice uprawnień oraz które funkcje z protokołu driver space są już stabilne?
2. Kod potwierdza osobną przestrzeń adresową, ale trzeba doprecyzować pełny cykl życia procesu/obrazu d
3. Jaki jest formalny format komunikatów między kernelem a driver space?
4. Jak wygląda cykl życia sterownika: discovery, init, start, stop, reset, unload i obsługa awarii?
5. Czy buddy, slab i heap tworzą jeden ciąguch alokacji, czy są niezależnymi allocatorami dla różnych t
6. Czy `DsMsg` i komendy z `kernel/src/driverspaceinit/abi/abi.rs` mają już gwarantowaną kompatybilność
7. Jakie są minimalne wymagania dla implementacji ARM64 i RISC-V?
8. Które sterowniki są obecnie działające na prawdziwym sprzęcie, a które są dopiero szkieletem?

12. Podsumowanie

TrangorgeOS ma już wyraźny podział na jądro, przestrzeń sterowników, biblioteki, pakiety urządzeń, narz

Największą wartością kolejnej iteracji dokumentacji będzie opisanie formalnych kontraktów: ABI, komunik

Referencje

[1]: <https://github.com/CTRL-F-org3/TrangorgeOS/tree/unstable> "TrangorgeOS – gałęź unstable na GitHubie"

[2]: <https://trangorgeos.website/> "Oficjalna strona TrangorgeOS"

[3]: <https://github.com/CTRL-F-org3/Trango>

... (truncated, file is very long)

doc.md

```
# TrangorgeOS Architecture Documentation
```

```
**Generated:** 2026-09-18 21:00:50
```

```
**Files Scanned:** `1709` | **API Items Extracted:** `4008`
```

```
**Languages:** Rust, C/C++, Ada SPARK, Odin, Nim, Assembly
```

```
---
```

```
## Table of Contents
```

```

- [c.rs](#c.rs)
- [comgrub](#comgrub)
- [comgrub/src](#comgrub-src)
- [comlimine](#comlimine)
- [comlimine/src](#comlimine-src)
- [drivers/amdgpu_driver](#drivers-amdgpu-driver)
- [drivers/audioplayer](#drivers-audioplayer)
- [drivers/intelgpu_driver](#drivers-intelgpu-driver)
- [drivers/netcam_driver](#drivers-netcam-driver)
- [drivers/vgpu](#drivers-vgpu)
- [drivers/wacomgraphic_driver](#drivers-wacomgraphic-driver)
- [driverspace/src](#driverspace-src)
- [driverspace_workspace/crates](#driverspace-workspace-crates)
- [driverspace_workspace/drivers](#driverspace-workspace-drivers)
- [driverspace_workspace/formal](#driverspace-workspace-formal)
- [driverspace_workspace/lib](#driverspace-workspace-lib)
- [driverspacelib/src](#driverspacelib-src)
- [kernel_Workspace/C_base](#kernel-workspace-c-base)
- [kernel_Workspace/Odin_base](#kernel-workspace-odin-base)
- [kernel_Workspace/base](#kernel-workspace-base)
- [kernel_Workspace/core](#kernel-workspace-core)
- [kernel_Workspace/kernel](#kernel-workspace-kernel)
- [kernel_Workspace/kernel-bin](#kernel-workspace-kernel-bin)
- [kstd/include](#kstd-include)
- [kstd/src](#kstd-src)
- [libs](#libs)
- [mp4_to_bmp/src](#mp4-to-bmp-src)
- [trangorgelibc/src](#trangorgelibc-src)
- [triang-lang/src](#triang-lang-src)
- [userspace-legacy/demo](#userspace-legacy-demo)
- [userspace-legacy/init](#userspace-legacy-init)
- [userspace-legacy/shell](#userspace-legacy-shell)
- [userspace-legacy/terminal](#userspace-legacy-terminal)

---

## c.rs

<details>
<summary><b>c.rs</b> (2 items)</summary>

#### `STRUCT`: **BlockHeader** <sub>line 57</sub>
```rust
struct BlockHeader* next; /* Wskaźnik do następnego bloku */
```

#### `STRUCT`: **BlockHeader** <sub>line 58</sub>
```rust
struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
```

</details>

## comgrub

<details>
<summary><b>comgrub/build.rs</b> (1 items)</summary>

#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```

</details>

## comgrub/src

<details>
<summary><b>comgrub/src/main.rs</b> (2 items)</summary>

#### `FN`: **kernel_main** <sub>line 11</sub>
```rust
fn kernel_main(magic: u32, info: *const u8) -> !;

```

```

```
#### `FN`: **panic** <sub>line 15</sub>
```rust
fn panic(_info: &PanicInfo) -> ! {
```
```

</details>

comlimine

```
<details>
<summary><b>comlimine/build.rs</b> (1 items)</summary>
```

```
#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```
```

</details>

comlimine/src

```
<details>
<summary><b>comlimine/src/main.c</b> (2 items)</summary>
```

```
#### `FUNCTION`: **kernel_main** <sub>line 6</sub>
```c
extern void kernel_main(uint64_t magic, void *info);
```
```

```
#### `FUNCTION`: **_start** <sub>line 28</sub>
```c
void _start(void) {
```
```

</details>

```
<details>
<summary><b>comlimine/src/main.rs</b> (1 items)</summary>
```

```
#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {}
```
```

</details>

drivers/amdgpu_driver

```
<details>
<summary><b>drivers/amdgpu_driver/src/main.rs</b> (1 items)</summary>
```

```
#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
```
```

</details>

drivers/audiodriver

```
<details>
<summary><b>drivers/audiodriver/build.rs</b> (1 items)</summary>
```

```
#### `FN`: **main** <sub>line 12</sub>
```rust
fn main() {
```
```

</details>

<details>

<summary>drivers/audiodriver/src/jacklib.rs (6 items)</summary>

```
#### `STRUCT`: **JackMgr** <sub>line 4</sub>
```rust
pub struct JackMgr {
 `
```

```
`IMPL`: **JackMgr** _{line 9}
```rust
impl JackMgr {
    `
```

```
#### `FN`: **tick** <sub>line 14</sub>
```rust
pub fn tick(&mut self) {
 `
```

```
`FN`: **set_amp** _{line 30}
```rust
pub fn set_amp(&mut self, on: bool) {
    `
```

```
#### `FN`: **present** <sub>line 35</sub>
```rust
pub fn present(&self) -> bool {
 `
```

```
`FN`: **amp_enabled** _{line 40}
```rust
pub fn amp_enabled(&self) -> bool {
    `
> Czy wzmacniacz (amp) jest aktualnie włączony.
```

</details>

<details>

<summary>drivers/audiodriver/src/lib.rs (14 items)</summary>

```
#### `FN`: **ad_init** <sub>line 7</sub>
```rust
fn ad_init(nam_va: u64, bm_va: u64) -> i32;
`
```

```
`FN`: **ad_play** _{line 8}
```rust
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
`
```

```
#### `FN`: **ad_capture** <sub>line 9</sub>
```rust
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
`
```

```
`FN`: **ad_stop** _{line 10}
```rust
fn ad_stop() -> i32;
`
```

```
#### `FN`: **ad_jack_present** <sub>line 11</sub>
```rust
fn ad_jack_present() -> i32;
`
```

```
`FN`: **ad_set_amp** _{line 12}
```rust
fn ad_set_amp(on: i32) -> i32;
`
```

```
#### `FN`: **ad_position** <sub>line 13</sub>
```rust
fn ad_position() -> u32;
`
```

```

`FN`: **init** _{line 16}
```rust
pub fn init(nam_va: u64, bm_va: u64) -> bool {
```

`FN`: **play** _{line 20}
```rust
pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
```

`FN`: **capture** _{line 24}
```rust
pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
```

`FN`: **stop** _{line 28}
```rust
pub fn stop() {
```

`FN`: **jack_present** _{line 32}
```rust
pub fn jack_present() -> bool {
```

`FN`: **set_amp** _{line 36}
```rust
pub fn set_amp(on: bool) {
```

`FN`: **position** _{line 40}
```rust
pub fn position() -> u32 {
```

```

</details>

<details>

<summary><b>drivers/audiodriver/src/main.rs</b> (1 items)</summary>

```

`FN`: **main** _{line 8}
```rust
fn main() {
```


> Punkt wejścia binarki pomocniczej.

>

> Właściwy sterownik działa w driver space przez `ds_entry` (patrz

> `driverspace/src/main.rs`); ta binarka jest cienkim wrapperem CLI, który

> pozwala sprawdzić stan gniazda jack z poziomu hosta.


```

</details>

<details>

<summary><b>drivers/audiodriver/src/odin/driver.odin</b> (8 items)</summary>

```

`STRUCT`: **Bdl_Entry** _{line 8}
```odin
Bdl_Entry :: struct {
```

`PROC`: **nam16** _{line 16}
```odin
nam16 :: proc(off: u32) -> ^u16 {
```


> mixer (NAM) – 16-bit

`PROC`: **bm8** _{line 21}
```odin
bm8 :: proc(off: u32) -> ^u8 {
```


> bus master – 8/16/32

`PROC`: **bm16** _{line 25}

```

```

```odin
bm16 :: proc(off: u32) -> ^u16 {
```

`PROC`: **bm32** _{line 29}
```odin
bm32 :: proc(off: u32) -> ^u32 {
```

`PROC`: **ad_init** _{line 50}
```odin
ad_init :: proc "C" (nam_va: u64, bm_va: u64) -> i32 {
```

`PROC`: **ad_stop** _{line 73}
```odin
ad_stop :: proc "C" () -> i32 {
```

`PROC`: **ad_position** _{line 80}
```odin
ad_position :: proc "C" () -> u32 {
```

</details>

<details>
<summary>drivers/audiodriver/src/odin/jack.odin (2 items)</summary>

`PROC`: **ad_jack_present** _{line 8}
```odin
ad_jack_present :: proc "C" () -> i32 {
```

`PROC`: **ad_set_amp** _{line 21}
```odin
ad_set_amp :: proc "C" (on: i32) -> i32 {
```

</details>

<details>
<summary>drivers/audiodriver/src/odin/microphone.odin (1 items)</summary>

`PROC`: **ad_capture** _{line 6}
```odin
ad_capture :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
```

</details>

<details>
<summary>drivers/audiodriver/src/odin/speaker.odin (1 items)</summary>

`PROC`: **ad_play** _{line 6}
```odin
ad_play :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
```

</details>

<details>
<summary>drivers/audiodriver/src/tone.rs (1 items)</summary>

`FN`: **fill_square** _{line 1}
```rust
pub fn fill_square(buf: &mut [u8], periods: u32) {
```

</details>

drivers/intelgpu_driver

```

<details>  
<summary><b>drivers/intelgpu\_driver/src/main.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 1}
```rust
fn main() {
    `
```

</details>

drivers/netcam_driver

<details>
<summary>drivers/netcam_driver/src/main.rs (1 items)</summary>

```
#### `FN`: **main** <sub>line 1</sub>
```rust
fn main() {
 `
```

</details>

## drivers/vgpu

<details>  
<summary><b>drivers/vgpu/vgpu.h</b> (7 items)</summary>

```
`FUNCTION`: **vgpu_init** _{line 22}
```c
bool vgpu_init(uint32_t w, uint32_t h);
`
```

```
#### `FUNCTION`: **vgpu_get_info** <sub>line 23</sub>
```c
vgpu_info_t vgpu_get_info(void);
`
```

```
`FUNCTION`: **vgpu_clear** _{line 25}
```c
void vgpu_clear(uint32_t color);
`
```

```
#### `FUNCTION`: **vgpu_pixel** <sub>line 26</sub>
```c
void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);
`
```

```
`FUNCTION`: **vgpu_surface_create** _{line 28}
```c
int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
`
```

```
#### `FUNCTION`: **vgpu_surface_present** <sub>line 29</sub>
```c
bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);
`
```

```
`FUNCTION`: **vgpu_process_ring** _{line 31}
```c
void vgpu_process_ring(volatile void *ring);
`
```

</details>

<details>
<summary>drivers/vgpu/vgpu_driver.c (11 items)</summary>

```
#### `FUNCTION`: **pci_void** <sub>line 12</sub>
```c
static void pci_void(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
`
```

```
`FUNCTION`: **pci_val** _{line 20}
```



```

```c
static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
```

`FUNCTION`: **vbe_write** _{line 33}
```c
static void vbe_write(uint16_t idx, uint16_t val)
```

`FUNCTION`: **vgpu_init** _{line 40}
```c
bool vgpu_init(uint32_t w, uint32_t h)
```

`FUNCTION`: **vgpu_get_info** _{line 98}
```c
vgpu_info_t vgpu_get_info(void)
```

`FUNCTION`: **vgpu_pixel** _{line 104}
```c
void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color)
```

`FUNCTION`: **vgpu_clear** _{line 113}
```c
void vgpu_clear(uint32_t color)
```

`FUNCTION`: **vgpu_surface_create** _{line 128}
```c
int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out)
```

`FUNCTION`: **vgpu_surface_present** _{line 168}
```c
bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y)
```

`FUNCTION`: **vgpu_process_ring** _{line 190}
```c
void vgpu_process_ring(volatile vgpu_slot_t *ring)
```

`FUNCTION`: **ds_entry** _{line 244}
```c
void ds_entry(uint64_t params_va)
```

</details>

drivers/wacomgraphic_driver

<details>
<summary>drivers/wacomgraphic_driver/src/main.rs (1 items)</summary>

`FN`: **main** _{line 1}
```rust
fn main() {
```

</details>

driverspace/src

<details>
<summary>driverspace/src/drivers/storage.rs (4 items)</summary>

`STRUCT`: **StorageDrv** _{line 9}
```rust
pub struct StorageDrv {
```

> Storage driver stub. It only satisfies the driver registry so that

```

> driver space can boot; a real disk driver can slot in later.

```
`IMPL`: **StorageDrv** _{line 13}
```rust
impl StorageDrv {
```
```

```
`IMPL`: **Driver** _{line 19}
```rust
impl Driver for StorageDrv {
```
```

```
`FN`: **init** _{line 20}
```rust
fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
```
```

</details>

<details>

<summary><b>driverspace/src/main.rs</b> (1 items)</summary>

```
`FN`: **panic** _{line 32}
```rust
fn panic(_info: &core::panic::PanicInfo) -> ! {
```
```

</details>

## driverspace\_workspace/crates

<details>

<summary><b>driverspace\_workspace/crates/ds-manager/src/main.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

## driverspace\_workspace/drivers

<details>

<summary><b>driverspace\_workspace/drivers/amdgpu-driver/src/main.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

<details>

<summary><b>driverspace\_workspace/drivers/audiodriver/src/main.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

<details>

<summary><b>driverspace\_workspace/drivers/intelgpu-driver/src/main.rs</b> (1 items)</summary>

```
`FN`: **main** _{line 1}
```rust
fn main() {
```
```

</details>

```

<details>
<summary>driverspace_workspace/drivers/netcam_driver/src/main.rs (1 items)</summary>

`FN`: **main** _{line 1}
```rust
fn main() {
```

</details>

<details>
<summary>driverspace_workspace/drivers/vgpu/src/main.rs (1 items)</summary>

`FN`: **main** _{line 1}
```rust
fn main() {
```

</details>

<details>
<summary>driverspace_workspace/drivers/wacomgraphic_driver/src/main.rs (1 items)</summary>

`FN`: **main** _{line 1}
```rust
fn main() {
```

</details>

driverspace_workspace/formal

<details>
<summary>driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (7 items)</summary>

`PACKAGE`: **body** _{line 2}
```ada
package body DS_Buddy_Math with
```
> formal/ds-spark-core/src/ds_buddy_math.adb

`FUNCTION`: **Power_Of_Two** _{line 6}
```ada
function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is
```

`FUNCTION`: **Is_Aligned** _{line 11}
```ada
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
```

`FUNCTION`: **Get_Buddy_Address** _{line 18}
```ada
function Get_Buddy_Address (
```

`FUNCTION`: **Get_Left_Child_Address** _{line 31}
```ada
function Get_Left_Child_Address (
```

`FUNCTION`: **Get_Right_Child_Address** _{line 40}
```ada
function Get_Right_Child_Address (
```

`FUNCTION`: **Safe_Calculate_DMA_End** _{line 50}
```ada
function Safe_Calculate_DMA_End (
```

</details>

```

```

<details>
<summary>driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (9 items)</summary>

`PACKAGE`: **DS_Buddy_Math** _{line 4}
```ada
package DS_Buddy_Math with
```

`TYPE`: **Phys_Addr** _{line 9}
```ada
type Phys_Addr is new Unsigned_64;
```
> Typy bazowe dla fizycznej pamieci (bare-metal)

`TYPE`: **Block_Size** _{line 10}
```ada
type Block_Size is new Unsigned_64;
```

`FUNCTION`: **Power_Of_Two** _{line 17}
```ada
function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
```
> Funkcja pomocnicza: Obliczanie 2^N (Shift left)

`FUNCTION`: **Is_Aligned** _{line 23}
```ada
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
```
> Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku

`FUNCTION`: **Get_Buddy_Address** _{line 32}
```ada
function Get_Buddy_Address (
```

`FUNCTION`: **Get_Left_Child_Address** _{line 47}
```ada
function Get_Left_Child_Address (
```

`FUNCTION`: **Get_Right_Child_Address** _{line 55}
```ada
function Get_Right_Child_Address (
```

`FUNCTION`: **Safe_Calculate_DMA_End** _{line 69}
```ada
function Safe_Calculate_DMA_End (
```

</details>

driverspace_workspace/lib

<details>
<summary>driverspace_workspace/lib/ds-formal-ffi/build.rs (1 items)</summary>

`FN`: **main** _{line 10}
```rust
fn main() {
```

</details>

<details>
<summary>driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (6 items)</summary>

`FN`: **ds_spark_safe_add_32** _{line 7}
```rust
pub fn ds_spark_safe_add_32(a: u32, b: u32) -> u32;
```

```

```
`FN`: **ds_spark_safe_array_index** _{line 10}
```rust
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
```
```

```
`FN`: **safe_add** _{line 21}
```rust
pub fn safe_add(a: u32, b: u32) -> u32 {
```
```

```
`FN`: **ds_spark_buddy_get_buddy** _{line 28}
```rust
pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
```
```

```
`FN`: **ds_spark_safe_dma_end** _{line 29}
```rust
pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
```
```

```
`FN`: **get_buddy** _{line 35}
```rust
pub fn get_buddy(addr: u64, exp: u64) -> u64 {
```
```

</details>

<details>  
<summary><b>driverspace\_workspace/lib/ds-fw-audio/src/lib.rs</b> (2 items)</summary>

```
`FN`: **add** _{line 1}
```rust
pub fn add(left: u64, right: u64) -> u64 {
```
```

```
`FN`: **it_works** _{line 10}
```rust
fn it_works() {
```
```

</details>

<details>  
<summary><b>driverspace\_workspace/lib/ds-fw-block/src/request.rs</b> (8 items)</summary>

```
`ENUM`: **RequestState** _{line 7}
```rust
pub enum RequestState {
```
```

```
`STRUCT`: **IoRequest** _{line 15}
```rust
pub struct IoRequest {
```
```

> High-level I/O request with metadata.

```
`IMPL`: **IoRequest** _{line 27}
```rust
impl IoRequest {
```
```

```
`FN`: **new** _{line 28}
```rust
pub fn new(id: u64, lba: u64, block_count: u32, buffer: *mut u8, len: usize, is_write: bool) -> Self {
```
```

```
`FN`: **state** _{line 42}
```rust
pub fn state(&self) -> RequestState {
```
```

```
`FN`: **mark_in_progress** _{line 52}
```rust
```

```

pub fn mark_in_progress(&self) {
    ...

    #### `FN`: **mark_completed** <sub>line 56</sub>
    ```rust
 pub fn mark_completed(&self) {
 ...

 #### `FN`: **mark_failed** _{line 63}
    ```rust
    pub fn mark_failed(&self) {
        ...

</details>

<details>
<summary><b>driverspace_workspace/lib/ds-fw-block/src/traits.rs</b> (11 items)</summary>

    #### `STRUCT`: **BlockRequest** <sub>line 6</sub>
    ```rust
 pub struct BlockRequest {
 ...
 }
 > Represents a single I/O request to a block device.

 #### `TRAIT`: **BlockDevice** _{line 16}
    ```rust
    pub trait BlockDevice {
        ...
    }
    > Core trait for all block devices.
    > Drivers must implement this to integrate with the block subsystem.

    #### `FN`: **read_blocks** <sub>line 18</sub>
    ```rust
 fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;
 ...
 > Read blocks from the device starting at LBA.

 #### `FN`: **write_blocks** _{line 21}
    ```rust
    fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;
    ...
    > Write blocks to the device starting at LBA.

    #### `FN`: **flush** <sub>line 24</sub>
    ```rust
 fn flush(&mut self) -> Result<(), DsError>;
 ...
 > Flush internal caches to physical media.

 #### `FN`: **geometry** _{line 27}
    ```rust
    fn geometry(&self) -> BlockGeometry;
    ...
    > Get device geometry (block size, total blocks).

    #### `FN`: **supports_trim** <sub>line 30</sub>
    ```rust
 fn supports_trim(&self) -> bool { false }
 ...
 > Check if device supports TRIM/discard.

 #### `FN`: **submit_request** _{line 33}
    ```rust
    fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
        ...
    }
    > Optional: Submit async request (for advanced schedulers).

    #### `STRUCT`: **BlockGeometry** <sub>line 40</sub>
    ```rust
 pub struct BlockGeometry {
 ...
 }

 #### `IMPL`: **BlockGeometry** _{line 46}

```

```

```rust
impl BlockGeometry {
```

`FN`: **total_bytes** _{line 47}
```rust
pub fn total_bytes(&self) -> u64 {
```

</details>

<details>
<summary>driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (2 items)</summary>

`FN`: **add** _{line 1}
```rust
pub fn add(left: u64, right: u64) -> u64 {
```

`FN`: **it_works** _{line 10}
```rust
fn it_works() {
```

</details>

<details>
<summary>driverspace_workspace/lib/ds-fw-input/src/lib.rs (2 items)</summary>

`FN`: **add** _{line 1}
```rust
pub fn add(left: u64, right: u64) -> u64 {
```

`FN`: **it_works** _{line 10}
```rust
fn it_works() {
```

</details>

<details>
<summary>driverspace_workspace/lib/ds-ipc/src/client.rs (4 items)</summary>

`STRUCT`: **ManagerClient** _{line 7}
```rust
pub struct ManagerClient {
```

`IMPL`: **ManagerClient** _{line 11}
```rust
impl ManagerClient {
```

`FN`: **request_mmio** _{line 16}
```rust
pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
```

`FN`: **bind_irq** _{line 41}
```rust
pub fn bin

... (truncated, file is very long)

```

driverspace_workspace/TODO.md

TODO.md: Driver Space Migration and Implementation Guide

Overview

This document outlines the pending tasks, architectural constraints, and execution steps for migrating and

The legacy monolithic implementations (`driverspacelib` and the old `driverspace` daemon) are deprecated.

1. Prerequisites

Before beginning driver implementation, the foundational libraries located in `lib/` must be fully implemented.

- [] **lib/kapi-abi**: Finalize all IPC message structures, opcodes, and error codes. Ensure `cbindgen` is configured correctly.
- [] **lib/kapi-syscall**: Implement architecture-specific SVC/syscall wrappers.
- [] **lib/ds-ipc**: Stabilize the channel, port, and shared memory abstractions.
- [] **lib/ds-mem**: Finalize the `no\_std` allocators (Buddy/Slab) and DMA mapping utilities.
- [] **lib/ds-fw**: Define the core traits for device classes (`BlockDevice`, `AudioDevice`, `DisplayDevice`).

2. Driver Implementation Checklist

The following drivers require migration from the legacy codebase or complete reimplementation using the new framework.

2.1. Graphics and Display

- [] **amdgpu_driver**
 - [] Implement MMIO register read/write wrappers.
 - [] Implement ring buffer management for command submission.
 - [] Implement Display PHY initialization.
 - [] Implement the `DisplayDevice` and `GpuDevice` traits from `ds-fw-gpu`.
- [] **intelgpu_driver**
 - [] Implement Graphics Technology (GT) and Display Engine (DE) initialization.
 - [] Implement GuC (Graphics Microcontroller) firmware loading.
 - [] Implement the `DisplayDevice` trait from `ds-fw-gpu`.
- [] **vgpu** (Virtual GPU)
 - [] Set up the Rust-to-C FFI bridge (`build.rs` and `ffi.rs`).
 - [] Implement virtual framebuffer memory management.
 - [] Ensure the C implementation strictly uses the `kapi-abi` headers.

2.2. Audio

- [] **audiodriver**
 - [] Implement DMA stream management for audio buffers.
 - [] Implement codec communication protocols (I2C / Intel HDA).
 - [] Integrate the Odin/C DSP components via the defined FFI boundaries.
 - [] Implement the `AudioDevice` trait from `ds-fw-audio`.

2.3. Input and HID

- [] **wacomgraphic_driver**
 - [] Implement Wacom-specific HID report parsing.
 - [] Implement raw-to-logical data conversion for pen pressure and tilt (X/Y).
 - [] Implement the `InputDevice` trait from `ds-fw-input`.

2.4. Networking and Multimedia

- [] **netcam_driver**
 - [] Implement USB Video Class (UVC) protocol parsing.
 - [] Implement isochronous USB transfer handling.
 - [] Implement device control interfaces (brightness, contrast, zoom).

3. Strict Architectural Rules for Contributors

All contributors must adhere to the following rules when writing driver code. Violations will result in immediate removal from the project.

- Unidirectional Dependencies**:
 - Drivers (`drivers/`) may **only** depend on libraries in `lib/`.
 - Drivers must **never** depend on the management daemons in `crates/`. Communication with the manager must be handled via the FFI.
- No Standard Library**:
 - All Rust driver code must be `#![no_std]`. The use of the Rust standard library is strictly prohibited.
- Zero Code Duplication**:
 - If a utility, protocol parser, or hardware abstraction is needed by more than one driver, it must be implemented in a shared library in `lib/`.
- Strict FFI Boundaries**:
 - If a driver requires C or Odin code, it must be isolated in a dedicated subdirectory (e.g., `c/` or `odin/`).
 - Compilation of foreign code must be handled via `build.rs` using the `cc` crate or equivalent tooling.
 - Rust wrappers for foreign functions must be isolated in an `ffi.rs` file.
- Capability and Resource Management**:
 - Drivers must not attempt to access hardware resources (MMIO, IRQs, DMA) directly via hardcoded addresses.

4. Execution Steps for a New Driver

When starting work on a driver, follow this exact sequence:

- Initialize the Crate**: Create the directory under `drivers/<driver\_name>/` and generate a new `Cargo.toml`.
- Update Workspace**: Add the new crate path to the `members` array in the root `driverspace\_workspace.toml`.
- Define Dependencies**: Add only the necessary `lib/` crates to the driver's `Cargo.toml` (e.g., `kapi`, `ds-fw-gpu`).
- Implement Entry Point**: Create `main.rs` (or `lib.rs`), initialize the driver runtime, and establish the driver's FFI interface.
- Implement the Trait**: Implement the required trait from the corresponding `ds-fw-\*` framework.
- Hardware Logic**: Implement the actual hardware register manipulation and state machines.

5. Current Status

- [] Workspace structure created and validated.
- [] Legacy `driverspacelib` code analyzed for extraction.
- [] `lib/kapi-abi` definitions finalized.
- [] First driver (`amdgpu\_driver` or `audiodriver`) migrated to the new workspace.

*For questions regarding the architecture or the new Kernel API, refer to the main Architecture Documenta

driverspace_workspace/Tree_readme.md

```
driverspace_workspace/
├── Cargo.toml
├── rust-toolchain.toml
├── lib/
│   ├── kapi-abi/
│   │   ├── Cargo.toml
│   │   ├── build.rs
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── primitives.rs
│   │       ├── opcodes.rs
│   │       ├── errors.rs
│   │       ├── capabilities.rs
│   │       ├── wire/
│   │       │   ├── mod.rs
│   │       │   ├── encode.rs
│   │       │   ├── decode.rs
│   │       │   └── endian.rs
│   │       └── payloads/
│   │           ├── mod.rs
│   │           ├── mem.rs
│   │           ├── ipc.rs
│   │           ├── dev.rs
│   │           └── irq.rs
│   ├── kapi-syscall/
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       └── arch/
│   │           ├── mod.rs
│   │           ├── x86_64.rs
│   │           ├── riscv64.rs
│   │           └── wrappers.rs
│   ├── ds-ipc/
│   │   ├── Cargo.toml
│   │   └── src/
│   │       ├── lib.rs
│   │       ├── channel.rs
│   │       ├── port.rs
│   │       ├── router.rs
│   │       ├── shared_mem.rs
│   │       ├── sync/
│   │       │   ├── mod.rs
│   │       │   ├── spinlock.rs
│   │       │   └── semaphore.rs
│   └── ds-mem/
│       ├── Cargo.toml
│       └── src/
│           ├── lib.rs
│           ├── alloc/
│           │   ├── mod.rs
│           │   ├── buddy.rs
│           │   ├── slab.rs
│           │   └── heap.rs
│           ├── dma/
│           │   ├── mod.rs
│           │   ├── buffer.rs
│           └── mapping.rs
└──
```

Główny workspace manifest
Wymuszenie nightly i specyficznych targetów

■ FUNDAMENT: API, INFRASTRUKTURA I FRAMEWORKI

Nowe API Jędra: Definicje binarne (Single Source of Truth)

Generowanie nagłówek C/Odin przez cbindgen

Tylko re-eksporty i flagi no_std
Podstawowe typy (Handle, CapId, Status)
Enumy z numerami wywołań (Syscall/IPC opcodes)
Wspólne kody błędów (DsError)
Struktury uprawnień (Capability tokens)
Serializacja/Deserializacja wiadomości

Zapis do bufora
Odczyt z bufora
Konwersje kolejności bajtów
Konkretne struktury wiadomości ([repr(C)])

Dania mapowania pamięci
Dania tworzenia kanałów
Dania rejestracji urządzeń
Dania obsługi przerwań

Surowe wywołania systemowe (SVC / INT)

Implementacje zależne od architektury

Makra asm! dla x86_64 (syscall/sysenter)
Makra asm! dla RISC-V (ecall)
Bezpieczne (lub unsafe) wrappery nad surowym asm

Infrastruktura IPC (Komunikacja między procesami)

Abstrakcja kanału (dwukierunkowego)
Abstrakcja portu (nasłuchiwanie)
Logika routowania wiadomości
Zarządzanie buforami pamięci współdzielonej
Primitives synchronizacji dla IPC

Spinlock (no_std)
Semafore dla kolejek wiadomości

Zarządzanie pamięcią w przestrzeni sterownika

Własne alokatory (brak std::alloc)

Alokator Buddy (dla dużych bloków)
Alokator Slab (dla małych, stałych obiektów)
Globalny heap sterownika
Obsługa DMA (Direct Memory Access)

Bufory DMA (pamięć fizyczna/ciągła)
Mapowanie pamięci dla urządzeń

```

■ ■ ■ ■ vmm.rs # Abstrakcje nad Wirtualn■ Pami■ci■ (stronicowanie)
■ ■ ■ ■ ds-log/ # System logowania
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ levels.rs # Enumy poziomów (Trace, Info, Error, Panic)
■ ■ ■ ■ formatter.rs # Formatowanie tekstu (bez std::fmt)
■ ■ ■ ■ transport.rs # Wysy■anie logów przez IPC do managera
■ ■ ■ ■ macros.rs # Makra: ds_log::info!, ds_log::error!
■ ■ ■ ■ ds-fw-block/ # Framework dla urz■dze■ blokowych (Dyski, USB Mass)
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ traits.rs # Trait BlockDevice (read, write, flush)
■ ■ ■ ■ request.rs # Struktura ■■dania I/O (LBA, rozmiar, bufor)
■ ■ ■ ■ queue.rs # Kolejka ■■da■ (I/O Scheduler)
■ ■ ■ ■ partition.rs # Abstrakcja tablicy partycji (MBR/GPT)
■ ■ ■ ■ ds-fw-audio/ # Framework dla Audio
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ traits.rs # Trait AudioDevice (play, record)
■ ■ ■ ■ stream.rs # Abstrakcja strumienia audio
■ ■ ■ ■ formats.rs # Formaty próbek (PCM, Float, Sample rates)
■ ■ ■ ■ ringbuffer.rs # Lock-free ring buffer dla danych audio
■ ■ ■ ■ ds-fw-gpu/ # Framework dla GPU / Display
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ traits.rs # Trait DisplayDevice, GpuDevice
■ ■ ■ ■ framebuffer.rs # Abstrakcja bufora ramki (pitch, format)
■ ■ ■ ■ modeset.rs # Ustawianie rozdzielczo■ci i trybu
■ ■ ■ ■ edid.rs # Parser EDID (dane monitora)
■ ■ ■ ■ cmd_buffer.rs # Abstrakcja bufora polece■ GPU (Command Submission)
■ ■ ■ ■ ds-fw-input/ # Framework dla HID (Klawiatura, Mysz, Wacom)
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ traits.rs # Trait InputDevice (poll_event)
■ ■ ■ ■ events.rs # Typy zdarze■ (KeyPress, MouseMove, PenPressure)
■ ■ ■ ■ scancodes.rs # Mapowanie scancode'ów na kody klawiszy
■ ■ ■ ■ hid_parser.rs # Parser raportów HID (USB HID descriptors)
■ ■ ■ ■ crates/ # ■ ZARZ■DZANIE: DAEMONY DRIVERSPACE
■ ■ ■ ■ ds-manager/ # G■ówny daemon zarz■dzaj■cy sterownikami
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ main.rs # Entry point, inicjalizacja p■tli zdarze■
■ ■ ■ ■ lifecycle/ # Zarz■dzanie cyklem ■ycia sterowników
■ ■ ■ ■ mod.rs
■ ■ ■ ■ spawn.rs # ■adowanie i startowanie procesów sterowników
■ ■ ■ ■ monitor.rs # Watchdog (restartowanie craszy sterowników)
■ ■ ■ ■ shutdown.rs # Bezpieczne zamykanie
■ ■ ■ ■ resources/ # Przydzielanie zasobów sprz■towych
■ ■ ■ ■ mod.rs
■ ■ ■ ■ irq.rs # Routing i przydzielanie IRQ
■ ■ ■ ■ mmio.rs # Mapowanie pami■ci urz■dze■
■ ■ ■ ■ dma.rs # Przydzielanie buforów DMA
■ ■ ■ ■ ipc_handler.rs # Nas■uchiwanie ■■da■ od j■dra i sterowników
■ ■ ■ ■ config.rs # Wczytywanie konfiguracji (np. z TOML/JSON)
■ ■ ■ ■ ds-registry/ # Rejestr urz■dze■ i sterowników
■ ■ ■ ■ Cargo.toml
■ ■ ■ ■ src/
■ ■ ■ ■ lib.rs
■ ■ ■ ■ database.rs # Przechowywanie mapy DeviceID -> Driver
■ ■ ■ ■ matching.rs # Logika dopasowywania (PCI ID, USB ID, ACPI)

```

```

■      ■■■ state.rs      # ■ledzenie stanu urz■dze■ (Pod■czzone, Usuni■te, B■d)
■
■■■ drivers/
■
■      ■■■ amdgpu_driver/
■      ■■■ Cargo.toml
■      ■■■ build.rs
■      ■■■ src/
■      ■■■ main.rs      # Entry point, inicjalizacja ds-runtime
■      ■■■ probe.rs     # Detekcja karty, odczyt BAR-ów
■      ■■■ mmio.rs      # Wrappery do odczytu/zapisu rejestrów MMIO
■      ■■■ irq.rs       # Obs■uga przerwa■ GPU
■      ■■■ registers/  # Definicje rejestrów (atomowo!)
■      ■      ■■■ mod.rs
■      ■      ■■■ gc.rs      # Graphics Controller
■      ■      ■■■ sdma.rs    # System DMA
■      ■      ■■■ nbio.rs   # Northbridge IO
■      ■      ■■■ ring_buffer.rs # Implementacja pier■cieni polece■ (Ring Buffers)
■      ■      ■■■ display/  # Podmodu■ wy■wietlania
■      ■      ■■■ mod.rs
■      ■      ■■■ phy.rs    # Inicjalizacja PHY (Physical layer)
■
■      ■■■ audiodriver/
■      ■■■ Cargo.toml
■      ■■■ build.rs
■      ■■■ c/
■      ■      ■■■ dsp.c
■      ■      ■■■ dsp.h
■      ■      ■■■ src/
■      ■      ■■■ main.rs
■      ■      ■■■ codec.rs  # Komunikacja z kodekiem audio (I2C/HDA)
■      ■      ■■■ dma_stream.rs # Zarz■dzanie strumieniami DMA dla audio
■      ■      ■■■ ffi.rs    # Funkcje `extern "C"` wywo■ywane przez C/Ok■adki
■
■      ■■■ intelgpu_driver/
■      ■■■ Cargo.toml
■      ■■■ src/
■      ■      ■■■ main.rs
■      ■      ■■■ probe.rs
■      ■      ■■■ gt.rs     # Graphics Technology (rdze■ GPU)
■      ■      ■■■ display.rs # Display Engine (DE)
■      ■      ■■■ guc.rs    # GuC (Graphics Microcontroller) firmware loading
■
■      ■■■ netcam_driver/
■      ■■■ Cargo.toml
■      ■■■ src/
■      ■      ■■■ main.rs
■      ■      ■■■ uvc.rs    # USB Video Class protocol parser
■      ■      ■■■ isoc.rs   # Obs■uga transferów izochronicznych USB
■      ■      ■■■ controls.rs # Panel sterowania (jasno■, kontrast, zoom)
■
■      ■■■ vgpu/
■      ■■■ Cargo.toml
■      ■■■ build.rs
■      ■■■ c/
■      ■      ■■■ vgpu_driver.c # G■ówna logika w C
■      ■      ■■■ vgpu.h
■      ■      ■■■ vgpu_mmio.c  # Operacje na pami■ci w C
■      ■      ■■■ src/
■      ■      ■■■ lib.rs      # Rust wrapper, eksportuj■cy ABI dla ds-manager
■      ■      ■■■ ffi.rs     # Definicje `extern "C"` dla funkcji z C
■
■      ■■■ wacomgraphic_driver/
■      ■■■ Cargo.toml
■      ■■■ src/
■      ■      ■■■ main.rs
■      ■      ■■■ hid_report.rs # Parsowanie specyficznych raportów Wacom
■      ■      ■■■ pressure.rs  # Konwersja surowych danych na nacisk
■      ■      ■■■ tilt.rs     # Obs■uga nachylenia rysika (Tilt X/Y)

```

driverspace_workspace/Workspace.md

Oto profesjonalna dokumentacja architektury, przygotowana w j■zyku angielskim, zgodnie z Twoimi wytycznymi

TrangorgeOS Driver Space Architecture and Workspace Documentation

1. Executive Summary

This document outlines the architectural redesign of the Driver Space subsystem within TrangorgeOS. The c

2. Workspace Topology

The `driverspace\_workspace` is divided into three primary pillars. This strict separation of concerns di

```text

```
driverspace_workspace/
■■■■ Cargo.toml # Workspace root configuration
■■■■ lib/ # Core libraries, frameworks, and the new Kernel API
■■■■ crates/ # Driver space management daemons and services
■■■■ drivers/ # Individual hardware driver implementations
```
```

3. Component Breakdown

3.1. `lib/` (Core Libraries and Kernel API)

The `lib/` directory serves as the foundational layer of the driver space. It contains the new Kernel API

```
* **Kernel API (`kapi-*`)**  
*   `kapi-abi`: The single source of truth for all data structures, IPC message formats, opcodes, and  
*   `kapi-syscall`: Low-level, architecture-specific wrappers for kernel transitions (e.g., SVC calls  
* **Infrastructure (`ds-*`)**  
*   `ds-ipc`: Inter-process communication primitives, channel management, and message routing logic.  
*   `ds-mem`: Memory management utilities, including custom allocators for `no_std` environments, DMA  
*   `ds-log`: Standardized logging infrastructure that routes driver logs to the central manager or k  
* **Device Frameworks (`ds-fw-*`)**  
*   `ds-fw-block`, `ds-fw-audio`, `ds-fw-gpu`, `ds-fw-input`: High-level abstractions for specific de
```

3.2. `crates/` (Driver Space Management)

The `crates/` directory contains the privileged user-space (or kernel-space) daemons responsible for orch

```
* `ds-manager`: The central daemon responsible for device enumeration, resource allocation (IRQs, MMIO  
* `ds-registry`: A database service that maintains the mapping of hardware device identifiers (e.g., PC
```

3.3. `drivers/` (Hardware Drivers)

The `drivers/` directory contains the actual implementations for specific hardware peripherals (e.g., `am

```
* Each driver is an independent crate (or an external project for C/Odin implementations).  
* Drivers implement the traits defined in the `ds-fw-*` frameworks.  
* Drivers communicate with the `ds-manager` and the kernel exclusively through the APIs provided in `li
```

4. The New Kernel API (`kapi`)

The legacy C headers (`libs/\*.h`) and the old `abi.rs` are deprecated. The new `kapi` (Kernel API) is des

Key characteristics of the new API:

1. **Explicit Message Passing:** All communication between the kernel, the manager, and the drivers is h
2. **Capability-Based Security:** Resource sharing (like memory or IRQs) is handled through explicit cap
3. **Language Agnostic:** The ABI is designed to be easily consumable by Rust, C, and Odin, facilitating

5. Architectural Rules and Dependency Graph

To maintain the integrity of the workspace, the following strict rules apply:

1. **Unidirectional Dependencies:**
* `drivers/` may only depend on `lib/`.
* `crates/` may only depend on `lib/`.
* **Crucial Rule:** `drivers/` must **never** depend on `crates/`. Drivers must not have compile-ti
2. **No Code Duplication:** If a utility, protocol parser, or hardware abstraction is needed by more tha
3. **Strict `no\_std` Compliance:** All crates within `lib/` and `crates/` must be `#![no\_std]`. Standar

6. Cross-Language Interoperability (FFI)

TrangorgeOS utilizes multiple languages (Rust, C, Odin, Nim). The `lib/kapi-abi` crate acts as the centra

```
* Rust structures must use `#[repr(C)]`.  
* A build script (`build.rs`) utilizing `cbindgen` should be implemented in `kapi-abi` to automatically
```

7. Summary

This modular workspace architecture transforms the TrangorgeOS driver space from a tightly coupled monoli

driverspace_workspace/formal/README.md

kernel_Workspace/C_base/kc-abi/docs/abi_spec.md

kernel_Workspace/C_base/kc-abi/docs/architecture.md

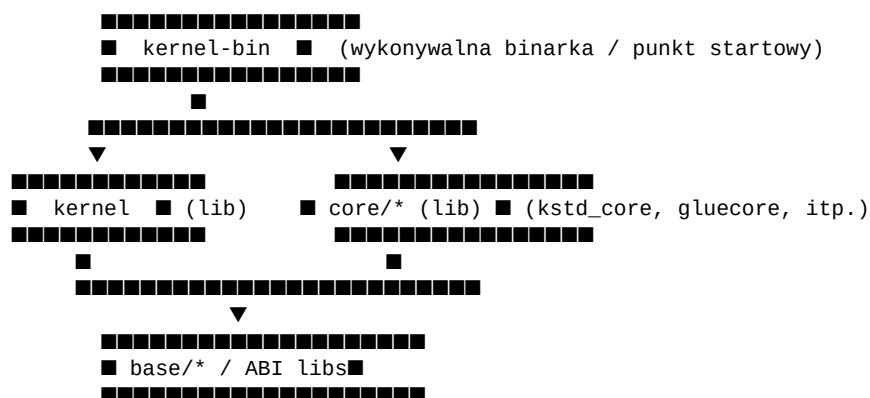
kernel_Workspace/C_base/kc-abi/docs/memory_model.md

kernel_Workspace/C_base/kc-abi/kc-abi/docs/abi_spec.md

kernel_Workspace/C_base/kc-abi/kc-abi/docs/architecture.md

kernel_Workspace/C_base/kc-abi/kc-abi/docs/memory_model.md

kernel_Workspace/README.md



kernel_Workspace/core/core.md

kernel_Workspace/kernel/README.md

...

kernel_Workspace/kernel/src/mm/MM_PLAN_ULEPSZENIA.md

Plan ulepszenia podsystemu MM w TrangorgeOS

Repozytorium: [CTRL-F-0rg3/TrangorgeOS](https://github.com/CTRL-F-0rg3/TrangorgeOS)

Analizowana gałęź: `stabilizing`

Analizowany commit: `7a0d836f2180874aac3e8c92493c791aec89a8b3`

Autor analizy: ctrl

Data analizy: 20 sierpnia 2026 r.

masz kropke gdzie ostatnio dodałem

1. Zakres i podsumowanie

Przeanalizowany został cały katalog `kernel/src/mm`, w tym: inicjalizacja pamięci architektury, bitmapy

Architektura jest sensownie rozdzielona na warstwy. Obecny przepływ startowy (`arch\_memory\_init` → pagin

> ****Najważniejsza rekomendacja:**** przed dodawaniem nowych funkcji należy ustabilizować kontrakty allocatorów

2. Ocena stanu obecnego

Obszar	Stan	Ocena
---	---	---
Rozdzielenie PMM/VMM/heap/process	Istnieje i jest czytelne	Dobra baza architektoniczna
Bitmapa ramek	Obsługuje pojedyncze i ciągłe zakresy	Wymaga testów granicznych i lepszej wydajności
Buddy allocator	Działający jako główna ścieżka heap	Brak synchronizacji SMP i dokładnego rozmiaru bloków
Slab allocator	Zaimplementowany, ale 'HEAP_USE_SLAB' wynosi '0'	Niewykorzystana optymalizacja; ma r
API alokacji	'kmalloc', 'kzalloc', 'kcalloc', 'krealloc', strony	Kontrakty 'aligned/pages' są niepe
Paging/VMM	Słabe mapowanie, translate, protect i address spaces	Brakuje pełnej transakcyjności i wali
DMA/contiguous	Istnieją osobne moduły	Wymagają bezpiecznego liczenia rozmiarów i modelu cache
Izolacja	SMEP/SMAP/NX i audyt PML4	Wzmocnienie mechanizmów nie jest raportowane jako błąd krytyczny
Testy	Jest 'kernel/src/testing.rs' i kilka self-testów	Brak automatycznej, szerokiej macierzy testów
Build/CI	W środowisku analizy 'cargo' nie było dostępnego	Nie udało się potwierdzić kompilacji; pot

3. Problemy wymagające naprawy

P0 – bezpieczeństwo i poprawność krytyczna

P0.1. Blokada wywołująca przerwanie nie są blokadami SMP

PMM, slab i przestrzenie adresowe używają wzorca 'pushfq; cli' oraz lokalnego licznika zagnieżdżenia. Wy

****Pliki:**** 'kernel/src/mm/alloc/physical/pmm.c', 'kernel/src/mm/alloc/heap/slab.c', 'kernel/src/mm/proces

****Zadanie:**** zastąpić blokady per-CPU rzeczywistymi spinlockami ticket/MCS albo istniejącym prymitywem S

****Kryterium akceptacji:**** test wielordzeniowy wykonuje równoległe co najmniej 100 000 alokacji i zwolnień

P0.2. Double-free w slabie może uszkodzić listę wolnych obiektów

'slab_free()' sprawdza zakres i wyrównanie wskaźnika, ale nie przechowuje informacji, czy konkretny obiekt

****Plik:**** 'kernel/src/mm/alloc/heap/slab.c', szczególnie ścieżka 'slab_free()'.

****Kryterium akceptacji:**** drugie 'slab_free(ptr)' nie modyfikuje allocatorów i generuje błąd diagnostyczny

P0.3. Brak walidacji overflow w API przestrzeni adresowych

W wielu miejscach granice są liczone jako 'addr + len', a następnie wyrównywane, np. w 'aspace_map_at',

****Plik:**** 'kernel/src/mm/process/address_space.c', okolice obliczeń 'a'/'b'.

****Zadanie:**** wprowadzić wspólną funkcję 'range_from_addr_len(addr, len, alignment, out_start, out_end)',

1. odrzuca 'len == 0';
2. sprawdza 'addr + len' bez overflow;
3. bezpiecznie wyrównuje początek w dół i koniec w górę;
4. sprawdza kanoniczność oraz pełne granice user space;
5. zwraca zakres półotwarty '[start, end)'.

****Kryterium akceptacji:**** testy dla 'UINT64_MAX', adresów niekanonicznych, przejścia przez granicę user/

P0.4. Nieatomowy rollback w 'aspace_brk()'

'aspace_brk()' najpierw tworzy lub rozszerza VMA, a dopiero potem wywołuje 'map_anon_pages()'. Jeżeli al

****Plik:**** 'kernel/src/mm/process/address_space.c', ścieżka powiększania sterty.

****Zadanie:**** zastosować transakcję: najpierw przygotować zakres i zmapować strony, a dopiero po pełnym s

****Kryterium akceptacji:**** wymuszony brak ramek w połowie operacji pozostawia identyczne VMA, 'brk' i lic

P1 – błądy kontraktów allocatorów

P1.1. 'kmalloc_aligned()' nie gwarantuje dowolnego wyrównania

'buddy_alloc_aligned()' zaokrągla potrzebny rozmiar do potęgi dwójki i zakłada, że baza buddy jest odpow

****Pliki:**** 'kernel/src/mm/alloc/api/alloc.c', 'kernel/src/mm/alloc/heap/buddy.c'.

****Zadanie:**** jasno zdefiniowa API: albo przyjmowa wyliczenie potęgi dwójki i to walidowa, albo doda

P1.2. ``kfree_pages()`` ignoruje parametr ``pages``

``kfree_pages(void *ptr, size_t pages)`` ignoruje ``pages`` i deleguje do ``heap_free()``. To tworzy mylący kon

****Plik:**** ``kernel/src/mm/alloc/api/alloc.c``.

P1.3. ``krealloc()`` nie przechowuje rzeczywistego rozmiaru zadan

Poza debug mode ``heap_usable_size()`` zwraca rozmiar zaokręglonego bloku buddy, a nie liczb bajtów zadan

P1.4. Overflow w alokacji contiguous i DMA

``contig_bytes_to_frames()`` i ``dma_bytes_to_frames()`` wykonuj ``bytes + ARCH_PAGE_SIZE - 1`` bez jawnej och

****Pliki:**** ``kernel/src/mm/alloc/special/contiguous.c``, ``kernel/src/mm/alloc/special/dma.c``.

****Kryterium akceptacji:**** największe wartości ``size_t``, ``UINT64_MAX``, długości niebędące wielokrotnośc

P1 – paging, TLB i mapowania

Należy przeprowadzić audyt wszystkich funkcji ``map``, ``unmap`` i ``protect`` pod kątem transakcyjności. Je

****Zadania:****

Zadanie	Oczekiwany rezultat
Ujednolicić <code>`map_range`</code> / <code>`unmap_range`</code>	Jedna polityka wyrównania, overflow i rollbacku
Dodać statusy bloków	Rozróżnienie <code>`invalid`</code> , <code>`already mapped`</code> , <code>`no memory`</code> , <code>`permission`</code>
Dokonać batch TLB	Jedno <code>`invlpg`</code> na adres lub pełny flush przy przepełnieniu batcha
Dodać shutdown SMP	IPI lub bezpieczny mechanizm synchronizacji aktywnych CR3
Walidować PTE flags	Zakaz USER w mapowaniach kernela i spójne NX/WRITE/DEVICE
Testować aliasy	Mapowanie tej samej ramki pod różne VA z kontrolą uprawnień

P1 – semantyka ``mmap``/VMA

Obecne ``mmap()`` rozpoznaje głównie ``MAP_FIXED`` i kieruje pozostałe przypadki do anonimowego mapowania.

****Zadania:****

- Walidować dozwolone kombinacje flag i odrzucać nieobsługiwane zamiast je ignorować.
- Rozdzielić VMA na prefix/zakres/suffix przy czyszczeniowym ``mprotect``.
- Wykonywać wyszukiwanie VMA pod tym samym blokadą co modyfikację.
- Ustalić, czy mapowanie anonimowe ma być eager czy demand-paged.
- Dodać guard pages dla stosu oraz limit wzrostu stosu.
- Wprowadzić copy-on-write dla ``MAP_PRIVATE``, gdy pojawi się fork/procesy potomne.

4. Ulepszenia architektury i wydajności

.

4.1. PMM strefowy zamiast jednej bitmapy dla całego maksimum fizycznego

Obecna bitmapa indeksuje PFN-y a nie do najwyższego użytecznego adresu, także przez dziury w fizycznej prze

4.2. Per-CPU cache dla małych alokacji

Po ustabilizowaniu słaba wartość dodać per-CPU magazyn/cache dla najczęstszych klas 16-256 B. Ograniczyć t

4.3. Lepszy allocator stron dla dużych bloków

Buddy jest dobrym źródłem dużych bloków, ale obecna sterła mapuje i odmapowuje strony przy każdej operac

4.4. Cache i DMA

``dma_alloc_coherent()`` potrzebuje jawnego modelu spójności zależnego od architektury. Należy rozróżni

4.5. Bezpieczne flagi ochrony

Należy ustalić jedną reprezentację ochrony. Obecnie występują osobne zestawy ``PROT_*`` i ``virt::*`` flags

5. Diagnostyka i obserwowalność

Debug allocator ma dobre podstawy: magic, poison, canary, leak table i statystyki. Ograniczeniem jest sta

Ulepszenie	Cel
---	---
Rozszerzy leak table lub użyj hash table	Brak cichego nieledzenia alokacji po przekroczeniu limitu
Doda alloc_id	Jednoznaczne ledzenie ycia obiektu
Doda guard pages dla dużych alokacji	Wykrywanie overrun/underrun poza canary
Raportowa ownera cache i CPU	Debugowanie SMP
Doda histogram rozmiarów	Strojenie klas slab/buddy
Wykrywa niezgodność par allocator/free	Natychmiastowe raportowanie błędów ownership
Eksportowa snapshot MM	Testy regresyjne: free/used, VMA, mapowania, TLB

6. Plan implementacji etapami

Etap	Zakres	Priorytet	Definicja ukończenia
---	---	---	---
A	Wspólne helpery overflow/range/alignment	P0	Wszystkie publiczne API zakresowe używaj helperów
B	Spinlocki SMP + kolejno blokad	P0	Test równoległy nie wykazuje race/double allocation
C	Transakcyjny map/unmap/`aspace_brk`	P0	Błęd w połowie operacji nie zmienia stanu
D	Bezpieczny slab z bitmapą obiektów	P0	Double-free i invalid-free są wykrywane
E	Kontrakty aligned/pages/realloc	P1	API ma dokumentowane i testowane gwarancje
F	Overflow DMA/contiguous + strefy PMM	P1	Testy graniczne przechodzą, DMA32 działa deterministycznie
G	Pełniejsze VMA/mmap/mprotect	P1	Człciowe zakresy są poprawnie dzielone
H	TLB shutdown SMP i batchowanie	P1	Zmiany mapowania są widoczne na wszystkich CPU
I	Per-CPU cache i optymalizacja buddy/slab	P2	Benchmark pokazuje mniejszą kontencję bez regresji
J	Demand paging/COW/huge pages	P2	Funkcje mają testy integracyjne z procesami

7. Minimalny zestaw testów regresyjnych

Należy dodać testy jednostkowe możliwe do uruchamiania poza jądrem dla bitmapy, regionów, alignmentu i c

Grupa	Przypadki
---	---
Bitmapa	pusta, pełna, jeden bit, granice 63/64/65, zakres przez granic słowa, double-free
PMM	pojedyncze ramki, contiguous, align 0/1/nie-potęga, max address, brak pamięci
Buddy	kałdy order, koalescencja, fragmentacja, invalid pointer, double-free, overflow align
Slab	kałda klasa, pełny/pusty slab, double-free, invalid pointer, równoległość
API heap	`calloc` overflow, `realloc(NULL)`, `realloc(ptr,0)`, aligned, pages
Mapowanie	człciowy błąd, rollback, overlap, unmap fragmentu, protect fragmentu
VMA	zakresy graniczne, `brk` rollback, guard page, siednie VMA
DMA	32-bit zone, długo overflow, unmap rollback, sync CPU/device
Ochrona	NX, SMEP, SMAP, USER/KERNEL PTE, zakaz W^X

8. Dokumentacja i CI

Dokumentacja architektury powinna używać gałzi `stabilizing`, a nie nieaktualnych linków do `unstable`.

Repozytorium powinno mieć CI, który buduje kernel w tym samym cross-toolchainie co Docker/QEMU, uruchamia

9. Kolejno rekomendowana dla autora

Najpierw należy wdrożyć helpery zakresów i overflow, następnie spinlocki SMP. Dopiero potem warto napraw

> **Nie rekomenduj jeszcze włączania `HEAP\_USE\_SLAB` w produkcyjnej ścieżce.** Najpierw trzeba dodać w

10. Odwołania do kodu

Najważniejsze ustalenia wynikają bezpośrednio z analizowanej gałzi:

- [API alokacji](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/api/allo>)
- [Buddy allocator](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/p>)
- [Slab allocator](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/s>)
- [PMM](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/physical/pmm.c>)
- [Przestrzenie adresowe](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/proce>)
- [DMA](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/special/dma.c>)
- [Dokumentacja architektury](<https://github.com/CTRL-F-0rg3/TrangorgeOS/blob/stabilizing/TrangorgeOS%20>)

References

[1]: <https://github.com/CTRL-F-0rg3/TrangorgeOS/tree/stabilizing> "TrangorgeOS – gałła stabilizing"
[2]: <https://github.com/CTRL-F-0rg3/TrangorgeOS/commit/7a0d836f2180874aac3e8c92493c791aec89a8b3> "Trangorg


```
[3]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/buddy.c "Buddy
[4]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/heap/slab.c "Slab al
[5]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/process/address_space.c "A
[6]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/physical/pmm.c "Phys
[7]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/kernel/src/mm/alloc/special/dma.c "DMA a
[8]: https://github.com/CTRL-F-org3/TrangorgeOS/blob/stabilizing/TrangorgeOS%20%E2%80%94%20dokumentacja%20
```

panic.md

```
# Kernel Hang / Panic Report – 2026-10-09
# Kernel Hang / Panic Report – LAPIC base sanity check
```

Key findings:

- `APIC\_BASE\_MSR` read reports x2APIC when bit 10 set; else LAPIC assumed at `base\_phys` passed in.
- `lapic::init(base\_phys)` uses `base\_phys` straight as virtual if it's `>= 0xFFFFF80000000000`.
- In SMP init, APs are booted with `send\_startup\_ipi` using `trampoline::TRAMPOLINE\_BASE >> 12` as vector

Open question:

- Where is actual LAPIC physical base loaded from? Current code does not show base-specific usage beyond

Next active items:

1. Confirm TRAMPOLINE_BASE value and that identity mapping covers it.
2. Confirm `build.rs` linker path behavior for x86_64.
3. Add serial console capture in QEMU and reproduce hang location.

Status

Kernel ****builds and boots**** in qemu (GTK display, 4 vCPUs) but ****freezes during runtime****. No console / serial output was captured from the GTK session – the VM stays quiet and appears hung.

Attempted serial capture with qemu (`-serial file:/tmp/qemu\_serial\_out.txt`) did not produce a readable guest log at the time of this report, so the exact stop point is inferred from the boot path rather than from live kernel prints.

Boot entry

```
`kernel_main(boot_info: &'static bootloader::BootInfo) -> !` (`src/main.rs`)
```

Sequence:

1. `init()`
2. `mm::init\_from\_boot\_info(boot\_info)` → “[mm] allocator initialized OK” expected
3. `init\_permissions()`
4. `gfx::init()` → “[gfx] framebuffer initialized OK” expected
5. (optional) `hdmi::init::init()`
6. `pci::init()`
7. `nic::runtime::init()`
8. `bluetooth::init::init()`
9. `fs::init()`
10. ****`cpu::init(boot\_info)`**** ← SMP bring-up, tight waits per AP
11. `testing::run\_all(TESTS)`
12. `println!("Welcome in my Galaxy!")`
13. `gfx::refresh()`, `terminal::init()`, `terminal::run()`

Where the hang manifests

In the GTK qemu session the kernel reaches `cpu::init(boot\_info)` (step 10) and then stops producing visible progress. The SMP initialization polls for each AP to reach its trampoline entry within a tight per-AP timeout (about 100 ms per AP, 3 APs with 4 vCPUs). If an AP does not become ready in time, the boot path blocks waiting for it without emitting a visible panic in the GTK display.

Observations from the code

- SMP init lives in `src/cpu/mod.rs` / `src/cpu/smp.rs` and reaches into `src/cpu/scheduler/smp/mod.rs` plus the migration / stopper / balancing subtrees. AP boot uses a trampoline at `TRAMPOLINE\_BASE`.
- The bootloader crate (`bootloader = 0.9` with `map\_physical\_memory` +

Changed technical context

- Trampoline base is `TRAMPOLINE\_BASE = 0x8000` in `kernel/src/cpu/trampoline.rs` and `.set TRAMPOLINE\_BASE`
- SMP bringup uses `send\_startup\_ipi(apic\_id, (trampoline::TRAMPOLINE\_BASE >> 12) as u8)`.
- Paging init in `kernel/src/mm/arch/x86\_64/paging.c` skips identity work when `boot_phys_offset == ARCH_
- Kernel MM bridge is compiled by `kernel/build.rs`, but on x86_64 it does ****not**** inject an explicit `-T
- Trampoline identity map is done relative to `TRAMPOLINE\_BASE` before paging is enabled.
- ACPI parsing path uses `phys\_offset` for RSDP/MADT/FADT reads; MADT can carry `lapic\_base\_override` (ty

Still blocked

- Cannot reproduce and capture exact panic address or serial log yet in this environment; recommended next

- `vga\_320x200`) maps physical memory and provides `BootInfo`. The kernel C bridge (`libmm.a`) is compiled by `build.rs` for x86_64.
- There is **no explicit x86_64 kernel linker script** in the repo at the moment — `build.rs` adds `-T ../riscv64-link.ld` only for riscv64. On x86_64 the bootloader's default layout is used. If that layout does not match kernel assumptions about load address / trampoline placement, early boot can fail silently.
- **566 Rust warnings** remain (Rust 2024 strictness around `&mut static` / shared references to mutable statics in `gfx/console.rs`, `vga\_buffer.rs`, `fs/mbr.rs`, and several scheduler files). These are not compile errors yet, but some are UB-adjacent and can corrupt early-boot state.
- Current `cargo run` uses the GTK display variant, so kernel `println!` / panic output is not visible from the host side.

Likely root causes (ranked)

1. **SMP AP bring-up timeout / deadlock** — an AP never reaches the trampoline or does not complete early init within the per-AP timeout, and the BSP waits indefinitely (or until a timeout path that may not panic). This is the most probable hang location with 4 vCPUs / 3 APs.
2. **Incorrect / incomplete physical memory mapping for the kernel image or trampoline region** — if `BootInfo` mapping is wrong, AP bootstrap code can fault silently without a visible triple fault in GTK mode.
3. **Undefined behavior from `&mut static` / shared-ref patterns** in early boot (GFX, scheduler, FS) corrupting state that SMP init / tests depend on.
4. **Missing explicit kernel linker script for x86_64** — the bootloader default layout may not match kernel expectations for section / trampoline placement, causing early code to run at wrong addresses.

Czerwone miejsce (hang point)

- `cpu::init(boot\_info)` in `src/cpu/mod.rs` / `src/cpu/smp.rs` — specifically the per-AP wait loop after the SMP startup IPI(s). If the AP never signals readiness, the BSP waits here and the system hangs.

Verification needed

- Run qemu with a **serial console** (`-serial stdio` or `-serial file:/tmp/qemu\_serial\_out.txt`) and `< /dev/null` so host can read kernel prints and any panic message.
- Determine whether the log stops at `[mm]`, `[gfx]`, `pci`, `nic`, `bluetooth`, `fs`, or at `cpu::init` / AP bring-up.
- Inspect the SMP trampoline path and per-AP early stack / GDT / IDT setup.
- Validate the bootloader-provided memory map and kernel load / trampoline addresses against kernel expectations.
- Consider adding an explicit x86_64 linker script if the bootloader layout is not what the kernel assumes.

Verification status

- Build: clean (exit 0) for `driverspacelib`, `driverspace`, `kernel`.
- Boot: `cargo run` builds the bootimage and launches qemu.
- Runtime: hung; no serial console output captured yet in this session.

Notes

- The GTK display session shows only qemu theme warnings; no kernel console

Architecture-specific physical-memory offset notes

- On x86_64, `mm::init` receives `boot\_phys\_offset` from `boot\_info.physical\_memory\_offset` and passes it
- Paging code in `kernel/src/mm/arch/x86\_64/paging.c` has an early exit when `boot_phys_offset == ARCH_DI
- If the bootloader does not map physical memory at the expected offset, the direct map and/or trampoline
- The LAPIC base can come from ACPI MADT; if that value is used directly as a virtual address somewhere w

Actionable next steps

1. Boot the kernel with `-serial stdio` and capture the exact panic / hang point, not just the GTK display
2. If there is no serial access yet, add an early, unconditional serial write right after paging enable a
3. Compare the bootloader-provided `physical\_memory\_offset` with the value the kernel expects before call
4. If APs are not starting, focus on the trampoline region identity mapping and the startup-IPI path, and

2026-09-14 — objdump evidence from debug ELF (`kernel`)

- Static data (confirmed in objdump relocation output):
 - ``boot_phys_offset``: ``0x5c6d88``
 - ``boot_phys_offset_valid``: ``0x5c6d80``
- Disassembly shows explicit checks against `<boot_phys_offset_valid>` and `<boot_phys_offset>` in main, paging, serial, and CPU bringup paths.
- Paging init path includes early-exit / map logic matching the C/Rust preamble already discussed.

Observability findings

- The debug kernel ELF (``target/x86_64-kernel/debug/kernel``) exists.
- Serial console capture remains the biggest gap:
 - ``/tmp/kernel_serial_out.txt`` is empty.
 - ``/tmp/kernel_serial_err.txt`` was too short to contain a clear panic line.

Interpretation

- Because the binary clearly looks at ``boot_phys_offset`` / ``boot_phys_offset_valid`` around paging init ``77b2...a5``, the most likely first failures are:
 1. Paging init fault when bootloader-provided offset is incompatible with ``ARCH_DIRECT_MAP_BASE``, OR
 2. SMP bringup fault when AP trampoline/stack/identity mapping is inconsistent with LAPIC/ACPI MADT info.
- Correlation is now feasible if serial output is captured and mapped back with the debug ELF.

Branch state

- ``git status``: modified: ``panic.md``; otherwise on ``stabilizing``.
- HEAD: ``183557c`` (``stabilizing``).

output is visible there.

Current working branch and state

- Branch: ``stabilizing`` (tracking remote ``origin/stabilizing``).
- Working tree has unstaged changes mainly in scheduler / SMP and some target/flycheck artifacts; kernel
 - ``kernel/src/cpu/scheduler/collections/mod.rs``
 - ``kernel/src/cpu/scheduler/smp/migration/mod.rs``
- The most relevant panic context is in ``panic.md``; the key code references are ``kernel/src/main.rs``, ``ke`

Current working branch and state

- Branch: ``stabilizing`` (tracking remote ``origin/stabilizing``).
- Working tree has unstaged changes mainly in scheduler / SMP and some target/flycheck artifacts; kernel
 - ``kernel/src/cpu/scheduler/collections/mod.rs``
 - ``kernel/src/cpu/scheduler/smp/migration/mod.rs``
- The most relevant panic context is in ``panic.md``; the key code references are ``kernel/src/main.rs``, ``ke`

TODO

References

References

- [] Capture reliable serial console output with ``-serial stdio``/``-serial file``.
- [] Add an early unconditional serial write after paging enable, before MM/SMP init.
- [] Compare bootloader ``physical_memory_offset`` vs ``ARCH_DIRECT_MAP_BASE`` and LAPIC direct-map expectations.
- [] Confirm trampoline identity mapping covers AP trampoline code/stack.
- [] If serial log appears, correlate RIP with ``kernel`` ELF using `addr2line/objdump`.
- [] If offset mismatch is confirmed, decide whether to support dynamic offset or force early remap before paging enable.
- [] Fix ``kernel/src/cpu/scheduler/collections/mod.rs`` and ``kernel/src/cpu/scheduler/smp/migration/mod.rs`` (``pub pub mod ...``) so ``cargo build --release`` can produce the release bootimage again.

- ``kernel/src/main.rs``
- ``kernel/src/cpu/smp.rs``
- ``kernel/src/cpu/lapic.rs``
- ``kernel/src/cpu/acpi.rs``
- ``kernel/src/mm/mod.rs``
- ``kernel/src/mm/arch/x86_64/paging.c``
- ``kernel/src/mm/arch/x86_64/memory.c``

- `kernel/src/cpu/trampoline.s`
- `kernel/src/cpu/trampoline.rs`
- `panic.md`

This report is intended as a continuation note for kernel hang debugging on x86_64 and is not yet a root-

- `kernel/src/main.rs`
- `kernel/src/cpu/smp.rs`
- `kernel/src/cpu/lapic.rs`
- `kernel/src/cpu/acpi.rs`
- `kernel/src/mm/mod.rs`
- `kernel/src/mm/arch/x86\_64/paging.c`
- `kernel/src/mm/arch/x86\_64/memory.c`
- `kernel/src/cpu/trampoline.s`
- `kernel/src/cpu/trampoline.rs`
- `panic.md`

This report is intended as a continuation note for kernel hang debugging on x86_64 and is not yet a root-

- [] Add a serial-handshake early print right after paging enable, before SMP bringup.
- [] Add a small test mode to boot with QEMU and a serial file/stdio in CI or locally, so the hang can be reproduced.
- [] Compare actual bootloader-provided `physical\_memory\_offset` with the offset used by paging init and ensure they match.
- [] Confirm trampoline identity map covers at least the trampoline code and stack range used by AP bringup.